

Michael Laube

CONSTRAINT

CREATE INDEX

ORDER BY

WHERE

Einstieg in SQL

Für
Ausbildung
und Beruf

CREATE
TABLE

SELECT * FROM

DELETE FROM

GROUP BY

CREATE VIEW

4. Auflage

- ▶ Datenbanken und SQL – ganz ohne Vorkenntnisse einsteigen
- ▶ SQL-Abfragen, Datenmodellierung, Tipps aus der Praxis
- ▶ Übungen und Musterlösungen für alle Datenbanksysteme



Inkl. Übungsdatenbank zum Download



Rheinwerk
Computing

Liebe Leserin, lieber Leser,

Sie haben sicherlich schon einmal mit einer relationalen Datenbank und *SQL* zu tun gehabt. Denn egal ob bei der Webentwicklung, der Programmierung oder der Systemadministration: Überall, wo große Datenmengen verarbeitet werden, werkelt ganz sicher eine Datenbank im Hintergrund. Eine zentrale Rolle spielt dabei die Structured Query Language und ihre verschiedenen Dialekte, die seit Jahrzehnten bei der Arbeit mit Datenbanken eingesetzt werden.

Daher ist es eine sehr gute Entscheidung, sich mit *SQL*, der richtigen Datenverarbeitung und der korrekten Datenmodellierung zu beschäftigen, denn dieses Wissen wird Ihnen in vielen Bereichen weiterhelfen.

Dabei steht Ihnen Michael Laube zur Seite, der als erfahrener Datenbankentwickler schon lange mit *SQL* arbeitet. Er erklärt Ihnen schrittweise und detailliert, wie Sie die Beispieldatenbank auf Ihrem Rechner einrichten, Rohdaten von Redundanzen befreien und effiziente Abfragen formulieren. Übungen und Musterlösungen aus der Praxis runden das Angebot ab und sorgen dafür, dass Ihnen die Syntax der Befehle leicht von der Hand geht und dass Ihnen auch große Datenmengen keine Probleme bereiten. So ist dieser Leitfaden ideal für das Selbststudium, die Lehre oder für den täglichen Einsatz in der Praxis geeignet.

Abschließend noch ein Wort in eigener Sache: Dieses Werk wurde mit großer Sorgfalt geschrieben, geprüft und produziert. Sollte dennoch einmal etwas nicht so funktionieren, wie Sie es erwarten, freue ich mich, wenn Sie sich mit mir in Verbindung setzen. Ihre Kritik und konstruktiven Anregungen sind jederzeit willkommen.

Ihr Christoph Meister

Lektorat Rheinwerk Computing

christoph.meister@rheinwerk-verlag.de

www.rheinwerk-verlag.de

Rheinwerk Verlag · Rheinwerkallee 4 · 53227 Bonn

Hinweise zur Benutzung

Dieses E-Book ist **urheberrechtlich geschützt**. Mit dem Erwerb des E-Books haben Sie sich verpflichtet, die Urheberrechte anzuerkennen und einzuhalten. Sie sind berechtigt, dieses E-Book für persönliche Zwecke zu nutzen. Sie dürfen es auch ausdrucken und kopieren, aber auch dies nur für den persönlichen Gebrauch. Die Weitergabe einer elektronischen oder gedruckten Kopie an Dritte ist dagegen nicht erlaubt, weder ganz noch in Teilen. Und auch nicht eine Veröffentlichung im Internet oder in einem Firmennetzwerk.

Die ausführlichen und rechtlich verbindlichen Nutzungsbedingungen lesen Sie im Abschnitt Rechtliche Hinweise.

Dieses E-Book-Exemplar ist mit einem **digitalen Wasserzeichen** versehen, einem Vermerk, der kenntlich macht, welche Person dieses Exemplar nutzen darf:

Exemplar Nr. 9iya-54nf-7kz6-p3xu
zum persönlichen Gebrauch für
Tong Wu,
wutong2017ok@gmail.com

Impressum

Dieses E-Book ist ein Verlagsprodukt, an dem viele mitgewirkt haben, insbesondere:

Lektorat Christoph Meister

Korrektorat Petra Biedermann, Reken

Herstellung E-Book Ralf Kepnik

Covergestaltung Silke Braun

Satz E-Book SatzPro, Krefeld

Wir hoffen sehr, dass Ihnen dieses Buch gefallen hat. Bitte teilen Sie uns doch Ihre Meinung mit und lesen Sie weiter auf den [Serviceseiten](#).

Bibliografische Information der Deutschen Nationalbibliothek:

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <http://dnb.dnb.de> abrufbar.

ISBN 978-3-367-10547-2 (E-Book)

ISBN 978-3-367-10549-6 (Bundle)

4., aktualisierte Auflage 2025

© Rheinwerk Verlag GmbH, Bonn 2025

www.rheinwerk-verlag.de

Inhalt

Materialien zum Buch	19
1 Grundlagen kennenlernen und verstehen	21
1.1 Die Tabelle als zentrales Element	21
1.2 Eine kleine Historie von SQL	23
1.3 Datenbanksysteme	24
1.4 SQL – ein Standard und seine Umsetzung	25
1.5 Zu diesem Buch	27
1.6 MySQL unter Windows installieren	28
1.7 Die MySQL-Übungsdatenbank anlegen	36
1.8 Eine erste Abfrage an die Datenbank senden	39
1.9 Kommentarfunktion	42
1.9.1 Übungen	44
2 Los geht's: Die Grundfunktionen der Tabellenabfrage (SELECT)	47
2.1 Mit einer SELECT-Anweisung Tabellen abfragen	47
2.1.1 Die Tabelle »mitarbeiter«	47
2.1.2 Wie frage ich eine Tabelle ab? (SELECT ... FROM)	47
2.1.3 Spalten einer Tabelle abfragen	48
2.1.4 Alle Spalten einer Tabelle abfragen	49
2.1.5 Übungen	50
2.2 Zeilen in einer Abfrage mit WHERE filtern	51
2.2.1 SQL-Vergleichsoperatoren	52
2.2.2 Spaltenwerte auf Gleichheit prüfen	55
2.2.3 Spaltenwerte auf Ungleichheit prüfen	58
2.2.4 Spaltenwerte auf kleiner als/gleich prüfen	60
2.2.5 Spaltenwerte auf größer als/gleich prüfen	62
2.2.6 Bedingungen mit dem NOT-Operator verneinen	64
2.2.7 Spaltenwerte auf ein Intervall prüfen (BETWEEN)	66

2.2.8	Spaltenwerte auf ein Muster prüfen (LIKE)	71
2.2.9	Spaltenwerte auf Mengenzugehörigkeit prüfen	76
2.2.10	Fehlende Spaltenwerte (NULL-Value)	79
2.2.11	Spaltenwerte auf NULL prüfen	82
2.2.12	Spaltenwerte auf »ist nicht NULL« prüfen	83
2.2.13	Spaltenwerte mit Spaltenwerten vergleichen	84
2.2.14	Übungen	86
2.3	Filterbedingungen mit AND (NOT) und OR (NOT) logisch verknüpfen	91
2.3.1	Der logische Verknüpfungsoperator AND	91
2.3.2	SQL-Bedingungen mit dem logischen AND-Operator verknüpfen	92
2.3.3	Der logische Verknüpfungsoperator OR	94
2.3.4	SQL-Bedingungen mit dem logischen OR-Operator verknüpfen	96
2.3.5	Der logische Verknüpfungsoperator AND NOT	98
2.3.6	SQL-Bedingungen mit dem AND NOT-Operator logisch verknüpfen	99
2.3.7	Der logische Verknüpfungsoperator OR NOT	99
2.3.8	SQL-Bedingungen mit dem logischen OR NOT-Operator verknüpfen	100
2.3.9	Logische Verknüpfungsoperatoren kombiniert anwenden	101
2.3.10	Übungen	106
2.4	Ergebniszeilen einer SELECT-Anweisung einschränken	109
2.4.1	Ergebniszeilen mit FETCH, LIMIT und TOP eingrenzen	109
2.4.2	Übungen	111
2.5	Datensätze sortiert abfragen	111
2.5.1	Aufsteigende Sortierung gemäß einer Spaltenangabe	113
2.5.2	Auf- und absteigende Sortierung mehrerer Spalten	115
2.5.3	Nach numerischen Spaltenwerten sortieren	116
2.5.4	Nach Datumswerten sortieren	117
2.5.5	Nicht definierte Werte in einer Sortierung beachten	118
2.5.6	ORDER BY mit einer WHERE-Klausel verwenden	119
2.5.7	Übungen	121
2.6	Konstanten in die Spaltenauswahlliste aufnehmen	124
2.6.1	Abfrage eines konstanten Textes	125
2.6.2	Konstanten und Spalten einer Tabelle gleichzeitig abfragen	125
2.6.3	Übungen	126
2.7	Spalten einen Alias zuordnen	127
2.7.1	Spalten in einer Abfrage mit einem Alias versehen	128
2.7.2	Ausgewählten Spalten einer Abfrage einen Alias zuordnen	128
2.7.3	Spalten und Konstanten einen Alias zuordnen	129
2.7.4	Übungen	130

2.8	Gleiche Ergebniszeilen ausschließen (DISTINCT)	131
2.8.1	Übungen	133

3 Zeilen einfügen (INSERT), ändern (UPDATE) und löschen (DELETE, TRUNCATE) 135

3.1	Zeilen mit einer INSERT-Anweisung einfügen	135
3.1.1	Spaltenwerte mit expliziter Spaltenangabe einfügen	136
3.1.2	Spaltenwerte ohne Spaltenangabe einfügen	140
3.1.3	Übungen	143
3.2	Zeilen mit einer UPDATE-Anweisung ändern	145
3.2.1	Einen Spaltenwert einer Zeile ändern	146
3.2.2	Mehrere Spaltenwerte einer Zeile gleichzeitig ändern	147
3.2.3	Spaltenwerte einer Spalte für mehrere Zeilen gleichzeitig ändern	148
3.2.4	Allen Spaltenwerten einer Spalte einen Wert zuordnen	150
3.2.5	Spaltenwerten mit einer UPDATE-Anweisung einen NULL-Wert zuweisen	151
3.2.6	Schlüsselwertspalten mit UPDATE einen neuen Wert zuweisen	152
3.2.7	Übungen	154
3.3	Zeilen mit einer DELETE-Anweisung löschen	156
3.3.1	Eine Zeile einer Tabelle löschen	157
3.3.2	Mehrere Zeilen einer Tabelle gleichzeitig löschen	158
3.3.3	Alle Zeilen einer Tabelle gleichzeitig löschen	158
3.3.4	Übungen	159
3.4	Alle Zeilen einer Tabelle mit einer TRUNCATE-Anweisung löschen	161
3.4.1	Die TRUNCATE-Anweisung anwenden	161
3.4.2	Übungen zum Thema »Alle Zeilen einer Tabelle mit einer TRUNCATE-Anweisung löschen«	162

4 Tabellen anlegen (CREATE TABLE) 165

4.1	Datentypen	165
4.1.1	Datentypen für ganze Zahlen	169
4.1.2	Datentypen für rationale Zahlen	171
4.1.3	Datentypen für Datum und Zeit	172
4.1.4	Datentypen für Zeichenketten	173
4.1.5	Übungen	176

4.2	Datentypen umwandeln	179
4.3	Explizite und implizite Typkonvertierung	180
4.3.1	Explizite Typkonvertierung	180
4.3.2	Implizite Typkonvertierung	181
4.3.3	Übungen	186
4.4	Einfache Tabellen mit CREATE TABLE erstellen	187
4.4.1	Zielstruktur der Tabelle	187
4.4.2	Tabellen mit der CREATE TABLE-Anweisung anlegen	188
4.4.3	Tabellen mit einer DROP-Anweisung löschen	191
4.4.4	Eine Tabelle mit einem Primärschlüssel ausstatten	192
4.4.5	Automatisch hochzählende numerische Primärschlüsselspalten festlegen	194
4.4.6	Reservierte Schlüsselwörter	197
4.4.7	Übungen	198
4.5	Spalten Einschränkungen (CONSTRAINTS) zuordnen	199
4.5.1	Spalten als Pflichtfelder (NOT NULL) definieren	199
4.5.2	Spalten mit einer UNIQUE-Einschränkung versehen	203
4.5.3	Standardwerte mit DEFAULT für Spalten festlegen	205
4.5.4	Bedingungen mit einer CHECK-Einschränkung für Spalten festlegen	207
4.5.5	Übungen	210
4.6	Spalten auf Tabellenebene Einschränkungen (CONSTRAINT) zuordnen	214
4.6.1	Einen Primärschlüssel auf Tabellenebene festlegen	214
4.6.2	Eine UNIQUE-Einschränkung auf Tabellenebene festlegen	218
4.6.3	Eine CHECK-Einschränkung auf Tabellenebene festlegen	221
4.6.4	Übungen	223

5 Mengenoperationen anwenden 229

5.1	Mengenoperationen auf Ergebnistabellen anwenden	229
5.1.1	Eine Vereinigungsmenge aus zwei Mengen bilden	229
5.1.2	Eine Schnittmenge bilden	233
5.1.3	Eine Differenzmenge bilden	235
5.2	Eine Vereinigungsmenge bilden (UNION)	236
5.2.1	Vereinigungsmengen von Ergebnistabellen bilden	237
5.2.2	Übungen	243

5.3	Die Schnittmenge von Ergebnistabellen bilden (INTERSECT)	245
5.3.1	Schnittmengen von Ergebnistabellen	245
5.3.2	Übungen	247
5.4	Eine Differenzmenge aus Ergebnistabellen bilden (EXCEPT)	250
5.4.1	Differenzmenge von Ergebnismengen bilden	250
5.4.2	Übungen	251
5.5	Mengenoperationen in Kombination mit einer WHERE-Klausel verwenden	253
5.5.1	Vor einer Vereinigungsoperation mit UNION filtern	253
5.5.2	Übung	254
5.6	Vereinigungsmengen in Kombination mit einer ORDER BY-Klausel	255
5.6.1	Übungen	256

6 Benutzer, Rollen und ihre Berechtigungen 259

6.1	Benutzer anlegen (CREATE USER)	260
6.1.1	Nutzer in einer MySQL- oder MariaDB-Datenbank anlegen	260
6.1.2	Nutzer in einer PostgreSQL-Datenbank anlegen	260
6.1.3	Nutzer in einer MS SQL Server-Datenbank anlegen	261
6.2	Benutzer entfernen	261
6.3	Eine Verbindung für einen Datenbankbenutzer erstellen	262
6.3.1	Verbindung für eine MySQL-Datenbank einrichten	262
6.3.2	Verbindung für eine MariaDB-Datenbank herstellen	263
6.3.3	Verbindung für eine PostgreSQL-Datenbank herstellen	264
6.3.4	Verbindung für eine MS SQL Server-Datenbank herstellen	267
6.4	Berechtigungen verwalten	268
6.4.1	Berechtigungen vergeben (GRANT)	268
6.4.2	Berechtigungen entziehen (REVOKE)	270
6.5	Mit Rollen Berechtigungen zuordnen	270
6.5.1	Rollen anlegen (CREATE ROLE)	270
6.5.2	Rollen mit Berechtigungen ausstatten	271
6.5.3	Rollen Datenbanknutzern zuordnen	271
6.5.4	Rollen Berechtigungen entziehen	272
6.5.5	Rollen entfernen	272
6.6	Übungen	273

7 Datenbanken modellieren 279

7.1 Anforderungskatalog	279
7.2 Entitäten identifizieren und modellhaft abbilden	280
7.2.1 Entitäten identifizieren	280
7.2.2 Informationen zu den Entitäten ermitteln	281
7.2.3 Schlüsselattribute für Entitäten identifizieren	281
7.2.4 Die Wertebereiche von Attributen erkennen	284
7.2.5 Zwischen Pflichtattributen und optionalen Attributen unterscheiden	286
7.3 Beziehungen zwischen Entitäten festlegen	287
7.3.1 Beziehungen im Entity-Relationship-Modell definieren	288
7.3.2 Kardinalitäten von Beziehungen erkennen	289
7.3.3 Eine besondere 1:n-Beziehung – oder Entitäten, die auf sich selbst verweisen	296
7.3.4 Starke und schwache Entitäten unterscheiden	298
7.3.5 Die Beziehungstypen »identifying« und »non-identifying«	301
7.4 Datenmodelle in der UML-Notation darstellen	302
7.5 Übungen	305

8 Datenmodelle optimieren (Normalisierung) 309

8.1 Redundanzen erkennen	309
8.1.1 Was ist eine Redundanz?	309
8.1.2 Was bedeutet Normalisierung?	311
8.2 Die 1. Normalform anwenden	312
8.3 Die 2. Normalform anwenden	314
8.4 Die 3. Normalform anwenden	316
8.5 Denormalisierung	318
8.6 Übungen	320

9 Datenmodelle in Tabellen überführen 323

9.1 Die Ausbildungsdatenbank anlegen	323
9.1.1 Eine neue Datenbank mit UTF-8-Zeichensatz anlegen (MySQL, MariaDB)	324
9.1.2 Eine neue Datenbank mit UTF-8-Zeichensatz anlegen (PostgreSQL)	324
9.1.3 Eine neue Datenbank mit Unicode-Zeichensatz anlegen (MS SQL Server)	324
9.1.4 Übung	325
9.2 Tabellen mit Beziehungen zu anderen Tabellen erstellen	325
9.2.1 Die Ausbildungsdatenbank im Modell erfassen	325
9.2.2 Tabellen erstellen, die in einer 1:1-Beziehung stehen	326
9.2.3 Tabellen erstellen, die in einer 1:n-Beziehung stehen	328
9.2.4 Tabellen erstellen, die in einer m:n-Beziehung stehen	330
9.2.5 Tabellen erstellen, die zu sich selbst in Beziehung stehen	331
9.3 Übung	332
9.4 Die referenzielle Integrität verstehen	334

10 Operationen auf Tabellen in Beziehungen anwenden 341

10.1 Zeilen in Tabellen einfügen, die in Beziehung zueinander stehen	341
10.1.1 Zeilen in die Tabelle »auszubildender« einfügen	341
10.1.2 Zeilen in die Tabelle »ausbildungsberuf« einfügen	342
10.1.3 Zeilen in die Tabelle »lehrfach« einfügen	342
10.1.4 Zeilen in die Tabelle »adresse« (inklusive der Beziehungen) einfügen	343
10.1.5 Zeilen in die Tabelle »ausbildungsvertrag« (inklusive der Beziehungen) einfügen	344
10.1.6 Zeilen in die Tabelle »beruflehrfach« (inklusive der Beziehungen) einfügen	344
10.1.7 Zeilen in die Tabelle »mitarbeiterausbildungsbetrieb« (inklusive der Beziehungen) einfügen	345
10.1.8 Übungen	347

10.2	Zeilen aus Tabellen, die in Beziehung stehen, mit JOIN verbunden abfragen	351
10.2.1	Zeilen mit einem INNER JOIN verbinden	352
10.2.2	Zeilen mit einem LEFT OUTER JOIN verbinden	354
10.2.3	Zeilen mit einem RIGHT OUTER JOIN verbinden	357
10.2.4	Zeilen mit einem FULL OUTER JOIN verbinden	360
10.2.5	Einen FULL OUTER JOIN unter MySQL oder MariaDB nachbilden	362
10.2.6	Zeilen mit einem CROSS JOIN verbinden	366
10.2.7	Zeilen von drei Tabellen mit einem INNER JOIN verbinden	368
10.2.8	Spalten in einem JOIN über Tabellennamen referenzieren	370
10.2.9	Spalten in einem JOIN über Tabellenaliasse referenzieren	373
10.2.10	Zeilen mit einem Self Join verbinden	375
10.2.11	Zeilen mit einem INNER JOIN ohne Schlüsselvergleich verbinden	377
10.2.12	Übungen	379
10.3	Beziehungen (Schlüsselbeziehungen) ändern	388
10.3.1	Beziehungen aus Zeilen aus einer Kindtabelle ändern	388
10.3.2	Beziehungen aus Zeilen einer Elterntabelle ändern (ON UPDATE CASCADE)	390
10.3.3	Übungen	394
10.4	Beziehungen (Schlüsselbeziehungen) aufheben oder löschen	399
10.4.1	Zeilen aus Kindtabellen auf NULL setzen	399
10.4.2	Zeilen aus Kindtabellen löschen	402
10.4.3	Zeilen aus Elterntabellen löschen	403
10.4.4	Übungen	406

11 Transaktionen 411

11.1	Forderungen an relationale Datenbanksysteme	412
11.2	Transaktionen verstehen	414
11.2.1	Allgemeiner Aufbau einer Transaktion	414
11.2.2	Einen atomaren Datenzustand mit Transaktionen sicherstellen	415
11.2.3	Transaktionen mit ROLLBACK rückgängig machen	416
11.2.4	Operationen mit Transaktionen isoliert ausführen	419
11.3	Übungen	421

12 Tabellenstrukturen verändern 423

12.1 Eine Tabelle umbenennen	423
12.2 Spalten einer Tabelle ändern	425
12.2.1 Eine Spalte umbenennen	425
12.2.2 Den Datentyp einer Spalte ändern	427
12.2.3 Eine Spalte als Primärschlüsselspalte definieren	429
12.2.4 Einer Spalte eine NOT NULL-Einschränkung zuordnen	430
12.2.5 Einer Spalte eine NULL-Einschränkung zuordnen	431
12.2.6 Einer Spalte einen Standardwert (DEFAULT VALUE) zuordnen	432
12.2.7 Einer Spalte eine UNIQUE-Einschränkung zuordnen	435
12.2.8 Eine Spalte mit einer CHECK-Einschränkung versehen	436
12.3 Spalten hinzufügen und entfernen	438
12.3.1 Einer Tabelle eine Spalte hinzufügen	438
12.3.2 Eine Spalte aus einer Tabelle entfernen	439
12.4 Beziehungen zwischen Tabellen herstellen und entfernen	440
12.5 Übungen	443

13 Mit SQL rechnen 451

13.1 Spaltenwerte addieren	452
13.2 Spaltenwerte subtrahieren	454
13.3 Spaltenwerte multiplizieren	455
13.4 Spaltenwerte dividieren	455
13.5 Den Restwert einer Division von Spaltenwerten berechnen	456
13.6 Nach dem Ergebnis einer Berechnung filtern	457
13.7 Nach dem Ergebnis einer Berechnung sortieren lassen	458
13.8 Übungen	459

14 Skalarfunktionen anwenden 463

14.1 Funktionen für Textwerte	464
14.1.1 Zeichenkette in Kleinbuchstaben umwandeln (LOWER)	465
14.1.2 Spaltenwerte in Großbuchstaben umwandeln (UPPER)	465

14.1.3	Spaltenwerte von führenden und endenden Leerzeichen befreien (TRIM)	466
14.1.4	Text aus Spaltenwerten extrahieren (SUBSTRING)	469
14.1.5	Textspaltenwerte verkettet ausgeben	471
14.1.6	Übungen	474
14.2	Funktionen für Zahlenwerte	476
14.2.1	Die Länge einer Zeichenkette ermitteln (CHAR_LENGTH, LEN)	477
14.2.2	Die Startposition einer Zeichenkette in einem Textwert ermitteln (POSITION, CHARINDEX)	478
14.2.3	Potenzen berechnen (POWER)	479
14.2.4	Eine Quadratwurzel berechnen (SQRT)	480
14.2.5	Übungen	481
14.3	Verschachtelte Funktionsaufrufe	482
14.3.1	Übungen	486

15 Bedingungslogik 487

15.1	Die CASE-Klausel	487
15.2	Bedingungslogik in einer Spaltenauswahlliste einer SELECT-Anweisung anwenden	488
15.3	Bedingungslogik in einer ORDER BY-Klausel anwenden	490
15.4	Übungen	492

16 Mit Zeit und Datum arbeiten 495

16.1	Datumsformate	496
16.2	Skalarfunktionen für Zeit- und Datumsangaben in SQL nutzen	496
16.2.1	Datum, Zeit und Zeitstempel vom Datenbankserver ermitteln lassen	497
16.2.2	Ergebnislisten mit einem Berichtsdatum versehen	498
16.2.3	Übungen	499
16.3	Zeit- und Datumsangaben formatieren	499
16.3.1	Datumsformatierung unter MySQL und MariaDB (DATE_FORMAT)	500
16.3.2	Datumsformatierung unter PostgreSQL (TO_CHAR)	505

16.3.3	Datumsformatierung unter MS SQL Server (FORMAT)	510
16.3.4	Übungen	514
16.4	Datumsangaben extrahieren (EXTRACT)	515
16.4.1	Übungen	519
16.5	Mit Datumsangaben rechnen	520
16.5.1	Mit Datumswerten rechnen unter MySQL und MariaDB	521
16.5.2	Mit Datumswerten rechnen unter PostgreSQL	523
16.5.3	Mit Datumswerten rechnen unter MS SQL Server	524
16.5.4	Übungen	526

17 Spaltenwerte gruppieren (GROUP BY) 529

17.1	Die Aggregatfunktion COUNT anwenden	530
17.1.1	Übungen	535
17.2	Die Aggregatfunktion SUM anwenden	535
17.2.1	Übungen	536
17.3	Die Aggregatfunktion AVG anwenden	537
17.3.1	Übungen	538
17.4	Die Aggregatfunktion MAX anwenden	539
17.4.1	Übungen	540
17.5	NULL-Werte berücksichtigen	541
17.5.1	Übungen	545
17.6	Nach aggregierten Werten einer Gruppierung filtern (HAVING)	545
17.6.1	Übungen	547
17.7	Nach zwei oder mehr Spalten gruppieren	548
17.7.1	Übungen	550

18 Mächtiges Werkzeug: Die Unterabfragen (Subqueries) 553

18.1	Unterabfragen, die in Korrelation zueinander stehen	554
18.1.1	Übungen	558
18.2	Unterabfragen, die nicht in Korrelation zueinander stehen	560
18.2.1	Übungen	565

18.3 Vergleichsoperatoren auf Unterabfragen mit ANY, SOME und ALL anwenden	566
18.3.1 Übungen	569
18.4 Auf die Existenz von Ergebniszeilen aus Unterabfragen prüfen (EXISTS)	571
18.4.1 Übungen	574

19 Views: Abfragen in virtuellen Tabellen speichern 575

19.1 Einfache Views anlegen	576
19.1.1 Übungen	579
19.2 Views und ORDER BY	581
19.2.1 Übungen	583
19.3 INSERT, UPDATE und DELETE auf Views anwenden	584
19.3.1 Eine INSERT-Anweisung auf Views anwenden	584
19.3.2 Eine UPDATE-Anweisung auf Views anwenden	588
19.3.3 Eine DELETE-Anweisung auf Views anwenden	590
19.3.4 Views, auf die keine INSERT-, DELETE- oder UPDATE-Anweisung angewendet werden kann	592
19.3.5 Übungen	593
19.4 Views entfernen oder ersetzen	596
19.4.1 Übungen	597

20 Abfragen mit einem Index optimieren 599

20.1 Einführung	599
20.2 Syntax: Index erstellen	602
20.3 Eine Tabelle mit vielen Zeilen generieren	602
20.4 Einen Index für eine Tabelle anlegen	604
20.5 Einen Index über mehrere Spalten anlegen	606
20.6 Den Index einer Tabelle löschen	609
20.7 Fremdschlüsselspalten indexieren	609
20.8 Übungen	613

21 Skalarfunktionen entwickeln 617

21.1 Funktionen mit einem Rückgabewert erstellen	618
21.1.1 MySQL und MariaDB	619
21.1.2 PostgreSQL	621
21.1.3 MS SQL Server	622
21.2 Funktionen mit Parameter entwickeln	623
21.2.1 MySQL und MariaDB	624
21.2.2 PostgreSQL	625
21.2.3 MS SQL Server	626
21.3 Funktionen entfernen	627
21.4 Übungen	628

22 Prozeduren entwickeln 631

22.1 Prozeduren ausführen	631
22.2 Prozeduren anlegen	632
22.2.1 Prozeduren anlegen (MySQL und MariaDB)	632
22.2.2 Prozeduren anlegen (PostgreSQL)	634
22.2.3 Prozeduren anlegen (MS SQL Server)	635
22.3 Übung	636
 Index	 639

Materialien zum Buch

Auf der Webseite zu diesem Buch stehen folgende Materialien für Sie zum Download bereit:

- ▶ alle Beispieldatenbanken für MySQL, Microsoft SQL Server sowie PostgreSQL
- ▶ Installationsanleitungen für Microsoft SQL Server und PostgreSQL

Gehen Sie auf www.rheinwerk-verlag.de/6020. Klicken Sie auf den Reiter MATERIALIEN ZUM BUCH. Sie sehen die herunterladbaren Dateien samt einer Kurzbeschreibung des Dateiinhalts. Klicken Sie auf den Button HERUNTERLADEN, um den Download zu starten. Je nach Größe der Datei (und Ihrer Internetverbindung) kann es einige Zeit dauern, bis der Download abgeschlossen ist.

Danke, Ingrid!

An dieser Stelle möchte ich mich zudem bei allen Mitwirkenden des Verlags für die gute Zusammenarbeit bedanken. Mein Dank gilt Korrektorin Frau Biedermann und natürlich dem Lektoratsteam im Rheinwerk Verlag. Herausgekommen ist ein Fachbuch über SQL, das Einsteigern Schritt für Schritt erklärt, was es mit SQL auf sich hat. Der »Einstieg in SQL« ist das Ergebnis einer hervorragenden Teamarbeit und nicht das Werk eines einzelnen.

Ich wünsche Ihnen viel Spaß beim Lesen, Verstehen und Ausprobieren. Und nochmal vielen Dank Christoph, Josha und Sebastian!

Michael Laube

Kapitel 1

Grundlagen kennenlernen und verstehen

Datenbanken und SQL haben in der Informatik die Aufgabe, Daten auf eine einfache Art und Weise zu verwalten. Zentrale Elemente sind die Abfragesprache SQL selbst, die Tabellen, in denen die Daten gespeichert werden, und schließlich die Datenbanksysteme, in denen die Tabellen und Daten hinterlegt sind.

Sie haben eine gute Wahl getroffen, als Sie sich entschieden haben, die Datenbankabfragesprache SQL zu lernen!

Datenbanken, die die Abfragesprache SQL unterstützen, sind weltweit verbreitet. Sie werden in zahlreichen Unternehmen und Institutionen verwendet, um Daten bzw. Informationen zu strukturieren und effektiv zu verwalten. Denken Sie nur an Versicherungen, Banken, Behörden und viele weitere Institutionen, die darauf angewiesen sind, Informationen sicher und dauerhaft zu verwalten.

Vermutlich interessiert Sie auch, wofür SQL überhaupt steht: *Structured Query Language*.

Structured Query Language (SQL)

S → Structured (engl. für »strukturiert«)

Q → Query (engl. für »Abfrage«)

L → Language (engl. für »Sprache«)

SQL ist eine *strukturierte Abfragesprache* für Datenbanken.



1.1 Die Tabelle als zentrales Element

Im Zentrum der Sprache SQL steht die *Tabelle*. Mit SQL können Sie folgende Hauptfunktionen auf eine Tabelle anwenden:

- Daten in eine Tabelle *einfügen*
- Daten aus einer Tabelle *abfragen*

- Daten in einer Tabelle *aktualisieren*
- Daten aus einer Tabelle *löschen*

In der Fachsprache werden diese Funktionen oftmals mit dem Begriff *CRUD* zusammengefasst, der für die englischen Wörter *Create*, *Read*, *Update* und *Delete* steht. Es sind die Grundfunktionen jedes Datenbanksystems.

Beginnen werden Sie in diesem Buch mit sogenannten **SELECT**-Abfragen, die auf eine Tabelle angewendet werden können. Die in ihrer Grundstruktur einfachen Abfragen ermöglichen es Ihnen, Daten einer Tabelle abzurufen bzw. abzufragen. Außerdem hält SQL alles bereit, was nötig ist, um Daten in Tabellen einzufügen, zu ändern und natürlich auch zu löschen.

Sie merken schon, die Tabelle ist das zentrale Element von SQL.

Tabellen und ihre Struktur

Wahrscheinlich haben Sie bereits einmal in einem anderen Zusammenhang mit Tabellen zu tun gehabt, beispielsweise mit Microsoft Excel. Dann wissen Sie, dass die Struktur einer Tabelle grundsätzlich sehr einfach ist: Eine Tabelle besteht aus *Zeilen* und *Spalten*. Eine Zeile, die auch als *Datensatz* bezeichnet wird, enthält Werte, die wiederum Spalten zugeordnet sind.

Betrachten wir zur Illustration ein Beispiel. In Abbildung 1.1 sehen Sie die Tabelle, die von der Öffentlichkeit vermutlich am meisten diskutiert wird: die Tabelle der ersten Fußball-Bundesliga.

PL. 	TEAM	SP.	S	U	N	TORE	DIFF.	PUNKTE
1 	 Bayern München	5	4	1	0	17:4	13	13
2 	 Eintracht Frankfurt	5	4	0	1	11:6	5	12
3 	 RB Leipzig	5	3	2	0	8:2	6	11
4 	 Bayer 04 Leverkusen (M, P)	5	3	1	1	14:10	4	10
5 	 Borussia Dortmund	5	3	1	1	11:9	2	10
6 	 1. FC Heidenheim	5	3	0	2	10:7	3	9
7 	 SC Freiburg	5	3	0	2	8:7	1	9
8 	 VfB Stuttgart	5	2	2	1	14:10	4	8

Abbildung 1.1 Diese Tabelle enthält die essenziellen Daten zur schönsten Nebensache der Welt. (Quelle: <https://www.kicker.de/bundesliga/tabelle>)

Hier sehen Sie auf den ersten Blick, dass EINTRACHT FRANKFURT am 5. Spieltag hinter BAYERN MÜNCHEN steht und dass sich der SC FREIBURG auf einem starken 7. Platz befindet. Die Informationen, die uns die Spalten der Tabelle liefern, sind:

- ▶ PL. – der Platz, auf dem sich Ihr Lieblingsclub befindet
- ▶ TEAM – die Bezeichnung des Clubs
- ▶ SP. – die Anzahl der gespielten Spiele
- ▶ S – die Anzahl der Siege
- ▶ U – die Anzahl der unentschieden gespielten Spiele
- ▶ N – die Anzahl der Niederlagen
- ▶ TORE – die Anzahl der bisher erzielten Tore
- ▶ DIFF. – die Tordifferenz
- ▶ PUNKTE – die Anzahl der bis dato geholten Punkte

Die Spaltenbezeichnungen im Tabellenkopf, z. B. PL., TEAM, SP. und S, stehen für die *Eigenschaften* der Tabelle. Sie sorgen dafür, dass der Analyst die Zahl der Spiele und die Zahl der Niederlagen auseinanderhalten kann. Darunter stehen die dazugehörigen Zeilen, die die Tabelle mit Inhalt füllen. Einen *Wert* in einer Tabelle ermitteln Sie, indem Sie den Schnittpunkt der Zeile und der Spalte betrachten. Sie wollen wissen, welcher Club auf Platz 2 ist? Kein Problem: Betrachten Sie Zeile 2 und die Spalte TEAM, und ermitteln Sie den Schnittpunkt. Hier finden Sie den Wert EINTRACHT FRANKFURT.

Zusammenfassung: Definition einer Tabelle

Eine *Tabelle* besteht aus einem Tabellennamen, Zeilen und Spalten.

Der *Name* einer Tabelle fasst die Eigenschaften zusammen, die in einer Tabelle hinterlegt sind, und beschreibt, welche Informationen gespeichert werden.

Die *Eigenschaften* (Spalten) einer Tabelle bilden eine Struktur, in der Informationen geordnet gespeichert werden.

Die Informationen (die Daten) einer Tabelle sind in den Zeilen zu finden. Ein *Wert* innerhalb einer Zeile wird gelesen, indem der Schnittpunkt zwischen einer Zeile und einer Spalte ermittelt wird.



1.2 Eine kleine Historie von SQL

Eine Tabelle habe ich Ihnen bereits vorgestellt. Jetzt widmen wir uns der Frage, wer auf die Idee kam, Tabellen zur Speicherung großer Datenmengen zu nutzen, und auch die Grundlagen für SQL schuf.

Edgar F. Codd entwickelte in den 1970er-Jahren ein mathematisches Modell auf Grundlage der Mengenlehre. Die Struktur einer Tabelle wie der Bundesligatabelle kann auf dieser Grundlage beschrieben werden: Wir betrachten eine Menge. Diese

Menge bezeichnen wir als *Bundesligatabelle* und stattdessen sie mit Elementen wie *Platz* und *Verein* aus. Dies lässt sich in folgender Kurzschreibweise darstellen:

Bundesligatabelle={Pl,Verein,Sp,g,u,v,...}

Mathematisch betrachtet, bedeutet das nichts anderes als eine Definition der Struktur der Bundesligatabelle. In den geschweiften Klammern ist die Menge der Elemente (Spalten) aufgelistet, die für die Informationen der Bundesligatabelle erforderlich sind.

Mit der mathematischen Beschreibung einer Tabelle ist es in der Datenverarbeitung natürlich nicht getan. Denn das Ganze muss noch zum Leben erweckt werden.

Deshalb entwickelte Edgar F. Codd auch eine Sprache, mit der Daten basierend auf einfachsten Operationen abgefragt, eingefügt, gelöscht oder geändert werden können. Das Resultat dieser Überlegungen bildete die Grundlage für das *relationale Datenbankmodell* und die Sprache SQL. Die Arbeit von Edgar F. Codd wurde schließlich gekrönt, indem sie zu einem Standard erhoben wurde. Diese Standards werden heute von der *ISO (International Organization for Standardization)* festgelegt, erweitert und publiziert. In [Abschnitt 1.4](#) erfahren Sie mehr über den SQL-Standard und seine Umsetzung durch unterschiedliche Datenbanksysteme.

1.3 Datenbanksysteme

Unterschiedlichste Anbieter haben zahlreiche Datenbanksysteme auf den Markt gebracht, die SQL unterstützen und Daten bzw. Informationen in Tabellen speichern. Datenbanksysteme, die Informationen in Tabellen speichern, werden als *relationale Datenbanksysteme* bezeichnet. Zu den bekanntesten Vertretern gehören:

- ▶ *IBM DB2*
- ▶ *Oracle DB*
- ▶ *Microsoft SQL Server*
- ▶ *Oracle MySQL*
- ▶ *MariaDB*
- ▶ *PostgreSQL*

Hierbei handelt es sich nur um eine kleine Auswahl von SQL-Datenbanken. Die Auswahl der Datenbanken habe ich bewusst getroffen, um Ihnen einerseits kommerzielle Anbieter und andererseits Open-Source-Vertreter von Datenbanksystemen vorstellen zu können.

Als führendes Datenbanksystem verwenden wir in diesem Buch MySQL. MySQL-Datenbanken sind sehr weit verbreitet. Das liegt unter anderem daran, dass die MySQL-

Datenbank kostenlos angeboten und oft im Verbund mit der Web-Programmiersprache *PHP* verwendet wird. Außerdem steht Ihnen die MySQL-Datenbank für die wichtigsten Betriebssysteme (beispielsweise unterschiedliche Linux-Distributionen, Windows-Systeme, Apple macOS) zur Verfügung. Ein weiterer Vorteil ist die umfangreiche Dokumentation, die Sie jederzeit online unter <http://dev.mysql.com/doc> abfragen können.

Falls Sie stattdessen lieber mit einer MariaDB-, PostgreSQL- oder einer Microsoft SQL Server-Datenbank lernen möchten, wird es Sie freuen zu erfahren, dass ich auf alle vier genannten Datenbanken eingehe und Ihnen jeweils eine Übungsdatenbank zur Verfügung stelle.

Die Datenbankserver MariaDB, PostgreSQL und Microsoft SQL Server sind ebenfalls sehr gut dokumentiert. Die Dokumentationen finden Sie unter:

- ▶ <https://mariadb.com/kb/en/documentation/>
- ▶ <http://www.postgresql.org/docs>
- ▶ <https://learn.microsoft.com/de-de/sql/sql-server/?view=sql-server-ver16>

MariaDB

MariaDB wurde im Jahr 2015 ins Leben gerufen, und es lohnt sich, einige Worte darüber zu verlieren. MariaDB ist ein sogenannter *Fork* von MySQL, also eine Abspaltung von diesem Projekt. Das MariaDB-Projekt bemüht sich, die volle Kompatibilität mit MySQL zu bewahren – die Unterschiede zwischen den beiden Datenbanken sind wirklich mit der Lupe zu suchen und betreffen hauptsächlich Lizenzfragen. (Fast) alle Erläuterungen, die Sie in diesem Buch zu MySQL finden, gelten daher ebenso für MariaDB. Sie finden das Projekt unter:

<https://mariadb.org>



1.4 SQL – ein Standard und seine Umsetzung

Der SQL-Standard der ISO wurde in der Vergangenheit als *ANSI-SQL-Standard* (ANSI: *American National Standards Institute*) bezeichnet. Der ISO-SQL-Standard soll auch international sicherstellen, dass Ihre Abfragen auf allen konformen Datenbanken zuverlässig funktionieren.

SQL: Ein Standard

Die Abfragesprache SQL ist im Standard ISO/IEC 9075-1:2016 festgelegt. Die Spezifikation kann unter www.iso.org gegen ein Entgelt bezogen werden.



Sie können davon ausgehen, dass unabhängig vom verwendeten Datenbanksystem der SQL-Sprachschatz zum größten Teil so umgesetzt wird, wie er im SQL-Standard beschrieben ist. Der kleinere Teil des Sprachumfangs, der nicht so realisiert wird, wie es der SQL-Standard vorsieht, wird als *SQL-Dialekt* bezeichnet. Alle drei hier behandelten Datenbanken haben einen eigenen Dialekt, der sich durch kleinere Unterschiede zum SQL-Standard-Sprachumfang bemerkbar macht.

Aus diesem Grund stelle ich Ihnen SQL anhand von vier unterschiedlichen Datenbanksystemen vor. Einerseits möchte ich Ihnen damit zeigen, dass SQL ein Standard ist und sich daher das Gelernte auf unterschiedliche Datenbanken anwenden lässt. Andererseits möchte ich Sie dafür sensibilisieren, dass SQL in einem kleinen Umfang abhängig vom jeweiligen Hersteller ist und damit im Detail unterschiedlich sein kann. Da es sich aber nur um kleine Unterschiede handelt, können Sie die hier vermittelten Grundlagen auch nutzen, um z. B. auf einer DB2-Datenbank von IBM oder einer Oracle-Datenbank mit SQL zu arbeiten.



SQL mit Hilfe von künstlicher Intelligenz lernen

Vielleicht kennen Sie schon KI-Helfer wie ChatGPT. Diese Werkzeuge können nicht nur Texte in normaler Sprache erstellen, sondern sie beherrschen auch Programmiersprachen wie SQL. Mit den richtigen Anfragen, den sogenannten *Prompts*, können Sie sich auch SQL-Aufrufe erstellen lassen und Informationen zu Ihrem Datenbanksystem erfragen.

Ich möchte Sie aber bitten, diese Möglichkeiten nur mit Bedacht und nicht verantwortungslos zu nutzen. Sie sollten keinesfalls KI-Code, den Sie nicht vollständig verstehen und geprüft haben, in Ihrer Datenbank nutzen. Denken Sie immer daran, dass die KI-Antworten fehlerhaft sein können und abhängig von den Eingaben sind. ChatGPT und andere Tools beruhen auf Modellen, die mit statistischen Wahrscheinlichkeiten der Sprache arbeiten. Damit lassen sich beeindruckende Ergebnisse erzielen, aber Ihr KI-Helfer »versteht« seine Aufgaben nicht wirklich und ist kein Ersatz für das eigene Lernen und Mitdenken.

Wenn Sie nach der Lektüre dieses Buchs die Grundlagen von SQL und Datenbanken gelernt haben, können Sie mit diesem Wissen die KI-Helfer sicherer nutzen und ihre Antworten prüfen und verstehen.



Zusammenfassung: SQL-Dialekte

SQL ist ein Standard, der durch die ISO (*International Organization for Standardization*) vorgegeben wird. Dennoch gibt es im Sprachschatz jeweils abhängig von der verwendeten Datenbank Abweichungen vom Standard. Diese Unterschiede werden als *SQL-Dialekte* bezeichnet.

1.5 Zu diesem Buch

Grundsätzlich gehen wir in diesem Buch vom SQL-Standard aus. Den Grundsprachschatz, den Sie im Rahmen dieses Einstiegs in SQL erlernen, können Sie universal auf allen SQL-Datenbanken anwenden. Bezogen auf die hier behandelten Datenbanksysteme werde ich Sie stets auf die kleinen Unterschiede aufmerksam machen. Und wenn Sie einmal auf den Fall stoßen, dass ein hier beschriebenes Beispiel bei Ihnen nicht funktioniert, weil Sie ein besonderes Feature einer bestimmten Datenbank einsetzen wollen: Glückwunsch, dann sind Sie wahrscheinlich kein Einsteiger mehr und spezialisieren sich auf ein Datenbanksystem eines Anbieters. Achten Sie in diesem Fall besonders auf die Dokumentation der von Ihnen eingesetzten Datenbank.

In Abschnitt 1.6 gebe ich Ihnen einen Leitfaden an die Hand, der Sie bei der Installation der MySQL-Datenbank unterstützt.

Der Leitfaden zur Installation enthält ausschließlich Hinweise, die für die Installation der jeweiligen Datenbank von Bedeutung sind. Von einer gewissen Affinität zu Windows- und Linux-Systemen gehe ich in diesem Fachbuch aus. Unter Windows können Sie die hier vorgestellten Datenbanken sehr einfach installieren. Ausführlicher werde ich Ihnen die Installation einer MySQL-Datenbank unter Windows beschreiben.

In den Downloadmaterialien zum Buch finden Sie das MySQL-Installationsprogramm in der folgenden Version hinterlegt:

- *MySQL 8.0.39 Community Edition, MySQL Workbench 8.0.38*

Falls Sie eine MariaDB-, PostgreSQL- oder MS SQL Server-Datenbank verwenden möchten, laden Sie die Installationsprogramme bitte direkt beim Hersteller herunter. Die entsprechende Installationsanleitung finden Sie in den Materialien zum Buch auf www.rheinwerk-verlag.de/6020. Beachten Sie, dass sich je nach Version Abweichungen ergeben können. Die Erklärungen in diesem Buch beziehen sich auf die Versionen:

- *MariaDB Server 11.5.2, HeidiSQL 12.8*
- *PostgreSQL 16.4, pgAdmin 4, v8.6*
- *MS SQL Server 2022 Express, SQL Server Management Studio 20.1*

In Abschnitt 1.7 zeige ich Ihnen anschließend, wie Sie die Übungsdatenbank auf den hier besprochenen Systemen anlegen und die Daten importieren.

In Abschnitt 1.8 erfahren Sie, wie Sie die MySQL Workbench nutzen, um mit SQL zu beginnen.

Sie werden in diesem Buch viele Kästen und SQL-Beispiele finden. In Hinweiskästen befinden sich besonders wichtige Tipps, die Sie sich unbedingt anschauen sollten.





Außerdem werde ich am Ende der Abschnitte die wichtigsten Informationen noch einmal für Sie zusammenfassen. Dies wird immer mit einem besonderen Icon gekennzeichnet.

In den SQL-Listings finden Sie sowohl eine kürzere einzeilige Schreibweise als auch die etwas längere, aber übersichtlichere Fassung, die sich über mehrere Zeilen erstreckt:

```
SELECT
    name,
    vorname,
    bonus
FROM
    mitarbeiter
WHERE bonus>500;
```

Listing 1.1 Ein auf mehrere Zeilen verteilter SQL-Befehl

Für die Datenbank bedeutet diese Anweisung genau das Gleiche wie der folgende Befehl:

```
SELECT name,vorname,bonus FROM mitarbeiter WHERE bonus>500;
```

Listing 1.2 Das gleiche Beispiel in der Kurzschreibweise

Wenn Sie nur rasch eine SQL-Anweisung ausprobieren wollen, reicht Ihnen wahrscheinlich die Kurzschreibweise aus. Bei längeren, komplexeren Anweisungen wird dies jedoch schnell unübersichtlich. Dann ist es hilfreich, die einzelnen Schritte in der übersichtlichen Schreibweise aufzuschlüsseln. Dabei ist es sinnvoll, jedes SQL-Schlüsselwort in eine Zeile zu schreiben. Egal, ob in der Kurzschreibweise oder der ausführlichen Darstellung: Die SQL-Schlüsselwörter finden Sie hier im Buch immer fett gedruckt und durch Großbuchstaben hervorgehoben. So können Sie sich rasch einen Überblick über die Beispiele verschaffen.

Was diese Schlüsselwörter genau bedeuten, erfahren Sie in den folgenden Kapiteln. Kümmern wir uns aber zunächst darum, die MySQL-Datenbank zu installieren.

1.6 MySQL unter Windows installieren

Um MySQL unter Windows zu installieren, führen Sie die folgenden Schritte aus:

Schritt 1: Laden Sie entweder den MySQL-Installer der MySQL 8.0.39 Community Edition von der Webseite zu diesem Buch unter www.rheinwerk-verlag.de/6020 herunter oder die aktuelle Version von der Website <https://dev.mysql.com/downloads/installer>. Achten Sie dabei darauf, dass das Auswahlfeld unter dem Punkt **SELECT**

OPERATION SYSTEM auf MICROSOFT WINDOWS eingestellt ist, um den MSI-Installer herunterladen zu können. Hier werden zwei Downloads angeboten: zum einem ein Web-Installer und zum anderen ein vollständiger Installer, der alles enthält, was Sie zur Installation brauchen. Wählen Sie den vollständigen Installer aus. Zu erkennen ist er einerseits am Namen und andererseits an der Größe der Installationsdatei von mehr als 310 MB. Die Anmeldeinformation auf der folgenden Seite können Sie gestrost ignorieren. Klicken Sie einfach weiter unten auf der Seite auf NO THANKS, JUST START MY DOWNLOAD, um die Datei ohne Anmeldung herunterzuladen.

Schritt 2: Öffnen Sie als Nächstes das Downloadverzeichnis, und führen Sie einen Doppelklick auf die soeben heruntergeladene Installationsdatei (mit der Endung *.msi*) aus. Falls Ihnen ein Dialogfenster mit einer Warnung angezeigt wird, in der Sie gefragt werden, ob Sie die Datei tatsächlich ausführen wollen, bestätigen Sie sie mit JA. Sollten während des Installationsprozesses weitere Warnmeldungen auftauchen, bestätigen Sie auch diese entsprechend.

Schritt 3: Im Fenster CHOOSING A SETUP TYPE sehen Sie die fünf möglichen Installationsvarianten (siehe [Abbildung 1.2](#)). Wählen Sie die Variante CUSTOM. Diese ermöglicht, nur die Softwarekomponenten auszuwählen, die Sie für den Einstieg in SQL benötigen. Klicken Sie auf NEXT, um zum nächsten Installationsschritt zu gelangen.

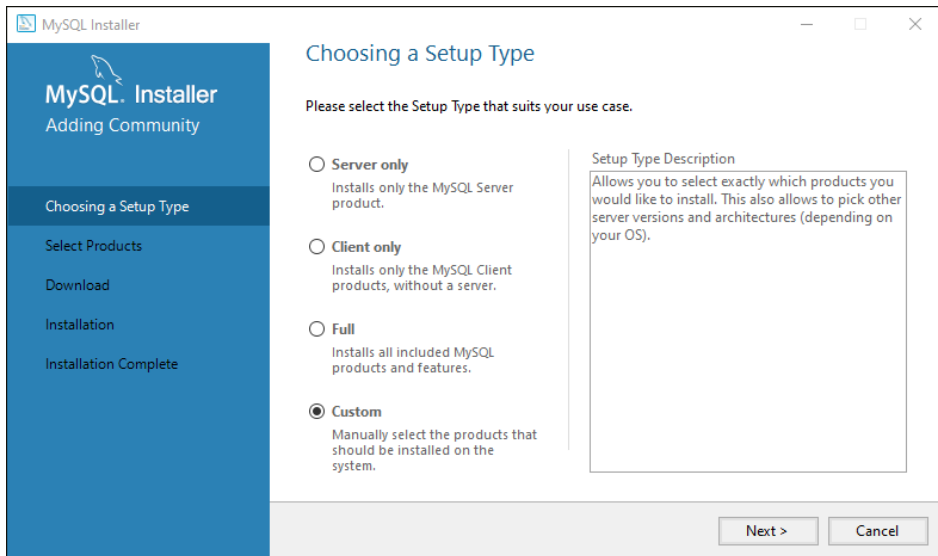


Abbildung 1.2 Installationstyp auswählen

Schritt 4: Im Fenster SELECT PRODUCTS (siehe [Abbildung 1.3](#)) sehen Sie eine Auswahl der Softwarekomponenten, die Sie installieren können.

Im linken Bereich mit der Überschrift AVAILABLE PRODUCTS ist ein Navigationsbaum zu sehen. Klicken Sie einfach auf den Knoten mit dem Pluszeichen, um den Baum so

weit aufzuklappen, dass die benötigten Softwarekomponenten auswählbar sind. Wählen Sie **MYSQL SERVERS • MYSQL SERVERS • MYSQL SERVER 8.0 • MYSQL SERVER 8.0.39–X64**. Klicken Sie im Anschluss auf den grün gewordenen Pfeil nach rechts, der sich zwischen den beiden Fenstern befindet, um das Produkt in das Fenster **PRODUCTS TO BE INSTALLED** zu verschieben.

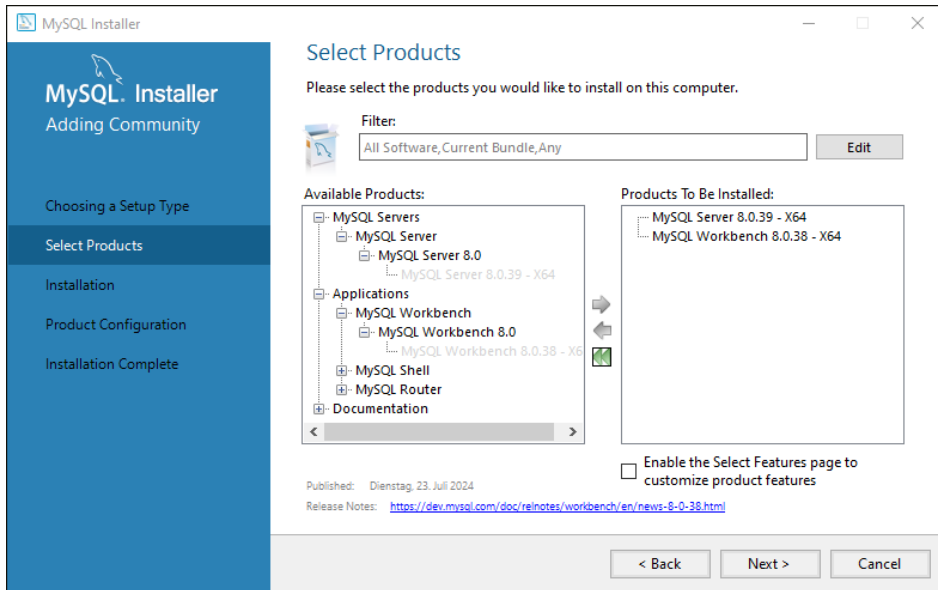


Abbildung 1.3 Zu installierende Komponenten auswählen

Wiederholen Sie diesen Schritt für **APPLICATIONS • MYSQL WORKBENCH • MYSQL WORKBENCH 8.0 • MYSQL WORKBENCH 8.0.39–X64**.

Sollten neuere Versionen von MySQL Server und MySQL Workbench zur Verfügung stehen, wählen Sie bitte diese anstelle der hier genannten.

Klicken Sie anschließend auf den Button **NEXT**, um mit der Installation fortzufahren.

Schritt 5: Im Anschluss sehen Sie das Fenster **INSTALLATION**. Darin werden nochmals die Softwareprodukte angezeigt, die Sie zur Installation ausgewählt haben. Mit einem Klick auf **EXECUTE** werden sie installiert. Sobald die Installation erfolgreich war, werden vor den Softwarekomponenten grüne Häkchen angezeigt, und die **STATUS**-Spalte zeigt **COMPLETE** an (siehe [Abbildung 1.4](#)). Klicken Sie erneut auf den Button **NEXT**, um zum nächsten Schritt zu gelangen.

Schritt 6: Jetzt befinden Sie sich im Fenster **PRODUCT CONFIGURATION**. Hier werden die Softwareprodukte aufgelistet, die zur Konfiguration bereitstehen. Da Sie nichts weiter tun können, klicken Sie direkt auf **NEXT**.

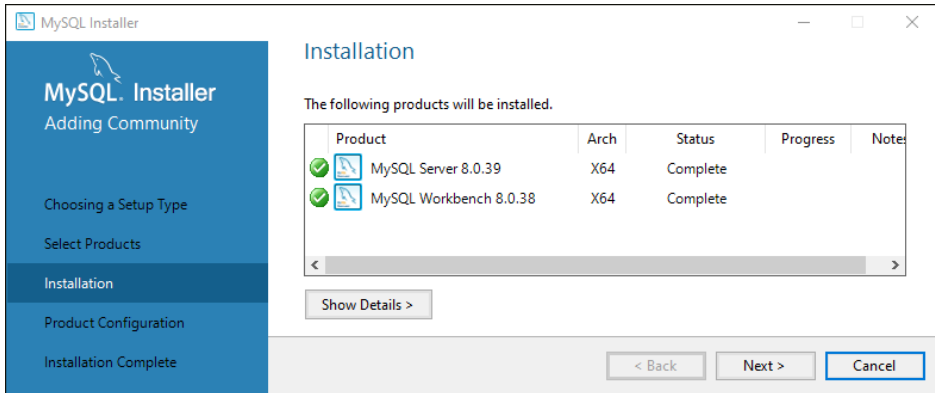


Abbildung 1.4 Die Installation der MySQL Server- und MySQL Workbench-Software wird bestätigt.

Schritt 7: Nach einem erneuten Klick auf NEXT öffnet sich das Fenster TYPE AND NETWORKING (siehe [Abbildung 1.5](#)). Der Konfigurationsdialog enthält einige voreingestellte Netzwerkeinstellungen. Ich empfehle Ihnen, diese beizubehalten und direkt auf NEXT zu klicken, um fortzufahren.

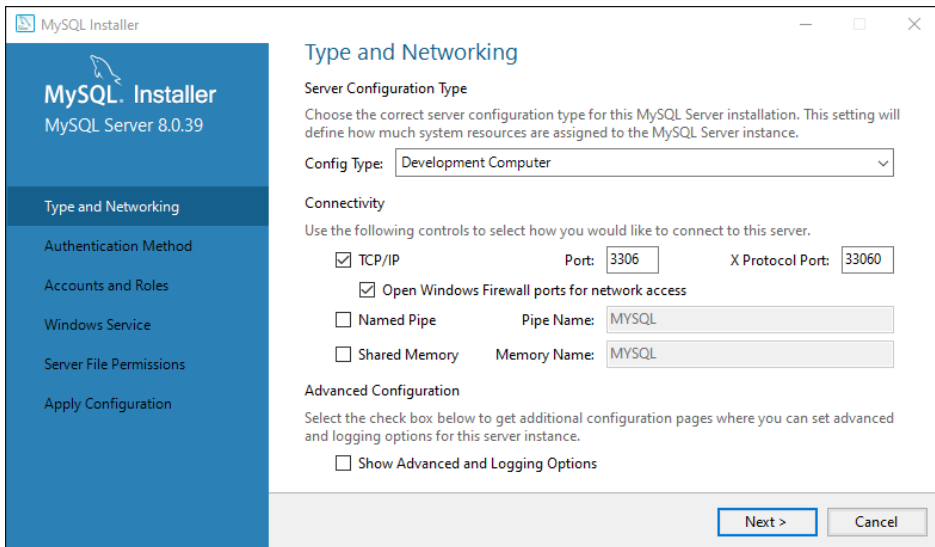


Abbildung 1.5 Typ und Netzwerkeinstellungen konfigurieren

Schritt 8: Im Fenster AUTHENTICATION METHOD (siehe [Abbildung 1.6](#)) können Sie zwischen zwei Optionen wählen. Belassen Sie es auch hier bei der Standardauswahl, und klicken Sie auf den Button NEXT, um die Installation fortzuführen.

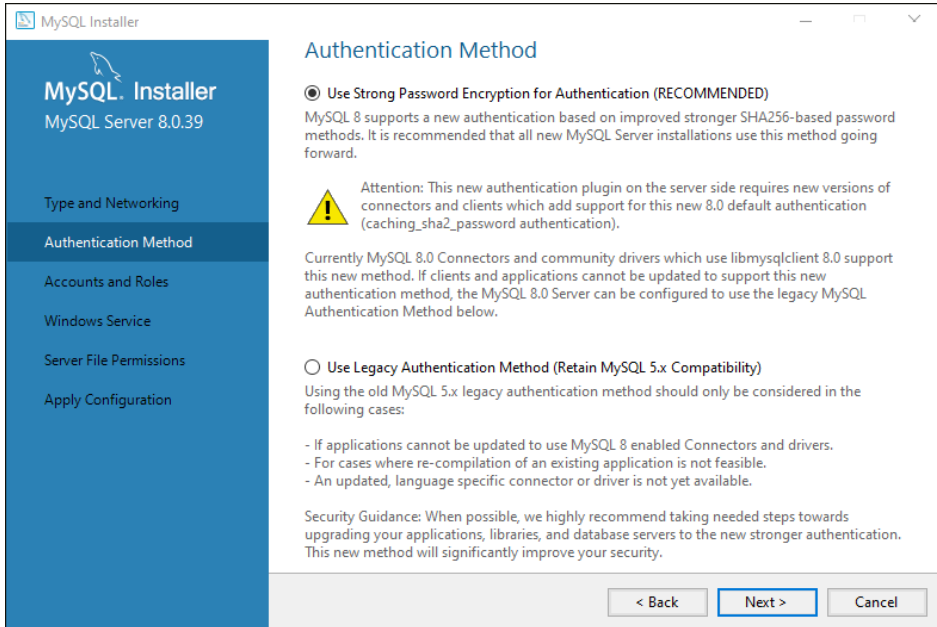


Abbildung 1.6 Die Authentifizierungsmethode auswählen

Schritt 9: Jetzt müssen Sie noch ein Passwort für unseren Superuser (*root*) festlegen. Geben Sie ein Passwort Ihrer Wahl in das Texteingabefeld für **MYSQL ROOT PASSWORD** ein, und bestätigen Sie es, indem Sie es noch einmal in das Texteingabefeld **REPEAT PASSWORD** eingeben.

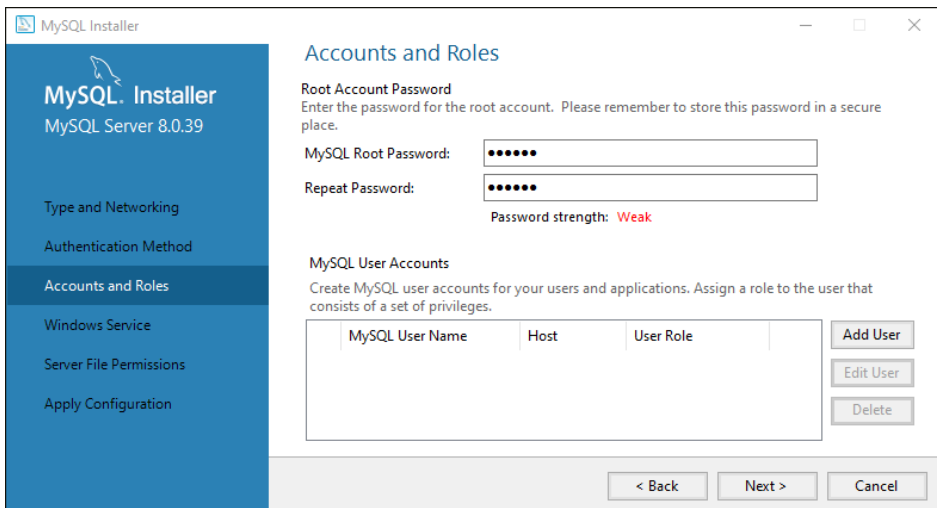


Abbildung 1.7 Ein Passwort für den Superuser (»root«) vergeben

Der Einfachheit halber empfehle ich Ihnen für den Übungsbetrieb, ein simples Passwort auszuwählen. Klicken Sie wieder auf den Button NEXT, um zum nächsten Installationsschritt zu gelangen.

Hinweis zur Nutzung des Administrators »root«

Im Produktivbetrieb einer Datenbank würden Sie aus Sicherheitsgründen natürlich niemals mit dem administrativen Nutzer *root* arbeiten, wenn Sie SQL-Abfragen durchführen. Für den Übungsbetrieb können Sie jedoch getrost den Administrator *root* verwenden. Mehr zu Benutzern, Rollen und Berechtigungen erfahren Sie in [Kapitel 6](#).

Schritt 10: In [Abbildung 1.8](#) sehen Sie das Konfigurationsfenster WINDOWS SERVICE. Der MySQL-Datenbankserver schlägt vor, sich als Windows-Dienst einzurichten. Idealerweise belassen Sie es dabei und klicken auf den Button NEXT, um fortzufahren.

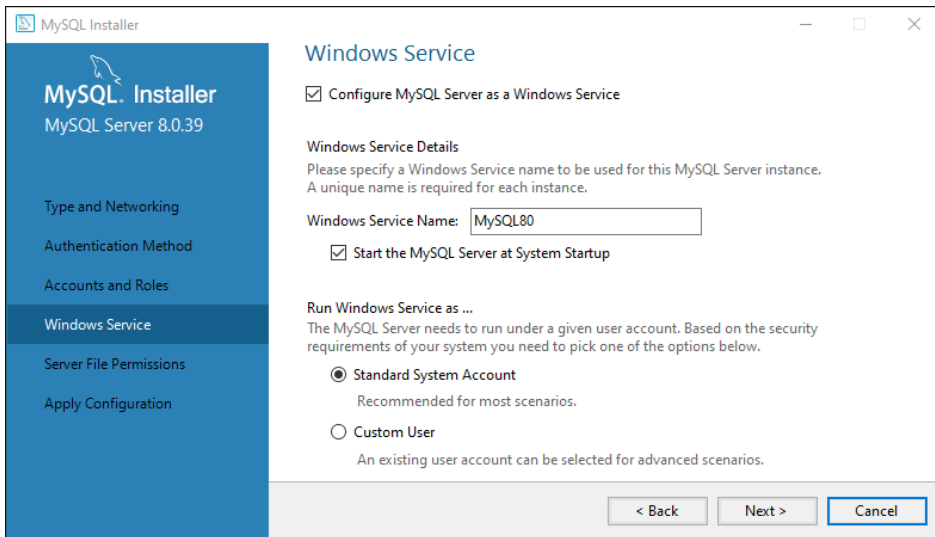


Abbildung 1.8 MySQL Server als Windows-Dienst einrichten

Schritt 11: Als Nächstes bestätigen Sie im Konfigurationsfenster SERVER FILE PERMISSIONS ([Abbildung 1.9](#)) eine Berechtigungsänderung für das Datenverzeichnis der MySQL-Datenbank, die durch das Installationsprogramm vorgenommen wird. Bitte belassen Sie diese Einstellung so wie vorgegeben, und klicken Sie auf den Button NEXT, um fortzufahren.

Schritt 12: Im Fenster APPLY CONFIGURATION müssen Sie mit einem Klick auf EXECUTE nur noch der Konfiguration zustimmen.

Gleich im Anschluss bestätigt Ihnen das Installationsprogramm mit grünen Häkchen, welche Konfigurationsschritte bereits ausgeführt worden sind (siehe [Abbil-](#)

dung 1.10). Sobald alle Schritte durchgeführt wurden, können Sie auf **FINISH** klicken, um die Konfiguration abzuschließen.

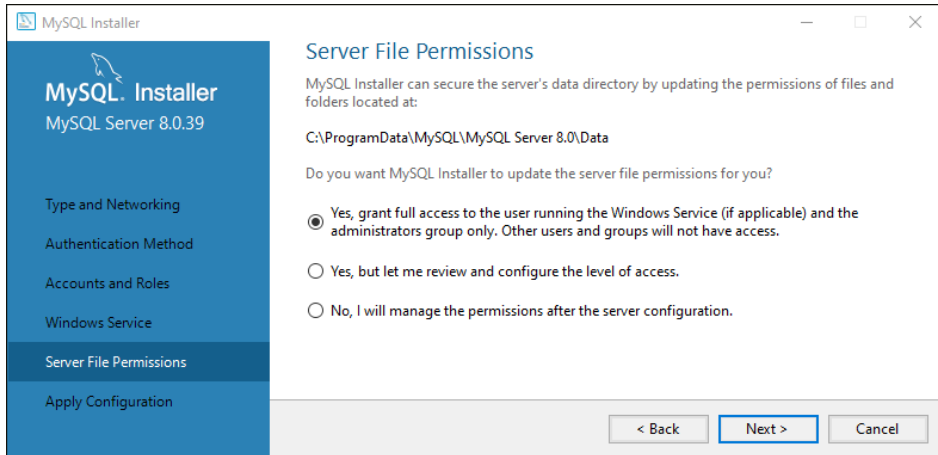


Abbildung 1.9 Bestätigung der Berechtigungseinstellung

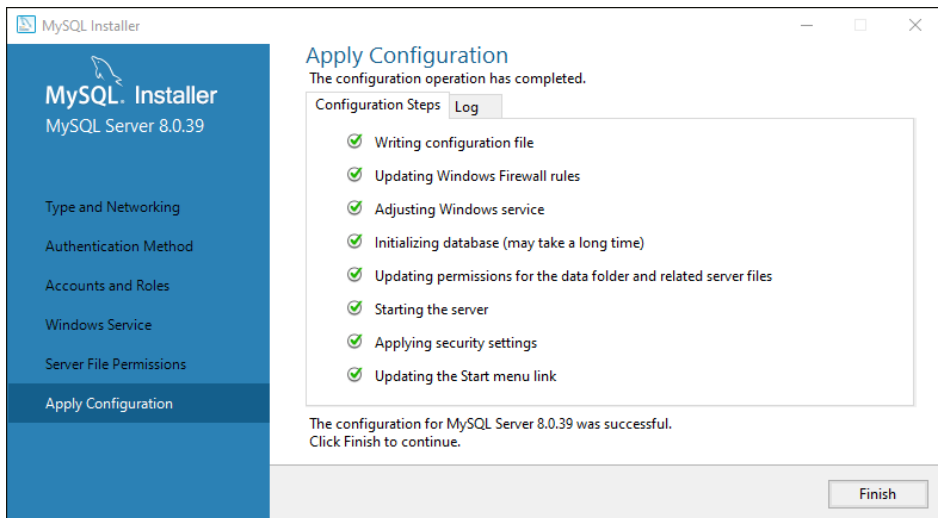


Abbildung 1.10 Bestätigung der Konfiguration

Abschließend erhalten Sie eine Bestätigung über die erfolgreiche Installation und Konfiguration von MySQL Server. Klicken Sie abermals auf **NEXT**.

Jetzt haben Sie es endlich geschafft! MySQL Server und die MySQL Workbench wurden installiert, und sämtliche erforderlichen Einstellungen wurden vorgenommen.

Aktivieren Sie, falls es noch nicht ausgewählt sein sollte, im nächsten Fenster das Kontrollkästchen **START MYSQL WORKBENCH AFTER SETUP** (siehe Abbildung 1.11).

Nun müssen Sie nur noch auf **FINISH** klicken, um die Installation abzuschließen und die MySQL Workbench zu starten.

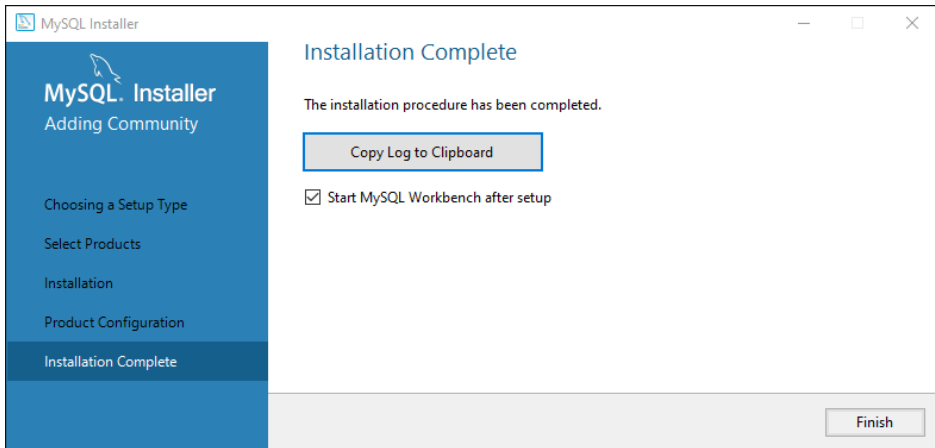


Abbildung 1.11 Bestätigung der Konfiguration

Im folgenden Willkommensfenster der MySQL Workbench sollten Sie darauf achten, im linken Navigationsbereich nur das Symbol mit dem Delphin auszuwählen (siehe Abbildung 1.12).

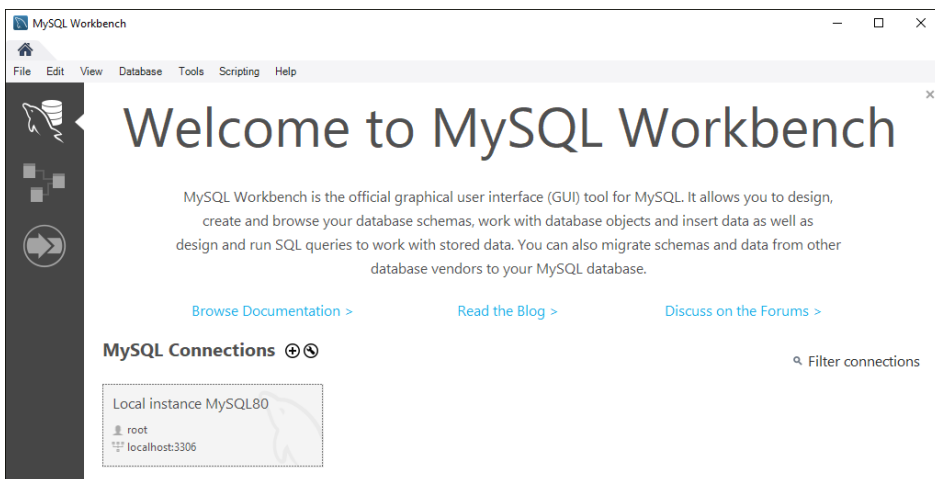


Abbildung 1.12 Die MySQL Workbench nach dem Start

Im Hauptfenster wird Ihnen eine grau hinterlegte Fläche mit der Bezeichnung **LOCAL INSTANCE** angezeigt, auf die eine MySQL-Versionsbezeichnung folgt. Wenn Sie auf diese Fläche klicken, öffnet sich nach der Eingabe des Passworts für den Benutzer **root** die Editoransicht. Sie ist im Prinzip ein Texteditor, in dem Sie SQL-Anweisungen eingeben und ausführen können.

1.7 Die MySQL-Übungsdatenbank anlegen

An dieser Stelle gehe ich davon aus, dass Sie, wie in Abschnitt 1.6 beschrieben, die MySQL-Datenbank installiert haben.

Die Übungsdatenbank trägt den selbsterklärenden Namen *uebungsdatenbank*. Beachten Sie, dass der Name vollständig kleingeschrieben ist. Der deutsche Umlaut »ü« wurde durch die Buchstaben »ue« ersetzt, da sich ansonsten Probleme mit Zeichensätzen ergeben können. Daher gehört es in der Programmierung zum guten Stil, Bezeichner stets ohne Umlaute zu verwenden.

Eine wichtige Bemerkung vorab: Um die Übungsdatenbank anzulegen, benötigen Sie *Administratorrechte* für die verwendete Datenbank. Wenn Sie der Installationsanleitung in Abschnitt 1.6 gefolgt sind, besitzen Sie bereits Administratorrechte. Während der Installation haben Sie für den Benutzer *root* ein Passwort vergeben.



Administratoren unter MySQL

Unter MySQL heißt der Nutzer, der über administrative Rechte verfügt, *root*.

Um die Übungsdatenbank erfolgreich anzulegen, benötigen Sie Administratorrechte für die Datenbank oder entsprechende Berechtigungen, die es Ihnen erlauben, Datenbanken oder Tabellen anzulegen und natürlich Daten dort einzufügen.

Kommen wir jetzt zu den Vorbereitungen, die für das Importieren der Übungsdatenbank notwendig sind. Laden Sie die Importskripte für die Übungsdatenbanken herunter. Beachten Sie, dass die Downloaddatei die Importskripte für alle vier Datenbanken enthält, die ich hier vorstelle.



Download der Importskripte

Sie finden die Importskripte auf der Webseite des Rheinwerk Verlags zu diesem Buch unter www.rheinwerk-verlag.de/6020.

Wählen Sie dort den Tab mit der Bezeichnung MATERIALIEN ZUM BUCH. Die Importskripte stehen jeweils als *ZIP*-Datei zum Download bereit. Bitte laden und entpacken Sie diese Datei. Das Entpacken der *ZIP*-Datei kann beispielsweise in Ihrem *Downloads*-Ordner erfolgen.

In dem soeben entpackten Verzeichnis *mysql* finden Sie die folgenden Dateien:

- ▶ *importskript.sql*
- ▶ *importskript_arbeitszeit.sql*
- ▶ *importskript_kreditinstitut.sql*

Wir benötigen zunächst nur die Datei *importskript.sql*, mit der wir die Übungsdatenbank anlegen.

Abbildung 1.13 zeigt das geöffnete Fenster der MySQL Workbench. Klicken Sie auf das VERZEICHNIS ÖFFNEN-Symbol unterhalb der Menüüberschriften.

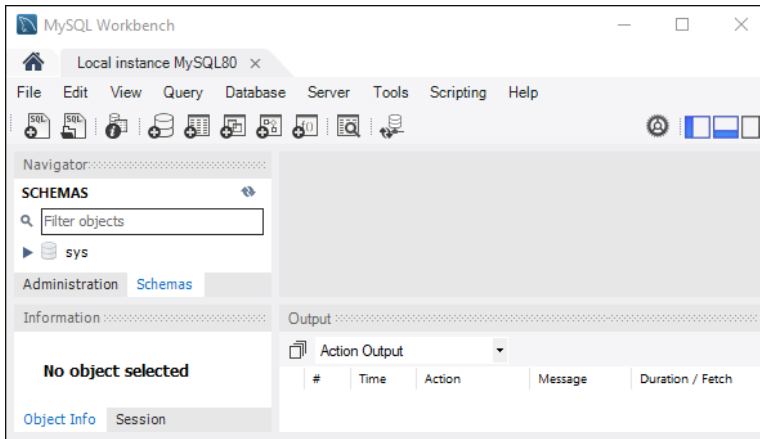


Abbildung 1.13 Die MySQL Workbench

Wählen Sie im Ordner *Downloads\mysql* die Datei *importskript.sql* aus (siehe [Abbildung 1.14](#)), und öffnen Sie sie mit einem Klick auf ÖFFNEN.

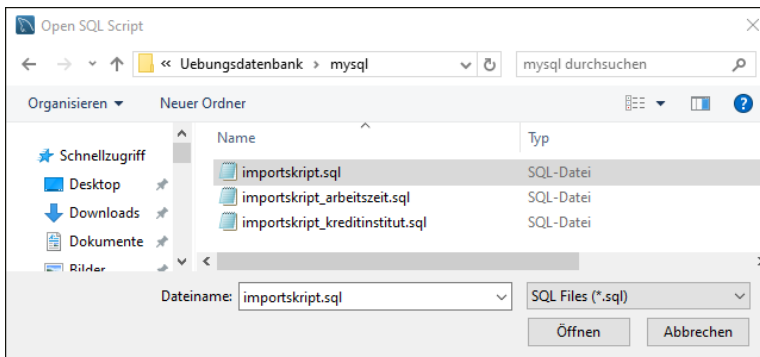


Abbildung 1.14 Das Importskript auswählen und öffnen

Stellen Sie nun sicher, dass sich der Cursor im geöffneten Importskript an der ersten Position der ersten Zeile befindet (siehe [Abbildung 1.15](#)). Klicken Sie danach auf den gelben Blitz, um alle Anweisungen, die im Importskript enthalten sind, auszuführen und die Übungsdatenbank anzulegen.

In [Abbildung 1.16](#) sehen Sie im Fenster ACTION OUTPUT, dass die Anweisungen, die für den Import erforderlich sind, ausgeführt wurden.

Nun müssen wir die Liste der Datenbanken aktualisieren – diese werden im MySQL-Sprachgebrauch auch *Schema* genannt. Klicken Sie hierfür auf den Reiter SCHEMAS, der sich unterhalb des NAVIGATOR-Fensters befindet. Nun sehen Sie im NAVIGATOR-

Fenster rechts neben der Überschrift SCHEMAS ein Aktualisierungssymbol. Wenn Sie darauf klicken, wird Ihnen die importierte Datenbank mit ihren Tabellen angezeigt.

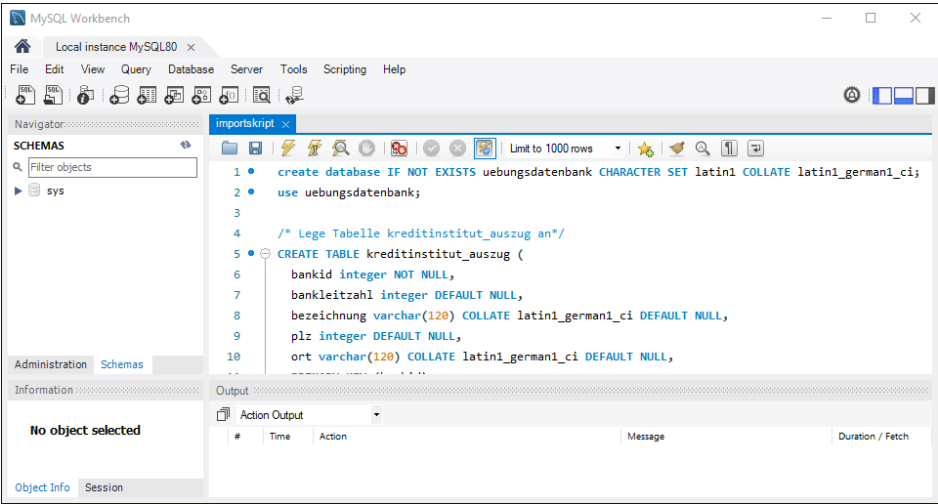


Abbildung 1.15 Das geöffnete Importskript in der MySQL Workbench

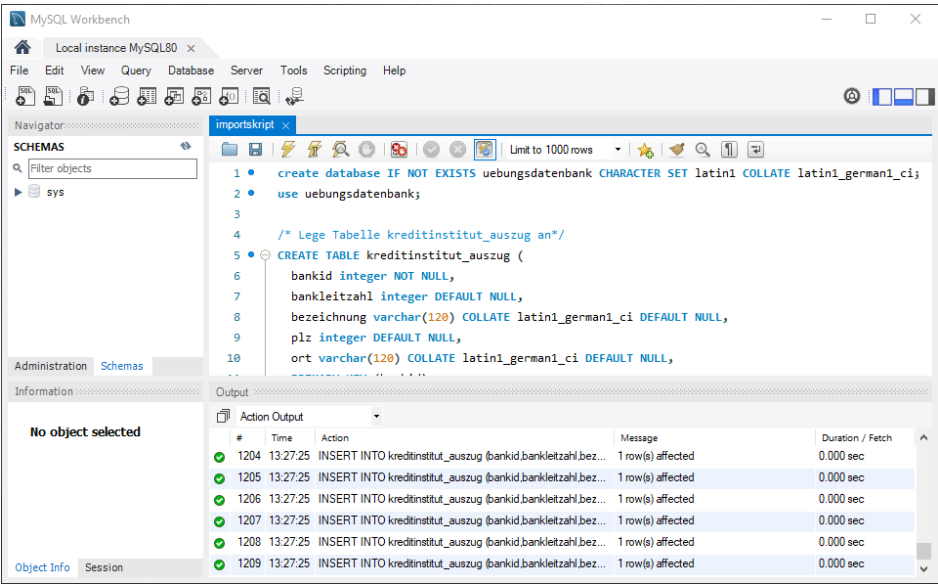


Abbildung 1.16 Das Importskript wurde ausgeführt.

Die aktive Datenbank wird im NAVIGATOR-Fenster wie in [Abbildung 1.17](#) immer in Fettschrift dargestellt. Stellen Sie zu Beginn Ihrer Arbeit stets sicher, dass Sie im NAVIGATOR-Fenster auf die *uebungsdatenbank* geklickt haben und diese fettgedruckt angezeigt wird.

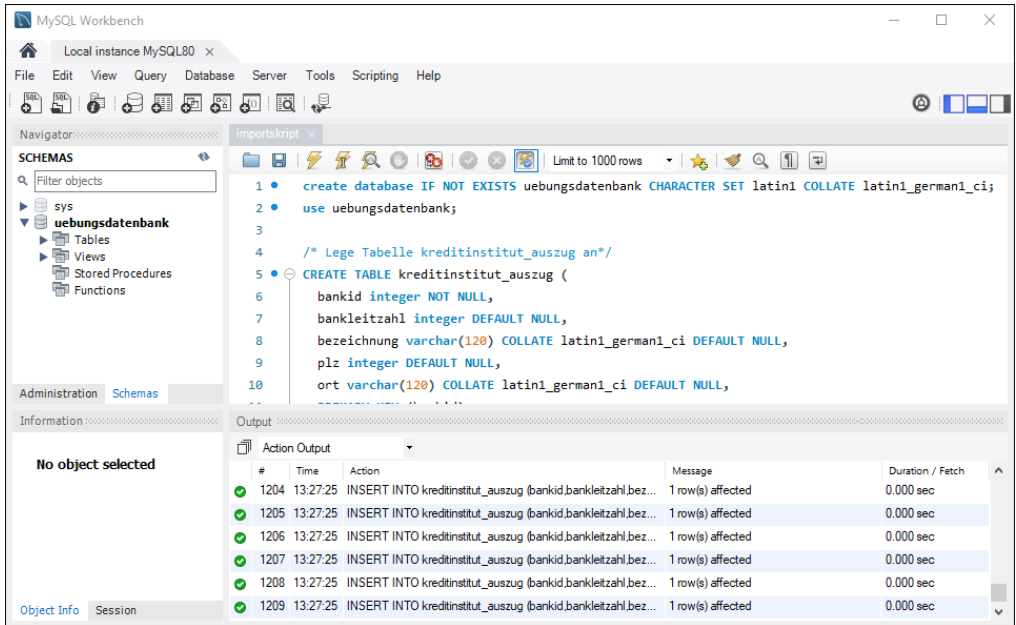


Abbildung 1.17 Das Navigator-Fenster wurde aktualisiert.

Wenn Sie zu einem späteren Zeitpunkt mit mehreren Datenbanken auf Ihrem MySQL-Datenbanksystem arbeiten, können Sie mit der MySQL Workbench auch auf sie entsprechend zugreifen.

Schließen Sie nun das geöffnete Importskript, indem Sie auf das Kreuz neben dem Tab IMPORTSKRIPT klicken. Alternativ können Sie auch die Tastenkombination **Strg** + **W** nutzen, um das Editorfenster zu schließen.

1.8 Eine erste Abfrage an die Datenbank senden

Wir werden nun eine erste einfache Abfrage der Tabelle *mitarbeiter* ausführen. Da wir in [Abschnitt 1.7](#) das Editorfenster geschlossen haben, müssen Sie ein neues Editorfenster in der MySQL Workbench aufrufen. In [Abbildung 1.18](#) sehen Sie die Menüleiste der MySQL Workbench. Unter den Menüpunkten sehen Sie ein paar Icons:

- ❶ Das erste Icon von links enthält ein Pluszeichen. Sie verwenden es, wenn Sie beabsichtigen, eine neue Datei zu erstellen, in der Sie Ihre SQL-Anweisungen notieren und ausführen.
- ❷ Das zweite Icon zeigt eine geöffnete Mappe. Hiermit öffnen Sie eine vorhandene SQL-Datei. Sie haben es bereits kennengelernt, als wir das Importskript geöffnet haben.

- 3 Das dritte Icon enthält ein »i« wie »Information«. Wenn Sie z. B. im Navigationsbaum eine Datenbank markieren und auf dieses Icon klicken, so erhalten Sie ausführliche Informationen wie die Dateigröße oder in Ihrer Datenbank enthaltene Tabellen.
- 4 Das vierte Icon symbolisiert eine Datenbank. Mit ihm können Sie eine neue Datenbank (ein neues Schema) anlegen.
- 5 Das fünfte Symbol zeigt eine Tabelle mit Pluszeichen. Mit ihm legen Sie eine neue Tabelle für eine ausgewählte Datenbank an.

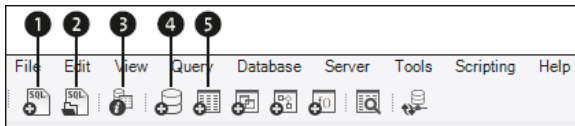


Abbildung 1.18 Die Menüleiste der MySQL Workbench

Nun wollen wir eine SQL-Abfrage an die Datenbank richten. Hierzu benötigen wir ein neues Editorfenster, das Sie mit einem Klick auf das erste Icon erzeugen. Alternativ können Sie auch das Tastaturkürzel `[Strg] + [T]` verwenden.

Am oberen Rand des Editorfensters sehen Sie nun wie in [Abbildung 1.19](#) eine weitere Leiste mit Icons.

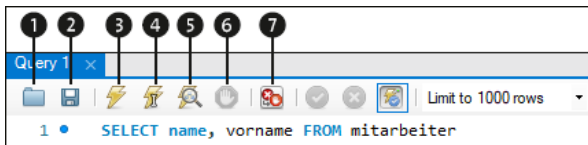


Abbildung 1.19 Eine SELECT-Abfrage ausführen

- 1 Mit dem Ordnersymbol-Icon öffnen Sie eine gespeicherte *.sql*-Datei, in der Sie SQL-Anweisungen notiert haben.
- 2 Mit dem Disketten-Icon speichern Sie Ihre aktuell bearbeiteten SQL-Anweisungen in einer Datei mit der Endung *.sql*.
- 3 Das Blitz-Icon dient zur Ausführung von SQL-Anweisungen. Diese Version des Blitzes führt alle SQL-Anweisungen aus, die im SQL-Editor notiert wurden, es sei denn, Sie haben explizit eine Anweisung mit der Maus markiert.
- 4 Das zweite Blitz-Icon (mit einem Cursor-Symbol) führt die SQL-Anweisung aus, die sich unterhalb des Cursors befindet.
- 5 Das Blitz-Icon mit einer Lupe führt die SQL-Anweisung aus und liefert Ihnen eine Beschreibung dessen, was die Datenbank mit dieser Anweisung an Arbeitspaketen zu bewältigen hat. Dieser Punkt ist für Sie im Augenblick nicht von Bedeutung.

- ⑥ Das Symbol mit der Hand bricht eine länger andauernde SQL-Anweisung ab. Diese Funktion werden Sie erst einmal nicht benötigen. Sie werden sie aber zu schätzen lernen, wenn Sie mit großen Datenvolumen zu tun haben.
- ⑦ Über das Symbol eines Abbruchzeichens mit einer stoppenden Hand bestimmen Sie, ob SQL-Anweisungen weiter ausgeführt werden sollen, wenn eine SQL-Anweisung einen Fehler während der Ausführung zurückliefert.

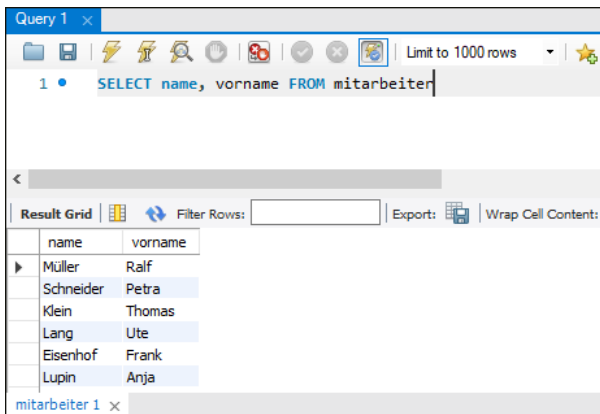
Steigen wir gleich ein, und probieren wir eine Anweisung in einem Beispiel aus. Geben Sie dazu die SQL-Anweisung aus [Listing 1.3](#) in Ihren Editor ein. Sie müssen an dieser Stelle noch nicht verstehen, was diese **SELECT**-Anweisung bewirkt, denn es kommt erst einmal nur darauf an, wie Sie SQL-Anweisungen eingeben und ausführen, um ein Ergebnis von der Datenbank zu erhalten.

```
SELECT name,vorname FROM mitarbeiter;
```

Listing 1.3 Eine SQL-Anweisung an die Datenbank richten

Jetzt müssen wir die im SQL-Editor notierte SQL-Anweisung nur noch an die Datenbank senden, um ein Ergebnis zu erhalten. Hierzu nutzen Sie die Icon-Leiste über dem Eingabeditor, die Sie ja bereits kennengelernt haben. Nutzen Sie das Blitz-Icon ③ aus [Abbildung 1.19](#), um die SQL-Anweisung, die im SQL-Editor notiert ist, an die Datenbank zu richten. Anstelle des Symbols können Sie alternativ auch eine Tastenkombination verwenden, um Ihre SQL-Anweisungen an das MySQL-Datenbanksystem zu senden: Halten Sie die Taste `[Strg]` gedrückt, und betätigen Sie die `[↵]`-Taste, um die SQL-Anweisung im Editor auszuführen.

Führen Sie also Ihre erste SQL-Anweisung aus, indem Sie sie an den MySQL-Datenbankserver senden. Sie erhalten dann von der Datenbank eine Ergebnistabelle zurück (siehe [Abbildung 1.20](#)).



The screenshot shows a SQL query editor window titled 'Query 1'. The query entered is `SELECT name, vorname FROM mitarbeiter;`. Below the query editor, there is a toolbar with various icons, including a lightning bolt icon (used for executing the query). Below the toolbar, there is a 'Result Grid' section. The 'Result Grid' shows a table with two columns: 'name' and 'vorname'. The table contains the following data:

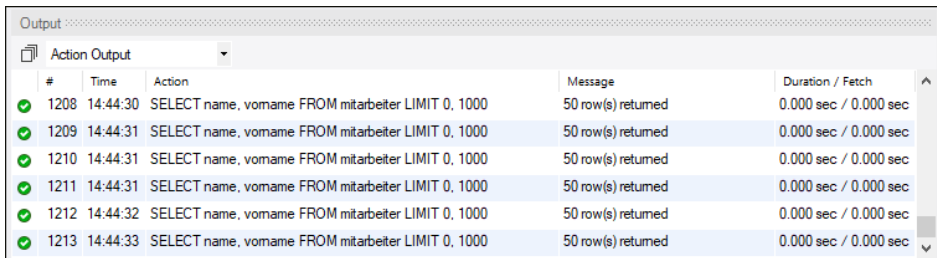
name	vorname
Müller	Ralf
Schneider	Petra
Klein	Thomas
Lang	Ute
Eisenhof	Frank
Lupin	Anja

Below the table, there is a tab labeled 'mitarbeiter 1'.

Abbildung 1.20 Ergebnistabelle nach der Ausführung einer SQL-Anweisung

Unterhalb der Ergebnistabelle sehen Sie ein Ausgabefenster wie in [Abbildung 1.21](#). In diesem Fenster wird Ihnen das Feedback der Datenbank zu den jeweiligen SQL-Anweisungen mitgeteilt. Der Haken in der ersten Spalte zeigt Ihnen, dass die jeweilige SQL-Anweisung erfolgreich ausgeführt wurde. Insgesamt hält das Ausgabefenster folgende Informationen für Sie bereit:

- ▶ den Zeitpunkt der Ausführung (TIME)
- ▶ die SQL-Anweisung, die ausgeführt wurde (ACTION)
- ▶ eine Nachricht, die z. B. ausgibt, wie viele Zeilen zurückgeliefert wurden (MESSAGE)
- ▶ die Dauer (DURATION / FETCH), die zur Ausführung der SQL-Anweisung benötigt wurde



#	Time	Action	Message	Duration / Fetch
✓ 1208	14:44:30	SELECT name, vomame FROM mitarbeiter LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
✓ 1209	14:44:31	SELECT name, vomame FROM mitarbeiter LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
✓ 1210	14:44:31	SELECT name, vomame FROM mitarbeiter LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
✓ 1211	14:44:31	SELECT name, vomame FROM mitarbeiter LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
✓ 1212	14:44:32	SELECT name, vomame FROM mitarbeiter LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec
✓ 1213	14:44:33	SELECT name, vomame FROM mitarbeiter LIMIT 0, 1000	50 row(s) returned	0.000 sec / 0.000 sec

Abbildung 1.21 Statusmeldungen zu den SQL-Anweisungen



Gebräuchliche Tastaturkürzel der MySQL Workbench

Ich verwende gern Tastaturkürzel, da sie ein effektiveres Arbeiten ermöglichen. Aus diesem Grund stelle ich Ihnen die hier verwendeten Tastaturkürzel kurz zusammengefasst vor:

[Strg] + [O] – ein bestehendes Skript öffnen

[Strg] + [T] – einen neuen Tabulator (Editorfenster) öffnen

[Strg] + [S] – ein Skript speichern

[Strg] + [W] – ein Editorfenster schließen

[Strg] + [↵] – eine SQL-Anweisung ausführen

1.9 Kommentarfunktion

Sie sollten Ihre SQL-Anweisungen, die Sie dauerhaft speichern möchten, ausreichend mit erklärenden Notizen versehen. Besonders bei komplexeren SQL-Anweisungen ist das sehr hilfreich: Nur so können Sie auch nach längerer Zeit noch nachvollziehen, wie Sie die Anweisung programmiert und auf welchen Wegen Sie das Ergebnis herbeigeführt haben.

Ihre Kommentare können dabei über verschiedenste Dinge informieren. Als Beispiel liste ich Ihnen einige Informationen auf, die Ihren SQL-Code bereichern und Ihnen in Zukunft die Arbeit damit vereinfachen werden:

- ▶ die anfordernde Stelle
- ▶ die Funktion der Abfrage
- ▶ das Datum der Fertigstellung
- ▶ die Durchführung von Änderungen und der Grund der Änderung
- ▶ der Name des Programmierers

Grundsätzlich verfügt SQL über zwei unterschiedliche Arten von Kommentaren. So gibt es einzeilige Kommentare, die mit einem doppelten Bindestrich (--) eingeleitet werden, auf den der Text des Kommentars folgt. Daneben gibt es Kommentare, die sich über mehrere Zeilen erstrecken können. Diese Kommentare beginnen mit den Zeichen /* und enden mit den Zeichen */. Alles, was dazwischen steht, wird vom Datenbanksystem ignoriert.

Kommentare in der Praxis nutzen

Sehen wir uns an, wie Sie einen einzeiligen Kommentar in der Praxis verwenden. Hierzu setzen wir einfach vor der **SELECT**-Abfrage einen einzeiligen Kommentar wie in [Listing 1.4](#) ein. Wenn Sie die Anweisung in Ihrem SQL-Client ausführen, ignoriert das Datenbanksystem den Kommentar und liefert Ihnen stattdessen nur das Ergebnis der **SELECT**-Abfrage als Ergebnistabelle zurück.

```
-- Spaltenauswahl: name, vorname der Tabelle mitarbeiter
SELECT name,vorname FROM mitarbeiter;
```

Listing 1.4 Einen einzeiligen Kommentar im SQL-Code anwenden

Einzeilige Kommentare sind zu Dokumentationszwecken nur eingeschränkt verwendbar, da Ihnen nur eine Zeile zur Verfügung steht. Aber dafür gibt es ja die mehrzeiligen Kommentare. Die nächste **SELECT**-Anweisung dokumentieren wir in [Listing 1.5](#) mit einem mehrzeiligen Kommentar, um sie mit verschiedenen Informationen anzureichern. Erläutert wird so, wer der Autor war, wann der Befehl erstellt wurde, und es folgen noch einige weitere nützliche Informationen.

```
/*
Autor: Michael Laube
Erstellungsdatum: 01.08.2024
Funktion: Liste mit den Namen und Vornamen der Mitarbeiter
Funktion geprüft: ja
Funktion erfüllt: ja
```

Nutzergruppen der Abfrage: Personalabteilung

*/

```
SELECT name,vorname FROM mitarbeiter;
```

Listing 1.5 Einen mehrzeiligen Kommentar im SQL-Code anwenden

Die Datenbank ignoriert bei der Ausführung der SQL-Anweisung auch den mehrzeiligen Kommentar. Wenn Sie den gesamten Inhalt einer *.sql*-Datei auskommentieren, passiert einfach gar nichts. Bei der Ausführung der leeren Anweisung wird Ihnen kein Ergebnis zurückgeliefert, da keine SQL-Anweisung vorhanden ist.



Zusammenfassung: SQL-Anweisungen kommentieren

Kommentare werden zu Dokumentationszwecken verwendet. In SQL haben Sie zwei Möglichkeiten, Kommentare zu Ihren SQL-Anweisungen zu verfassen:

- Die erste Möglichkeit besteht darin, einzeilige Kommentare zu verfassen, die mit einem doppelten Bindestrich (--) eingeleitet werden.
- Die zweite Möglichkeit ist, mehrzeilige Kommentare zu verfassen. Diese beginnen stets mit einem Slash, gefolgt von einem Sternchen (/*), und enden mit einem Sternchen, gefolgt von einem Slash (*/).

1.9.1 Übungen

Übung 1

Geben Sie die SQL-Anweisung wie in [Listing 1.6](#) gezeigt in Ihre MySQL Workbench ein. Vor der **SELECT**-Anweisung soll folgender einzeiliger Kommentar notiert werden:

Alle Spalten der Tabelle mitarbeiter abfragen

```
SELECT * FROM mitarbeiter;
```

Listing 1.6 Alle Spalten einer Tabelle abfragen

Führen Sie die Abfrage in Ihrem SQL-Client aus, um die Auswirkungen des einzeiligen Kommentars auf das Ergebnis der Abfrage zu ermitteln.

Übung 2

In dieser Übung sollen sämtliche Mitarbeiter ermittelt werden, die zur Abteilung *einkauf* gehören. Bitte schreiben Sie die SQL-Anweisung wie in [Listing 1.7](#) angegeben in Ihr Editorfenster:

```
SELECT
    name,vorname,abteilung
FROM
```



```

    mitarbeiter
WHERE
    abteilung='einkauf';

```

Listing 1.7 Eine weitere SQL-Abfrage, die mit einem mehrzeiligen Kommentar versehen werden muss

Vor der **SELECT**-Anweisung notieren Sie einen Kommentar, der so strukturiert ist:

Programmierer: Ihr Name

Datum: Das aktuelle Datum

Funktion: Filtern aller Mitarbeiter, die zur Abteilung Einkauf gehören

Beachten Sie auch die Zeilenumbrüche.

Führen Sie die Abfrage aus, und ermitteln Sie, welche Auswirkungen der mehrzeilige Kommentar auf das Ergebnis der Abfrage hat.

Lösung zu Übung 1

```

-- Alle Spalten der Tabelle mitarbeiter abfragen
SELECT * FROM mitarbeiter;

```

Listing 1.8 Verwendung eines einzeiligen Kommentars in SQL

Der einzeilige Kommentar hat keine Auswirkungen auf das Ergebnis der Abfrage. Die Datenbank liefert Ihnen wie gewohnt eine Ergebnistabelle gemäß Ihrer **SELECT**-Abfrage zurück.

Lösung zu Übung 2

```

/*
Programmierer: Ihr Name
Datum: Das aktuelle Datum
Funktion: Filtern aller Mitarbeiter, die zur Abteilung Einkauf gehören
*/
SELECT
    name,vorname,abteilung
FROM
    mitarbeiter
WHERE
    abteilung='einkauf';

```

Listing 1.9 Verwendung eines mehrzeiligen Kommentars in SQL

Der mehrzeilige Kommentar hat bei der Ausführung der **SELECT**-Anweisung keinerlei Auswirkungen auf das Ergebnis der Abfrage.

Kapitel 2

Los geht's: Die Grundfunktionen der Tabellenabfrage (SELECT)

Nachdem wir die Grundvoraussetzungen geschaffen haben, steige ich mit Ihnen in das Basiswissen ein. Sie werden sehen, es macht viel Spaß, mit SQL zu arbeiten.

Dieses Kapitel vermittelt Ihnen die Grundkenntnisse in SQL. Sie lernen, erste Abfragen zu formulieren und an die Datenbank zu senden, um Ergebnislisten zu erhalten.

2.1 Mit einer SELECT-Anweisung Tabellen abfragen

Wir beginnen mit einfachen SQL-Abfragen. Für den Einstieg stellen wir die Tabelle *mitarbeiter* in den Fokus. Sie ist sehr einfach, aber wir können mit ihr schon viel unternehmen.

2.1.1 Die Tabelle »mitarbeiter«

Die Tabelle *mitarbeiter* speichert Datensätze von Mitarbeitern eines Unternehmens. So stehen uns die Daten immer zentral in der Datenbank zur Verfügung, und wir können sie abrufen, wenn wir sie benötigen. Die Tabelle enthält Spalten, die den Mitarbeiter identifizieren (*name*, *vorname*, *gebdatum*), seinen Wohnort speichern (*strasse*, *hausnummer*, *ort*), die Bankdaten enthalten und noch einiges mehr.

Im Folgenden beschäftigen wir uns mit der ersten SQL-Anweisung, nämlich mit einer einfachen Abfrage.

2.1.2 Wie frage ich eine Tabelle ab? (SELECT ... FROM)

Wir beginnen mit der einfachsten aller Abfragen: Sie fragen die Spalten *name* und *vorname* der Tabelle *mitarbeiter* ab. Eine Auswahl von Spalten einer Tabelle wird dabei als *Projektion* bezeichnet. Sie können es sich tatsächlich so vorstellen, dass die ausgewählten Spalten der Tabelle von der Datenbank in Tabellenform auf Ihren Rechner projiziert werden.

2.1.3 Spalten einer Tabelle abfragen

Unser Ziel ist es, die Spalten *name* und *vorname* der Tabelle *mitarbeiter* auszuwählen und abzufragen, um eine Ergebnistabelle mit den Werten zu erhalten. Wir beginnen mit dem Schlüsselwort **SELECT**, um eine Abfrage einzuleiten. In [Listing 2.1](#) sehen Sie eine vollständige **SELECT**-Anweisung, wie Sie sie bereits verwendet haben:

```
SELECT name,vorname FROM mitarbeiter;
```

Listing 2.1 Unsere erste **SELECT**-Anweisung mit den ausgewählten Spalten »name« und »vorname« der Tabelle »mitarbeiter«

Hinter dem Wort **SELECT** folgen die Spaltenbezeichnungen, die jeweils durch ein Komma separiert werden. Das Schlüsselwort **FROM** teilt der Datenbank mit, aus welcher Tabelle die Spalten abgefragt werden. Schließlich wird die Tabelle hinter der **FROM**-Klausel namentlich genannt. In unserem Beispiel handelt es sich um die Tabelle *mitarbeiter*.

Als Ergebnis erhalten Sie [Tabelle 2.1](#), in deren Zeilen die Namen und Vornamen der Mitarbeiter zu finden sind.

name	vorname
Müller	Ralf
Schneider	Petra
Klein	Thomas
Lang	Ute

Tabelle 2.1 Ergebnistabelle einer einfachen **SELECT**-Abfrage



Ausgabe der Abfragen

Der Übersichtlichkeit halber zeige ich Ihnen immer die ersten vier Zeilen einer Ergebnistabelle – es sei denn, es ist für das Verständnis erforderlich, mehr Zeilen anzuzeigen. Die Beispiele, die ich Ihnen erläutere, können Sie in der *uebungsdatenbank* jederzeit nachvollziehen.

Wenn Sie nur eine Spalte abfragen wollen, benötigen Sie natürlich kein Komma. Die nächste **SELECT**-Anweisung enthält ausschließlich eine Spalte, die in der Spaltenauswahlliste einer **SELECT**-Abfrage ausgewählt wird. Als Ergebnis erhalten Sie [Tabelle 2.2](#) zurück.

```
SELECT name FROM mitarbeiter;
```

Listing 2.2 Eine Spalte einer Tabelle abfragen

name
Müller
Schneider
Klein
Lang

Tabelle 2.2 Ergebnistabelle für eine Spalte in der Spaltenauswahlliste

Sie können die Spalten natürlich in beliebiger Reihenfolge abfragen. So, wie Sie die Spalten in der Spaltenauswahlliste der **SELECT**-Anweisung angeben, werden sie auch in der Ergebnisliste zurückgeliefert. Die Spalten einer Tabelle können Sie auch doppelt abfragen, wenn es erforderlich ist. Vergessen Sie nicht das Komma, wenn es sich um mindestens zwei Spalten handelt.

SQL ist case-insensitive

Die Abfragesprache SQL ist *case-insensitive*. Das heißt, es spielt überhaupt keine Rolle, ob Sie eine SQL-Anweisung in Großbuchstaben, in Kleinbuchstaben oder auch gemischt notieren. Mit allen folgenden Beispielen erzielen Sie dasselbe Ergebnis:

```
SELECT name,vorname FROM mitarbeiter;
select name,vorname from mitarbeiter;
sEleCt nAme,vorName fRom mitaRbeiter;
```



2.1.4 Alle Spalten einer Tabelle abfragen

Wie sieht es aus, wenn Sie alle Spalten einer Tabelle mit einer **SELECT**-Anweisung abfragen wollen? Im umständlichsten Fall schreiben Sie alle Spalten der Tabelle in die Spaltenauswahlliste der **SELECT**-Anweisung. Je nach Anzahl der Spalten kann das sehr mühsam sein. SQL stellt dafür das Sternchen (*) (oder *Asterisk*, was schlicht und ergreifend »Sternchen« heißt) als Platzhalter für alle Spalten einer Tabelle bereit. Wählen Sie nun alle Spalten der Tabelle *mitarbeiter* mit dem Asterisk wie folgt aus:

```
SELECT * FROM mitarbeiter;
```

Listing 2.3 Alle Spalten der Tabelle »mitarbeiter« abfragen

Sie erhalten alle Spalten der Tabelle *mitarbeiter* zurück (siehe Tabelle 2.3).

mitarbeiterid	name	vorname	gebdatum
1	Müller	Ralf	1970-12-20
2	Schneider	Petra	1965-03-07
3	Klein	Thomas	1970-12-20
4	Lang	Ute	1985-07-19

Tabelle 2.3 Alle Spalten der Tabelle »mitarbeiter« abfragen



Zusammenfassung: Tabellen abfragen (SELECT)

Der Aufbau einer SELECT-Anweisung entspricht folgendem Muster:

```
SELECT spalte1,spalte2 FROM tabelle1;
```

In einer SELECT-Abfrage wählen Sie Spalten aus, indem Sie in einer Spaltenauswahl-liste explizit die Spaltennamen (durch je ein Komma getrennt) notieren. Wollen Sie alle Spalten einer Tabelle mit SELECT abfragen, verwenden Sie das Stellvertreterzeichen *, das für alle Spalten steht.

SQL-Anweisungen sind case-insensitive, können also in Großbuchstaben, Kleinbuchstaben oder gemischt geschrieben werden.

2.1.5 Übungen

Übung 1

In der ersten Übung interessieren wir uns dafür, wann ein Mitarbeiter eingestellt wurde und wann ein Mitarbeiter das Unternehmen verlassen hat. Formulieren Sie hierzu eine SELECT-Abfrage, in der Sie die Spalten *name*, *vorname*, *eintrittsdatum* und *austrittsdatum* der Tabelle *mitarbeiter* abfragen.

Übung 2

Wir erweitern die Abfrage aus Übung 1: Uns interessiert jetzt auch der Austrittsgrund der Mitarbeiter. Ergänzen Sie die Abfrage der ersten Übung um die Spalte *austrittsgrund*.

Übung 3

In der nächsten Übung sollen sämtliche Spalten der Tabelle *mitarbeiter* abgefragt werden. Ich gebe Ihnen einen kleinen Tipp: Um das zu realisieren, nutzen Sie den oben vorgestellten Platzhalter.

Lösung zu Übung 1

SELECT

```
name,  
vorname,  
eintrittsdatum,  
austrittsdatum
```

FROM

```
mitarbeiter;
```

Listing 2.4 Spaltenauswahlabfrage zur Ermittlung des Eintritts- und Austrittsdatums

Lösung zu Übung 2

SELECT

```
name,  
vorname,  
eintrittsdatum,  
austrittsdatum,  
austrittsgrund
```

FROM

```
mitarbeiter;
```

Listing 2.5 Spaltenauswahlabfrage, die um die Spalte »austrittsgrund« erweitert wurde

Lösung zu Übung 3

SELECT * FROM mitarbeiter;

Listing 2.6 Lösung zur Auswahl sämtlicher Spalten einer Tabelle

2.2 Zeilen in einer Abfrage mit WHERE filtern

Bisher haben Sie gelernt, wie Sie Spalten aus einer Tabelle selektieren. Wie sieht es aber aus, wenn Sie an Zeilen interessiert sind, deren Spaltenwerte ein bestimmtes Kriterium erfüllen?

Sie wollen eine bestimmte Mitarbeiterin mit sämtlichen Kontaktdaten aus der Tabelle *mitarbeiter* ermitteln. Der Name der Mitarbeiterin lautet Ute Lang. Sie könnten sich zuerst sämtliche Mitarbeiterdatensätze der Tabelle anzeigen lassen, um sie anschließend Zeile für Zeile durchzugehen, bis Sie auf die Daten zur Mitarbeiterin Ute Lang stoßen. Dies ist natürlich sehr zeitaufwendig und mühsam, besonders wenn Sie es mit der Mitarbeiterliste eines Großkonzerns mit 9.000 Mitarbeitern zu tun haben. Gut, dass es eine Möglichkeit gibt, die Datenbank gezielt nach der Mitarbeiterin Ute Lang suchen zu lassen und uns das Ergebnis in einer Zeile zu präsentieren.

Im Grunde müsste die Datenbank die Tabelle *mitarbeiter* ja nur Zeile für Zeile durchgehen und prüfen, ob der Spaltenwert der Spalte *name* gleich Lang ist. Dies ist vergleichbar mit Rechenoperationen. Jede Zeile wird nach dem Ausdruck ausgewertet, wobei das Ergebnis wahr oder falsch sein kann. Geprüft wird in diesem Beispiel die folgende Bedingung:

```
name= 'Lang'
```

Für diesen Fall hält der Sprachschatz von SQL natürlich eine Option bereit, nämlich die **WHERE**-Klausel. Wir könnten also auch sagen:

»Gib mir den Datensatz der Tabelle *mitarbeiter* zurück, wo (= *where*) der Name gleich Lang ist.«

Um eine Bedingung zu formulieren, die auf Gleichheit prüft, verwenden wir wie in der Mathematik den Vergleichsoperator (=). Das Auffinden von Zeilen anhand von Spaltenwerten, die ein bestimmtes Filterkriterium erfüllen, gehört zu den wichtigsten Aufgaben, die Sie in SQL durchführen werden. Erst wenn Sie eine Ergebnistabelle auf die Zeilen einschränken können, die Sie wirklich benötigen, haben Sie die Basis, um weitere Operationen vornehmen zu können. So lassen sich etwa das Sortieren anhand einer bestimmten Spalte, Berechnungen und viele weitere Operationen erst mit der **WHERE**-Klausel sinnvoll durchführen.

Die Auswertung einer Bedingung, die zum Ziel hat, Zeilen in der Ergebnistabelle einzugrenzen, wird als *Selektion* bezeichnet.

Sehen wir uns in [Abschnitt 2.2.1](#) an, welche Vergleichsoperatoren Ihnen zum Filtern zur Verfügung stehen.

2.2.1 SQL-Vergleichsoperatoren

Sie können zahlreiche unterschiedliche Vergleiche vornehmen. Hierzu zählen:

- ▶ numerische Werte vergleichen
- ▶ Texte vergleichen
- ▶ Textmuster vergleichen
- ▶ Datumsangaben vergleichen
- ▶ auf Intervalle prüfen
- ▶ auf Existenz in Teilmengen prüfen
- ▶ auf »Wert ist definiert« prüfen

In der Abfragesprache SQL ist es glücklicherweise möglich, diese Vergleiche mit einer übersichtlichen Anzahl von Vergleichsoperatoren durchzuführen. In [Tabelle 2.4](#) bietet ich Ihnen einen Überblick und eine Beschreibung zu den Vergleichsoperatoren.

Vergleichsoperator	Funktion
Wert1=Wert2	Prüft auf Gleichheit zweier Werte. Die Aussage ist wahr, wenn der linke Wert1 gleich dem rechten Wert2 ist.
Wert1<>Wert2 oder Wert1!=Wert2	Prüft auf Ungleichheit zweier Werte. Die Aussage ist wahr, wenn der linke Wert1 ungleich dem rechten Wert2 ist. Die Aussage ist also immer wahr, wenn Ungleichheit zwischen den Werten besteht. Alternativ zum <>-Operator kann der !=-Vergleichsoperator zur Prüfung der Ungleichheit zweier Werte verwendet werden.
Wert1<Wert2	Prüft, ob der linke Wert1 kleiner ist als der rechte Wert2. Die Aussage ist also wahr, wenn der linke Wert1 kleiner als der rechte Wert2 ist.
Wert1<=Wert2	Prüft, ob der linke Wert1 kleiner als oder gleich dem rechten Wert2 ist. Die Aussage ist also wahr, wenn der linke Wert1 kleiner als oder gleich dem rechten Wert2 ist.
Wert1>Wert2	Prüft, ob der linke Wert1 größer als der rechte Wert2 ist. Die Aussage ist also wahr, wenn der linke Wert1 größer als der rechte Wert2 ist.
Wert1>=Wert2	Prüft, ob der linke Wert1 größer als oder gleich dem rechten Wert2 ist. Die Aussage ist also wahr, wenn der linke Wert1 größer als oder gleich dem rechten Wert2 ist.
Wert BETWEEN Wert1 AND Wert2	Prüft, ob sich ein Wert innerhalb eines gegebenen Wertebereichs befindet, der durch Wert1 und Wert2 definiert ist. Die Aussage ist wahr, wenn der Wert innerhalb der begrenzenden Werte liegt, die mit AND angegeben werden.
Wert NOT BETWEEN Wert1 AND Wert2	Prüft, ob sich ein Wert nicht innerhalb des Wertebereichs befindet, der durch Wert1 und Wert2 definiert ist. Die Aussage ist wahr, wenn der Wert außerhalb der begrenzenden Werte liegt, die mittels AND angegeben werden.
Wert LIKE Muster	Prüft, ob ein Wert mit einem Textmuster übereinstimmt. Die Aussage ist wahr, wenn der Wert links vom LIKE -Operator dem Muster rechts vom LIKE -Operator entspricht.

Tabelle 2.4 Vergleichsoperatoren in SQL

Vergleichsoperator	Funktion
Wert NOT LIKE Muster	Prüft, ob ein Wert nicht mit einem Muster übereinstimmt. Die Aussage ist wahr, wenn der Wert links vom NOT LIKE -Operator ungleich dem Muster rechts vom NOT LIKE -Operator ist, also keine Übereinstimmung mit dem Muster erkannt werden kann.
Wert IN (Wert1,Wert2,...)	Prüft, ob ein Wert in einer geklammerten Liste vorkommt. Die Aussage ist wahr, wenn der Wert links vom IN -Operator in der Liste rechts vom IN -Operator enthalten ist.
Wert NOT IN (Wert1,Wert2,...)	Prüft, ob ein Wert nicht in einer geklammerten Liste vorkommt. Die Aussage ist wahr, wenn der Wert links vom NOT IN -Operator nicht in der Liste rechts vom NOT IN -Operator enthalten ist.
Wert IS NULL	Prüft, ob ein Wert nicht definiert ist. Die Aussage ist wahr, wenn der Wert nicht definiert, also NULL ist.
Wert IS NOT NULL	Prüft, ob der Wert definiert ist, also nicht leer ist. Die Aussage ist wahr, wenn der Wert definiert, also nicht leer ist.

Tabelle 2.4 Vergleichsoperatoren in SQL (Forts.)

Das sehen wir uns einmal etwas genauer an. Hierzu werden wir den Wahrheitsgehalt mehrerer Aussagen prüfen. Die Aussagen werden mit relationalen Vergleichsoperatoren und ganzen Zahlen wie 1, 2, 3 etc. formuliert. Betrachten Sie die in [Tabelle 2.5](#) angegebenen Aussagen, und vollziehen Sie das Ergebnis nach.

Aussage	Operator in SQL	Ergebnis der Aussage
1 ist gleich 1.	1=1	wahr
1 ist gleich 2.	1=2	falsch
2 ist ungleich 1.	2<>1	wahr
1 ist ungleich 1.	1<>1	falsch
2 ist ungleich 1.	2!=1	wahr
1 ist ungleich 1.	1!=1	falsch
1 ist kleiner als 2.	1<2	wahr

Tabelle 2.5 Aussage auf »wahr« oder »falsch« prüfen

Aussage	Operator in SQL	Ergebnis der Aussage
3 ist kleiner als 3.	3<3	falsch
2 ist kleiner als 2.	2<2	falsch
2 ist kleiner als oder gleich 3.	2<=3	wahr
3 ist kleiner als oder gleich 3.	3<=3	wahr
4 ist kleiner als oder gleich 3.	4<=3	falsch
2 ist größer als 1.	2>1	wahr
0 ist größer als 1.	0>1	falsch
1 ist größer als 1.	1>1	falsch
3 ist größer als oder gleich 2.	3>=2	wahr
2 ist größer als oder gleich 2.	2>=2	wahr
1 ist größer als oder gleich 2.	1>=2	falsch
2 liegt zwischen 1 und 3.	2 BETWEEN 1 AND 3	wahr
5 liegt zwischen 1 und 3.	5 BETWEEN 1 AND 3	falsch
0 ist zwischen 1 und 3.	0 BETWEEN 1 and 3	falsch

Tabelle 2.5 Aussage auf »wahr« oder »falsch« prüfen (Forts.)

Eine **WHERE**-Klausel macht nichts anderes. Sie prüft, ob die Ergebnisse von Ausdrücken wahr oder falsch sind. In die Ergebnisliste schaffen es nur die Zeilen einer Tabelle, die einer Bedingung entsprechen, deren Aussage also wahr ist.

2.2.2 Spaltenwerte auf Gleichheit prüfen

Wir wollen nun die telefonischen Kontaktdaten einer Mitarbeiterin ermitteln. Hierzu formulieren wir eine **SELECT**-Anweisung mit einer **WHERE**-Klausel und einer Bedingung, in der die Spaltenwerte der Spalte *name* der Tabelle *mitarbeiter* auf Gleichheit mit dem Wert Lang verglichen werden.

Die Spalte *name* der Tabelle *mitarbeiter* speichert sogenannte *Zeichenketten*, die Sie in [Abschnitt 4.1.4](#) noch viel genauer kennenlernen werden. Betrachten Sie den Namen Lang. Er entspricht einer Kette von Zeichen, die vier Zeichen lang ist. Solche Zeichenketten werden in SQL immer mit einfachen Hochkommas umschlossen.



Zeichenketten

Zeichenketten (oder *Strings*) sind nichts anderes als eine Aneinanderreihung von beliebigen Zeichen. Das können Buchstaben, Zahlen oder Sonderzeichen sein. Zeichenketten stehen in Hochkommas. Zu beachten ist, dass enthaltene Nummern nicht als Zahlen betrachtet werden, sondern als eine Folge von einzelnen Zeichen. Mehr zu dieser Unterscheidung finden Sie in [Kapitel 4](#), »[Tabellen anlegen \(CREATE TABLE\)](#)«.

Beispiele für Zeichenketten:

- ▶ Meier
- ▶ 1. FC Köln
- ▶ 1323
- ▶ erde@mars.de

In unserem Filterkriterium in [Listing 2.7](#) werden die Spaltenwerte der Spalte *name* mit der Zeichenkette *Lang* verglichen:

SELECT

```
name,  
vorname,  
festnetz,  
mobilnetz
```

FROM

```
mitarbeiter
```

WHERE

```
name='Lang';
```

Listing 2.7 Auf Gleichheit prüfen und filtern

In [Tabelle 2.6](#) sehen Sie das Ergebnis der **SELECT**-Abfrage aus [Listing 2.7](#).

name	vorname	festnetz	mobilnetz
Lang	Ute	8977/99999999999999	2345/55555555555555

Tabelle 2.6 Mitarbeiter, deren Name »Lang« ist

Es wird nur der Datensatz zurückgegeben, der der Bedingung entspricht; in diesem Fall die Zeile der Mitarbeiterin Ute Lang.

Wie sieht es aus, wenn Sie eine Bedingung für eine Spalte numerischen Typs formulieren wollen? Wir könnten ja an den Mitarbeitern interessiert sein, die im Postleitzahlenbereich 50000 wohnhaft sind.

Formulieren wir also eine **SELECT**-Abfrage, die in der Spaltenauswahlliste die Spalten *name*, *vorname* und *plz* der Tabelle *mitarbeiter* enthält. Außerdem teilen wir der Datenbank in einer **WHERE**-Klausel mit, dass wir nur an den Mitarbeitern interessiert sind, die im Postleitzahlenbereich 50000 wohnhaft sind. Spaltenwerte der Spalte *plz* sind in der *uebungsdatenbank* numerischer Art, sie enthalten ausschließlich ganzzahlige Werte. Wenn Sie ganzzahlige Werte mit Spaltenwerten vergleichen, benötigen Sie keine umschließenden Hochkommas. Das gilt für jegliche Art von numerischen Werten, also auch für Dezimalzahlen.

SELECT

```
name,
vorname,
plz
```

FROM

```
mitarbeiter
```

WHERE

```
plz=50000;
```

Listing 2.8 Numerische Werte auf Gleichheit prüfen und filtern

name	vorname	plz
Müller	Ralf	50000
Klein	Thomas	50000
Lang	Ute	50000
Eisenhof	Frank	50000

Tabelle 2.7 Alle Mitarbeiter, die im PLZ-Bereich 50000 leben

Wie sieht es mit Datumswerten aus? Eventuell interessieren uns nur die Mitarbeiter, die am 1. Januar 1985 eingestellt wurden. Somit haben Sie die Gelegenheit, in einer **WHERE**-Klausel Spaltenwerte nach einem Datumswert zu filtern. In der Spaltenauswahlliste wählen Sie die Spalten *name*, *vorname* und *eintrittsdatum* aus und formulieren eine **WHERE**-Klausel, in der die Bedingung `eintrittsdatum='1985-01-01'` lautet. Beachten Sie hier das Datumsformat, das in der Form *Jahr-Monat-Tag* oder *yyyy-mm-dd* notiert ist. Dieses Datumsformat entspricht der Norm ISO 8601, die einen internationalen Standard für die Schreibweise von Datumswerten festlegt. Auch Datumswerte werden in SQL wie Zeichenketten immer von Hochkommas umschlossen. Das gilt auch für unser Filterkriterium, dessen Wert ja einem Datum entspricht.

SELECT

```
name,  
vorname,  
eintrittsdatum
```

FROM

```
mitarbeiter
```

WHERE

```
eintrittsdatum='1985-01-01';
```

Listing 2.9 Datumswerte auf Gleichheit prüfen und filtern

name	vorname	eintrittsdatum
Sebastiansen	Inge	1985-01-01
Paulus	Sebastian	1985-01-01

Tabelle 2.8 Mitarbeiter, die am 01.01.1985 eingestellt wurden

Bis jetzt haben Sie Spaltenwerte auf Gleichheit mit von Ihnen vorgegebenen Filterkriterien verglichen. Damit haben Sie schon einen mächtigen Vergleichsoperator kennengelernt. Sehen wir uns jetzt an, wie Sie auf Ungleichheit prüfen.

2.2.3 Spaltenwerte auf Ungleichheit prüfen

Auf Gleichheit zu prüfen, ist einfach. Spaltenwerte auf Ungleichheit zu prüfen, ist allerdings nicht schwieriger. Sie verwenden lediglich einen anderen Vergleichsoperator, der aus zwei spitzen Klammern besteht: <>. Alternativ können Sie auch als Vergleichsoperator ein Ausrufezeichen verwenden, dem ein Gleichheitszeichen (!=) folgt. Dieser Operator prüft ebenfalls auf Ungleichheit. Es werden also zwei Vergleichsoperatoren für einen Zweck angeboten.

Im nächsten Beispiel ermitteln wir alle Mitarbeiter, die nicht verheiratet sind. Hierzu prüfen wir die Spalte *verheiratet* der Tabelle *mitarbeiter*, in der die Werte *ja* und *nein* vorkommen können, auf Ungleichheit mit dem Wert *ja*. In der Spaltenauswahlliste wählen wir die Spalten *name*, *vorname*, *steuerklasse* und *verheiratet* aus:

SELECT

```
name,  
vorname,  
steuerklasse,  
verheiratet
```

FROM

```
mitarbeiter
```

WHERE

```
verheiratet<>'ja';
```

Listing 2.10 Auf Ungleichheit prüfen und filtern

name	vorname	steuerklasse	verheiratet
Lang	Ute	1	nein
Lupin	Anja	2	nein
Schmidt	Werner	1	nein
Funke	Doris	1	nein

Tabelle 2.9 Mitarbeiter, die nicht verheiratet sind

In der **SELECT**-Abfrage aus [Listing 2.10](#) haben Sie Spalten mit Textwerten auf Ungleichheit geprüft. Jetzt werden wir eine weitere Abfrage formulieren, in der Sie die numerischen Werte, die in der Spalte *steuerklasse* enthalten sind, auf Ungleichheit mit dem Wert 3 prüfen. Hierzu nutzen wir die Spalte *steuerklasse* der Tabelle *mitarbeiter* in der **WHERE**-Klausel und formulieren die Bedingung `steuerklasse<>3`:

SELECT

```
name,
vorname,
steuerklasse,
verheiratet
```

FROM

```
mitarbeiter
```

WHERE

```
steuerklasse<>3;
```

Listing 2.11 Numerische Werte auf Ungleichheit prüfen und filtern

name	vorname	steuerklasse	verheiratet
Lang	Ute	1	nein
Lupin	Anja	2	nein
Lempe	Dirk	1	ja
Schmidt	Werner	1	nein

Tabelle 2.10 Alle Mitarbeiter ungleich Steuerklasse 3

Natürlich können Sie auch Datumsangaben in einer Bedingung auf Ungleichheit prüfen. Dies werden wir mit einer **SELECT**-Anweisung probieren, die in einer **WHERE**-Klausel die Spalte *eintrittsdatum* auf Ungleichheit mit dem Datumswert 13. Januar 1990 prüft. Auch hier müssen Sie auf die Datumsschreibweise achten. Wir verwenden die Schreibweise gemäß der Norm ISO 8601. Achten Sie darauf, auch Datumsangaben in einfache Hochkommas zu setzen:

```
SELECT
    name,
    vorname,
    eintrittsdatum
FROM
    mitarbeiter
WHERE
    eintrittsdatum <> '1990-01-13';
```

Listing 2.12 Datumswerte auf Ungleichheit prüfen und filtern

name	vorname	eintrittsdatum
Schneider	Petra	1995-08-27
Lang	Ute	2005-06-07
Eisenhof	Frank	1982-07-08
Lupin	Anja	1998-02-17

Tabelle 2.11 Mitarbeiter, deren »eintrittsdatum« ungleich dem 13.01.1990 ist

2.2.4 Spaltenwerte auf kleiner als/gleich prüfen

Der Vergleichsoperator *kleiner als* < prüft, ob der linke Wert in einer Bedingung kleiner ist als der rechte Wert. Der Vergleichsoperator *kleiner als/gleich* <= prüft, ob der linke Wert kleiner als oder gleich dem rechten Wert ist.

Das wollen wir uns gleich zunutze machen, um die Mitarbeiter zu ermitteln, die einen Bonus von weniger als 500 € erhalten haben:

```
SELECT
    name,
    vorname,
    bonus
FROM
    mitarbeiter
```


WHERE

```
bonus<500;
```

Listing 2.13 Numerische Werte auf »kleiner als« prüfen und filtern

name	vorname	bonus
Lang	Ute	280.50
Ludwig	Heinz	127.45

Tabelle 2.12 Alle Boni unter 500 €

Das probieren wir jetzt auch mit einem Datumswert aus. Mit dem Kleiner-als-Operator können wir Zeilen herausfiltern, deren Datumswerte kleiner sind als ein von uns vorgegebener Wert. Wir blicken also von unserem Vergleichsdatumswert in die Vergangenheit. Ermitteln wir doch einfach mal alle Mitarbeiter, die vor dem 1978-01-01 eingestellt wurden:

SELECT

```
name,
vorname,
eintrittsdatum
```

FROM

```
mitarbeiter
```

WHERE

```
eintrittsdatum<'1978-01-01';
```

Listing 2.14 Datumswerte auf »kleiner als« prüfen und filtern

name	vorname	eintrittsdatum
Müller	Iris	1970-03-15
Schmidt	Werner	1970-03-01
Jule	Stefanie	1975-01-05
Serenius	Ingo	1972-04-11

Tabelle 2.13 Mitarbeiter, deren »eintrittsdatum« vor dem 01.01.1978 liegt

Wir haben uns bereits die Frage gestellt, welche Mitarbeiter vor dem 1978-01-01 eingestellt wurden. Wenn der 1. Januar 1978 als Stichtag in die Suche einbezogen werden soll, können Sie eine **WHERE**-Klausel formulieren, die exakt diese Mitarbeiter filtert, indem Sie den Kleiner-gleich-Operator `<=` verwenden.

Das Eintrittsdatum wird so begrenzt, dass alle Datensätze der Tabelle *mitarbeiter* abgefragt werden, die einem Datumswert entsprechen, der gleich dem oder kleiner als der Datumswert 1978-01-01 ist:

```
SELECT
    name,
    vorname,
    eintrittsdatum
FROM
    mitarbeiter
WHERE
    eintrittsdatum<='1978-01-01';
```

Listing 2.15 Datumswerte auf »kleiner als/gleich« prüfen und filtern

name	vorname	eintrittsdatum
Schmidt	Werner	1970-03-01
Jule	Stefanie	1975-01-05
Serenius	Ingo	1972-04-11
Bücher	Herbert	1978-01-01

Tabelle 2.14 Mitarbeiter, deren »eintrittsdatum« vor dem oder am 01.01.1978 war

2.2.5 Spaltenwerte auf größer als/gleich prüfen

Der *Größer-als*-Operator `>` und der *Größer-gleich*-Operator `>=` werden nach dem gleichen Prinzip wie der *Kleiner-als*- bzw. *Kleiner-gleich*-Vergleichsoperator in der **WHERE**-Klausel angewendet. Damit prüfen wir jetzt, welche Mitarbeiter eine Bonuszahlung von mehr als 500 € erhalten haben:

```
SELECT
    name,
    vorname,
    bonus
FROM
    mitarbeiter
WHERE
    bonus>500;
```

Listing 2.16 Numerische Werte auf »größer als« prüfen und filtern

In der Ergebnistabelle sind jetzt ausschließlich die Bonuszahlungen enthalten, die einem Wert von größer als 500 € entsprechen.

name	vorname	bonus
Eisenhof	Frank	888.00
Lupin	Anja	2000.00
Lempe	Dirk	1800.00
Müller	Iris	7800.00

Tabelle 2.15 Mitarbeiter, die einen Bonus erhalten haben, der größer als 500 € ist

Welche Mitarbeiter wurden nach dem 1. Januar 1978 eingestellt? Um diese Frage zu beantworten, formulieren wir eine **WHERE**-Klausel, deren Bedingung nur die Mitarbeiter filtert, die nach dem 1978-01-01 eingestellt wurden. Hierzu verwenden wir in der **WHERE**-Klausel den Größer-als-Operator wie in [Listing 2.17](#) dargestellt.

Der Vergleichsoperator prüft, ob die Spaltenwerte der Spalte *eintrittsdatum* größer sind als der Datumswert 1978-01-01:

```
SELECT
    name,
    vorname,
    eintrittsdatum
FROM
    mitarbeiter
WHERE
    eintrittsdatum > '1978-01-01';
```

Listing 2.17 Datumswerte auf »größer als« prüfen und filtern

name	vorname	eintrittsdatum
Müller	Ralf	1990-01-13
Schneider	Petra	1995-08-27
Klein	Thomas	1990-01-13
Lang	Ute	2005-06-07

Tabelle 2.16 Mitarbeiter, die nach dem 01.01.1978 eingestellt wurden

In [Listing 2.18](#) verändern wir die Abfrage aus [Listing 2.17](#) ein wenig, indem wir den Vergleichsoperator »größer als/gleich« `>=` in der **WHERE**-Klausel verwenden:

```
SELECT
    name,
    vorname,
    eintrittsdatum
FROM
    mitarbeiter
WHERE
    eintrittsdatum>='1978-01-01';
```

Listing 2.18 Datumswerte auf »größer als/gleich« prüfen und filtern

name	vorname	eintrittsdatum
Santus	Cornelia	1992-05-22
Oberscheidt	Michael	1984-07-31
Dorsten	Klaus	1997-04-21
Bücher	Herbert	1978-01-01

Tabelle 2.17 Mitarbeiter, die nach dem oder am 01.01.1978 eingestellt wurden

2.2.6 Bedingungen mit dem NOT-Operator verneinen

Die Überschrift hört sich zugegebenermaßen etwas merkwürdig an. Aber tatsächlich bietet SQL Ihnen die Möglichkeit, Bedingungen wie etwa `spalte1=wert1` oder `spalte2>wert2` in der **WHERE**-Klausel zu verneinen, um damit eine negierte Ergebnisliste zu erreichen. Eine derartige Negierung bedeutet die Umkehrung des Aussageergebnisses. Sie realisieren sie, indem Sie hinter dem Schlüsselwort **WHERE** einfach **NOT** notieren.

Eine Bedingung wie **WHERE NOT** `name='Schulze'` würde die Datenbank wie folgt interpretieren: Prüfe sämtliche Spaltenwerte der Spalte *name* auf Gleichheit mit dem Text Schulze, und gib die Zeilen zurück, wo der Spaltenwert der Spalte *name* nicht gleich Schulze ist. Die Aussage der Bedingung der **WHERE**-Klausel wird also negiert ausgewertet.

Die **WHERE**-Klausel in [Listing 2.19](#) filtert somit die Zeilen der Mitarbeiter heraus, deren Name ungleich Winter ist, obwohl die Filterbedingung die Spaltenwerte der Spalte *name* auf Gleichheit mit dem Namen Winter prüft:

```
SELECT
    name,
```

```

    vorname
FROM
    mitarbeiter
WHERE NOT
    name='Winter';

```

Listing 2.19 Das Ergebnis einer Bedingung negiert auswerten

name	vorname
Müller	Ralf
Schneider	Petra
Klein	Thomas
Lang	Ute

Tabelle 2.18 Das Ergebnis enthält sämtliche Datensätze der Tabelle »mitarbeiter« bis auf die Mitarbeiterin »Elise Winter« (Tabelle gekürzt).

Im nächsten Schritt formulieren wir eine Bedingung, die die Spaltenwerte der Spalte *name* auf Ungleichheit mit dem Namen *Winter* prüft. Gleich im Anschluss negieren wir das Ergebnis der Prüfung mit dem **NOT**-Operator:

```

SELECT
    name,
    vorname
FROM
    mitarbeiter
WHERE NOT
    name<>'Winter';

```

Listing 2.20 Einen Textwert auf Ungleichheit prüfen und das Ergebnis negieren

name	vorname
Winter	Elise

Tabelle 2.19 Die negierte Abfrage »name« ist nicht »Winter«.

In diesem Fall haben wir also für die **WHERE**-Klausel eine Bedingung formuliert, die auf Ungleichheit mit dem Namen *Winter* prüft, und die Aussage der Bedingung mit dem **NOT**-Operator der **WHERE**-Klausel umgekehrt. Folglich erhalten wir exakt den Datensatz, den wir in der Bedingung ausgeschlossen haben.

Die Verneinung **NOT** einer Bedingung in einer **WHERE**-Klausel können Sie natürlich auch für die bereits eingeführten Operatoren **<=**, **>**, **>=** und die noch folgenden Vergleichsoperatoren anwenden, ganz unabhängig davon, ob Sie numerische Werte, Textwerte oder Datumswerte vergleichen. Es handelt sich ja lediglich um eine Negierung der Bedingung, die Sie formuliert haben.

In [Listing 2.13](#) haben Sie bereits eine Bedingung in der **WHERE**-Klausel verwendet, in der die Spalte *bonus* auf kleiner als 500 geprüft wird. Exakt dieses Listing werden wir nun in leicht abgewandelter Form mit dem **NOT**-Operator verwenden, um die Zeilen zu ermitteln, deren Werte in der Spalte *bonus* gleich oder größer als 500 € sind:

```
SELECT
  name,
  vorname,
  bonus
FROM
  mitarbeiter
WHERE NOT
  bonus<500;
```

Listing 2.21 Eine numerische Bedingung auswerten und in der **WHERE**-Klausel negieren

name	vorname	bonus
Schneider	Petra	500.00
Klein	Thomas	500.00
Eisenhof	Frank	888.00
Lupin	Anja	2000.00

Tabelle 2.20 Boni, die nicht kleiner als »500« sind

2.2.7 Spaltenwerte auf ein Intervall prüfen (BETWEEN)

SQL bietet einen weiteren nützlichen Vergleichsoperator, der es uns ermöglicht, Bedingungen zu formulieren, die auf Intervalle prüfen. Halten wir uns einmal kurz vor Augen, was ein *Intervall* überhaupt ist. Mit den Vergleichsoperatoren größer als (**>**), größer als/gleich (**>=**), kleiner als (**<**) und kleiner als/gleich (**<=**) haben Sie bereits Bedingungen ausgewertet, die auf ein offenes Intervall prüfen.

Ein Intervall hingegen hat nicht nur zu einer Seite eine Begrenzung. Stellen Sie sich vor, Sie würden gern die Mitarbeiter ermitteln, die einen Umsatz von 200.000 € bis 600.000 € erzielen. Dann haben Sie schon ein numerisches Intervall festgelegt, dessen Grenzen bei 200.000 € und 600.000 € liegen. Um dies in einer **WHERE**-Klausel

auszudrücken, verwenden Sie das Schlüsselwort **BETWEEN** (dt. »zwischen«), um auf ein Intervall zu prüfen.

In [Listing 2.22](#) sehen Sie, wie Sie innerhalb einer **WHERE**-Klausel Spaltenwerte auf ein Intervall prüfen können. Zuerst geben Sie die Spalte an, deren Werte auf das Intervall geprüft werden sollen. Dann folgt das Schlüsselwort **BETWEEN**. Die Grenzen des Intervalls legen Sie danach fest. Zwischen den Werten, die das Intervall begrenzen, notieren Sie das Schlüsselwort **AND**. Die Datenbank interpretiert das Ganze dann wie folgt: »Filtere alle Ergebniszeilen, wo der Wert der Spalte *spalte1* zwischen dem Wert *wert1* und *wert2* liegt.« Beachten Sie bitte an dieser Stelle, dass die Start- und Endwerte (*wert1* und *wert2*) inklusiv enthalten sind. Wenn wir anstelle des **BETWEEN**-Operators Vergleichsoperatoren nutzen würden, könnten wir den Ausdruck also wie folgt formulieren: Die Werte der Spalte *umsatz* sind größer als/gleich 200.000 € und kleiner als/gleich 600.000 €.

Sie können auch eine Bedingung aufstellen, die auf Werte prüft, die sich außerhalb eines Intervalls befinden. Hierzu verwenden Sie einfach die **NOT BETWEEN**-Klausel:

```
WHERE spalte1 BETWEEN wert1 AND wert2;
WHERE spalte1 NOT BETWEEN wert1 AND wert2;
```

Listing 2.22 Spaltenwerte auf ein Intervall prüfen

Wie wären z. B. die Mitarbeiter zu ermitteln, deren Umsatz 200.000 € bis 600.000 € beträgt? In [Listing 2.23](#) prüft dazu eine **SELECT**-Anweisung mit einer **WHERE**-Klausel auf dieses Intervall. Hier wenden wir die **WHERE**-Klausel in Verbindung mit dem **BETWEEN**-Vergleichsoperator an. Hinter das Wort **BETWEEN** schreiben Sie einfach das Intervall, das Sie wünschen, in unserem Beispiel also 200.000 € und 600.000 €:

```
SELECT
    name,
    vorname,
    umsatz
FROM
    mitarbeiter
WHERE
    umsatz
    BETWEEN 200000 AND 600000;
```

Listing 2.23 Auf ein numerisches Intervall prüfen und filtern

Beachten Sie hier die linke und rechte Begrenzung des Intervalls. Das bedeutet: Wenn ein Mitarbeiter einen Umsatz von exakt 200.000 € oder 600.000 € erzielt hätte, wäre er auch in der Ergebnisliste enthalten.

name	vorname	umsatz
Nolte	Reinhard	355000.80
Sebastiansen	Inge	570000.00
Haier	Peter	347000.89
Klein	Bärbel	234000.00

Tabelle 2.21 Mitarbeiter, die einen Umsatz im Bereich von 200.000 € bis 600.000 € erzielt haben

Den **BETWEEN**-Vergleichsoperator können Sie auch hervorragend nutzen, um Datum-sintervalle zu prüfen. Sehen wir uns die Abfrage in [Listing 2.24](#) genauer an. Hier werden die Mitarbeiter ermittelt, die im Zeitraum vom 1981-01-01 bis 1984-12-31 eingestellt wurden:

```
SELECT
    name,
    vorname,
    eintrittsdatum
FROM
    mitarbeiter
WHERE
    eintrittsdatum
    BETWEEN
        '1981-01-01' AND '1984-12-31';
```

Listing 2.24 Auf ein Datumsintervall prüfen und filtern

name	vorname	eintrittsdatum
Eisenhof	Frank	1982-07-08
Haier	Peter	1984-09-07
Blume	Siegmar	1982-02-12
Kanis	Herbert	1984-10-15

Tabelle 2.22 Mitarbeiter, die zwischen dem 01.01.1981 und dem 31.12.1984 eingestellt wurden

Eine Bedingung, die auf ein Intervall prüft, können Sie auch verneinen. Hierzu modifizieren Sie die **SELECT**-Abfrage aus [Listing 2.23](#) und fügen vor dem Schlüsselwort

BETWEEN einfach den **NOT**-Operator ein. Wörtlich gesprochen bedeutet das so viel wie »nicht innerhalb von zwei vorgegebenen Intervallbegrenzungen«. Wir werden also die Mitarbeiter ermitteln, die weniger als 200.000 € und mehr als 600.000 € Umsatz erzielt haben. Das heißt, wir filtern nur die Zeilen, deren Wert der Spalte *umsatz* außerhalb des Bereichs von 200000 bis 600000 liegt:

SELECT

```
name,
vorname,
umsatz
```

FROM

```
mitarbeiter
```

WHERE

```
umsatz
```

NOT BETWEEN

```
200000 AND 600000;
```

Listing 2.25 Auf ein numerisches Intervall prüfen, das Ergebnis negieren und filtern

name	vorname	umsatz
Jule	Stefanie	700000.00
Petersen	Hans	156000.00
Steinmacher	Ilse	700898.03
Überall	Georg	678000.54

Tabelle 2.23 Der Umsatz liegt außerhalb des Bereichs von 200.000 € bis 600.000 €.

Streng genommen, haben Sie es hier mit zwei Intervallen zu tun: Das erste Intervall prüft die Spaltenwerte der Spalte *umsatz* auf 0 bis kleiner als 200000, und das zweite Intervall prüft die Werte der Spalte *umsatz* von größer als 600000 bis zum Maximalwert der gezahlten Bonuszahlungen. Liegt ein Spaltenwert der Spalte *umsatz* innerhalb eines der beiden Prüfintervalle, so wird die Zeile in die Ergebnistabelle übernommen. Diese Besonderheit tritt nur auf, wenn Sie eine Bedingung auf ein Intervall prüfen und das Ergebnis der Bedingung negieren.

In der letzten **SELECT**-Abfrage haben Sie einen Wertebereich negiert und dadurch eine Ergebnisliste erhalten, in der Sie Werte innerhalb der beiden Begrenzungen von 0 bis 199999 und von 600001 bis zum maximalen Umsatzeintrag eingegrenzt haben. Nun werden wir diese Negierung negieren und damit aufheben. Es ist vielleicht etwas widersprüchlich, da wir damit ja im Grunde die von uns erwünschte Bedingung wieder

rückgängig machen. Vermutlich gibt es für die jetzt folgende **SELECT**-Abfrage, in der ich den **NOT**-Operator hinter dem **WHERE**-Schlüsselwort hinzugefügt habe, kaum einen Anwendungsfall in der Praxis. Die Möglichkeit besteht aber dennoch, und ich möchte sie Ihnen nicht vorenthalten.

In [Listing 2.26](#) negieren wir also das Ergebnis der folgenden Bedingung: »Gib mir sämtliche Datensätze der Mitarbeiter zurück, deren Umsatz nicht zwischen 200.000 € und 600.000 € liegt.« Mit der Negierung dieser Gesamtaussage erreichen wir, dass wir eben doch die Mitarbeiter ermitteln, deren Umsatz zwischen 200.000 € und 600.000 € liegt.

```
SELECT
    name,
    vorname,
    umsatz
FROM
    mitarbeiter
WHERE NOT
    umsatz
NOT BETWEEN
    200000 AND 600000;
```

Listing 2.26 Negiert auf ein numerisches Intervall prüfen und dann die Negierung mit einer Negierung aufheben und das Ergebnis filtern

name	vorname	umsatz
Nolte	Reinhard	355000.80
Sebastiansen	Inge	570000.00
Haier	Peter	347000.89
Klein	Bärbel	234000.00

Tabelle 2.24 Negativierung des Ergebnisses aller Mitarbeiter, deren Umsatz nicht zwischen 200.000 € und 600.000 € liegt

Sie sehen also, dass eine bestehende Negierung eines **NOT BETWEEN**-Vergleichsoperators mit einer **WHERE NOT**-Klausel wieder aufgehoben werden kann. Sie werden im weiteren Verlauf noch andere Vergleichsoperatoren kennenlernen, die Sie auf die gleiche Art und Weise negieren können. Die bereits vorgestellten Vergleichsoperatoren können Sie natürlich auch auf diese Art und Weise anwenden und die aus den Vergleichen resultierenden Aussagen negieren.

2.2.8 Spaltenwerte auf ein Muster prüfen (LIKE)

Innerhalb einer **WHERE**-Klausel haben Sie die Möglichkeit, Spaltenwerte, die eine Zeichenkette enthalten, mit einem Muster zu vergleichen und nur die Zeilen zu filtern, deren Spaltenwerte dem Muster entsprechen. Für diese Musterprüfungen wird der **LIKE**-Vergleichsoperator verwendet. Der Vergleichsoperator **LIKE** bedeutet *wie* oder *gleich*. Er prüft, ob das Muster mit den Spaltenwerten übereinstimmt. Um Muster zu formulieren, bietet SQL Ihnen zwei Platzhalter an, die auf Text-Spaltenwerte angewendet werden können. Der Platzhalter %, auf Englisch auch *wildcard* oder *joker* genannt, steht für eine beliebige Anzahl beliebiger Zeichen in einer Zeichenkette.

Wenn Sie z. B. eine Textspalte in der **WHERE**-Klausel mit einem Muster auswerten möchten, dann könnte dieses Vergleichsmuster etwa wie folgt aussehen: '%strasse%' oder '%weg%'.

Wie sind diese Muster zu interpretieren? Das Muster lässt sich so verstehen: »Prüfe, ob eine Spalte Textwerte enthält, die mit beliebigen Zeichen beginnen, gefolgt von der Zeichenkette *strasse*, und die mit beliebigen Zeichen enden.« Das zweite Muster lässt sich genauso lesen: »Beliebige Zeichen umschließen die Zeichenkette *weg*.«

Oder gehen wir davon aus, dass Sie alle Mitarbeiter ermitteln möchten, deren Name mit *m* beginnt. Das Textmuster hierzu sähe wie folgt aus: 'm%'. Übersetzt heißt dies: »Wähle alle Werte aus, die mit einem *m* beginnen und mit einer beliebigen Anzahl beliebiger Zeichen enden.«

Den Platzhalter % können Sie auch mehrfach in Ihrem Textmuster verwenden. Betrachten Sie das Textmuster '%leistung%gut%'. Das Textmuster entspricht folgender Filteraussage: »Die Zeichenkette beginnt mit beliebigen Zeichen, gefolgt von der Zeichenkette *leistung*. Es folgen wieder beliebige Zeichen. Hinter der beliebigen Zeichenkette folgt die Zeichenkette *gut*, und der Textwert endet mit beliebigen Zeichen.« Mit dem Platzhalter % können Sie also sehr detaillierte Ähnlichkeitsprüfungen durchführen, die sich auf die Textspaltenwerte beziehen.

Es gibt noch einen Platzhalter, mit dem Sie Ähnlichkeiten in Textwerten prüfen können: den Underscore oder, auf Deutsch gesagt, den Unterstrich *_*. Der Unterstrich steht für exakt ein beliebiges Zeichen. Ein Textmuster wie 'd_i%' bedeutet, dass alle Spaltenwerte auf Übereinstimmung mit einem Muster geprüft werden, das mit einem *d* beginnt, gefolgt von einem beliebigen Zeichen, gefolgt von einem *i*, gefolgt von beliebigen Zeichen.

Groß- und Kleinschreibung in der Mustererkennung

Beachten Sie, dass sowohl eine MS SQL Server-Datenbank als auch eine MySQL- oder MariaDB-Datenbank die Muster, mit denen Werte verglichen werden, unabhängig von der Groß- und Kleinschreibung auswertet. Das Muster 'a%' führt also zum gleichen Ergebnis wie das Muster 'a%'.



Bei einer PostgreSQL-Datenbank müssen Sie hingegen die Groß- und Kleinschreibung beachten. In diesem Fall würden die Muster 'A%' und 'a%' zu unterschiedlichen Ergebnissen führen. PostgreSQL bietet für diesen Fall den Vergleichsoperator `ILIKE`. Mit ihm können Sie Mustervergleiche unabhängig von der Groß- und Kleinschreibung durchführen. Der Operator entspricht allerdings nicht dem Standard.



Zusammenfassung: Ähnlichkeitsprüfung mit den Platzhaltern % und _

- ▶ Der %-Platzhalter steht für eine beliebige Anzahl von beliebigen Zeichen.
- ▶ Der _-Platzhalter steht für exakt ein beliebiges Zeichen.
- ▶ Je nach verwendetem Datenbanksystem wird bei der Prüfung eines Musters zwischen Groß- und Kleinschreibung unterschieden. Hier müssen Sie gegebenenfalls die Dokumentation Ihres Datenbanksystems zurate ziehen.

Schauen wir uns in den folgenden Beispielen an, wie diese Platzhalter in Abfragen verwendet werden.

Die Syntax der nächsten **WHERE**-Klausel zeigt Ihnen, wie Sie auf Spaltenwerte prüfen, die mit einem a beginnen und mit einer beliebigen Anzahl von beliebigen Zeichen enden:

```
WHERE spalte1 LIKE 'a%'
```

Listing 2.27 Syntax für die Nutzung des »%«-Platzhalters in einem Muster

Wenn Sie nur ein Zeichen in einer Zeichenkette als Platzhalter verwenden wollen, nutzen Sie den Underscore wie in [Listing 2.28](#):

```
WHERE spalte1 LIKE 'a_ton'
```

Listing 2.28 Syntax für die Nutzung des Underscores »_« als Platzhalter in einem Muster

Negiert sieht das Ganze dann wie in [Listing 2.29](#) aus:

```
WHERE spalte1 NOT LIKE 'a%'
```

oder

```
WHERE spalte1 NOT LIKE 'a_ton'
```

Listing 2.29 Ähnlichkeitsprüfung mit den Platzhaltern »%« und »_«

Hier liefert die Datenbank sämtliche Zeilen zurück, die nicht dem Textmuster entsprechen – in diesem Fall also Werte, die nicht mit einem a beginnen beziehungsweise nicht ein beliebiges Zeichen zwischen einem a und der Zeichenkette ton aufweisen.

Sehen wir uns [Listing 2.30](#) genauer an. Hier werden die Spaltenwerte der Spalte *vorname* gemäß dem Muster 'K%' auf Übereinstimmung geprüft. Das Muster trifft zu, wenn der Vorname mit einem K beginnt und auf eine beliebige Zeichenkette endet. An dieser Stelle sei noch einmal erwähnt, dass der **LIKE**-Vergleichsoperator ausschließlich Zeichenketten miteinander vergleicht.

```
SELECT
    name,
    vorname
FROM
    mitarbeiter
WHERE
    vorname LIKE 'K%';
```

Listing 2.30 Mit dem Platzhalter »%« auf ein Muster prüfen und filtern

name	vorname
Dorsten	Klaus

Tabelle 2.25 Alle Mitarbeiter, deren Vorname mit »K« beginnt

Natürlich können Sie das Stellvertreterzeichen % auch am Anfang eines Textmusters anwenden. Hierzu passen Sie die Abfrage aus [Listing 2.30](#) ein wenig an, indem Sie das Textmuster in '%r' ändern. Die Interpretation des Textmusters in [Listing 2.31](#) lautet: »Prüfe alle Werte der Spalte *vorname* darauf, ob sie mit einer beliebigen Zeichenkette beginnen und mit dem Zeichen r enden.«

```
SELECT
    name,
    vorname
FROM
    mitarbeiter
WHERE
    vorname LIKE '%r';
```

Listing 2.31 Mit dem Stellvertreterzeichen »%« ein Vergleichsmuster beginnen und filtern

name	vorname
Schmidt	Werner
Haier	Peter

Tabelle 2.26 Alle Mitarbeiter, deren Vorname mit »r« endet

name	vorname
Blume	Siegmar
Gelz	Oliver

Tabelle 2.26 Alle Mitarbeiter, deren Vorname mit »r« endet (Forts.)

Bis jetzt haben wir einen Platzhalter – oder *Joker* – am Anfang und am Ende einer Zeichenkette angewendet. Sie werden sich nun sicherlich fragen, wie es aussieht, wenn Sie den Platzhalter mitten in einem Textmuster verwenden möchten. Sehen wir uns an, wie wir so etwas machen können.

Unser nächstes Ziel besteht darin, ein Vergleichsmuster zu entwerfen, das die Spaltenwerte der Spalte *strasse* prüft. Wir formulieren das Textmuster '*S%straße*', um es mit den Werten der Spalte *strasse* zu vergleichen. Mit dem in [Listing 2.32](#) verwendeten Suchmuster ermitteln wir die Straßen, die mit einem *S* beginnen und mit der Zeichenkette *straße* enden. Mit dem Prozentzeichen legen wir für das Muster außerdem fest, dass zwischen dem *S* und der Zeichenkette *straße* beliebige Zeichen in beliebiger Anzahl erlaubt sind:

```
SELECT
  name,
  vorname,
  strasse,
  hausnummer
FROM
  mitarbeiter
WHERE
  strasse LIKE 'S%straße';
```

Listing 2.32 Mit dem Platzhalter »%« innerhalb einer Zeichenkette auf ein Muster prüfen und filtern

Mit dem formulierten Suchmuster für die Spaltenwerte der Spalte *strasse* stimmen vier Straßenbezeichnungen überein (siehe [Tabelle 2.27](#)).

name	vorname	strasse	hausnummer
Schneider	Petra	Sonnenstraße	189
Funke	Doris	Spiralstraße	75
Nolte	Reinhard	Sombrerostraße	34z
Steinmacher	Ilse	Supernovastraße	346

Tabelle 2.27 Alle Straßen, die mit »S« beginnen und auf »straße« enden

Wir haben mit dem Platzhalter % schon einige Vergleiche durchgeführt, die Sie in den Übungen sicherlich noch mehrfach anwenden werden. Nicht ignorieren wollen wir aber auch die Beispiele zum Platzhalter ...

Der Underscore ist ein Platzhalter, der ein einzelnes beliebiges Zeichen repräsentiert. Er kann in dem zu vergleichenden Textmuster ebenfalls mehrfach vorkommen. Zu Beginn formulieren wir ein Vergleichsmuster, das mit einem beliebigen Zeichen beginnt, gefolgt von der Zeichenkette `eltstraße`. In [Listing 2.33](#) sehen Sie, wie Sie den Underscore in einem Vergleichsmuster verwenden:

```
SELECT
    name,
    vorname,
    strasse,
    hausnummer
FROM
    mitarbeiter
WHERE
    strasse LIKE '_eltstraße';
```

Listing 2.33 Zu Beginn einer Zeichenkette auf das Stellvertreterzeichen Underscore prüfen und filtern

name	vorname	strasse	hausnummer
Sternental	Marlene	Weltstraße	97
Heinrich	Hans	Geltstraße	65

Tabelle 2.28 Alle Straßen, die ein beliebiges Zeichen vor »eltstraße« haben

Es handelt sich lediglich um zwei Zeilen, bei denen der Spaltenwert der Spalte *strasse* mit dem Vergleichsmuster übereinstimmt, das wir formuliert haben. Die Spaltenwerte beginnen jeweils mit den Zeichen W und G und enden mit der von uns vorgegebenen Zeichenkette `eltstraße`.

Die Ähnlichkeitsprüfung mit dem `LIKE`-Vergleichsoperator können Sie ebenfalls wie Vergleichsausdrücke mit `WHERE` negieren. Hierzu wenden Sie einfach das Schlüsselwort `NOT` vor dem Schlüsselwort `LIKE` an.

Achtung: Verwechslungsgefahr

Verwechseln Sie diese Verneinung nicht mit der Verneinung, die Sie auf eine gesamte Bedingung anwenden können.



Stellen Sie sich vor, Sie möchten aus der Spalte *strasse* nur die Werte erhalten, die nicht auf *straße* enden. Sehen Sie sich hierzu die **SELECT**-Anweisung in [Listing 2.34](#) an. In der **WHERE**-Klausel sehen Sie, dass die **LIKE**-Klausel zunächst die Spaltenwerte ermittelt, die dem Muster '%straße' entsprechen. Mit dem Schlüsselwort **NOT** vor der **LIKE**-Klausel (**NOT LIKE**) kehren wir die Aussage wieder um und erhalten so alle Zeilen in der Ergebnisliste, in der die Spaltenwerte der Spalte *strasse* nicht diesem Muster entsprechen:

```
SELECT
    name,
    vorname,
    strasse,
    hausnummer
FROM
    mitarbeiter
WHERE
    strasse
NOT LIKE
    '%straße';
```

Listing 2.34 Auf ein Muster prüfen, die Aussage negieren und filtern

[Tabelle 2.29](#) enthält die zurückgelieferten Zeilen, deren Werte der Spalte *strasse* nicht auf die Zeichenfolge *straße* enden. Alles, was auf *weg* endet oder eine Straße mit der Bezeichnung *Uranus* ist, hat es in diesem Beispiel in die Ergebnisliste geschafft.

name	vorname	strasse	hausnummer
Lupin	Anja	Saturnweg	1
Lempe	Dirk	Venusweg	100
Müller	Iris	Merkurweg	770
Ludiwig	Heinz	Uranus	27c

Tabelle 2.29 Eine negierte Textmusterprüfung

2.2.9 Spaltenwerte auf Mengenzugehörigkeit prüfen

In der **WHERE**-Klausel einer **SELECT**-Abfrage gibt es noch eine weitere Möglichkeit, Bedingungen zu formulieren, um Spaltenwerte auf Zugehörigkeit in einer Liste zu prüfen. Hierzu notieren Sie einfach hinter dem Schlüsselwort **WHERE** die auszuwertende Spalte und das Schlüsselwort **IN** (»in etwas enthalten«), gefolgt von einer öffnenden Klammer und einer kommaseparierten Liste mit Werten. Hinter dem letzten Wert

schließen Sie die Klammer und signalisieren so, dass die Liste von Werten vollständig ist.

```
WHERE spalte1 IN ('wert1', 'wert2', ...)
```

Listing 2.35 Syntax für die Prüfung von Spaltenwerten auf Zugehörigkeit in einer Liste mit Text-, Zahlen- oder Datumswerten

In [Listing 2.36](#) sehen Sie, wie Sie die Mitgliedschaftsaussage von Spaltenwerten negiert auswerten können. Hierzu schreiben Sie einfach vor die **IN**-Klausel das Schlüsselwort **NOT**. Das bedeutet, dass Zeilen, deren Spaltenwert einer auszuwertenden Spalte in der Liste enthalten ist, nicht mit in die Ergebnisliste aufgenommen werden.

```
WHERE spalte1 NOT IN ('wert1', 'wert2', ...)
```

Listing 2.36 Negierung der Zugehörigkeitsaussage

Stellen Sie sich vor, Sie bilden eine Arbeitsgruppe von Mitarbeitern, die einem bestimmten Projekt zugeordnet sind. Zu Ihrem Projektteam gehören folgende Mitarbeiter:

- ▶ Ralf Müller
- ▶ Petra Schneider
- ▶ Frank Eisenhof
- ▶ Simone Klarfeld
- ▶ Werner Schmidt
- ▶ Doris Funke
- ▶ Rolf Stein
- ▶ Bärbel Klein

Aus den Nachnamen bilden Sie eine kommaseparierte Liste, die wie folgt aussieht: Müller, Schneider, Eisenhof, Klarfeld, Schmidt, Funke, Stein, Klein. Wenn Sie eine **SELECT**-Abfrage an die Datenbank senden, die prüft, wo die Spalte *name* einem Wert in Ihrer Liste entspricht, erhalten Sie sämtliche Zeilen der Mitarbeiter, die sich in Ihrer geklammerten Liste befinden. Vorausgesetzt natürlich, diese Mitarbeiterwerte befinden sich in der Spalte *name* in der Tabelle *mitarbeiter*.

```
SELECT
    *
FROM
    mitarbeiter
WHERE
    name
```

```
IN (  
    'Müller',  
    'Schneider',  
    'Eisenhof',  
    'Klarfeld',  
    'Schmidt',  
    'Funke',  
    'Stein',  
    'Klein'  
);
```

Listing 2.37 WHERE-Klausel, die Spaltenwerte auf Mitgliedschaft in einer Liste prüft

Tabelle 2.30 zeigt uns nur die Mitarbeiter, die in der Liste hinter dem **IN**-Vergleichsoperator enthalten sind. Sie können so eine Liste, die auf Zugehörigkeit prüft, auch für numerische Werte oder Datumswerte verwenden, um die Spaltenwerte einer ausgewählten Spalte mit einer Liste von Spaltenwerten zu vergleichen.

mitarbeiterid	name	vorname	gebdatum
1	Müller	Ralf	1970-12-20
2	Schneider	Petra	1965-03-07
3	Klein	Thomas	1970-12-20
5	Eisenhof	Frank	1962-05-15

Tabelle 2.30 Eine Zugehörigkeitsprüfung mit dem Vergleichsoperator IN

Den Mitgliedschaftsvergleich mithilfe einer Liste können Sie auch negiert anwenden. Die Aussage der Bedingung zu negieren heißt, dass die Datenbank sämtliche Ergebniszeilen ermittelt, in denen die zu prüfenden Spaltenwerte nicht in der Liste hinterlegt sind. Hierzu formulieren wir eine **SELECT**-Abfrage, die sämtliche Mitarbeiter aus der Tabelle *mitarbeiter* ermittelt, die nicht in den Abteilungen Personal, Vertrieb, Controlling oder Verkauf beschäftigt sind. Um das zu erreichen, notieren Sie einfach das Schlüsselwort **NOT** vor dem **IN**-Operator, der ja auf Mengenzugehörigkeit prüft. Unser Ziel ist es, sämtliche Mitarbeiter zu ermitteln, die in der verbliebenen Abteilung Einkauf tätig sind:

```
SELECT  
    name,  
    vorname,  
    abteilung  
FROM  
    mitarbeiter
```

WHERE

```

abteilung
  NOT IN
  (
    'Personal',
    'Vertrieb',
    'Controlling',
    'Verkauf'
  );

```

Listing 2.38 Negiert auf Mitgliedschaft in einer Liste prüfen

Das Ergebnis der Abfrage aus [Listing 2.38](#) ist in [Tabelle 2.31](#) dargestellt. Es werden ausschließlich die Mitarbeiter ermittelt und in die Ergebnisliste aufgenommen, die nicht in den genannten Abteilungen tätig sind. In der Liste enthalten sind also nur die verbleibenden Mitarbeiter der Abteilung Einkauf, die nicht in der Zugehörigkeitsauflistung enthalten ist.

name	vorname	abteilung
Müller	Ralf	Einkauf
Schneider	Petra	Einkauf
Klein	Thomas	Einkauf
Lang	Ute	Einkauf

Tabelle 2.31 Eine Prüfung mit dem NOT IN-Vergleichsoperator**2.2.10 Fehlende Spaltenwerte (NULL-Value)**

In [Abschnitt 2.2.11](#) werden wir nach Werten filtern, die nicht definiert bzw. nicht vorhanden sind. Um sich mit fehlenden Spaltenwerten vertraut zu machen, halten Sie sich Ihr Adressbuch vor Augen – sei es auf Papier oder in einer Adressverwaltung.

Mit Kontaktdaten wird in der Regel auf unterschiedlichste Art und Weise umgegangen: Die einen geben Ihnen ihre Telefonnummer, die anderen geben Ihnen ihre E-Mail-Adresse. Auf einem Blatt Papier bleiben hier einfach die Felder frei, denen kein Eintrag zugeordnet ist. Der Platz für die Informationen ist jedoch in der Tabelle durch leere Kästchen reserviert. Wenn Sie die Informationen erhalten, können Sie sie einfach nachtragen.

In Tabellen relationaler Datenbanksysteme wird dieser Zustand als **NULL** beschrieben. **NULL** heißt nichts anderes, als dass der Wert nicht bekannt ist oder schlicht und ergreifend nicht zur Verfügung steht. In einer Zeile, in der Daten vorhanden sind, jedoch

das eine oder andere Feld keine Einträge enthält, wird diesem Feld der Eintrag **NULL**, also »nicht definiert«, zugeordnet.

Besonders wichtig ist, dass ein **NULL**-Wert nichts mit dem numerischen Wert 0 zu tun hat. Bei dem Wert 0 handelt es sich um einen konkreten Wert, während **NULL** einen fehlenden Wert darstellt. Wichtig ist auch, dass Sie zwischen leeren Zeichenketten und **NULL**-Werten unterscheiden können. Eine leere Zeichenkette stellt wieder einen konkreten Wert dar, nämlich eben den einer leeren Zeichenkette.

Unsichtbare Zeichen wie ein Leerzeichen, ein Tab oder ein Zeilenumbruch haben auch nichts mit einem **NULL**-Wert gemein, da es sich hierbei ja auch wieder um Zeichen handelt, die Sie zwar nicht optisch wahrnehmen können, die aber dennoch existieren.

Betrachten wir **NULL** im Zusammenhang mit der **WHERE**-Klausel, so haben **NULL**-Werte Auswirkungen auf die Auswertung von Bedingungen. Das Ergebnis einer Bedingung in einer **WHERE**-Klausel kann wahr (true) oder falsch (false) sein. Wie wir eben festgestellt haben, gibt es Werte, die nicht bekannt bzw. nicht definiert sind. Sollten Sie also auf die Idee kommen, solche Spaltenwerte mit Vergleichsoperatoren wie `=`, `<>`, `<` oder `>` zu vergleichen, so wird dies immer zu einem nicht definierten Ergebnis führen. Das Ergebnis ist also weder wahr noch falsch, sondern nicht bekannt.

Nehmen wir einmal einen Vergleich, wie wir ihn in der **WHERE**-Klausel verwenden könnten, in die Spaltenauswahlliste auf, um zu sehen, welches Ergebnis ein Ausdruck liefert, der auf Gleichheit mit **NULL** prüft. Bedenken Sie, dass wir eigentlich Werte wie wahr oder falsch erwarten würden, die ein Ausdruck ja als logische Aussage zur Folge haben kann.

```
SELECT
  name,
  name=NULL,
  name='Klein'
FROM
  mitarbeiter;
```

Listing 2.39 Auswertung von nicht definierten Werten in einer Spaltenauswahlliste

Sehen wir uns Tabelle 2.32 an.

name	name=NULL	name='Klein'
Müller	NULL	0
Schneider	NULL	0

Tabelle 2.32 Ein Vergleich mit **NULL**

name	name=NULL	name='Klein'
Klein	NULL	1
Lang	NULL	0

Tabelle 2.32 Ein Vergleich mit NULL (Forts.)

Die Abfrage in [Listing 2.39](#) enthält in der Spaltenauswahlliste zwei Ausdrücke, die ausgewertet werden.

Der eine Ausdruck vergleicht die Spaltenwerte der Spalte *name* auf Gleichheit mit NULL. In der dritten Spalte sehen Sie das Ergebnis eines Ausdrucks, der die Spaltenwerte der Spalte *name* auf Gleichheit mit der Zeichenkette *Klein* prüft. In der Programmierung werden die Zahlen 0 und 1 synonym für falsch beziehungsweise wahr verwendet.

Das Ergebnis der ersten Auswertung in der zweiten Spalte überrascht uns jedoch ein wenig: Es wird NULL zurückgegeben. Die Aussage des Vergleichs ist also weder wahr noch falsch, denn die Aussage ist einfach nicht definiert. Das Ergebnis der zweiten Auswertung in der dritten Spalte ist schon eher wie erwartet. Sie sehen hier also, dass der Vergleich mit der Zeichenkette *Klein* einmal zu einer wahren Aussage führt. Die restlichen Vergleiche führen zu einer unwahren Aussage.

NULL in Bedingungen

In der Logik kann es nicht nur zu einer wahren oder falschen Aussage kommen. Dies ist für die Beschäftigung mit SQL von großer Bedeutung, da es Auswirkungen darauf hat, wie Bedingungen ausgewertet werden. Wenn Sie also komplexe Bedingungen formulieren, sollten Sie sich dies unbedingt vor Augen führen.



Drei-Werte-Logik

Es gibt neben wahr und falsch einen dritten Wert, der das Ergebnis einer Aussage repräsentieren kann. Dabei handelt sich um einen nicht definierten Wert bzw. einen nicht vorhandenen Wert, den wir in SQL als NULL bezeichnen. Dieses Phänomen wird als *Drei-Werte-Logik* bezeichnet.

Beachten Sie, dass in einer WHERE-Klausel nur Bedingungen, die als wahr ausgewertet werden, in der Ergebnistabelle Berücksichtigung finden.



Sehen Sie sich die SELECT-Abfrage aus [Listing 2.40](#) an, die in der Spaltenauswahlliste lediglich zwei NULL-Werte auf Gleichheit prüft (sie funktioniert allerdings nur auf einer MySQL-, MariaDB oder PostgreSQL-Datenbank):

```
SELECT NULL=NULL;
```

Listing 2.40 Zwei NULL-Werte auf Gleichheit überprüfen

In [Tabelle 2.33](#) sehen Sie das Ergebnis der Abfrage. **NULL**-Werte sind nie gleich, denn ein **NULL**-Wert bedeutet ja, dass kein Wert definiert ist. Ein nicht definierter Wert kann natürlich nicht mit irgendetwas auf Gleichheit geprüft werden. Sie erhalten also wieder einen **NULL**-Wert als Ergebnis des Vergleichs anstelle von `wahr` oder `falsch`. Dieses Ergebnis zeigt also noch einmal deutlich, dass es eine Drei-Werte-Logik gibt und nicht nur eine Zwei-Werte-Logik mit den Werten `wahr` oder `falsch`.

NULL=NULL
NULL

Tabelle 2.33 Gleichheitsprüfung zweier **NULL**-Werte

Sie kennen jetzt die Auswirkungen von **NULL**-Werten auf die Aussagen von Bedingungen. Dies ist aber nicht ihr einziger Anwendungszweck. **NULL**-Werte können auch als Reservierung von Speicherplatz im Sinne einer Adresszuordnung für eine fehlende Information betrachtet werden. Die Reservierung gilt, bis eine Information eingetragen wird. Den Speicherbedarf von Werten sehen wir uns in [Abschnitt 4.1](#) genauer an.



Zusammenfassung: Nicht definierte Werte (NULL)

Spaltenwerte, die nicht definiert sind, werden als **NULL**-Werte bezeichnet. **NULL**-Werte können also als Platzhalter für einen echten Wert betrachtet werden. Beachten Sie Folgendes bei der Auswertung von Bedingungen in einer **WHERE**-Klausel:

- ▶ **NULL**-Werte haben nichts mit dem realen numerischen Wert 0 zu tun.
- ▶ **NULL**-Werte haben nichts mit leeren Zeichenketten gemeinsam.
- ▶ **NULL**-Werte führen in einem Ausdruck zu einem dritten, nicht logischen Zustand (nicht bekannt, nicht definiert).

Es gibt noch einige andere Auswirkungen, die zu beachten sind, wenn **NULL**-Werte als Spaltenwerte vorkommen. In [Abschnitt 2.5](#) erfahren Sie, was Sie bei Sortierungen beachten müssen, wenn die zu sortierende Spalte **NULL**-Werte enthält.

2.2.11 Spaltenwerte auf **NULL** prüfen

In diesem Abschnitt werden Sie Zeilen filtern, in denen Werte einer Spalte nicht definiert sind, also dem Wert **NULL** entsprechen. Die Syntax zum Vergleichen von Spaltenwerten mit **NULL** ist einfach: Hinter der **WHERE**-Klausel geben Sie die Spalte an, deren Spaltenwerte Sie mit **NULL** vergleichen wollen. Um diese Werte auf **NULL** zu prüfen,

notieren Sie einfach hinter der zu prüfenden Spaltenbezeichnung den **IS NULL**-Vergleichsoperator. In [Listing 2.41](#) sehen Sie diese Syntax:

```
WHERE spalte IS NULL;
```

Listing 2.41 Syntax zur Prüfung auf nicht definierte Werte einer Spalte

Jetzt formulieren wir eine Abfrage mit einer **WHERE**-Klausel, in der wir genau nach diesen nicht definierten Spaltenwerten filtern werden, indem wir in der **WHERE**-Klausel die Spalte *umsatz* auf den Wert **NULL** filtern. Mitarbeitern, die nicht in den Abteilungen Verkauf oder Vertrieb beschäftigt sind, wird kein Umsatz zugeordnet. Nachvollziehen können Sie dies in der Spalte *abteilung*. Hier finden Sie Abteilungen wie die Einkaufsabteilung, deren Mitarbeitern in der Spalte *umsatz* jeweils ein **NULL**-Wert zugewiesen ist.

```
SELECT
    name,
    vorname,
    abteilung,
    umsatz
FROM
    mitarbeiter
WHERE
    umsatz IS NULL;
```

Listing 2.42 Auf Spaltenwerte prüfen, die nicht definiert sind

name	vorname	abteilung	umsatz
Müller	Ralf	Einkauf	NULL
Schneider	Petra	Einkauf	NULL
Klein	Thomas	Einkauf	NULL
Lang	Ute	Einkauf	NULL

Tabelle 2.34 Alle Mitarbeiter, denen keine Vertriebs- oder Verkaufsaktivitäten zugeordnet sind

2.2.12 Spaltenwerte auf »ist nicht NULL« prüfen

In diesem Abschnitt lernen Sie, wie Sie nach Spaltenwerten filtern, die nicht **NULL** sind. In [Listing 2.43](#) sehen Sie die Syntax zur Anwendung des **IS NOT NULL**-Vergleichsoperators. Der Vergleichsoperator **IS** wird hier um das Schlüsselwort **NOT** erweitert, um eine

Negation der Bedingung zu erreichen. Umgangssprachlich könnten Sie das mit »ist nicht NULL« übersetzen.

WHERE spalte **IS NOT NULL**

Listing 2.43 Spaltenwerte auf »nicht NULL« prüfen

Nachdem wir die Spalte *umsatz* der Tabelle *mitarbeiter* auf den Wert **NULL** geprüft haben, bietet es sich jetzt an, sie auf »nicht NULL« zu prüfen und zu filtern. Hierzu modifizieren wir die **SELECT**-Abfrage aus [Listing 2.42](#) so, dass wir die Datensätze abfragen, in denen die Spaltenwerte der Spalte *umsatz* ungleich **NULL** sind.

```
SELECT
  name,
  vorname,
  abteilung,
  umsatz
FROM
  mitarbeiter
WHERE
  umsatz IS NOT NULL;
```

Listing 2.44 **SELECT**-Anweisung, deren Bedingung die Spalte »umsatz« auf »ungleich NULL«, also »nicht NULL«, prüft

Die Datenbank sendet Ihnen ein Ergebnis wie in [Tabelle 2.35](#).

name	vorname	abteilung	umsatz
Nolte	Reinhard	Vertrieb	355000.80
Jule	Stefanie	Vertrieb	700000.00
Petersen	Hans	Vertrieb	156000.00
Sebastiansen	Inge	Vertrieb	570000.00

Tabelle 2.35 Alle Mitarbeiter, die den Abteilungen Vertrieb und Verkauf angehören und einen Umsatz erzielt haben (gekürzte Darstellung)

2.2.13 Spaltenwerte mit Spaltenwerten vergleichen

Bisher haben Sie Spaltenwerte mit Zahlen-, Datums- und Textwerten verglichen. Sie können auch die Spaltenwerte einer Zeile miteinander vergleichen. Hierzu formulieren wir eine **SELECT**-Abfrage wie in [Listing 2.45](#), in der wir prüfen, wo die Bonuszahlungen der Mitarbeiter größer sind als die erzielten Umsätze, um zu vermeiden, dass zukünftig nicht gerechtfertigte Bonuszahlungen ausgezahlt werden.

SELECT

```
name,
vorname,
umsatz,
bonus
```

FROM

```
mitarbeiter
```

WHERE

```
bonus>umsatz;
```

Listing 2.45 SELECT-Anweisung, deren Bedingung prüft, ob der Spaltenwert von »bonus« größer ist als der von »umsatz«

name	vorname	umsatz	bonus
Oberscheidt	Michael	150000.00	200000.00

Tabelle 2.36 Der Mitarbeiter Michael Oberscheidt erhält einen Bonus, der größer ist als der von ihm erwirtschaftete Umsatz.

Zusammenfassung: WHERE-Klausel

Die **WHERE**-Klausel wird immer dann verwendet, wenn Sie nur Ergebniszeilen wünschen, die einem bestimmten Filterkriterium entsprechen. Hierzu vergleichen Sie die Spaltenwerte einer Spalte mithilfe eines Vergleichsoperators mit einem oder mehreren anderen Werten. Ist die Aussage eines Vergleichs wahr, so wird der Datensatz, also die Zeile, mit in die Ergebnisliste aufgenommen.

Folgende Vergleichsoperatoren (in rechteckigen Klammern mit optionalen Zusätzen) können Sie in einer **WHERE**-Klausel verwenden:

- ▶ **=** prüft auf Gleichheit.
- ▶ **<>** und **!=** prüfen beide auf Ungleichheit.
- ▶ **<** prüft auf »kleiner als«.
- ▶ **<=** prüft auf »kleiner als/gleich«.
- ▶ **>** prüft auf »größer als«.
- ▶ **>=** prüft auf »größer als/gleich«.
- ▶ **NOT** negiert eine Aussage oder ein Ergebnis einer Bedingung.
- ▶ **[NOT] BETWEEN** prüft, ob ein Wert in einem Intervall enthalten ist.
- ▶ **[NOT] LIKE** prüft auf ein Muster.
- ▶ **[NOT] IN** prüft, ob ein Wert in einer Menge enthalten ist.
- ▶ **IS [NOT] NULL** prüft, ob ein Wert definiert ist.



In [Abschnitt 2.3](#) widmen wir uns dem Thema der logischen Verknüpfung von mehreren Bedingungen in einer **WHERE**-Klausel. Sie werden lernen, wie Sie mit den logischen Operatoren **AND** und **OR** verknüpfte Aussagen von zwei oder mehreren Bedingungen in einer Gesamtaussage auswerten können.

2.2.14 Übungen

Übung 1

Ermitteln Sie die Mitarbeiter, die in der Abteilung Vertrieb tätig sind. Formulieren Sie hierzu eine **WHERE**-Klausel, die die Spaltenwerte der Spalte *abteilung* auf Gleichheit mit dem Textausdruck 'Vertrieb' prüft. In der Spaltenauswahlliste wählen Sie sämtliche Spalten der Tabelle *mitarbeiter* mit dem Stellvertreterzeichen Asterisk (*) aus.

Übung 2

Überprüfen Sie, welche Mitarbeiter bereits 30 Tage ihres Urlaubs in Anspruch genommen haben. Vergleichen Sie hierzu in einer **WHERE**-Klausel die Spaltenwerte der Spalte *urlaubgenommen* der Tabelle *mitarbeiter* mit dem numerischen Wert 30. In der Ergebnistabelle geben Sie die Spalten *name*, *vorname* und *urlaubgenommen* aus.

Übung 3

Ihnen fällt nicht der Mitarbeiter ein, den Sie am '1978-05-22' eingestellt haben? Formulieren Sie dazu eine **SELECT**-Abfrage, in der Sie die Werte der Spalte *eintrittsdatum* der Tabelle *mitarbeiter* auf Gleichheit mit dem genannten Datumswert prüfen. In die Spaltenauswahlliste nehmen Sie die Spalten *name*, *vorname* und *eintrittsdatum* auf.

Übung 4

Im Rahmen einer Datenermittlung soll aufgelistet werden, welche Mitarbeiter ihr Konto nicht bei der Stadtsparkasse Köln haben. Formulieren Sie eine **WHERE**-Klausel, in der Sie die Spaltenwerte der Spalte *bankinstitut* der Tabelle *mitarbeiter* auf Ungleichheit mit der Zeichenkette 'Stadtsparkasse Köln' prüfen. In der Ergebnistabelle der **SELECT**-Abfrage sollen die Spalten *name*, *vorname* und *bankinstitut* enthalten sein.

Übung 5

Ermitteln Sie alle Mitarbeiter, die keine Überstunden oder Minusstunden auf dem Stundenkonto aufweisen. In der Ergebnisliste sollen die Spalten *name*, *vorname* und *ueberstunden* der Tabelle *mitarbeiter* enthalten sein.

Übung 6

Natürlich interessiert uns auch, wer Überstunden auf seinem Arbeitskonto angehäuft hat. Prüfen Sie diesen Sachverhalt mithilfe einer Bedingung, in der die Werte

der Spalte *ueberstunden* auf »größer als 0« verglichen werden. In der Ergebnisliste der **SELECT**-Abfrage sollen die Spalten *name*, *vorname* und *ueberstunden* der Tabelle *mitarbeiter* enthalten sein.

Übung 7

Formulieren Sie eine **WHERE**-Klausel, die die Mitarbeiter ermittelt, die seit dem '2000-01-01' in dem Unternehmen beschäftigt sind. Nutzen Sie für den Vergleich die Spalte *eintrittsdatum* der Tabelle *mitarbeiter*, und geben Sie die Spalten *name*, *vorname* und *eintrittsdatum* in der Ergebnistabelle aus.

Übung 8

Fragen Sie sämtliche Mitarbeiter ab, die ein oder mehrere Kinder haben. In Ihrer Bedingung verwenden Sie den Vergleichsoperator (=) und den **NOT**-Operator. Dazu ein kleiner Hinweis: Die Bedingung soll die Spaltenwerte der Spalte *anzahlkinder* auf Gleichheit mit dem Wert 0 prüfen. Die Bedingung verneinen Sie im Anschluss in der **WHERE**-Klausel gleich wieder mit dem **NOT**-Operator. In der Spaltenauswahlliste nehmen Sie die Spalten *mitarbeiterid*, *name*, *vorname* und *anzahlkinder* der Tabelle *mitarbeiter* auf.

Übung 9

Formulieren Sie als Nächstes eine **SELECT**-Abfrage, die die Mitarbeiter ermittelt, die zwischen dem '1990-01-01' und dem '2000-01-01' eingestellt wurden. In der **WHERE**-Klausel verwenden Sie zum Filtern die Spalte *eintrittsdatum*, deren Spaltenwerte Sie gegen das Datumsintervall prüfen. In die Ergebnisliste nehmen Sie die Spalten *mitarbeiterid*, *name*, *vorname* und *eintrittsdatum* der Tabelle *mitarbeiter* auf.

Übung 10

Formulieren Sie ein Textmuster, das die Spalte *geschlecht* auf ein beliebiges Zeichen prüft. In die Spaltenauswahl nehmen Sie die Spalten *name*, *vorname* und *geschlecht* der Tabelle *mitarbeiter* auf.

Lösung zu Übung 1

```
SELECT
    *
FROM
    mitarbeiter
WHERE
    abteilung='Vertrieb';
```

Listing 2.46 Datensätze nach einem Textwert auf Gleichheit filtern

Lösung zu Übung 2

SELECT

```
name,  
vorname,  
urlaubgenommen
```

FROM

```
mitarbeiter
```

WHERE

```
urlaubgenommen=30;
```

Listing 2.47 Datensätze nach einem numerischen Literal auf Gleichheit filtern

Lösung zu Übung 3

SELECT

```
name,  
vorname,  
eintrittsdatum
```

FROM

```
mitarbeiter
```

WHERE

```
eintrittsdatum='1978-05-22';
```

Listing 2.48 Datensätze nach einem Datumsliteral auf Gleichheit filtern

Lösung zu Übung 4

Sie erhalten eine leere Ergebnisliste zurück, da kein Mitarbeiter bei einem anderen Institut ein Konto hat. Alternativ zum <>-Ungleichheitsoperator können Sie natürlich auch den !=-Ungleichheitsoperator verwenden.

SELECT

```
name,  
vorname,  
bankinstitut
```

FROM

```
mitarbeiter
```

WHERE

```
bankinstitut<>'Stadtsparkasse Köln';
```

Listing 2.49 Datensätze mit einem Ungleich-Vergleichsoperator gegen ein Textliteral prüfen und filtern

Lösung zu Übung 5

SELECT

name,
vorname,
ueberstunden

FROM

mitarbeiter

WHERE

ueberstunden<=0;

Listing 2.50 Datensätze mit einem Kleiner-gleich-Vergleichsoperator gegen ein numerisches Literal prüfen und filtern

Lösung zu Übung 6

SELECT

name,
vorname,
ueberstunden

FROM

mitarbeiter

WHERE

ueberstunden>0;

Listing 2.51 Datensätze mit einem Größer-gleich-Vergleichsoperator gegen ein numerisches Literal prüfen und filtern

Lösung zu Übung 7

SELECT

name,
vorname,
eintrittsdatum

FROM

mitarbeiter

WHERE

eintrittsdatum >= '2000-01-01';

Listing 2.52 Datensätze mit einem Größer-gleich-Vergleichsoperator gegen ein Datumsliteral prüfen und filtern

Lösung zu Übung 8

SELECT

```
mitarbeiterid,  
name,  
vorname,  
anzahlkinder
```

FROM

```
mitarbeiter
```

WHERE NOT

```
anzahlkinder=0;
```

Listing 2.53 Datensätze mit einem negierten Vergleich auf einen Wert ungleich 0 filtern

Lösung zu Übung 9

SELECT

```
name,  
vorname,  
eintrittsdatum
```

FROM

```
mitarbeiter
```

WHERE

```
eintrittsdatum  
    BETWEEN '1990-01-01' AND '2000-01-01';
```

Listing 2.54 Datensätze auf ein Datumsintervall zwischen zwei Datumsangaben filtern

Lösung zu Übung 10

SELECT

```
name,  
vorname,  
geschlecht
```

FROM

```
mitarbeiter
```

WHERE

```
geschlecht LIKE '_';
```

Listing 2.55 Datensätze mit einem einstelligen Platzhalter filtern

2.3 Filterbedingungen mit AND (NOT) und OR (NOT) logisch verknüpfen

Sie haben schon einige Vergleichsoperatoren kennengelernt und können bereits etliche Filterkriterien formulieren. Der nächste Schritt ist, zwei Bedingungen in Ihrer **WHERE**-Klausel zu verwenden und sie logisch miteinander zu verknüpfen. SQL bietet Ihnen hierzu die logischen Verknüpfungsoperatoren **AND** (»und«) und **OR** (»oder«), mit denen Sie in einer **WHERE**-Klausel gleich mehrere Bedingungen verwenden können. An dieser Stelle noch der Hinweis, dass für die Verknüpfungsoperatoren, die Sie kennenlernen werden, die Gesetze der *booleschen Algebra* gelten. Die am häufigsten verwendeten Szenarien dieser Algebra stelle ich Ihnen im Zusammenhang mit der Formulierung von zusammengesetzten Filterkriterien vor. Sehen wir uns zunächst an, wie Sie zwei Bedingungen mit dem **AND**-Operator logisch verknüpfen.

2.3.1 Der logische Verknüpfungsoperator AND

Der logische Verknüpfungsoperator **AND** verknüpft innerhalb einer **WHERE**-Klausel Bedingungen und ist dann wahr, wenn sämtliche Aussagen der Bedingungen, die Sie mit **AND** verknüpfen, wahr sind. Eine derartige Verknüpfung von Bedingungen wird als *Konjunktion* bezeichnet. Das Wort »Konjunktion« stammt aus dem Lateinischen und bedeutet so viel wie »Verbindung«. Sie verbinden so zwei Aussagen von Bedingungen. Nur dann, wenn Bedingung 1 und Bedingung 2 gleichzeitig wahr sind, ist die Gesamtaussage ebenfalls wahr. Betrachten Sie hierzu Tabelle 2.37.

Bedingung 1	Bedingung 2	Ergebnis
wahr	wahr	wahr
falsch	falsch	falsch
falsch	wahr	falsch
wahr	falsch	falsch

Tabelle 2.37 Wahrheitstabelle für die AND-Verknüpfung

Das Ergebnis der Gesamtaussage ist nur dann wahr, wenn die Aussagen von Bedingung 1 und Bedingung 2 wahr sind. Die übrigen drei verknüpften Bedingungen können nicht in ihrer Gesamtaussage wahr sein, da sie dieser Anforderung nicht genügen. Eine solche Tabelle, die wir hier verwendet haben, um logische Aussagen mit **AND** zu verknüpfen, wird als *Wahrheitstabelle* bezeichnet.

Zum besseren Verständnis der AND-Verknüpfung von logischen Aussagen betrachten Sie Abbildungung 2.1. Sie sehen eine Batterie, zwei Schalter und eine Glühbirne, die durch

Symbole dargestellt werden. Die Schalter 1 und 2 sind beide nicht betätigt. Es kann also kein Strom fließen, um die Glühbirne zum Leuchten zu bringen. Schließen Sie nur Schalter 1 oder nur Schalter 2, so bringt Sie das auch nicht weiter: Die Glühbirne würde nicht leuchten.

Damit geben wir uns natürlich nicht zufrieden. Wir wollen die Glühbirne zum Leuchten bringen. In [Abbildung 2.2](#) sind Schalter 1 und Schalter 2 geschlossen. Der Strom kann also fließen, und die Glühbirne leuchtet.

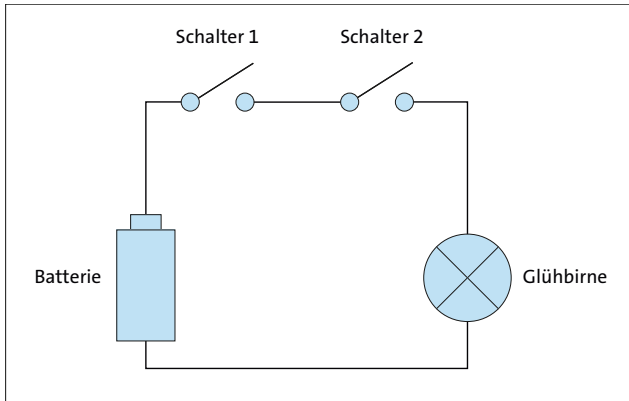


Abbildung 2.1 Logische UND-Verknüpfung mit zwei geöffneten Schaltern in einer elektrischen Schaltung

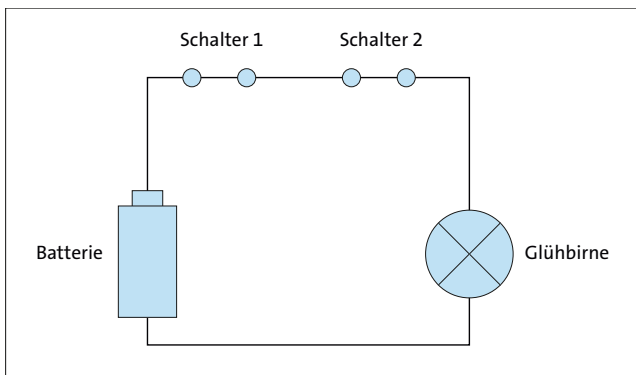


Abbildung 2.2 Logische UND-Verknüpfung mit zwei geschlossenen Schaltern in einer elektrischen Schaltung

2.3.2 SQL-Bedingungen mit dem logischen AND-Operator verknüpfen

Schauen wir uns nun an, wie sich diese logische Bedingung in SQL darstellen lässt.

Als Erstes sollen die Mitarbeiter, die Kinder haben und trotzdem Überstunden leisten, ermittelt werden, um ihnen einen Bonus von 1.000 € zuzuordnen. Hierzu for-

mulieren wir eine **SELECT**-Abfrage, die in der Spaltenauswahlliste die Spalten *name*, *vorname*, *ueberstunden* und *anzahlkinder* enthält. Außerdem formulieren wir eine **WHERE**-Klausel, die prüft, ob die Spaltenwerte der Spalte *ueberstunden* und die Spaltenwerte der Spalte *anzahlkinder* größer als 0 sind. Es handelt sich also um zwei Bedingungen, die mit einem **AND**-Verknüpfungsoperator (*ueberstunden*>0 **AND** *anzahlkinder*>0) verbunden werden. Ausschließlich dann, wenn beide Bedingungen erfüllt sind – also nur, wenn die Aussagen der Bedingungen *ueberstunden*>0 und *anzahlkinder*>0 wahr sind –, schafft es eine Zeile in die Ergebnistabelle.

SELECT

```
name,
vorname,
ueberstunden,
anzahlkinder
```

FROM

```
mitarbeiter
```

WHERE

```
ueberstunden>0 AND anzahlkinder>0;
```

Listing 2.56 Zwei Ergebnisse von Bedingungen in der **WHERE**-Klausel mit einem logischen **AND** verknüpfen

name	vorname	ueberstunden	anzahlkinder
Müller	Ralf	3	4
Lupin	Anja	15	2
Müller	Iris	80	2
Jule	Stefanie	21	1

Tabelle 2.38 Sämtliche Mitarbeiter, die Überstunden machen und (**AND**) Kinder haben

Natürlich können Sie auch mehr als zwei Bedingungen miteinander verknüpfen, wie das nächste Beispiel zeigt, in dem drei Bedingungen mit dem logischen **AND**-Operator verknüpft werden. Hierzu wollen wir die Mitarbeiter der Abteilung Verkauf ermitteln, um ihnen einen Sonderbonus zukommen zu lassen.

Neben dieser Bedingung sollen ausschließlich die Mitarbeiter ausgegeben werden, die einen Umsatz von mehr als 500.000 € erzielen und Überstunden leisten. In die Spaltenauswahlliste nehmen wir die Spalten *name*, *vorname*, *umsatz*, *ueberstunden* und *abteilung* auf. Wir haben es hier also mit drei Bedingungen zu tun, die wir sprachlich wie folgt auswerten können: »Die Gesamtaussage in der **WHERE**-Klausel ist erfüllt, wenn die Aussage der Bedingung *umsatz*>500000 und die Aussage der Bedingung

ueberstunden>0 und die Aussage der Bedingung abteilung='Verkauf' wahr sind.« Es müssen also alle drei Aussagen der Bedingungen wahr sein, damit die Gesamtaussage der **WHERE**-Klausel wahr ist.

SELECT

```
name,  
vorname,  
umsatz,  
ueberstunden,  
abteilung
```

FROM

```
mitarbeiter
```

WHERE

```
umsatz>500000 AND ueberstunden>0 AND abteilung='Verkauf';
```

Listing 2.57 Drei Bedingungen mit dem logischen AND-Operator in der WHERE-Klausel verknüpfen

name	vorname	umsatz	ueberstunden	abteilung
Sonnenfeld	Udo	650000.00	10	Verkauf
Dorsten	Klaus	600000.00	2	Verkauf
Bücher	Herbert	1200000.80	48	Verkauf
Sternental	Marlene	600000.00	5	Verkauf

Tabelle 2.39 Alle Mitarbeiter, die im Verkauf arbeiten und (AND) Überstunden leisten und (AND) über 500.000 € Umsatz erzielen

2.3.3 Der logische Verknüpfungsoperator OR

Der logische Verknüpfungsoperator **OR** verknüpft ebenfalls zwei oder mehrere Bedingungen in einer **WHERE**-Klausel. Aber wie es das Wort »or« schon vermuten lässt, ist bei Bedingungen die Gesamtaussage wahr, wenn Bedingung 1 **oder** Bedingung 2 wahr ist. Wenn beide Bedingungen erfüllt sind, ist die Gesamtaussage der mit **OR** verknüpften Bedingungen ebenfalls wahr. Nur dann, wenn keine der Bedingungen 1 oder 2 wahr sind, ist die logische Gesamtaussage falsch. Eine derartige Verknüpfung wird als *Disjunktion* bezeichnet. Das Wort Disjunktion bedeutet so viel wie »Trennung« oder »Unterscheidung«. Die Wahrheitstabelle zeigt Ihnen die verschiedenen Möglichkeiten im Überblick:

Bedingung 1	Bedingung 2	Ergebnis
wahr	falsch	wahr
falsch	wahr	wahr
wahr	wahr	wahr
falsch	falsch	falsch

Tabelle 2.40 Wahrheitstabelle für die »OR«-Verknüpfung

Nutzen wir auch hier einen Schaltkreis als Beispiel (siehe [Abbildung 2.3](#)). Die Anordnung der Schalter in unserer elektrischen Schaltung hat sich etwas geändert. Wieder ist keiner der Schalter geschlossen. Aber Sie erkennen, dass der Stromkreis sich schließen wird und unsere Glühbirne leuchtet, wenn Sie einen der beiden Schalter betätigen.

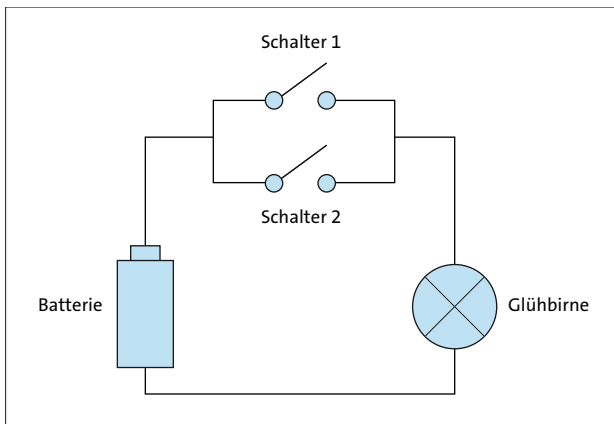


Abbildung 2.3 Logische ODER-Verknüpfung mit zwei offenen Schaltern in einer elektrischen Schaltung

In [Abbildung 2.4](#) sehen Sie Schalter 1 betätigt. Der Stromkreis ist geschlossen, und die Glühbirne leuchtet. Das Gleiche erreichen Sie, wenn Sie ausschließlich Schalter 2 betätigen – die Glühbirne würde ebenfalls leuchten. Sie könnten also Schalter 1 oder Schalter 2 betätigen, damit die Glühbirne leuchtet. Betätigen Sie beide Schalter, ist der Stromkreis ebenfalls geschlossen.

Sie haben logische UND- und ODER-Beziehungen anhand von elektrischen Schaltkreisen kennengelernt. Wie sieht das Ganze in einer **WHERE**-Klausel in SQL aus?

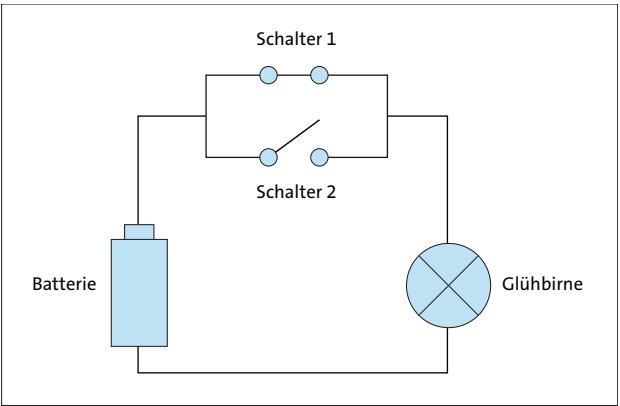


Abbildung 2.4 Logische ODER-Verknüpfung mit einem betätigten Schalter in einer elektrischen Schaltung

2.3.4 SQL-Bedingungen mit dem logischen OR-Operator verknüpfen

Die Mitarbeiter der Abteilungen Personal und Einkauf sollen nun auch mit Bonuszahlungen bedacht werden. Bei dem Filterkriterium handelt es sich um zwei Bedingungen, die die Spaltenwerte der Spalte *abteilung* auf Gleichheit mit den Werten Personal und Einkauf prüfen. Wie in [Listing 2.58](#) dargestellt, realisieren wir die logische Verknüpfung mit einem OR-Verknüpfungsoperator, da entweder die eine Bedingung oder die andere Bedingung erfüllt sein muss, um zu einer wahren Gesamtaussage zu kommen.

```
SELECT
    name,
    vorname,
    abteilung
FROM
    mitarbeiter
WHERE
    abteilung='Einkauf' OR abteilung='Personal';
```

Listing 2.58 Zwei Bedingungen mit dem logischen OR-Operator verknüpfen

name	vorname	abteilung
Eisenhof	Frank	Einkauf
Lupin	Anja	Einkauf

Tabelle 2.41 Mitarbeiter, die der Abteilung Einkauf oder (OR) Personal angehören

name	vorname	abteilung
Lempe	Dirk	Personal
Müller	Iris	Personal

Tabelle 2.41 Mitarbeiter, die der Abteilung Einkauf oder (OR) Personal angehören (Forts.)

Wir haben eine Abteilung vergessen und wollen das natürlich nicht so stehen lassen. Hierzu formulieren wir eine **SELECT**-Abfrage, in der wir auch sämtliche Mitarbeiter berücksichtigen, die der Abteilung Controlling zugeordnet sind. Die Spaltenauswahlliste enthält die Spalten *name*, *vorname* und *abteilung*.

SELECT

```
name,
vorname,
abteilung
```

FROM

```
mitarbeiter
```

WHERE

```
abteilung='Einkauf' OR
abteilung='Personal' OR
abteilung='Controlling';
```

Listing 2.59 Drei Bedingungen mit dem logischen OR-Operator verknüpfen

Der logische **OR**-Operator zur Verknüpfung von Bedingungen ist in einer **WHERE**-Klausel ebenfalls beliebig oft nutzbar.

name	vorname	abteilung
Schneider	Ralf	Einkauf
Schneider	Petra	Einkauf
Lempe	Dirk	Personal
Müller	Iris	Personal
Stein	Rolf	Controlling
Groß	Vera	Controlling

Tabelle 2.42 Mitarbeiter, die der Abteilung Einkauf oder (OR) Personal oder (OR) Controlling angehören

2.3.5 Der logische Verknüpfungsoperator AND NOT

Eine weitere Möglichkeit besteht darin, eine Bedingung mit dem AND NOT-Verknüpfungsoperator mit einer anderen Bedingung zu verknüpfen. Das bedeutet: Wenn die eine Aussage wahr ist und die folgende Aussage nicht wahr ist, ist die Gesamtaussage wahr. Sehen Sie sich hierzu [Tabelle 2.43](#) an, die die Wahrheitswerte auflistet.

Bedingung 1	Bedingung 2	Ergebnis
wahr	wahr	falsch
wahr	falsch	wahr
falsch	wahr	falsch
falsch	falsch	falsch

Tabelle 2.43 Wahrheitstabelle für den logischen AND NOT-Operator

Es gibt nur eine Kombination, die im Ergebnis wahr ist, nämlich dann, wenn Bedingung 1 wahr ist und Bedingung 2 nicht wahr ist. Um das Ganze besser nachvollziehen zu können, sehen wir uns [Abbildung 2.5](#) an.

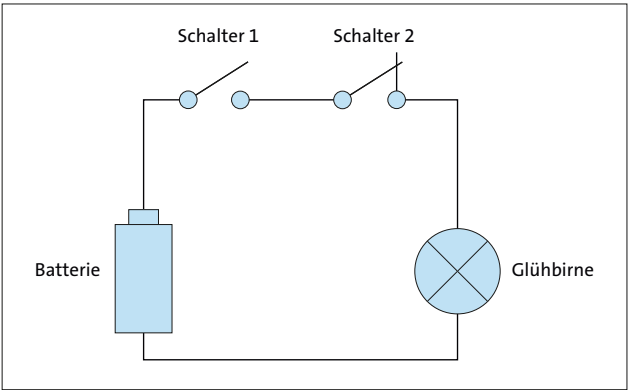


Abbildung 2.5 Logische AND NOT-Verknüpfung in einer elektrischen Schaltung

In unserem elektrischen Schaltkreis hat sich etwas verändert. Schalter 2 ist kein schließender Schalter mehr, sondern ein öffnender Schalter. Das heißt, wenn Sie ihn betätigen, wird der Stromkreis unterbrochen, und die Glühlampe leuchtet nicht. Ist also Schalter 1 betätigt und damit analog gesehen zu einer Aussage wahr und Schalter 2 nicht betätigt und damit analog zu einer Aussage falsch, so ist der Stromkreis geschlossen, und die Glühlampe leuchtet. Die Gesamtaussage der logischen Verknüpfung ist also wahr.

2.3.6 SQL-Bedingungen mit dem AND NOT-Operator logisch verknüpfen

Kommen wir zur Praxis. Wir werden die Mitarbeiter ermitteln, deren Werte der Spalte *austrittsdatum* nicht gleich **NULL** sind, die also aus der Firma ausgetreten sind. In einer weiteren Bedingung prüfen wir die Werte der Spalte *austrittsgrund* auf Gleichheit mit dem Wert 'ruhestand'. Diese Bedingung ist also in ihrer logischen Aussage wahr, wenn der Wert gleich 'ruhestand' ist. Uns interessieren in diesem Fall aber alle Mitarbeiter, die über ein Austrittsdatum verfügen und deren Austrittsgrund nicht ruhestand ist. Um unser Filterergebnis zu realisieren, verknüpfen wir die zwei Bedingungen und damit ihre Aussagen mit dem **AND NOT**-Verknüpfungsoperator. Hier wird die Aussage der Bedingung *austrittsgrund* gleich *ruhestand* also negiert verknüpft. Wenn die Aussage der Bedingung falsch ist, wird sie durch den logischen Verknüpfungsoperator **AND NOT** zu einer wahren Gesamtaussage führen.

SELECT

```
name,
vorname,
austrittsdatum,
austrittsgrund
```

FROM

```
mitarbeiter
```

WHERE

```
austrittsdatum IS NOT NULL AND NOT austrittsgrund='ruhestand';
```

Listing 2.60 Zwei Bedingungen mit dem AND NOT-Operator verknüpfen

name	vorname	austrittsdatum	austrittsgrund
Lempe	Dirk	2002-01-31	Veränderung
Klarfeld	Simone	1997-01-31	Veränderung
Haier	Peter	2011-07-31	Abfindungsprogramm
Groß	Vera	2005-03-31	Veränderung

Tabelle 2.44 Sämtliche Mitarbeiter, deren Spaltenwerte in der Spalte »austrittsdatum« nicht NULL sind und für die die Spalte »austrittsgrund« nicht den Spaltenwert »ruhestand« aufweist

2.3.7 Der logische Verknüpfungsoperator OR NOT

Nun lernen Sie den **OR NOT**-Verknüpfungsoperator kennen. Eine wahre Gesamtaussage einer **ODER NICHT**-Verknüpfung aus zwei Bedingungen ergibt sich entweder aus einer wahren Aussage einer Bedingung 1 oder aus einer nicht wahren Aussage einer Bedingung 2, wie in Tabelle 2.45 dargestellt:

Bedingung 1	Bedingung 2	Ergebnis
wahr	falsch	wahr
falsch	wahr	falsch
wahr	wahr	wahr
falsch	falsch	wahr

Tabelle 2.45 Wahrheitstabelle für den logischen OR NOT-Operator

Sehen Sie sich zum besseren Verständnis Abbildung 2.6 an. Hier handelt es sich wieder um eine elektrische Schaltung, in der Sie eine logische **OR NOT**-Verknüpfung mit einem einfachen elektrischen Schaltkreis nachgebildet sehen. Beide Schalter sind nicht betätigt, allerdings ist Schalter 2 wieder ein Ausschalter. Für uns bedeutet das also, dass Aussage 1 falsch und Aussage 2 falsch ist. Da es sich bei Schalter 2 jedoch um einen Ausschalter handelt, ist der Stromkreis dennoch geschlossen. Die Gesamtaussage ist also wahr, obwohl Aussage 2 falsch ist; die Glühbirne leuchtet. Die **OR NOT**-Verknüpfung könnten wir also wie folgt sprachlich formulieren: »Die Gesamtaussage ist wahr, wenn Schalter 1 betätigt wird (wahr ist) oder Schalter 2 nicht betätigt wird (die Aussage also falsch ist).«

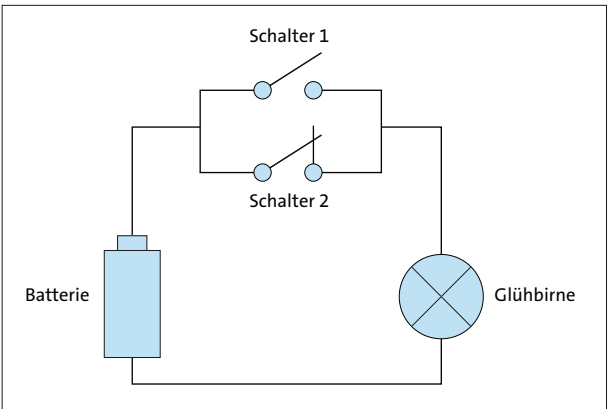


Abbildung 2.6 Logische ODER NICHT-Verknüpfung mit einem nicht betätigten einfachen Schalter und einem Öffner in einer elektrischen Schaltung

2.3.8 SQL-Bedingungen mit dem logischen OR NOT-Operator verknüpfen

Kommen wir zur Praxis: Unser Ziel ist es, die Mitarbeiter zu ermitteln, die zur Abteilung *Einkauf* gehören, oder die, die nicht wegen des Abfindungsprogramms die Firma verlassen haben.

SELECT

```
name,
vorname,
abteilung,
austrittsgrund
```

FROM

```
mitarbeiter
```

WHERE

```
abteilung='Einkauf' OR NOT austrittsgrund='abfindungsprogramm';
```

Listing 2.61 Zwei Bedingungen mit dem logischen OR NOT-Operator verknüpfen

An diesem Beispiel sehen Sie schon, dass es nur selten sinnvolle Szenarien für eine **OR NOT**-Klausel gibt. Wenn Sie Abfragen mit **OR NOT** schreiben, prüfen Sie genau, ob Sie nicht stattdessen **AND NOT** meinen. Das ist meistens wesentlich zielführender und kommt in der Praxis häufig vor, da so die Bedingungen ja kombiniert werden.

name	vorname	abteilung	austrittsgrund
Eisenhof	Frank	Einkauf	Ruhestand
Lupin	Anja	Einkauf	NULL
Jule	Stefanie	Vertrieb	Ruhestand
Eisenhof	Linda	Vertrieb	Ruhestand

Tabelle 2.46 Alle Mitarbeiter, die entweder zur Abteilung Einkauf gehören oder nicht aufgrund eines Abfindungsprogramms die Firma verlassen haben.

2.3.9 Logische Verknüpfungsoperatoren kombiniert anwenden

Logische Verknüpfungsoperatoren wie **AND**, **OR**, **AND NOT** oder **OR NOT** lassen sich auch kombiniert für die Verknüpfung unterschiedlichster Bedingungen in einer **WHERE**-Klausel anwenden.

Das sehen wir uns jetzt näher an: Vollzeitbeschäftigte Mitarbeiter verfügen im Jahr über 30 Urlaubstage. Teilzeitbeschäftigte erhalten hingegen 15 Urlaubstage. Wenn das Jahr zu Ende geht, ist sicherzustellen, dass Vollzeitbeschäftigte mindestens 20 Tage ihres Urlaubs in Anspruch genommen haben. Zudem ist sicherzustellen, dass Teilzeitbeschäftigte mindestens 10 Tage genutzt haben. Es sind also sämtliche Mitarbeiter zu ermitteln, die entweder über 30 Urlaubstage verfügen und weniger als 20 Tage davon genommen haben oder die 15 Urlaubstage zur Verfügung hatten und davon weniger als 10 Tage genommen haben.

In der Tabelle *mitarbeiter* finden Sie die Spalten *urlaubstage* und *urlaubgenommen*. Diese beiden Spalten eignen sich hervorragend für die Bedingungen *urlaubstage=30* und *urlaubgenommen<20* oder *urlaubstage=15* und *urlaubgenommen<10*.

In Listing 2.62 sehen Sie die gemischte Anwendung der logischen Verknüpfungsoperatoren **AND** und **OR** und die Umsetzung der hier geschilderten Anforderungen:

SELECT

```
name,  
vorname,  
urlaubstage,  
urlaubgenommen
```

FROM

```
mitarbeiter
```

WHERE

```
urlaubstage=30 AND urlaubgenommen<20  
OR  
urlaubstage=15 AND urlaubgenommen<10;
```

Listing 2.62 Vier Bedingungen und ihre Ergebnisse mit den logischen Operatoren AND und OR verknüpfen

name	vorname	urlaubstage	urlaubgenommen
Sommer	Werner	15	8
Oberscheidt	Michael	15	7
Heinrich	Hans	15	8
Groß	Vera	30	19
Friedrichs	Martin	30	14
Kanis	Herbert	30	18

Tabelle 2.47 Sämtliche Mitarbeiter, die entweder Anspruch auf 30 Urlaubstage hatten und (AND) davon weniger als 20 Tage genommen haben oder (OR) die Anspruch auf 15 Urlaubstage hatten und (AND) davon weniger als 10 Tage genommen haben

Sie kennen die Regel, dass in einer Berechnung, in der eine Multiplikation und eine Addition vorkommen, immer die Punkt- vor der Strichrechnung durchzuführen ist. Ähnliches gilt, wenn Sie mehr als zwei Bedingungen logisch miteinander verknüpfen: Die logische **AND**-Verknüpfung hat immer Vorrang vor der **OR**-Verknüpfung.

Es gibt nur eine Ausnahme: Indem Sie Klammern setzen, können Sie die Reihenfolge der Auswertung für Bedingungen ändern. Auch hier kennen Sie die alte Regel aus der

Mathematik, dass Sie durch Klammersetzung der Strichrechnung Vorrang vor der Multiplikation gewähren können. Diese einfachen Vorrangregeln der Mathematik sind in Abbildung 2.7 dargestellt. In der Logik haben sie vergleichbare Auswirkungen auf das Ergebnis.

$2 \times 3 + 2 = 8$ $2 \times (3 + 2) = 10$	} Punkt-vor-Strich- Rechnung in der Mathematik
---	--

Abbildung 2.7 Multiplizieren hat Vorrang vor einer Addition.

In Tabelle 2.48 wird die Aussage A1 einer Bedingung mit der Aussage A2 einer Bedingung mithilfe des **AND**-Operators verknüpft. Diese Verknüpfung wird immer zuerst ausgewertet. Die Aussage der **AND**-Verknüpfung wird dann mithilfe des nachrangigen **OR**-Operators mit der Aussage A3 logisch verknüpft. Im letzten Schritt kommen wir zur Gesamtaussage A4.

A1	A2	A3	A4 = A1 AND A2 OR A3
F	F	F	F
F	F	W	W
F	W	F	F
F	W	W	W
W	F	F	F
W	F	W	W
W	W	F	W
W	W	W	W

Tabelle 2.48 Wahrheitstabelle für eine AND- mit einer OR-Verknüpfung

Sehen wir uns jetzt an, welche Auswirkungen der Vorrang des **AND**-Operators bei der Auswertung von mehreren verknüpften Bedingungen in einer **WHERE**-Klausel hat. Hierzu ermitteln wir die Mitarbeiter, die einer falschen Steuerklasse zugeordnet sind. Zunächst ist es erforderlich, die Steuerklasse zu prüfen. In diesem Fall prüfen wir in einer Bedingung die Werte der Spalte *steuerklasse* auf Gleichheit mit dem Wert 1. Mit einem logischen **AND** verknüpfen wir das Ergebnis dieser Prüfung mit einer weiteren Bedingung, die prüft, ob die Anzahl der Kinder größer als 0 ist. Zuletzt werten wir eine dritte Bedingung aus, nämlich ob die gefundenen Mitarbeiter verheiratet sind. Die Aussage dieser Bedingung verknüpfen wir mithilfe eines logischen **OR**-Operators mit

dem Ergebnis aus der logischen **AND**-Verknüpfung. In die Spaltenauswahlliste nehmen wir die Spalten *name*, *vorname*, *steuerklasse*, *anzahlkinder* und *verheiratet* auf.

```
SELECT
    name,
    vorname,
    steuerklasse,
    anzahlkinder,
    verheiratet
FROM
    mitarbeiter
WHERE
    steuerklasse=1
AND
    anzahlkinder>0
OR
    verheiratet='ja';
```

Listing 2.63 Auswirkungen der Auswertungsreihenfolge von Bedingungen, die mit AND und OR verknüpft sind

Wenn Sie das Beispiel aus [Listing 2.63](#) nicht sofort nachvollziehen können, ist das überhaupt nicht schlimm. Die Auflösung folgt in der Betrachtung der Ergebnistabelle:

name	vorname	steuerklasse	anzahlkinder	verheiratet
Müller	Ralf	3	4	ja
Schneider	Petra	3	3	ja
Klein	Thomas	3	4	ja
Eisenhof	Frank	3	4	ja

Tabelle 2.49 Die Ausgabe von [Listing 2.63](#) führt aufgrund der Vorrangregel nicht zu dem gewollten Ergebnis.

Ihnen fällt sofort auf, dass auch Mitarbeiter mit der Steuerklasse 3 in der Ergebnisliste enthalten sind. Das entspricht nicht ganz der Anforderung, die wir gestellt hatten. Ich habe Sie hier bewusst in eine Disjunktionsfalle gelockt, um Ihnen zu zeigen, dass Sie die Vorrangregeln immer berücksichtigen müssen.

Was ist hier passiert? In der ersten Bedingung wird auf die Steuerklasse geprüft. Hier ist nur der Wert 1 zulässig. Daraus folgt, dass das Ergebnis bzw. die Aussage der Bedin-

gung nur dann wahr ist, wenn die Steuerklasse gleich 1 ist. Dieses Ergebnis wird mithilfe des logischen **AND**-Operators mit der Bedingung verknüpft, die prüft, ob die Anzahl der Kinder größer als 0 ist. Zuletzt folgt eine Bedingung, die mithilfe des **OR**-Operators verknüpft wird. Sie prüft, ob der Mitarbeiter verheiratet ist.

Sie wissen bereits, dass die **AND**-Verknüpfungen von Bedingungen zuerst ausgewertet werden und zu einer wahren oder falschen Aussage führen. Mit der **AND**-Verknüpfung werden die Bedingungen »Steuerklasse ist gleich 1« und »Anzahl der Kinder ist größer als 0« verknüpft. Ist eine der Aussagen der Bedingungen nicht wahr, so ist die Gesamtaussage der **AND**-Verknüpfung ebenfalls nicht wahr. Hier fehlt also noch die **OR**-Verknüpfung zu dem Ergebnis der zuvor ausgeführten Konjunktion. Es gibt leider nur einen Haken: Ist die Aussage der Konjunktion falsch, wird nur noch die Bedingung der **OR**-Klausel ausgewertet. In die Ergebnisliste schaffen es dann alle Mitarbeiter, die verheiratet sind und somit auch der Steuerklasse 3 zugeordnet sind.

Mit diesem Ergebnis wollen wir uns natürlich nicht zufriedengeben. In [Listing 2.64](#) sehen Sie eine modifizierte **WHERE**-Klausel, in der hinter der **AND**-Verknüpfung die Auswertung einer **OR**-Verknüpfung in einem Klammersausdruck steht. Jetzt wird zuerst der Wert der Steuerklasse ausgewertet. Dann wird die **AND**-Verknüpfung mit dem Ergebnis der **OR**-Verknüpfung in der Klammer ausgewertet.

WHERE

```
steuerklasse=1
```

AND

```
(
```

```
anzahlkinder>0
```

OR

```
verheiratet='ja'
```

```
);
```

Listing 2.64 Den Vorrang einer Auswertung von verknüpften Bedingungen durch Klammern beeinflussen

Das Resultat in [Tabelle 2.50](#) sieht schon besser aus.

name	vorname	steuerklasse	anzahlkinder	verheiratet
Lempe	Dirk	1	2	ja
Paulus	Sebastian	1	1	nein
Serenius	Ingo	1	7	nein

Tabelle 2.50 Die Konjunktionsfalle wurde mithilfe von Klammern vermieden.

Hier sehen Sie nur die Mitarbeiter, denen fälschlicherweise die Steuerklasse 1 zugeordnet wurde. Alle Spaltenwerte der Spalte *anzahlkinder* sind größer als 0. Außerdem ist der Mitarbeiter Dirk Lempe verheiratet.



Zusammenfassung: Bedingungen mit logischen Operatoren verknüpfen

Filterabfragen können aus mehreren Bedingungen bestehen, die mithilfe der folgenden logischen Operatoren miteinander verknüpft werden können:

- ▶ AND
- ▶ OR
- ▶ AND NOT
- ▶ OR NOT

Die Verknüpfungsoperatoren können auch in Kombination verwendet werden. Es kommt ganz auf die Anforderung der Filterfunktionalität an, die zu realisieren ist.

Eine **AND**-Verknüpfung wird immer vorrangig vor einer **OR**-Verknüpfung ausgewertet. Indem Sie Klammern setzen, können Sie, wie in der Mathematik, den Vorrang ändern.

Die Anwendung der **WHERE**-Klausel haben Sie in etlichen Ausprägungen kennengelernt. Sie sind jetzt in der Lage, mehrere Bedingungen in einer **WHERE**-Klausel mit logischen Verknüpfungsoperatoren auszuwerten. In [Abschnitt 2.4](#) werden Sie erfahren, wie Sie die Anzahl der von einer Abfrage zurückgelieferten Zeilen gemäß einem von Ihnen vorgegebenen Wert eingrenzen können.

2.3.10 Übungen

Übung 1

In der Abteilung Vertrieb soll geprüft werden, ob eine geschlechterspezifische Verteilung gewährleistet ist. Dafür sollen zuerst sämtliche Mitarbeiter ermittelt werden, die männlich sind und der Abteilung Vertrieb angehören. Prüfen Sie hierzu in der Bedingung der **WHERE**-Klausel die Spalte *abteilung* auf Gleichheit mit dem Wert `vertrieb`. Die erste Bedingung verknüpfen Sie mithilfe eines **AND**-Operators mit einer Bedingung, die die Spalte *geschlecht* auf Gleichheit mit dem Wert `'m'` prüft. In der Spaltenauswahlliste sollen die Spalten *name*, *vorname*, *geschlecht*, *abteilung* der Tabelle *mitarbeiter* ausgewählt werden. In der zweiten Abfrage formulieren Sie die Bedingung so, dass die Spalte *geschlecht* auf Gleichheit mit `'w'` geprüft wird.

Übung 2

Die Personalabteilung muss eine statistische Erhebung durchführen und fordert Sie auf, sämtliche Mitarbeiter zu ermitteln, die entweder der Krankenkasse »MH Plus

Bonn« oder der Krankenkasse »IKK gesund plus« angehören. In die Spaltenauswahlliste sind die Spalten *name*, *vorname* und *krankenversicherung* aufzunehmen. In den Bedingungen prüfen Sie die Spalte *krankenversicherung* auf Gleichheit mit den Werten MH Plus Bonn bzw. IKK gesund plus und verknüpfen die Bedingungen mit dem OR-Verknüpfungsoperator.

Übung 3

In einer weiteren Angelegenheit bittet die Personalabteilung Sie, die Mitarbeiter zu ermitteln, die vor dem 01.01.1955 geboren wurden und noch nicht im Ruhestand sind, um ihnen ein attraktives Angebot für eine Vorruhestandsregelung anzubieten. In die Ergebnisliste nehmen Sie hierzu die Spalten *name*, *vorname*, *gebdatum* und *austrittsgrund* auf. Formulieren Sie erstens eine Bedingung, die die Spalte *gebdatum* auf kleiner als '1955-01-01' prüft.

Verknüpfen Sie zweitens die erste Bedingung mit dem logischen Verknüpfungsoperator AND, und prüfen Sie in einer weiteren Bedingung, ob die Spalte *austrittsgrund* nicht definiert ist, also einen NULL-Wert besitzt.

Übung 4

Die Personalabteilung möchte alleinerziehende Mütter mit einen Bonus von 5.000 € unterstützen. Darum bittet die Personalabteilung Sie, alle Mitarbeiterinnen zu ermitteln, die nicht verheiratet sind und Kinder haben. Um diese Anforderung zu erfüllen, formulieren Sie eine SELECT-Abfrage, die in der WHERE-Klausel die Spalten *geschlecht*, *verheiratet* und *anzahlkinder* auf Gleichheit mit dem Wert 'w', auf Gleichheit mit dem Wert 'nein' und auf einen Wert größer als 0 prüft. Verknüpfen Sie die einzelnen Bedingungen mit dem logischen AND-Operator. In die Spaltenauswahlliste nehmen Sie die Spalten *name*, *vorname*, *verheiratet*, *geschlecht*, *anzahlkinder* und den numerischen Wert 5000 auf.

Lösung zu Übung 1

SELECT

name,
vorname,
geschlecht,
abteilung

FROM

mitarbeiter

WHERE

abteilung='Vertrieb'

AND

```
geschlecht='m';  
/* geschlecht='w';*/
```

Listing 2.65 Nach Bedingungen filtern, die mit AND verknüpft sind

Lösung zu Übung 2

SELECT

```
name,  
vorname,  
krankenversicherung
```

FROM

```
mitarbeiter
```

WHERE

```
krankenversicherung='MH Plus Bonn'
```

OR

```
krankenversicherung='IKK gesund plus';
```

Listing 2.66 Nach Bedingungen filtern, die mit OR verknüpft sind

Lösung zu Übung 3

SELECT

```
name,  
vorname,  
gebdatum,  
austrittsgrund
```

FROM

```
mitarbeiter
```

WHERE

```
gebdatum<'1955.01.01' AND austrittsgrund IS NULL;
```

Listing 2.67 Bedingungen mit dem logischen AND-Operator verknüpfen

Lösung zu Übung 4

SELECT

```
name,  
vorname,  
verheiratet,  
geschlecht,  
anzahlkinder,  
5000
```



```

FROM
    mitarbeiter
WHERE
    geschlecht='w'
    AND
    verheiratet='nein'
    AND
    anzahlkinder>0;

```

Listing 2.68 Mehr als zwei Bedingungen in einer WHERE-Klausel miteinander verknüpfen

2.4 Ergebniszeilen einer SELECT-Anweisung einschränken

Manchmal ist es hilfreich, sich nur eine begrenzte Anzahl von Zeilen einer Tabelle von einer Datenbank als Liste zurückliefern zu lassen. Bereits so können Sie sehen, welche Werte die einzelnen Spalten enthalten.

Stellen Sie sich vor, Sie haben Ihren ersten Arbeitstag als SQL-Entwickler in einem Versandhaus und möchten sich mit den Daten vertraut machen, die in den Tabellen Ihres Unternehmens verwaltet werden. Da die Tabellen, die sich im Einsatz befinden, sehr große Datenmengen (z. B. Millionen von Zeilen von Kundendaten) enthalten können, sollten Sie die Anzahl der zurückzuliefernden Zeilen eingrenzen, um die Leistungsfähigkeit der Datenbank nicht allzu sehr zu beanspruchen.

Unterschiede zwischen den Datenbanken

Die MySQL- oder MariaDB-Datenbank und MS SQL Server setzen die Funktionalität leider nicht standardgemäß um.



2.4.1 Ergebniszeilen mit FETCH, LIMIT und TOP eingrenzen

Als Erstes betrachten wir in [Listing 2.69](#), wie Sie die Anzahl der Zeilen, die durch eine SELECT-Anweisung zurückgegeben werden, für eine MySQL- und MariaDB-Datenbank einschränken:

```
SELECT name,vorname FROM mitarbeiter LIMIT 2;
```

Listing 2.69 Mit dieser Syntax schränken Sie die zurückzugebenden Zeilen in der Ergebnisliste einer MySQL- oder MariaDB-Datenbank ein.

Hier gibt es etwas Neues zu entdecken: Hinter dem Tabellennamen *mitarbeiter* finden Sie das Schlüsselwort **LIMIT**. Limitieren heißt eingrenzen, was wir auf die Anzahl

der Zeilen anwenden wollen. Das erreichen wir, indem wir hinter dem Schlüsselwort **LIMIT** die Anzahl der Zeilen angeben, die in der Ergebnisliste enthalten sein sollen. Wenn Sie die Anzahl der Zeilen in der Ergebnisliste auf fünf Zeilen reduzieren möchten, schreiben Sie einfach **LIMIT 5** hinter die Tabellenangabe. In diesem Beispiel begrenzen wir die Anzahl der Zeilen, die von der **SELECT**-Abfrage zurückgegeben werden, auf den Wert 2.

Wenn Sie mit Microsoft SQL Server arbeiten, können Sie leider ebenfalls nicht nach dem SQL-Standard die Anzahl der zurückzuliefernden Zeilen begrenzen. Hier verwenden Sie wie in [Listing 2.70](#) die **TOP n**-Klausel, um die Anzahl der Zeilen z. B. mit **TOP 5** auf fünf zu begrenzen. Die **TOP**-Klausel notieren Sie vor der Spaltenauswahlliste einer **SELECT**-Anweisung. Auch hier begrenzen wir die Anzahl der Zeilen, die durch die **SELECT**-Anweisung zurückgegeben werden, auf den Wert 2:

```
SELECT TOP 2 name,vorname FROM mitarbeiter;
```

Listing 2.70 Mit dieser Syntax schränken Sie die zurückzugebenden Zeilen in einer Ergebnisliste von Microsoft SQL Server ein.

Diejenigen, die mit einer PostgreSQL-Datenbank arbeiten, finden ein SQL-konformes Beispiel zur Begrenzung der Anzahl der zurückzuliefernden Zeilen in [Listing 2.71](#). Hinter der Tabellenbezeichnung steht die **FETCH**-Klausel. Mit **FETCH FIRST 2 ROWS ONLY** teilen Sie der Datenbank mit, dass nur die ersten zwei Zeilen aus der Tabelle *mitarbeiter* abgefragt werden. Hier begrenzen wir die Anzahl der Zeilen wieder auf 2:

```
SELECT name,vorname FROM mitarbeiter FETCH FIRST 2 ROWS ONLY;
```

Listing 2.71 Mit dieser Syntax schränken Sie die zurückzugebenden Zeilen in der Ergebnisliste einer PostgreSQL-Datenbank ein.



Zusammenfassung: Zeilen einer Abfrage in einer Ergebnisliste begrenzen

Der SQL-Standard bietet mit der **FETCH**-Klausel die Möglichkeit, die Zeilen einer Ergebnisliste einer Abfrage zu begrenzen. Nicht alle Datenbanken realisieren die Funktion so, wie sie im SQL-Standard festgelegt wurde.

- ▶ Bei der MySQL- und MariaDB-Datenbank nutzen Sie die **LIMIT**-Klausel, um die Anzahl der Zeilen der Ergebnisliste zu begrenzen.
- ▶ Bei der MS SQL Server-Datenbank nutzen Sie die **TOP**-Klausel, um die Anzahl der Zeilen der Ergebnisliste zu begrenzen.
- ▶ Bei der PostgreSQL-Datenbank nutzen Sie die **FETCH**-Klausel, um die Anzahl der Zeilen der Ergebnisliste zu begrenzen.

2.4.2 Übungen

Übung 1

Mit welchem Schlüsselwort begrenzen Sie – bezogen auf den von Ihnen verwendeten SQL-Dialekt – die Anzahl der Zeilen, die eine `SELECT`-Abfrage zurückliefern soll, auf einen von Ihnen festgelegten Wert?

Übung 2

Fragen Sie in einer `SELECT`-Anweisung alle Spalten der Tabelle *krankenkasse* ab. Formulieren Sie für das von Ihnen verwendete Datenbanksystem eine `SELECT`-Anweisung, in der Sie die Anzahl der Ergebniszeilen auf 10 einschränken.

Lösung zu Übung 1

MySQL und MariaDB: `LIMIT`

PostgreSQL: `FETCH`

MS SQL Server: `TOP`

Lösung zu Übung 2

```
SELECT * FROM krankenkasse LIMIT 10;
```

Listing 2.72 In einer MySQL- und MariaDB-Datenbank Zeilen begrenzen

```
SELECT * FROM krankenkasse FETCH FIRST 10 ROWS ONLY;
```

Listing 2.73 In einer PostgreSQL-Datenbank Zeilen begrenzen

```
SELECT TOP 10 * FROM krankenkasse;
```

Listing 2.74 In einer MS SQL Server-Datenbank Zeilen begrenzen

2.5 Datensätze sortiert abfragen

In diesem Abschnitt beschäftigen wir uns mit `SELECT`-Abfragen, die eine Ergebnistabelle anhand eines von Ihnen festgelegten Kriteriums sortiert zurückgeben. Sie kennen sicherlich einen der bekanntesten Vertreter der aufsteigenden alphabetischen Sortierung: das Telefonbuch.

Die Spalten, die als Sortierkriterien verwendet werden, können Textwerte, Datumswerte oder auch numerische Werte enthalten. Dementsprechend werden folgende unterschiedliche Sortierregeln verwendet:

- ▶ alphabetische Sortierung
- ▶ numerische Sortierung
- ▶ kalendarische Sortierung

Auch müssen Sie beim Thema der Sortierung (engl. *collation*) berücksichtigen, nach welchem Zeichensatz sortiert werden soll. Ein Zeichensatz umfasst alle Zeichen, die von der Datenbank genutzt werden können. In dem Zeichensatz, den wir verwenden, sind etwa die Ziffern 0 bis 9, die Buchstaben a–z und A–Z sowie einige Sonderzeichen enthalten. In Deutschland müssen Sie beispielsweise die Umlaute beachten. Sie werden bei der Sortierung wie folgt aufgelöst:

- ▶ Aus ä wird ae.
- ▶ Aus ö wird oe.
- ▶ Aus ü wird ue.

Es gibt auf der Welt eine Vielzahl von Alphabeten. Denken Sie nur an chinesische Schriftzeichen oder an das arabische Alphabet. Und natürlich gibt es auch eine Vielzahl weiterer Auflistungen von Zeichen, die die Grundlage für einen Sprachschatz bilden. Beachten Sie also, dass in diesen Fällen andere Sortierregeln gelten.

Die Anwendungsfälle für sortierte Abfragen sind vielfältig. Vielleicht führen Sie ein Haushaltsbuch, in dem Sie Ihre einzelnen Positionen, die Sie einkaufen, mit einer Bezeichnung, dem Einkaufsdatum und dem Einkaufspreis aufführen. Hier könnten Sie z. B. die Ausgaben interessieren, die die höchsten Kosten verursacht haben, um zu ermitteln, wo Sie gegebenenfalls Kosten einsparen können.

Ein weiteres Beispiel wäre ein Adressbuch, das in der Regel aufsteigend alphabetisch sortiert unsere Kontakte verwaltet. Wir machen das schon ganz automatisch, indem wir die Reihenfolge des Alphabets (A, a, B, b etc.) anwenden. Auch ein Geschäftsführer eines Unternehmens wird eine Bilanz, in der die Quartals- und Umsatzzahlen sortiert sind, nützlicher finden als eine chaotische Auflistung.

Sie wissen, dass Sie Werte aufsteigend und absteigend sortieren können. Sehen wir uns ein Beispiel an, in dem aufsteigend nach den Werten der Spalte *umsatz* sortiert wurde. Sie sehen in der letzten Zeile sofort, dass das Unternehmen im 3. Quartal am meisten Umsatz gemacht hat. Ebenso ersichtlich ist, dass im 4. Quartal der schwächste Umsatz erzielt wurde.

Quartal	Umsatz
4	130000 EUR
2	140000 EUR

Tabelle 2.51 Die Umsatzzahlen eines Quartals, aufsteigend sortiert

Quartal	Umsatz
1	150000 EUR
3	160000 EUR

Tabelle 2.51 Die Umsatzzahlen eines Quartals, aufsteigend sortiert (Forts.)

Betrachten wir jetzt die gleichen Umsatzzahlen in einer weiteren Tabelle. Sie wurde absteigend sortiert.

Quartal	Umsatz
3	160000 EUR
1	150000 EUR
2	140000 EUR
4	130000 EUR

Tabelle 2.52 Die Umsatzzahlen eines Quartals, absteigend sortiert

Auch hier sehen Sie sofort in der ersten Zeile, dass das Unternehmen im 3. Quartal den höchsten Umsatz verzeichnet hat. Ihnen stellt sich jetzt sicher die Frage, wie Sie eine Datenbank dazu veranlassen können, Ergebnislisten sortiert zurückzuliefern.

In SQL verwenden Sie in Ihren **SELECT**-Abfragen die Klausel **ORDER BY** (engl. für »sortiere nach«), um Ergebnislisten sortiert zurückzuerhalten.

2.5.1 Aufsteigende Sortierung gemäß einer Spaltenangabe

Wenden wir uns der Praxis zu, und probieren wir die aufsteigende Sortierung mit der **ORDER BY**-Klausel in einer **SELECT**-Abfrage aus. Die Verwendung der **ORDER BY**-Klausel ist recht einfach. Fügen Sie sie hinter der Tabellenbezeichnung der **SELECT**-Abfrage an. Gleich im Anschluss müssen Sie nur noch die Spalte oder die Spalten angeben, nach der bzw. denen sortiert werden soll. In [Listing 2.75](#) fragen wir mit einer einfachen **SELECT**-Abfrage eine Liste ab, in der die Spaltenwerte der Spalte *name* als aufsteigendes Sortierkriterium Verwendung finden:

```
SELECT
  name,
  vorname
FROM
  mitarbeiter
```

ORDER BY

name;

Listing 2.75 Eine Ergebnisliste mit einer SELECT-Abfrage und einer ORDER BY-Klausel aufsteigend sortiert bei der Datenbank anfordern

Name	vorname
Engels	Sabine
Friedrichs	Martin
Funke	Doris
Funke	Andreas

Tabelle 2.53 Aufsteigende Sortierung gemäß den Werten der Spalte »name«

Natürlich gibt es auch die Möglichkeit der absteigenden Sortierung gemäß den Spaltenwerten einer festgelegten Spalte. Damit kommen wir zu den beiden Schlüsselwörtern **ASC** (*ascending*, engl. für »aufsteigend«) und **DESC** (*descending*, engl. für »absteigend«). Hierzu betrachten wir noch einmal das Codefragment aus [Listing 2.75](#), das wir verwendet haben, um eine aufsteigend sortierte Liste anzufordern.

In [Listing 2.76](#) sehen Sie drei Versionen einer **ORDER BY**-Klausel. Die erste Version haben Sie ja bereits verwendet, um aufsteigend gemäß den Spaltenwerten der Spalte *name* der Tabelle *mitarbeiter* sortieren zu lassen. Die zweite Version der **ORDER BY**-Klausel ist neu: Hinter der Spaltenangabe *name* finden Sie jetzt das Schlüsselwort **ASC**. Mit dem Schlüsselwort **ASC** teilen Sie der Datenbank explizit mit, dass Sie eine aufsteigende Sortierung wünschen. Geben Sie hinter der Spaltenangabe das Schlüsselwort **ASC** nicht an, so wird standardmäßig von einer aufsteigenden Sortierung ausgegangen.

Das dritte Codefragment zeigt Ihnen, wie Sie mit dem Schlüsselwort **DESC** die Datenbank dazu veranlassen, absteigend zu sortieren.

ORDER BY

name;

ORDER BY

name **ASC**;

ORDER BY

name **DESC**;

Listing 2.76 ORDER BY mit expliziter Angabe für eine aufsteigende bzw. absteigende Sortierung einer Spalte

Die soeben dargestellte Möglichkeit einer absteigenden Sortierung mithilfe der **ORDER BY**-Klausel wenden wir im nächsten Beispiel an. Hierzu erweitern wir die **ORDER BY**-Klausel um das Schlüsselwort **DESC**, um eine absteigende Sortierung zu erreichen:

```
SELECT
    name,
    vorname
FROM
    mitarbeiter
ORDER BY name DESC;
```

Listing 2.77 Eine Spaltenauswahl absteigend sortiert nach der Spalte »name« bei der Datenbank anfordern

name	vorname
Winter	Elise
Winscheidt	Elke
Überall	Georg
Trost	Elvira

Tabelle 2.54 Absteigend alphabetische Sortierung der Spalte »name« durch das Schlüsselwort **DESC** in der **ORDER BY**-Klausel

2.5.2 Auf- und absteigende Sortierung mehrerer Spalten

Wie fordern Sie eine Ergebnisliste an, die nach mehreren Spalten sortiert ist? Um das zu erreichen, fügen Sie einfach eine kommaseparierte Spaltenliste hinter die **ORDER BY**-Klausel ein. Die Abfrage in [Listing 2.78](#) fordert bei der Datenbank eine Ergebnisliste an, die nach den Spalten *name* und *vorname* sortiert wird. Die Spalten, die das Sortierkriterium repräsentieren, sind einfach durch Kommas getrennt angegeben:

```
SELECT
    name,
    vorname
FROM
    mitarbeiter
ORDER BY
    name,
    vorname;
```

Listing 2.78 Eine Sortierung mehrerer Spalten bei der Datenbank anfordern

name	vorname
Blume	Siegmar
Blume	Tanja
Bücher	Herbert
Bücher	Ilse
Dorsten	Klaus
Dorsten	Stephanie
Eisenhof	Frank
Eisenhof	Linda

Tabelle 2.55 Aufsteigend alphabetische Sortierung zweier Spalten

Die zurückgelieferte Liste wurde nach den Werten der Spalten *name* und *vorname* aufsteigend sortiert ausgegeben. Hier wird zunächst nach den Werten der Spalte *name* sortiert. In der Sortierung wird der Name *Blume* also vor *Bücher* ausgegeben. In diesem Fall wird nicht nur nach den Werten der Spalte *name* sortiert, sondern auch nach den Werten der Spalte *vorname*. Es gibt zwei Mitarbeiter mit dem Nachnamen *Blume*: *Siegmar* und *Tanja*. Da wir in der **ORDER BY**-Klausel nach Namen und Vornamen sortieren, erscheinen *Siegmar* und *Tanja* aufsteigend sortiert. Wenn Sie also der **ORDER BY**-Klausel in diesem Fall zwei Spalten gemäß Ihrer gewünschten Reihenfolge übergeben, so werden zuerst die Spaltenwerte der ersten Spalte alphabetisch sortiert. Diese Sortierung der ersten Spalte hat dann natürlich Vorrang in der Auswertung.

2.5.3 Nach numerischen Spaltenwerten sortieren

Bisher haben Sie ausschließlich Sortierkriterien formuliert, in denen Textwerte enthalten sind. Wie sieht es aus, wenn Sie eine **SELECT**-Abfrage formulieren möchten, die die Sortierung einer numerischen Spalte bei der Datenbank anfordert?

In der Tabelle *mitarbeiter* finden Sie einige numerische Spalten, darunter die Spalten *umsatz* und *monatslohn*. Im nächsten Beispiel wollen wir eine absteigende Sortierung der numerischen Spaltenwerte der Spalte *umsatz* erreichen. Mit der **SELECT**-Abfrage aus [Listing 2.79](#) realisieren Sie genau dies. Um die absteigende Sortierung bei der Datenbank anzufordern, geben Sie wie zuvor das Schlüsselwort **DESC** hinter der Spaltengabe *umsatz* an:


```

SELECT
    name,
    vorname,
    umsatz
FROM
    mitarbeiter
ORDER BY
    umsatz
DESC;

```

Listing 2.79 Eine absteigend sortierte Ergebnisliste mit numerischen Werten anfordern

Name	vorname	umsatz
Heinrich	Hans	1200000.80
Bücher	Herbert	1200000.80
Mey	Lisa	1000000.00
Steinmacher	Ilse	700898.03

Tabelle 2.56 Absteigend numerische Sortierung der Spalte »umsatz« durch das Schlüsselwort DESC in der ORDER BY-Klausel

2.5.4 Nach Datumswerten sortieren

Was uns jetzt noch fehlt, ist ein Sortierkriterium, das nach Datumswerten sortiert. Die Tabelle *mitarbeiter* enthält gleich drei Spalten, in denen Datumswerte gespeichert werden:

- ▶ *gebdatum*
- ▶ *eintrittsdatum*
- ▶ *austrittsdatum*

Jetzt formulieren wir eine Abfrage, die bei der Datenbank eine Ergebnistabelle anfordert, die nach Datumswerten sortiert ist. In [Listing 2.80](#) sehen Sie eine **SELECT**-Abfrage mit einer **ORDER BY**-Klausel, die eine aufsteigende Sortierung der Spaltenwerte der Spalte *eintrittsdatum* bei der Datenbank anfordert:

```

SELECT
    name,
    vorname,
    eintrittsdatum
FROM
    mitarbeiter

```

ORDER BY

 eintrittsdatum;

Listing 2.80 Eine ORDER BY-Klausel auf Datumsspalten anwenden

name	vorname	eintrittsdatum
Schmidt	Werner	1970-03-01
Müller	Iris	1970-03-15
Heinrich	Hans	1971-03-30
Serenius	Ingo	1972-04-11

Tabelle 2.57 Aufsteigende kalendarische Sortierung von Datumswerten

2.5.5 Nicht definierte Werte in einer Sortierung beachten

In Abschnitt 2.2.10 habe ich bereits den Wert **NULL** besprochen, der in der Spalte *austrittsdatum* wichtig ist, denn natürlich ist diese nicht immer mit einem Wert versehen. Wie wirkt sich das auf die Sortierung aus? Betrachten Sie hierzu die **SELECT**-Abfrage in Listing 2.81. Sie fordert von der Datenbank eine Ergebnistabelle an, die aufsteigend nach den Spaltenwerten der Spalte *austrittsdatum* sortiert ist.

SELECT

 name,
 vorname,
 austrittsdatum

FROM

 mitarbeiter

ORDER BY

 austrittsdatum;

Listing 2.81 NULL-Werte in einer aufsteigenden Sortierung berücksichtigen

name	vorname	austrittsdatum
Überall	Georg	NULL
Steinmacher	Ilse	NULL
Schmidt	Werner	1975-03-31
Stein	Rolf	1985-01-31

Tabelle 2.58 NULL-Werte stehen bei aufsteigenden Sortierungen in einer MySQL-, MariaDB- und MS SQL Server-Datenbank immer an erster Stelle.

Bei absteigender Sortierreihenfolge würden die **NULL**-Werte bei einer MySQL- oder MariaDB-Datenbank und MS SQL Server-Datenbank zuerst in der Liste auftauchen. Bei einer PostgreSQL-Datenbank werden die **NULL**-Werte des Sortierkriteriums zuletzt in der Liste angezeigt. Wenn Sie mit einer **ORDER BY**-Klausel die Datenbank nach einer Spalte sortieren lassen, die **NULL**-Werte enthält, findet bei den Spaltenwerten, die gleich **NULL** sind, natürlich keine Sortierung statt. Sie haben ja bereits gelernt, dass ein **NULL**-Wert kein definierter Wert ist.

2.5.6 ORDER BY mit einer WHERE-Klausel verwenden

Natürlich lassen sich Einträge auch durch Bedingungen filtern, bevor sie sortiert werden. Wenn Sie nur die Mitarbeiter der Abteilung Controlling in einer Ergebnistabelle darstellen und diese Zeilen nach den Namen der Mitarbeiter sortieren möchten, brauchen Sie genau diese beiden Arbeitsschritte.

Die **ORDER BY**-Klausel haben Sie bisher einfach hinter der Tabellenbezeichnung angegeben. Die **WHERE**-Klausel haben Sie ebenfalls direkt hinter der Tabellenbezeichnung notiert. Wenn wir die Sortierung und die Filterfunktion getrennt anwenden würden, sähe das Ganze wie in [Listing 2.82](#) und [Listing 2.83](#) aus:

```
SELECT
    name,
    abteilung
FROM
    mitarbeiter
ORDER BY
    name;
```

Listing 2.82 Eine sortierte Ergebnistabelle anfordern

```
SELECT
    name,
    abteilung
FROM
    mitarbeiter
WHERE
    abteilung='Controlling';
```

Listing 2.83 Eine gefilterte Ergebnistabelle anfordern

Natürlich geht das einfacher. Wir wollen die **WHERE**-Klausel und die **ORDER BY**-Klausel jetzt gleichzeitig in einer **SELECT**-Abfrage verwenden. Als Erstes müssen Sie die **WHERE**-Klausel angeben, denn erst dann, wenn die Datenbank die gefilterten Datensätze er-

mittelt hat, soll die soeben gefilterte Liste mit einer **ORDER BY**-Klausel sortiert und Ihnen als Ergebnistabelle zurückgeliefert werden. Die Lösung sehen Sie in [Listing 2.84](#):

```
SELECT
    name,
    abteilung
FROM
    mitarbeiter
WHERE
    abteilung='Controlling'
ORDER BY
    name;
```

Listing 2.84 Eine SELECT-Abfrage gleichzeitig mit einer WHERE-Klausel und ORDER BY-Klausel verwenden

name	abteilung
Blume	Controlling
Engels	Controlling
Funke	Controlling
Gelz	Controlling

Tabelle 2.59 Alle Mitarbeiter, die der Abteilung Controlling angehören (WHERE), sortiert (ORDER BY) nach Namen

Umgekehrt geht es nicht, da eine Abfrage wie in [Listing 2.85](#) nicht sinnvoll wäre. Die Ausführung führt daher zu einem Fehler. Achten Sie also stets darauf, dass Sie in einer **SELECT**-Abfrage die **WHERE**-Klausel zuerst anwenden. Erst dann folgt die **ORDER BY**-Klausel.

```
SELECT
    name,
    abteilung
FROM
    mitarbeiter
ORDER BY
    name
WHERE
    abteilung='Controlling';
```

Listing 2.85 Falsche Reihenfolge bei der Verwendung einer WHERE- und einer ORDER BY-Klausel!



Zusammenfassung: Ergebnislisten bei der Datenbank sortiert anfordern

SQL bietet für **SELECT**-Anweisungen die **ORDER BY**-Klausel an, mit der Sie eine sortierte Liste bei der Datenbank anfordern. Sortiert wird nach den angegebenen Sortierkriterien. Mit den Schlüsselwörtern **DESC** und **ASC** fordern Sie eine absteigende oder eine aufsteigende Sortierung an. Geben Sie keines dieser Schlüsselwörter in der **ORDER BY**-Klausel an, so wird standardmäßig aufsteigend sortiert.

Zu beachten ist, dass bei einer aufsteigenden Sortierung nicht definierte Werte (**NULL**-Werte) bei einer MySQL-, MariaDB- und MS SQL Server-Datenbank zuerst gelistet werden. Bei einer absteigenden Sortierung werden sie an das Ende der Ergebnisliste angefügt. Die Sortierung der PostgreSQL-Datenbank verhält sich umgekehrt.

Vor dem Sortieren filtert eine **WHERE**-Klausel in einer **SELECT**-Anweisung die gewünschte Tabelle. Dabei muss die **ORDER BY**-Klausel stets nach der **WHERE**-Klausel notiert werden.

2.5.7 Übungen

Übung 1

Formulieren Sie eine **SELECT**-Abfrage, in der Sie die Spalten *name*, *vorname* und *abteilung* der Tabelle *mitarbeiter* auswählen. Als Sortierkriterium geben Sie die Spalte *abteilung* an. Die Mitarbeiter sollen aufsteigend sortiert nach den Abteilungen, denen sie angehören, in der Ergebnistabelle aufgeführt werden.

Übung 2

Ändern Sie die von Ihnen in Übung 1 erstellte Abfrage, und fordern Sie stattdessen bei der Datenbank eine Liste an, die absteigend nach den Spaltenwerten der Spalte *abteilung* der Tabelle *mitarbeiter* sortiert ist.

Übung 3

Formulieren Sie eine Abfrage, in der Sie die Inhalte der Spalten *name*, *vorname*, *abteilung* und die Spalte *bonus* der Tabelle *mitarbeiter* anfragen. Die Ergebnistabelle soll aufsteigend nach den Werten der Spalten *abteilung* und *bonus* sortiert sein.

Übung 4

Jetzt sollen die Mitarbeiter, die das Unternehmen verlassen haben, absteigend sortiert nach den Werten der Spalte *austrittsdatum* der Tabelle *mitarbeiter* in einer Ergebnistabelle ausgegeben werden. In die Spaltenauswahlliste nehmen Sie hierzu die Spalten *name*, *vorname* und *austrittsdatum* auf. Beachten Sie gegebenenfalls die Unterschiede bei der Sortierung der einzelnen Datenbanken.

Übung 5

Erstellen Sie eine Übersicht über die Mitarbeiter, die sich bereits im Ruhestand befinden. Formulieren Sie ein Filterkriterium, das die Spaltenwerte der Spalte *austrittsgrund* auf Gleichheit mit dem Wert *ruhestand* vergleicht. Die Zeilen sollen in der Ergebnistabelle nach den Werten der Spalte *austrittsdatum* aufsteigend sortiert sein.

Lösung zu Übung 1

```
SELECT
    name,
    vorname,
    abteilung
FROM
    mitarbeiter
ORDER BY
    abteilung;
```

Listing 2.86 SELECT-Abfrage mit aufsteigender Sortierung der Spalte »abteilung«

Lösung zu Übung 2

```
SELECT
    name,
    vorname,
    abteilung
FROM
    mitarbeiter
ORDER BY
    abteilung
DESC;
```

Listing 2.87 SELECT-Abfrage mit absteigender Sortierung der Spalte »abteilung«

Lösung zu Übung 3

```
SELECT
    name,
    vorname,
    abteilung,
    bonus
FROM
    mitarbeiter
```

ORDER BY

```
abteilung,bonus;
```

Listing 2.88 SELECT-Abfrage mit einem aus zwei Spalten zusammengesetzten Sortierkriterium

Lösung zu Übung 4**SELECT**

```
name,  
vorname,  
austrittsdatum
```

FROM

```
mitarbeiter
```

ORDER BY

```
austrittsdatum
```

DESC;

Listing 2.89 SELECT-Abfrage, die bei der Datenbank eine Liste anfordert, die nach einer Datumsangabe absteigend sortiert ist

Die MySQL-, MariaDB- und MS SQL Server-Datenbank fügen die Zeilen, in denen das Sortierkriterium gleich NULL ist, bei einer absteigenden Sortierung an das Ende der Ergebnistabelle. Bei PostgreSQL werden die Zeilen, in denen das Sortierkriterium NULL ist, bei einer absteigenden Sortierung zuerst in der Liste angezeigt.

Lösung zu Übung 5**SELECT**

```
name,  
vorname,  
austrittsgrund,  
austrittsdatum
```

FROM

```
mitarbeiter
```

WHERE

```
austrittsgrund='ruhestand'
```

ORDER BY

```
austrittsdatum;
```

Listing 2.90 Eine WHERE- und eine ORDER BY-Klausel gleichzeitig in einer SELECT-Abfrage verwenden

2.6 Konstanten in die Spaltenauswahlliste aufnehmen

Sie haben bestimmt bereits die eine oder andere Konstante kennengelernt. Beispielsweise beginnt Wasser bei einer Temperatur von 100 °C zu kochen. Wenn Sie es kochen sehen, wissen Sie, dass Sie Ihr Frühstücksei in das Wasser geben können. Es handelt sich bei dieser Temperaturangabe um eine konstante Größe. Eine Konstante ist nichts anderes als ein Wert, der unverändert bleibt. In der Sprache SQL und in anderen Programmiersprachen wird ein solcher Wert, der sich nicht ändert, ebenfalls als *Konstante* bezeichnet. Viele Dinge können bei SQL eine Konstante sein:

- ▶ ein Datum, z. B. ein Geburtsdatum wie '01.03.1960'
- ▶ eine ganze Zahl, z. B. eine konstante Anzahl von 20 in einer Verpackung
- ▶ ein Text, z. B. 'Herzlichen Glückwunsch zum Geburtstag'
- ▶ eine Kommazahl, z. B. 20.99 % Rabatt auf alles für Geburtstagskinder

Datums- und Textwerte setzen Sie, wie Sie bereits wissen, in einfache Hochkommas. Das gilt auch für Konstanten, die Datums- und Textwerte enthalten.

Sie haben die Schreibweise bereits bei der Verwendung von Textsuchkriterien in einer **WHERE**-Klausel kennengelernt.

Zahlen notieren Sie ohne Hochkommas. Zahlen haben Sie bereits in einer **WHERE**-Klausel verwendet. Auch Kommazahlen wie 3,5 werden einfach so in die Spaltenauswahlliste übernommen. Dabei müssen Sie aber beachten, dass Kommazahlen in SQL eben nicht mit dem in Deutschland üblichen Kommazeichen angegeben werden. Stattdessen wird ein einfacher Punkt anstelle des Kommas benutzt. Aus 3,5 wird also 3.5.

Passen Sie unbedingt auf, wenn Sie doch Zahlen in einfache Anführungszeichen setzen, denn dadurch ändert sich ihre Bedeutung für die Datenbank grundlegend. Sie interpretiert diesen Eintrag nun als Textwert, was große Auswirkungen auf Sortier- und Rechenoperationen hat. Kurz gesagt: '3.5' stimmt nicht mit 3.5 überein. In [Kapitel 4, »Tabellen anlegen \(CREATE TABLE\)«](#), erfahren Sie mehr über die sogenannten Datentypen.



Zusammenfassung: Konstanten

Es gibt unterschiedliche Arten von Konstanten bzw. Werten, die in der Spaltenauswahlliste und an anderer Stelle verwendet werden können. Zu ihnen zählen Textwerte, Datumswerte und numerische Werte.

- ▶ Text- und Datumswerte werden in SQL immer in Hochkommas gesetzt.
- ▶ Dezimalzahlen werden mit einem Punkt anstelle eines Kommas notiert.

2.6.1 Abfrage eines konstanten Textes

Probieren wir das Arbeiten mit Konstanten gleich aus, indem wir eine `SELECT`-Abfrage wie in [Listing 2.91](#) formulieren, in der lediglich eine Glückwunschkformel in der Spaltenauswahlliste hinterlegt ist:

```
SELECT 'Herzlichen Glückwunsch zum Geburtstag';
```

Listing 2.91 Konstante verwenden

Herzlichen Glückwunsch zum Geburtstag
Herzlichen Glückwunsch zum Geburtstag

Tabelle 2.60 Ergebnistabelle für eine Konstante in der Spaltenauswahlliste

Unterschiede zwischen den Datenbanken

Verschiedene SQL-Werkzeuge reagieren unterschiedlich auf diese Eingabe. Die MySQL Workbench und der HeidiSQL-Editor verwenden den Textwert sogar als Überschrift für die Spaltenbezeichnung. Wenn Sie andere Programme einsetzen, sieht dies jedoch unterschiedlich aus. Der *Microsoft SQL Server Editor* würde hier keine Spaltennamen ausgeben. Der SQL-Editor des *pgAdmin 4-Tools* würde uns eine Spaltenbezeichnung *column unknown* (engl. für »Spalte unbekannt«) bieten. Die Spaltenbezeichnung kann also je nach verwendetem Datenbanksystem unterschiedlich sein.



2.6.2 Konstanten und Spalten einer Tabelle gleichzeitig abfragen

Stellen Sie sich vor, Sie möchten den Mitarbeitern einen guten Morgen wünschen. In SQL könnten Sie dies mit der Abfrage aus [Listing 2.92](#) einfach realisieren, indem Sie eine konstante Zeichenkette, die Guten Morgen lautet, vor der Spalte *name* in die Spaltenauswahlliste aufnehmen.

```
SELECT
    'Guten Morgen ',
    name
FROM
    mitarbeiter;
```

Listing 2.92 Literale und die Spalte »name« der Tabelle »mitarbeiter« abfragen

Guten Morgen	name
Guten Morgen	Müller
Guten Morgen	Schneider
Guten Morgen	Klein
Guten Morgen	Lang

Tabelle 2.61 Jeder Mitarbeiter wird konstant mit »Guten Morgen« begrüßt.



Zusammenfassung: Konstanten in der Spaltenauswahlliste verwenden

In eine Spaltenauswahlliste einer **SELECT**-Anweisung können Sie beliebige Konstanten aufnehmen, um sie sich in der Ergebnisliste anzeigen zu lassen. Dabei kann es sich um einen Text, eine Zahl oder auch um ein Datum handeln.

2.6.3 Übungen

Übung 1

Formulieren Sie eine **SELECT**-Abfrage, in der Sie ausschließlich folgende Konstanten ausgeben: 'Hallo', gefolgt von ' Du'. Diese Abfrage bezieht sich auf keine Tabelle.

Übung 2

Formulieren Sie eine **SELECT**-Abfrage, in der Sie die Spalten *name* und *vorname*, gefolgt von der Textkonstanten ' hat am ', der Spalte *gebdatum* und der Textkonstanten ' Geburtstag' abfragen. Die Spalten sollen aus der Tabelle *mitarbeiter* stammen.

Übung 3

Aufgrund ihrer besonderen Leistungen erhalten sämtliche Mitarbeiter jeweils 120 Leistungspunkte. Formulieren Sie für die Tabelle *mitarbeiter* eine Abfrage, die den Namen, den Vornamen, den Textwert ' erhält ' und die 120 Bonuspunkte als konstante Zahl in der Ergebnisliste ausgibt.

Lösung zu Übung 1

```
SELECT 'Hallo', ' Du';
```

Listing 2.93 Konstanten (Literele) in einer **SELECT**-Abfrage

Lösung zu Übung 2

SELECT

```
name,
vorname,
' hat am ',
gebdatum,
' Geburtstag'
```

FROM

```
mitarbeiter;
```

Listing 2.94 Konstanten in Verbindung mit Spaltenwerten abfragen

Lösung zu Übung 3

SELECT

```
name,
vorname,
' erhält ',
120,
' Bonuspunkte'
```

FROM

```
mitarbeiter;
```

Listing 2.95 Lösung zur Abfrage von Konstanten- und Spaltenwerten

2.7 Spalten einen Alias zuordnen

Jetzt zeige ich Ihnen, wie Sie den Spalten und Konstanten in einer Spaltenauswahl-liste einer **SELECT**-Abfrage einen Alias zuordnen. Ein sogenannter *Alias* (engl. für »Deckname«) wird verwendet, um Spalten oder Literalen einen sprechenden Namen zuzuordnen. Gehen wir das einfach mal gedanklich anhand der drei Spalten *name*, *vorname* und *gebdatum* der Tabelle *mitarbeiter* durch. Für die Spaltenbezeichnung *name* könnten Sie beispielsweise die Bezeichnung *namemitarbeiter* vergeben, dementsprechend der Spalte *vorname* die Bezeichnung *vornamemitarbeiter* zuordnen und die Spalte *gebdatum* *geburtsdatummitarbeiter* nennen.

Bei Konstanten sind Aliasse sehr nützlich, da sie ja über keinen definierten Spaltennamen verfügen und auf diese Weise eindeutig bezeichnet werden können. Der Einsatz von Aliassen kann auch sinnvoll sein, um Spaltenbezeichnungen in der Ergebnisliste abzukürzen. Und zu guter Letzt kann so auch ein sprechenderer Name für eine Spalte einer Tabelle vergeben werden, um den Inhalt der Spaltenwerte besser zu beschreiben.

2.7.1 Spalten in einer Abfrage mit einem Alias versehen

Probieren wir es einfach aus. In [Listing 2.96](#) sehen Sie eine Spaltenauswahlliste, in der sämtlichen Spalten, die abgefragt werden, Spaltenaliasse zugeordnet sind. Ein Spaltenalias wird hinter der Spalte notiert, auf die er Anwendung finden soll. Dazwischen geben Sie noch das Schlüsselwort **AS** an. Sie könnten also z. B. eine Spalte mit der Bezeichnung *name* in eine Spalte mit der Bezeichnung *mitarbeitername* umbenennen. Das Schlüsselwort **AS** können Sie optional verwenden. Das heißt, die Angabe ist nicht unbedingt erforderlich.

```
SELECT
  name AS mitarbeitername,
  vorname AS mitarbeitervorname,
  gebdatum AS mitarbeitergeburtsdatum
FROM
  mitarbeiter;
```

Listing 2.96 Alle Spalten einer SELECT-Abfrage mit einem Alias versehen

mitarbeitername	mitarbeitervorname	mitarbeitergeburtsdatum
Müller	Ralf	1970-12-20
Schneider	Petra	1965-03-07
Klein	Thomas	1970-12-20
Lang	Ute	1985-07-19

Tabelle 2.62 Alternative Bezeichnungen aller Spalten mittels »AS«

2.7.2 Ausgewählten Spalten einer Abfrage einen Alias zuordnen

Sie müssen nicht unbedingt sämtlichen Spalten oder Werten in der Spaltenauswahlliste einen Alias zuordnen. Sie könnten auch die Spalten *name* und *vorname* ohne Alias in der Spaltenauswahlliste angeben und lediglich die Spalte *gebdatum* mit einem Alias versehen. [Listing 2.97](#) zeigt eine Anwendung, in der nur einer ausgewählten Spalte ein Alias zugeordnet wird:

```
SELECT
  name,
  vorname,
  gebdatum AS mitarbeitergebdatum
FROM
  mitarbeiter;
```

Listing 2.97 Nur bestimmten Spalten in einer Abfrage einen Alias zuordnen

name	vorname	mitarbeitergebdatum
Müller	Ralf	1970-12-20
Schneider	Petra	1965-03-07
Klein	Thomas	1970-12-20
Lang	Ute	1985-07-19

Tabelle 2.63 Eine explizite Aliaszuweisung an eine Spalte

2.7.3 Spalten und Konstanten einen Alias zuordnen

In [Listing 2.98](#) sehen Sie nun einerseits Spalten, denen ein Alias zugeordnet wurde, und andererseits Konstanten, die einen Text repräsentieren, denen ein Alias zugeordnet wurde. Bei Konstanten – egal, ob es sich um Text-, Zahlen- oder Datumskonstanten handelt – ist die Verwendung von Spaltenaliasen besonders sinnvoll, da ihnen keine Spaltenüberschriften zugeordnet sind.

SELECT

```
name AS Name,
vorname AS Vorname,
' hat am ' AS hatam,
gebdatum AS Geburtstag,
' Geburtstag!' AS Happybirthday
```

FROM

```
mitarbeiter;
```

Listing 2.98 Spalten und Konstanten mit Aliassen versehen

In [Tabelle 2.64](#) werden Ihnen jetzt auch für die Konstanten in der Spaltenauswahlliste die Spaltenaliasse *hatam* und *Happybirthday* angezeigt, die Sie ihnen zugeordnet haben.

Name	Vorname	hatam	Geburtstag	Happybirthday
Müller	Ralf	hat am	1970-12-20	Geburtstag!
Schneider	Petra	hat am	1965-03-07	Geburtstag!
Klein	Thomas	hat am	1970-12-20	Geburtstag!
Lang	Ute	hat am	1985-07-19	Geburtstag!

Tabelle 2.64 Konstanten und Spalten besitzen nun Aliasbezeichnungen.



Zusammenfassung: Verwendung von Aliassen in der Spaltenauswahlliste

Den einzelnen Spalten einer Spaltenauswahlliste können Sie alternative Bezeichner zuordnen, die als *Alias* bezeichnet werden. Das ist dann hilfreich, wenn Sie die Spalten mit lesbareren Spaltenüberschriften ausstatten wollen oder gerne individuelle Abkürzungen für die Spaltenüberschriften verwenden möchten. Bei Text-, Zahlen- und Datumswerten können Sie Spaltenaliasse verwenden, um ihnen eine feste Spaltenbezeichnung zuzuordnen.



Zusammenfassung: Spalten einen Alias zuordnen

Spalten oder Konstanten aus einer Spaltenauswahlliste einer **SELECT**-Abfrage können mit einem Alias versehen werden, der in der Ergebnistabelle anstelle des Spaltennamens ausgegeben wird. Der zu vergebende Alias wird in der Spaltenauswahlliste direkt hinter der jeweiligen Spalte oder Konstante notiert.

```
SELECT spalte1 spalte1_alias, spalte2 spalte2_alias  
FROM tabelle1;
```

Optional können Sie auch das Schlüsselwort **AS** verwenden, um Spaltenaliasse deutlicher zu kennzeichnen:

```
SELECT spalte1 AS spalte1_alias, spalte2 AS spalte2_alias  
FROM tabelle1;
```

Nachdem Sie gelernt haben, wie Sie Konstanten und Spaltenbezeichnungen in einer Ergebnistabelle mit alternativen Bezeichnern versehen, werde ich Ihnen im folgenden [Abschnitt 2.8](#) zeigen, wie Sie Ergebniszeilen einer **SELECT**-Anweisung eindeutig ermitteln.

2.7.4 Übungen

Übung 1

Formulieren Sie eine **SELECT**-Abfrage, die die Spalten *name*, *vorname*, *strasse*, *hausnummer*, *plz*, *ort* aus der Tabelle *mitarbeiter* abfragt und den jeweiligen Spalten folgende Spaltenaliasse zuordnet:

name → Alias: *Name*

vorname → Alias: *Vorname*

strasse → Alias: *Strasse*

hausnummer → Alias: *Hausnummer*

plz → Alias: *Postleitzahl*

ort → Alias: *Ort*

Übung 2

Formulieren Sie eine **SELECT**-Abfrage, in der Sie mithilfe einer Textkonstanten sämtliche Mitarbeiter aus der Tabelle *mitarbeiter* willkommen heißen. Ordnen Sie diesem Textwert den Alias *Begrueßung* zu. In der Spaltenauswahlliste sollen auch die Spalten *name* und *vorname* enthalten sein.

Lösung zu Übung 1

SELECT

```
name AS Name,
vorname AS Vorname,
strasse AS Strasse,
hausnummer AS Hausnummer,
plz AS Postleitzahl,
ort AS Ort
```

FROM

```
mitarbeiter;
```

Listing 2.99 Den Spalten in einer **SELECT**-Abfrage Ersatznamen zuordnen

Lösung zu Übung 2

SELECT

```
'Herzlich willkommen ' as Begrueßung,
name,
vorname
```

FROM

```
mitarbeiter;
```

Listing 2.100 Einem Zeichenketten-Literal in einer **SELECT**-Abfrage einen Spaltenalias zuordnen

2.8 Gleiche Ergebniszeilen ausschließen (DISTINCT)

Wenn Sie eine Abfrage für eine Tabelle formulieren, kann es je nach Spaltenauswahl zu gleichen Ergebniszeilen kommen. Sie möchten die Zeilen jedoch eindeutig ermitteln. Betrachten Sie als Erstes die Tabelle *mitarbeitergleich*, die 6 Zeilen enthält. Jeweils 3 Zeilen sind identisch.

name	vorname
Müller	Ralf
Müller	Ralf
Müller	Ralf
Klein	Thomas
Klein	Thomas
Klein	Thomas

Tabelle 2.65 Die Tabelle »mitarbeitergleich« enthält nicht eindeutige Zeilen.

Eine **SELECT**-Abfrage wie in [Listing 2.101](#) würde also zu ebendieser Ergebnistabelle führen.

```
SELECT name,vorname FROM mitarbeitergleich;
```

Listing 2.101 Zeilen mit gleichem Inhalt abfragen

Zeilen sind dann gleich, wenn alle Spaltenwerte einer Zeile mit allen Spaltenwerten einer oder mehrerer Zeilen exakt übereinstimmen. Je nach Auswertung kann dieses Abfrageergebnis durchaus erwünscht sein. Wenn jedoch eine eindeutige Ergebnistabelle gefordert ist, ist es wünschenswert, mit einer **SELECT**-Anweisung eine Abfrage formulieren zu können, bei der ausschließlich eindeutige Zeilen als Ergebnistabelle zurückgegeben werden. »Eindeutig« bedeutet, dass die Ergebnistabelle nur Zeilen enthält, die in allen Spaltenwerten nicht mehrfach vorkommen.

SQL sieht für diese Anforderung die **DISTINCT**-Klausel vor. Die **DISTINCT**-Klausel notieren Sie vor der Spaltenauswahlliste oder dem Asterisk-Operator. In [Listing 2.102](#) sehen Sie eine **SELECT**-Abfrage, in der eine **DISTINCT**-Klausel verwendet wird. Das Schlüsselwort **DISTINCT** wurde hier vor der Auswahl der Spalten *name* und *vorname* notiert.

```
SELECT DISTINCT name,vorname FROM mitarbeitergleich;
```

Listing 2.102 Zeilen eindeutig abfragen

name	vorname
Müller	Ralf

Tabelle 2.66 Die Mitarbeiter Ralf Müller und Thomas Klein werden jetzt eindeutig ausgegeben.

name	vorname
Klein	Thomas

Tabelle 2.66 Die Mitarbeiter Ralf Müller und Thomas Klein werden jetzt eindeutig ausgegeben. (Forts.)

In der Tabelle *mitarbeitergleich* sind die Spalten *name* und *vorname* enthalten. Wenn Sie sich Schreibarbeit sparen möchten, können Sie natürlich auch den Asterisk-Spaltenauswahloperator verwenden, den Sie in [Abschnitt 2.1.4](#) kennengelernt haben. Die Abfrage aus [Listing 2.103](#) führt also exakt zu dem gleichen Ergebnis wie die Abfrage aus [Listing 2.102](#). Die **DISTINCT**-Klausel wurde hier vor dem Asterisk notiert, der stellvertretend sämtliche Spalten der Tabelle *mitarbeitergleich* auswählt.

SELECT DISTINCT * FROM mitarbeitergleich;

Listing 2.103 Den Asterisk-Operator mit der **DISTINCT**-Klausel verwenden

Zusammenfassung: Zeilen eindeutig abfragen

Wenn Sie eine eindeutige Ergebnistabelle benötigen, können Sie die **DISTINCT**-Klausel verwenden, um gleiche Ergebniszeilen eindeutig abzufragen. Die **DISTINCT**-Klausel wird in einer **SELECT**-Anweisung vor der Spaltenauswahlliste oder dem Asterisk-Operator notiert.



2.8.1 Übungen

Übung 1

[Tabelle 2.67](#) (*produktgleich*) enthält zwei verschiedene Produkte, die jeweils in drei Zeilen enthalten sind. Formulieren Sie eine **SELECT**-Anweisung, mit der Sie die Spalten *produkt*, *beschreibung* und *marktbewertung* eindeutig abfragen.

produkt	beschreibung	marktbewertung
Schraube 1	3×20	1
Schraube 1	3×20	1
Schraube 1	3×20	2
Schraube 2	5×30	2
Schraube 2	5×30	2

Tabelle 2.67 Mehrfach vorkommende Zeilen in der Tabelle »produktgleich«

produkt	beschreibung	marktbewertung
Schraube 2	5×30	2

Tabelle 2.67 Mehrfach vorkommende Zeilen in der Tabelle »produktgleich« (Forts.)

Übung 2

Tabelle 2.68 zeigt Ihnen das Ergebnis der Abfrage aus Übung 1. Aus welchem Grund werden Ihnen zwei Ergebniszeilen für das Produkt Schraube 1 zurückgegeben?

produkt	beschreibung	marktbewertung
Schraube 1	3×20	1
Schraube 1	3×20	2
Schraube 2	5×30	2

Tabelle 2.68 Ergebniszeilen, die nicht exakt identisch sind

Lösung zu Übung 1

```
SELECT DISTINCT produkt,beschreibung,marktbewertung
FROM produktgleich;
```

Listing 2.104 Zeilen eindeutig mit der DISTINCT-Klausel abfragen

Lösung zu Übung 2

Die Zeilen für das Produkt Schraube 1 sind nicht exakt identisch. Die Spaltenwerte der Spalte *marktbewertung* sind verschieden. Also sind die beiden Zeilen für das Produkt Schraube 1 in der Gesamtheit der Spaltenwerte nicht gleich.

Kapitel 3

Zeilen einfügen (INSERT), ändern (UPDATE) und löschen (DELETE, TRUNCATE)

Bisher haben wir uns mit der Abfrage (SELECT) von Tabellen befasst. Jetzt ist es an der Zeit, uns damit zu beschäftigen, wie Sie Datensätze in Tabellen einfügen (INSERT), ändern (UPDATE) und löschen (DELETE) können.

Bislang haben Sie Tabellen mit einer **SELECT**-Anweisung abgefragt. Jetzt beginnen wir in Abschnitt 3.1 mit einer **INSERT**-Anweisung, mit der Sie neue Zeilen in Tabellen einfügen. In Abschnitt 3.2 erfahren Sie, wie Sie Zeilen einer Tabelle mit der **UPDATE**-Anweisung ändern. Zu guter Letzt zeige ich Ihnen in Abschnitt 3.3 und Abschnitt 3.4, wie Sie Zeilen einer Tabelle mit der **DELETE**- bzw. **TRUNCATE**-Anweisung löschen.

In produktiven Datenbanken müssen Sie die Anweisungen **UPDATE**, **DELETE** und **TRUNCATE** mit Bedacht anwenden und sehr vorsichtig sein, da sie die Datensätze dauerhaft verändern. Die Anweisungen können bei fehlerhafter Anwendung großen Schaden anrichten.

Zum Glück arbeiten wir hier ja in unserer Übungsdatenbank, wo es nicht so schlimm ist, wenn einige Daten überschrieben oder gelöscht werden.

3.1 Zeilen mit einer INSERT-Anweisung einfügen

Bestimmt haben Sie sich bisher schon häufiger die Frage gestellt, wie die Datensätze, mit denen wir arbeiten, in die Tabellen kommen. SQL sieht dafür die einfache Einfügeanweisung **INSERT** vor.

Es liegt daher nahe, dass Einfügeoperationen sehr häufig zum Einsatz kommen. Wenn etwa ein Mitarbeiter in einem Unternehmen eingestellt wird, gibt es eine Menge neuer Informationen, die in einer oder mehreren Tabellen abgespeichert werden müssen. Wir nutzen die Tabelle *qualifikationen*, um die **INSERT**-Anweisung kennenzulernen. Die Tabelle *qualifikationen* finden Sie in der Übungsdatenbank. Sie ist

sehr einfach strukturiert und eignet sich hervorragend, um Sie mit der Einfügeanweisung **INSERT** vertraut zu machen. Sie ist leer und sieht wie [Tabelle 3.1](#) aus.

qid	bezeichnung	kuerzel	kategorie

Tabelle 3.1 Die Tabelle »qualifikationen«

Die erste Spalte mit der Bezeichnung *qid* (Qualifikations-ID) stellt einen sogenannten *Primärschlüssel* dar. Mit Primärschlüsseln werden wir uns in [Abschnitt 4.4](#) noch ausführlicher befassen. Um Datensätze in die Tabelle *qualifikationen* einzufügen, ist es jedoch erforderlich, dass Sie ein paar Eigenschaften eines Primärschlüssels kennen. Ich werde also ein paar nützliche Informationen vorziehen.

Ein Primärschlüssel identifiziert eine Zeile einer Tabelle eindeutig. Es ist unbedingt empfehlenswert, jede Tabelle mit einem Primärschlüssel zu versehen, um Zeilen in einer Tabelle eindeutig ermitteln zu können. Sie haben es hier mit einem numerischen Primärschlüssel zu tun, den Sie einfach hochzählen können, wenn Sie neue Zeilen in die Tabelle *qualifikationen* einfügen.

Ich muss Ihnen noch eine Zusatzinformation geben: Wenn Sie neue Zeilen in eine Tabelle einfügen, kann es passieren, dass Sie eine Fehlermeldung erhalten, da Sie die maximal zulässige Zeichenanzahl überschreiten. Hier sind für die Spalten *bezeichnung*, *kuerzel* und *kategorie* jeweils 500, 4 und 200 Zeichen erlaubt.

Für Spalten, die Text oder Zeichenketten speichern, können Sie bei der Tabellenerstellung definieren, wie viele Zeichen zur Verfügung stehen. Warum dies sinnvoll ist und wie es genau funktioniert, werde ich in [Abschnitt 4.1](#) erklären.

Fügen wir also Datensätze in die Tabelle *qualifikationen* ein. Diese Tabelle verfügt über Spalten, die Texte oder ganze Zahlen enthalten können.

3.1.1 Spaltenwerte mit expliziter Spaltenangabe einfügen

In [Listing 3.1](#) sehen Sie die erste Einfügeoperation mit einer **INSERT**-Anweisung:

```
INSERT INTO qualifikationen (qid,bezeichnung,kuerzel,kategorie)
VALUES (1,'Word','msw','Office');
```

Listing 3.1 Datensätze in eine Tabelle einfügen

Diese Anweisung beginnt mit dem Schlüsselwort **INSERT**. So teilen Sie der Datenbank mit, dass Sie eine Zeile in eine Tabelle einfügen. Es folgt das Schlüsselwort **INTO**. Mit

dem Schlüsselwort **INTO** und der Tabellenbezeichnung geben Sie an, in welche Tabelle Sie eine Zeile einfügen werden. Darauf folgt die Angabe der Spalten, in die Sie Werte eintragen werden. Die Spalten werden einfach in einer Auflistung angegeben und mit Kommas voneinander getrennt. Die komplette Liste der Spaltenbezeichnungen wird mit einem einfachen Klammerpaar () umschlossen.

Was jetzt noch fehlt, sind die Werte zu den Spalten; Sie geben sie hinter der Liste der Spalten an, indem Sie das Schlüsselwort **VALUES** verwenden. Hinter dem Schlüsselwort **VALUES** folgt wieder eine Liste, die mit Kommas unterteilt und von einem Klammerpaar umschlossen ist. Diesmal stehen in dieser Liste die Werte, die Sie den Spalten in der Spaltenliste zuordnen wollen. Die Spaltenwerte werden gemäß der Reihenfolge der Spaltenliste, die Sie hinter der Tabellenbezeichnung notiert haben, in die Werteliste eingetragen.

Wenn Sie die **INSERT**-Anweisung aus [Listing 3.1](#) ausführen, wird die erste Zeile mit den von Ihnen vorgegebenen Spaltenwerten in die Tabelle *qualifikationen* eingetragen.

Um zu prüfen, ob die Zeile auch tatsächlich in die Tabelle *qualifikationen* eingefügt wurde, formulieren wir eine einfache **SELECT**-Abfrage, in der wir die Spalten *qid*, *bezeichnung*, *kuerzel* und *kategorie* abfragen:

```
SELECT qid,bezeichnung,kuerzel,kategorie
FROM qualifikationen;
```

Listing 3.2 So prüfen Sie, ob eine Zeile in die Tabelle »qualifikationen« eingefügt wurde.

qid	bezeichnung	kuerzel	kategorie
1	Word	msw	Office

Tabelle 3.2 Die neue Zeile wurde mit Ihrer ersten **INSERT**-Anweisung in die Tabelle eingefügt.

Ein Vorteil der expliziten Spaltenangabe in einer **INSERT**-Anweisung ist, dass Sie die Reihenfolge der Spalten, die Sie hinter der Tabellenbezeichnung in Klammern angeben, frei bestimmen können. Wir machen uns das zunutze und fügen einen weiteren Datensatz in die Tabelle *qualifikationen* ein. Diesmal verwenden wir allerdings eine geänderte Reihenfolge der Spaltenangaben in der Spaltenliste. Natürlich müssen wir dabei darauf achten, dass wir die gleiche Reihenfolge in der **VALUES**-Klausel anwenden. Beachten Sie, dass wir den Datensatz wieder mit einem eindeutigen Schlüsselwert versehen müssen. In [Listing 3.3](#) sehen Sie die beliebig festgelegte Spaltenreihenfolge der Spaltenliste, die wie folgt lautet: *kuerzel*, *qid*, *kategorie* und *bezeichnung*. Die Spaltenwerte hinter dem **VALUES**-Schlüsselwort werden jetzt in exakt dieser Reihenfolge angegeben, um die neue Zeile erfolgreich in die Tabelle *qualifikationen* einzufügen.

```
INSERT INTO qualifikationen (kuerzel,qid,kategorie,bezeichnung)
VALUES ('mex',2,'Office','Excel');
```

Listing 3.3 Eine neue Zeile in die Tabelle »qualifikationen« einfügen, mit einer geänderten Reihenfolge der Spaltenliste

qid	bezeichnung	kuerzel	kategorie
1	Word	msw	Office
2	Excel	mex	Office

Tabelle 3.3 Der zusätzliche Datensatz wurde unter Verwendung einer geänderten Spaltenreihenfolge mit einer INSERT-Anweisung eingefügt.

Sie müssen auch nicht alle Spaltenwerte gleichzeitig eintragen. Das ist ein großer Vorteil der expliziten Spaltenangabe in einer **INSERT**-Anweisung. In [Listing 3.4](#) sehen Sie, wie Sie einen Datensatz in die Tabelle *qualifikationen* mit einer **INSERT**-Anweisung einfügen, der lediglich die Werte für die Spalten *qid* und *bezeichnung* übergeben werden:

```
INSERT INTO qualifikationen (qid,bezeichnung)
VALUES (3,'Outlook');
```

Listing 3.4 Eine Zeile mit ausgewählten Spaltenwerten in die Tabelle »qualifikationen« einfügen

qid	bezeichnung	kuerzel	kategorie
1	Word	msw	Office
2	Excel	mex	Office
3	Outlook	NULL	NULL

Tabelle 3.4 Für den dritten Eintrag wurden nur Werte für die Spalten »qid« und »bezeichnung« eingefügt.

Für uns von Interesse ist die Zeile mit dem Wert 3 in der Spalte *qid*. Sie sehen hier, dass für die Spalten, die wir ausgelassen haben, automatisch **NULL**-Werte eingefügt wurden. Unter *kuerzel* und *kategorie* hatten wir ja keine konkreten Werte für die Spalten angegeben. Daher sind diese Werte nun **NULL** und somit nicht definiert.

Im nächsten Beispiel formulieren wir eine **INSERT**-Anweisung, die eine Zeile ausschließlich mit der Spalte und dem Wert für die Spalte *qid* einfügt:

```
INSERT INTO qualifikationen (qid)
VALUES (4);
```

Listing 3.5 Eine Zeile einfügen, die nur einen Primärschlüsselwert enthält

qid	bezeichnung	kuerzel	kategorie
1	Word	msw	Office
2	Excel	mex	Office
3	Outlook	NULL	NULL
4	NULL	NULL	NULL

Tabelle 3.5 Nicht angegebene Spaltenwerte erhalten automatisch einen NULL-Wert.

Die Spalten, die nicht in der **INSERT**-Anweisung angegeben wurden, wurden automatisch mit **NULL**-Werten befüllt.

Wir können **NULL**-Werte aber auch explizit in die Tabelle *qualifikationen* mit einer **INSERT**-Anweisung einfügen. Fügen wir also eine weitere Qualifikation in die Tabelle *qualifikationen* ein, indem wir sowohl einen Wert für den Primärschlüssel und die Spalte *bezeichnung* als auch **NULL** für die Spalten *kuerzel* und *kategorie* eintragen:

```
INSERT INTO qualifikationen (qid,bezeichnung,kuerzel,kategorie)
VALUES (5,'MS Project',NULL,NULL);
```

Listing 3.6 Explizite Angabe von **NULL**-Werten in einer **INSERT**-Anweisung

qid	bezeichnung	kuerzel	kategorie
1	Word	msw	Office
2	Excel	mex	Office
3	Outlook	NULL	NULL
4	NULL	NULL	NULL
5	MS Project	NULL	NULL

Tabelle 3.6 Die Spalten »kuerzel« und »kategorie« in Zeile 5 wurden explizit mit **NULL**-Werten gefüllt.

3.1.2 Spaltenwerte ohne Spaltenangabe einfügen

Wenn Sie für alle Spalten Werte eintragen, ist es nicht erforderlich, die Spalten in einer Liste hinter der Tabellenbezeichnung anzugeben. In diesem Fall ist die **VALUES**-Klausel mit der entsprechenden Angabe der Spaltenwerte ausreichend. An dieser Stelle ist allerdings die Reihenfolge der Spaltenwerte wichtig, die in die neue Zeile eingetragen werden sollen. Sie muss der Reihenfolge entsprechen, die bei der Tabellen-erstellung festgelegt wurde.

Unsere erste Aufgabe ist also, die Reihenfolge der Spalten herauszufinden. Wie kommen wir an diese Information?

Glücklicherweise hat die *International Standardization Organization (ISO)* einen Standard verabschiedet, der festlegt, dass relationale Datenbanken sogenannte *Views* (oder *Sichten*) mit Metadaten zur Verfügung stellen sollen. Auf das Thema Sichten bzw. Views werde ich in Kapitel 19 ausführlich eingehen. An dieser Stelle benötigen wir jedoch schon ein paar Informationen, um die Reihenfolge der Spalten zu ermitteln.

Auf Views können Sie wie auf eine Tabelle zugreifen. Sie sind zudem einem *Schema* mit der Bezeichnung *information_schema* zugeordnet. Der Begriff »Schema« wird je nach verwendeter Datenbank unterschiedlich interpretiert. Im Kontext einer MySQL- oder MariaDB-Datenbank ist der Begriff »Schema« synonym mit dem Begriff »Datenbank«. Informationen über Datenbanken sind also in der Datenbank *information_schema* hinterlegt.



Schema (MS SQL Server und PostgreSQL)

MS SQL Server

Vor der Version 2005 von MS SQL Server entsprach ein *Schema* einem Datenbankbenutzer. Das hat sich seitdem geändert. Ein Schema stellt einen Behälter dar, der Datenbankobjekte wie z. B. Tabellen enthält. Es existiert unabhängig von einem Datenbankbenutzer. Es ist ein Namensraum, dem Sie beliebige Datenbankobjekte zuordnen können. Ein Schema wiederum wird einem oder mehreren Datenbankbenutzern zugeordnet. Einer oder mehreren Datenbanken können ein oder mehrere Schemas zugewiesen werden.

PostgreSQL

Eine PostgreSQL-Datenbank fasst ebenfalls Datenbankobjekte wie beispielsweise Tabellen unter einem Namen zusammen. Es handelt sich also auch um einen Behälter, in dem z. B. mehrere Tabellen zusammengefasst werden können. Diese Behälter können ebenfalls einem oder mehreren Benutzern zugeordnet werden. Auch hier ist die Möglichkeit gegeben, einer oder mehreren Datenbanken ein oder mehrere Schemas zuzuweisen, um sie Datenbankbenutzern zuzuordnen.

Die Sicht, die für uns interessant ist, hat den Namen *columns*; wir lesen sie mit einer **SELECT**-Abfrage wie in [Listing 3.7](#) aus:

```
SELECT
    table_name,column_name,ordinal_position
FROM
    information_schema.columns
WHERE
    table_name = 'qualifikationen'
ORDER BY
    ordinal_position;
```

Listing 3.7 Metadaten von Spalten einer Tabelle abfragen

SELECT fragt in diesem Fall nur die Spalten ab, die für uns von Interesse sind. Dies sind die Einträge *table_name*, *column_name* und *ordinal_position*. In der **FROM**-Klausel geben wir die View *information_schema.columns* an. Sie enthält Informationen zu den Spalten aller Tabellen, die in Ihrer Datenbank enthalten sind. Abschließend grenzen Sie das Ergebnis mit einer **WHERE**-Klausel ein, damit Sie nur Spalten der Tabelle *qualifikationen* erhalten. Zu guter Letzt wenden Sie eine **ORDER BY**-Klausel an, um nach der Position (*ordinal_position*) zu sortieren.

table_name	column_name	ordinal_position
qualifikationen	qid	1
qualifikationen	bezeichnung	2
qualifikationen	kuerzel	3
qualifikationen	kategorie	4

Tabelle 3.7 Metainformationen von Spalten einer Tabelle

Sie sehen also die Tabellenbezeichnung (*table_name*), die Spaltennamen (*column_name*) und die Positionen der Spalten (*ordinal_position*). Aus diesem letzten Wert lässt sich die Reihenfolge der Spalten ablesen. *qid* ist an erster Position, *bezeichnung* an zweiter, *kuerzel* an dritter und *kategorie* ganz am Ende an vierter Position.

Mit den Informationen, die uns jetzt zur Verfügung stehen, können wir eine **INSERT**-Anweisung formulieren, die nur die Spaltenwerte für die **VALUES**-Klausel enthält. So können wir einen weiteren Datensatz in die Tabelle *qualifikationen* einfügen:

```
INSERT INTO qualifikationen
VALUES (6, 'SAP-Basis-Know-how', 'SAP', 'Office');
```

Listing 3.8 Zeilen ohne Spaltenangaben mit einer INSERT-Anweisung einfügen

qid	bezeichnung	kuerzel	kategorie
1	Word	msw	Office
2	Excel	mex	Office
3	Outlook	NULL	NULL
4	NULL	NULL	NULL
5	MS Project	NULL	NULL
6	SAP-Basis-Know-how	SAP	Office

Tabelle 3.8 In der letzten Zeile sehen Sie den neuen Datensatz mit dem Schlüsselwert »6« für die Spalte »qid«.



Zusammenfassung: Neue Zeilen mit INSERT in eine Tabelle einfügen

Die **INSERT**-Anweisung sieht zwei Varianten vor, Zeilen einzufügen:

- ▶ Variante 1: Die Spalten, in die Werte eingefügt werden sollen, werden einzeln in einer kommaseparierten Liste angegeben.
- ▶ Variante 2: Für alle Spalten einer Tabelle soll ein Spaltenwert eingefügt werden. Hier ist keine kommaseparierte Liste von Spaltenangaben erforderlich. Sie geben die Spaltenwerte hinter der **VALUES**-Klausel in einer kommaseparierten Liste an, die Sie in Klammern setzen. Hier müssen Sie darauf achten, dass die Reihenfolge der Spaltenwerte der Reihenfolge der Spalten entspricht, wie sie bei der Tabellen-erstellung festgelegt wurde. Da in diesem Fall die Spalten nicht in einer kommaseparierten Liste angegeben werden, müssen allen Spalten Werte zugeordnet werden.

Für beide Varianten gilt, dass Spaltenwerte, die nicht bekannt sind, explizit mit einem **NULL**-Wert in der **VALUES**-Klausel angegeben werden können.

Ebenfalls für beide Varianten gilt, dass Schlüsselwertspalten immer mit einem eindeutigen Wert versehen werden müssen, der noch nicht in der Schlüsselwertspalte eingefügt wurde.

Für die Variante 1 gilt, dass Spalten, die nicht in der Spaltenliste enthalten sind und denen demzufolge keine Spaltenwerte zugeordnet sind, automatisch von der Datenbank mit einem **NULL**-Wert versehen werden.

Für die Variante 1 gilt auch, dass Spalten in der Spaltenliste in beliebiger Reihenfolge angegeben werden können. Die Werte in der **VALUES**-Klausel müssen der Reihenfolge entsprechen.

3.1.3 Übungen

Übung 1

Nehmen Sie eine weitere Qualifikation in die Tabelle *qualifikationen* auf. Die neue Zeile soll mit einer expliziten Spaltenangabe in die Tabelle eingefügt werden. In die Spaltenliste soll Folgendes aufgenommen werden: *qid*, *bezeichnung*, *kuerzel* und *kategorie*. Wir möchten die Qualifikation SQL eintragen, die mit dem Kürzel ITE in die Kategorie Informatik aufgenommen werden soll. Wenn Sie das Kapitel praktisch nachvollzogen haben, nutzen Sie den Schlüsselwert 7 für die Spalte *qid*, um die neue Zeile mit einem eindeutigen Schlüssel in die Tabelle *qualifikationen* einzutragen.

Übung 2

In jedem größeren Unternehmen gibt es Administratoren. Diese Qualifikation werden wir als Nächstes in die Tabelle *qualifikationen* eintragen. Für die Schlüsselwertspalte *qid* verwenden Sie den Wert 8. Als Bezeichnung geben Sie die Zeichenkette Administrator an. Hier sollen das Kürzel ADA und die Kategorie Support eingetragen werden. In der ersten Übung haben Sie in der **INSERT**-Anweisung folgende Spaltenliste genutzt: *qid*, *bezeichnung*, *kuerzel* und *kategorie*. In dieser Übung verwenden Sie die folgende Spaltenliste, deren Reihenfolge der Spalten sich von der ersten Liste unterscheidet: *bezeichnung*, *qid*, *kategorie* und *kuerzel*. Achten Sie darauf, dass Sie die Spaltenwerte auch gemäß der neuen Spaltenliste hinter der **VALUES**-Klausel notieren.

Übung 3

Bei expliziter Angabe von Spalten in einer **INSERT**-Anweisung haben Sie die Möglichkeit, nur eine Auswahl von Spaltenwerten in die Tabelle *qualifikationen* einzufügen. Das machen wir uns jetzt zunutze, indem wir eine neue Zeile eintragen, in der lediglich der Schlüsselwert und die Bezeichnung für die zu ergänzende Qualifikation bekannt sind. Als Schlüsselwert verwenden Sie den Wert 9. Als Bezeichnung tragen Sie für die Spalte *bezeichnung* den Wert Catering ein. Es handelt sich also nur um zwei Spaltenwerte, die eingetragen werden. Welche Beobachtung machen Sie in Bezug auf die Spaltenwerte der Spalten *kuerzel* und *kategorie*, wenn Sie die Tabelle *qualifikationen* nach dem Einfügen mit einer **SELECT**-Anweisung abfragen?

Übung 4

In dieser Übung soll eine zusätzliche Qualifikation erfasst werden, von der wir nur die Bezeichnung wissen. Die Bezeichnung soll Projektleitung lauten. Als eindeutigen Schlüssel weisen wir für die Spalte *qid* den Wert 10 zu. In der Spaltenliste geben Sie folgende Spalten an: *qid*, *bezeichnung*, *kuerzel* und *kategorie*. Für die **VALUES**-Klausel nutzen Sie die beiden genannten Spaltenwerte sowie **NULL**-Werte für die übrigen Spalten *kuerzel* und *kategorie*, für die keine Spaltenwerte bekannt sind.

Übung 5

In diesem Kapitel haben Sie auch eine Form der **INSERT**-Anweisung kennengelernt, in der auf eine Spaltenliste verzichtet wird. Diese Form der **INSERT**-Anweisung werden wir jetzt nutzen, um eine weitere Qualifikation in die Tabelle *qualifikationen* einzutragen. Die Qualifikation soll die Bezeichnung *Personalverwaltung* erhalten. PEV soll als Abkürzung verwendet werden, und die Kategorie lautet *Personal*. Als eindeutigen Schlüsselwert können Sie den Wert 11 für die Spalte *qid* verwenden. Denken Sie daran, in der **VALUES**-Klausel die Werte gemäß der Reihenfolge der Spalten einzutragen, die Sie erst noch ermitteln müssen, um die Spaltenwerte korrekt in die Tabelle *qualifikationen* einzutragen.

Lösung zu Übung 1

```
INSERT INTO qualifikationen (qid,bezeichnung,kuerzel,kategorie)
VALUES (7,'SQL','ITE','Informatik');
```

Listing 3.9 Eine Zeile mit expliziter Spaltenangabe in eine Tabelle einfügen

Lösung zu Übung 2

```
INSERT INTO qualifikationen (bezeichnung,qid,kategorie,kuerzel)
VALUES ('Administrator',8,'Support','ADA');
```

Listing 3.10 Eine neue Zeile mit einer anders sortierten Spaltenliste mithilfe einer **INSERT**-Anweisung in eine Tabelle einfügen

Lösung zu Übung 3

```
INSERT INTO qualifikationen (qid,bezeichnung)
VALUES (9,'Catering');
```

```
/* Prüfabfrage */
SELECT qid, bezeichnung, kuerzel, kategorie
FROM qualifikationen;
```

Listing 3.11 Nicht alle Spaltenwerte für eine Zeile in eine Tabelle einfügen

Sie beobachten nach der Prüfabfrage, dass die Datenbank für die Spaltenwerte *kuerzel* und *kategorie* automatisch **NULL**-Werte in die neue Zeile eingetragen hat.

Lösung zu Übung 4

```
INSERT INTO qualifikationen (qid,bezeichnung,kuerzel,kategorie)
VALUES (10,'Projektleitung',NULL,NULL);
```

Listing 3.12 Fehlende Spaltenwerte in einer **INSERT**-Anweisung durch **NULL**-Werte ersetzen

Lösung zu Übung 5

```

/* Spaltenreihenfolge für die INSERT-Anweisung ermitteln */
SELECT
    table_name,column_name,ordinal_position
FROM
    information_schema.columns
WHERE
    table_name ='qualifikationen'
ORDER BY
    ordinal_position;
/* Datensatz gemäß der ermittelten Spaltenreihenfolge einfügen */
INSERT INTO qualifikationen
VALUES (11,'Personalverwaltung','PEV','Personal');
```

Listing 3.13 Eine neue Zeile ohne eine Spaltenliste mit einer INSERT-Anweisung eintragen

3.2 Zeilen mit einer UPDATE-Anweisung ändern

Natürlich wissen Sie, dass Daten nicht statisch sind. Es könnte z. B. das erfreuliche Ereignis einer Heirat einer Mitarbeiterin oder eines Mitarbeiters bevorstehen. Ebenso könnte es vorkommen, dass ein Mitarbeiter in den Ruhestand geht oder sich beruflich anderweitig orientiert. Die Informationen, die die Spaltenwerte von Zeilen in Tabellen ausmachen, können sich also ändern. SQL sieht hierfür eine **UPDATE**-Anweisung vor, die ebendies leistet. Um uns mit Aktualisierungsanweisungen vertraut zu machen, verwenden wir die Tabelle *qualglobal*, in der die Qualifikationen multinational verwaltet werden sollen. Einen Ausschnitt sehen Sie in Tabelle 3.9. Sie hat die gleiche Spaltenstruktur wie die Tabelle *qualifikationen* und enthält 7 Zeilen, an denen wir das Aktualisieren von Werten mit einer **UPDATE**-Anweisung üben können.

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	DAT	Informatik
2	Second Level Support	SLS	Informatik
3	First Level Support	FLS	Informatik
4	SAP Help Desk	SHD	Office
5	Excel	EXC	Office

Tabelle 3.9 Die Tabelle »qualglobal«

qid	bezeichnung	kuerzel	kategorie
6	Word	WOR	Office
7	Power Point	POW	Office

Tabelle 3.9 Die Tabelle »qualglobal« (Forts.)

3.2.1 Einen Spaltenwert einer Zeile ändern

In der ersten Zeile von [Tabelle 3.9](#) sehen Sie, dass Datenbankprogrammierung das Kürzel DAT zugeordnet ist. Dieses Kürzel soll jetzt durch das Kürzel SQL ersetzt werden. Hierzu ist es erforderlich, die erste Zeile der Tabelle zu aktualisieren und den Wert der Spalte *kuerzel* in SQL zu ändern. In Zusammenhang mit der Aktualisierungsanweisung **UPDATE** spielt die **WHERE**-Klausel eine wichtige Rolle, damit Sie gezielt einzelne Zeilen aktualisieren können. Die Aktualisierungsanweisung in [Listing 3.14](#) leistet genau das.

```
UPDATE qualglobal SET kuerzel='SQL' WHERE kuerzel='DAT';
```

Listing 3.14 Einen Spaltenwert einer Zeile mit UPDATE aktualisieren

Falls Sie beim Ausführen der Anweisung einen Fehler erhalten, deaktivieren Sie den *Safe_mode* unter **EDIT • PREFERENCES • SQL EDITOR**, und starten Sie die MySQL Workbench neu, damit die Änderung wirksam wird.

Sehen wir genauer hin, um zu verstehen, wie Sie die **UPDATE**-Anweisung verwenden. Eingeleitet wird die Anweisung mit dem Schlüsselwort **UPDATE**. Es folgt die Bezeichnung der *Tabelle*, deren *Spaltenwert* (oder auch deren Spaltenwerte) Sie aktualisieren wollen.

Hinter der Tabellenbezeichnung, die hier *qualglobal* lautet, notieren wir das Schlüsselwort **SET**, mit dem wir der Datenbank mitteilen, dass wir jetzt für eine Spalte eine Wertzuweisung vornehmen werden. Umgangssprachlich könnten Sie formulieren, dass Sie einen neuen Wert für eine Spalte festlegen. Die Wertzuweisung wiederum besteht aus einer Spaltenangabe, die die Spalte definiert, der ein neuer Wert zugewiesen werden soll. Dies geschieht mit einem Gleichheitszeichen, das hier als Wertzuweisungsoperator genutzt wird, und einem konkreten Wert. In diesem Beispiel ordnen wir dem Wert der Spalte *kuerzel* die neue Zeichenkette SQL zu.

Wir müssen natürlich sichergehen, dass wir auch die richtige Zeile aktualisieren. Die **WHERE**-Klausel haben Sie bereits genutzt, um Zeilen mit einer **SELECT**-Abfrage zu filtern und nur die Zeilen in einer Ergebnistabelle auszugeben, die bestimmten Kriterien entsprachen. Wir nutzen hier ebenfalls die **WHERE**-Klausel, um das gleiche Ziel zu erreichen wie bei der **SELECT**-Abfrage, diesmal allerdings mit der Absicht, die Spaltenwerte gefilterter Zeilen zu aktualisieren. Durch die **WHERE**-Klausel wird also eine Bedingung

ausgewertet. Wenn die Bedingung erfüllt ist, aktualisiert die Datenbank die Zeile. In unserem Fall bedeutet das: Wenn die Bedingung `kuerzel='DAT'` erfüllt ist, wird die Aktualisierung in der `SET`-Klausel durchgeführt. Das führt dazu, dass der Wert der Spalte `kuerzel` auf `SQL` gesetzt wird.

Wenn Sie die `UPDATE`-Anweisung aus [Listing 3.14](#) ausgeführt haben, können Sie eine einfache Prüfabfrage wie in [Listing 3.15](#) schreiben, um sich zu vergewissern, dass die Aktualisierung erfolgreich verlaufen ist. Wenn die Aktualisierung nicht erfolgreich war, wird natürlich keine Ergebniszeile ausgegeben, da wir in der `WHERE`-Klausel auf den aktualisierten Wert prüfen:

```
SELECT qid, bezeichnung, kuerzel, kategorie
FROM qualglobal
WHERE kuerzel='SQL';
```

Listing 3.15 Prüfabfrage für das Ergebnis einer Aktualisierung mit `UPDATE`

Sie sehen in [Tabelle 3.10](#), dass der Wert für die Spalte `kuerzel` von `DAT` in `SQL` geändert wurde. Betrachten Sie noch einmal die `WHERE`-Klausel aus [Listing 3.15](#). Gäbe es hier mehr als eine Zeile, die die Bedingung erfüllt, so würden diese Zeilen allesamt aktualisiert.

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	Informatik

Tabelle 3.10 Der Wert der Spalte »kuerzel« wurde in »SQL« geändert.

3.2.2 Mehrere Spaltenwerte einer Zeile gleichzeitig ändern

In [Tabelle 3.9](#) sehen Sie in der vierten Zeile die Qualifikation `SAP Help Desk`. In der Zeile sehen Sie auch, dass der Spalte `kuerzel` der Wert `SHD` und der Spalte `kategorie` der Wert `Office` zugeordnet ist. Die Werte der Spalte `kuerzel` und `kategorie` sollen geändert werden. Der Wert der Spalte `kuerzel` soll in den Wert `SAP` und der Wert der Spalte `kategorie` in den Wert `Informatik` geändert werden:

```
UPDATE qualglobal
SET kuerzel='SAP',kategorie='Informatik'
WHERE kuerzel='SHD';
```

Listing 3.16 Mehrere Spaltenwerte gleichzeitig mit einer `UPDATE`-Anweisung aktualisieren

Als Erstes betrachten wir die `SET`-Klausel der `UPDATE`-Anweisung. Hier finden Sie zwei Wertzuweisungen, die durch ein Komma separiert wurden. So wird der Spalte `kuerzel` der Spaltenwert `SAP` und der Spalte `kategorie` der Wert `Informatik` zugewiesen. Falls

Sie noch weiteren Spalten Werte zuordnen möchten, fügen Sie ein weiteres Komma gefolgt von einer weiteren Wertzuweisung an die Liste der SET-Klausel an.

Die WHERE-Klausel sorgt auch hier dafür, dass wir die richtige Zeile aktualisieren. In diesem Fall wird nur eine Zeile aktualisiert, und zwar diejenige, deren Wert der Spalte *kuerzel* gleich SHD ist.

qid	bezeichnung	kuerzel	kategorie
4	SAP Help Desk	SAP	Informatik

Tabelle 3.11 Die geänderte Zeile in der Tabelle »qualglobal«

Die Spaltenwerte für die Spalten *kuerzel* und *kategorie* wurden in die Werte SAP und Informatik geändert. Sie können hier wieder die Prüfabfrage aus [Listing 3.15](#) verwenden, um das Ergebnis darzustellen. Sie müssen nur die WHERE-Klausel dahingehend anpassen, dass die Spaltenwerte der Spalte *kuerzel* gegen den Wert SAP geprüft werden, um die aktualisierte Zeile zurückzuerhalten.

3.2.3 Spaltenwerte einer Spalte für mehrere Zeilen gleichzeitig ändern

Wenn in einer Spalte über mehrere Zeilen die gleichen Werte enthalten sind, können Sie diese Werte ebenfalls mit einer UPDATE-Anweisung ändern. Die wichtigste Rolle spielt in diesem Fall die Filterfunktion der WHERE-Klausel, mit der Sie bestimmen, für welche Zeilen die Spaltenwerte mit der UPDATE-Anweisung aktualisiert werden sollen. In den ersten drei Zeilen von [Tabelle 3.9](#) sind in der Spalte *kategorie* die Spaltenwerte Informatik eingetragen, und mit SAP Help Desk kam noch ein Informatik-Eintrag in Zeile 4 hinzu. Diese vier Spaltenwerte sollen nun durch den Wert Informationstechnologie aktualisiert werden:

```
UPDATE qualglobal
SET kategorie='Informationstechnologie'
WHERE kategorie='Informatik';
```

Listing 3.17 Mehrere Spaltenwerte einer Spalte gleichzeitig ändern

In [Listing 3.17](#) ist nichts Neues zu sehen. Es handelt sich um eine UPDATE-Anweisung mit einer Wertzuweisung in der SET-Klausel und einer WHERE-Klausel, die für die Werte der Spalte *kategorie* mittels einer Bedingung daraufhin überprüft, ob sie dem Wert Informatik entsprechen. Die WHERE-Klausel legt also mit ihrer Filterfunktion fest, welche Zeilen aktualisiert werden. In diesem Fall erfüllen vier Zeilen die Bedingung der WHERE-Klausel, und exakt diesen Zeilen wird in der SET-Klausel der Spaltenwert Informationstechnologie der Spalte *kategorie* zugewiesen. Wichtig ist an dieser Stelle, dass

Sie erkennen, dass es ganz von der Bedingung in der **WHERE**-Klausel abhängt, ob Sie eine oder mehrere Zeilen einer Tabelle aktualisieren.

Es können natürlich auch mehrere unterschiedliche Spaltenwerte über mehrere Spalten aktualisiert werden. In Zeile 5 bis 7 von Tabelle 3.9 sind Qualifikationen von Microsoft-Office-Produkten enthalten.

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	Informationstechnologie
2	Second Level Support	SLS	Informationstechnologie
3	First Level Support	FLS	Informationstechnologie
4	SAP Help Desk	SAP	Informationstechnologie

Tabelle 3.12 Alle vier Spaltenwerte der Spalte »kategorie«, die zuvor der Bedingung »kategorie='Informatik'« entsprachen, wurden auf den Wert »Informationstechnologie« aktualisiert.

Diesen Qualifikationen sind in der Spalte *kuerzel* folgende Kürzel als Werte zugeordnet: EXC, WOR und POW. Die Spaltenwerte der Spalte *kuerzel* sollen jetzt in den Wert MSO geändert werden. Es soll also keine unterschiedlichen Kürzel mehr für die unterschiedlichen Office-Anwendungen geben. Die Spaltenwerte der Spalte *kategorie* sollen hingegen in den Spaltenwert Microsoft Office geändert werden.

```
UPDATE qualglobal
SET kuerzel='MSO',kategorie='Microsoft Office'
WHERE kategorie='Office';
```

Listing 3.18 Mehrere Spaltenwerte mehrerer Zeilen gleichzeitig aktualisieren

qid	bezeichnung	kuerzel	kategorie
5	Excel	MSO	Microsoft Office
6	Word	MSO	Microsoft Office
7	PowerPoint	MSO	Microsoft Office

Tabelle 3.13 Die geänderten Werte für die Spalten »kuerzel« und »kategorie«

Die Spaltenwerte wurden für alle Zeilen gemäß den Wertzuweisungen in der **SET**-Klausel aktualisiert. Für die betroffenen Datensätze bedeutet das, dass die Anwendungen Word, Excel und PowerPoint jetzt mit dem Kürzel MSO und der Kategorie Microsoft Office ausgestattet sind.

3.2.4 Allen Spaltenwerten einer Spalte einen Wert zuordnen

Sämtliche Spaltenwerte einer Spalte in einen von Ihnen vorgegebenen Wert zu ändern, ist mit einer **UPDATE**-Anweisung sehr einfach zu realisieren. Sie lassen einfach die einschränkende **WHERE**-Klausel weg, und schon aktualisiert die **UPDATE**-Anweisung gemäß der Wertzuweisung in der **SET**-Klausel alle Spaltenwerte der Spalte, der Sie in der Wertzuweisung einen Wert zugewiesen haben.



MySQL Workbench: SAFE UPDATE/DELETE deaktivieren

Wenn Sie sämtliche Spaltenwerte einer Spalte mit einer **UPDATE**- oder **DELETE**-Operation ohne **WHERE**-Klausel ändern wollen, müssen Sie eine Sicherheitseinstellung der MySQL Workbench anpassen: Unter **EDIT • PREFERENCES** wählen Sie hierzu den Menüeintrag **SQL EDITOR** aus. Unter der Überschrift **OTHER** deaktivieren Sie die Checkbox **SAFE UPDATES**, um einen Spaltenwert über alle Zeilen zu aktualisieren.

Denken Sie daran, die MySQL Workbench nach der Deaktivierung der **SAFE UPDATES** zu beenden und neu zu starten. Die Änderung ist sonst nicht wirksam.

Wenn Sie eine MariaDB-Datenbank mit dem HeidiSQL-Editor nutzen, erfolgt lediglich eine Sicherheitsabfrage, wenn keine einschränkende **WHERE**-Klausel notiert wurde.

Die **UPDATE**-Anweisung in [Listing 3.19](#) ist so aufgebaut, dass sie alle Spaltenwerte der Spalte *kategorie* in den Wert *Derzeit keine Kategorie* ändert (siehe [Tabelle 3.14](#)):

```
UPDATE qualglobal
SET kategorie='Derzeit keine Kategorie';
```

Listing 3.19 Allen Spaltenwerten einer Spalte einen neuen Wert zuweisen

Sie können das leicht an der Wertzuweisung der **SET**-Klausel nachvollziehen. Da ja alle Spaltenwerte der Spalte *kategorie* geändert werden sollen, benötigt die **UPDATE**-Anweisung keine eingrenzende **WHERE**-Klausel, mit der Datensätze gezielt für eine Aktualisierung gefiltert werden.

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	Derzeit keine Kategorie
2	Second Level Support	SLS	Derzeit keine Kategorie
3	First Level Support	FLS	Derzeit keine Kategorie
4	SAP Help Desk	SAP	Derzeit keine Kategorie

Tabelle 3.14 Sämtliche Spaltenwerte der Spalte »kategorie« wurden mit dem Wert »Derzeit keine Kategorie« versehen.

qid	bezeichnung	kuerzel	kategorie
5	Excel	MSO	Derzeit keine Kategorie
6	Word	MSO	Derzeit keine Kategorie
7	PowerPoint	MSO	Derzeit keine Kategorie

Tabelle 3.14 Sämtliche Spaltenwerte der Spalte »kategorie« wurden mit dem Wert »Derzeit keine Kategorie« versehen. (Forts.)

Dies ist wirklich eine einfache Methode, sämtliche Spaltenwerte einer Spalte zu ändern. Beachten Sie, dass so eine **UPDATE**-Anweisung ohne eingrenzende **WHERE**-Klausel schnell ausgeführt ist: Es kann so ein großer Schaden entstehen, wenn fälschlicherweise alle Spaltenwerte einer oder mehrerer Spalten auf einen Wert gesetzt werden. Aber selbst wenn Sie eine **WHERE**-Klausel in einer **UPDATE**-Anweisung anwenden, kann es gefährlich werden. Was halten Sie von einer Bedingung, die wie folgt aussieht: **WHERE 8=8**?

Richtig, diese **WHERE**-Klausel ist für die **UPDATE**-Anweisung von keinerlei Bedeutung, da sie immer wahr ist. Die bloße Existenz einer **WHERE**-Klausel reicht also nicht aus, um sicherzustellen, dass nur die richtigen Datensätze aktualisiert werden.

WHERE-Klausel prüfen

Ein kleiner Tipp von mir: Wenn Sie vorhaben, einen oder mehrere Werte einer Spalte zu aktualisieren, sollten Sie immer zuerst die von Ihnen formulierte **WHERE**-Klausel mittels einer **SELECT**-Anweisung anwenden, um zu sehen, ob tatsächlich nur die Datensätze gefiltert werden, die Sie wirklich aktualisieren möchten.



3.2.5 Spaltenwerten mit einer UPDATE-Anweisung einen NULL-Wert zuweisen

In der Wertzuweisung einer **UPDATE**-Klausel können Sie Spalten auch einen **NULL**-Wert zuweisen. In [Listing 3.20](#) sehen Sie in der **SET**-Klausel der **UPDATE**-Anweisung eine Wertzuweisung, die der Spalte *kategorie* den nicht definierten Wert **NULL** zuweist:

```
UPDATE qualglobal
SET kategorie=NULL;
```

Listing 3.20 Spaltenwerten in einer **UPDATE**-Anweisung **NULL**-Werte zuweisen

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	NULL
2	Second Level Support	SLS	NULL
3	First Level Support	FLS	NULL
4	SAP Help Desk	SAP	NULL
5	Excel	MSO	NULL
6	Word	MSO	NULL
7	PowerPoint	MSO	NULL

Tabelle 3.15 Die Felder der Spalte »kategorie« wurden mit NULL-Werten überschrieben.

In diesem Beispiel haben wir ebenfalls keine einschränkende **WHERE**-Klausel verwendet. Die **UPDATE**-Anweisung funktioniert an dieser Stelle nur, weil die Tabelle bis auf die Schlüsselwertspalte über keinerlei Einschränkungen verfügt. Diese Art von Einschränkungen werden Sie in [Abschnitt 4.5](#) kennenlernen.

3.2.6 Schlüsselwertspalten mit UPDATE einen neuen Wert zuweisen

Die Tabelle *qualglobal* hält sich an die Empfehlung, dass jede Tabelle mit einer *Schlüsselwertspalte* ausgestattet sein sollte. Darin werden die eindeutigen Schlüsselwerte gespeichert, die die Zeilen der Tabelle identifizieren. Wenn Sie Spaltenwerte von Schlüsselwertspalten aktualisieren möchten, haben Sie ein paar Dinge zu beachten:

- Was nicht geht, ist, einer Schlüsselwertspalte einen **NULL**-Wert zuzuweisen. Ich denke, dass ist nachvollziehbar, da Werte von Schlüsselwertspalten immer eindeutig sein müssen und nicht mit Werten versehen werden können, die nicht definiert sind.
- Auch ist es nicht möglich, einen Schlüsselwert mit einer **UPDATE**-Anweisung in einen bereits bestehenden Schlüsselwert zu ändern, der in einer anderen Zeile einem Datensatz zugeordnet ist. Ein Schlüsselwert darf nicht mehrfach vorkommen, da er sonst seine Eindeutigkeit verliert. Die Datenbank würde einen derartigen Versuch mit einer Fehlermeldung quittieren.

Bisher habe ich Ihnen dargestellt, was nicht geht. Kommen wir jetzt zu dem, was wir machen können. Sie können den Wert einer Schlüsselwertspalte in jeden Wert ändern, der sich von den bereits bestehenden Schlüsselwerten unterscheidet.

Das probieren wir gleich aus. In [Tabelle 3.15](#) haben Sie bereits gesehen, dass die Schlüsselwerte 1 bis 7 Verwendung finden. Die numerischen Werte von 1 bis 7 könnten Sie

also nicht in der Wertzuweisung einer **SET**-Klausel zuweisen. In [Listing 3.21](#) weisen wir der Schlüsselwertspalte *qid* daher den Wert 100 zu:

```
UPDATE qualglobal
SET qid=100
WHERE qid=5;
```

Listing 3.21 Einen Wert einer Schlüsselwertspalte mit einer UPDATE-Anweisung ändern

Dieser Wert unterscheidet sich von allen Werten, die bereits Verwendung finden. In der **WHERE**-Klausel teilen wir der Datenbank mit, dass wir die Zeile mit dem Spaltenwert 5 in der Spalte *qid* aktualisieren wollen.

So ganz nebenbei lernen Sie hier, dass Sie Schlüsselwerte hervorragend in der **WHERE**-Klausel einer **UPDATE**-Anweisung verwenden können, um einen beliebigen Spaltenwert einer Tabelle zu aktualisieren, da ein Schlüsselwert ja immer eindeutig ist und exakt eine Zeile identifiziert.

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	NULL
2	Second Level Support	SLS	NULL
3	First Level Support	FLS	NULL
4	SAP Help Desk	SAP	NULL
6	Word	MSO	NULL
7	PowerPoint	MSO	NULL
100	Excel	MSO	NULL

Tabelle 3.16 Die Qualifikation »Excel« hat einen neuen Schlüsselwert »100« für die Spalte »qid« erhalten. Folglich gibt es den Schlüsselwert »5« jetzt nicht mehr.

Zusammenfassung: Spaltenwerte mit einer UPDATE-Anweisung aktualisieren

Mit einer **UPDATE**-Anweisung aktualisieren Sie Spaltenwerte einer Tabelle. Hier können folgende Szenarien Anwendung finden:

- ▶ einen Spaltenwert in einer Zeile ändern
- ▶ mehrere Spaltenwerte einer Zeile gleichzeitig ändern
- ▶ Spaltenwerte einer Spalte über mehrere Zeilen ändern



- ▶ allen Werten einer Spalte einen Wert zuweisen
- ▶ Spaltenwerten einen NULL-Wert zuweisen
- ▶ Schlüsselwertspalten aktualisieren

Wenn bestehende Spaltenwerte einer oder mehrerer Zeilen mit einer **UPDATE**-Anweisung geändert werden sollen, wird die **WHERE**-Klausel mit einem Filterkriterium angewendet. So wird sichergestellt, dass nur die ausgewählten Zeilen aktualisiert werden. Eine Aktualisierung mit der **UPDATE**-Anweisung wird oft zusammen mit der **WHERE**-Klausel genutzt. Stellen Sie zudem immer mit einer **SELECT**-Anweisung sicher, dass in der später auszuführenden **UPDATE**-Anweisung nur die Zeilen geändert werden, die auch geändert werden sollen.

Sollen alle Spaltenwerte einer oder mehrerer Spalten mit neuen Werten versehen werden, ist keine **WHERE**-Klausel erforderlich. Sie benötigen nur die **SET**-Klausel und die Wertzuweisungsliste. Auch hier müssen Sie unbedingt sicherstellen, dass auch tatsächlich eine derartige Aktualisierung stattfinden soll. Passen Sie auf, dass Sie nicht ungewollt Daten in Ihrer Tabelle überschreiben.

3.2.7 Übungen

Wenn Sie prüfen möchten, ob Sie die grundlegenden Operationen mit einer **UPDATE**-Anweisung beherrschen, versuchen Sie sich an den folgenden Übungen.

Übung 1

Die Zeile mit dem Schlüsselwert 2 der Spalte *qid* der Tabelle *qualglobal* identifiziert einen Datensatz, in dem der Spalte *bezeichnung* der Wert *Second Level Support* zugeordnet ist. Ändern Sie den Wert der Spalte *bezeichnung* mithilfe einer **UPDATE**-Anweisung in den Wert *Second Level Helpdesk*. Achten Sie darauf, dass Sie den bereits angegebenen Schlüsselwert der Spalte *qid* in der **WHERE**-Klausel verwenden, um die richtige Zeile zu aktualisieren. Formulieren Sie auch eine Prüfabfrage, die ausschließlich die Zeile mit dem Schlüsselwert 2 der Spalte *qid* ermittelt, um das Ergebnis der Aktualisierung mit einer **UPDATE**-Anweisung zu prüfen.

Übung 2

Jetzt soll die Zeile mit dem Schlüsselwert 3 der Spalte *qid* geändert werden. Der Wert *First Level Support* der Spalte *bezeichnung* soll in den Wert *First Level Helpdesk* geändert werden. Der Spalte *kuerzel* ist der Wert *FLS* zugeordnet. Ändern Sie den Wert in der gleichen **UPDATE**-Anweisung, und weisen Sie der Spalte *kuerzel* den Wert *FLH* zu. Formulieren Sie hier ebenfalls eine Prüfabfrage, die die geänderte Zeile mithilfe einer passenden **WHERE**-Klausel ermittelt, um die Aktualisierung zu prüfen.

Übung 3

Das Unternehmen beabsichtigt, auf eine freie Office-Suite umzusteigen. Daher soll den Werten der Spalte *kuerzel*, die derzeit dem Wert *MSO* entsprechen, ein **NULL**-Wert zugewiesen werden. Ermitteln Sie zur Überprüfung der aktualisierten Werte in einer **WHERE**-Klausel einer **SELECT**-Abfrage die Spaltenwerte der Spalte *kuerzel*, die gleich **NULL** sind.

Übung 4

Allen Werten der Spalte *kategorie* sind derzeit **NULL**-Werte zugeordnet. In dieser Übung soll ebenfalls allen Spaltenwerten der Spalte *kategorie* der Wert *Wird überarbeitet* zugewiesen werden. Hier können Sie mit einer einfachen **SELECT**-Abfrage das Ergebnis Ihrer Aktualisierung überprüfen. Sie benötigen keine einschränkende **WHERE**-Klausel, da wir ja alle Zeilen der Tabelle *qualglobal* aktualisieren wollen.

Übung 5

In dieser Übung soll ein Wert der Schlüsselwertspalte *qid* geändert werden. Setzen Sie den Schlüsselwert 100 der Spalte *qid* wieder auf den Wert 5 zurück.

Lösung zu Übung 1

```
UPDATE qualglobal
SET bezeichnung='Second Level Helpdesk'
WHERE qid=2;

SELECT qid,bezeichnung,kuerzel,kategorie
FROM qualglobal
WHERE qid=2;
```

Listing 3.22 Einen Spaltenwert einer Zeile mit einer UPDATE-Anweisung ändern

Lösung zu Übung 2

```
UPDATE qualglobal
SET bezeichnung='First Level Helpdesk',kuerzel='FLH'
WHERE qid=3;

SELECT qid,bezeichnung,kuerzel,kategorie
FROM qualglobal
WHERE qid=3;
```

Listing 3.23 Zwei Spaltenwerte gleichzeitig mit einer UPDATE-Anweisung aktualisieren

Lösung zu Übung 3

```
UPDATE qualglobal
SET kuerzel=NULL
WHERE kuerzel='MSO';

SELECT qid,bezeichnung,kuerzel,kategorie
FROM qualglobal
WHERE kuerzel IS NULL;
```

Listing 3.24 Einem Spaltenwert NULL zuweisen

Lösung zu Übung 4

```
UPDATE qualglobal
SET kategorie='Wird überarbeitet';

SELECT qid,bezeichnung,kuerzel,kategorie
FROM qualglobal;
```

Listing 3.25 Alle Spaltenwerte einer Spalte ändern

Lösung zu Übung 5

```
UPDATE qualglobal
SET qid=5
WHERE qid=100;

SELECT qid,bezeichnung,kuerzel,kategorie
FROM qualglobal
WHERE qid=5;
```

Listing 3.26 Einen Wert einer Schlüsselspalte ändern

3.3 Zeilen mit einer DELETE-Anweisung löschen

Eine grundlegende Operation in SQL fehlt Ihnen noch, nämlich die **DELETE**-Anweisung. Mit der **DELETE**-Anweisung können Sie gezielt einzelne Zeilen, mehrere Zeilen oder alle Zeilen einer Tabelle löschen. Die ersten beiden Löschoperationen werden ähnlich wie bei der **UPDATE**-Anweisung unter Zuhilfenahme der **WHERE**-Klausel durchgeführt, die sicherstellt, dass auch die richtigen Zeilen gelöscht werden. Wenn Sie alle Zeilen einer Tabelle löschen wollen, dann findet die **WHERE**-Klausel – ähnlich wie in der **UPDATE**-Anweisung – keine Verwendung.

3.3.1 Eine Zeile einer Tabelle löschen

Die Qualifikation SAP Help Desk ist nicht mehr erforderlich. Daher entfernen wir sie mit einer **DELETE**-Anweisung, wie in [Listing 3.27](#) zu sehen ist:

```
DELETE FROM qualglobal
WHERE qid=4;
```

Listing 3.27 Eine Zeile in einer Tabelle löschen

Die Anweisung beginnt mit dem Wort **DELETE**, das die Datenbank informiert, dass eine Löschoperation durchgeführt werden soll. Wir müssen der Datenbank natürlich noch mitteilen, aus welcher Tabelle wir eine Zeile löschen werden. Hierzu notieren Sie das Schlüsselwort **FROM** hinter dem **DELETE**-Schlüsselwort und geben direkt danach die Bezeichnung der Tabelle an. Hier beabsichtigen wir, eine Zeile aus der Tabelle *qualglobal* zu löschen.

Jetzt müssen wir nur noch die Zeile eingrenzen, damit auch nur die Zeile mit der Qualifikation SAP Help Desk gelöscht wird. Das machen wir ähnlich wie in der **UPDATE**-Anweisung, indem wir hinter dem Tabellennamen *qualglobal* eine **WHERE**-Klausel mit einer Bedingung angeben. In diesem Fall lautet die Bedingung *qid=4*. [Tabelle 3.17](#) zeigt das Ergebnis.

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	Wird überarbeitet
2	Second Level Helpdesk	SLS	Wird überarbeitet
3	First Level Helpdesk	FLH	Wird überarbeitet
5	Excel	NULL	Wird überarbeitet
6	Word	NULL	Wird überarbeitet
7	PowerPoint	NULL	Wird überarbeitet

Tabelle 3.17 Die Zeile mit dem Spaltenwert »4« der Spalte »qid« existiert nicht mehr, da sie erfolgreich aus der Tabelle gelöscht wurde.

Bei der Anwendung von **DELETE**-Anweisungen müssen Sie sehr vorsichtig agieren. Formulieren Sie die **WHERE**-Klausel mit Bedacht. Auch in diesem Fall empfiehlt es sich, die Bedingungen der **WHERE**-Klausel vorher mit einer **SELECT**-Anweisung zu überprüfen, um sicherzugehen, dass in der Löschoperation tatsächlich nur die Datensätze gelöscht werden, die auch gelöscht werden sollen.

3.3.2 Mehrere Zeilen einer Tabelle gleichzeitig löschen

Wenn Sie beabsichtigen, mehrere Zeilen einer Tabelle mit einer **DELETE**-Anweisung zu löschen, spielt der Spaltenwert eine Rolle, der in der Bedingung der **WHERE**-Klausel geprüft wird. Wenn die Bedingung in mehreren Zeilen erfüllt ist, so werden auch mehrere Zeilen gelöscht.

In [Listing 3.28](#) werden diejenigen Zeilen mit einer **DELETE**-Anweisung gelöscht, die aufgrund der Einführung einer alternativen Office-Suite obsolet geworden sind. In der **WHERE**-Klausel formulieren wir hierzu eine Bedingung, die die Zeilen filtert, deren Spaltenwerte der Spalte *kuerzel* gleich **NULL** sind:

```
DELETE FROM qualglobal
WHERE kuerzel IS NULL;
```

Listing 3.28 Mehrere Zeilen einer Tabelle löschen

qid	bezeichnung	kuerzel	kategorie
1	Datenbankprogrammierung	SQL	Wird überarbeitet
2	Second Level Helpdesk	SLS	Wird überarbeitet
3	First Level Helpdesk	FLH	Wird überarbeitet

Tabelle 3.18 Nach der **DELETE**-Anweisung sind die Zeilen mit den Qualifikationen der obsolet gewordenen Office-Suite nicht mehr enthalten.

3.3.3 Alle Zeilen einer Tabelle gleichzeitig löschen

Wenn Sie eine Tabelle mit aktuelleren Daten versehen wollen, kann es erforderlich sein, alle enthaltenen Zeilen zu löschen. Das könnte dann der Fall sein, wenn monatlich aktuelle Daten aus einer Datei importiert werden.

Sie löschen alle Zeilen einer Tabelle, indem Sie eine **DELETE**-Anweisung ausschließlich mit der **FROM**-Klausel und dem darauf folgenden Tabellennamen verwenden. In [Listing 3.29](#) werden mit der **DELETE**-Anweisung alle Zeilen aus der Tabelle *qualglobal* entfernt:

```
DELETE FROM qualglobal;
```

Listing 3.29 Alle Zeilen einer Tabelle löschen

Hier ist keine einschränkende **WHERE**-Klausel erforderlich. Wie Sie sich denken können, sollte eine **DELETE**-Anweisung ohne **WHERE**-Klausel ebenfalls nur ausgeführt werden, wenn Sie ganz sicher sind, dass alle Zeilen einer Tabelle gelöscht werden sollen.

qid	bezeichnung	kuerzel	kategorie

Tabelle 3.19 Alle Zeilen der Tabelle »qualglobal« wurden durch die DELETE-Anweisung gelöscht.

Zusammenfassung: Datensätze einer Tabelle mit einer DELETE-Anweisung löschen



Mit einer **DELETE**-Anweisung löschen Sie eine, mehrere oder alle Zeilen einer Tabelle. Wenn Sie eine oder mehrere Zeilen gezielt aus einer Tabelle entfernen wollen, legen Sie mit der **WHERE**-Klausel die Löschbedingung fest.

Vor der Durchführung einer **DELETE**-Anweisung mit einer **WHERE**-Klausel ist stets sicherzustellen, dass die **WHERE**-Klausel tatsächlich nur die Zeilen für die **DELETE**-Anweisung filtert, die gelöscht werden sollen. Eine einfache Möglichkeit, dies zu überprüfen, besteht darin, eine **SELECT**-Anweisung mit der **WHERE**-Klausel auszuführen. So können Sie testen, ob die **DELETE**-Anweisung nur auf die gewünschten Daten angewendet wird.

Wenn alle Zeilen einer Tabelle mit einer **DELETE**-Anweisung gelöscht werden sollen, so ist vorher ebenfalls zu prüfen, ob dies tatsächlich so erwünscht ist! Es ist in der Regel nicht möglich, eine Löschoperation rückgängig zu machen.

3.3.4 Übungen

Wenn Sie die Beispiele oben nachvollzogen haben, enthält die Tabelle *qualglobal* jetzt keine Datensätze mehr – eine denkbar schlechte Voraussetzung, um Übungen durchzuführen. Abhilfe schafft hier die für diesen Zweck angelegte Tabelle *qualglobalarchiv*, in der archivierte Qualifikationen gespeichert sind.

Übung 1

Als Erstes soll die Zeile mit dem Schlüsselwert 2 der Spalte *qid* der Tabelle *qualglobalarchiv* mit einer **DELETE**-Anweisung gelöscht werden. Gleich im Anschluss überprüfen Sie mit einer **SELECT**-Abfrage die noch vorhandenen Zeilen der Ergebnistabelle.

Übung 2

In dieser Übung sollen mehrere Zeilen der Tabelle *qualglobalarchiv* gleichzeitig mit einer **DELETE**-Anweisung gelöscht werden. Die Werte der Spalte *qid* für die zu löschen den Zeilen lauten: 5, 6 und 7.

Ein Tipp für Sie: Es gibt mehrere Möglichkeiten, die Bedingung der **WHERE**-Klausel zu formulieren. Eine davon ist, Spaltenwerte auf Mitgliedschaft in einer Werteliste zu prüfen. Alternativ könnten Sie auch eine Bedingung für die **WHERE**-Klausel formulieren, in der Sie einzelne Bedingungen auf Gleichheit mit einem der hier vorgegebenen Werte prüfen und die einzelnen Bedingungen mit einem logischen **OR**-Operator auswerten.

Prüfen Sie auch hier das Ergebnis Ihrer Löschoperation mit einer **SELECT**-Abfrage, um das Ergebnis der Löschoperation nachzuvollziehen.

Übung 3

Löschen Sie alle verbleibenden Zeilen der Tabelle *qualglobalarchiv*. Überprüfen Sie das Ergebnis dieser Löschoperation mit einer **SELECT**-Abfrage.

Lösung zu Übung 1

```
DELETE FROM qualglobalarchiv WHERE qid=2;
```

```
SELECT qid, bezeichnung, kuerzel, kategorie FROM qualglobalarchiv;
```

Listing 3.30 Gezielt eine Zeile in einer Tabelle mit einer **DELETE**-Anweisung löschen

Lösung zu Übung 2

```
/* Variante 1 */  
DELETE FROM qualglobalarchiv WHERE qid=5 OR qid=6 OR qid=7;  
/* Variante 2 */  
DELETE FROM qualglobalarchiv WHERE qid IN (5,6,7);
```

```
SELECT qid, bezeichnung, kuerzel, kategorie FROM qualglobalarchiv;
```

Listing 3.31 Mehrere Zeilen einer Tabelle gleichzeitig mit einer **DELETE**-Anweisung löschen

Lösung zu Übung 3

```
DELETE FROM qualglobalarchiv;
```

```
SELECT qid, bezeichnung, kuerzel, kategorie FROM qualglobalarchiv;
```

Listing 3.32 Alle Zeilen einer Tabelle mit einer **DELETE**-Anweisung löschen

3.4 Alle Zeilen einer Tabelle mit einer TRUNCATE-Anweisung löschen

Die **TRUNCATE**-Anweisung wird genutzt, um alle Zeilen einer Tabelle zu löschen. Sie haben richtig gelesen: Wenn Sie die **TRUNCATE**-Anweisung für eine Tabelle ausführen, werden in der Regel alle Zeilen dieser Tabelle gelöscht. Ob das Löschen wirklich unwiederbringlich Ihre Daten entfernt, hängt vom verwendeten Datenbanksystem ab.

Was ist der Nutzen einer solchen rabiaten SQL-Anweisung? Die Durchführung einer **TRUNCATE**-Anweisung ist viel schneller, da sie keine Informationen speichert, die es ermöglichen, das Löschen wieder rückgängig zu machen. Dies hat Auswirkungen auf das Thema *Transaktionen*, mit dem wir uns in [Kapitel 11](#), »Transaktionen«, befassen werden. Dort werden Sie auch erfahren, was es mit dem Rückgängigmachen auf sich hat.

3.4.1 Die TRUNCATE-Anweisung anwenden

Damit Sie mit der **TRUNCATE**-Anweisung alle Zeilen einer Tabelle löschen können, enthält die Übungsdatenbank die Tabelle *hardwarearchiv*. In [Tabelle 3.20](#) sehen Sie die Tabellenstruktur und die enthaltenen Datensätze. Unser Ziel ist es, alle Datensätze bzw. Zeilen der Tabelle zu löschen.

hardwareid	hardwarebezeichnung	name	vorname
1	Drucker	Lempe	Dirk
2	PC	Lempe	Dirk
3	Notebook	Lempe	Dirk
4	Drucker	Müller	Ralf
5	PC	Müller	Ralf
6	Notebook	Müller	Ralf

Tabelle 3.20 Die Tabelle »hardwarearchiv«

In [Listing 3.33](#) wird die **TRUNCATE**-Anweisung auf die Tabelle *hardwarearchiv* angewendet:

```
TRUNCATE TABLE hardwarearchiv;
```

Listing 3.33 Alle Zeilen einer Tabelle mit **TRUNCATE** löschen

Die Anwendung der **TRUNCATE**-Anweisung ist sehr einfach. Eingeleitet wird sie mit dem Schlüsselwort **TRUNCATE**. Es folgen das Schlüsselwort **TABLE** und der Name der Tabelle, die Sie löschen möchten. Wenn Sie diese Anweisung ausgeführt haben, enthält die Tabelle *hardwarearchiv* keine Datensätze mehr:

hardwareid	hardwarebezeichnung	name	vorname

Tabelle 3.21 Sämtliche Datensätze wurden aus der Tabelle gelöscht.



Die **TRUNCATE**-Anweisung löscht Daten dauerhaft

Wenn eine Tabelle sehr viele Datensätze bzw. Zeilen enthält, wird die **TRUNCATE**-Anweisung genutzt, um sämtliche Zeilen zu löschen.

Beachten Sie, dass das Löschen nicht mehr rückgängig gemacht werden kann!

Dieses Verhalten hängt jedoch vom verwendeten Datenbanksystem ab:

- ▶ MySQL-, MariaDB- und MS SQL Server-Datenbanken erlauben es nicht, die Löschoperation mit **TRUNCATE** wieder rückgängig zu machen.
- ▶ Die MySQL- und MariaDB-Datenbank löscht die alte Tabelle einfach und legt eine neue Tabelle mit der alten Struktur an.
- ▶ Die MS SQL Server-Datenbank schreibt keine Änderungsprotokolle und erreicht somit eine hohe Effizienz des Löschvorgangs.
- ▶ Wenn eine PostgreSQL-Datenbank verwendet wird, ist die Ausführung einer **TRUNCATE**-Anweisung unter bestimmten Bedingungen wieder rückgängig zu machen.

3.4.2 Übungen zum Thema »Alle Zeilen einer Tabelle mit einer **TRUNCATE**-Anweisung löschen«

Übung 1

Die Zeilen der Tabelle *hardwarearchiv* stehen uns leider nicht mehr zur Verfügung. Um diese Übung durchzuführen, enthält die Übungsdatenbank die Tabelle *hardware*, in der exakt die gleichen Zeilen enthalten sind wie in der Tabelle *hardwarearchiv*. Löschen Sie alle Zeilen der Tabelle *hardware* unter Verwendung der **TRUNCATE**-Anweisung.

Übung 2

Warum ist die Anwendung der **TRUNCATE**-Anweisung sehr effizient, wenn sie auf Tabellen mit sehr vielen Zeilen angewendet wird?

Lösung zu Übung 1

`TRUNCATE TABLE hardware;`

Listing 3.34 Alle Zeilen einer Tabelle mit einer TRUNCATE-Anweisung löschen

Lösung zu Übung 2

Die **TRUNCATE**-Anweisung sichert in der Regel keine Informationen über die gelöschten Datensätze bzw. Zeilen. Es besteht also keine Möglichkeit, eine **TRUNCATE**-Anweisung wieder rückgängig zu machen.

Kapitel 4

Tabellen anlegen (CREATE TABLE)

Bisher haben Sie Tabellen abgefragt (SELECT), Datensätze bzw. Zeilen in Tabellen eingefügt (INSERT) oder geändert (UPDATE) und aus Tabellen gelöscht (DELETE). Im Folgenden zeige ich Ihnen, wie Sie neue Tabellen in der Datenbank erstellen (CREATE).

Vielleicht haben Sie sich auch schon die Frage gestellt, wie die Tabellen erstellt wurden, mit denen wir bisher gearbeitet haben. Denn natürlich ist es gar nicht ungewöhnlich, dass in einer Datenbank neue Tabellen angelegt werden müssen.

Wie Sie bereits wissen, verfügen Tabellen stets über eine eindeutige Tabellenbezeichnung und können verschiedene Informationsarten in den unterschiedlichen Spalten enthalten: ganze Zahlen, rationale Zahlen, Text oder Datumsangaben. Was es mit den unterschiedlichen Informationsarten auf sich hat, klären wir in [Abschnitt 4.1](#). Danach zeige ich Ihnen in [Abschnitt 4.2](#) und [Abschnitt 4.3](#), wie Sie einen Datentyp in einen anderen konvertieren können. In [Abschnitt 4.4](#) lernen Sie, wie Sie eine Tabelle anlegen. In [Abschnitt 4.5](#) und [Abschnitt 4.6](#) stelle ich Ihnen noch verschiedene Möglichkeiten der Einschränkung vor.

4.1 Datentypen

Ein *Datentyp* besteht aus Elementen, die einem Wertebereich zugeordnet sind, und Operatoren, die auf diese Elemente angewendet werden können. Mit *Operatoren* sind mathematische Operatoren wie +, -, *, / gemeint, die auf numerische Wertebereiche angewendet werden können. Einer Spalte, die in einer Tabelle angelegt werden soll, muss immer ein Datentyp zugeordnet werden. Dieser Datentyp enthält Informationen darüber, welche Elemente die Spalte speichern kann.

Sehen wir uns hierzu einen ganzzahligen Datentyp des MySQL- bzw. MariaDB-Datenbanksystems an. Der Datentyp **TINYINT SIGNED** ordnet einer Spalte einen ganzzahligen Wertebereich (mit Vorzeichen) von -128 bis 127 zu. Wenn ausschließlich positive ganze Zahlen (ohne Vorzeichen) als Spaltenwerte gespeichert werden, so lässt der Datentyp **TINYINT UNSIGNED** eine Zahl aus dem Wertebereich von 0 bis 255 für einen Spaltenwert zu. Datentypen, die in einer MySQL- bzw. MariaDB Datenbank **UNSIGNED** sein können, sind standardmäßig immer **SIGNED**.

Einer Spalte vom Datentyp **TINYINT UNSIGNED** können Spaltenwerte im Bereich von 0 bis 255 zugewiesen werden. **UNSIGNED** bedeutet, dass der Wert ohne Vorzeichen gespeichert wird und daher nicht negativ werden kann. Wenn Sie stattdessen nur den Datentyp **TINYINT** verwenden, können Sie auch negative Zahlen darstellen. Dazu brauchen Sie dann ein Vorzeichen.

Sie merken schon: Wenn Sie den Datentyp **TINYINT** mit einem Vorzeichen verwenden, so teilt sich der Wertebereich von -128 bis 127 auf.

Wozu brauchen wir unterschiedliche Datentypen? Machen sie die Verwaltung von Tabellen nicht viel komplizierter?

Die Antwort liegt in den Anforderungen, die große Datenmengen an die Hardware stellen. Können Sie sich vorstellen, wie viel Speicherplatz der Wert 200 benötigt, um ihn als Information speichern zu können? Die Antwort hängt vom verwendeten Datentyp ab: Beim Datentyp **TINYINT** haben Sie 1 *Byte* zur Verfügung. Mit diesem Byte, das exakt aus 8 *Bit* besteht, können Sie also Werte von -128 bis 127 bzw. 0 bis 255 abbilden. Jede Zahl, die innerhalb dieser Wertebereiche gespeichert wird, benötigt also ein Byte Speicherplatz. Um zu verstehen, warum das so ist, müssen wir uns kurz ansehen, wie ein Computer Daten verarbeitet und speichert.

In der Informatik werden jegliche Arten von Informationen im *Dualsystem* gespeichert. Ein anderer Begriff dafür ist *Binärsystem*.

Wie kommt eine Zahl 200 in verschiedenen Zahlensystemen zustande? Das hat zwar nicht direkt etwas mit Datenbanken zu tun, ist für das bessere Verständnis von Datentypen jedoch wichtig.

Im täglichen Leben arbeiten Sie mit dem Dezimalsystem, also betrachten wir dieses zuerst. Das Dezimalsystem arbeitet mit der Basis 10. Halten wir uns einmal vor Augen, wie Sie die Zahl 200 im Dezimalsystem auf Basis der Zahl 10 herleiten können. In Tabelle 4.1 wird die Zahl 200 im Dezimalsystem dargestellt.

Potenz	10 ³	10 ²	10 ¹	10 ⁰
Wert der Potenz	1.000	100	10	1
Multiplikator	0	2	0	0

Tabelle 4.1 Den Wert 200 im Dezimalsystem ermitteln

Die Zahl 200 wird hier aus der Potenz zur Basis 10 und dem jeweiligen Exponenten 0, 1, 2, ... und letztendlich der Summe der Potenzwerte gebildet. In der ersten Zeile sind die Potenzen mithilfe der Basis und des Exponenten dargestellt. Die Werte, die sich jeweils ergeben, sind in der zweiten Zeile zu finden. Wenn Sie also den Wert 200 im Dezimalsystem abbilden wollen, wählen Sie nullmal den Wert der Potenz 10⁰,

nullmal den Wert der Potenz 10^1 und zweimal den Wert der Potenz 10^2 . Wenn Sie die Potenzen von der rechten Spalte ausgehend betrachten, sehen Sie, dass zuerst die Anzahl der Einer, dann die Anzahl der Zehner, gefolgt von der Anzahl der Hunderter angegeben wird. Zuerst werden die Werte multipliziert, die sich aus dem Multiplikator 0 bis 9 bezogen auf eine Potenz ergeben. Abschließend wird die Summe aus den einzelnen Werten gebildet, um den Dezimalwert zu berechnen.

Das hört sich viel komplizierter an, als es eigentlich ist, denn jeder kann den Wert 200 nachvollziehen. Ähnlich sieht es aus, wenn wir den Wert 1.526 im Dezimalsystem nachvollziehen wollen. [Tabelle 4.2](#) zeigt Ihnen, wie Sie ihn ermitteln. Wieder wählen wir die Multiplikatoren zu den jeweiligen Potenzen, errechnen den Wert und ermitteln schließlich die Summe der Werte, die sich aus den Multiplikationen ergeben.

Potenz	10^3	10^2	10^1	10^0
Wert der Potenz	1.000	100	10	1
Multiplikator	1	5	2	6

Tabelle 4.2 Den Wert 1.526 im Dezimalsystem ermitteln

Während wir im Dezimalsystem zehn Ziffern verwenden (nämlich 0 bis 9), haben wir im Dualsystem nur zwei Ziffern zur Verfügung. Dabei handelt es sich um die 0 und die 1. Ein Bit kann den Wert 1 oder 0 annehmen und ist damit die kleinste Einheit. Um die Größe von Festplatten anzugeben, wird meistens ein Vielfaches der Einheit Byte verwendet. Ein Byte besteht aus 8 Bit.

PCs sind grundsätzlich nur dazu in der Lage, mit dem Dualsystem zu arbeiten, und speichern Daten in diesen beiden Zuständen 0 und 1. Auch die Datenverarbeitung funktioniert so. Die Recheneinheiten, also die *Central Processing Units (CPUs)*, bestehen aus Transistoren, wie sie in elektrischen Schaltkreisen schon lange ihre Arbeit verrichten. Diese kennen nur zwei Zustände: an und aus. Das Dualsystem nutzt eben diese beiden Zustände und ordnet ihnen die 1 und die 0 zu.

Zahlen werden im Dualsystem genauso dargestellt wie im Dezimalsystem, auch wenn es auf den ersten Blick nicht so scheint. Der Wert 11001000 steht also für die gleiche Zahl wie der Wert 200, nur in unterschiedlichen Systemen. Wie sich der Wert 200 im Dualsystem genau aufschlüsselt, zeigt Ihnen [Tabelle 4.3](#).

Potenz	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Wert der Potenz	128	64	32	16	8	4	2	1
Multiplikator	1	1	0	0	1	0	0	0

Tabelle 4.3 Den Wert 200 im Dualsystem bzw. Binärsystem ermitteln

Was hat das nun mit den Datentypen zu tun?

Zu Anfang dieses Kapitels habe ich bereits den Datentyp `TINYINT` einer MySQL- bzw. MariaDB-Datenbank eingeführt. Dieser Datentyp hat 1 Byte, also 8 Bit, zur Verfügung, um ganze Zahlen zu speichern. Daher liegt sein Wertebereich zwischen 0 und 255.

Um den maximalen Wertebereich nachzuvollziehen, betrachten Sie Tabelle 4.4.

Potenz	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Wert der Potenz	128	64	32	16	8	4	2	1
Multiplikator	1	1	1	1	1	1	1	1

Tabelle 4.4 Den Maximalwert eines Bytes im Dualsystem bzw. Binärsystem ermitteln

Uns stehen 8 Bit zur Verfügung, denen wiederum die Potenzen von 2^0 bis 2^7 zugeordnet sind. Ordnen wir den acht Potenzen jeweils eine 1 zu, so erhalten wir den maximalen Zahlenbereich, der mit einem Byte dargestellt werden kann. Wir bilden also die Summe aus den Werten 128, 64, 32, 16, 8, 4, 2 und 1 und erhalten den maximal darstellbaren Wert 255 des Datentyps `TINYINT`.

Den Maximalwert haben wir ermittelt. Kommen wir zum Minimalwert. Um die Zahl 0 darzustellen, setzen wir einfach sämtliche Bits des Bytes auf den Wert 0 wie in Tabelle 4.5.

Potenz	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Wert der Potenz	128	64	32	16	8	4	2	1
Multiplikator	0	0	0	0	0	0	0	0

Tabelle 4.5 Der Minimalwert in der Dualdarstellung



Zusammenfassung: Berechnung eines maximalen Werts im Binärsystem

Im *Binärsystem* lassen sich zwei Zustände darstellen. Der Wert bildet die Basis unserer Rechnung. Der Exponent ist die Anzahl der Bits, die wir zur Verfügung haben. Sie berechnen den Maximalwert, indem Sie die Basis mit dem Exponenten potenzieren und vom Wert der Potenz den Wert 1 subtrahieren. Die Formel lautet also:

$$2^{\text{Bits}} - 1$$

Bei 8 Bit bzw. 1 Byte ergibt dies:

$$2^8 - 1 = 255$$

Bei 16 Bit bzw. 2 Byte ergibt dies:

$$2^{16} - 1 = 65.535$$

Bei 32 Bit bzw. 4 Byte ergibt dies:

$$2^{32} - 1 = 4.294.967.295$$

Wenn wir einen sehr großen Wert wie etwa 1.000.000 speichern wollen, brauchen wir dafür einen Datentyp, der mindestens vier Byte groß ist. Gleichzeitig wäre es Verschwendung, den kleinen Wert 7 in einen so großen Datentyp zu stecken. Hierfür würde auch ein Byte mehr als ausreichen.

4.1.1 Datentypen für ganze Zahlen

Wir wissen, dass wir ganze Zahlen verarbeiten wollen. Welchen Datentyp wir wählen sollten, hängt vor allen Dingen mit seinem Wertebereich zusammen. Verschaffen wir uns daher zunächst in Tabelle 4.6 einen Überblick über die MySQL- und MariaDB-Datenbanktypen für ganze Zahlen.

Datentyp	Speicherbedarf	minimaler Wert bis maximaler Wert
TINYINT mit Vorzeichen	1 Byte	−128 bis 127
TINYINT ohne Vorzeichen	1 Byte	0 bis 255
SMALLINT mit Vorzeichen	2 Byte	−32.768 bis 32.767
SMALLINT ohne Vorzeichen	2 Byte	0 bis 65.535
MEDIUMINT mit Vorzeichen	3 Byte	−8.388.608 bis 8.388.607
MEDIUMINT ohne Vorzeichen	3 Byte	0 bis 16.777.215
INT mit Vorzeichen	4 Byte	−2.147.483.648 bis 2.147.483.647
INT ohne Vorzeichen	4 Byte	0 bis 4.294.967.295
BIGINT mit Vorzeichen	8 Byte	−9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807
BIGINT ohne Vorzeichen	8 Byte	0 bis 18.446.744.073.709.551.615

Tabelle 4.6 Datentypen für ganze Zahlen und ihre Wertebereiche



Unterschiede zwischen den Datenbanksystemen

Nicht alle Datenbanksysteme verwenden die hier aufgezählten Datentypen gleich. So können die ganzzahligen Datentypen, die MySQL und MariaDB anbieten, mit den Vorzeichen (**SIGNED**) oder ohne Vorzeichen (**UNSIGNED**) zur Anwendung kommen. Bei PostgreSQL ist hingegen zu beachten, dass es dort nur die drei numerischen Datentypen **SMALLINT**, **INTEGER** und **BIGINT** gibt. MS SQL Server kennt zusätzlich **TINYINT**. Es ist auch möglich, dass andere Datenbanksysteme nicht alle Datentypen einsetzen, die hier aufgelistet wurden. Schauen Sie in der Dokumentation Ihres Datenbanksystems nach, welche Datentypen Ihnen zur Verfügung stehen.

Wir veranschaulichen uns das Ganze an einem Beispiel: Wenn Sie die Mitarbeiter eines Unternehmens mit eindeutigen Personalnummern versehen möchten, dann können Sie sich ungefähr an der Mitarbeiterzahl orientieren. Wenn das Unternehmen 1.500 Mitarbeiter hat, könnten Sie einer Spalte *personalnummer* den Datentyp **SMALLINT** zuordnen. Ohne Vorzeichen ermöglicht Ihnen dieser Datentyp einen ganzzahligen numerischen Bereich von 0 bis 65.535. Das passt.

Wenn Sie hingegen eine Volkszählung in Deutschland vornehmen wollen, reicht dieser Wertebereich nicht mehr aus. Der Datentyp **INT** ohne Vorzeichen wäre hingegen geeignet, um bis zu 4.294.967.295 Bürgerinnen und Bürger zu erfassen. Deutschland hat um die 80.000.000 Einwohner. Der Wertebereich des Datentyps **INT** wäre also mehr als ausreichend.

Warum ordnen wir nun nicht einfach jeder Spalte, die ganzzahlige numerische Werte speichern soll, immer den Datentyp **INT** zu? So könnte man ja auf Nummer sicher gehen, und es würde eigentlich immer passen.

Die Antwort habe ich bereits angedeutet: Der Speicherplatz wird sehr schnell knapp. Nun werden Sie vielleicht einwenden, dass Festplatten heute groß sind und Speicherplatz billig ist. Das mag sein, aber auch aktuelle Hardware kann schnell an ihre Grenzen kommen, wenn wir die Datentypen falsch wählen.

Ganz bewusst habe ich mit Ihnen einen maximalen Wertebereich ermittelt. Wie wir gesehen haben, hängt der maximale Wert einer Ganzzahl von der Anzahl der Bytes ab, die zur Verfügung stehen. Der Datentyp **INT** einer MySQL- oder MariaDB-Datenbank benötigt vier Byte, um eine Zahl darzustellen. Bleiben wir bei unserem Beispiel der Volkszählung. Wir multiplizieren 80.000.000 Einwohner mit 4 Byte. Das ergibt 320.000.000 Byte oder ungefähr 320 Megabyte. Und im Moment betrachten wir hier nur eine Spalte, in der die Einwohner durchnummeriert würden! Wenn noch andere Spalten hinzukommen, würde der Speicherbedarf enorm ansteigen und auch moderne Festplatten schnell füllen. Sie sollten daher unbedingt im Voraus planen, mit welchen Werten Sie arbeiten werden, und darauf achten, passende Datentypen auszuwählen.

4.1.2 Datentypen für rationale Zahlen

Bei rationalen Zahlen handelt sich um Zahlen, die sich eindeutig durch einen Bruch darstellen lassen. Rationale Zahlen können in SQL durch zwei verschiedene Kategorien von Datentypen abgebildet werden: durch die *Fließkomma*-Datentypen und die *Fixkomma*-Datentypen. In [Tabelle 4.7](#) stelle ich Ihnen die Fixkomma-Datentypen vor. Sie werden dann eingesetzt, wenn eine große Genauigkeit erforderlich ist. Im Englischen ist der entsprechende Begriff für »Fließkomma« *floating point*.

Datentyp
DECIMAL(<i>p</i> , <i>s</i>)
NUMERIC(<i>p</i> , <i>s</i>)

Tabelle 4.7 Fixkomma-Datentypen

Fließkommazahlen, die mit den Datentypen `FLOAT` und `DOUBLE` abgebildet werden, sind jedoch nicht genau; sie werden in den Dokumentationen der hier behandelten Datenbanksysteme wie folgt beschrieben: »Annäherung« (MySQL), »ungenau« (PostgreSQL), »nicht mit exakter Präzision« (MariaDB) und »letztendlich ungefähre Zahlen« (MS SQL Server).

Dahinter steckt ein grundlegendes Problem, wie Computer mit Zahlen umgehen. Es betrifft alle Programmiersprachen und -systeme. Für Sie ist wichtig, zu wissen, dass bei Berechnungen mit Fließkommazahlen Ungenauigkeit entsteht. Besonders auffällig ist dies, wenn Sie die Ergebnisse von Rechnungen vergleichen, die eigentlich gleich sein sollten, aber wegen minimaler Abweichungen in der Berechnung nicht übereinstimmen.

Die beiden Datentypen `DECIMAL` und `NUMERIC` gehören zu den Fixkomma-Datentypen und unterscheiden sich nicht, denn sie speichern exakt die gleichen numerischen Werte. Der Datentyp `NUMERIC` wurde für eine MySQL- oder MariaDB-Datenbank aus dem Datentyp `DECIMAL` implementiert. Sie können beide Typen also synonym verwenden, um rationale Zahlen abzubilden. Was für MySQL- und MariaDB-Datenbanken gilt, muss natürlich nicht für andere Datenbanken gelten. Es ist durchaus möglich, dass es in der Implementierung der Datentypen Unterschiede gibt.

Hinter der Datentypbezeichnung `DECIMAL` sehen Sie die Buchstaben *p* (*precision*) und *s* (*scale*). Die Variable *p* gibt die Gesamtanzahl der Ziffern des numerischen Wertes an. Die Variable *s* nennt uns die Stellen rechts vom Dezimalpunkt. Die Gehaltsangabe 60.000,35 € könnte in einer Spalte des Datentyps `DECIMAL` gespeichert werden, der 7 Gesamtstellen und 2 Stellen hinter dem Komma vorsieht. Die korrekte Angabe wäre also `DECIMAL(7,2)`.

4.1.3 Datentypen für Datum und Zeit

Datumsdatentypen werden oft genutzt. Denken Sie nur an die Spalten *gebdatum*, *eintrittsdatum* oder *austrittsdatum* der Tabelle *mitarbeiter*. In Tabelle 4.8 sind die jeweiligen Wertebereiche für die Datums- und Zeitdatentypen der MySQL- und MariaDB-Datenbanken beschrieben.

Datentyp	Wertebereich	
	von	bis
DATE	1000-01-01	9999-12-31
DATETIME	1000-01-01 00:00:00.000000	9999-12-31 23:59:59.999999
TIMESTAMP	1970-01-01 00:00:01.000000	2038-01-19 03:14:07.999999
TIME	-838:59:59.000000	838:59:59.000000
YEAR	1901	2155

Tabelle 4.8 Datentypen für Datums- und Zeitangaben



Unterschiede zwischen den Datenbanken

Auch bei den Zeitdatentypen gibt es einige Unterschiede zwischen den einzelnen Datenbanken. Sowohl die PostgreSQL- als auch die MS SQL Server-Datenbank stellen Ihnen einige weitere Typen zur Verfügung. Andere Schreibweisen sind je nach Lokalisierung der Datenbank möglich.

Beachten Sie, dass MySQL- und MariaDB-Datenbanken für Datumsangaben konsequent den Standard *ISO 8601* nutzen. PostgreSQL und MS SQL Server verwenden ebenfalls diese Schreibweise:

yyyy-mm-dd (Jahr, Monat, Tag)

Die Zeichenkette *yyyy* steht für das Jahr, *mm* für den Monat und *dd* für den Tag des jeweiligen Datums. Aus dem 16. März 2016 wird also 2016-03-16.

Auch die Zeitangaben werden diesem Standard folgend formatiert:

hh:mm:ss.f

Die Zeichenkette *hh* steht für die Stunden, *mm* für die Minuten, *ss* für die Sekunden, und das Zeichen *f* wird für den gebrochenen Anteil einer Sekunde verwendet. Aus Viertel nach elf wird also 11:15:12.5.

Vielleicht sind Sie in Tabelle 4.8 über den Wertebereich des Datentyps **TIME** gestolpert: Ein Tag hat doch nur 24 Stunden und keine 838 Stunden. Die Entwickler von MySQL und MariaDB haben sich dabei allerdings etwas gedacht. Schließlich ist es möglich,

Intervalle über eine Anzahl von Stunden zu definieren, die länger als die 24 Stunden eines Tages dauern.

4.1.4 Datentypen für Zeichenketten

Sie haben bereits etliche Spalten mit einer `SELECT`-Anweisung abgefragt, die Zeichenketten enthielten. So haben wir in den vorherigen Beispielen oftmals die Werte der Spalten *name* und *vorname* der Tabelle *mitarbeiter* ausgelesen. Diese Spalten speichern Zeichenketten. Zeichenketten bestehen aus einer Aneinanderreihung von einzelnen Zeichen, die sich zu Wörtern, Namen, Nummern wie ISBNs oder Bezeichnungen zusammensetzen. Wenn ich hier von Zeichenketten spreche, können Sie umgangssprachlich an *Wörter* denken. Hinzu kommen auch Ziffern, denn auch »Woert3r« wäre eine korrekte Zeichenkette.

Wenn Sie Zeichenketten speichern, ist der verwendete Zeichensatz von großer Bedeutung. Die Übungsdatenbank wurde mit dem Zeichensatz UTF-8 angelegt. Die UTF-8-Kodierung ist die am weitesten verbreitete Implementierung für Unicode-Standardzeichen. Der Unicode-Standard legt für beinahe jedes nur vorstellbare Zeichen eine eindeutige Nummer fest.

Sie können sich vorstellen, dass es sehr viele Zeichen gibt. Zunächst einmal fallen uns die Zeichen der europäischen Sprachen ein. Hinzu kommen die Sprachen des asiatischen Raums, die eine Vielzahl von Zeichen verwenden. Und schließlich gibt es eine ganze Anzahl von Zeichen, die in keinem Alphabet vorkommen, aber für die Arbeit mit Computern sehr wichtig sind. Denken Sie nur an das @.

Hier liegt die Stärke von Unicode: Die unterschiedlichen Codelisten enthalten für diese Vielzahl von Sprachen und Dialekten Zeichensätze, in denen die Zeichen einer Sprache hinterlegt sind. Solche Zeichensätze stellen für uns die Basis dar, um mit Zeichenketten arbeiten zu können. Für die deutsche Sprache sind die Zeichencodelisten *Basic Latin* und *Latin 1 Supplement* wichtig, da hier sämtliche Zeichen hinterlegt sind, die wir brauchen. Hierzu zählen z. B. die Umlaute. Tabelle 4.9 zeigt Ihnen einen Auszug aus dem Basic-Latin-Zeichensatz.

Dezimalwert	Hexadezimalwert	Zeichen	Dezimalwert	Hexadezimalwert	Zeichen
32	20		80	50	P
33	21	!	81	51	Q
34	22	"	82	52	R

Tabelle 4.9 Auszug aus dem Basic-Latin-Zeichensatz

Dezimalwert	Hexadezimalwert	Zeichen	Dezimalwert	Hexadezimalwert	Zeichen
35	23	#	83	53	S
36	24	\$	84	54	T
37	25	%	85	55	U
38	26	&	86	56	V
39	27	'	87	57	W
40	28	(	88	58	X
41	29	)	89	59	Y
42	2a	*	90	5a	Z
43	2b	+	91	5b	[
44	2c	,	92	5c	\
45	2d	-	93	5d	]
46	2e	.	94	5e	^
47	2f	/	95	5f	_
48	30	0	96	60	`
49	31	1	97	61	a
50	32	2	98	62	b
51	33	3	99	63	c
52	34	4	100	64	d
53	35	5	101	65	e
54	36	6	102	66	f
55	37	7	103	67	g
56	38	8	104	68	h
57	39	9	105	69	i
58	3a	:	106	6a	j

Tabelle 4.9 Auszug aus dem Basic-Latin-Zeichensatz (Forts.)

Den Zeichen sind Dezimalwerte und Hexadezimalwerte zugeordnet, um sie im dualen Zahlensystem abbilden zu können. Alle Zeichen, die in der Tabelle enthalten sind, lassen sich mit nur einem Byte darstellen. Die Dezimalwerte, die den Zeichen zugeordnet werden, entsprechen dem Sortierkriterium für die **ORDER BY**-Klausel.

Die Datentypen **CHAR** und **VARCHAR** bestimmen, wie viele Zeichen Sie in einer Zeichenkette als Wert speichern können.

Tabelle 4.10 zeigt Ihnen, wie diese Datentypen funktionieren. Mit der Variable **L** ordnen Sie **CHAR** oder **VARCHAR** die Länge der Zeichenkette zu, die Sie in einer Spalte speichern wollen. **L** ist dabei ein ganzzahliger numerischer Wert in einfachen Klammern. Der größtmögliche Wert, den Sie für **CHAR** in einer MySQL- oder MariaDB-Datenbank vergeben können, ist 255.

Datentyp	Maximale Zeichenkettenlänge
CHAR(L)	255
VARCHAR(L)	65.535 (seit MySQL 5.0.3)

Tabelle 4.10 Datentypen für Zeichenketten in einer MySQL-Datenbank

Nehmen wir an, Sie wollen die Namen von Mitarbeitern speichern und ordnen einer Spalte den Datentyp **CHAR(100)** zu. Der Spaltenwert **Dirk**, der ja nur vier der hundert zur Verfügung stehenden Zeichen nutzt, wird dann mit Leerzeichen aufgefüllt. Obwohl der eingegebene Wert nur vier Zeichen lang ist, würde der ganze Wert trotzdem 100 Byte verbrauchen.

In der MySQL-Datenbank merken Sie das allerdings nicht. Wenn die MySQL-Datenbank Spaltenwerte des Type **CHAR** speichert, werden die verbleibenden Zeichen nicht genutzt, jedoch wird der Speicherplatz so, wie er festgelegt wurde (in diesem Fall 100 Byte), belegt.

Ähnlich funktioniert der Datentyp **VARCHAR**. Auch hier bestimmen Sie über eine Variable die Größe des Datentyps, die maximal 65.535 Zeichen umfassen darf. Wenn Sie Ihrer Spalte mit Mitarbeiterdaten jedoch den Datentyp **VARCHAR(100)** zuordnen, dann wäre der Wert **Dirk** zwar auch nur vier Zeichen lang, die verbleibenden 96 Zeichen werden dann allerdings nicht mit Leerzeichen aufgefüllt.

«

Unterschiede in den Datenbanksystemen

Die PostgreSQL-Datenbank speichert sowohl in **CHAR** als auch in **VARCHAR** ein Maximum von 10.485.760 Zeichen. MS SQL Server-Datenbanken können immerhin bis zu 8.000 Zeichen in den beiden Datentypen speichern.



Zusammenfassung: Datentypen, die Spalten zugeordnet werden können

Datentypen werden verwendet, um Spalten Wertebereiche unterschiedlicher Elemente zuzuordnen. Solche Werte können ganze Zahlen, rationale Zahlen, Datums- und Zeitangaben oder Zeichenketten sein.

Diese Spaltenwerte werden im *Binärsystem* gespeichert. Beachten Sie bei der Planung einer Tabelle, wie groß die Werte sein werden, die Sie speichern möchten, und denken Sie daran, dass die umfangreichen Datenmengen schnell viel Speicherplatz und Rechenzeit benötigen.

Für eine MySQL- oder MariaDB-Datenbank stehen beispielsweise die folgenden Datentypen zur Verfügung:

- ▶ **TINYINT** mit und ohne Vorzeichen
- ▶ **SMALLINT** mit und ohne Vorzeichen
- ▶ **MEDIUMINT** mit und ohne Vorzeichen
- ▶ **INT** mit und ohne Vorzeichen
- ▶ **BIGINT** mit und ohne Vorzeichen
- ▶ **DECIMAL**(*p*, *s*)
- ▶ **NUMERIC**(*p*, *s*)
- ▶ **DATE**
- ▶ **DATETIME**
- ▶ **TIMESTAMP**
- ▶ **TIME**
- ▶ **YEAR**

Die in einer Datenbank verwendeten Zeichen sind in einem Zeichensatz hinterlegt. In einer MySQL- oder MariaDB-Datenbank können Sie unter anderem folgende Zeichensätze verwenden:

- ▶ **latin1**, CP1252 West European
- ▶ **latin2**, ISO 8859-2 Central European
- ▶ **utf8**, UTF-8 Unicode

4.1.5 Übungen

Übung 1

Zu Anfang dieses Kapitels habe ich Ihnen erläutert, wie Sie Dezimalzahlen im Dualsystem darstellen können. Stellen Sie in dieser Übung die Dezimalzahl 178 als Dualzahl dar. So viel vorweg: Sie kommen mit einem Byte aus.

Übung 2

Berechnen Sie den ganzzahligen Maximalwert, den Sie mit 3 Byte darstellen können.

Übung 3

Nennen Sie drei Datentypen, die Sie Spalten für ganzzahlige Spaltenwerte zuordnen können.

Übung 4

Welche Datentypen können Sie verwenden, um Spalten Wertebereiche mit rationalen Zahlen zuzuordnen?

Übung 5

Nennen Sie drei Datentypen, die Sie Spalten zuordnen können, um Datums- und Zeitangaben zu speichern.

Übung 6

Mit welchen Datentypen können Sie Spalten Werte von Zeichenketten zuordnen? Wie unterscheiden sich die Datentypen?

Lösung zu Übung 1

Den Dezimalwert 178 erhalten Sie, wenn Sie zunächst den größten Wert mit dem höchstrangigen Bit betrachten. Das höchste Bit eines Bytes hat den Wert 128 und ist daher die Basis für die Summenbildung. Dem achten Bit können Sie also eine 1 zuordnen. Das siebte Bit kommt nicht infrage, da die Summe, die sich ergeben würde, den geforderten Dezimalwert von 178 überschreitet. Wir ordnen dem siebten Bit also eine 0 zu. Das sechste Bit hingegen stellt mit der Potenz 2^5 den Wert 32 dar. Zusammen mit 128 erhalten Sie den Wert 160. Folglich können Sie dem sechsten Bit ebenfalls eine 1 zuordnen. Das fünfte Bit stellt mit der Potenz 2^4 den Wert 16 dar. In Summe erhalten Sie also den Wert 176. Auch diesem Bit können wir den Wert 1 zuordnen. Die Potenzen 2^3 und 2^2 mit den Werten 8 und 4 würden in der Summenbildung den geforderten Wert überschreiten. Folglich müssen wir den Potenzen 2^3 und 2^2 eine 0 zuordnen. Die Potenz 2^1 mit dem Wert 2 führt uns zum Ziel. Die Summe von 176 und 2 ist 178. Der Potenz 2^1 ordnen wir also eine 1 zu. Die letzte Potenz, 2^0 , benötigen Sie nicht zur Summenbildung und ordnen ihr daher eine 0 zu.

Die Dezimalzahl 178 entspricht also dem Dualwert 10110010.

Potenz	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Wert der Potenz	128	64	32	16	8	4	2	1
Multiplikator	1	0	1	1	0	0	1	0

Lösung zu Übung 2

Der Maximalwert, der sich aus 3 Byte ergibt, lautet 16.777.215. Die Rechnung dazu sieht so aus:

$$2^{24} - 1 = 16.777.215$$

Lösung zu Übung 3

Für die MySQL- bzw. MariaDB-Datenbank kämen drei der folgenden Datentypen in Betracht: **TINYINT**, **SMALLINT**, **MEDIUMINT**, **INT** und **BIGINT**.

Für die PostgreSQL-Datenbank kämen die folgenden Datentypen in Betracht: **SMALLINT**, **INTEGER** und **BIGINT**.

Für die MS SQL Server-Datenbank kämen drei der folgenden Datentypen in Betracht: **TINYINT**, **SMALLINT**, **INT** und **BIGINT**.

Lösung zu Übung 4

DECIMAL und **NUMERIC**

Lösung zu Übung 5

Für die MySQL- bzw. MariaDB-Datenbank kämen drei der folgenden Datentypen in Betracht: **DATE**, **TIME**, **DATETIME** und **TIMESTAMP**.

Für die PostgreSQL-Datenbank kämen drei der folgenden Datentypen in Betracht: **DATE**, **TIME**, **TIMESTAMP** und **INTERVAL**.

Für die MS SQL Server-Datenbank kämen drei der folgenden Datentypen in Betracht: **DATE**, **TIME**, **DATETIME** und **SMALLDATETIME**.

Lösung zu Übung 6

CHAR und **VARCHAR**

Mit dem Datentyp **CHAR** wird einer Spalte die Länge der Zeichenkette zugeordnet. Ist eine Zeichenkette kürzer, so werden die freien Zeichen mit Leerzeichen aufgefüllt. Der Datentyp **VARCHAR** bekommt ebenfalls die Länge der zu speichernden Zeichenkette übergeben. Dieser Datentyp füllt allerdings die nicht genutzten Zeichen nicht mit Leerzeichen auf. Demzufolge benötigt er weniger Speicherplatz, da er ja nur die Zeichen speichert, die tatsächlich vorhanden sind.

4.2 Datentypen umwandeln

Datenbanken müssen unterscheiden können, ob es sich bei den Daten um Textwerte, ganze Zahlen, Kommazahlen oder etwa Datumswerte handelt. Wenn eine Datenbank beispielsweise zwei Zahlenwerte miteinander addieren soll, muss sie anders handeln, als wenn zwei Datumswerte addiert werden. Und Sie haben ja bereits in [Kapitel 3](#) gelernt, dass sich Textwerte, die mit Hochkommas gekennzeichnet werden, nicht zu Zahlen addieren lassen. Wie also geht eine Datenbank mit Text um, der zu einer Zahl addiert werden soll? Und was würde passieren, wenn Sie versuchen, Zahlen mit unterschiedlichen Wertebereichen zu addieren, etwa eine Zahl vom Typ **INTEGER** und eine Zahl vom Typ **SMALLINT**? Hier kommen die sogenannten *Konvertierungsfunktionen* zum Einsatz, denen wir uns jetzt widmen werden.

SQL bietet die **CAST**-Konvertierungsfunktion an, mit der Sie einen Wert eines Datentyps in einen anderen Datentyp umwandeln. Das setzt allerdings voraus, dass die Werte tatsächlich in den Zieldatentyp umgewandelt werden können: Ein Datum lässt sich z. B. nicht in eine ganze Zahl umwandeln oder umgekehrt.

Die **CAST**-Funktion kann bei allen Operationen oder Wertzuweisungen zum Einsatz kommen. Die folgende Auflistung zeigt Ihnen, wo es notwendig sein kann, eine Typkonvertierung vorzunehmen:

- ▶ bei der Verwendung von arithmetischen Operatoren (+, -, *, /, %)
- ▶ bei Einfügeoperationen mit **INSERT** oder **UPDATE**
- ▶ bei Vergleichen mithilfe einer Bedingung in einer **WHERE**-Klausel
- ▶ bei der Sortierung mithilfe einer **ORDER BY**-Klausel
- ▶ bei der Anwendung einer **UNION**-Klausel
- ▶ bei Aufrufen von Skalarfunktionen mit Parametern

Um die Additionen aus unserem Beispiel erfolgreich durchzuführen, muss die Datenbank entweder die Zahl, die dem Datentyp **SMALLINT** zugeordnet ist, in den Datentyp **INTEGER** konvertieren, oder Sie als Anwender müssten dafür Sorge tragen, dass die Zahl, die dem Datentyp **SMALLINT** zugeordnet ist, in den Datentyp **INTEGER** umgewandelt wird.

Eine Typkonvertierung muss sich immer an der Zahl orientieren, die über den größeren Wertebereich verfügt. Dies gilt sowohl für einzelne Werte als auch für ganze Spalten. Dies wird bereits durch das einfache Beispiel der Addition deutlich, in dem beide Werte nach der Konvertierung dem größeren Datentyp entsprechen.

Zusammenfassung: Typkonvertierung mit CAST

Eine Typkonvertierung ist immer dann erforderlich, wenn Wertebereiche, die Spalten zugeordnet werden, nicht identisch sind.



Für die Typkonvertierung steht in SQL die Funktion **CAST** zur Verfügung, die Spaltenwerte in einen anderen Wertebereich umwandelt.

4.3 Explizite und implizite Typkonvertierung

Wenn die Datenbank die Typkonvertierung automatisch übernimmt, wird dies als *implizite Typkonvertierung* bezeichnet. Wenn Sie hingegen in Ihren SQL-Anweisungen Konvertierungen vornehmen, nennt man dies *explizite Typkonvertierung*.

4.3.1 Explizite Typkonvertierung

Wenn Sie eine **CAST**-Funktion einsetzen, teilen Sie der Datenbank *explizit* (ausdrücklich) mit, dass der Spalten- oder Literalwert in einen anderen Datentyp konvertiert werden soll:

```
SELECT CAST(5 AS DECIMAL(6,3)) AS konvertierungsergebnis;
```

Listing 4.1 Typkonvertierung mit der CAST-Funktion

Hier wird ein ganzzahliges numerisches Literal 5 mit dem Aufruf **DECIMAL(6,3)** konvertiert. Das bedeutet, dass eine ganze Zahl in einen Datentyp konvertiert werden soll, der maximal insgesamt sechsstellig ist und drei Nachkommastellen hat.

In der Spaltenauswahlliste der **SELECT**-Abfrage sehen Sie den Funktionsaufruf der **CAST**-Konvertierungsfunktion. Als Parameter übergeben wir als Erstes die Zahl 5, die in einen Zieldatentyp konvertiert werden soll. Es folgen das Schlüsselwort **AS** sowie der Zieldatentyp, in den konvertiert werden soll.

In der einzelnen Ergebnisliste in Tabelle 4.11 sehen Sie, dass die Datenbank die **CAST**-Funktion erfolgreich ausgeführt hat. Der ganzzahlige Spaltenwert vom Typ **INTEGER** wurde in eine Dezimalzahl des Typs **DECIMAL** umgewandelt. Er verfügt über drei Nachkommastellen und eigentlich auch über drei Stellen vor dem Komma, von denen wir aber nur eine sehen.

konvertierungsergebnis
5.000

Tabelle 4.11 Die Konvertierung einer ganzen Zahl in eine Dezimalzahl

Wie sähe es nun aus, wenn Sie das Ganze umgekehrt versuchen würden? Probieren wir es doch einfach aus:


```

/* MySQL und MariaDB */
SELECT CAST(3.4 AS SIGNED INTEGER) AS vondezimalzuganzzahl;
/* PostgreSQL und MS SQL Server */
SELECT CAST(3.4 AS INTEGER) AS vondezimalzuganzzahl;

```

Listing 4.2 Eine Dezimalzahl in eine ganze Zahl umwandeln

Die Datenbanksysteme gehen davon aus, dass eine Dezimalzahl in eine ganze Zahl umgewandelt werden soll, und geben Ihnen dementsprechend lediglich den ganzzahligen Anteil der Dezimalzahl als Ergebnis zurück.

vondezimalzuganzzahl
3

Tabelle 4.12 Umwandlung einer Dezimalzahl in eine Ganzzahl
(MySQL, MariaDB, PostgreSQL und MS SQL Server)

Sie sehen also, dass die Konvertierung von Datentypen nicht ganz einfach ist. Achten Sie bei der Konvertierung auf den Wertebereich, in den umgewandelt werden soll. Wenn Sie eine Dezimalzahl in einen ganzzahligen Datentyp umwandeln, sollte Ihnen stets bewusst sein, dass Ihnen die Werte hinter dem Komma verlorengehen.

4.3.2 Implizite Typkonvertierung

Wenn Sie die Typkonvertierung der Datenbank überlassen, nennt man dies *implizite Typkonvertierung*. Sie versucht, die Datentypen automatisch anzupassen, wenn keine Übereinstimmung (*Konformität*) von Datentypen gegeben ist. Das gilt sowohl für Wertzuweisungen als auch für mathematische Operationen, z. B. in einer Spaltenauswahlliste.

Eine Typkonvertierung brauchen Sie beispielsweise, wenn Sie mit einer Zeichenkette rechnen wollen. Hier käme eine einfache Addition von Zahlen in Betracht, in der die Datenbank die Zeichenketten implizit umwandelt. Wenn Sie sich die Frage stellen, in welchen Datentyp implizit umgewandelt wurde, so ist dies nicht so leicht zu beantworten. Es ist auch nicht garantiert, dass je nach Datenbank eine implizite Typkonvertierung immer erfolgreich funktioniert. Und leider gibt es hier wieder Unterschiede zwischen den einzelnen Datenbanken.

In [Tabelle 4.13](#) sehen Sie in der Spalte »Abfrage« jeweils in Kurzform eine **SELECT**-Abfrage, die das Ergebnis eines Ausdrucks berechnet. Die erste **SELECT**-Abfrage berechnet eine Addition aus der ganzen Zahl 4 und der Ziffer 9, die als Zeichenkettenliteral angegeben wird. Alle vier hier behandelten Datenbanken führen eine implizite Typkonvertierung durch und konvertieren die Zeichenkette 9 in eine ganze Zahl, um anschließend die Addition durchzuführen.

Abfrage	Ergebnis MySQL	Ergebnis PostgreSQL	Ergebnis MS SQL Server	Ergebnis MariaDB
SELECT 4 + '9';	13	13	13	13
SELECT 4 + '09';	13	13	13	13
SELECT 4 + ' 09.00 ';	13	Konvertierungsfehler	Konvertierungsfehler	13
SELECT 4 + '9.53';	13.53	Konvertierungsfehler	Konvertierungsfehler	13.53
SELECT 4.53 + '9';	13.5300000 00000001	13.53	13.53	13.5300000 00000001
SELECT '3.47' + '9.53';	13	Konvertierungsfehler	Verbindet Zeichenketten: 3.479.53	13

Tabelle 4.13 Implizite Typkonvertierung in MySQL-, MariaDB-, PostgreSQL- und MS SQL Server-Datenbanken

Die zweite **SELECT**-Abfrage unterscheidet sich durch eine vorangestellte 0, die vor der Ziffer 9 innerhalb der Zeichenkette notiert wurde. Auch in diesem Fall kommen die hier behandelten Datenbanken zu dem Ergebnis 13.

Die dritte **SELECT**-Abfrage enthält sogar ein führendes und endendes Leerzeichen vor bzw. nach den Ziffern 09.00. Hier kommen MySQL- und MariaDB-Datenbanken zu dem richtigen Ergebnis, 13. Bei PostgreSQL- und MS SQL Server-Datenbanken tritt ein Konvertierungsfehler auf.

In der Spaltenauswahlliste der vierten **SELECT**-Abfrage addieren wir zu einer ganzen Zahl 4 eine Fließkommazahl 9.53, die als Zeichenkettenliteral angegeben ist. MySQL- und MariaDB-Datenbanken führen für den Wert 9.53, der als Zeichenkette notiert wurde, wieder eine implizite Typkonvertierung durch und kommen zu dem Ergebnis 13.53. PostgreSQL- und MS SQL Server-Datenbanken melden hingegen einen Konvertierungsfehler, da sie implizit versuchen, den zweiten Parameter in einen **INTEGER**-Datentyp umzuwandeln. Auch wenn 9.53 wie eine Zahl aussehen mag: Hier ist es eine Zeichenkette, und die beiden Datenbanken tun sich schwer damit, eine solche Kette mit einem Komma (bzw. Punkt) in einen Zahlenwert umzuwandeln.

In der Spaltenauswahlliste der fünften **SELECT**-Abfrage versuchen wir, die Fließkommazahl 4.53 zur Ziffer 9 zu addieren, die als Zeichenkette notiert ist. Die MySQL-Datenbank führt die Addition aus und gibt eine Fließkommazahl zurück, die nicht exakt dem erwarteten Ergebnis entspricht. PostgreSQL- und MS SQL Server-Daten-

banken führen ebenfalls eine implizite Konvertierung durch und liefern Ihnen mit dem Wert 13.53 das Ergebnis, das exakt der Addition der beiden Werte entspricht.

In der letzten `SELECT`-Abfrage wird eine Addition von zwei Fließkommazahlen durchgeführt, die als Zeichenketten notiert sind. MySQL- und MariaDB-Datenbanken führen eine implizite Typkonvertierung durch und kommen zum erwarteten Ergebnis 13. Die PostgreSQL-Datenbank gibt einen Fehler aus, mit dem sie mitteilt, dass sie implizit keinen geeigneten Konvertierungstyp finden kann. Die MS SQL Server-Datenbank hingegen nutzt den `+`-Operator auch zum Verbinden von Zeichenketten. Genau das passiert hier: Die Datenbank geht davon aus, dass Zeichenketten verbunden werden sollen, und führt diese Verkettung aus. Darauf werde ich in Kapitel 14, »Skalarfunktionen anwenden«, noch genauer eingehen.

Implizite Typkonvertierung

Für die implizite Typkonvertierung wendet jede Datenbank unterschiedliche Regeln an. Um ganz sicherzugehen, sollten Sie bei der bewussten Nutzung der impliziten Typkonvertierung die jeweilige Dokumentation der verwendeten Datenbank zurate ziehen.

Bei Berechnungen mit größeren Zahlen kann es durchaus zu Unschärfen in der Berechnung kommen. Besonders in diesen Fällen sollten Sie die explizite Typkonvertierung nutzen. So haben Sie es selbst in der Hand, ob Sie beispielsweise eine Zahl, die als Zeichenkette in einer Datenbank gespeichert wurde, in einen `INTEGER`-, `DECIMAL`- oder `NUMERIC`-Typ umwandeln. Wenn Sie die explizite Typkonvertierung einsetzen, müssen Sie aber darauf achten, die richtigen Datentypen zu wählen. Wenn Sie also zwei Zahlen addieren, müssen Sie sicherstellen, dass die Zahl mit dem kleineren Wertebereich in den Wertebereich der Zahl umgewandelt wird, deren Wertebereich größer ist.

Exemplarisch zeige ich Ihnen in Tabelle 4.14 eine Übersicht über implizite und explizite Typwandlungen, die der Dokumentation der MS SQL Server-Datenbank entnommen ist: <https://learn.microsoft.com/en-us/sql/t-sql/data-types/data-type-conversion-database-engine>

Alle vier hier behandelten Datenbanken verwenden jedoch andere Regeln, um eine implizite Typkonvertierung durchzuführen. Das heißt, die Ergebnisse aus Tabelle 4.13 sind in keiner Form als Indikator für die Qualität der Datenbanken zu werten. Sie sollen Sie lediglich darauf aufmerksam machen, dass Datenbanken unterschiedlicher Hersteller andere Regeln zur Typkonvertierung nutzen, und Sie zudem für dieses Thema sensibilisieren. Beachten Sie, dass es ja auch noch viele andere Hersteller von Datenbanken gibt, die ebenfalls möglicherweise andere Regeln für Typkonvertierungen umgesetzt haben.



From/To	binary	varbinary	char	varchar	nchar	nvarchar	datetime	smalldatetime	decimal	numeric	float	real	bigint	int(INT4)	smallint(INT2)	tinyint(INT1)	money	smallmoney	bit	timestamp	uniqueidentifier	image	ntext	text	sql_variant
binary		•	•	•	•	•	•	•	•	•	×	×	•	•	•	•	•	•	•	•	•	×	×	•	
varbinary	•		•	•	•	•	•	•	•	•	×	×	•	•	•	•	•	•	•	•	•	×	×	•	
char	Δ	Δ		•	•	•	•	•	•	•	•	•	•	•	•	•	Δ	Δ	•	Δ	•	•	•	•	
varchar	Δ	Δ	•		•	•	•	•	•	•	•	•	•	•	•	•	Δ	Δ	•	Δ	•	•	•	•	
nchar	Δ	Δ	•	•		•	•	•	•	•	•	•	•	•	•	•	Δ	Δ	•	Δ	•	×	•	•	
nvarchar	Δ	Δ	•	•	•		•	•	•	•	•	•	•	•	•	•	Δ	Δ	•	Δ	•	×	•	•	
datetime	Δ	Δ	•	•	•	•		•	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	×	×	×	•	
smalldatetime	Δ	Δ	•	•	•	•			Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	Δ	×	×	×	•	
decimal	•	•	•	•	•	•	•	•	*	*	•	•	•	•	•	•	•	•	•	•	×	×	×	•	
numeric	•	•	•	•	•	•	•	•	*	*	•	•	•	•	•	•	•	•	•	•	×	×	×	•	
float	•	•	•	•	•	•	•	•	•	•		•	•	•	•	•	•	•	•	×	×	×	×	•	
real	•	•	•	•	•	•	•	•	•	•	•		•	•	•	•	•	•	•	×	×	×	×	•	
bigint	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	×	×	×	•	
int(INT4)	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	•	×	×	×	•	

Tabelle 4.14 Implizite und explizite Typkonvertierung im MS SQL-Server

From/To	binary	varbinary	char	varchar	nchar	nvarchar	datetime	smalldatetime	decimal	numeric	float	real	bigint	int(INT4)	smallint(INT2)	tinyint(INT1)	money	smallmoney	bit	timestamp	uniqueidentifier	image	ntext	text	sql_variant
binary	•																								
varbinary		•																							
char			•																						
varchar				•																					
nchar					•																				
nvarchar						•																			
datetime							•																		
smalldatetime								•																	
decimal									•																
numeric										•															
float											•														
real												•													
bigint													•												
int(INT4)														•											
smallint(INT2)															•										
tinyint(INT1)																•									
money																	•								
smallmoney																		•							
bit																			•						
timestamp																				•					
uniqueidentifier																					•				
image																						•			
ntext																							•		
text																								•	
sql_variant																									

Δ explizite Typkonvertierung; • implizite Typkonvertierung; × Typkonvertierung nicht erlaubt;
 * Benötigt ein explizites CAST, um Rundungsfehler und fehlende Genauigkeit einer impliziten
 Typkonvertierung zu verhindern.

Tabelle 4.14 Implizite und explizite Typkonvertierung im MS SQL-Server (Forts.)



Zusammenfassung: Spaltenwerte in andere Datentypen umwandeln

Wenn Sie mit Spaltenwerten arbeiten, müssen Sie sicherstellen, dass sie zueinander passen. Dies kommt z. B. bei Berechnungen vor oder auch bei Vergleichen.

SQL bietet die Möglichkeit, solche Unstimmigkeiten mit der **CAST**-Funktion zu bereinigen, die einen Wert eines Typs in einen anderen Typ umwandelt.

Die umzuwandelnden Werte müssen allerdings auch tatsächlich konvertierbar sein. Sie können z. B. keine Kommazahl in eine ganze Zahl umwandeln. Hier gibt es allerdings Ausnahmen zu beachten, die von Datenbank zu Datenbank unterschiedlich sein können.

Es wird zwischen *impliziter* und *expliziter Typkonvertierung* unterschieden. Wenn die Datentypen innerhalb einer Operation oder eines Vergleichs in einer **WHERE**-Klausel nicht kompatibel sind und keine Konvertierungsfunktion verwendet wird, versucht eine Datenbank automatisch, die Datentypen passend zu machen. Die implizite Typkonvertierung birgt immer die Gefahr, dass Ungenauigkeiten entstehen.

Eine explizite Typkonvertierung wird mit der Skalarfunktion **CAST** realisiert. Hier wird ein Spalten- oder Literalwert aus einem gegebenen Datentyp in einen Zieldatentyp konvertiert, der explizit angegeben werden muss.

4.3.3 Übungen

An dieser Stelle können Sie das Gelernte in einigen Übungen anwenden.

Übung 1

Verwenden Sie die Kurzform einer **SELECT**-Anweisung ohne **FROM**-Klausel, um ein Textliteral mit dem Wert '5' zu einem ganzzahligen numerischen Literal 20 zu addieren. Wenden Sie die explizite Typkonvertierung nach **INTEGER** an. Die Leser, die eine MySQL- oder MariaDB-Datenbank verwenden, nutzen bitte den Datentyp **UNSIGNED INTEGER**.

Übung 2

Verwenden Sie in dieser Übung ebenfalls die Kurzform der **SELECT**-Anweisung, und addieren Sie zu einer ganzzahligen Zahl 1000000 den Textwert '5.8'. Hierbei verlassen Sie sich auf die implizite Typkonvertierung des Datenbanksystems. Schildern Sie, welches Ergebnis die Addition liefert, und begründen Sie, wie dieses Ergebnis zustande kommt.

Lösung zu Übung 1

```
SELECT CAST('5' AS UNSIGNED INTEGER) + 20 AS summe; /* MySQL, MariaDB */
SELECT CAST('5' AS INTEGER) + 20 AS summe; /* MS SQL Server, PostgreSQL */
```

Listing 4.3 Einen Textwert »5« in eine ganze Zahl umwandeln

Lösung zu Übung 2

```
SELECT 100000 + '5.8' AS summe;
```

Listing 4.4 Implizite Typkonvertierung verwenden

Leser, die mit PostgreSQL oder MS SQL Server arbeiten, werden die Beobachtung machen, dass die Datenbanken hier keine implizite Typkonvertierung durchführen, sondern eine Fehlermeldung ausgeben.

Die Datenbanksysteme MySQL und MariaDB sehen das etwas anders: Sie liefern Ihnen ein Ergebnis, wie Sie es von der Berechnung erwarten, und wenden eine implizite Typkonvertierung an.

4.4 Einfache Tabellen mit CREATE TABLE erstellen

In diesem Abschnitt widmen wir uns ganz dem Thema der Tabellenerstellung mit der Anweisung **CREATE TABLE** und wie wir Tabellen mit **DROP TABLE** wieder löschen.

4.4.1 Zielstruktur der Tabelle

In diesem Beispiel geht es zunächst nur darum, eine einfache Tabelle mit einer **CREATE TABLE**-Anweisung in der Datenbank zu erstellen. Weitere Möglichkeiten, eine Tabelle zu erstellen, finden Sie in [Kapitel 7](#) und [Kapitel 8](#).

Die Tabelle, die wir erstellen, soll die Bezeichnung *dienstwagen* erhalten. Bei der Planung gehen wir davon aus, dass ein Dienstwagen einem Mitarbeiter fest zugeordnet wird. Außerdem gehen wir davon aus, dass jeder Dienstwagen, der einem Mitarbeiter zugeordnet wird, auch zur Kostenstelle des Mitarbeiters gehört.

So ergeben sich schon vier unterschiedliche Kategorien von Informationen, die für die Tabelle *dienstwagen* abgeleitet werden können, nämlich die Kategorien »Dienstwagen«, »Kostenstelle«, »Fahrzeugtyp« und »Mitarbeiter«. Aus diesen Kategorien ergeben sich die Informationen, die in den Spalten der zu erstellenden Tabelle gespeichert werden sollen.

Der Kategorie »Dienstwagen« ordnen wir die Spalten *dienstwagenid*, *kennzeichen* und *erstzulassung* zu. Zur Kategorie »Kostenstelle« gehört die Spalte *kostenstelle*. Der

»Fahrzeugtyp« wird durch die Spalten *hersteller*, *modell*, *kw*, *motor* und *getriebe* abgebildet. Jetzt fehlt noch der »Mitarbeiter«, dem der Dienstwagen zugewiesen wird. Ihn bilden wir in der Spalte *mitarbeiter* ab.

Den Spalten der zu erstellenden Tabelle *dienstwagen* müssen jetzt die passenden Datentypen zugeordnet werden (siehe [Tabelle 4.15](#)). In der zweiten Spalte der Tabelle sehen Sie die Zuordnung der Datentypen zu den jeweiligen Spaltenbezeichnungen. Die Datentypen wurden passend zu den Informationen ausgewählt, die in den Spalten gespeichert werden sollen.

Die dritte Spalte mit der Bezeichnung *primärschlüssel* legt den eindeutigen ganzzahligen Primärschlüssel der Tabelle *dienstwagen* fest. Der Spalte mit der Bezeichnung *dienstwagenid* wurde entsprechend ein X zugeordnet.

In der vierten Spalte sehen Sie Beispielwerte für die jeweiligen Spalten.

Spaltenbezeichnung	Datentyp	Primärschlüssel	Beispielwert
<i>dienstwagenid</i>	SMALLINT	X	1
<i>kennzeichen</i>	CHAR(9)		BN-XX-000
<i>erstzulassung</i>	DATE		2014-01-21
<i>kostenstelle</i>	SMALLINT		1
<i>hersteller</i>	VARCHAR(200)		Toyota
<i>modell</i>	VARCHAR(200)		Yaris
<i>leistung</i>	DECIMAL(5,2)		110.48
<i>motor</i>	VARCHAR(100)		Diesel
<i>getriebe</i>	VARCHAR(100)		Manuell
<i>mitarbeiter</i>	VARCHAR(300)		Meier

Tabelle 4.15 Struktur der Tabelle »dienstwagen«

4.4.2 Tabellen mit der CREATE TABLE-Anweisung anlegen

Legen Sie nun mit **CREATE TABLE** die Tabelle *dienstwagen* wie folgt an:

```
CREATE TABLE dienstwagen(  
  dienstwagenid SMALLINT PRIMARY KEY,  
  kennzeichen CHAR(9),  
  erstzulassung DATE,  
  kostenstelle SMALLINT,  
  hersteller VARCHAR(200),
```



```
modell VARCHAR(200),
leistung DECIMAL(5,2),
motor VARCHAR(100),
getriebe VARCHAR(100),
mitarbeiter VARCHAR(300)
);
```

Listing 4.5 Eine Tabelle mit einer CREATE TABLE-Anweisung erstellen

Die SQL-Anweisung beginnt mit den Schlüsselwörtern **CREATE TABLE**. Hinter diesen Schlüsselwörtern geben wir die Tabellenbezeichnung an. Es folgt eine geöffnete Klammer, hinter der wir die Spalten notieren, die die Tabelle enthalten soll. Hinter den Spaltenbezeichnungen geben wir jeweils den Datentyp an, wie wir ihn für die jeweilige Spalte festgelegt haben.

Außer in der Spalte *dienstwagenid* bestehen die Spaltendefinitionen bisher nur aus der Spaltenbezeichnung und dem Datentyp. Mehrere Angaben trennen Sie jeweils durch ein Komma. So sehen Sie, dass in der ersten Spaltendefinition hinter der Datentypangabe für die Spalte *dienstwagenid* die Schlüsselwörter **PRIMARY KEY** folgen, worüber Sie in [Abschnitt 4.4.4](#) mehr erfahren. Hinter der letzten Spaltendefinition ist kein Komma erforderlich, da hier ja keine weitere Spaltendefinition in der Liste der Spaltendefinitionen angegeben wird.

Ganz am Ende steht eine schließende Klammer, die die **CREATE TABLE**-Anweisung abschließt. Kurz gesagt handelt es sich um eine geklammerte Liste von Spaltendefinitionen, in der Spaltenbezeichnungen und Datentypen kommasepariert festgelegt werden.

Wenn Sie [Listing 4.5](#) ausführen, wird die Tabelle *dienstwagen*, mit der Struktur, die Sie in [Tabelle 4.16](#) sehen, in der *uebungsdatenbank* erstellt.

dienstwagenid	kennzeichen	erstzulassung	kostenstelle	hersteller	...

Tabelle 4.16 Die Struktur der Tabelle »dienstwagen« in der Übungsdatenbank

Mit folgender **SELECT**-Anweisung können Sie überprüfen, ob die Tabelle auch tatsächlich erfolgreich angelegt wurde. Hierzu nutzen wir die Tabelle *tables* mit der vorangestellten Bezeichnung *information_schema*.

```
SELECT table_name FROM information_schema.tables
WHERE table_name='dienstwagen';
```

Listing 4.6 Prüfung, ob die Tabelle »dienstwagen« in der Datenbank erstellt wurde

TABLE_NAME
<i>dienstwagen</i>

Tabelle 4.17 Ergebnistabelle für die Prüfabfrage aus Listing 4.6

Für die Spalten *dienstwagenid*, *kennzeichen* und *erstzulassung* fügen wir folgende Werte mit der Anweisung **INSERT** ein:

```
INSERT INTO dienstwagen
(dienstwagenid,kennzeichen,erstzulassung)
VALUES (1, 'XX-XX-XX1', '2014-05-04');
```

```
INSERT INTO dienstwagen
(dienstwagenid,kennzeichen,erstzulassung)
VALUES (2, 'XX-XX-XX1', '2014-08-05');
```

Listing 4.7 Datensätze in die Tabelle »dienstwagen« einfügen

Beachten Sie, dass die Spaltenwerte in den **INSERT**-Anweisungen exakt den Zuordnungen der Datentypen zu den Spalten entsprechen. Das bedeutet, dass der Spalte *dienstwagenid* auch ein Wert aus dem Wertebereich des Datentyps **SMALLINT** zugeordnet wird. Der Spalte *kennzeichen* wird eine neunstellige Zeichenkette übergeben, die exakt der Datentypzuordnung **CHAR(9)** entspricht. Der Spalte *erstzulassung* übergeben wir ein Datum in der Form, wie es der Datentyp **DATE** von uns gemäß dem Standard ISO 8601 erwartet.

Prüfen wir abschließend den Inhalt der Tabelle *dienstwagen* mit einer einfachen **SELECT**-Abfrage:

```
SELECT dienstwagenid,kennzeichen,erstzulassung
FROM dienstwagen;
```

Listing 4.8 Datensätze der Tabelle »dienstwagen« abfragen

dienstwagenid	kennzeichen	erstzulassung
1	XX-XX-XX1	2014-05-04
2	XX-XX-XX1	2014-08-05

Tabelle 4.18 Ergebnistabelle der Prüfabfrage für zwei eingefügte Datensätze

Es wurden zwei Zeilen in die Tabelle *dienstwagen* eingefügt. Die Spaltenwerte wurden gemäß der **INSERT**-Anweisung in die jeweiligen Spalten eingetragen.

Was passiert, wenn wir einen ungültigen Wert für einen Datentyp eingeben? Das können wir ausprobieren, indem wir durch eine fehlerhafte Eingabe mit einer **INSERT**-Anweisung eine Datentypverletzung provozieren.

In [Listing 4.9](#) notieren wir in der **VALUES**-Klausel für die Spalte *erstzulassung* einen Spaltenwert, der der Zeichenkette *abc* entspricht. Obwohl die Datenbank ein Datum vom Typ **DATE** erwartet, ordnen wir also eine Zeichenkette zu:

```
INSERT INTO dienstwagen
(dienstwagenid,kennzeichen,erstzulassung)
VALUES (3, 'XX-XX-XX1', 'abc');
```

Listing 4.9 Datentypverletzung mit einer **INSERT**-Anweisung provozieren

Die MySQL-Datenbank antwortet Ihnen mit einer Fehlermeldung und beschwert sich, dass es sich bei *abc* nicht um einen korrekten Datumswert handelt:

Error Code: 1292.

Incorrect date value: 'abc' for column 'erstzulassung' at row 1

Diese **INSERT**-Anweisung funktioniert auch nicht bei den anderen hier behandelten Datenbanken. Achten Sie also stets darauf, dass Sie die richtigen Spaltenwerte in der **VALUES**-Klausel der **INSERT**-Anweisung eintragen, wenn Sie Datensätze einfügen. Das gilt einerseits für die inhaltliche Richtigkeit, aber natürlich andererseits auch für die Übereinstimmung der Spaltenwerte mit den Datentypen, die Sie während der Tabellenerstellung den Spalten zugeordnet haben.

4.4.3 Tabellen mit einer **DROP**-Anweisung löschen

Der SQL-Sprachschatz hält für uns neben der **CREATE TABLE**-Anweisung auch die **DROP TABLE**-Anweisung bereit, mit der Sie Tabellen löschen. Die Anwendung ist einfach: Sie leiten die Anweisung mit den Schlüsselwörtern **DROP TABLE** ein und notieren dahinter die Bezeichnung der zu löschenden Tabelle:

```
DROP TABLE dienstwagen;
```

Listing 4.10 Eine Tabelle mit der **DROP TABLE**-Anweisung löschen

Führen Sie die Anweisung aus, um die Tabelle *dienstwagen* zu entfernen. Wir werden sie gleich im Anschluss erneut erstellen.

Mit der **DROP TABLE**-Anweisung können Sie jede Tabelle Ihrer Übungsdatenbank löschen. Gehen Sie in produktiven Datenbanken sehr behutsam mit der **DROP TABLE**-Anweisung um. Verwenden Sie sie nur, wenn Sie ganz sicher sind, dass eine Tabelle nicht mehr benötigt wird.

Ich habe Ihnen bereits dargestellt, dass eine Tabelle immer mit einem eindeutigen Schlüssel ausgestattet sein muss. Er dient zur Identifikation der einzelnen Zeilen bzw. Datensätze einer Tabelle. Sie haben bereits gelernt, wie Sie Tabellen mit einer Bezeichnung und einer Liste von Spaltenbezeichnungen und Datentypzuordnungen erstellen können.

4.4.4 Eine Tabelle mit einem Primärschlüssel ausstatten

Im Folgenden zeige ich Ihnen, wie Sie die Tabelle *dienstwagen* mit einer eindeutigen Schlüsselwertspalte erstellen, die jeder Zeile der Tabelle einen Schlüsselwert zuordnet und die Zeile damit identifizierbar macht.

Sie haben bereits in [Listing 4.5](#) eine Tabelle erstellt, die einen Primärschlüssel enthält. Rufen wir uns nochmals die erste Spaltendefinition in Erinnerung:

```
dienstwagenid SMALLINT PRIMARY KEY,
```

Neben dem Namen der Spalte und dem Datentyp **SMALLINT** werden die Schlüsselwörter **PRIMARY KEY** zugeordnet. Mit dieser Schlüsselwortfolge bestimmen Sie den eindeutigen Primärschlüssel einer Tabelle.

In der Praxis bedeutet das, dass Sie jeder Zeile nur einen eindeutigen ganzzahligen Wert in der Spalte *dienstwagenid* zuordnen können. Wenn Sie eine **PRIMARY KEY**-Einschränkung einer Spalte zuordnen, wacht die Datenbank darüber, dass die Schlüsselwerte immer eindeutig bleiben. Sollten Sie also versuchen, zwei Zeilen mit den gleichen Werten für die Spalte *dienstwagenid* einzufügen, so würde die Datenbank das Einfügen ablehnen.

Grundsätzlich können Sie jede beliebige Spalte eines beliebigen Datentyps als Primärschlüssel definieren. Hier ist natürlich darauf zu achten, dass Sie die Spalte auswählen, die als Schlüssel für die Zeilen einer Tabelle infrage kommt.

Verwenden Sie die **CREATE TABLE**-Anweisung aus [Listing 4.5](#), um die Tabelle *dienstwagen* in der Übungsdatenbank erneut anzulegen.

Unter Berücksichtigung des eindeutigen Primärschlüssels fügen wir zwei Zeilen mit den Primärschlüsselwerten 1 und 2 in der Spalte *dienstwagenid* in die Tabelle *dienstwagen* ein:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen)
VALUES (1, 'XX-XX-XX1');
INSERT INTO dienstwagen (dienstwagenid,kennzeichen)
VALUES (2, 'XX-XX-XX2');
```

Listing 4.11 Zeilen mit eindeutigen Schlüsselwerten in die Tabelle »dienstwagen« einfügen

Der Spalte *kennzeichen* ordnen wir die Werte XX-XX-XX1 und XX-XX-XX2 zu. Den anderen Spalten weisen wir keine Werte zu. Das ist hier auch nicht erforderlich, da für diese Spalten während der Tabellenerstellung keine weiteren Einschränkungen festgelegt wurden. Es ist nämlich möglich, die Eingaben noch weiter einzuschränken (mehr dazu folgt in [Abschnitt 4.5](#)).

Die soeben eingefügten Datensätze werden wir als Nächstes mit einem **SELECT**-Befehl aus der Tabelle *dienstwagen* abfragen:

```
SELECT dienstwagenid,kennzeichen FROM dienstwagen;
```

Listing 4.12 Zeilen einer Tabelle mit eindeutigem Schlüssel abfragen

In [Tabelle 4.19](#) sehen Sie das Ergebnis der Prüfabfrage. Den eingefügten Zeilen wurden für die Spalte *dienstwagenid* die Spaltenwerte 1 und 2 zugeordnet.

dienstwagenid	kennzeichen
1	XX-XX-XX1
2	XX-XX-XX2

Tabelle 4.19 Prüfabfrage für die eindeutig zugewiesenen Werte aus [Listing 4.11](#)

Ich habe Ihnen bereits geschildert, dass die Datenbank, wenn ein Primärschlüssel in der Tabellenerstellungsanweisung festgelegt wurde, darüber wacht, dass die Schlüsselwerte der Primärschlüsselspalte immer eindeutig sind. Im nächsten Schritt werden wir diese Garantie auf den Prüfstand stellen, indem wir versuchen, einen weiteren Datensatz mit dem Primärschlüsselwert 2 in die Tabelle *dienstwagen* einzufügen:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen)
VALUES (2, 'XX-XX-XX3');
```

Listing 4.13 Provokation einer Primärschlüsselverletzung

Die MySQL-Datenbank liefert Ihnen eine Fehlermeldung, wenn Sie versuchen, einen Primärschlüsselwert in die Tabelle *dienstwagen* einzutragen, der bereits existiert:

```
Error Code: 1062. Duplicate entry '2' for key 'PRIMARY'
```

Die Datenbank nimmt also sehr gewissenhaft die Aufgabe wahr, über die Eindeutigkeit von Primärschlüsselwerten zu wachen.

Wie sieht es aus, wenn Sie versuchen, einen **NULL**-Wert als Primärschlüsselwert anzugeben? Das probieren wir im nächsten Beispiel aus, indem wir in [Listing 4.14](#) versuchen, einen Datensatz einzufügen, dessen Primärschlüsselwert für die Spalte *dienstwagenid* in der **INSERT**-Anweisung gleich **NULL** ist:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen)
VALUES (NULL, 'XX-XX-XX3');
```

Listing 4.14 Versuch, einen NULL-Wert als Primärschlüsselwert einzufügen

Die Datenbank reagiert bei dem Versuch, einen **NULL**-Wert als Primärschlüsselwert in eine Primärschlüsselspalte einzufügen, mit der Fehlermeldung, dass der Spaltenwert der Spalte *dienstwagenid* nicht **NULL** sein darf:

Error Code: 1048. Column 'dienstwagenid' cannot be null

Da wir im nächsten Abschnitt die Tabelle *dienstwagen* erneut anlegen, entfernen Sie die Tabelle nun mit der **DROP**-Anweisung aus [Listing 4.10](#).



Zusammenfassung: Primärschlüssel

Jede Zeile einer Tabelle sollte unbedingt über einen eindeutigen Schlüssel identifizierbar sein. Um dies sicherzustellen, werden die Spalten, die als Schlüsselspalten infrage kommen, bereits während der Tabellenerstellung mit einer **CREATE TABLE**-Anweisung als Primärschlüssel festgelegt.

Ein Primärschlüsselwert muss innerhalb der Primärschlüsselspalte immer eindeutig sein. Das bedeutet, dass er auf keinen Fall doppelt oder mehrfach vorkommen darf. Die Datenbank wacht über die Spaltenwerte, die als Primärschlüssel festgelegt wurden, und stellt sicher, dass jeder Primärschlüsselwert eindeutig ist.

NULL-Werte sind nicht als Primärschlüsselwert zulässig, da es sich hier um nicht definierte Werte handelt.

4.4.5 Automatisch hochzählende numerische Primärschlüsselspalten festlegen

Beim Thema Primärschlüssel sollten die sogenannten *Autoinkrement-Schlüssel* nicht fehlen. Wenn Sie eine numerische Spalte als Primärschlüsselspalte verwenden, die ganzzahlige Werte als Schlüsselwerte speichert, so können Sie in der **CREATE TABLE**-Anweisung einen Autoinkrement-Schlüssel verwenden, der beim Einfügen eines neuen Datensatzes in eine Tabelle den Wert dieser Spalte automatisch um den Wert 1 erhöht.

Beachten Sie dabei, dass diese automatische Hochzählung nur mit Zahlenwerten funktioniert. Eine Spalte, die mit Datums- oder Zeichenwerten gefüllt wird, können Sie zwar auch als Primärschlüsselspalte verwenden, Sie müssen dann jedoch auf die automatische Hochzählung verzichten.

Eine Autoinkrement-Primärschlüsselspalte für eine MySQL- und MariaDB-Datenbank legen Sie wie folgt fest:

```
CREATE TABLE dienstwagen(  
    dienstwagenid SMALLINT AUTO_INCREMENT PRIMARY KEY,  
    kennzeichen CHAR(9),  
    erstzulassung DATE  
);
```

Listing 4.15 Eine Autoinkrement-Spalte für eine Tabelle festlegen (MySQL, MariaDB)

Unterschiede in PostgreSQL und MS SQL

Wenn Sie eine PostgreSQL-Datenbank verwenden, nutzen Sie die Anweisung aus [Listing 4.16](#), um die Tabelle *dienstwagen* mit einem automatisch hochzählenden eindeutigen Schlüssel zu versehen. Bei dem Datentyp **SERIAL** handelt es sich nicht um einen einfachen ganzzahligen Datentyp, sondern um einen automatisch hochzählenden ganzzahligen Datentyp.

```
CREATE TABLE dienstwagen(  
    dienstwagenid SERIAL PRIMARY KEY,  
    kennzeichen CHAR(9),  
    erstzulassung DATE  
);
```

Listing 4.16 Eine Autoinkrement-Spalte für eine Tabelle festlegen (PostgreSQL)

Die MS SQL Server-Datenbank bietet, wie in [Listing 4.17](#) angegeben, das Schlüsselwort **IDENTITY**(startwert, inkrementwert). Das Schlüsselwort **IDENTITY** geben Sie zwischen dem Datentyp und der **PRIMARY KEY**-Festlegung an. In der Klammer notieren Sie den initialen numerischen Startwert und durch ein Komma getrennt die Schrittweite, die durch die automatische Schlüsselvergabe genutzt wird, um Werte hochzuzählen. Löschen Sie die Tabelle *dienstwagen*, bevor Sie die neue Tabelle mit der Autoinkrement-Primärschlüsselspalte anlegen.

```
CREATE TABLE dienstwagen(  
    dienstwagenid SMALLINT IDENTITY(1,1) PRIMARY KEY,  
    kennzeichen CHAR(9),  
    erstzulassung DATE  
);
```

Listing 4.17 Eine Autoinkrement-Spalte für eine Tabelle festlegen (MS SQL Server)

Wir nutzen hier eine etwas schlankere Version der Tabelle *dienstwagen*. Um für eine MySQL- oder MariaDB-Datenbank eine Autoinkrement-Spalte als Primärschlüsselspalte zu vereinbaren, nutzen Sie hinter der Datentypangabe das Schlüsselwort **AUTO_INCREMENT**. Dann erst folgen die Schlüsselwörter **PRIMARY KEY**.

Danach fügen wir zwei Datensätze mit den **INSERT**-Anweisungen aus [Listing 4.18](#) ein. Beachten Sie, dass die Primärschlüsselwerte von der Datenbank automatisch eingetragen werden.

```
INSERT INTO dienstwagen (kennzeichen,erstzulassung)
VALUES ('XX-XX-XX1', '2014-04-01');
```

```
INSERT INTO dienstwagen (kennzeichen,erstzulassung)
VALUES ('XX-XX-XX2', '2014-06-20');
```

Listing 4.18 Datensätze in Tabellen mit Autoinkrement-Primärschlüsselspalte einfügen

Prüfen Sie nun mit einer **SELECT**-Abfrage, ob die automatische Vergabe von Primärschlüsselwerten tatsächlich funktioniert hat:

```
SELECT dienstwagenid,kennzeichen,erstzulassung
FROM dienstwagen;
```

Listing 4.19 Prüfabfrage für automatisch vergebene Primärschlüsselwerte

Sie werden sehen, dass die Primärschlüsselwerte jeweils automatisch durch die Datenbank hochgezählt wurden:

dienstwagenid	kennzeichen	erstzulassung
1	XX-XX-XX1	2014-04-01
2	XX-XX-XX2	2014-06-20

Tabelle 4.20 Ergebnis der Abfrage aus [Listing 4.19](#)



Zusammenfassung: Automatische ganzzahlige Primärschlüsselwerte generieren

Während der Tabellenerstellung können Sie mit einer **CREATE TABLE**-Anweisung festlegen, ob automatisch ein ganzzahlig hochzählender Primärschlüssel eingefügt werden soll.

Um dies zu erreichen, wendet die Datenbank den folgenden Ausdruck an, um inkrementell die Primärschlüsselwerte hochzuzählen:

$$X = X + 1$$

Die automatische Schlüsselwertgenerierung wird von Datenbank zu Datenbank unterschiedlich umgesetzt. In den Tabellenerstellungsanweisungen werden für die hier behandelten Datenbanken die folgenden Schlüsselwörter angewendet:

- ▶ AUTO_INCREMENT für eine MySQL- oder MariaDB-Datenbank
- ▶ SERIAL für eine PostgreSQL-Datenbank
- ▶ IDENTITY für eine MS SQL Server-Datenbank

Wenn Sie Primärschlüsselspalten verwenden, die z. B. vom Typ **CHAR**, **VARCHAR** oder **DATE** sind, steht Ihnen die Autoinkrement-Funktionalität nicht zur Verfügung.

4.4.6 Reservierte Schlüsselwörter

Jede Datenbank legt fest, welche Wörter nicht als Bezeichner für Datenbankobjekte verwendet werden dürfen. Das gilt auch für Datenbanken, die hier nicht explizit besprochen werden.

Wenn Sie die Beispiele dieses Buchs nachverfolgen, können Sie natürlich allen Datenbankobjekten andere Namen geben. Vielleicht gefallen Ihnen andere Tabellennamen besser als die hier vorgeschlagenen. Diese können Sie sicherlich ändern, obwohl ich Ihnen das nicht empfehlen würde. Wenn Sie im Rahmen dieser Einführung andere Tabellenbezeichnungen verwenden sollten, kann das beim praktischen Nachvollziehen der Übungen zu verwirrenden Nebeneffekten führen. Möglich ist es aber natürlich.

Bevor Sie beginnen, Tabellen anders zu benennen, als ich es hier vorschlage, muss ich Ihnen eine Liste von reservierten Schlüsselwörtern vorstellen, die Sie nicht verwenden dürfen. In der Regel handelt es sich dabei um Wörter, die dem SQL-Sprachschatz entstammen. Es ist also wirklich keine gute Idee, eine Tabelle *select* zu nennen.

Die reservierten Schlüsselwörter von MySQL finden Sie in der offiziellen Dokumentation unter <https://dev.mysql.com/doc/refman/9.1/en/keywords.html>.

Zusammenfassung: Tabellen mit einer CREATE TABLE-Anweisung erstellen

Sie erstellen neue Tabellen in einer Datenbank mit der **CREATE TABLE**-Anweisung. Bevor eine Tabelle in einer Datenbank neu angelegt wird, sind einige Fragen zu beantworten. Hierzu zählen:

- ▶ Welche Spalten soll die Tabelle enthalten?
- ▶ Welche Datentypen sollen den Spalten zugeordnet werden?
- ▶ Welche Spalte soll als Primärschlüsselspalte verwendet werden?
- ▶ Soll die Primärschlüsselspalte mit einer Autoinkrement-Funktion (nur für numerische Datentypen möglich) versehen werden?

Für den Tabellennamen und die Spaltenbezeichner dürfen Sie keine reservierten Schlüsselwörter verwenden. Wenn eine Tabelle nicht mehr erforderlich ist, kann sie mit einer **DROP TABLE**-Anweisung gelöscht werden.



4.4.7 Übungen

Übung 1

In dieser Übung erstellen Sie eine Tabelle, die sich aus der Tabelle *dienstwagen* ableiten lässt. Dort finden sich folgende Spalten: *hersteller*, *modell*, *leistung*, *motor* und *getriebe*. Hier bietet es sich an, eine Tabelle *fahrzeugtyp* anzulegen, in der die Eigenschaften einzelner Fahrzeugtypen zusammengefasst werden. In Tabelle 4.21 sehen Sie die Datentypen, die wir den Spalten zuordnen können.

Spaltenbezeichnung	Datentyp	Primärschlüssel	Beispielwert
<i>typid</i>	SMALLINT	X	1
<i>hersteller</i>	VARCHAR(200)		Toyota
<i>modell</i>	VARCHAR(200)		Yaris
<i>leistung</i>	DECIMAL(5,2)		110.48
<i>motor</i>	VARCHAR(100)		Diesel
<i>getriebe</i>	VARCHAR(100)		Manuell

Tabelle 4.21 Tabellenstruktur für die anzulegende Tabelle »fahrzeugtyp«

Legen Sie mit einer `CREATE TABLE`-Anweisung eine Tabelle *fahrzeugtyp* an, die mit den Spalten und Datentypzuordnungen aus Tabelle 4.21 ausgestattet ist. Vergessen Sie auch nicht, der Spalte *typid* die Primärschlüsseinschränkung zuzuweisen, so wie es in der Tabellenstruktur angegeben ist.

Übung 2

Fügen Sie zwei Zeilen mit beliebigen Fahrzeugtypen in die neu erstellte Tabelle *fahrzeugtyp* ein. Gleich im Anschluss fragen Sie mit einer `SELECT`-Abfrage die eingefügten Zeilen der Tabelle *fahrzeugtyp* ab.

Übung 3

Als Nächstes löschen Sie die soeben erstellte Tabelle *fahrzeugtyp* wieder.

Lösung zu Übung 1

```
CREATE TABLE fahrzeugtyp(  
  typid SMALLINT PRIMARY KEY,  
  hersteller VARCHAR(200),  
  modell VARCHAR(200),  
  leistung DECIMAL(5,2),
```

```
motor VARCHAR(100),  
getriebe VARCHAR(100)  
);
```

Listing 4.20 Mit einer CREATE TABLE-Anweisung eine einfache Tabelle erstellen

Lösung zu Übung 2

```
INSERT INTO fahrzeugtyp  
VALUES (1, 'VW', 'Polo', 100.78, 'Benzin', 'Automatik');  
  
INSERT INTO fahrzeugtyp  
VALUES (2, 'Toyota', 'Yaris', 75.00, 'Benzin', 'Manuell');  
  
SELECT typid, hersteller, modell, leistung, motor, getriebe  
FROM fahrzeugtyp;
```

Listing 4.21 Datensätze in die neue Tabelle einfügen und abfragen

Lösung zu Übung 3

```
DROP TABLE fahrzeugtyp;
```

Listing 4.22 Eine Tabelle mit einer DROP TABLE-Anweisung löschen

4.5 Spalten Einschränkungen (CONSTRAINTS) zuordnen

Bisher haben Sie nur eine Einschränkung kennengelernt, die Sie Spalten zuordnen können. Es handelt sich um die **PRIMARY KEY**-Einschränkung, die einen eindeutigen Schlüsselwert pro Zeile bzw. Datensatz sicherstellt. Wie sieht es aus, wenn Sie wünschen, dass ein Spaltenwert immer eingetragen wird? Wie können Sie festlegen, dass ein Spaltenwert immer eindeutig ist, auch wenn es sich nicht um eine Schlüsselwertspalte handelt? Können wir bereits während der Tabellenerstellung Spalten mit Standardwerten ausstatten? Diese Fragen kann ich mit einem klaren »Ja« beantworten. Wie das funktioniert, erfahren Sie im Folgenden.

4.5.1 Spalten als Pflichtfelder (NOT NULL) definieren

Wir bleiben bei unserem Beispiel und nutzen wieder die Tabelle *dienstwagen*. Sie soll nun mit Pflichtfeldern versehen werden. Wenn Sie die Tabelle im Rahmen des Themas Primärschlüssel angelegt haben, löschen Sie sie mit einer einfachen **DROP**-Anweisung wieder.

Die Spalten *kennzeichen* und *erstzulassung* sollen in der Tabellenerstellungsanweisung als Pflichtfelder definiert werden. In [Listing 4.23](#) sehen Sie in Zeile 3 und 4 die Definitionen der Spalten *kennzeichen* und *erstzulassung*:

```
CREATE TABLE dienstwagen(  
    dienstwagenid SMALLINT PRIMARY KEY,  
    kennzeichen CHAR(9) NOT NULL,  
    erstzulassung DATE NOT NULL,  
    kostenstelle SMALLINT NULL,  
    hersteller VARCHAR(200) NULL,  
    modell VARCHAR(200) NULL,  
    leistung DECIMAL(5,2) NULL,  
    motor VARCHAR(100) NULL,  
    getriebe VARCHAR(100) NULL,  
    mitarbeiter VARCHAR(300) NULL  
);
```

Listing 4.23 Pflichtfelder in einer Tabellenerstellungsanweisung mit NOT NULL definieren

Um diese Spalten zu Pflichtfeldern zu machen, notieren Sie hinter dem Datentyp die Schlüsselwörter **NOT NULL**. Sie geben also an, dass die Spaltenwerte dieser Spalten nie undefiniert sein dürfen. Somit erzwingen Sie, dass in einer **INSERT**- oder **UPDATE**-Anweisung niemals **NULL**-Werte in diese Spalten eingefügt werden können. Oder denken Sie an einen Einfügevorgang mit einer **INSERT**-Anweisung, die mit einer expliziten Spaltenangabe verwendet wird: Spalten, die nicht explizit angegeben werden und somit auch keine Spaltenwertzuordnung erhalten, werden automatisch mit **NULL**-Werten ausgestattet.

Wie verhalten sich Spalten, denen keine **NOT NULL**-Einschränkung zugeordnet ist? Standardmäßig werden Spalten in einer **CREATE TABLE**-Anweisung mit der **NULL**-Einschränkung versehen. Diese Einschränkung muss nicht explizit angegeben werden und erlaubt **NULL**-Werte in Einfüge- und Änderungsoperationen mit einer **INSERT**- oder einer **UPDATE**-Anweisung. In der Tabellenerstellungsanweisung aus [Listing 4.24](#) sind den Spalten, die **NULL**-Werte in einer **INSERT**- oder **UPDATE**-Anweisung zulassen, **NULL**-Einschränkungen hinter der Datentypzuordnung zugewiesen.

Eigentlich ist diese explizite Angabe einer **NULL**-Einschränkung nicht erforderlich, da sie standardmäßig angewendet wird, wenn keine explizite **NOT NULL**-Einschränkung angegeben wurde. Die **NULL**-Einschränkung gehört auch nicht zum SQL-Standard. Die meisten Datenbanken (auch die hier behandelten) nutzen diese Angabe, um diejenigen Spalten explizit auszeichnen zu können, die **NULL**-Werte zulassen sollen. In diesem Beispiel wird sie verwendet, um Ihnen den Unterschied zur **NOT NULL**-Anweisung zu zeigen.

Nachdem Sie die neue Tabelle *dienstwagen* mit der Tabellenerstellungsanweisung aus [Listing 4.23](#) in der Übungsdatenbank *uebungsdatenbank* angelegt haben, werden wir den Spalten *kennzeichen* und *erstzulassung* Datensätze zuordnen. Dies sehen Sie in [Listing 4.24](#):

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung)
VALUES (1, 'XX-XX-XX1', '2015-01-14');
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung)
VALUES (2, 'XX-XX-XX2', '2015-01-14');
```

Listing 4.24 Zeilen unter Berücksichtigung einer NOT NULL-Einschränkung einfügen

Überprüfen wir nun, ob die beiden Zeilen tatsächlich in die neue Tabelle *dienstwagen* eingefügt wurden:

```
SELECT dienstwagenid, kennzeichen, erstzulassung FROM dienstwagen;
```

Listing 4.25 Prüfabfrage zur INSERT-Anweisung

dienstwagenid	kennzeichen	erstzulassung
1	XX-XX-XX1	2015-01-14
2	XX-XX-XX2	2015-01-14

Tabelle 4.22 Ergebnis der Prüfabfrage aus [Listing 4.25](#)

Gemäß den **NOT NULL**-Einschränkungen haben wir für *kennzeichen* und *erstzulassung* Spaltenwerte eingefügt. Die nicht in der **INSERT**-Anweisung angegebenen Spalten wurden von der Datenbank mit **NULL**-Werten ausgestattet. Das können Sie einfach nachvollziehen, indem Sie die Prüfabfrage ein wenig ändern und alle Spalten der Tabelle *dienstwagen* abfragen.

Jetzt würden wir gerne wissen, ob die **NOT NULL**-Einschränkung erwartungsgemäß funktioniert. Um die **NOT NULL**-Einschränkung zu testen, lassen wir einfach die Spaltenangabe und den dazugehörigen Wert für die Spalte *erstzulassung* in der **INSERT**-Anweisung weg, obwohl wir die Spalte in der Tabellenerstellungsanweisung mit einer **NOT NULL**-Einschränkung zum Pflichtfeld gemacht haben:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen)
VALUES (3, 'XX-XX-X3');
```

Listing 4.26 INSERT-Anweisung zum Prüfen der NOT NULL-Einschränkung

Wenn Sie die Anweisung aus [Listing 4.26](#) ausgeführt haben, werden Sie von der Datenbank eine Fehlermeldung erhalten:

Error Code: 1364. Field 'erstzulassung' doesn't have a default value

Die Fehlermeldung mag zunächst ein wenig merkwürdig erscheinen. Bemängelt wird ja ein fehlender Standardwert. Die MySQL- und MariaDB-Datenbank gehen davon aus, dass die Zuordnung eines Standardwerts bei der **NOT NULL**-Einschränkung fehlt. Die PostgreSQL- und Microsoft SQL Server-Datenbank liefern hier sinngemäß in der Form »NULL-Werte sind für die Spalten xxx nicht zugelassen« eine treffendere Fehlermeldung.

Sie sehen also, dass die Einhaltung der **NOT NULL**-Einschränkung, die Sie der Spalte *erstzulassung* zugeordnet haben, von der Datenbank bei **INSERT**-Anweisungen sichergestellt wird.

Die **NOT NULL**-Einschränkung hat auch Auswirkungen auf die Verwendung der Aktualisierungsanweisung mit **UPDATE**. Wir könnten auf die Idee kommen, die Zeile mit dem Wert 1 für die Spalte *dienstwagenid* zu aktualisieren. Das Kennzeichen bleibt gleich, aber da ein neues Auto angeschafft wird, ist in Kürze ein neuer Eintrag in der Spalte *erstzulassung* zu erwarten. Zwischenzeitlich soll der Spalte *erstzulassung* der Wert **NULL** zugewiesen werden. Wenn Sie die **UPDATE**-Anweisung aus [Listing 4.27](#) ausführen, kommt es, wie zu erwarten, zu einer Fehlermeldung.

```
UPDATE dienstwagen SET erstzulassung=NULL
WHERE dienstwagenid=1;
```

Listing 4.27 Aktualisierung eines Spaltenwertes mit einer NULL-Wertzuzuweisung für eine Spalte mit **NOT NULL**-Einschränkung

Die MySQL-Datenbank reagiert auf Ihren Versuch, den Spaltenwert der Spalte *erstzulassung* mit einem **NULL**-Wert zu aktualisieren, mit folgender Fehlermeldung:

Error Code: 1048. Column 'erstzulassung' cannot be null



Zusammenfassung: Pflichtfelder

Überall da, wo Spaltenwerte unbedingt erforderlich sind, können Sie bereits während der Tabellenerstellung mit einer **NOT NULL**-Einschränkung sicherstellen, dass hier immer Werte eingetragen werden. Das gilt für Einfügeoperationen mit **INSERT** wie für Aktualisierungsoperationen mit einer **UPDATE**-Anweisung.

4.5.2 Spalten mit einer UNIQUE-Einschränkung versehen

Sie haben bereits Einschränkungen im Zusammenhang mit Primärschlüsseln kennengelernt, die sicherstellen, dass ein Wert in einer Spalte nur einmal vorkommt. Ähnlich funktioniert es, wenn Sie auch normalen Spalten eine Eindeutigkeits-einschränkung zuordnen möchten. Während der Tabellenerstellung nutzen Sie dazu die **UNIQUE**-Einschränkung.

Wenn es erforderlich ist, löschen Sie die Tabelle *dienstwagen* wieder, um sie diesmal mit einer **UNIQUE**-Einschränkung neu anzulegen.

In [Listing 4.28](#) sehen Sie in der letzten Spaltendefinition, bei der Spalte *mitarbeiter*, hinter der Angabe des Datentyps das Schlüsselwort **UNIQUE**:

```
CREATE TABLE dienstwagen(
  dienstwagenid SMALLINT PRIMARY KEY,
  kennzeichen CHAR(9) NOT NULL,
  erstzulassung DATE NOT NULL,
  kostenstelle SMALLINT,
  hersteller VARCHAR(200),
  modell VARCHAR(200),
  leistung DECIMAL(5,2),
  motor VARCHAR(100),
  getriebe VARCHAR(100),
  mitarbeiter VARCHAR(200) UNIQUE
);
```

Listing 4.28 Eine Tabelle mit einer UNIQUE-Einschränkung anlegen

Die **UNIQUE**-Einschränkung wird immer hinter einer **NULL**- oder **NOT NULL**-Einschränkung notiert, vorausgesetzt, diese Einschränkungen sind vorhanden.

Normalform

An dieser Stelle noch ein kleiner Hinweis: Die Spalte *mitarbeiter* der Tabelle *dienstwagen* erfüllt nicht das Kriterium der ersten Normalform. Was das ist, erfahren Sie in [Kapitel 8, »Datenmodelle optimieren \(Normalisierung\)«](#).

Unterschiede bei MySQL, PostgreSQL und MS SQL Server

Beachten Sie, dass die MS SQL Server-Datenbank **NULL**-Werte, die mit **UNIQUE** eingeschränkt werden, anders interpretiert, als es die Datenbanksysteme MySQL, MariaDB und PostgreSQL tun. Die MS SQL Server-Datenbank betrachtet **NULL**-Werte als gleiche Werte. Das bedeutet: Wenn Sie versuchen, mehrere **NULL**-Werte in die Spalte *mitar-*

beiter einzufügen, so würde dies von MS SQL Server abgelehnt. PostgreSQL-, Maria-DB- und MySQL-Datenbanken lassen mehrfache NULL-Werte für eine Spalte zu, die mit einer UNIQUE-Einschränkung versehen wurde.

Schauen Sie sich zunächst die UNIQUE-Einschränkung der Spalte *mitarbeiter* und ihre Auswirkungen auf INSERT- und UPDATE-Anweisungen an.

Als Erstes fügen wir in der neu angelegten Tabelle wieder zwei Datensätze ein – den Mitarbeitern Petra Schneider und Ralf Müller ordnen wir einen Dienstwagen zu:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung,mitarbeiter)
VALUES (1,'XX-XX-XX1','2015-01-01','Ralf Müller');

INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung,mitarbeiter)
VALUES (2,'XX-XX-XX2','2015-01-01','Petra Schneider');
```

Listing 4.29 Datensätze unter Beachtung der UNIQUE-Einschränkung einfügen

Wenn Sie beide INSERT-Anweisungen ausgeführt haben, prüfen Sie mit der Abfrage aus Listing 4.30, ob die Datensätze eingefügt wurden:

```
SELECT dienstwagenid,kennzeichen,erstzulassung,mitarbeiter
FROM dienstwagen;
```

Listing 4.30 Prüfabfrage für die Datensätze, die unter Berücksichtigung der UNIQUE-Klausel eingefügt wurden

In Tabelle 4.23 sehen Sie, dass die Abfrage erfolgreich war. Die Spaltenwerte, die der Spalte *mitarbeiter* zugewiesen wurden, sind eindeutig und entsprechen somit der UNIQUE-Einschränkung.

dienstwagenid	kennzeichen	erstzulassung	mitarbeiter
1	XX-XX-XX1	2015-01-01	Ralf Müller
2	XX-XX-XX2	2015-01-01	Petra Schneider

Tabelle 4.23 Ergebnis der Prüfabfrage aus Listing 4.30

Jetzt werden wir die UNIQUE-Einschränkung herausfordern und prüfen, ob sie tatsächlich keine doppelten Werte für die Spalte *mitarbeiter* zulässt. Dazu nutzen wir die beiden INSERT-Anweisungen aus Listing 4.31:


```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung,mitarbeiter)
VALUES (3, 'XX-XX-XX1', '2015-01-01', 'Ralf Müller');
```

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung,mitarbeiter)
VALUES (4, 'XX-XX-XX2', '2015-01-01', 'Petra Schneider');
```

Listing 4.31 Provokation einer UNIQUE-Einschränkungsverletzung

Die MySQL-Datenbank quittiert den Versuch, gleiche Spaltenwerte für die Spalte *mitarbeiter* einzufügen, mit der folgenden Fehlermeldung:

Error Code: 1062. Duplicate entry 'Ralf Müller' for key 'mitarbeiter'

4.5.3 Standardwerte mit DEFAULT für Spalten festlegen

Wenn Sie beabsichtigen, Spalten bereits während der Tabellenerstellung einen Standardwert zuzuweisen, können Sie das Schlüsselwort **DEFAULT** verwenden. Sehen wir uns die Spaltendefinition der Spalte *kostenstelle* in [Listing 4.32](#) an:

```
CREATE TABLE dienstwagen(
  dienstwagenid SMALLINT PRIMARY KEY,
  kennzeichen CHAR(9) NOT NULL,
  erstzulassung DATE NOT NULL,
  kostenstelle SMALLINT DEFAULT 0,

  hersteller VARCHAR(200),
  modell VARCHAR(200),
  leistung DECIMAL(5,2),
  motor VARCHAR(100),
  getriebe VARCHAR(100),
  mitarbeiter VARCHAR(200) UNIQUE
);
```

Listing 4.32 Eine Spalte einer Tabelle während der Tabellenerstellung mit einem Standardwert ausstatten

Hinter dem Datentyp werden hier das Schlüsselwort **DEFAULT** und der Standardwert angegeben, der der Spalte zugeordnet werden soll. In diesem Fall wurde der Standardwert 0 der Spalte *kostenstelle* zugewiesen.

UNIQUE und DEFAULT

Wenn eine **DEFAULT**-Einschränkung für eine Spalte festgelegt wird, sollte dieser Spalte nicht gleichzeitig eine **UNIQUE**-Einschränkung zugeordnet werden, da sich beide Einschränkungen funktional ausschließen.



Die Zuordnung von Standardwerten wollen wir natürlich ausprobieren. Löschen Sie die Tabelle *dienstwagen* wieder aus der Datenbank, um sie diesmal mit einer Standardwerteinschränkung anzulegen.

Als Nächstes fügen wir wieder zwei Datensätze mittels einer **INSERT**-Anweisung in die Tabelle *dienstwagen* ein. Die Spalte *kostenstelle* wird in [Listing 4.33](#) bewusst nicht in der **INSERT**-Anweisung berücksichtigt, weil wir prüfen wollen, ob der Standardwert 0 der Spalte *kostenstelle* zugeordnet wird.

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung,mitarbeiter)
VALUES (1,'XX-XX-XX1','2015-01-01','Ralf Müller');
```

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,erstzulassung,mitarbeiter)
VALUES (2,'XX-XX-XX2','2015-01-01','Petra Schneider');
```

Listing 4.33 Zwei Datensätze einfügen, ohne Werte für eine Spalte anzugeben, der Standardwerte zugeordnet sind

Ob die Zuweisung des Standardwerts für die Spalte *kostenstelle* funktioniert hat, erfahren wir wieder mit einer einfachen Prüfabfrage:

```
SELECT dienstwagenid,kennzeichen,erstzulassung,kostenstelle
FROM dienstwagen;
```

Listing 4.34 Prüfabfrage zu einer Standardwertzuweisung, die während der Tabellenerstellung festgelegt wurde

dienstwagenid	kennzeichen	erstzulassung	kostenstelle
1	XX-XX-XX1	2015-01-01	0
2	XX-XX-XX2	2015-01-01	0

Tabelle 4.24 Das Ergebnis enthält die Standardwerte für die Spalte »kostenstelle«.

Wir haben in den **INSERT**-Anweisungen aus [Listing 4.33](#) keine Werte für die Spalte *kostenstelle* angegeben. Dennoch sehen Sie, dass in die Spalte *kostenstelle* gemäß der Standardwertzuweisung der Wert 0 eingefügt wurde.



DEFAULT und NOT NULL

Wenn Sie eine **DEFAULT**-Einschränkung für eine Spalte festlegen, können Sie dieser Spalte gleichzeitig eine **NOT NULL**-Einschränkung zuweisen. Das ist dann sinnvoll, wenn Sie sicherstellen wollen, dass der Standardwert nicht durch einen **NULL**-Wert überschrieben wird, etwa bei einer **INSERT**- oder **UPDATE**-Anweisung. Hier können Sie den standardmäßig gesetzten Wert explizit durch **NULL** oder konkrete Werte ersetzen.

4.5.4 Bedingungen mit einer CHECK-Einschränkung für Spalten festlegen

Bereits während der Tabellenerstellung können Sie mit einer **CHECK**-Einschränkung für Spalten Plausibilitätsprüfungen festlegen. Die den Spalten zugeordneten Plausibilitätsprüfungen gelten für die **INSERT**- und die **UPDATE**-Anweisungen. Wenn Sie versuchen, einen Wert in einer Spalte, der eine Plausibilitätsprüfung zugeordnet ist, einzufügen oder zu ändern, dann wird die Datenbank automatisch überprüfen, ob der Spaltenwert der Bedingung entspricht, die in der Plausibilitätsprüfung für die Spalte hinterlegt wurde. Entspricht der Spaltenwert nicht der festgelegten Bedingung, so wird die Einfüge- oder Änderungsoperation von der Datenbank abgelehnt.

Unterstützung der CHECK-Einschränkung

MariaDB, PostgreSQL und MS SQL Server unterstützen **CHECK**-Einschränkungen, wie es der Standard vorsieht.

Für MySQL steht die **CHECK**-Einschränkung erst ab der Version 8.0.16 zur Verfügung. Wenn Sie eine **CHECK**-Einschränkung in einer Tabellenerstellungsanweisung einer Spaltendefinition vor der Version 8.0.16 zuordnen, führt das zwar zu keinem Fehler, die Funktionalität der Plausibilitätsprüfung gemäß einer angegebenen Bedingung ist aber nicht gegeben.

Sehen wir uns nun [Listing 4.35](#) an:

```
CREATE TABLE dienstwagen(
  dienstwagenid SMALLINT PRIMARY KEY,
  kennzeichen CHAR(9) NOT NULL,
  erstzulassung DATE,
  kostenstelle SMALLINT DEFAULT 0,
  hersteller VARCHAR(200),
  modell VARCHAR(200),
  leistung DECIMAL(5,2),
  motor VARCHAR(100)
    CHECK(motor='Diesel' OR motor='Benzin' OR motor='Super'),
  getriebe VARCHAR(100)
    CHECK(getriebe='Automatikgetriebe' OR getriebe='Schaltgetriebe'),
  mitarbeiter VARCHAR(200)
);
```

Listing 4.35 Tabellenerstellungsanweisung mit **CHECK**-Einschränkung für Spaltenwerte

Von besonderem Interesse sind die Definitionen der Spalten *motor* und *getriebe*. Hinter der Datentypangabe wird die **CHECK**-Einschränkung angegeben. In einer Klammer wird dort die Bedingung aufgeführt, die geprüft werden soll. Werden mehrere Bedingungen geprüft, verknüpfen Sie sie mit logischen Operatoren wie **AND** und **OR**.

Der Spalte *motor* ordnen wir eine Plausibilitätsprüfung zu, die prüft, ob der Spaltenwert einem der folgenden Werte entspricht: *Diesel*, *Benzin* oder *Super*. Sie können nur einen der Werte eintragen, nicht mehrere gleichzeitig. Das Gleiche gilt bei der Spalte *getriebe*: Die **CHECK**-Einschränkung wacht darüber, dass ausschließlich Spaltenwerte in die Spalte *getriebe* eingetragen werden, die entweder dem Wert *Automatikgetriebe* oder dem Wert *Schaltgetriebe* entsprechen.

Das gilt natürlich nicht nur für die **INSERT**-Anweisung. Wenn Sie Aktualisierungen mit einer **UPDATE**-Anweisung durchführen, werden die Spaltenwerte ebenfalls auf die Bedingungen in der **CHECK**-Einschränkung geprüft. Werden die Bedingungen nicht erfüllt, so wird die Datenbank eine Aktualisierung eines Datensatzes ablehnen.

Als Nächstes soll die Tabelle *dienstwagen* gemäß der **CREATE TABLE**-Anweisung aus [Listing 4.35](#) in der Übungsdatenbank angelegt werden. Falls die Tabelle noch in Ihrer Datenbank existiert, löschen Sie sie einfach.

Beachten Sie, dass Spaltenwerte für die Spalte *erstzulassung* in dieser Tabellenerstellungsanweisung optional anzugeben sind. Das hat lediglich den Hintergrund, dass wir die **INSERT**-Anweisungen und die zu erwartende Ergebnistabelle übersichtlicher darstellen können. Dieser Tatsache ist auch die fehlende **UNIQUE**-Einschränkung für die Spalte *mitarbeiter* geschuldet.

Wenn die Tabelle *dienstwagen* erstellt wurde, bietet es sich an, den Spalten *motor* und *getriebe* Werte zuzuweisen, die der **CHECK**-Einschränkung entsprechen. Der Übersichtlichkeit halber fügen wir mit den **INSERT**-Anweisungen in [Listing 4.36](#) lediglich für die Spalten *dienstwagenid*, *kennzeichen*, *motor* und *getriebe* Werte in die Tabelle *dienstwagen* ein:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen, motor,getriebe)
VALUES (1,'XX-XX-XX1','Diesel','Automatikgetriebe');
```

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,motor,getriebe)
VALUES (2,'XX-XX-XX2','Benzin','Schaltgetriebe');
```

Listing 4.36 Datensätze unter Berücksichtigung von **CHECK**-Einschränkungen einfügen

Prüfen wir nun das Ergebnis der Einfügeoperationen:

```
SELECT dienstwagenid,kennzeichen,motor,getriebe
FROM dienstwagen;
```

Listing 4.37 Prüfabfrage zu **INSERT**-Anweisungen, die eine **CHECK**-Einschränkung berücksichtigen

In [Tabelle 4.25](#) sehen Sie, dass die **VALUES**-Klauseln der **INSERT**-Anweisungen eingefügt wurden, da die Werte der Spalten *motor* und *getriebe* exakt der **CHECK**-Einschränkung dieser Spalten entsprechen.

dienstwagenid	kennzeichen	motor	getriebe
1	XX-XX-XX1	Diesel	Automatikgetriebe
2	XX-XX-XX2	Benzin	Schaltgetriebe

Tabelle 4.25 Ergebnistabelle mit Zeilen, die unter Berücksichtigung einer **CHECK**-Einschränkung in eine Tabelle eingefügt wurden

Anschließend fordern wir die **CHECK**-Einschränkung heraus, die wir für die Tabelle *dienstwagen* definiert haben, indem wir versuchen, einen Datensatz einzufügen, dessen Spaltenwerte nicht den Bedingungen der **CHECK**-Einschränkungen entsprechen. Versuchen wir jetzt, den Wert **E10** als Spaltenwert in die Spalte *motor* einzufügen:

```
INSERT INTO dienstwagen (dienstwagenid,kennzeichen,motor,getriebe)
VALUES (3, 'XX-XX-XX2', 'E10', 'Schaltgetriebe');
```

Listing 4.38 **INSERT**-Anweisung mit einer Verletzung der **CHECK**-Einschränkung der Spalte »motor«

Die Datenbank liefert Ihnen nach dem Einfügeversuch der **INSERT**-Anweisung aus [Listing 4.38](#) folgende Fehlermeldung:

```
Error Code: 3819. Check constraint 'dienstwagen_chk_1' is violated.
```

Die Datenbank nimmt ihre Aufgabe also gewissenhaft wahr und überwacht die Einhaltung der **CHECK**-Einschränkung der Spalte *motor* in der Tabelle *dienstwagen*.

Zusammenfassung: Tabellen auf Spaltenebene Einschränkungen zuordnen

Sie haben die Möglichkeit, bereits während der Tabellenerstellung mit einer **CREATE TABLE**-Anweisung Einschränkungen für Spaltenwerte zu definieren, die für **INSERT**- und **UPDATE**-Anweisungen gelten. Folgende Einschränkungsarten können auf Spaltenebene für Tabellen genutzt werden:

- ▶ **NOT NULL** lässt keine **NULL**-Werte für die Spaltenwerte einer Spalte zu.
- ▶ **UNIQUE** sorgt dafür, dass jeder Spaltenwert einer Spalte eindeutig ist.
- ▶ **DEFAULT** weist einer Spalte einen initialen Standardwert zu.
- ▶ **CHECK** bietet die Möglichkeit, Plausibilitätsprüfungen (Bedingungen) zu formulieren.



4.5.5 Übungen

Wenn Sie das soeben Gelernte überprüfen möchten, so empfehle ich Ihnen, die folgenden Übungsaufgaben zu lösen.

Übung 1

In dieser Übung soll eine Tabelle erstellt werden, in der die Geschenke für besonders langjährige Mitarbeiter verwaltet werden. Die Tabelle soll die Bezeichnung *mitarbeitergeschenk* erhalten. Die Tabellenstruktur sehen Sie in Tabelle 4.26.

Spaltenbezeichnung	Datentyp	Primär-schlüssel	NOT NULL	Beispielwert
<i>geschenkid</i>	SMALLINT	X		1
<i>artikel</i>	VARCHAR(200)		X	Uhr
<i>preis</i>	DECIMAL(5,2)		X	150.00
<i>jahrezugehoerigkeit</i>	SMALLINT		X	10

Tabelle 4.26 Struktur der Tabelle »mitarbeitergeschenk«

Beachten Sie in der **CREATE TABLE**-Anweisung die Primärschlüsseleinschränkung der Spalte *geschenkid* und die **NOT NULL**-Einschränkung der Spalten *artikel*, *preis* und *jahrezugehoerigkeit*.

Wenn Sie die Tabelle erstellt haben, fügen Sie zwei gültige Datensätze unter Berücksichtigung der Primärschlüssel- und **NOT NULL**-Einschränkung in die Tabelle *mitarbeiterjubilaeum* ein.

Übung 2

Stellen Sie sicher, dass die Werte der Spalte *artikel* der Tabelle *mitarbeitergeschenk* immer eindeutig sind. Erstellen Sie hierzu die Tabelle *mitarbeitergeschenk* gemäß den Angaben aus Tabelle 4.26, und versehen Sie die Spalte *artikel* zusätzlich mit einer Einschränkung, die dafür sorgt, dass jeder Wert der Spalte *artikel* nur einmal vorkommt. Nachdem Sie die Tabelle *mitarbeitergeschenk* angelegt haben, fügen Sie zwei Datensätze ein, die den festgelegten Einschränkungen entsprechen.

Zusätzlich soll eine weitere Zeile eingefügt werden, in der ein Artikel vorkommt, der einem der zuvor eingefügten Datensätze entspricht. Achten Sie darauf, dass der neue Datensatz bis auf die Eindeigkeitseinschränkung keine weitere Einschränkung ver-

letzt, die in der Tabellenerstellungsanweisung vorkommt. Beschreiben Sie kurz die Fehlermeldung der Datenbank und was sie aussagt.

Zuvor löschen Sie die in Übung 1 erstellte Tabelle, um sie neu anlegen zu können.

Übung 3

Die Spalte *preis* der Tabelle *mitarbeitergeschenk* soll in einer weiteren Version der Tabelle standardmäßig mit dem Wert 0.00 unter Beibehaltung der **NOT NULL**-Einschränkung ausgestattet werden. Sie können also die Tabellenstruktur verwenden, wie sie in Tabelle 4.26 aufgezeigt ist, bis auf die zusätzliche Standardwertzuordnung der Spalte *preis*.

Fügen Sie auch hier einen Datensatz mit expliziter Spaltenangabe ein. In der **INSERT**-Anweisung lassen Sie die Angabe der Spalte *preis* weg, um anschließend mit einer einfachen **SELECT**-Abfrage zu prüfen, ob die Spalte *preis* einen Standardwert zugeordnet bekommen hat.

Vergessen Sie nicht, zuvor die Tabelle *mitarbeitergeschenk* wieder zu löschen.

Übung 4

Die Spalte *jahrezugehoerigkeit* der Tabelle *mitarbeitergeschenk* soll ausschließlich die Spaltenwerte 5, 10, 15 und 20 für die Anzahl der Jubiläumsjahre aufnehmen können. Hier ist also eine Bedingung zu formulieren, die jeweils die Werte der Spalte *jahrezugehoerigkeit* gegen die angegebenen Spaltenjahre prüft. Überlegen Sie, welchen logischen Operator Sie brauchen, um die Werte miteinander zu verknüpfen.

Fügen Sie anschließend in die erstellte Tabelle zwei Datensätze ein, die der Einschränkung der Spalte *jahrezugehoerigkeit* genügen. Zu guter Letzt fügen Sie eine Zeile ein, die nicht der Prüfung entspricht. Beschreiben Sie kurz die Fehlermeldung und die Auswirkung auf die **INSERT**-Anweisung.

Auch hier müssen Sie die Tabelle *mitarbeitergeschenk* aus der letzten Übung löschen, um sie wieder erstellen zu können.

Lösung zu Übung 1

```
CREATE TABLE mitarbeitergeschenk(  
geschenkid SMALLINT PRIMARY KEY,  
artikel VARCHAR(200) NOT NULL,  
preis DECIMAL(5,2) NOT NULL,  
jahrezugehoerigkeit SMALLINT NOT NULL  
);
```

```
INSERT INTO mitarbeitergeschenk  
  (geschenkid, artikel, preis, jahrezugehoerigkeit)  
VALUES (1, 'Aktentasche', 120.54, 5);
```

```
INSERT INTO mitarbeitergeschenk  
  (geschenkid, artikel, preis, jahrezugehoerigkeit)  
VALUES (2, 'Uhr', 200.99, 10);
```

Listing 4.39 Eine Tabelle mit einem Primärschlüssel und einer NOT NULL-Einschränkung erstellen

Lösung zu Übung 2

```
CREATE TABLE mitarbeitergeschenk(  
  geschenkid SMALLINT PRIMARY KEY,  
  artikel VARCHAR(200) UNIQUE,  
  preis DECIMAL(5,2),  
  jahrezugehoerigkeit SMALLINT NOT NULL  
);
```

```
INSERT INTO mitarbeitergeschenk  
  (geschenkid, artikel, preis, jahrezugehoerigkeit)  
VALUES (1, 'Aktentasche', 120.54, 5);
```

```
INSERT INTO mitarbeitergeschenk  
  (geschenkid, artikel, preis, jahrezugehoerigkeit)  
VALUES (2, 'Uhr', 200.99, 10);
```

```
INSERT INTO mitarbeitergeschenk  
  (geschenkid, artikel, preis, jahrezugehoerigkeit)  
VALUES (3, 'Uhr', 265.00, 15);
```

```
/*  
Die Datenbank teilt Ihnen in der Fehlermeldung mit,  
dass Sie versucht haben, einen doppelten Schlüsselwert  
in eine Spalte einzutragen, bei der während der  
Tabellenerstellung der Schlüsselwert eingeschränkt wurde.  
Die Fehlermeldung sagt aus, dass nur eindeutige Werte für  
die Spalte "artikel" gültig sind.  
*/
```

Listing 4.40 Eine Tabelle mit einer UNIQUE-Einschränkung für eine Spalte erstellen und eine Verletzung der Einschränkung provozieren

Lösung zu Übung 3

```
CREATE TABLE mitarbeitergeschenk (
geschenkid SMALLINT PRIMARY KEY,
artikel VARCHAR(200) NOT NULL,
preis DECIMAL(5,2) NOT NULL DEFAULT 0.0,
jahrezugehoerigkeit SMALLINT NOT NULL
);
```

```
INSERT INTO mitarbeitergeschenk
(geschenkid, artikel, jahrezugehoerigkeit)
VALUES (1, 'Aktentasche', 5);
```

```
SELECT geschenkid, artikel, preis, jahrezugehoerigkeit
FROM mitarbeitergeschenk;
```

Listing 4.41 Einer Spalte einer Tabelle einen Standardwert zuordnen

Lösung zu Übung 4

```
CREATE TABLE mitarbeitergeschenk (
geschenkid SMALLINT PRIMARY KEY,
artikel VARCHAR(200) NOT NULL,
preis DECIMAL(5,2) NOT NULL,
jahrezugehoerigkeit SMALLINT NOT NULL CHECK(
jahrezugehoerigkeit=5 OR jahrezugehoerigkeit=10 OR
jahrezugehoerigkeit=15 OR jahrezugehoerigkeit=20)
);
```

```
INSERT INTO mitarbeitergeschenk
(geschenkid, artikel, preis, jahrezugehoerigkeit)
VALUES (1, 'Aktentasche', 150.50, 5);
```

```
INSERT INTO mitarbeitergeschenk
(geschenkid, artikel, preis, jahrezugehoerigkeit)
VALUES (2, 'Uhr', 200.50, 10);
```

```
INSERT INTO mitarbeitergeschenk
(geschenkid, artikel, preis, jahrezugehoerigkeit)
VALUES (3, 'Schreibtischset', 300.00, 7);
```

Listing 4.42 Eine Tabelle mit einer CHECK-Einschränkung für eine Spalte erstellen

Bei dem Versuch, eine Zeile einzufügen, die nicht der **CHECK**-Einschränkung entspricht, erhalten Sie die Fehlermeldung, dass eine **CHECK**-Einschränkung verletzt wurde und die Zeile somit nicht eingefügt wurde.

4.6 Spalten auf Tabellenebene Einschränkungen (CONSTRAINT) zuordnen

Bisher haben wir nur Spalten mit Einschränkungen gesehen. Sie können Einschränkungen aber auch auf Tabellenebene zuordnen. Tabelleneinschränkungen werden dann angewendet, wenn mehrere Spalten von der Einschränkung bzw. dem **CONSTRAINT** betroffen sein können.

Sehen wir uns noch einmal kurz an, welche Einschränkungen Sie bis jetzt kennengelernt haben:

- ▶ **PRIMARY KEY**
- ▶ **NOT NULL**
- ▶ **UNIQUE**
- ▶ **DEFAULT**
- ▶ **CHECK**

Auf Tabellenebene können nur die Einschränkungen **PRIMARY KEY**, **UNIQUE** und **CHECK** festgelegt werden. Eine **DEFAULT**-Einschränkung, die einen Standardwert festlegt, oder eine **NOT NULL**-Einschränkung, die eine Spalte als Pflichtfeld definiert, können nicht als Tabelleneinschränkung definiert werden.

4.6.1 Einen Primärschlüssel auf Tabellenebene festlegen

Sehen wir uns in [Listing 4.43](#) die **CREATE TABLE**-Anweisung für die Tabelle *dienstwagen* an, mit der Sie einen Primärschlüssel auf Tabellenebene definieren:

```
CREATE TABLE dienstwagen(  
    dienstwagenid SMALLINT,  
    kennzeichen CHAR(9) NOT NULL,  
    erstzulassung DATE NOT NULL,  
    kostenstelle SMALLINT,  
    hersteller VARCHAR(200),  
    modell VARCHAR(200),  
    leistung DECIMAL(5,2),  
    motor VARCHAR(100),  
    getriebe VARCHAR(100),
```

```

mitarbeiter VARCHAR(200),
CONSTRAINT pk_dienstwagenid PRIMARY KEY (dienstwagenid)
);

```

Listing 4.43 Primärschlüssel auf Tabellenebene definieren

Als Erstes fällt Ihnen sicherlich auf, dass der Spalte *dienstwagenid* keine Primärschlüsseleinschränkung mehr zugeordnet ist. Stattdessen folgt hinter der letzten Spaltendefinition für die Spalte *mitarbeiter* keine weitere Spalte, so wie wir es bisher gewohnt sind, sondern eine Einschränkungdefinition, die immer mit dem Schlüsselwort **CONSTRAINT** beginnt.

Hinter dem Schlüsselwort **CONSTRAINT** wird die Bezeichnung für die Einschränkung festgelegt. In diesem Fall ordnen wir dem **CONSTRAINT** die Bezeichnung *pk_dienstwagenid* zu, um zu kennzeichnen, dass es sich um eine Primärschlüsseleinschränkung handelt. Hinter der **CONSTRAINT**-Bezeichnung wird die eigentliche Einschränkung angegeben. Hier definieren wir einen Primärschlüssel genau wie bei einer Spaltendefinition, nämlich mit dem Schlüsselwort **PRIMARY KEY**. Jetzt müssen wir nur noch angeben, welche Spalte als Primärschlüssel verwendet werden soll. Hinter den Schlüsselwörtern **PRIMARY KEY** übergeben wir hierzu in einer Klammer die Spalte, die als Primärschlüssel festgelegt werden soll – in diesem Fall die Spalte *dienstwagenid*.

Legen wir nun die Tabelle *dienstwagen* mit einer Primärschlüsseleinschränkung auf Tabellenebene an, so wie Sie es in [Listing 4.43](#) sehen. Falls es erforderlich sein sollte, nutzen Sie die **DROP TABLE**-Anweisung, um eine bestehende Tabelle *dienstwagen* zu entfernen.

Führen Sie nun die in [Listing 4.44](#) angegebenen **INSERT**-Anweisungen aus, um Datensätze in die Tabelle *dienstwagen* einzutragen:

```

INSERT INTO dienstwagen
(dienstwagenid,kennzeichen,erstzulassung)
VALUES (1,'XX-XX-XX1','2014-08-05');

```

```

INSERT INTO dienstwagen
(dienstwagenid,kennzeichen,erstzulassung)
VALUES (2,'XX-XX-XX1','2014-08-05');

```

Listing 4.44 Datensätze unter Berücksichtigung der Primärschlüsseleinschränkung einfügen

Wenn Sie eine Primärschlüsselverletzung durch einen bereits vorhandenen Schlüssel provozieren, wird die Datenbank eine entsprechende Fehlermeldung ausgeben. Die Primärschlüsseldefinition, die auf Tabellenebene festgelegt wurde, verhält sich also wie auf Spaltenebene.

Nicht jede Primärschlüsseldefinition ist so einfach wie die, die wir für die Tabelle *dienstwagen* definiert haben. Betrachten Sie [Tabelle 4.27](#), in der eine Tabellenstruktur die einzelnen Niederlassungen eines Unternehmens beschreibt. Die Niederlassungen des Unternehmens können in unterschiedlichen Ländern ansässig sein. Einem Land können also mehrere Niederlassungen zugeordnet werden.

Spaltenbezeichnung	Datentyp	Primärschlüssel	Beispielwert
<i>land</i>	VARCHAR(200)	X	Deutschland
<i>zweigstellenid</i>	SMALLINT	X	1
<i>namezweigstelle</i>	VARCHAR(200)		Niederlassung X

Tabelle 4.27 Struktur einer Tabelle, deren Primärschlüssel aus mehreren Spalten besteht

Der Tabelle soll die Bezeichnung *zweigstellen* zugeordnet werden. Die eigentliche Struktur der Tabelle wird durch die Spalten *land*, *zweigstellenid* und *namezweigstelle* festgelegt. In der Spalte »Datentyp« sehen Sie die entsprechenden Datentypen, die den Spalten zugeordnet werden.

Die Spalte »Primärschlüssel« zeichnet jeweils durch ein X die Spalten *land* und *zweigstellenid* als Primärschlüssel aus. Das ist neu. Bisher haben wir immer nur einer Spalte eine Primärschlüsseinschränkung zugeordnet. Ein Primärschlüssel, der aus mehreren Spalten zusammengesetzt ist, bedeutet, dass Zeilen durch eine Kombination von Spaltenwerten eindeutig identifiziert werden. In diesem Beispiel sind dies die Spalten *land* und *zweigstellenid*.

Ein solcher Primärschlüssel, der aus einer Kombination von Spalten gebildet wird, wird als *zusammengesetzter Primärschlüssel* bezeichnet. Ein zusammengesetzter Primärschlüssel kann aus zwei oder mehreren Spalten bestehen und kann nur auf Tabellenebene definiert werden.

Eine Tabellenerstellungsanweisung wie in [Listing 4.45](#) ist also nicht möglich und führt zu einer Fehlermeldung. Auf Spaltenebene kann ein Primärschlüssel nur einer Spalte zugeordnet werden.

```
CREATE TABLE zweigstellen(  
  land VARCHAR(200) PRIMARY KEY,  
  zweigstellenid SMALLINT PRIMARY KEY,  
  namezweigstelle VARCHAR(200)  
);
```

Listing 4.45 Fehlerhafte Tabellenerstellungsanweisung mit einer Primärschlüsselfestlegung über zwei Spalten

Stattdessen müssen wir bereits auf Tabellenebene festlegen, dass sich der Primärschlüssel aus einer Kombination mehrerer Spalten zusammensetzt. Wenn Sie einen Primärschlüssel auf Tabellenebene festlegen, müssen Sie die Spalten, die als zusammengesetzter Primärschlüssel festgelegt werden sollen, durch Kommas getrennt in der Klammer angeben. Sehen Sie sich dazu den Eintrag hinter der letzten Spaltendefinition in [Listing 4.46](#) an:

```
CREATE TABLE zweigstellen(
  land VARCHAR(200),
  zweigstellenid SMALLINT,
  namezweigstelle VARCHAR(200),
  CONSTRAINT pk_land_zweigstellen PRIMARY KEY (land,zweigstellenid)
);
```

Listing 4.46 Tabellen mit einem zusammengesetzten Primärschlüssel ausstatten

Probieren wir in [Listing 4.47](#) aus, was passiert, wenn wir in die Tabelle *zweigstellen* gemäß der Primärschlüsseinschränkung über die Spalten *land* und *zweigstellenid* vier Zeilen einfügen:

```
INSERT INTO zweigstellen VALUES ('Deutschland',1,'Zweigstelle 1');
```

```
INSERT INTO zweigstellen VALUES ('Deutschland',2,'Zweigstelle 2');
```

```
INSERT INTO zweigstellen VALUES ('Holland',1,'Zweigstelle 3');
```

```
INSERT INTO zweigstellen VALUES ('Holland',2,'Zweigstelle 4');
```

Listing 4.47 Datensätze gemäß den Einschränkungen eines zusammengesetzten Primärschlüssels einfügen

Die ersten beiden **INSERT**-Anweisungen fügen in die Tabelle *zweigstellen* jeweils Zweigstellen ein, die in Deutschland ansässig sind, die dritte und vierte Zeile Zweigstellen, die in Holland ansässig sind. Die Kombination der Primärschlüsselwerte über die Spalten *land* und *zweigstellenid* ist also eindeutig und identifiziert jeweils eine Zweigstelle.

Wir nutzen wieder eine einfache Abfrage, um das Ergebnis der Einfügeoperationen zu überprüfen:

```
SELECT land,zweigstellenid,namezweigstelle FROM zweigstellen;
```

Listing 4.48 Prüfabfrage für eine Tabelle mit einem zusammengesetzten Primärschlüssel

In [Tabelle 4.28](#) ist zu sehen, dass tatsächlich jeder aus zwei Spaltenwerten zusammengesetzte Schlüssel eindeutig eine Zweigstelle identifiziert.

land	zweigstellenid	namezweigstelle
Deutschland	1	Zweigstelle 1
Deutschland	2	Zweigstelle 2
Holland	1	Zweigstelle 3
Holland	2	Zweigstelle 4

Tabelle 4.28 Ergebnistabelle mit Datensätzen, die einer zusammengesetzten Primärschlüsseleinschränkung entsprechen

Wenn Sie versuchen, eine weitere Zweigstelle mit einem Spaltenwert 2 für die Spalte *zweigstellenid* in Deutschland einzufügen, so würde die Datenbank den Wert nicht einfügen, da eine Primärschlüsselverletzung vorläge. Stattdessen würde sie Ihnen folgende Fehlermeldung ausgeben:

Error Code: 1062. Duplicate entry 'Deutschland-2' for key 'PRIMARY'

Sie sehen also, dass für zusammengesetzte Primärschlüssel, die aus mehreren Spalten bestehen, ebenfalls zuverlässig sichergestellt ist, dass nur eindeutige Schlüsselwerte in die Tabelle eingetragen werden.

4.6.2 Eine UNIQUE-Einschränkung auf Tabellenebene festlegen

Auf Tabellenebene können Sie mehrere Spalten mit einer **UNIQUE**-Einschränkung ausstatten. Betrachten Sie die letzte Zeile der Spalten- bzw. Einschränkungsliste unserer **CREATE TABLE**-Anweisung in [Listing 4.49](#):

```
CREATE TABLE zweigstellen(
  globalid SMALLINT PRIMARY KEY,
  land VARCHAR(200),
  zweigstellenid SMALLINT,
  namezweigstelle VARCHAR(200),
  CONSTRAINT uq_landzweigstellenid UNIQUE (land,zweigstellenid)
);
```

Listing 4.49 Eine UNIQUE-Einschränkung für mehrere Spalten festlegen

Hier ist der **CONSTRAINT** mit der Bezeichnung *uq_landzweigstellenid* zu sehen. Hinter der Bezeichnung finden Sie die **UNIQUE**-Einschränkung. Von Klammern umschlossen werden die Spalten *land* und *zweigstellenid* in einer Liste der **UNIQUE**-Einschränkung auf Tabellenebene übergeben. Es gilt also, dass die Kombination dieser beiden Spaltenwerte immer eindeutig sein muss.

Diese Eindeutigkeit erzielen wir ähnlich wie für Primärschlüsselwerte, die aus mehreren Spalten bestehen. Um eine Eindeutigkeit zu erreichen, reicht es also aus, wenn ein Wert der Kombination aus Spaltenwerten sich von den bereits bestehenden Werten unterscheidet. Die **UNIQUE**-Einschränkung auf Tabellenebene verhält sich also wie eine Primärschlüsselzuordnung auf Tabellenebene, die ebenfalls aus mehreren Spalten festgelegt wird. Aus diesem Grund enthält [Listing 4.49](#) in der Tabellenerstellungsanweisung einen Primärschlüssel *globalid*. Die **UNIQUE**-Einschränkung stellt sicher, dass die Kombination der Spaltenwerte für die Spalten *land* und *zweigstellenid* eindeutig ist.

Führen Sie die Tabellenerstellungsanweisung aus [Listing 4.49](#) aus, um die Tabelle *zweigstellen* anzulegen. Löschen Sie die Tabelle *zweigstellen* mit einer **DROP TABLE**-Anweisung, wenn sie noch aus unserem vorherigen Beispiel in der Datenbank vorhanden ist.

Als Nächstes fügen wir in [Listing 4.50](#) vier Zeilen in die Tabelle *zweigstellen* ein, die der **UNIQUE**-Einschränkung entsprechen:

```
INSERT INTO zweigstellen
VALUES (1, 'Deutschland', 1, 'Zweigstelle 1');
```

```
INSERT INTO zweigstellen
VALUES (2, 'Deutschland', 2, 'Zweigstelle 2');
```

```
INSERT INTO zweigstellen
VALUES (3, 'Holland', 1, 'Zweigstelle 3');
```

```
INSERT INTO zweigstellen
VALUES (4, 'Holland', 2, 'Zweigstelle 4');
```

Listing 4.50 Datensätze unter Berücksichtigung der **UNIQUE**-Einschränkung auf Tabellenebene einfügen

Mit der **SELECT**-Abfrage aus [Listing 4.51](#) können Sie nachvollziehen, dass die Zeilen unter Berücksichtigung der **UNIQUE**-Einschränkung für die Spalten *land* und *zweigstellenid* eingefügt wurden:

```
SELECT globalid, land, zweigstellenid, namezweigstelle
FROM zweigstellen;
```

Listing 4.51 Prüfabfrage für eine Tabelle, der auf Tabellenebene eine **UNIQUE**-Einschränkung für zwei Spalten zugeordnet wurde

In [Tabelle 4.29](#) sehen Sie das Ergebnis der Prüfabfrage. Die Spaltenwerte der Spalten *land* und *zweigstellenid* sind jeweils in der Kombination miteinander eindeutig und

genügen somit der **UNIQUE**-Einschränkung, die wir auf Tabellenebene den Spalten zugeordnet haben.

globalid	land	zweigstellenid	namezweigstelle
1	Deutschland	1	Zweigstelle 1
2	Deutschland	2	Zweigstelle 2
3	Holland	1	Zweigstelle 3
4	Holland	2	Zweigstelle 4

Tabelle 4.29 Ergebnistabelle mit Datensätzen, die unter Berücksichtigung einer **UNIQUE**-Einschränkung in eine Tabelle eingefügt wurden

Danach provozieren wir eine Verletzung der **UNIQUE**-Einschränkung, indem wir versuchen, für die Spalten *land* und *zweigstellenid* eine bereits bestehende Kombination von Spaltenwerten in die Tabelle *zweigstellen* einzutragen. [Listing 4.52](#) gibt für die Spalten *land* und *zweigstellenid* die bereits bestehende Spaltenwertekombination Deutschland und 1 als Werte an. Die Primärschlüsseinschränkung wird in der **INSERT**-Anweisung beachtet, indem wir den eindeutigen Wert auf den Wert 5 erhöhen, da wir ja nur beabsichtigen, eine **UNIQUE**-Einschränkungsverletzung zu provozieren und keine Primärschlüsselverletzung.

```
INSERT INTO zweigstellen
VALUES (5, 'Deutschland', 1, 'Zweigstelle 5');
```

Listing 4.52 Eine Verletzung der **UNIQUE**-Einschränkung provozieren

Die Datenbank reagiert mit einer Fehlermeldung, die verlauten lässt, dass Sie gerade versucht haben, einen doppelten Eintrag in die Spaltenwertkombination der Spalten *land* und *zweigstellenid* einzufügen, und damit gegen die **UNIQUE**-Einschränkung verstoßen, die Sie während der Tabellenerstellung festgelegt haben:

```
Error Code: 1062. Duplicate entry 'Deutschland-1' for key 'uq_
landzweigstellenid'
```

Wenn Sie einen weiteren gültigen Datensatz unter Berücksichtigung der **UNIQUE**-Einschränkung mit der Bezeichnung *uq_landzweigstellenid* eintragen möchten, so müssten Sie entweder den Spaltenwert für das Land oder die Zweigstellen-ID ändern, damit die Spaltenwertkombination wieder eindeutig ist. Wir versuchen mit der **INSERT**-Anweisung aus [Listing 4.53](#), den Spaltenwert 3 für die Spalte *zweigstellenid* zu verwenden, um eine weitere Zeile einzufügen:


```
INSERT INTO zweigstellen
VALUES (5, 'Deutschland', 3, 'Zweigstelle 5');
```

Listing 4.53 Eine weitere Zeile unter Berücksichtigung der UNIQUE-Einschränkung einfügen

Mit der Prüfabfrage aus [Listing 4.51](#) erhalten Sie ein Ergebnis wie in [Tabelle 4.30](#), das den neuen gültigen Datensatz enthält. Wie die Spaltenwerte `Deutschland` und `3` für die Spalten `land` und `zweigstellenid` zeigen, hat das Einfügen der neuen Zeile funktioniert.

globalid	land	zweigstellenid	namezweigstelle
1	Deutschland	1	Zweigstelle 1
2	Deutschland	2	Zweigstelle 2
3	Holland	1	Zweigstelle 3
4	Holland	2	Zweigstelle 4
5	Deutschland	3	Zweigstelle 5

Tabelle 4.30 Ergebnistabelle für eine Einfügeanweisung unter Berücksichtigung der UNIQUE-Einschränkung

4.6.3 Eine CHECK-Einschränkung auf Tabellenebene festlegen

Die Plausibilitätsprüfung, die Sie mit einer CHECK-Einschränkung bereits während der Tabellenerstellung der Datenbank mitteilen können, funktioniert vom Grundsatz her wie die Zuweisung dieser Einschränkung auf Spaltenebene. Sie formulieren eine oder mehrere Bedingungen, die auf Spaltenwerte bei Einfüge- und Aktualisierungsanweisungen zutreffen müssen. Ein Beispiel dazu finden Sie in [Listing 4.54](#):

```
CREATE TABLE zweigstellen(
  globalid SMALLINT PRIMARY KEY,
  land VARCHAR(200),
  zweigstellenid SMALLINT,
  namezweigstelle VARCHAR(200),
  mitarbeiterplanzahl SMALLINT,
  mitarbeiteristzahl SMALLINT,
  CONSTRAINT ch_mitarbeiterzahl CHECK(mitarbeiterplanzahl>=mitarbeiteristzahl)
);
```

Listing 4.54 Auf Tabellenebene eine CHECK-Einschränkung festlegen

Dort sehen Sie hinter der Spaltendefinition *namezweigstelle* eine Einschränkung auf Tabellenebene, in der die hinzugekommenen Spalten *mitarbeiterplanzahl* und *mitarbeiteristzahl* miteinander verglichen werden. Datensätze sollen nur eingefügt oder aktualisiert werden dürfen, wenn sie die Bedingung *mitarbeiterplanzahl*>*mitarbeiteristzahl* erfüllen.

Wie Sie es bei den anderen Tabelleneinschränkungen kennengelernt haben, leiten wir die Einschränkung mit dem Schlüsselwort **CONSTRAINT** ein. Hinter der **CONSTRAINT**-Bezeichnung geben Sie die **CHECK**-Einschränkung an. Innerhalb der einfachen Klammern der **CHECK**-Einschränkung sehen Sie hier die Bedingung, die für die **CHECK**-Einschränkung gelten soll.

Ihnen ist sicherlich nicht entgangen, dass wir hier eine Bedingung formulieren, in der wir zwei Spaltenwerte miteinander vergleichen. Es ist daher sinnvoll, Einschränkungen auf Tabellenebene zu formulieren.

In [Listing 4.55](#) sehen Sie **INSERT**-Anweisungen mit Spaltenwerten für die Spalten *mitarbeiterplanzahl* und *mitarbeiteristzahl*, die der Bedingung der **CHECK**-Einschränkung entsprechen:

```
INSERT INTO zweigstellen  
  (globalid, mitarbeiterplanzahl, mitarbeiteristzahl)  
VALUES (1,120,115);
```

```
INSERT INTO zweigstellen  
  (globalid, mitarbeiterplanzahl, mitarbeiteristzahl)  
VALUES (2,50,43);
```

Listing 4.55 Datensätze gemäß einer **CHECK**-Einschränkung mit einer Bedingung einfügen, die zwei Spalten enthält

Es werden so zwei Datensätze in die Tabelle *zweigstellen* eingefügt, deren Werte in den Spalten *mitarbeiterplanzahl* und *mitarbeiteristzahl* der Bedingung *mitarbeiterplanzahl*>*mitarbeiteristzahl* entsprechen. Die Einschränkungen, die für die zu erstellende Tabelle gelten, ermöglichen es uns, eine **INSERT**-Anweisung zu formulieren, in der die Spalten *globalid*, *mitarbeiterplanzahl* und *mitarbeiteristzahl* explizit angegeben werden. Die anderen Spalten sind mit keinerlei verbindlichen Einschränkungen versehen.

Wenn Sie die Spalten *globalid*, *mitarbeiterplanzahl* und *mitarbeiteristzahl* mit einer einfachen **SELECT**-Anweisung abfragen, erhalten Sie ein Ergebnis wie in [Tabelle 4.31](#). Die Spaltenwerte konnten ohne Probleme eingefügt werden, da sie der Bedingung *mitarbeiterplanzahl*>*mitarbeiteristzahl* entsprechen.

globalid	mitarbeiterplanzahl	mitarbeiteristzahl
1	120	115
2	50	43

Tabelle 4.31 Ergebnisliste, in der die Spaltenwerte enthalten sind, die in einer Bedingung einer Tabelleneinschränkung geprüft werden

Der nächste Schritt soll uns Gewissheit darüber verschaffen, dass die Datenbank tatsächlich über die aufgestellten Bedingungen wacht, die wir in der **CREATE TABLE**-Anweisung festgelegt haben. Dazu formulieren wir eine **INSERT**-Anweisung, in der die Bedingung `mitarbeiterplanzahl >= mitarbeiteristzahl` nicht erfüllt ist, da die **INSERT**-Anweisung den Spalten `mitarbeiterplanzahl` und `mitarbeiteristzahl` die Werte 120 bzw. 150 zuweist. Das sehen Sie in [Listing 4.56](#):

```
INSERT INTO zweigstellen
(globalid,mitarbeiterplanzahl,mitarbeiteristzahl)
VALUES (3,120,150);
```

Listing 4.56 Provokation einer CHECK-Einschränkungsverletzung

Diese Einfügeanweisung wird von den Datenbanken mit einer Fehlermeldung quittiert. Der Wert 120 ist eben nicht größer als oder gleich 150.

Zusammenfassung: Einschränkungen auf Tabellenebene festlegen

Die Einschränkungen **PRIMARY KEY**, **UNIQUE** und **CHECK** können Sie über mehrere Spalten auf Tabellenebene festlegen.

Für die **PRIMARY KEY**- und **UNIQUE**-Einschränkung gilt, dass die Kombination der Spaltenwerte immer eindeutig sein muss, damit ein Datensatz mit einer **INSERT**-Anweisung eingefügt oder mit einer **UPDATE**-Anweisung geändert werden kann.

Die **CHECK**-Einschränkung auf Tabellenebene wird genutzt, um Bedingungen über mehrere Spalten zu formulieren, die erfüllt sein müssen, damit ein Datensatz mit einer **INSERT**-Anweisung eingefügt oder mit einer **UPDATE**-Anweisung geändert werden kann.



4.6.4 Übungen

Vielleicht haben Sie ja Lust, das Erlernte nun zu üben.

Übung 1

In [Tabelle 4.32](#) finden Sie die Struktur der zu erstellenden Tabelle.

Spaltenbezeichnung	Datentyp	Primärschlüssel	Beispielwert
<i>maschinenid</i>	SMALLINT	X	1
<i>variante</i>	SMALLINT	X	1
<i>bezeichnung</i>	VARCHAR(200)		Drehmaschine

Tabelle 4.32 Struktur der Tabelle »produktionsmaschinen« mit einem zusammengesetzten Primärschlüssel

Mit dieser Tabelle sollen die Produktionsmaschinen des Unternehmens verwaltet werden. Sie soll folgende Spalten enthalten: *maschinenid*, *variante* und *bezeichnung*. Die Produktionsmaschinen lassen sich anhand einer Maschinen-Identifikationsnummer und einer Variantennummer eindeutig identifizieren. Eine Maschine des gleichen Bautyps kann also in unterschiedlichen Konfigurationen im Unternehmen eingesetzt werden. Die Spalten *maschinenid* und *variante* bilden also die Grundlage für einen zusammengesetzten Primärschlüssel. Die Tabelle soll die Bezeichnung *produktionsmaschinen* erhalten. Fügen Sie für zwei unterschiedliche Maschinen in zwei Varianten gültige Datensätze in die Tabelle *produktionsmaschinen* ein, nachdem Sie die Tabelle in der Datenbank angelegt haben.

Übung 2

Nutzen Sie in dieser Übung die **UNIQUE**-Einschränkung auf Tabellenebene. Die Struktur der Tabelle *produktionsmaschinen* hat sich geändert (siehe [Tabelle 4.33](#)), damit wir die **UNIQUE**-Einschränkung auch verwenden können. Anstelle eines zusammengesetzten Primärschlüssels verwenden wir jetzt einen eindeutigen numerischen Primärschlüssel, der durch die Spalte *globalid* realisiert wird.

Spaltenbezeichnung	Datentyp	Primärschlüssel	UNIQUE	Beispielwert
<i>globalid</i>	SMALLINT	X		1
<i>maschinenid</i>	SMALLINT		X	1
<i>variante</i>	SMALLINT		X	1
<i>bezeichnung</i>	VARCHAR(200)			Drehmaschine

Tabelle 4.33 Struktur der Tabelle »produktionsmaschinen« mit einer UNIQUE-Einschränkung über mehrere Spalten

In dieser Übung soll durch eine **UNIQUE**-Einschränkung sichergestellt werden, dass die Kombination der Spaltenwerte der Spalten *maschinenid* und *variante* stets eindeutig bleibt.

Fügen Sie auch in die zu erstellende Tabelle vier Datensätze ein, die den Tabelleneinschränkungen entsprechen. Löschen Sie zuvor die bereits angelegte gleichnamige Tabelle aus der Datenbank.

Übung 3

Der Tabelle *produktionsmaschinen* sind die Spalten *laufzeit* und *maxlaufzeit* zugeordnet. Für **INSERT**- und **UPDATE**-Anweisungen soll die Plausibilitätsregel *laufzeit* < *maxlaufzeit* gelten.

Stellen Sie eine Bedingung auf, in der Spaltenwerte von zwei Spalten auf eine Plausibilitätsregel hin geprüft werden.

Spaltenbezeichnung	Datentyp	Primärschlüssel	Beispielwert
<i>globalid</i>	SMALLINT	X	1
<i>laufzeit</i>	INT		1138045
<i>maxlaufzeit</i>	INT		10000000
<i>bezeichnung</i>	VARCHAR(200)		Drehmaschine

Tabelle 4.34 Struktur der Tabelle »produktionsmaschinen« mit einer CHECK-Einschränkung über mehrere Spalten

Fügen Sie nach dem Anlegen der Tabelle zwei Datensätze in die Tabelle *produktionsmaschinen* ein, die der Plausibilitätsregel entsprechen.

Zuvor löschen Sie wieder die gleichnamige Tabelle aus Übung 2.

Lösung zu Übung 1

```
CREATE TABLE produktionsmaschinen(
    maschinenid SMALLINT,
    variante SMALLINT,
    bezeichnung VARCHAR(200),
    CONSTRAINT pk_maschine PRIMARY KEY(maschinenid,variante)
);
```

```
INSERT INTO produktionsmaschinen (maschinenid,variante,bezeichnung)
VALUES (1,1,'Drehmaschine 1.0');
```

```
INSERT INTO produktionsmaschinen (maschinenid,variante,bezeichnung)
VALUES (1,2,'Drehmaschine 1.1');
```

```
INSERT INTO produktionsmaschinen (maschinenid,variante,bezeichnung)
VALUES (2,1,'Drehmaschine 2.0');
```

```
INSERT INTO produktionsmaschinen (maschinenid,variante,bezeichnung)
VALUES (2,2,'Drehmaschine 2.1');
```

Listing 4.57 Eine Tabelle mit einem zusammengesetzten Primärschlüssel anlegen

Lösung zu Übung 2

```
CREATE TABLE produktionsmaschinen(
    globalid SMALLINT,
    maschinenid SMALLINT,
    variante SMALLINT,
    bezeichnung VARCHAR(200),
    CONSTRAINT pk_maschine PRIMARY KEY(globalid),
    CONSTRAINT u_maschinevariante UNIQUE(maschinenid,variante)
);
```

```
INSERT INTO produktionsmaschinen
(globalid,maschinenid,variante,bezeichnung)
VALUES (1,1,1,'Drehmaschine 1.0');
```

```
INSERT INTO produktionsmaschinen
(globalid,maschinenid,variante,bezeichnung)
VALUES (2,1,2,'Drehmaschine 1.1');
```

```
INSERT INTO produktionsmaschinen
(globalid,maschinenid,variante,bezeichnung)
VALUES (3,2,1,'Drehmaschine 2.0');
```

```
INSERT INTO produktionsmaschinen
(globalid,maschinenid,variante,bezeichnung)
VALUES (4,2,2,'Drehmaschine 2.1');
```

Listing 4.58 Eine UNIQUE-Einschränkung über mehrere Spalten einer Tabelle festlegen

Lösung zu Übung 3

```
CREATE TABLE produktionsmaschinen(  
    globalid SMALLINT,  
    laufzeit INT,  
    maxlaufzeit INT,  
    bezeichnung VARCHAR(200),  
    CONSTRAINT pk_maschine PRIMARY KEY(globalid),  
    CONSTRAINT check_laufzeit_maschinevariante CHECK(laufzeit<maxlaufzeit)  
);  
  
INSERT INTO produktionsmaschinen(globalid,laufzeit,maxlaufzeit,bezeichnung)  
VALUES (1,10234,1500000,'Drehmaschine 1.0');  
  
INSERT INTO produktionsmaschinen(globalid,laufzeit,maxlaufzeit,bezeichnung)  
VALUES (2,23789,2500000,'Drehmaschine 1.0');
```

Listing 4.59 Eine CHECK-Einschränkung mit einer Bedingung über zwei Spalten auf Tabellenebene festlegen

Kapitel 5

Mengenoperationen anwenden

Mussten Sie im Mathematikunterricht auch Schnittmengen und Teilmengen ermitteln? Kennen Sie noch Vereinigungsmengen? Dann wird Ihnen das Thema dieses Kapitels sehr vertraut vorkommen. Wir werden Zeilen mit Mengenoperationen vereinen und Schnittmengen sowie Differenzmengen bilden.

In SQL haben Sie die Möglichkeit, Mengenoperationen wie **UNION**, **INTERSECT** oder **EXCEPT** auf Ergebnistabellen anzuwenden. Halten wir uns zu Beginn dieses Kapitels erst einmal kurz vor Augen, was es mit diesen Mengenoperationen auf sich hat, bevor ich im weiteren Verlauf genauer auf die jeweilige Mengenoperation eingehen werde.

5.1 Mengenoperationen auf Ergebnistabellen anwenden

In diesem Abschnitt sehen wir uns die grundlegenden Mengenoperationen an. Zu ihnen zählen:

- ▶ die Vereinigungsmenge
- ▶ die Schnittmenge
- ▶ die Differenzmenge

Damit Sie die grundlegenden Mengenoperationen auch nachvollziehen können, sind in der Übungsdatenbank zwei Tabellen mit den Bezeichnungen *mengea* und *mengeb* hinterlegt. Beide Tabellen enthalten eine Spalte *element*. Wenn Sie in SQL z. B. Vereinigungsmengen bilden, beziehen sie sich immer auf Spaltenwerte. In diesen Beispielen nutzen wir die Spaltenwerte der Spalte *element*, um sie beispielsweise zu vereinen oder auch die Schnitt- oder eine Differenzmenge zu bilden.

5.1.1 Eine Vereinigungsmenge aus zwei Mengen bilden

Beginnen wir mit der Vereinigungsoperation. In [Abbildung 5.1](#) sind die Mengen A und B gegeben. Menge A enthält die ganzzahligen Elemente 1, 2, 3, 4, 5, 6 und 8. Menge B wiederum enthält die ganzzahligen Elemente 2, 5, 6, 7, 8, 9 und 10.

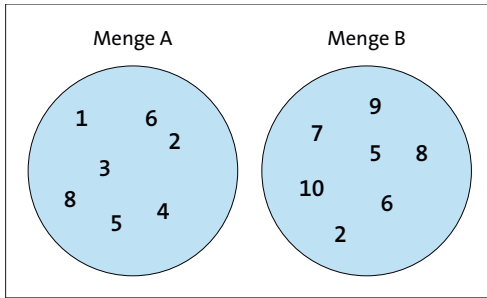


Abbildung 5.1 Die Mengen A und B

Wenn wir eine *Vereinigungsmenge* bilden, so vereinigen wir sämtliche Elemente einer Menge A mit sämtlichen Elementen einer Menge B. Aus den beiden Mengen ergibt sich eine neue Menge C, die die Elemente aus den Mengen A und B vereint enthält. In Abbildung 5.2 sehen Sie die Vereinigungsmenge C mit ihren Elementen, die sich aus der Vereinigung der beiden Mengen A und B ergeben hat.

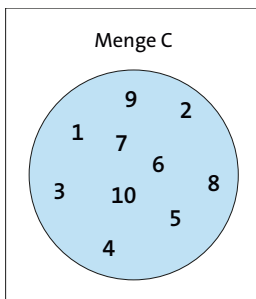


Abbildung 5.2 Vereinigungsmenge C mit den Elementen aus den Mengen A und B

Sie können auch die Elemente aus mehr als zwei Mengen vereinigen. In dieser Vereinigungsmenge sind die in den Mengen vorkommenden Elemente eindeutig enthalten.

Es ist auch möglich, eine mehrdeutige Ergebnismenge zu erstellen. Dann kämen die Elemente, die in den Mengen A und B gleichzeitig enthalten sind, auch in der Ergebnismenge mehrfach vor. Dazu brauchen Sie das Schlüsselwort **UNION ALL**, auf das ich gleich noch zu sprechen komme. Sehen wir uns aber zunächst an, wie eine eindeutige Vereinigungsmenge in SQL dargestellt wird.

SQL ermöglicht uns mit der **UNION**-Klausel, Ergebnislisten von **SELECT**-Abfragen zu vereinen. Um die **UNION**-Vereinigungsoperation auszuprobieren, sind in der *uebungsdatenbank* die Tabellen *mengea* und *mengemb* mit den hier dargestellten Elementen hinterlegt. Beide Tabellen enthalten eine Spalte namens *element*. In den Spalten liegen die Spaltenwerte, die jeweils den Mengen A und B zugeordnet sind.

Als Erstes sehen wir uns den Inhalt der Tabelle *mengea* mit einer einfachen **SELECT**-Abfrage an:

```
SELECT element FROM mengea;
```

Listing 5.1 Die Tabelle »mengea« abfragen

element
1
2
3
4
5
6
8

Tabelle 5.1 Ergebnis der **SELECT**-Abfrage auf die Tabelle »mengea«

Als Nächstes betrachten wir die Elemente, die in der Tabelle *mengeb* enthalten sind:

```
SELECT element FROM mengeb;
```

Listing 5.2 Die Tabelle »mengeb« abfragen

element
2
5
6
7
8
9
10

Tabelle 5.2 Ergebnis der **SELECT**-Abfrage auf die Tabelle »mengeb«

Unser nächstes Ziel ist, eine Vereinigungsmenge mit der **UNION**-Klausel zu realisieren:

```
SELECT element FROM mengea
UNION
SELECT element FROM mengeb;
```

Listing 5.3 Die Inhalte zweier Ergebnistabellen von SELECT-Abfragen vereinigen

Sehen wir uns zunächst die erste Abfrage an. In der **SELECT**-Anweisung wählen wir die Spalte *element* der Tabelle *mengea* aus. Es folgt das Schlüsselwort **UNION**, mit dem wir festlegen, dass wir die Ergebnistabelle der ersten Abfrage eindeutig mit einer Ergebnistabelle einer zweiten Abfrage vereinigen werden. Diese zweite Abfrage wählt ebenfalls die Spalte *element* aus, diesmal allerdings aus der Tabelle *mengeb*.

element
1
2
3
4
5
6
8
7
9
10

Tabelle 5.3 Ergebnistabelle der Vereinigung von »mengea« und »mengeb«

Sie haben es hier mit einer Vereinigung zu tun, die nur eindeutige Elemente in der Ergebnismenge zulässt. Das bedeutet: Wenn Elementwerte in der Ergebnismenge (Tabelle) mehrfach vorkommen, so werden sie jeweils auf einen Wert verdichtet.

Wollen Sie in der Vereinigungsmenge auch die Duplikate sehen, müssen Sie die **UNION**-Klausel um eine weitere Angabe ergänzen: Hinter der **UNION**-Klausel geben Sie mit dem Schlüsselwort **ALL** an, dass Sie eine Vereinigung aller Elemente, also der Spaltenwerte, wünschen. Es werden nun auch gleiche Elemente in die Ergebnismenge der Vereinigung mit aufgenommen:

```
SELECT element FROM mengea  
UNION ALL  
SELECT element FROM mengeb;
```

Listing 5.4 Eine Vereinigungsoperation von Werten, die auch Duplikate zulässt

In [Tabelle 5.4](#) sehen Sie, dass jetzt auch die mehrfach vorkommenden ganzen Zahlen 2, 6 und 8 in der vereinigten Ergebnistabelle wiederholt vorkommen.

element
1
2
2
3
4
5
5
6
6
7
8
8
9
10

Tabelle 5.4 Die Anweisung ALL zeigt alle Zahlen der Vereinigungsmenge an – auch die Duplikate.

5.1.2 Eine Schnittmenge bilden

Ausgangspunkt ist wieder eine Betrachtung von zwei Mengen. Eine Schnittmenge wird aus den Elementen gebildet, die in beiden Mengen enthalten sind.

Die in [Abbildung 5.3](#) gezeigte Schnittmenge (der überlappende Bereich) der Mengen A und B sagt aus, dass die Elemente 5, 6, 2 und 8 in beiden Mengen enthalten sind.

Eine Schnittmenge von zwei Mengen ermitteln Sie in SQL mit der **INTERSECT**-Klausel.

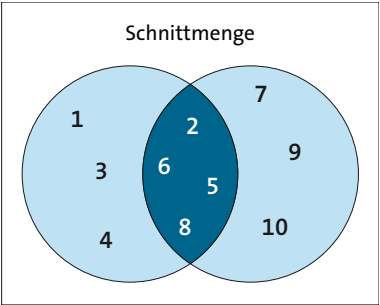


Abbildung 5.3 Schnittmenge aus den Mengen A und B



Kein INTERSECT-Operator in der MySQL-Datenbank vor Version 8.0.31

MySQL verfügt erst seit Version 8.0.31 über die **INTERSECT**-Klausel. Falls Sie eine MySQL-Datenbank vor der Version 8.0.31 nutzen, sollten Sie diesen Abschnitt dennoch aufmerksam lesen, denn die **INTERSECT**-Klausel ist Bestandteil des SQL-Standards.

Die auf Grundlage der MySQL-Datenbank entwickelte MariaDB-Datenbank verfügt seit der Version 10.3 ebenfalls über den **INTERSECT**-Operator.

In [Listing 5.5](#) sehen Sie die erste Abfrage, in der die Werte der Spalte *element* aus der Tabelle *mengea* ermittelt werden. Es folgt das Schlüsselwort **INTERSECT**, mit dem wir der Datenbank mitteilen, dass wir eine Schnittmenge mit den Spaltenwerten einer weiteren Abfrage ermitteln werden. Hinter dem Schlüsselwort **INTERSECT** geben wir schließlich die zweite **SELECT**-Abfrage an, in der wir ebenfalls die Spaltenwerte der Spalte *element* aus der Tabelle *mengeb* abfragen:

```
SELECT element FROM mengea
INTERSECT
SELECT element FROM mengeb;
```

Listing 5.5 Eine Schnittmenge aus zwei Abfragen ermitteln

element
8
5
6
2

Tabelle 5.5 Ergebnis für die zwei **SELECT**-Abfragen aus [Listing 5.5](#)

In der Tabelle sind ausschließlich die Elemente 8, 5, 6 und 2 enthalten, die jeweils in den Mengen A und B vorkommen und somit die Schnittmenge bilden.

5.1.3 Eine Differenzmenge bilden

In SQL steht Ihnen zur Differenzbildung der **EXCEPT**-Mengenoperator zur Verfügung. Damit subtrahieren Sie Spaltenwerte, die Ihnen von **SELECT**-Abfragen zurückgeliefert werden, voneinander und bilden so eine Differenz.

Kein EXCEPT-Operator in der MySQL-Datenbank-vor Version 8.0.31

MySQL unterstützt den **EXCEPT**-Mengenoperator ab Version 8.0.31. Falls Sie eine MySQL-Datenbank in einer älteren Version (z. B. 8.0.30) verwenden, sollten Sie aber auch diesen Abschnitt aufmerksam lesen, da die **EXCEPT**-Klausel zum SQL-Standard gehört. Die MariaDB-Datenbank bietet die **EXCEPT**-Klausel seit Version 10.3 an.

Wir betrachten zunächst wieder die Mengen A und B aus [Abbildung 5.4](#). Wenn Menge B von Menge A subtrahiert wird, werden aus Menge A sämtliche Elemente entfernt, die auch in Menge B enthalten sind. Es bleiben in Menge A nur noch die Elemente 1, 3 und 4 erhalten. Die Differenzbildung können Sie auch als »Menge A ohne Menge B« umschreiben.

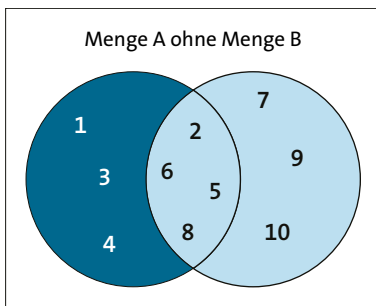


Abbildung 5.4 Menge A ohne Menge B

In [Listing 5.6](#) zeige ich Ihnen, wie Sie dies in SQL umsetzen:

```
SELECT element FROM mengea
EXCEPT
SELECT element FROM mengeb;
```

Listing 5.6 Differenz zwischen den Spaltenwerten von Menge A und von Menge B mit einer **EXCEPT**-Klausel ermitteln

Sie finden die Differenzmenge der Werte der Spalte *element* aus der ersten Abfrage und der Werte der Spalte *element* aus der zweiten Abfrage, indem Sie das Schlüssel-

wort **EXCEPT** hinter der Abfrage notieren, aus der die Spaltenwerte entfernt werden sollen.

Hinter das Schlüsselwort **EXCEPT** schreiben Sie die Abfrage, die Ihnen die Spaltenwerte liefert, die von der ersten Abfrage subtrahiert werden sollen. Voraussetzung ist natürlich, dass es etwas zum Subtrahieren gibt. In der zweiten Abfrage müssen also Spaltenwerte vorhanden sein, die auch in der ersten Abfrage vorhanden sind.

element
4
1
3

Tabelle 5.6 Ergebnis mit dem EXCEPT-Mengenoperator aus [Listing 5.6](#)

Das Resultat der Differenzoperation besteht nur noch aus den Elementen, die ausschließlich in Menge A enthalten sind. Die Elemente, die auch in Menge B enthalten waren, wurden von Menge A subtrahiert.



Zusammenfassung: Mengenoperationen

Es gibt drei Mengenoperationen, die in SQL auf Ergebniszeilen mit Spaltenwerten angewendet werden können:

- ▶ Vereinigungsmenge (**UNION**)
- ▶ Schnittmenge (**INTERSECT**)
- ▶ Differenzmenge (**EXCEPT**)

Achtung: MySQL unterstützt die Bildung einer Schnittmenge und Differenzmenge erst ab Version 8.0.31!

5.2 Eine Vereinigungsmenge bilden (UNION)

Bisher haben Sie die Mengenoperationen nur auf Werte einer Spalte angewandt. Wie sieht es aber aus, wenn wir Mengenoperationen mit mehreren Spalten und folglich mehreren Spaltenwerten durchführen wollen?

Wenn Sie Zeilen aus zwei Ergebnistabellen, die aus mehreren Spalten bestehen, ohne weitere Angaben einer Mengenoperation wie **UNION** unterziehen, werden nach der Vereinigung alle Spaltenwerte einer Zeile mit allen Spaltenwerten der restlichen Zei-

len verglichen. Wenn eine Zeile mehrfach mit gleichen Spaltenwerten ermittelt wird, wird sie auf eine Zeile verdichtet.

Optional können Sie hinter der **UNION**-Klausel die aus Kapitel 2 bekannte **DISTINCT**-Klausel notieren. Wenn Sie nur die **UNION**-Klausel verwenden, erhalten Sie die Zeilen eindeutig zurück, ohne mehrfach vorkommende gleiche Zeilen. **DISTINCT** wird also bereits standardmäßig verwendet, ohne dass Sie das Schlüsselwort explizit angeben, Sie können es aber notieren, um eine Anweisung klarer zu machen.

5.2.1 Vereinigungsmengen von Ergebnistabellen bilden

Wir wollen das Ganze nun in die Praxis überführen. Als Erstes sehen wir uns zwei Tabellen an. In der ersten Tabelle, *produkt*, sind die Artikel des Unternehmens hinterlegt. In der zweiten Tabelle, *produktarchiv*, sind die Artikel hinterlegt, die einmal vom Unternehmen vertrieben wurden, aber aus dem Programm genommen wurden. Alle Informationen der Tabelle *produkt* finden Sie in Tabelle 5.7.

INTEGER ↓ artikelid	VAR-CHAR(300) ↓ artikelbezeichnung	DECIMAL(8,2) ↓ preis	VAR-CHAR(200) ↓ kategorie	VAR-CHAR(300) ↓ legierung
1	Schraube 1	2.56	Schrauben	Zink/Eisen
2	Schraube 2	2.30	Schrauben	Kupfer/Eisen
3	Schraube 3	2.21	Schrauben	Nickel/Eisen
4	Schraube 4	2.11	Schrauben	Gold/Eisen
5	Schraube 5	1.50	Schrauben	Silber/Eisen

Tabelle 5.7 Die Tabelle »produkt«

Die Tabelle *produkt* eignet sich hervorragend, um die Spaltenwerte *artikelbezeichnung*, *kategorie* und *legierung* abzufragen.

SELECT artikelbezeichnung,kategorie,legierung **FROM** produkt;

Listing 5.7 Eine erste Abfrage für eine Mengenoperation auf Zeilen- und Spaltenebene formulieren

Als Nächstes betrachten wir das *produktarchiv* in Tabelle 5.8. Die Spalten, Datentypen und Struktur dieser Tabelle sind identisch mit denjenigen der Tabelle *produkt*.

Auch die Tabelle *produktarchiv* kann wie die Tabelle *produkt* abgefragt werden:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv;
```

Listing 5.8 Eine zweite Abfrage für eine Mengenoperation auf Zeilen- und Spaltenebene formulieren

INTEGER ↓ artikelid	VAR-CHAR(300) ↓ artikelbezeichnung	DECIMAL(8,2) ↓ preis	VAR-CHAR(200) ↓ kategorie	VAR-CHAR(300) ↓ legierung
100	Schraube A	2.56	Schrauben	Zink/Eisen
101	Schraube 2	2.30	Schrauben	Kupfer/Eisen
102	Schraube B	2.21	Schrauben	Nickel/Eisen
103	Schraube 4	2.11	Schrauben	Gold/Eisen
104	Schraube C	1.50	Schrauben	Silber/Eisen

Tabelle 5.8 Die Tabelle »produktarchiv«

Eine Mengenoperation mit Zeilen aus zwei Ergebnistabellen und deren Spaltenwerten setzt voraus, dass die Struktur der ersten und zweiten Ergebnistabelle identisch ist. Die Struktur der zweiten **SELECT**-Anweisung muss also so gestaltet sein, dass die Spalten in der gleichen Reihenfolge angeordnet werden wie die Spalten der ersten Abfrage. Das gilt für alle ausgewählten Spalten und die jeweils zugeordneten Datentypen. Die Reihenfolge der Spalten der ersten Abfrage mit ihren Datentypen muss also mit der Reihenfolge der Spalten der zweiten Abfrage mit ihren Datentypen übereinstimmen. Das heißt aber nicht, dass die Spaltennamen identisch sein müssen.

In [Listing 5.9](#) sehen Sie, wie die zwei **SELECT**-Anweisungen mit einer **UNION**-Klausel vereinigt werden:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
UNION
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv;
```

Listing 5.9 Die Ergebnisse aus zwei Abfragen vereinen

[Tabelle 5.9](#) zeigt uns das Ergebnis der Vereinigungsoperation mit einer **UNION**-Klausel.

artikelbezeichnung	kategorie	legierung
Schraube 1	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen
Schraube 3	Schrauben	Nickel/Eisen
Schraube 4	Schrauben	Gold/Eisen
Schraube 5	Schrauben	Silber/Eisen
Schraube A	Schrauben	Zink/Eisen
Schraube B	Schrauben	Nickel/Eisen
Schraube C	Schrauben	Silber/Eisen

Tabelle 5.9 Ergebnis der Vereinigungsoperation der beiden Tabellen

Wenn Sie [Tabelle 5.7](#) und [Tabelle 5.8](#) betrachten, sehen Sie, dass die Artikel Schraube 2 und Schraube 4 jeweils in den Tabellen *produkt* und *produktarchiv* enthalten sind. Eine Vereinigungsoperation von **SELECT**-Abfragen ist standardmäßig immer eindeutig. Das gleiche Ergebnis erreichen Sie mit dem optionalen Schlüsselwort **DISTINCT**.

Wenn Sie eine Vereinigungsoperation mit **UNION** durchführen möchten, die auch die mehrfach vorkommenden Zeilen enthält, geben Sie hinter der **UNION**-Klausel das Schlüsselwort **ALL** an:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
UNION ALL
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv;
```

Listing 5.10 Alle Zeilen (auch die mehrfach vorkommenden) einer Vereinigung abfragen

artikelbezeichnung	kategorie	legierung
Schraube 1	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen
Schraube 3	Schrauben	Nickel/Eisen
Schraube 4	Schrauben	Gold/Eisen

Tabelle 5.10 Ergebnis der Abfrage aus [Listing 5.10](#)

artikelbezeichnung	kategorie	legierung
Schraube 5	Schrauben	Silber/Eisen
Schraube A	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen
Schraube B	Schrauben	Nickel/Eisen
Schraube 4	Schrauben	Gold/Eisen
Schraube C	Schrauben	Silber/Eisen

Tabelle 5.10 Ergebnis der Abfrage aus [Listing 5.10](#) (Forts.)

Das Ergebnis enthält jetzt auch die mehrfach vorkommenden Datensätze, deren Spaltenwerte identisch sind.

Bisher haben wir ausschließlich Beispiele betrachtet, die inhaltlich und formal sinnvoll waren. Im nächsten Beispiel zeige ich Ihnen, dass es völlig ausreicht, die **UNION**-Klausel formal korrekt zu verwenden. Sie können also auch Spaltenwerte von Zeilen vereinigen, die inhaltlich nicht füreinander geschaffen sind. Als formale Voraussetzung gilt, dass die Anzahl, Reihenfolge und Datentypen der Spalten übereinstimmen.

Sehen wir uns als Nächstes [Listing 5.11](#) an:

```
SELECT artikelbezeichnung, legierung
FROM produkt
UNION
SELECT legierung, artikelbezeichnung
FROM produktarchiv;
```

Listing 5.11 Spalten von Zeilen vereinigen, die inhaltlich nicht zusammenpassen

Hier interessieren uns die Spaltenauswahllisten der beiden **SELECT**-Abfragen, in der einmal die Spalten *artikelbezeichnung* und *legierung*, dann aber die Spalten *legierung* und *artikelbezeichnung* abgefragt werden. Den Spalten wurde während der Tabellen-erstellung jeweils der Datentyp **VARCHAR(300)** zugeordnet. Die Datentypen passen also, um die Vereinigungsoperation durchzuführen.

artikelbezeichnung	legierung
Schraube 1	Zink/Eisen
Schraube 2	Kupfer/Eisen

Tabelle 5.11 Inhaltlich unsinnige Vereinigungsoperation

artikelbezeichnung	legierung
Schraube 3	Nickel/Eisen
Schraube 4	Gold/Eisen
Schraube 5	Silber/Eisen
Zink/Eisen	Schraube A
Kupfer/Eisen	Schraube 2
Nickel/Eisen	Schraube B
Gold/Eisen	Schraube 4
Silber/Eisen	Schraube C

Tabelle 5.11 Inhaltlich unsinnige Vereinigungsoperation (Forts.)

In den beiden Spalten sind jetzt alle Einträge vorhanden, was sicherlich nicht gewollt war und kein sinnvolles Ergebnis darstellt. Schlimmstenfalls werden Ihre Daten im Ergebnis inkonsistent, da sich nun Informationen in den »falschen« Spalten finden. Eine Datenbank kann also nur formal sicherstellen, dass die Vereinigung korrekt erfolgt. Für die inhaltliche Richtigkeit Ihrer Vereinigungsoperationen sind jedoch Sie verantwortlich.

Als Nächstes sehen wir uns an, wie sich Zeilen, die ausschließlich **NULL**-Werte enthalten, in Vereinigungsoperationen verhalten. In [Listing 5.12](#) fügen wir zwei Datensätze mit **NULL**-Werten für die Spalten *artikelbezeichnung*, *preis*, *kategorie* und *legierung* in die Tabellen *produkt* und *produktarchiv* ein:

```
INSERT INTO produkt
VALUES (6,NULL,NULL,NULL,NULL);

INSERT INTO produktarchiv
VALUES (105,NULL,NULL,NULL,NULL);
```

Listing 5.12 Jeweils eine Zeile mit **NULL**-Werten in die Tabellen »produkt« und »produktarchiv« einfügen

Nutzen Sie wieder die Vereinigungsabfrage aus [Listing 5.10](#), um zu prüfen, wie sich eine **UNION ALL**-Operation verhält, wenn in beiden Ergebnistabellen ausschließlich **NULL**-Werte für die Spaltenauswahlwerte enthalten sind.

In [Tabelle 5.12](#) sehen Sie das Ergebnis der Vereinigungsabfrage.

artikelbezeichnung	kategorie	legierung
Schraube 1	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen
Schraube 3	Schrauben	Nickel/Eisen
Schraube 4	Schrauben	Gold/Eisen
Schraube 5	Schrauben	Silber/Eisen
NULL	NULL	NULL
Schraube A	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen
Schraube B	Schrauben	Nickel/Eisen
Schraube 4	Schrauben	Gold/Eisen
Schraube C	Schrauben	Silber/Eisen
NULL	NULL	NULL

Tabelle 5.12 Vereinigung von zwei Ergebnistabellen, die auch Zeilen enthalten, in denen nur NULL-Werte stehen

In den Tabellen *produkt* und *produktarchiv* ist jeweils ein Datensatz enthalten, dessen Werte für die Spalten *artikelbezeichnung*, *kategorie* und *legierung* **NULL** sind. Die Ergebnistabelle zeigt, dass auch diese Zeilen in einer **UNION ALL**-Vereinigungsoperation berücksichtigt werden. Wenn Sie eine Schnittmengenoperation mit dem **INTERSECT**-Operator oder eine Differenzmengenoperation mit dem **EXCEPT**-Operator durchführen, würden diese Zeilen mit **NULL**-Werten genauso in den jeweiligen Operationen berücksichtigt. Das gilt allerdings nur für und neue Versionen des MySQL-Datenbanksystems, da MySQL diese Mengenoperationen erst seit Version 8.0.31 unterstützt.

Mengenoperationen könnten Sie mit einer **UNION**-Klausel auf beliebig viele Ergebnistabellen anwenden. In [Listing 5.13](#) werden die Ergebnistabellen von drei **SELECT**-Abfragen mit ihren Spaltenwerten mithilfe einer **UNION**-Klausel vereinigt:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
UNION ALL
SELECT artikelbezeichnung,kategorie,legierung
```

```
FROM produktarchiv
UNION ALL
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt;
```

Listing 5.13 Mehr als zwei Ergebnistabellen vereinen

Zusammenfassung: Regeln für Mengenoperationen

Für die Spaltenauswahllisten der an Mengenoperationen beteiligten **SELECT**-Abfragen gilt:

- ▶ Die Spalten und ihre Datentypzuordnungen müssen in jeder Abfrage übereinstimmen.
- ▶ **NULL**-Werte verhalten sich besonders und stellen daher eine Ausnahme dar. Da ein **NULL**-Wert ja kein Wert ist und somit auch keinem Datentyp zuzuordnen ist, können Sie ihn mit Spaltenwerten vereinen. In diesem Fall gilt die Regel nicht, dass die Datentypen übereinstimmen müssen.
- ▶ Die Anzahl der Spalten in der Spaltenauswahlliste muss in jeder an einer Mengenoperation beteiligten **SELECT**-Abfrage gleich sein.
- ▶ Auch die Vereinigung von Zeilen, deren Spalten ausschließlich **NULL**-Werte enthalten, ist möglich.
- ▶ Eine **UNION**-Vereinigungsoperation und auch die **INTERSECT**- und **EXCEPT**-Operationen vereinigen standardmäßig die Zeilen von zwei **SELECT**-Abfragen eindeutig. Optional können Sie hinter der **UNION**-Klausel das Schlüsselwort **DISTINCT** angeben. Kommt eine exakt gleiche Zeile in beiden Ergebnistabellen vor, so wird sie zu einer Zeile verdichtet.
- ▶ Wenn auch Duplikate in einer Vereinigungsoperation berücksichtigt werden sollen (also nicht zu einer Zeile verdichtet werden sollen), verwenden Sie die **UNION**-Klausel mit dem Schlüsselwort **ALL**.
- ▶ Es gibt keine Begrenzung für die Anzahl der Vereinigungen von Ergebnislisten.



5.2.2 Übungen

Übung 1

In den Tabellen *niederlassungbelgien* und *niederlassungholland* sind Mitarbeiter gespeichert, die zu den Niederlassungen gehören. Formulieren Sie jeweils eine Abfrage, in der Sie die Spalten *name* und *vorname* der Tabellen *niederlassungbelgien* und *niederlassungholland* abfragen. Anschließend bilden Sie mit diesen beiden Abfragen eine eindeutige Vereinigungsmenge (also ohne Duplikate).

Übung 2

Formulieren Sie nochmals zwei Abfragen, in denen Sie jeweils die Spalten *name* und *vorname* der Tabellen *niederlassungholland* und *niederlassungbelgien* abfragen. Bilden Sie anschließend mit diesen beiden Abfragen eine Vereinigungsmenge mit Duplikaten.

Übung 3

In dieser Übung formulieren Sie drei **SELECT**-Abfragen, in denen Sie jeweils die Spalten *name* und *vorname* der Tabellen *niederlassungholland*, *niederlassungbelgien* und *niederlassungschweiz* abfragen. Im Anschluss bilden Sie mit den drei **SELECT**-Abfragen eine eindeutige Vereinigungsmenge (ohne Duplikate).

Lösung zu Übung 1

```
/* Die Spalten name und vorname der Tabelle niederlassungholland abfragen */  
SELECT name,vorname FROM niederlassungholland;
```

```
/* Die Spalten name und vorname der Tabelle niederlassungbelgien abfragen */  
SELECT name,vorname FROM niederlassungbelgien;
```

```
/* Abfrageergebnisse eindeutig vereinen */  
SELECT name,vorname FROM niederlassungholland  
UNION  
SELECT name,vorname FROM niederlassungbelgien;
```

Listing 5.14 Eine eindeutige Vereinigungsmenge aus zwei Ergebnistabellen bilden

Lösung zu Übung 2

```
/* Abfrageergebnisse mit Duplikaten vereinen */  
SELECT name,vorname FROM niederlassungholland  
UNION ALL  
SELECT name,vorname FROM niederlassungbelgien;
```

Listing 5.15 Eine Vereinigungsmenge mit Duplikaten bilden

Lösung zu Übung 3

```
/* Abfrageergebnisse eindeutig vereinen */  
SELECT name,vorname FROM niederlassungholland  
UNION  
SELECT name,vorname FROM niederlassungbelgien
```


UNION

```
SELECT name,vorname FROM niederlassungschweiz;
```

Listing 5.16 Eine Vereinigungsmenge aus drei Ergebnistabellen bilden

5.3 Die Schnittmenge von Ergebnistabellen bilden (INTERSECT)

Die **UNION**-Vereinigungsoperation vereinigt die Zeilen von zwei Abfragen, indem sie die eindeutigen Zeilen in einer Vereinigungsmenge ermittelt. Zeilen, die in beiden Abfragen als Ergebniszeilen vorkommen, werden in eine Zeile verdichtet. Exakt diese Zeilen, die die **UNION**-Klausel verdichten würde, ermittelt die **INTERSECT**-Klausel als *Schnittmenge* aus zwei oder mehreren Ergebnistabellen. In [Abschnitt 5.3.1](#) nutzen wir wieder die Tabellen *produkt* und *produktarchiv*, um die Schnittmenge von Zeilen (Zeilen mit exakt gleichen Spaltenwerten) aus den Ergebnistabellen zu ermitteln.

Keine Schnittmengenoperation in der MySQL-Datenbank vor Version 8.0.31

Die Schnittmengenoperation wird von der MySQL-Datenbank ab Version 8.0.31 angeboten. Für den Fall, dass Sie eine Version kleiner als 8.0.31 verwenden, zeige Ihnen in [Abschnitt 5.3.1](#) eine alternative Variante, die das Ergebnis der Schnittmengenoperation nachbildet. Diese Alternative wird auch immer wieder in den Lösungen zu den Aufgaben auftauchen.



5.3.1 Schnittmengen von Ergebnistabellen

Die Schnittmenge aus Zeilen von zwei Ergebnistabellen erhalten Sie, wenn in beiden Ergebnistabellen exakt gleiche Zeilen existieren. Die einzelnen Spaltenwerte der Spalten, die in den Spaltenauswahllisten abgefragt werden, müssen also übereinstimmen. In [Listing 5.17](#) nutzen wir die **INTERSECT**-Klausel, um die Schnittmenge aus den Ergebnistabellen von zwei **SELECT**-Abfragen zu ermitteln:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
INTERSECT
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv;
```

Listing 5.17 Eine Schnittmenge aus den Zeilen von zwei Ergebnistabellen mit **INTERSECT** ermitteln (MariaDB, PostgreSQL, MS SQL Server)

[Listing 5.18](#) zeigt Ihnen, wie Sie mit verbundenen Zeilen die Ermittlung einer Schnittmenge für eine MySQL-Datenbank (kleiner als Version 8.0.31) nachbilden können,

indem Sie einen **INNER JOIN** verwenden.¹ Bei diesem Thema geht es um das Verbinden von Zeilen in Tabellen, dem wir uns intensiv in den Kapiteln 7 bis 10 widmen werden. Schlagen Sie gegebenenfalls dort zunächst nach, wenn Sie die Abfrage genau verstehen wollen.

```
SELECT p.artikelbezeichnung,p.kategorie,p.legierung
FROM produkt p INNER JOIN produktarchiv pa
ON
    p.artikelbezeichnung=pa.artikelbezeichnung
    AND
    p.kategorie=pa.kategorie
    AND p.legierung=pa.legierung
;
```

Listing 5.18 Eine Schnittmenge aus den Zeilen von zwei Tabellen mit einem **INNER JOIN** ermitteln (MySQL < 8.0.31)

Der Nachbau leistet genau das, was wir von einer Schnittmengenoperation erwarten würden. Die Zeilen der Tabellen müssten exakt identisch sein, sonst schaffen sie es nicht in die Ergebnistabelle. Das erreichen wir, indem wir jeden Spaltenwert einer Zeile mit dem Pendant einer Zeile auf Gleichheit prüfen und die Ergebnisse der Ausdrücke mit **AND** verbinden.

In der Spaltenauswahlliste werden jeweils die Spalten *artikelbezeichnung*, *kategorie* und *legierung* der Tabellen *produkt* und *produktarchiv* abgefragt. Immer dann, wenn die Spaltenwerte einer Zeile aus einer Abfrage exakt mit den Spaltenwerten einer weiteren Zeile aus einer anderen Abfrage übereinstimmen, kann eine Schnittmenge zwischen diesen Zeilen ermittelt werden.

Tabelle 5.13 zeigt Ihnen das Ergebnis der Schnittmengenoperation.

artikelbezeichnung	kategorie	legierung
NULL	NULL	NULL
Schraube 2	Schrauben	Kupfer/Eisen
Schraube 4	Schrauben	Gold/Eisen

Tabelle 5.13 Ergebnis für die Schnittmengenbildung der zwei Ergebnistabellen

Es gibt insgesamt drei Zeilen, die gemäß der Spaltenauswahlliste über identische Werte in den Tabellen *produkt* und *produktarchiv* verfügen. Die Zeilen der Artikel Schraube 2 und Schraube 4 sind also in beiden Tabellen enthalten. Die Zeile mit den

¹ Ein Leser hat mich auf diese Idee gebracht. Vielen Dank dafür!

NULL-Werten stammt aus dem vorherigen Abschnitt, in dem wir in die beiden Tabellen *produkt* und *produktarchiv* Datensätze mit NULL-Werten eingefügt haben. Diese Werte stimmen ebenfalls in beiden Tabellen überein.

NULL-Werte werden von der MySQL-Variante in [Listing 5.18](#) nicht berücksichtigt, da sämtliche Kombinationen von Spaltenwerten und NULL-Werten verglichen werden müssten. Diese Komplexität ist Ihnen an dieser Stelle nicht hilfreich.

Wenn es keine übereinstimmenden Zeilen gibt, dann erhalten Sie eine leere Menge als Ergebnistabelle zurück. Eine leere Menge entspricht einer Tabelle ohne Ergebniszeilen.

Zusammenfassung: Schnittmengen aus Ergebnistabellen bilden

Eine Schnittmenge aus zwei Ergebnistabellen kommt immer dann zustande, wenn die Spaltenwerte der Spaltenauswahlliste in zwei Ergebnistabellen übereinstimmen.

Wenn es keine übereinstimmenden Zeilen gibt, erhalten Sie eine leere Schnittmenge zurück.



5.3.2 Übungen

Falls Sie eine MySQL-Datenbank Version kleiner als 8.0.31 verwenden, können Sie für die Übungen die Lösung aus [Listing 5.18](#) verwenden.

Übung 1

In den Tabellen *niederlassungholland* und *niederlassungbelgien* sind Mitarbeiter gespeichert. Formulieren Sie zwei einfache SELECT-Abfragen, in denen Sie die Spalten *name* und *vorname* der beiden Tabellen abfragen. Es gibt Mitarbeiter, die in beiden Niederlassungen tätig sind. Ermitteln Sie die Schnittmenge der Ergebnistabellen der beiden Abfragen.

Übung 2

In dieser Übung formulieren Sie drei einfache Abfragen, in der die Spalten *name* und *vorname* der Tabellen *niederlassungholland*, *niederlassungbelgien* und *niederlassungschweiz* abgefragt werden. Bilden Sie im Anschluss eine Schnittmenge aus allen drei Ergebnistabellen.

Übung 3

In dieser Übung sollen wieder die Mitarbeiter ermittelt werden, die in den Tabellen *niederlassungholland* und *niederlassungbelgien* enthalten sind. Hierzu formulieren Sie zwei SELECT-Abfragen, in denen die Spalten *name*, *vorname* und *status* der ge-

nannten Tabellen abgefragt werden. Begründen Sie, warum Sie in Übung 1 ein anderes Ergebnis erhalten haben.

Lösung zu Übung 1

```
/* Die Spalten name und vorname der Tabelle niederlassungholland abfragen */  
SELECT name,vorname FROM niederlassungholland;
```

```
/* Die Spalten name und vorname der Tabelle niederlassungbelgien abfragen */  
SELECT name,vorname FROM niederlassungbelgien;
```

```
/* Eine Schnittmenge der Ergebnistabellen bilden */  
SELECT name,vorname FROM niederlassungholland  
INTERSECT  
SELECT name,vorname FROM niederlassungbelgien;
```

Listing 5.19 Eine Schnittmenge aus zwei Ergebnistabellen bilden

```
SELECT nh.name, nh.vorname  
FROM  
    niederlassungholland nh  
    INNER JOIN  
    niederlassungbelgien nb  
    ON  
    nh.name=nb.name  
    AND  
    nh.vorname=nb.vorname;
```

Listing 5.20 Eine Schnittmenge aus zwei Ergebnistabellen bilden (MySQL Version < 8.0.31)

Lösung zu Übung 2

```
/* Schnittmenge aus drei Ergebnistabellen bilden */  
SELECT name,vorname FROM niederlassungholland  
INTERSECT  
SELECT name,vorname FROM niederlassungbelgien  
INTERSECT  
SELECT name,vorname FROM niederlassungschweiz;
```

Listing 5.21 Eine Schnittmenge aus drei Ergebnistabellen bilden

```
SELECT nh.name, nh.vorname  
FROM
```

```
niederlassungholland nh
INNER JOIN
niederlassungbelgien nb
ON
    nh.name=nb.name
AND
    nh.vorname=nb.vorname
INNER JOIN niederlassungschweiz nc
ON
    nb.name=nc.name
AND
    nb.vorname=nc.vorname;
```

Listing 5.22 Eine Schnittmenge aus drei Ergebnistabellen bilden (MySQL Version < 8.0.31)

Lösung zu Übung 3

```
SELECT name,vorname,status FROM niederlassungholland
INTERSECT
SELECT name,vorname,status FROM niederlassungbelgien;
```

Listing 5.23 Eine Schnittmenge über drei Spaltenwerte von Ergebniszeilen aus Abfragen bilden

```
SELECT nh.name, nh.vorname,nh.status
FROM
    niederlassungholland nh
INNER JOIN
    niederlassungbelgien nb
ON
    nh.name=nb.name
AND
    nh.vorname=nb.vorname
AND
    nh.status=nb.status;
```

Listing 5.24 Eine Schnittmenge über drei Spaltenwerte von Ergebniszeilen aus Abfragen bilden (MySQL Version < 8.0.31)

Die Antwort ist einfach: Es gibt keine Schnittmenge mehr. Es gibt keine Zeile, in der alle Spaltenwerte übereinstimmend als Ergebnis in der ersten und zweiten Abfrage vorkommen.

5.4 Eine Differenzmenge aus Ergebnistabellen bilden (EXCEPT)

In [Abschnitt 5.1.3](#) wurden von einer Menge A die Elemente subtrahiert, die auch in einer Menge B enthalten sind. Sehen wir uns nun in [Abschnitt 5.4.1](#) an, wie Sie mit der EXCEPT-Klausel aus zwei Ergebnistabellen Differenzmengen bilden.



Keine Differenzmengenoperation in der MySQL-Datenbank vor Version 8.0.31

MySQL bietet die EXCEPT-Klausel ab Version 8.0.31 an. Wenn Sie eine MySQL-Datenbank kleiner als 8.0.31 verwenden, sollten Sie den Abschnitt dennoch aufmerksam weiterlesen. Auch hier werde ich Ihnen Alternativen zum Nachbau aufzeigen.

5.4.1 Differenzmenge von Ergebnismengen bilden

In [Listing 5.25](#) sehen Sie zwei SELECT-Abfragen, für deren Ergebnistabellen mit der EXCEPT-Klausel eine Differenzmenge gebildet wird. [Listing 5.26](#) zeigt eine alternative Möglichkeit, die Differenzmenge mit einem LEFT OUTER JOIN für die MySQL-Datenbank Version kleiner als 8.0.31 nachzubilden.

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
EXCEPT
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv;
```

Listing 5.25 Eine Differenz zwischen Ergebnistabellen von zwei Abfragen ermitteln

```
SELECT p.artikelbezeichnung,p.kategorie,p.legierung
FROM produkt p
LEFT OUTER JOIN
produktarchiv pa
ON
    p.artikelbezeichnung=pa.artikelbezeichnung
    AND
    p.kategorie=pa.kategorie
    AND
    p.legierung=pa.legierung
WHERE
    pa.artikelbezeichnung IS NULL AND pa.kategorie IS NULL AND pa.legierung IS
NULL;
```

Listing 5.26 Eine Differenz zwischen Ergebnistabellen ermitteln (MySQL Version < 8.0.31)

Es werden alle Zeilen ermittelt, deren Kombination von Spaltenwerten ausschließlich in der ersten Abfrage enthalten ist. Alle Zeilen, die auch in der Ergebnistabelle der zweiten Abfrage enthalten sind, werden aus der ersten Ergebnistabelle entfernt.

Tabelle 5.14 enthält nur die Artikel Schraube 1, Schraube 3 und Schraube 5. Die Artikel Schraube 2 und Schraube 4 sind in beiden Ergebnistabellen enthalten und wurden daher durch die EXCEPT-Klausel entfernt. Wenn in Ihren Ergebnistabellen Zeilen vorkommen, deren Spaltenwerte ausschließlich NULL-Werte enthalten, so werden sie ebenfalls durch die EXCEPT-Mengenoperation subtrahiert.

artikelbezeichnung	kategorie	legierung
Schraube 1	Schrauben	Zink/Eisen
Schraube 3	Schrauben	Nickel/Eisen
Schraube 5	Schrauben	Silber/Eisen

Tabelle 5.14 Ergebnis für die Differenzoperation aus Listing 5.25



Zusammenfassung: Differenzmengen bilden (EXCEPT)

Mit dem EXCEPT-Mengenoperator werden alle Zeilen einer ersten Abfrage ermittelt, die nicht in den Zeilen einer zweiten Abfrage enthalten sind.

5.4.2 Übungen

Übung 1

Formulieren Sie zunächst zwei SELECT-Abfragen, in denen die Spalten *name* und *vorname* der Tabellen *niederlassungholland* und *niederlassungbelgien* abgefragt werden. Als Nächstes bilden Sie eine Differenzmenge aus diesen beiden Abfragen. Ermitteln Sie die Mitarbeiter, die ausschließlich in der Niederlassung Holland tätig sind.

Übung 2

Formulieren Sie zunächst drei SELECT-Abfragen, in denen die Spalten *name* und *vorname* der Tabellen *niederlassungholland*, *niederlassungbelgien* und *niederlassungschweiz* abgefragt werden. Ermitteln Sie anschließend, welche Mitarbeiter ausschließlich in der Tabelle *niederlassungholland* und nicht in den Tabellen *niederlassungbelgien* und *niederlassungschweiz* vorhanden sind. Aus dem Ergebnis dieser Differenzmenge sind ausschließlich die Mitarbeiter zu ermitteln, die nicht in der Tabelle *niederlassungschweiz* enthalten sind.

Lösung zu Übung 1

```
SELECT name,vorname FROM niederlassungholland  
EXCEPT  
SELECT name,vorname FROM niederlassungbelgien;
```

Listing 5.27 Eine einfache Differenzmenge aus zwei Ergebnistabellen bilden

```
SELECT nh.name, nh.vorname  
FROM niederlassungholland nh  
LEFT OUTER JOIN  
niederlassungbelgien nb  
ON  
    nh.name=nb.name  
    AND  
    nh.vorname=nb.vorname  
WHERE  
    nb.name IS NULL AND nb.vorname IS NULL;
```

Listing 5.28 Eine einfache Differenzmenge aus zwei Ergebnistabellen bilden (MySQL Version < 8.0.31)

Lösung zu Übung 2

```
SELECT name,vorname FROM niederlassungholland  
EXCEPT  
SELECT name,vorname FROM niederlassungbelgien  
EXCEPT  
SELECT name,vorname FROM niederlassungschweiz;
```

Listing 5.29 Eine Differenzmenge aus drei Ergebnistabellen bilden

```
SELECT nh.name, nh.vorname  
FROM niederlassungholland nh  
LEFT OUTER JOIN  
niederlassungbelgien nb  
ON  
    nh.name=nb.name  
    AND  
    nh.vorname=nb.vorname  
LEFT OUTER JOIN  
niederlassungschweiz nc  
ON  
(nh.name=nc.name AND nh.vorname=nc.vorname)
```


OR

```
(nb.name=nc.name AND nb.vorname=nc.vorname)
```

WHERE

```
nb.name IS NULL AND nc.name IS NULL;
```

Listing 5.30 Eine Differenzmenge aus drei Ergebnistabellen bilden (MySQL Version < 8.0.31)

5.5 Mengenoperationen in Kombination mit einer WHERE-Klausel verwenden

In diesem Abschnitt werden wir die Vereinigungsoperation **UNION** für zwei **SELECT**-Abfragen anwenden und gleichzeitig die **WHERE**-Klausel nutzen, um Zeilen zu filtern.

5.5.1 Vor einer Vereinigungsoperation mit UNION filtern

Wenn Sie der **UNION**-Klausel vor der Vereinigungsoperation von Ergebnistabellen etwas Arbeit abnehmen möchten, können Sie in jeder Abfrage eine **WHERE**-Klausel unterbringen. Das gilt auch für die Schnittmengen- oder die Differenzmengenbildung. Die Vereinigungsoperation bzw. Schnittmengen- oder Differenzmengenbildung bezieht sich dann auf die bereits gefilterten Abfragen.

In [Listing 5.31](#) stelle ich Ihnen so eine Variante vor:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
WHERE artikelbezeichnung='Schraube 1'
UNION ALL
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv
WHERE artikelbezeichnung='Schraube 2';
```

Listing 5.31 Vereinigung von Ergebnislisten mit zwei WHERE-Klauseln

Hier sind zwei **SELECT**-Abfragen jeweils mit einer **WHERE**-Klausel ausgestattet. Es wird also zuerst eine gefilterte Ergebnisliste ermittelt, bevor die **UNION**-Mengenoperation zur Anwendung kommt. **UNION** wird so entlastet, da ein Teil der Vergleiche wegfällt. Bei größeren Vergleichs- und Filteroperationen kommen Sie deutlich schneller an das Ergebnis und verringern den Datenbestand, der von den Operatoren ausgewertet werden muss.

Das Ergebnis der Vereinigung sehen Sie in [Tabelle 5.15](#).

artikelbezeichnung	kategorie	legierung
Schraube 1	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen

Tabelle 5.15 Ergebnis der Vereinigungsoperation mit UNION in Verbindung mit der WHERE-Klausel

Für die zu vereinigenden Abfragen wurden zuerst die Filteroperationen durchgeführt. Dann wurden die Ergebnistabellen vereinigt. In diesem Fall wurde daher nur jeweils eine Zeile pro Ergebnistabelle vereinigt, sodass wir nach der Vereinigung zwei Zeilen als Ergebnis vorfinden. Der Artikel *Schraube 1* findet sich also in der Ergebnisliste der ersten SELECT-Abfrage und der Artikel *Schraube 2* in der Ergebnisliste der zweiten SELECT-Abfrage.



Zusammenfassung: Mengenoperationen in Kombination mit einer WHERE-Klausel nutzen

Die an einer Mengenoperation beteiligten SELECT-Abfragen können jeweils eine WHERE-Klausel zugeordnet bekommen, um gemäß einer Bedingung Zeilen zu filtern. Zuerst werden immer die SELECT-Abfragen mit der WHERE-Klausel von einer Datenbank ausgeführt. Dann werden die Mengenoperationen auf die Ergebnistabellen durchgeführt.

Wenn Sie Ihren Datenbestand auf diesem Weg vorfiltern, werden die Operationen viel schneller abgearbeitet, da weniger Zeilen verglichen werden müssen.

5.5.2 Übung

Neben den Spalten *name* und *vorname* findet sich in den Tabellen *niederlassungholland* und *niederlassungbelgien* die Spalte *status*. In dieser Spalte werden die Spaltenwerte *aktiv* und *inaktiv* gespeichert, da ja ein Mitarbeiter nicht gleichzeitig in zwei Niederlassungen tätig sein kann. Formulieren Sie zunächst zwei Abfragen, in denen die Spalten *name*, *vorname* und *status* der Tabellen *niederlassungholland* und *niederlassungbelgien* abgefragt werden.

Ergänzen Sie die Abfragen um eine WHERE-Klausel, in der lediglich die Zeilen gefiltert werden, bei denen in der Spalte *status* der Wert *aktiv* steht. Im letzten Schritt vereinigen Sie die beiden Ergebnistabellen eindeutig (ohne Duplikate).

Lösung zur Übung

```
SELECT name,vorname,status FROM niederlassungholland
  WHERE status='aktiv'
UNION
SELECT name,vorname,status FROM niederlassungbelgien
  WHERE status='aktiv';
```

Listing 5.32 Eine WHERE-Klausel in Kombination mit dem Vereinigungsoperator UNION verwenden

5.6 Vereinigungsmengen in Kombination mit einer ORDER BY-Klausel

Eine **ORDER BY**-Klausel wird in Mengenoperationen wie **UNION**, **INTERSECT** oder **EXCEPT** immer hinter der letzten **SELECT**-Anweisung notiert. Die **ORDER BY**-Klausel bezieht sich grundsätzlich auf alle Ergebniszeilen, die z. B. von der **UNION**-Klausel vereinigt werden. Das heißt, dass die Sortierung gemäß einem von Ihnen vorgegebenen Sortierkriterium erst erfolgt, wenn alle Mengenoperationen wie **UNION**, **INTERSECT** und **EXCEPT** abgeschlossen wurden.

In [Listing 5.33](#) werden die Zeilen von zwei Ergebnistabellen mit einer **UNION**-Klausel vereinigt. Hinter der letzten Abfrage sehen Sie die **ORDER BY**-Klausel, in der für das Sortierkriterium die Spalte *artikelbezeichnung* angegeben ist:

```
SELECT artikelbezeichnung,kategorie,legierung
FROM produkt
UNION
SELECT artikelbezeichnung,kategorie,legierung
FROM produktarchiv
ORDER BY artikelbezeichnung;
```

Listing 5.33 Vereinigte Ergebnislisten sortiert abfragen

artikelbezeichnung	kategorie	legierung
Schraube 1	Schrauben	Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen
Schraube 3	Schrauben	Nickel/Eisen

Tabelle 5.16 Das Ergebnis der **SELECT**-Abfrage aus [Listing 5.33](#). Die Ergebnistabelle wurde gemäß den Spaltenwerten der Spalte »artikelbezeichnung« aufsteigend sortiert.

artikelbezeichnung	kategorie	legierung
Schraube 4	Schrauben	Gold/Eisen
Schraube 5	Schrauben	Silber/Eisen
Schraube A	Schrauben	Zink/Eisen
Schraube B	Schrauben	Nickel/Eisen
Schraube C	Schrauben	Silber/Eisen
NULL	NULL	NULL

Tabelle 5.16 Das Ergebnis der SELECT-Abfrage aus Listing 5.33. Die Ergebnistabelle wurde gemäß den Spaltenwerten der Spalte »artikelbezeichnung« aufsteigend sortiert. (Forts.)



Zusammenfassung: Mengenoperationen in Kombination mit einer ORDER BY-Klausel nutzen

Eine ORDER BY-Klausel wird bei Mengenoperationen wie UNION, INTERSECT und EXCEPT immer hinter der letzten SELECT-Anweisung notiert, um die resultierende Ergebnistabelle sortiert zurückzuerhalten.

5.6.1 Übungen

Übung 1

Bilden Sie die Vereinigungsmenge aus zwei SELECT-Abfragen, die jeweils die Spalten *name* und *vorname* der Tabellen *niederlassungholland* und *niederlassungbelgien* abfragen. Die Zeilen der Ergebnistabelle sollen sortiert nach den Spaltenwerten der Spalte *name* ausgegeben werden.

Übung 2

Auch diese Übung können Sie leider nicht ausführen, falls Sie eine MySQL-Datenbank Version kleiner als 8.0.31 verwenden. In [Abschnitt 5.4](#) haben Sie eine Möglichkeit kennengelernt, eine EXCEPT-Operation für eine MySQL-Datenbank kleiner als 8.0.31 nachzubilden. Diese Lösungsmöglichkeit können Sie auch für diese Übung verwenden.

Formulieren Sie eine SELECT-Abfrage, in der die Mitarbeiter ermittelt werden, die ausschließlich in der Tabelle *niederlassungholland* und nicht in der Tabelle *niederlassungbelgien* enthalten sind. In die Spaltenauswahlliste nehmen Sie hierzu die Spalten *name* und *vorname* auf, die in beiden Tabellen enthalten sind. Die Zeilen der Ergebnistabelle sollen sortiert nach den Spaltenwerten der Spalte *name* ausgegeben werden.

Lösung zu Übung 1

```
SELECT name,vorname FROM niederlassungholland
UNION
SELECT name,vorname FROM niederlassungbelgien
ORDER BY name;
```

Listing 5.34 Ergebnistabellen mit UNION vereinigen und die Zeilen gemäß den Spaltenwerten einer Spalte sortiert ausgeben

Lösung zu Übung 2

```
SELECT name,vorname FROM niederlassungholland
EXCEPT
SELECT name,vorname FROM niederlassungbelgien
ORDER BY name;
```

Listing 5.35 Die Zeilen einer Differenzmenge gemäß den Spaltenwerten einer Spalte sortiert ausgeben

```
SELECT nh.name,nh.vorname
FROM niederlassungholland nh
LEFT OUTER JOIN
niederlassungbelgien nb
ON
    nh.name=nb.name
AND
    nh.vorname=nb.vorname
WHERE
nb.name IS NULL AND nb.vorname IS NULL
ORDER BY nh.name;
```

Listing 5.36 Die Zeilen einer Differenzmenge gemäß den Spaltenwerten einer Spalte sortiert ausgeben (MySQL Version < 8.0.31)

Kapitel 6

Benutzer, Rollen und ihre Berechtigungen

Bei jeder Speicherung von Daten stellt sich die Frage, wer eigentlich auf welche Daten zugreifen darf – nicht zuletzt, um den Anforderungen an Datensicherheit zu genügen. Dieses Kapitel zeigt Ihnen daher, wie Sie Benutzer inklusive der notwendigen Berechtigungen anlegen oder wieder entfernen können.

Features einer SQL-Datenbank sind die zentrale Datenhaltung und dass mehrere Benutzer gleichzeitig auf einen einheitlichen Datenbestand zugreifen können. Wer jedoch diese Daten lesen, ändern oder in die Datenbank einfügen kann, hängt vom Anwendungskontext ab. So sind beispielsweise für die Mitarbeiter einer Personalabteilung andere Daten von Interesse als für die Mitarbeiter der Abteilung Controlling.

Zuerst widmen wir uns daher der Frage, wer überhaupt einen Datenbankzugriff erhalten soll. Diese Personen müssen als Benutzer in einer Datenbank geführt werden. Im Anschluss stellt sich die Frage, welche Berechtigungen diese Benutzer erhalten sollen.

Oder anders ausgedrückt: Wer (*Benutzer*) soll auf welche Daten (*Tabellen*) wie (*Berechtigungsart*) zugreifen können? Soll der Benutzer die Datensätze einer Tabelle nur lesen oder auch ändern können? Oder darf er sogar neue Datensätze einfügen oder bestehende löschen? In [Abbildung 6.1](#) sehen Sie, dass wir hier drei Aspekte zu beachten haben: die Datenbanknutzer, die Tabellen und die auszuführenden Anweisungen bzw. Berechtigungen.

Voraussetzungen für das Anlegen und Löschen von Datenbankbenutzern

Sie benötigen Administratorberechtigungen, um Datenbankbenutzer anzulegen, zu löschen oder die jeweiligen Rechte zu erteilen.



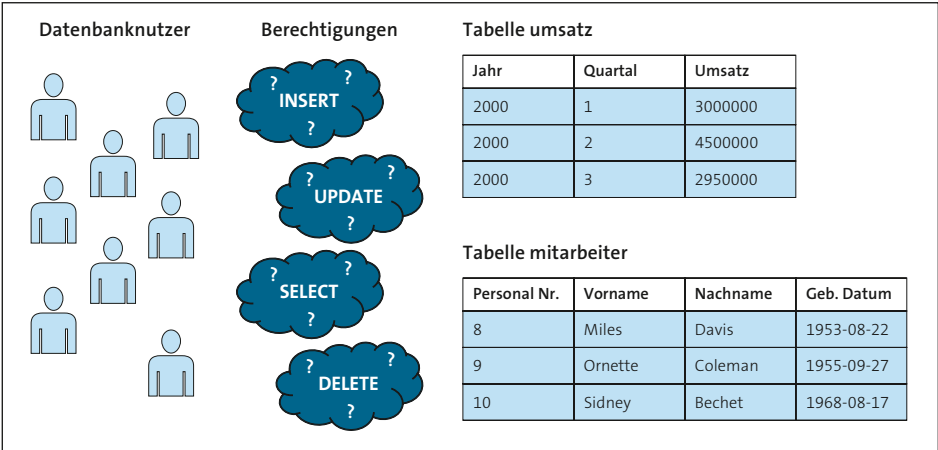


Abbildung 6.1 Tabellen, Datenbanknutzer und Berechtigungen verwalten

6.1 Benutzer anlegen (CREATE USER)

Wie man einen neuen Datenbankbenutzer anlegt, unterscheidet sich je nach verwendeter Datenbank. Aus diesem Grund erkläre ich Ihnen für alle der hier behandelten Datenbanken in einem eigenen Abschnitt, wie Sie einen Nutzer anlegen.

6.1.1 Nutzer in einer MySQL- oder MariaDB-Datenbank anlegen

In einer MySQL- oder MariaDB-Datenbank legen Sie einen Nutzer mit der **CREATE USER**-Anweisung an. Der Anweisung folgt, wie in [Listing 6.1](#) dargestellt, ein Benutzername, den Sie frei wählen können. Um ein Passwort für diesen Benutzer festzulegen, ergänzen Sie hinter dem Benutzernamen **IDENTIFIED BY**, gefolgt von dem Wunschpasswort, das Sie in einfache Hochkommas setzen:

```
CREATE USER hans IDENTIFIED BY 'secret';
```

Listing 6.1 Einen neuen Benutzer in einer MySQL- bzw. MariaDB-Datenbank anlegen

6.1.2 Nutzer in einer PostgreSQL-Datenbank anlegen

Wenn Sie eine PostgreSQL-Datenbank verwenden, nutzen Sie die **CREATE USER**-Anweisung aus [Listing 6.2](#). Dahinter folgt der Name des neuen Benutzers. Jetzt müssen Sie nur noch ein Passwort zuordnen. Hierzu notieren Sie im Anschluss das Schlüsselwort **PASSWORD** und vergeben ein Passwort, das Sie in einfache Hochkommas setzen:

```
CREATE USER hans PASSWORD 'secret';
```

Listing 6.2 Einen neuen Benutzer in einer PostgreSQL-Datenbank anlegen

6.1.3 Nutzer in einer MS SQL Server-Datenbank anlegen

Wenn Sie eine MS SQL Server-Datenbank verwenden, ist es zuerst erforderlich, einen Login anzulegen, bevor Sie einen Benutzer anlegen können. Hierfür notieren Sie zuerst die **CREATE LOGIN**-Anweisung, gefolgt vom Namen des Logins. Dem Login muss noch ein Passwort zugeordnet werden. Hierzu notieren Sie hinter dem Benutzernamen die Schlüsselwörter **WITH PASSWORD** und weisen mit einem Gleichheitszeichen dem Login ein Passwort zu, das Sie in einfache Hochkommas setzen:

```
CREATE LOGIN hans WITH PASSWORD = 'secret';
```

Listing 6.3 Einen Login für eine MS SQL Server-Datenbank anlegen

Jetzt müssen wir mittels **CREATE USER** nur noch den eigentlichen Benutzer anlegen und einem Login zuweisen. Hinter der Anweisung folgt der Benutzername, der dem Login zugewiesen werden soll. (Es liegt natürlich nahe, den gleichen Namen für Login und Benutzer zu verwenden.) Die Zuweisung erfolgt durch die Schlüsselwörter **FOR LOGIN** und die Nennung des entsprechenden Logins:

```
CREATE USER hans FOR LOGIN hans;
```

Listing 6.4 Einen neuen Datenbankbenutzer in einer MS SQL Server-Datenbank anlegen und einem Login zuordnen

6.2 Benutzer entfernen

In [Listing 6.5](#) sehen Sie die Anweisung, die einen Benutzer sowohl in einer MySQL- und einer MariaDB- als auch in einer PostgreSQL-Datenbank wieder entfernt. Hierfür verwenden Sie die **DROP**-Anweisung, mit der Sie auch andere Datenbankobjekte, z. B. Tabellen oder **VIEWS**, löschen können. Um einen Benutzer zu entfernen, geben Sie hinter der Anweisung das Schlüsselwort **USER** an, gefolgt von dem Benutzernamen, den Sie entfernen möchten:

```
DROP USER hans;
```

Listing 6.5 Einen Benutzer entfernen (MySQL, MariaDB, PostgreSQL)

Im Falle einer MS SQL Server-Datenbank müssen Sie zunächst den Login mit der **DROP**-Anweisung entfernen, indem Sie hinter die Anweisung das Schlüsselwort **LOGIN** setzen. Erst im Anschluss lässt sich der Benutzer mittels **DROP USER** löschen. Dabei müssen Sie stets die Reihenfolge aus [Listing 6.6](#) einhalten.

```
DROP LOGIN hans;
DROP USER hans;
```

Listing 6.6 Einen Login und einen Benutzer aus einer MS SQL Server-Datenbank entfernen

6.3 Eine Verbindung für einen Datenbankbenutzer erstellen

Bevor wir eine Datenbankverbindung anlegen können, müssen wir den soeben in [Abschnitt 6.2](#) gelöschten Benutzer erneut anlegen. Wählen Sie hierfür das zu Ihrer Datenbank passende Listing aus [Abschnitt 6.1](#).

6.3.1 Verbindung für eine MySQL-Datenbank einrichten

Wenn Sie mit einer MySQL-Datenbank arbeiten, wählen Sie die erste Registerkarte in der MySQL Workbench aus. In [Abbildung 6.2](#) sehen Sie die bereits bestehende Verbindung. Um eine neue Verbindung einzurichten, klicken Sie auf das Pluszeichen.

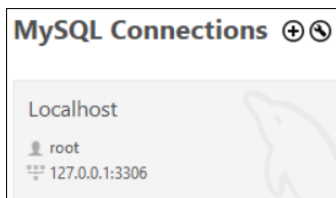


Abbildung 6.2 Aktive MySQL-Datenbankverbindungen

Als Nächstes richten Sie die Verbindung für den Datenbankbenutzer ein. Geben Sie hierzu den Verbindungsnamen (CONNECTION NAME) ein, der der Verbindung zugeordnet werden soll (hier *Verbindung_Hans*). In das Feld USERNAME tragen Sie den Benutzernamen (hier *hans*) ein, für den Sie die Datenbankverbindung einrichten wollen. Klicken Sie auf OK, um die neue Verbindung fertigzustellen.

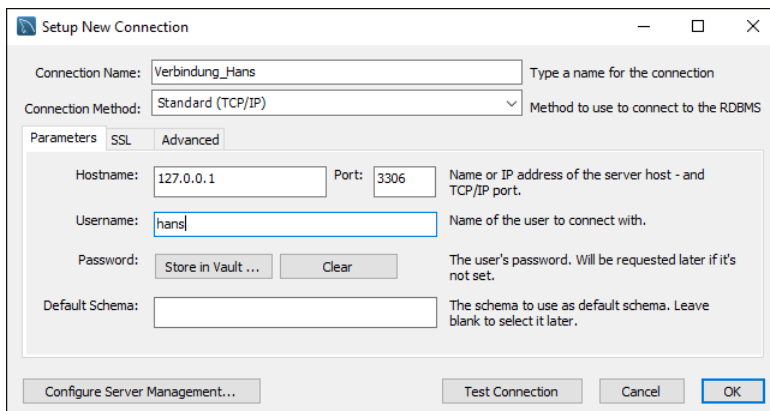


Abbildung 6.3 Neue Verbindung für den Datenbankbenutzer konfigurieren

Nachdem Sie die neue Verbindung eingerichtet haben, steht sie Ihnen im Startfenster der MySQL Workbench zur Verfügung. Die grau hinterlegte Schaltfläche ermöglicht Ihnen jetzt, sich als Benutzer *hans* an der Datenbank anzumelden. Allerdings verfügt

der Benutzer *hans* derzeit noch über keinerlei Berechtigungen, auf Tabellen zuzugreifen. Wie Sie Berechtigungen für einen Benutzer vergeben, lernen Sie in [Abschnitt 6.4](#).

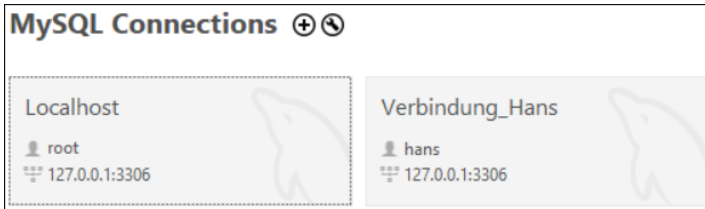


Abbildung 6.4 Die neu angelegte Datenbankverbindung in der Auswahlliste

6.3.2 Verbindung für eine MariaDB-Datenbank herstellen

Wenn Sie mit einer MariaDB-Datenbank arbeiten, nutzen Sie den Client *HeidiSQL*. Dort sehen Sie unterhalb der Menüleiste zwei Stecker-Icons. Über das linke Icon stellen Sie die Verbindung her, indem Sie im Auswahlmenü den Eintrag **NEUES FENSTER** wählen. Es öffnet sich der Verbindungsmanager wie in [Abbildung 6.5](#) gezeigt. (Schneller geht es mit der Tastenkombination **Strg** + **N**.)

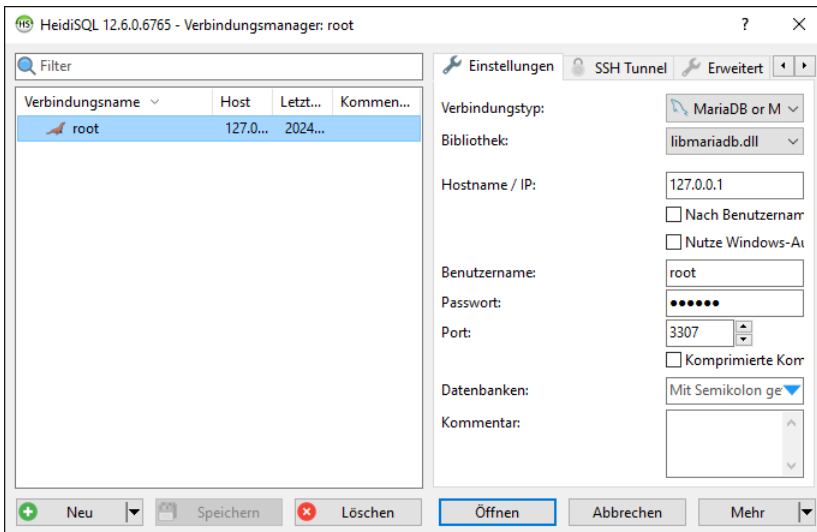


Abbildung 6.5 Der Verbindungsmanager des HeidiSQL-Clients der MariaDB-Datenbank

Im Verbindungsmanager klicken Sie auf das Pluszeichen (+), um eine neue Verbindung einzurichten.

Im linken Bereich des Verbindungsmanagers wird nun eine weitere Verbindung angezeigt. Geben Sie hier einen VERBINDUNGSNAMEN ein; in diesem Beispiel habe ich als Name die Bezeichnung *Verbindung_Hans* eingetragen. Wechseln Sie dann zur Registerkarte **EINSTELLUNGEN**, auf der ebenfalls Eingaben erforderlich sind. Tragen Sie

dort, wie in [Abbildung 6.6](#) gezeigt, den entsprechenden Wert in das Feld BENUTZER-NAME ein, z. B. *hans*. Als Nächstes müssen Sie im Feld PASSWORT das zuvor von Ihnen festgelegte Passwort eingeben.

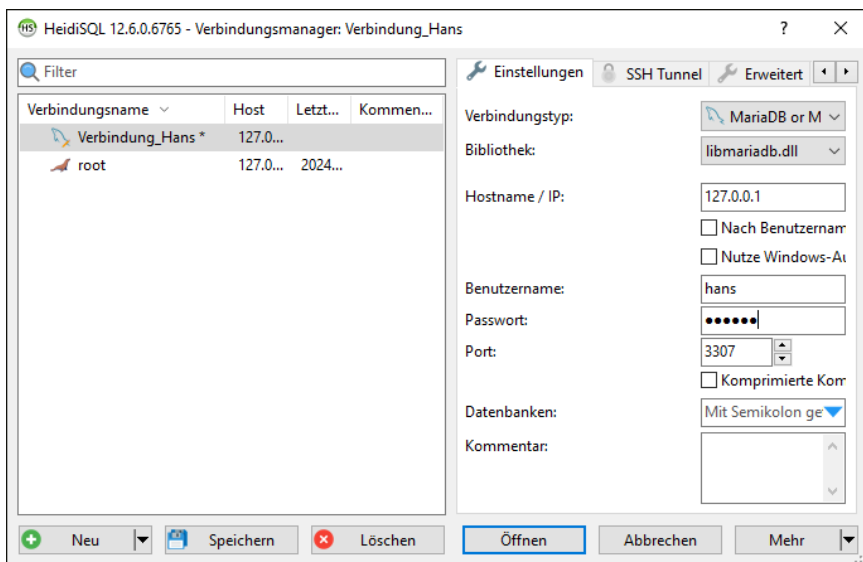


Abbildung 6.6 Eine neue Verbindung für den Datenbanknutzer »hans« einrichten

Jetzt müssen Sie nur noch den PORT angeben, auf dem der MariaDB-Server läuft. Wenn Sie kein anderes Datenbanksystem nutzen, lautet der richtige Port 3306. In diesem Beispiel habe ich jedoch den Wert 3307 eingetragen, da ich auf meinem Computer auch gleichzeitig eine MySQL-Datenbank betreibe. Indem Sie den Port sowohl auf dem Server als auch auf dem Client anpassen, können Sie also auch mehrere Datenbanksysteme auf einem Rechner nutzen.

Klicken Sie abschließend auf **SPEICHERN**, um die Verbindung einzurichten.

6.3.3 Verbindung für eine PostgreSQL-Datenbank herstellen

Wenn Sie mit einer PostgreSQL-Datenbank arbeiten, starten Sie das Tool *pgAdmin*. In [Abbildung 6.7](#) sehen Sie die Server in einem Navigationsbaum aufgelistet. Der Begriff »Server« ist hier ein wenig irreführend, da es sich hierbei um eine Verbindung handelt. Unterhalb des Knotens SERVERS sehen Sie derzeit ausschließlich einen Server mit der Bezeichnung **POSTGRESQL 16**.

Nach dem Start wird Ihnen die Registerkarte **DASHBOARD** angezeigt. Um einen weiteren Server anzulegen, klicken Sie auf das Symbol mit der Beschriftung **ADD NEW SERVER**.



Abbildung 6.7 Einen neuen Server (Verbindung) hinzufügen

Geben Sie im Anschluss im Feld NAME den gewünschten Verbindungsnamen (hier *Verbindung_Hans*) ein. Wie Sie in [Abbildung 6.8](#) sehen, weist der pgAdmin-Client Sie auf die weiteren notwendigen Angaben hin. Klicken Sie für den nächsten Konfigurationsschritt in der Menüleiste auf CONNECTION.

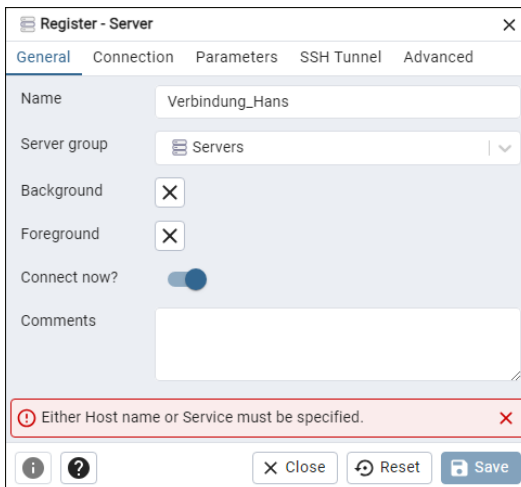


Abbildung 6.8 Eine Verbindung für die PostgreSQL-Datenbank konfigurieren

Wie Sie in [Abbildung 6.9](#) erkennen, müssen Sie für die Verbindung zu einer PostgreSQL-Datenbank mehrere Informationen angeben. Als Erstes sollten Sie den HOST eingeben. Tippen Sie hierfür *localhost* in das Namensfeld ein (anstelle des Namens können Sie auch die lokale IP-Adresse 127.0.0.1 verwenden). Als PORT behalten Sie den Standardwert 5432 bei. Unter MAINTENANCE DATABASE geben Sie den Namen der Datenbank (hier *uebungsdatenbank*) ein, mit der Sie eine Verbindung herstellen möchten. Zu guter Letzt geben Sie noch den Benutzernamen (USERNAME) und das dazugehörige Passwort (PASSWORD) an. Ich habe den Benutzernamen *hans* und das Passwort *secret* gewählt. Bestätigen Sie Ihre Eingaben mit einem Klick auf den Button SAVE.

The screenshot shows the 'Register - Server' dialog box with the 'Connection' tab selected. The following fields are visible and filled:

- Host name/address: localhost
- Port: 5432
- Maintenance database: uebungsdatenbank
- Username: hans
- Password: (masked)
- Kerberos authentication: ☐ (disabled)
- Save password?: ☐ (disabled)
- Role: (empty field)
- Service: (empty field)

At the bottom of the dialog are three buttons: 'Close', 'Reset', and 'Save'.

Abbildung 6.9 Die Verbindungsparameter

Wenn Sie den Server bzw. die Verbindung eingerichtet haben, wird Ihnen wie in [Abbildung 6.10](#) unter dem Knoten SERVERS die VERBINDUNG_HANS angezeigt.

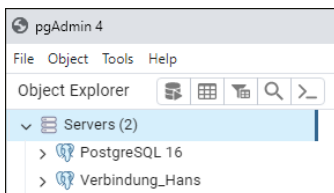


Abbildung 6.10 Die »Verbindung_Hans« zur »uebungsdatenbank« steht zur Verfügung.

Daneben befindet sich ein Pfeil, der nach rechts zeigt. Mit einem Klick auf diesen Pfeil öffnen Sie die Substruktur, die den Knoten **DATABASES** enthält. Standardmäßig wird hier die *uebungsdatenbank*, die in der Konfiguration angegeben wurde, als aktiv angezeigt. Da wir noch keine Berechtigungen für den verbundenen Benutzer *hans* vergeben haben, ist es derzeit noch nicht möglich, mit ihm Tabellenabfragen auszuführen.

6.3.4 Verbindung für eine MS SQL Server-Datenbank herstellen

Wenn Sie die MS SQL Server-Datenbank verwenden, müssen Sie keine Verbindung einrichten. Sie können sich einfach mit dem Login, den Sie in [Abschnitt 6.1.3](#) angelegt haben, an der Datenbank anmelden (siehe [Abbildung 6.11](#)). Wählen Sie hierfür als **SERVER TYPE** den Eintrag **DATABASE ENGINE** aus. Der **SERVER NAME** ist bereits standardmäßig ausgefüllt. Er setzt sich aus Ihrem Computernamen und der Serverbezeichnung zusammen. MS SQL Server bietet mehrere Möglichkeiten zur Authentifizierung an. Wählen Sie hier den Eintrag **SQL SERVER AUTHENTICATION** aus.

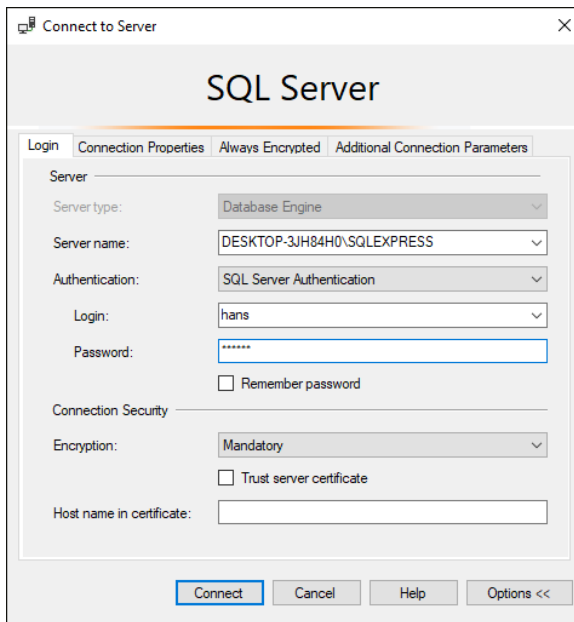


Abbildung 6.11 Anmeldedialog von MS SQL Server

Jetzt kommen wir zu der Eingabe der Nutzerdaten, die wir für den Nutzer *hans* festgelegt haben. In die beiden Texteingabefelder **LOGIN** und **PASSWORD** geben Sie den Benutzernamen »hans« und das Passwort »secret« ein, um die Verbindung zu MS SQL Server herzustellen. Beachten Sie bitte, dass es noch nicht möglich ist, Tabellen abzufragen, da noch keine Berechtigungen für den Benutzer vergeben wurden.

6.4 Berechtigungen verwalten

Wie eingangs erwähnt, ist die Berechtigungsvergabe immer mit der Frage verbunden, wer auf welche Daten Zugriff erhalten soll. So kann es in einer Unternehmensstruktur beispielsweise Tabellen geben, auf deren Daten nur die Mitarbeiter der Personalabteilung zugreifen dürfen. Daneben können natürlich auch Tabellen existieren, die lediglich für andere Abteilungen wie Controlling oder Marketing sichtbar sein dürfen. Doch neben der Frage, wer überhaupt Zugriff erhalten soll, muss auch definiert werden, welche Art von Zugriff der oder die Benutzer erhalten sollen. Darf derjenige die Informationen nur auslesen oder sie auch verändern? Darf er Datensätze anlegen und bestehende sogar löschen? Der Übersicht halber listen wir Ihnen hier noch einmal kurz die unterschiedlichen Zugriffsarten bzw. Berechtigungsarten auf, die Sie jedem Datenbanknutzer zuordnen können:

- ▶ **SELECT** erlaubt einem Nutzer, Datensätze aus einer Tabelle zu lesen.
- ▶ **INSERT** erlaubt einem Nutzer, Datensätze in eine Tabelle zu schreiben.
- ▶ **UPDATE** erlaubt einem Nutzer, Datensätze in einer Tabelle zu ändern.
- ▶ **DELETE** erlaubt einem Nutzer, Datensätze in einer Tabelle zu löschen.

In SQL weisen Sie diese Berechtigungen einem Benutzer mit dem Kommando **GRANT** zu. Dabei sind drei Informationen relevant:

- ▶ die Berechtigungsart
- ▶ die Tabelle, auf die die Berechtigung angewendet werden darf
- ▶ der Benutzer, dem der Zugriff gestattet werden soll

6.4.1 Berechtigungen vergeben (GRANT)

Zunächst melden Sie sich als Administrator (*root*, *postgres*, *sa*) oder als Benutzer mit Administratorenrechten an der Übungsdatenbank (*uebungsdatenbank*) an, um den eben erstellten Datenbankbenutzer (*hans*) mit einer Berechtigung auszustatten. Nun vergeben Sie mit einer **GRANT**-Anweisung dem Datenbanknutzer *hans* die **SELECT**- und **UPDATE**-Berechtigung für die Tabelle *mitarbeiter*:

```
GRANT SELECT,UPDATE ON mitarbeiter TO hans;
```

Listing 6.7 Zuweisen der Berechtigungen »Lesen« und »Ändern« für den Benutzer »hans«

Sehen wir uns im nächsten Schritt [Listing 6.7](#) etwas genauer an. Eingeleitet wird die Berechtigungsvergabe mit der Anweisung **GRANT**. Es folgt eine durch Kommas separierte Aufzählung der Berechtigungsarten, die Sie vergeben wollen (hier **SELECT** und

UPDATE). Dahinter steht das Schlüsselwort **ON**, auf das der Name der Tabelle folgt, auf die die Berechtigung angewendet werden darf (hier *mitarbeiter*). Zu guter Letzt müssen Sie noch festlegen, wem die Berechtigungen zugeordnet werden sollen. Hierfür notieren Sie hinter dem Tabellennamen das Schlüsselwort **TO**, gefolgt von dem Benutzernamen (hier *hans*).

Jetzt wollen wir überprüfen, ob die Berechtigungen tatsächlich vergeben worden sind. Hierfür melden Sie sich, wie in [Abschnitt 6.3](#) beschrieben, als Benutzer *hans* an der Datenbank an und testen den Lesezugriff mit einer einfachen **SELECT**-Abfrage, die alle Spalten der Tabelle *mitarbeiter* auswählt. Vergessen Sie bitte nicht, die *uebungsdatenbank* vorher auszuwählen.

```
SELECT name, vorname FROM mitarbeiter;
```

Listing 6.8 Überprüfung der Leseberechtigung für den Benutzer »hans«

Als Ergebnis liefert Ihnen die MySQL-Datenbank eine Liste aller Mitarbeiter, die in [Tabelle 6.1](#) auszugsweise dargestellt ist.

name	vorname
Klein	Thomas
Lang	Ute
Eisenhof	Frank

Tabelle 6.1 Ergebnis der Abfrage aus [Listing 6.8](#)

Die Zugriffsberechtigung auf die Tabelle *mitarbeiter* ist nun erteilt. Aber was passiert eigentlich, wenn der Benutzer *hans* auf eine Tabelle zugreifen will, für die er keine Berechtigung besitzt? Um zu prüfen, wie die Datenbank reagiert, probieren wir das einfach mal aus und formulieren, wie in [Listing 6.9](#) angegeben, eine **SELECT**-Anweisung, in der die Tabelle *umsatz* abgefragt werden soll:

```
SELECT quartal,umsatz FROM umsatz;
```

Listing 6.9 Eine Tabelle abfragen, für die keine Berechtigungen erteilt wurden

Die Datenbank teilt Ihnen sofort mit, dass ein Zugriff auf die Tabelle nicht gestattet ist bzw. abgelehnt wird:

```
Error Code: 1142. SELECT command denied to user 'hans'@'localhost' for table 'umsatz'
```

6.4.2 Berechtigungen entziehen (REVOKE)

Um einem Benutzer die erteilten Berechtigungen wieder zu entziehen, nutzen Sie die SQL-Anweisung **REVOKE**. Bevor Sie die Berechtigung entziehen können, müssen Sie sicherstellen, dass Sie als Datenbanknutzer *root* (*postgres* oder *sa*) angemeldet sind. Sehen Sie sich nun das Beispiel aus [Listing 6.10](#) an, in dem wir die im vorigen Abschnitt vergebenen Berechtigungen wieder aufheben. Hinter der **REVOKE**-Anweisung folgt die Aufzählung der Berechtigungsarten, die entzogen werden sollen (**SELECT** und **UPDATE**). Mit dem Wort **ON** geben wir wieder an, auf welche Tabelle (*mitarbeiter*) die Anweisung angewendet werden soll. Jetzt fehlt nur noch der Benutzer, dem die Berechtigung entzogen werden sollen. Hierzu notieren wir hinter dem Tabellennamen das Schlüsselwort **FROM**, gefolgt von dem Benutzernamen (*hans*):

```
REVOKE SELECT,UPDATE ON mitarbeiter FROM hans;
```

Listing 6.10 Dem Datenbankbenutzer »hans« werden die Berechtigungen »Lesen« und »Ändern« entzogen.

Wenn Sie nun die **SELECT**-Abfrage aus [Listing 6.8](#) ausführen, wird Ihnen die Datenbank mitteilen, dass der Datenbanknutzer *hans* keine Berechtigung mehr besitzt, um eine **SELECT**-Abfrage auf die Tabelle *mitarbeiter* auszuführen.

6.5 Mit Rollen Berechtigungen zuordnen

Sie haben nun gelernt, wie Sie Zugriffsberechtigungen mit einer **GRANT**-Anweisung für einen Benutzer vergeben. Allerdings wäre es doch sehr mühsam, wenn Sie für eine große Anzahl von Benutzern diese Berechtigungen alle einzeln vergeben müssten. Mit sogenannten *Rollen* können Sie Berechtigungen zentral vergeben, indem Sie einem Datenbankbenutzer eine bestimmte Rolle zuweisen. Diesen Rollen wiederum können Sie Berechtigungen auf eine oder mehrere Tabellen erteilen. Sie werden sehen, dass die Berechtigungsverwaltung hierdurch ungemein einfacher wird.

6.5.1 Rollen anlegen (CREATE ROLE)

Eine Rolle legen Sie mithilfe der **CREATE ROLE**-Anweisung an. Dahinter geben Sie eine Rollenbezeichnung an. Legen Sie nun, wie in [Listing 6.11](#) zu sehen, die Rolle *personal* an:

```
CREATE ROLE personal;
```

Listing 6.11 Eine Rolle anlegen

6.5.2 Rollen mit Berechtigungen ausstatten

Als Nächstes wollen wir der Rolle *personal* für die Tabelle *abteilung* die Berechtigungen *Lesen*, *Schreiben* und *Aktualisieren* erteilen. Die Berechtigungsvergabe für eine Rolle entspricht der für einen Benutzer. Wenn Sie [Listing 6.12](#) mit [Listing 6.7](#) vergleichen, sehen Sie, dass Sie anstelle des Benutzernamens nur den Rollennamen angeben müssen.

```
GRANT SELECT, INSERT, UPDATE
ON abteilung
TO personal;
```

Listing 6.12 Zugriffsberechtigungen für Rollen erteilen

6.5.3 Rollen Datenbanknutzern zuordnen

Jetzt müssen wir der Rolle nur noch einen Datenbanknutzer zuordnen. In einer MySQL-, MariaDB- oder PostgreSQL-Datenbank benutzen Sie hierfür ebenfalls die **GRANT**-Anweisung. Hinter der Anweisung folgt der Name der Rolle, die Sie zuweisen wollen. Im Anschluss geben Sie das Schlüsselwort **TO** und den Benutzer an, dem Sie die Rolle zuweisen möchten. Sollten Sie eine MS SQL Server-Datenbank verwenden, nutzen Sie die **ALTER ROLE**-Anweisung, gefolgt von dem Rollennamen und den Schlüsselwörtern **ADD MEMBER**, auf die der Benutzername folgt.

```
/* Rollen Nutzern zuweisen (MySQL, MariaDB, PostgreSQL)*/
GRANT personal TO hans;
/* Rollen Nutzern zuweisen (MS SQL Server)*/
ALTER ROLE personal ADD MEMBER hans;
```

Listing 6.13 Eine Datenbankrolle einem Datenbanknutzer zuordnen

Melden Sie sich nun als Benutzer *hans* an der Datenbank an, um die Zugriffsberechtigung mit einer **SELECT**-Abfrage wie in [Listing 6.14](#) zu überprüfen:

```
/*
Achtung, bei MySQL und MariaDB vor der Abfrage auszufuehren
SET ROLE 'personal';
USE uebungsdatenbank;
*/
SELECT * FROM abteilung;
```

Listing 6.14 Überprüfung der Zugriffsberechtigung für den Benutzer »hans« in der Rolle »personal«



Achtung bei einer MySQL- und MariaDB-Datenbank

Wenn Sie eine MySQL- oder MariaDB-Datenbank verwenden, muss der Benutzer vor einer Abfrage gegebenenfalls seine Rolle und die zu verwendende Datenbank nochmals explizit nennen, bevor er eine Abfrage ausführen kann:

```
SET ROLE 'personal';
USE uebungsdatenbank;
```

6.5.4 Rollen Berechtigungen entziehen

Wenn Sie einer Rolle Zugriffsberechtigungen wieder entziehen wollen, können Sie dies mit der **REVOKE**-Anweisung tun, die Sie bereits in [Abschnitt 6.4.2](#) kennengelernt haben. Wie bei der **GRANT**-Anweisung geben Sie hierbei lediglich die Rolle anstelle des Benutzernamens an.

In [Listing 6.15](#) entziehen wir der Rolle *personal* mittels der **REVOKE**-Anweisung die zuvor gesetzten Berechtigungen auf die Tabelle *abteilung*. Sie müssen als Datenbankadministrator angemeldet sein, um die **REVOKE**-Anweisung auszuführen.

```
REVOKE SELECT,INSERT,UPDATE ON abteilung FROM personal;
```

Listing 6.15 Einer Rolle Berechtigungen entziehen

Wenn Sie nach der Ausführung der **REVOKE**-Anweisung versuchen, mit dem Datenbanknutzer *hans* eine Abfrage auf die Tabelle *abteilung* durchzuführen, wird die MySQL-Datenbank die Abfrage verweigern und mit einer Fehlermeldung quittieren.

6.5.5 Rollen entfernen

Eine nicht mehr benötigte Rolle können Sie mit der **DROP ROLE**-Anweisung entfernen. Notieren Sie hierfür einfach, wie in [Listing 6.16](#) gezeigt, die Rollenbezeichnung (hier *personal*) hinter der Anweisung:

```
/*
```

Achtung: Bei einer MS SQL Server-Datenbank ist vor der Ausführung einer **DROP ROLE**-Anweisung sicherzustellen, dass die Rolle keine Mitglieder mehr hat. Mit `sp_droprolemember` wird das Mitglied "hans" aus der Rolle "personal" entfernt.
EXEC sp_droprolemember 'personal', 'hans';

```
*/
```

```
DROP ROLE personal;
```

Listing 6.16 Eine Datenbankrolle entfernen



Zusammenfassung: Berechtigungen, Benutzer und Rollen

Einem Datenbankbenutzer werden festgelegte *Berechtigungen* auf Tabellenebene (oder auf anderen Datenbankobjekten) zugeteilt. Die Berechtigungen entsprechen den SQL-Anweisungen. Folgende Berechtigungen können vergeben werden: Lesen (SELECT), Schreiben (INSERT), Ändern (UPDATE) und Löschen (DELETE).

Die Berechtigungsvergabe wird mit der **GRANT**-Anweisung durchgeführt:

```
GRANT Berechtigung1, Berechtigung2 ON Tabelle TO Benutzername;
```

Entzogen werden Berechtigungen mit der **REVOKE**-Anweisung:

```
REVOKE Berechtigung1, Berechtigung2 ON Tabelle FROM Benutzername;
```

Die meisten relationalen Datenbanksysteme nutzen *Rollen*, um mehreren Benutzern dieselben Rechte verfügbar zu machen. Die Berechtigungen werden hierbei nicht einem Datenbankbenutzer zugeordnet, sondern einer Rolle. Diese wird dann wiederum einem Datenbanknutzer zugeordnet. So erhält der Datenbankbenutzer indirekt die Berechtigungen seiner Rolle. Rollen werden nach folgendem Muster angelegt:

```
CREATE ROLE Rollenname;
```

Die Berechtigungen werden wie schon für den Benutzer mit **GRANT** erstellt bzw. mit **REVOKE** entfernt:

```
GRANT Berechtigung1, Berechtigung2 ON Tabelle TO Rollenname;
```

```
REVOKE Berechtigung1, Berechtigung2 ON Tabelle FROM Rollenname;
```

Den Benutzern wird im Anschluss eine Rolle zugeordnet:

```
GRANT Rollenname TO Benutzername; /* MySQL, PostgreSQL*/
```

```
ALTER ROLE Rollenname ADD MEMBER Benutzername; /* MS SQL Server*/
```

Entzogen werden kann die Berechtigung mit:

```
REVOKE SELECT, INSERT, UPDATE ON Tabelle FROM Rollenname;
```

Löschen lässt sich eine Rolle mit:

```
DROP ROLE Rollenname;
```

6.6 Übungen

Übung 1

Legen Sie gemäß der Datenbank, mit der Sie arbeiten, einen neuen Datenbanknutzer *thomas* an. Diesem Nutzer ordnen Sie das Passwort *secret* zu. Erstellen Sie, falls es die Datenbank erfordert, eine neue Verbindung für den Benutzer *thomas*.

Übung 2

Ordnen Sie dem in Übung 1 erstellten neuen Benutzer *thomas* mit der **GRANT**-Anweisung Berechtigungen zum Lesen, Ändern und Schreiben für die Tabelle *artikel* zu. Prüfen Sie im Anschluss an die Berechtigungsvergabe mit einer einfachen **SELECT**-Abfrage, ob die Zuordnung erfolgreich war. Bedenken Sie dabei, dass Sie sich als Datenbankbenutzer *thomas* anmelden müssen.

Übung 3

Entfernen Sie sämtliche Berechtigungen, die Sie dem neuen Benutzer *thomas* in Übung 2 zugeordnet haben, mit der **REVOKE**-Anweisung. Beachten Sie, dass Sie hierfür als Administrator angemeldet sein müssen.

Übung 4

Der Datenbankbenutzer *thomas* hat seine Schuldigkeit getan: Löschen Sie ihn aus der Benutzerverwaltung. Verwenden Sie hierzu die **DROP USER**-Anweisung. Sollte dem Benutzer noch ein Login (MS SQL Server) zugeordnet sein, müssen Sie es vorab löschen.

Übung 5

Legen Sie eine Benutzerrolle *vertrieb* an. Dieser Rolle ordnen Sie die Berechtigungen *Lesen*, *Schreiben* und *Aktualisieren* für die Tabelle *artikel* zu.

Legen Sie im Anschluss als Administrator zwei neue Datenbankbenutzer an: *frank* und *ralf*. Ordnen Sie sie nun der Rolle *vertrieb* zu. Testen Sie, ob der Zugriff auf die Tabelle *artikel* für die beiden neuen Mitarbeiter gegeben ist, indem Sie sich als Datenbanknutzer *frank* oder *ralf* an der Datenbank anmelden.

Übung 6

Entfernen Sie die **INSERT**- und **UPDATE**-Berechtigungen der Rolle *vertrieb*, die der Tabelle *artikel* zugeordnet sind, um zu gewährleisten, dass den Mitgliedern der Rolle *vertrieb* nur ein lesender Zugriff auf die Tabelle *artikel* gestattet ist.

Übung 7

Die Rolle *vertrieb* soll nicht mehr genutzt werden können; entfernen Sie sie.

Lösung zu Übung 1

```
/* MySQL- und MariaDB-Datenbank */  
CREATE USER thomas IDENTIFIED BY 'secret';
```

```

/* PostgreSQL-Datenbank */
CREATE USER thomas PASSWORD 'secret';

/* MS SQL Server-Datenbank */
CREATE LOGIN thomas WITH PASSWORD = 'secret';
CREATE USER thomas FOR LOGIN thomas;

```

Listing 6.17 Datenbankbenutzer »thomas« anlegen

Lösung zu Übung 2

```

GRANT SELECT, INSERT, UPDATE ON artikel TO thomas;

SELECT * FROM artikel;

```

Listing 6.18 Dem Benutzer »thomas« Berechtigungen zuordnen und die Berechtigungsvergabe mit einer Abfrage testen

Lösung zu Übung 3

```

REVOKE SELECT, INSERT, UPDATE ON artikel FROM thomas;

```

Listing 6.19 Dem Benutzer »thomas« Berechtigungen entziehen

Lösung zu Übung 4

```

/* MySQL-, MariaDB- und PostgreSQL-Datenbank */
DROP USER thomas;

/* MS SQL Server-Datenbank */
DROP LOGIN thomas;

DROP USER thomas;

```

Listing 6.20 Benutzer aus einer Datenbank entfernen

Lösung zu Übung 5

```

/* Rolle anlegen */
CREATE ROLE vertrieb;

/* Der Rolle Berechtigungen auf die Tabelle artikel geben*/
GRANT SELECT, INSERT, UPDATE ON artikel TO vertrieb;

/* Datenbanknutzer anlegen (MySQL, MariaDB)*/
CREATE USER frank IDENTIFIED BY 'secret';

```

```
CREATE USER ralf IDENTIFIED BY 'secret';

/* Datenbanknutzer anlegen (PostgreSQL)*/
CREATE USER frank PASSWORD 'secret';
CREATE USER ralf PASSWORD 'secret';

/* Datenbanknutzer anlegen (MS SQL Server)*/
CREATE LOGIN frank WITH PASSWORD = 'secret';
CREATE USER frank FOR LOGIN frank;

CREATE LOGIN ralf WITH PASSWORD = 'secret';
CREATE USER ralf FOR LOGIN ralf;
/* Die Rolle Vertrieb den neuen Nutzern zuordnen */

/* MySQL-,MariaDB- und PostgreSQL-Datenbank */
GRANT vertrieb TO ralf;
GRANT vertrieb TO frank;

/* MS SQL Server-Datenbank */
ALTER ROLE vertrieb ADD MEMBER frank;
ALTER ROLE vertrieb ADD MEMBER ralf;

/*
Wenn Sie eine MySQL- oder MariaDB-Datenbank verwenden, führen Sie zunächst
die Anweisung
SET ROLE vertrieb;
aus.
Als Nächstes müssen Sie für die Sitzung die zu verwendende Datenbank angeben.
USE uebungsdatenbank;
*/

SELECT * FROM artikel;
```

Listing 6.21 Rollen anlegen, Rechte zuweisen und Rollen Benutzern zuordnen

Lösung zu Übung 6

```
REVOKE INSERT,UPDATE ON artikel FROM vertrieb;
```

Listing 6.22 Die INSERT- und UPDATE-Berechtigung der Rolle »vertrieb« auf die Tabelle »artikel« entfernen

Lösung zu Übung 7

```
/*
```

Achtung bei PostgreSQL

Sie müssen zunächst auch die SELECT-Berechtigung auf die Tabelle "artikel" entfernen, um die Rolle "vertrieb" löschen zu können.

```
REVOKE SELECT ON artikel FROM vertrieb;
```

Achtung bei MS SQL Server

Wenn Sie beabsichtigen, eine Rolle zu entfernen, müssen Sie zunächst sämtliche der Rolle zugeordnete Mitglieder aus der Rolle löschen.

```
EXEC sp_droprolemember 'vertrieb', 'frank';
```

```
EXEC sp_droprolemember 'vertrieb', 'ralf';
```

```
*/
```

```
DROP ROLE vertrieb;
```

Listing 6.23 Die Rolle »vertrieb« entfernen

Kapitel 7

Datenbanken modellieren

Datenbanken und ihre Struktur existieren nicht per se. Auch sie haben ihren Ursprung. Um ein Modell zu entwickeln, muss zuvor klar sein, für welchen Zweck es entwickelt wird und welche Aspekte der Realität abgebildet werden sollen.

Die Entwicklung einer Datenbank, die Informationen zu einem bestimmten Thema speichert, erfordert einiges an gedanklicher Vorarbeit, bevor Sie damit beginnen können, Tabellen zu erstellen.

Gehen wir von folgender Situation aus, der Sie in ähnlicher Form durchaus begegnen könnten: In einem Unternehmen soll der Ausbildungsbetrieb datentechnisch in einer Ausbildungsdatenbank verwaltet werden. Bis auf die Tatsache, dass diese entwickelt werden soll und auch den Namen *Ausbildungsdatenbank* tragen könnte, sind noch keine Informationen bekannt. Wir müssen uns also zunächst damit befassen, wie die Daten aufgebaut sind, welche Anforderungen an die Ordnung gestellt werden und wie wir alles in einem Modell abbilden können.

Die Ausbildungsdatenbank soll *Ausbildungsfächer*, *Auszubildende*, *Ausbilder* und einiges mehr verwalten. Wir haben es hier also mit real existierenden Dingen zu tun, die wir datentechnisch abbilden müssen. Aus diesem Grund betrachten wir im nächsten Abschnitt zunächst einen kleinen Anforderungskatalog.

7.1 Anforderungskatalog

Die Ausbildungsdatenbank soll unterschiedliche Informationen verwalten. Hierzu zählen:

- ▶ Ausbildungsberufe
- ▶ Auszubildende
- ▶ Adresse des Auszubildenden
- ▶ Ausbilder
- ▶ Ausbildungsfächer
- ▶ Ausbildungsstandorte
- ▶ Praxisseminare
- ▶ Vermittlung theoretischer Ausbildungsinhalte

Hierbei handelt es sich nur um eine kleine Auswahl von Dingen und Personen in unterschiedlichen Rollen, die wir nutzen werden, um unseren ersten Schritt in der Modellierung der Ausbildungsdatenbank zu machen. Wichtig ist, dass den Dingen oder Personen, die verwaltet werden sollen, wiederum Informationen zugeordnet werden können, die spezifisch für die jeweiligen Dinge bzw. Personen sind.



Zusammenfassung: Anforderungskatalog als Basis eines Datenmodells

Ein Anforderungskatalog stellt die Grundlage für ein Datenmodell dar. Es muss zuerst klar sein, welche Sachverhalte datentechnisch abgebildet werden sollen, bevor ein Datenmodell erstellt werden kann. Ein Anforderungskatalog resultiert wiederum aus einer Anforderungsanalyse.

7.2 Entitäten identifizieren und modellhaft abbilden

Wenn Sie eine neue Datenbank entwickeln, ist es sinnvoll, zuerst ein Modell zu erstellen, ein sogenanntes *Entity-Relationship-Modell*. Es ist ein Werkzeug, das uns bei der Erstellung eines Datenmodells unterstützt. Diese Form eines Modells, das uns ermöglicht, Informationsstrukturen modellhaft abzubilden, wurde Mitte der 1970er-Jahre von Peter Chan vorgestellt.

Wir haben es mit real existierenden Dingen bzw. Objekten (*Entitäten*) zu tun, die in Beziehung (*Relationship*) zueinander stehen. Nehmen wir beispielsweise einen Auszubildenden. Sie können ihn exakt einem Ausbildungsberuf zuordnen. Einem Ausbildungsberuf wiederum können Sie mehrere Ausbildungsfächer zuweisen. Sie merken schon: Mit einem Entity-Relationship-Modell können Sie nicht nur Objekte darstellen, die Sie datentechnisch mit den jeweiligen Eigenschaften eines Objekts abbilden wollen, sondern auch modellhaft Beziehungen abbilden, die zwischen diesen Objekten existieren.

Ein Modell ist immer das Ergebnis eines Entwurfs und bildet ab, was in der Realität umgesetzt werden soll. Ein Entwurf wiederum entwickelt sich aus einer Phase, in der wir die Dinge oder Objekte, die es datentechnisch zu verwalten gilt, identifizieren und analysieren. Die Frage lautet also zunächst: Wie lassen sich Entitäten identifizieren?

7.2.1 Entitäten identifizieren

Aus den Anforderungen, die an die Ausbildungsdatenbank gestellt wurden, können die Entitäten identifiziert werden. In unserem Beispiel werden wir es also mit den Entitäten *auszubildender*, *ausbildungsberuf* usw. zu tun haben.

Zunächst müssen die Objekte, die in der Datenbank abzubilden sind, in ein Modell überführt werden. Dazu nutzen wir die *Entity-Relationship-Notation*. Gemäß dieser

Schreibweise werden Objekte als Rechteck notiert. In dem Rechteck wird die Bezeichnung oder der Name des abzubildenden Objekts notiert. Somit erhalten wir schon einmal eine Übersicht über die Entitäten, deren Informationen verarbeitet werden sollen. Als Beispiel sehen Sie in [Abbildung 7.1](#) die modellhafte Darstellung von drei Entitäten, die aus den Anforderungen übernommen wurden. Es handelt sich um die Entitäten *auszubildender*, *adresse* und *ausbildungsberuf*.

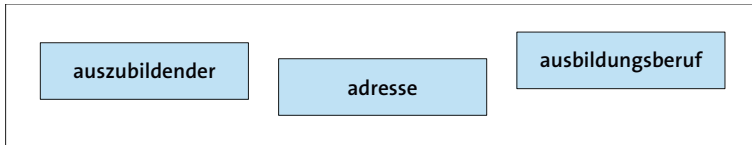


Abbildung 7.1 Auszug aus einem Entity-Relationship-Modell

7.2.2 Informationen zu den Entitäten ermitteln

Die Entität *auszubildender* soll die Informationen bzw. Eigenschaften enthalten, die zur Verwaltung eines Auszubildenden erforderlich sind. Überlegen wir uns also als Erstes, um welche Informationen es sich handeln könnte. Zunächst fallen Ihnen sicherlich der Name, der Vorname und das Geburtsdatum eines Auszubildenden ein. Das ist schon einmal ein guter Anfang, um ein Entity-Relationship-Modell zu erweitern. Diese zusätzlichen Eigenschaften, die zur weiteren Beschreibung eines Objekts erforderlich sind, werden in einem Entity-Relationship-Modell als *Attribute* bezeichnet. Attribute notieren Sie in Ellipsen unterhalb der Entitätsbezeichnung (siehe [Abbildung 7.2](#)). Von der Entitätsbezeichnung ausgehend, zeigen Pfeile auf die jeweils zu einer Entität gehörenden Attribute.

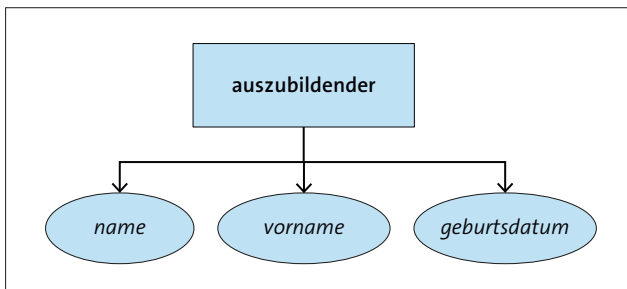


Abbildung 7.2 Einer Entität Attribute zuordnen

7.2.3 Schlüsselattribute für Entitäten identifizieren

Bevor wir damit beginnen, Schlüsselattribute für Entitäten zu identifizieren, stellen wir uns zunächst die Frage, welche Funktion Schlüsselattribute erfüllen. Die Entität *auszubildender* enthält derzeit die Attribute *name*, *vorname* und *geburtsdatum*.

Jetzt betrachten wir das Modell in einer vereinfachten Form, in der der Entität *auszubildender* ausschließlich die Attribute *name* und *vorname* zugeordnet sind (siehe Tabelle 7.1).

name	vorname
Müller	Ralf
Klein	Sabine
Lang	Peter
Müller	Ralf
Klein	Sabine

Tabelle 7.1 Auszubildende mit den Eigenschaften »name« und »vorname«

Sie sehen, dass die Tabelle zwei Auszubildende enthält, denen jeweils der gleiche Vorname und Name zugeordnet sind. In diesem Fall handelt es sich um die Auszubildenden Ralf Müller und Sabine Klein. In unserem Ausbildungsbetrieb existieren also jeweils zwei Auszubildende mit gleichen Namen und Vornamen. Wie können wir diese Auszubildenden unterscheiden?

Die Antwort lautet: gar nicht. Sie haben keine Möglichkeit, die Auszubildenden zu unterscheiden.

Die Lösung unseres Problems der Nichtunterscheidbarkeit besteht darin, ein oder mehrere Attribute zu identifizieren, die einen Auszubildenden eindeutig machen. Attribute, die Datensätze eindeutig bestimmen, werden als *Schlüsselattribute* bezeichnet.

Unser ursprüngliches Modell enthält ja zum Glück noch ein weiteres Attribut: das Geburtsdatum. In Tabelle 7.2 sehen Sie eine weitere Spalte, die dem Attribut *geburtsdatum* aus unserer Entität *auszubildender* entstammt. Mit den hier hinterlegten Informationen haben wir jetzt die Möglichkeit, jede Zeile exakt einem Auszubildenden zuzuordnen.

name	vorname	geburtsdatum
Müller	Ralf	01.04.2001
Klein	Sabine	10.05.2002
Lang	Peter	11.03.2001

Tabelle 7.2 Auszubildende mit den zugeordneten Eigenschaften »name«, »vorname« und »geburtsdatum«

name	vorname	geburtsdatum
Müller	Ralf	01.04.2001
Klein	Sabine	20.05.2003

Tabelle 7.2 Auszubildende mit den zugeordneten Eigenschaften »name«, »vorname« und »geburtsdatum« (Forts.)

Können wir wirklich jede Zeile exakt einem Auszubildenden zuordnen? Ein weiterer Blick in [Tabelle 7.2](#) offenbart uns, dass wieder zwei identische Zeilen existieren. Hier werden jeweils für die beiden Auszubildenden Ralf Müller, die zufällig am gleichen Tag Geburtstag haben, die gleichen Werte in den Spalten notiert. Die Kombination aus Name, Vorname und Geburtsdatum stellt für uns also immer noch kein geeignetes Kriterium dar, das einen Auszubildenden und die ihm zugeordneten Eigenschaften eindeutig identifizieren könnte, zumal in diesem Fall alle verfügbaren Eigenschaften der Entität *auszubildender* genutzt würden, um einen eindeutigen Schlüssel zu bilden, der einen Auszubildenden exakt identifiziert.

Machen wir uns also auf die Suche nach einem weiteren Schlüsselkandidaten, den wir verwenden können, um einen Auszubildenden eindeutig zu identifizieren.

Als *Schlüsselkandidaten* bezeichnen wir hier einen gegebenenfalls infrage kommenden eindeutigen Schlüssel. Wir haben erkannt, dass sich keines der Attribute, die wir der Entität *auszubildender* zugeordnet haben, als Schlüsselattribut eignet.

Was halten Sie davon, die Auszubildenden jeweils mit einer Nummer zu versehen, die sie eindeutig identifiziert? Wir könnten dazu der Entität *auszubildender* ein weiteres Attribut *ausid* zuordnen. In [Abbildung 7.3](#) sehen Sie das erweiterte Entity-Relationship-Modell, in dem das Attribut *ausid* als *Schlüsselattribut* hinzugefügt wurde. Das Schlüsselattribut wurde hier durch eine Raute (#) gekennzeichnet.

Ein derartiges Schlüsselattribut wird auch als *künstlicher Schlüssel* oder *Surrogatschlüssel* bezeichnet. Ein künstlicher Schlüssel hat die Eigenart, dass er nicht aus den Attributen abgeleitet werden kann, die einer Entität zugeordnet sind.

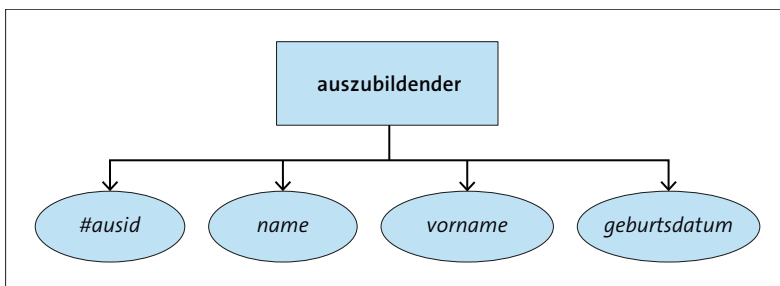


Abbildung 7.3 Eine Entität mit einem eindeutigen Schlüssel versehen

Ein *natürlicher Schlüssel* hingegen wird aus den Attributen abgeleitet, die wir einer Entität zuordnen können. Wenn wir uns also für einen Schlüssel entschieden hätten, der aus den Attributen *name*, *vorname* und *geburtsdatum* besteht, so würde es sich um einen natürlichen Schlüssel handeln. Dieses Beispiel zeigt Ihnen, wo der Unterschied zwischen einem künstlichen Schlüssel und einem natürlichen Schlüssel liegt. Diese Attributkombination ist nicht für einen eindeutigen Schlüssel geeignet.

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001
2	Klein	Sabine	10.05.2002
3	Lang	Peter	11.03.2001
4	Müller	Ralf	01.04.2001
5	Klein	Sabine	20.05.2003

Tabelle 7.3 Die Tabelle »auszubildender« mit eindeutigem Schlüssel

7.2.4 Die Wertebereiche von Attributen erkennen

In Kapitel 4, »Tabellen anlegen (CREATE TABLE)«, haben Sie bereits Datentypen und Wertebereiche kennengelernt. In diesem Abschnitt betrachten wir den Wertebereich, der auch als *Domain* bezeichnet wird, im Zusammenhang mit der Zuordnung von Attributen zu Entitäten. Betrachten Sie zunächst wieder die Entität *auszubildender*. Dieser Entität haben wir die Attribute *ausid*, *name*, *vorname* und *geburtsdatum* zugeordnet. Bei der Planung Ihrer Datenbank ist es wichtig, dass Sie den Attributen passende Wertebereiche zuordnen.

Dem Attribut *ausid* können ganzzahlige numerische Werte in einem Wertebereich von 0 bis 65.535 zugewiesen werden. Mit dem Wertebereich gehen wir auf Nummer sicher, um unsere Ausbildungsdatenbank langfristig in Betrieb zu nehmen.

Die Attribute *name* und *vorname*, die jeweils eine Folge von Zeichen wie z. B. Diether aufnehmen sollen, werden als *Zeichenketten* bezeichnet, also eine Aneinanderkettung von einzelnen Zeichen. Hier müssen Sie sich fragen, wie viele Zeichen für eine Zeichenkette dieser Attribute reserviert werden sollen. Beispielsweise können für die Attribute *Name* und *Vorname* 120 Zeichen freigehalten werden. So wird die maximale Länge dieses Attributs festgelegt.

Im Attribut *geburtsdatum* finden sämtliche Datumsangaben vom 01.01.1900 bis 31.12.2050 Platz. Somit hätten wir auch hier für das Attribut *geburtsdatum* einen Wertebereich bestimmt, der alle potenziellen Datumswerte repräsentiert.

Betrachten Sie als Nächstes [Tabelle 7.4](#), in der die Attribute, die Wertebereiche und der Datentyp der Entität *auszubildender* aufgeführt sind.

Attribut	Mögliche Werte	Datentyp
<i>ausid</i>	0 bis 65535	SMALLINT
<i>name</i>	0 bis 120 Zeichen	CHAR(120)
<i>vorname</i>	0 bis 120 Zeichen	CHAR(120)
<i>geburtsdatum</i>	01.01.1900 bis 31.12.2050	DATE

Tabelle 7.4 Wertebereiche für Attribute von Entitäten auswählen

Die Datentypen haben Sie bereits in [Abschnitt 4.1](#) kennengelernt. Sie wissen daher, dass die verschiedenen Datentypen unterschiedlich viel Platz verbrauchen. Sie sollten immer Datentypen wählen, die möglichst gut zum reservierten Wertebereich passen. Für *ausid* sind folgende Möglichkeiten sinnvoll:

- ▶ TINYINT – Wertebereich von 0 bis 255
- ▶ SMALLINT – Wertebereich von 0 bis 65.535
- ▶ MEDIUMINT – Wertebereich von 0 bis 16.777.215
- ▶ INT bzw. INTEGER – Wertebereich von 0 bis 4.294.967.295
- ▶ BIGINT – Wertebereich 0 bis 18.446.744.073.709.551.615

Wir gehen in unserem Beispiel von einer langfristigen Planung aus, somit ist der Datentyp **SMALLINT** für das Primärschlüsselattribut *ausid* mehr als ausreichend. **MEDIUMINT** und alle weiteren Alternativen wären hingegen viel zu groß und würden daher Speicherplatz verschwenden. **TINYINT** hingegen würde den Betrieb unserer Ausbildungsdatenbank einschränken.

Den Attributen *name* und *vorname* wird jeweils ein Wertebereich zugeordnet, der aus einer Folge von einzelnen Zeichen bestehen kann, mit einer maximalen Länge von 120 Zeichen. Dem Attribut *name* ordnen wir dementsprechend den Datentyp **CHAR** zu. Dem Attribut *geburtsdatum* wird der Datentyp **DATE** zugeordnet.

Zusammenfassung: Wertebereiche für Attribute

Ein Wertebereich (auch *Domain* genannt) entspricht allen Werten, die einem Attribut zugeordnet werden können. Es gibt unterschiedliche Wertebereiche, die jeweils einem Attribut einer Entität zugeordnet werden können. Hierzu gehören:

- ▶ Zeichenketten einer definierten Länge
- ▶ ganze Zahlen, die mit einer Begrenzung der kleinsten und größten Zahl ausgestattet sind



- ▶ Dezimalzahlen, die ebenfalls mit einer Begrenzung der kleinsten und größten Zahl ausgestattet sind
- ▶ Datumsangaben, die von einem verwendeten Kalender abhängig sind

7.2.5 Zwischen Pflichtattributen und optionalen Attributen unterscheiden

Sie wissen bereits, dass Sie einer Entität ein Attribut zuordnen können, das einen eindeutigen Schlüssel darstellt. Es handelt sich hierbei um ein Primärschlüsselattribut. Wenn eine Entität mit einem Primärschlüsselattribut in eine Tabelle überführt wird, ist ein Primärschlüsselwert verbindlich anzugeben.

Wie sieht es mit den anderen Attributen und ihren Werten aus, wenn eine Entität in eine Tabelle überführt wurde? Wann sind die Attributwerte verbindlich anzugeben, und wann sind sie optional? Die gute Nachricht ist, dass Sie bereits während der Modellierung einer Entität die Möglichkeit haben, Attribute als Pflichtattribute oder als optionale Attribute zu definieren. Betrachten Sie hierzu [Abbildung 7.4](#).

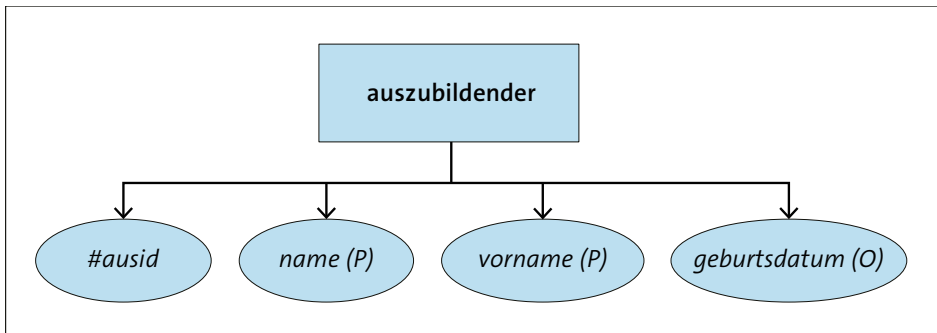


Abbildung 7.4 Pflichtattribute und optionale Attribute definieren

Der Entität *auszubildender* sind die Attribute *name*, *vorname* und *geburtsdatum* zugeordnet. In der Modellierungsphase liegt es nun an Ihnen, zu bestimmen, welche Attribute als Pflichtattribute und welche Attribute als optionale Attribute definiert werden. In diesem Beispiel habe ich die Attribute *name* und *vorname* mit einem »P« gekennzeichnet und somit festgelegt, dass es sich um Pflichtattribute handelt. Das Attribut *geburtsdatum* hingegen habe ich mit einem »O« gekennzeichnet, um es als ein optionales Attribut anzugeben.

Wie Sie Pflichtfelder mit einer **CREATE TABLE**-Anweisung angeben, haben Sie bereits in [Kapitel 4](#), »[Tabellen anlegen \(CREATE TABLE\)](#)«, erfahren: Hierzu ordnen Sie einer Spalte hinter der Datentypangabe einfach einen **NOT NULL**-Constraint zu.



Zusammenfassung: Entitäten modellieren

Um eine neue Datenbank zu modellieren, müssen Sie als Erstes die »Dinge« bzw. Objekte ermitteln, deren Informationen in der Datenbank gespeichert werden sollen. Hierbei könnte es sich ganz allgemein um eine Person, einen Arbeitnehmer oder einen Anstellungsvertrag handeln. Ein Objekt oder ein real existierendes »Ding« wird in der Datenbankmodellierung als *Entität* bezeichnet. Eine Entität wird im *Entity-Relationship-Diagramm* als Rechteck notiert.

Einer Entität werden Informationen oder, anders gesagt, Eigenschaften zugeordnet, die im unmittelbaren Bezug zu dem zu modellierenden Objekt stehen. Diese Eigenschaften werden in der Modellierungsphase als *Attribute* bezeichnet. Attribute, die einer Entität zuzuordnen sind, werden als Oval notiert und mit einer Verbindungslinie zu einer Entität versehen.

Die für eine Entität festgelegte Menge von Attributen muss mit einem *Schlüsselattribut* versehen werden. Während der Modellierung werden hier zunächst *Schlüsselkandidaten* untersucht, die aus den Elementen der Menge der Attribute hervorgehen. Wenn ein oder mehrere Attribute als Schlüsselkandidat infrage kommen, so spricht man von einem *natürlichen Schlüssel*. Kann hingegen aus der Menge der zur Verfügung stehenden Attribute kein natürlicher Schlüssel abgeleitet werden, so kommt ein *künstlicher Schlüssel* infrage. So ein künstlicher Schlüssel wird als *Surrogatschlüssel* bezeichnet und kann z. B. durch ganzzahlige Werte abgebildet werden. Ein *Schlüsselattribut* ist im Modell zu kennzeichnen, um es von den anderen Attributen unterscheidbar zu machen. In diesem Buch verwende ich das Zeichen #, um ein Schlüsselattribut kenntlich zu machen.

Unabhängig davon, ob es sich um einfache Attribute oder Schlüsselattribute handelt, muss jedem Attribut ein geeigneter *Wertebereich* (ein Datentyp) zugeordnet werden. In der Modellierung unterscheiden sich *optionale Attribute* von Attributen, die verbindlich anzugeben sind. *Pflichtattribute* sind ebenfalls im Entity-Relationship-Diagramm zu kennzeichnen, um sie unterscheidbar zu machen. Hier habe ich für sie den Buchstaben »P« ergänzt.

7.3 Beziehungen zwischen Entitäten festlegen

Sie haben bisher drei Entitäten festgelegt, die derzeit noch in keinerlei Bezug zueinander stehen. Sie könnten z. B. einem Auszubildenden weder einen Ausbildungsberuf noch eine Adresse zuordnen. Um das zu ändern, müssen wir die Entitäten in Beziehung zueinander setzen. Um ein vollständiges Beispiel darstellen zu können, fügen wir dem Entity-Relationship-Modell wie in Abbildung 7.5 die Entitäten *lehrfach* und *ausbildungsvertrag* hinzu.

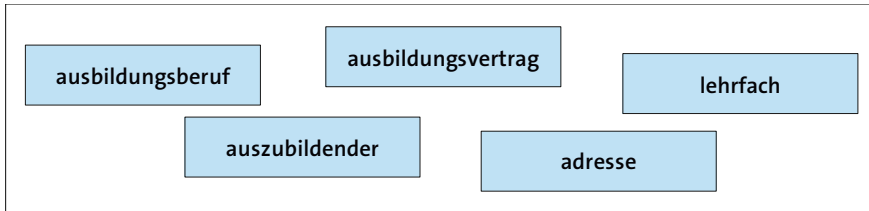


Abbildung 7.5 Entitäten, die nicht in Beziehung zueinander stehen

7.3.1 Beziehungen im Entity-Relationship-Modell definieren

Wir müssen uns natürlich zuerst überlegen, welche Entitäten in Beziehung zueinander stehen. Betrachten wir zunächst die Entität *auszubildender*, und überlegen wir uns, in welchen Beziehungen diese Entität stehen könnte.

Einem *auszubildenden* kann ein *ausbildungsvertrag* zugeordnet werden. Somit haben wir schon mal eine erste Beziehung erkannt. Des Weiteren können Sie einem *auszubildenden* eine *adresse* zuordnen. Somit hätten wir schon eine zweite Beziehung erkannt. Einem *auszubildenden* *lehrfächer* zuzuordnen, wäre hingegen nicht sinnvoll, da *lehrfächer* *ausbildungsberufen* zugeordnet werden. Stattdessen ordnen wir also einem *ausbildungsberuf* *lehrfächer* zu. Somit haben wir eine dritte Beziehung zwischen den Entitäten *ausbildungsberuf* und *lehrfach* erkannt.

Einem *ausbildungsvertrag*, einem *ausbildungsberuf* oder einem *lehrfach* eine *adresse* zuzuordnen, wäre wiederum nicht sinnvoll. Hier existieren also keine Beziehungen.

In [Abbildung 7.6](#) sehen Sie, wie Sie in einem Entity-Relationship-Modell die bereits ermittelten Beziehungen zwischen Entitäten darstellen können. Wenn Entitäten in Beziehung zueinander stehen, werden sie durch eine einfache Linie miteinander verbunden.

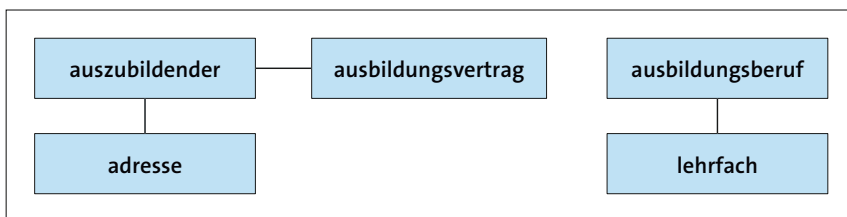


Abbildung 7.6 Beziehungen zwischen Entitäten herstellen

Vielleicht denken Sie jetzt, wir könnten doch einen Ausbildungsvertrag auch in Beziehung zu einem Ausbildungsberuf sehen. Stimmt! Ich bevorzuge an dieser Stelle allerdings ein einfaches Modell, um Ihnen die Beziehungen zwischen Entitäten zu erläutern. Machen wir es nicht zu kompliziert.

7.3.2 Kardinalitäten von Beziehungen erkennen

In Kapitel 1, »Grundlagen kennenlernen und verstehen«, haben Sie die Bundesliga-tabelle als Beispiel für eine einfache Tabelle kennengelernt. Zugegeben: Diese Tabelle hat nichts mit einer Ausbildungsdatenbank gemeinsam. Ein Torverhältnis in einem Bundesligaspiel eignet sich aber, um Ihnen den Begriff der *Kardinalität* zu erklären.

Stellen Sie sich vor, in einem Spiel treffen die Rivalen Bayern München und Borussia Dortmund aufeinander. Dortmund gewinnt das Spiel 5:0. Mit diesem Verhältnis zeigen Sie, dass Dortmund 5 Tore erzielt hat. Bayern geht in diesem hypothetischen Fall mit 0 Toren vom Platz. Das Endergebnis würden wir in der Form »Dortmund:Bayern 5:0« notieren.

Das Verhältnis bzw. die Kardinalität von Entitäten, die in Beziehung zueinander stehen, ist nicht immer so eindeutig zu sehen. Um ein solches Verhältnis zu identifizieren, sollten Sie sich folgende Frage stellen: »Wie viele Entitäten eines Typs kann ich einer Entität eines anderen Typs zuordnen?«

Frage zur Ermittlung von Kardinalitäten für eine Beziehung

Um eine Kardinalität einer Beziehung zwischen Entitäten zu ermitteln, sollten Sie sich für jede Beziehung zweimal fragen: »Wie viele Entitäten eines Typs kann ich einer Entität eines anderen Typs zuordnen?«

In unserem Beispiel lauten die Fragen also:

- ▶ Wie viele Ausbildungsverträge können einem oder mehreren Auszubildenden zugeordnet werden?
- ▶ Wie viele Auszubildende können einem oder mehreren Ausbildungsverträgen zugeordnet werden?

Beantwortet werden sie wie folgt:

- ▶ Jedem Auszubildenden kann nur exakt ein Ausbildungsvertrag zugeordnet werden.
- ▶ Jedem Ausbildungsvertrag kann nur exakt ein Auszubildender zugeordnet werden.

Eine 1:1-Beziehung erkennen

Im Entity-Relationship-Modell unserer zu modellierenden Ausbildungsdatenbank finden sich ebenfalls Entitäten und deren Beziehungen, die quantitativ ermittelt werden können.

Um Ihnen die *1:1-Beziehung* nahezubringen, betrachten wir das Beispiel der Azubis und ihrer Verträge. Einem Auszubildenden wird dabei immer exakt ein Ausbildungsvertrag mit einer Vertragsnummer zugeordnet, und ein Vertrag gilt genauso selbst-

verständlich nur für exakt einen Azubi. Hierbei handelt es sich also um eine klassische 1:1-Beziehung.

Eine 1:1-Beziehung wird im Entity-Relationship-Modell durch vertikale Striche am Anfang und am Ende einer Beziehungslinie dargestellt. Mit dieser Notation kennzeichnen Sie, dass einem Auszubildenden exakt ein Ausbildungsvertrag zugeordnet werden kann. Das gilt in diesem Fall auch umgekehrt, da einem Ausbildungsvertrag nur ein Auszubildender zugeordnet ist.

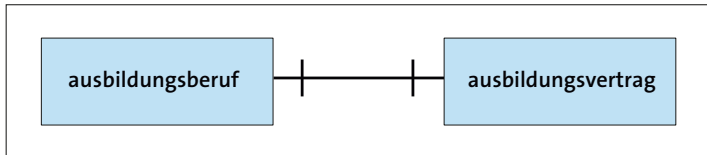


Abbildung 7.7 Die Entitäten »auszubildender« und »ausbildungsvertrag« stehen in einer 1:1-Beziehung.

Betrachten Sie die Attribute der Entität *ausbildungsvertrag* in [Abbildung 7.8](#) genauer. Hier finden Sie ein Attribut mit der Bezeichnung *ausid*. Dieses Attribut dient als Verweis auf den Primärschlüssel *ausid* der Entität *auszubildender*. Das Attribut *ausid* der Entität *ausbildungsvertrag* können wir also als einen fremden Schlüssel bezeichnen, der auf ein Primärschlüsselattribut in einer anderen Entität zeigt. Aus diesem Grund wird das Attribut *ausid* der Entität *ausbildungsvertrag* als *Fremdschlüssel* oder *Foreign Key* bezeichnet.

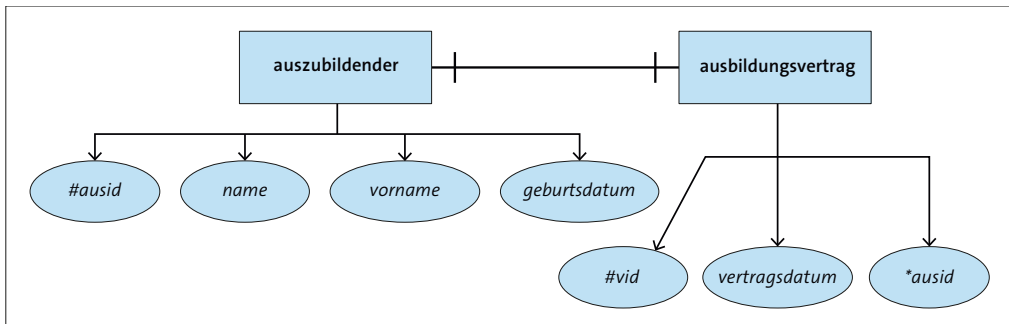


Abbildung 7.8 Ein vollständiges Entitätenmodell für die Beziehung der Entitäten »auszubildender« und »ausbildungsvertrag«

Als Nächstes sehen wir uns die Tabellen an, die aus dem Entitätenmodell der 1:1-Beziehung resultieren. Zunächst betrachten wir [Tabelle 7.5, auszubildende](#). Hier sehen Sie nichts Neues: Die Tabelle verfügt über die Spalten *name*, *vorname* und *geburtsdatum*, die jeweils abhängig von der Primärschlüsselspalte *ausid* sind.

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001
2	Klein	Sabine	10.05.2002
3	Lang	Peter	11.03.2001
4	Müller	Ralf	01.04.2001
5	Klein	Sabine	20.05.2003

Tabelle 7.5 Die Tabelle »auszubildende«

Tabelle 7.6, *ausbildungsverträge*, steht zur Tabelle *auszubildende* in einer 1:1-Beziehung. In dieser Tabelle wird in jeder Zeile der Fremdschlüsselwert *ausid* gespeichert, der dem Primärschlüsselwert *ausid* der Tabelle *auszubildende* entspricht. Somit ist sichergestellt, dass jede Zeile der Tabelle *ausbildungsvertrag* auf eine Zeile in der Tabelle *auszubildende* verweist und somit eine 1:1-Beziehung gegeben ist.

vid	vertragsdatum	ausid
1	01.03.2015	3
2	01.03.2015	5
3	01.03.2015	1
4	01.03.2015	2
5	01.03.2015	4

Tabelle 7.6 Die Tabelle »ausbildungsvertrag«

Eine 1:n-Beziehung erkennen

Die *1:n-Beziehung* erkläre ich Ihnen anhand eines einfachen Beispiels, in dem wir einen Auszubildenden in ein Verhältnis zu einer oder mehreren Adressen setzen. Es kann ja durchaus vorkommen, dass ein Azubi das Elternhaus verlässt und in einer anderen Stadt eine Ausbildung beginnt. Dies ist ein klassisches Beispiel für eine 1:n-Beziehung. Das Verhältnis besteht in diesem Fall darin, dass einem Azubi mehrere Wohnorte zugeordnet werden.

In [Abbildung 7.9](#) sehen Sie ausgehend von der Entität *auszubildender* einen vertikalen Strich auf der Beziehungslinie. Ausgehend von der Entität *adresse* sehen Sie die sogenannte *Krähfußnotation*, die hier für mehrere Adressen steht, die einem Auszubildenden zugeordnet werden können. Diese Notation wird im Entity-Relationship-Modell verwendet, um eine n-fache Beziehung zu einer Entität darzustellen.

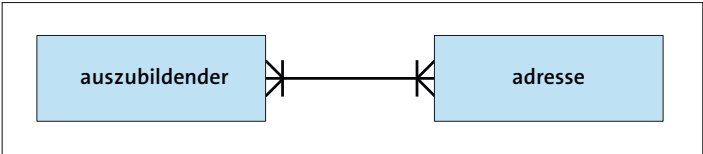


Abbildung 7.9 Die Entitäten »auszubildender« und »adresse« stehen in einer 1:n-Beziehung.

In Abbildung 7.10 sehen Sie ein vollständiges Entitäten-Modell für die Entitäten *auszubildender* und *adresse*. Darin werden die 1:n-Beziehung zwischen den Entitäten *auszubildender* und *adresse* sowie die Attribute dargestellt, die den Entitäten zugeordnet sind.

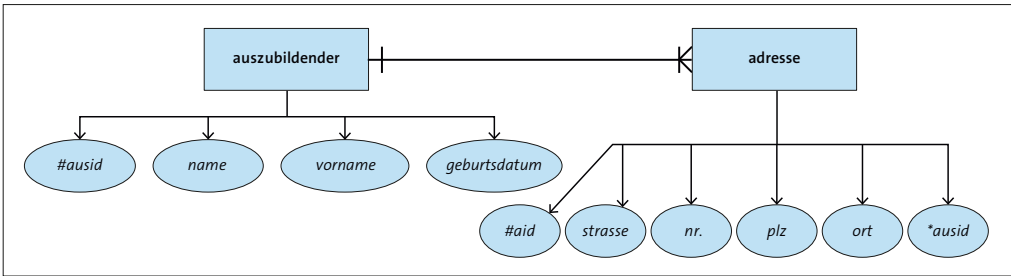


Abbildung 7.10 Ein vollständiges Entitätenmodell für die Beziehung der Entitäten »auszubildender« und »adresse«

Betrachten Sie die Attribute der Entität *adresse*. Auch hier findet sich ein Attribut *ausid*. Dieses Attribut dient ebenfalls als Verweis auf den Primärschlüssel *ausid* der Entität *auszubildender*. Das Attribut *ausid* der Entität *adresse* stellt also das Fremdschlüsselattribut dar, mit dem wir auf das Primärschlüsselattribut der Tabelle *auszubildender* verweisen und somit eine 1:n-Beziehung zwischen den Entitäten *auszubildender* und *adresse* realisieren.

Sehen wir uns als Nächstes auch hier wieder Tabellen an, die die Entitäten *auszubildender* und *adresse* in ihrer Beziehung zueinander repräsentieren. Die Tabelle *auszubildender* steht zur Tabelle *adresse* in einer 1:n-Beziehung. Auch hier wird aus jeder Zeile der Tabelle *adresse* mit einem Fremdschlüsselwert der Spalte *ausid* auf einen Primärschlüsselwert der Spalte *ausid* der Tabelle *auszubildender* verwiesen.

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001
2	Klein	Sabine	10.05.2002

Tabelle 7.7 Die Tabelle »auszubildender«

ausid	name	vorname	geburtsdatum
3	Lang	Peter	11.03.2001
4	Müller	Ralf	01.04.2001
5	Klein	Sabine	20.05.2003

Tabelle 7.7 Die Tabelle »auszubildender« (Forts.)

aid	strasse	nr.	plz	ort	ausid
1	Mondstraße	8	50827	Köln	1
2	Sternstraße	10	50127	Bonn	1
3	Sonnenstraße	1	50129	Bonn	2
4	Sternstraße	25	50126	Bonn	2
5	Jupiterstraße	7	50827	Köln	3

Tabelle 7.8 Die Tabelle »adresse«

Sehen wir uns die Zeilen genauer an. Die Spalte *aid* (kurz für »Adress-ID«) ist die Primärschlüsselspalte der Tabelle *adresse*. Die Adressen mit den Primärschlüsselwerten 1 und 2 sind in der Fremdschlüsselspalte *ausid* jeweils dem Fremdschlüsselwert 1 zugeordnet. Ein Blick auf die Tabelle *auszubildender* offenbart, dass es sich bei dem Auszubildenden mit dem Primärschlüsselwert 1 der Primärschlüsselspalte *ausid* um den Auszubildenden Ralf Müller handelt, dem somit zwei Adressen zugeordnet sind. In der Tabelle *adresse* findet sich ein weiterer Auszubildender, dem zwei Adressen zugewiesen sind, sowie ein Auszubildender, für den nur eine Adresse hinterlegt ist.

Eine m:n-Beziehung erkennen

Mit der *m:n-Beziehung* können Sie ein Verhältnis beschreiben, in dem einem Ausbildungsberuf mehrere Lehrfächer zugeordnet werden können. Einem Lehrfach wiederum können mehrere Ausbildungsberufe zugeordnet werden. Hier haben wir es also mit einer klassischen m:n-Beziehung zu tun.

In [Abbildung 7.11](#) sehen Sie in eine m:n-Beziehung zwischen den Entitäten *ausbildungsberuf* und *lehrfach*, in der das m:n-Verhältnis zwischen den Relationen mit der Krähenfußnotation beschrieben wird.

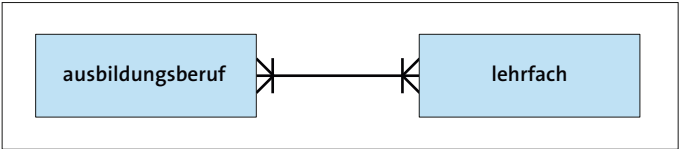


Abbildung 7.11 Die Entitäten »ausbildungsberuf« und »lehrfach« stehen in einer m:n-Beziehung.

Betrachten wir auch hier ein vollständiges Entity-Relationship-Modell der Entitäten *ausbildungsberuf* und *lehrfach*, das auch die Attribute enthält. Abbildung 7.12 zeigt die Entitäten *ausbildungsberuf* und *lehrfach* in einer m:n-Beziehung.

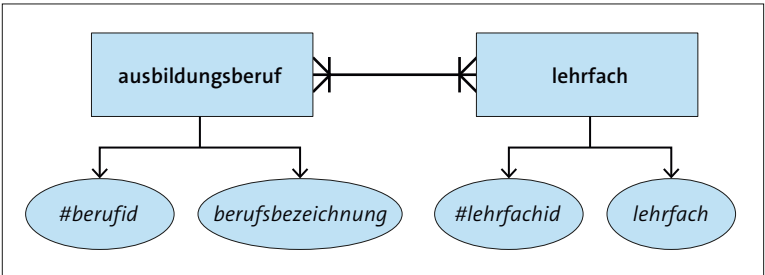


Abbildung 7.12 Ein vollständiges Entitätenmodell für die Beziehung der Entitäten »ausbildungsberuf« und »lehrfach«

Beiden Entitäten wurden Primärschlüsselattribute zugeordnet. Zum einem sehen Sie das Attribut *berufid*, das das Primärschlüsselattribut für die Entität *ausbildungsberuf* darstellt.

Zum anderen sehen Sie das Attribut *lehrfachid*, das Primärschlüsselattribut für die Entität *lehrfach*. Somit verfügen wir für beide Entitäten über Primärschlüsselattribute, über die wir eine m:n-Beziehung realisieren können.

Auch hier sehen wir uns zum besseren Verständnis wieder Tabellen an, die die Entitäten *ausbildungsberuf* und *lehrfach* in ihrer Beziehung zueinander repräsentieren. Die Tabelle, die die Entität *Ausbildungsberuf* repräsentiert, enthält zwei Spalten: eine Primärschlüsselspalte mit der Bezeichnung *berufid* und eine Spalte *berufsbezeichnung*. Hier sind also keine Fremdschlüsselspalten enthalten.

berufid	berufsbezeichnung
1	Elektroniker
2	Industriekaufmann

Tabelle 7.9 Die Tabelle »ausbildungsberuf«

berufid	berufsbezeichnung
3	Schlosser
4	Betriebswirt

Tabelle 7.9 Die Tabelle »ausbildungsberuf« (Forts.)

Als Nächstes sehen wir uns die Tabelle *lehrfach* an. Auch diese Tabelle ist mit einer Primärschlüsselspalte ausgestattet. Es handelt sich um die Spalte *lehrfachid*. Außerdem enthält sie die Spalte *lehrfach*, in der Lehrfächer aufgeführt sind. Auch hier sehen Sie keine Spalte, die einen Fremdschlüssel enthält, der eine Beziehung zu der Tabelle *ausbildungsberuf* herstellen könnte.

lehrfachid	lehrfach
1	Deutsch
2	Kommunikation
3	Mathematik
4	Mechanik
5	Buchhaltung
6	Englisch
7	Integrierte Schaltungstechnik

Tabelle 7.10 Die Tabelle »lehrfach«

Vermutlich stellen Sie sich jetzt völlig zu Recht die Frage, wie denn die m:n-Beziehung zwischen den Tabellen hergestellt wird, die im Entity-Relationship-Modell festgelegt wurde.

Die Antwort lautet: Indem wir eine weitere Tabelle einführen, in der wir die Primärschlüsselwerte der Tabellen *ausbildungsberuf* und *lehrfach* in Beziehung setzen. Der neu einzuführenden Tabelle geben wir die Bezeichnung *beruflehrfach*.

Die neu eingeführte Tabelle (siehe [Tabelle 7.11](#)) enthält die zwei Spalten *berufid* und *lehrfachid*. Die Wertkombinationen dieser beiden Spalten dürfen nur eindeutig vorkommen. Daher stellt die Kombination aus beiden Spalten einen Primärschlüssel dar. Es handelt sich also um einen Primärschlüssel, der aus mehreren Spalten besteht.

Mit den Spalten, die nicht nur den Primärschlüssel der Tabelle *beruflehrfach* darstellen, können wir jetzt Beziehungen zwischen den Tabellen *ausbildungsberuf* und *lehrfach* herstellen, da die Spaltenwerte der Spalte *berufid* auf die Primärschlüsselspalte

berufid der Tabelle *ausbildungsberuf* (siehe Tabelle 7.9) und die Spaltenwerte der Spalte *lehrfachid* auf die Primärschlüsselwerte der Tabelle *lehrfach* (siehe Tabelle 7.10) verweisen. Die beiden Spalten stellen also auch jeweils einen fremden Schlüssel dar, dessen Werte auf einen Primärschlüsselwert in einer anderen Tabelle verweisen. Somit können wir festhalten, dass es sich bei den Spalten *berufid* und *lehrfachid* auch um Fremdschlüsselspalten handelt.

Sehen wir uns ein paar konkrete Beispiele an. In der Tabelle *ausbildungsberuf* finden Sie in der Spalte *berufid* den Wert 1, der dem Ausbildungsberuf *Elektroniker* zugeordnet ist. In der Tabelle *beruflehrfach* sehen Sie in der Spalte *berufid* drei Werte, die dem Wert 1 entsprechen. In den betroffenen Zeilen können Sie unter der Spalte *lehrfachid* die jeweils zugeordneten Lehrfächer ermitteln, die wiederum dem Ausbildungsberuf *Elektroniker* zugeordnet sind. Es handelt sich um die Lehrfächer mit den Fremdschlüsselwerten 1, 3 und 7. Diese verweisen auf die Primärschlüsselwerte 1, 3 und 7 in der Tabelle *lehrfach*. Dem Ausbildungsberuf *Elektroniker* sind also die Fächer *Deutsch*, *Mathematik* und *Integrierte Schaltungstechnik* zugeordnet.

In der Tabelle *beruflehrfach* finden Sie weitere Beispiele, in denen Schlüsselwerte in Beziehung zueinander gesetzt werden und somit eine m:n-Beziehung abbilden.

berufid	lehrfachid
1	1
1	3
1	7
2	1
2	2
2	6
4	1
4	2
4	5

Tabelle 7.11 Die Tabelle »beruflehrfach«

7.3.3 Eine besondere 1:n-Beziehung – oder Entitäten, die auf sich selbst verweisen

Eine Entität kann in Beziehung zu sich selbst stehen. Auch hierzu liefert die Ausbildungsdatenbank ein Beispiel, denn sie soll auch sämtliche Mitarbeiter verwalten, die den Ausbildungsbetrieb sicherstellen. Somit können wir schon einmal eine Entität

mitarbeiter festlegen. In der Entität *mitarbeiter ausbildungsbetrieb* soll die Hierarchie der Mitarbeiter abgebildet werden. In einem Entity-Relationship-Modell wird eine derartige Beziehung einer Entität zu sich selbst durch eine Linie festgelegt, die von der Entität auf die Entität verweist. Auch hier haben wir es mit einer Beziehung zu tun, für die wir eine Kardinalität bestimmen können. Einem Vorgesetzten sollen n Mitarbeiter zugeordnet werden können. Einem oder mehreren Mitarbeitern hingegen soll ein Vorgesetzter zugeordnet werden. In [Abbildung 7.13](#) sehen Sie die Entität *mitarbeiter ausbildungsbetrieb* und deren Beziehung zu sich selbst gemäß einem Entity-Relationship-Modell.

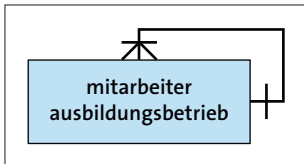


Abbildung 7.13 Die Entität »mitarbeiter ausbildungsbetrieb« steht in einer 1:n-Beziehung zu sich selbst.

mitarbeiterid	name	vorname	vorgesetzterid
1	Müller	Alfred	
2	Ungern	Peter	1
3	Erdenschein	Claudia	1
4	Sternenschein	Ute	1
5	Augustus	Frank	1
6	Erdenfels	Christine	
7	Hoffnung	Ralf	6
8	Freud	Erika	6
9	Bergfels	Diether	6
10	Lemon	Reinhold	6

Tabelle 7.12 Die Tabelle »mitarbeiter ausbildungsbetrieb«

In [Tabelle 7.12](#) sehen Sie zunächst eine Primärschlüsselspalte mit der Bezeichnung *mitarbeiterid*. In dieser Spalte werden also die eindeutigen Primärschlüsselwerte gespeichert. Es folgen die Spalten *name* und *vorname*, in denen die Namen und Vornamen der Mitarbeiter des Ausbildungsbetriebs hinterlegt sind.

Die nächste Spalte, *vorgesetzterid*, hat die Aufgabe, eine Beziehung zwischen den Mitarbeitern und ihren Vorgesetzten zu schaffen. Es handelt sich also um eine Fremdschlüsselspalte. Sehen Sie sich die Spaltenwerte der Spalte *vorgesetzterid* der Mitarbeiter Peter Ungern, Claudia Erdenschein, Ute Sternenschein und Frank Augustus an. Den Mitarbeitern des Ausbildungsbetriebs ist jeweils ein Vorgesetzter mit der ID 1 zugeordnet. Der Wert 1 aus der Spalte *vorgesetzterid* stellt die Beziehung zum Wert 1 in der Spalte *mitarbeiterid* her. Der Vorgesetzte dieser Mitarbeiter ist also Alfred Müller.

7.3.4 Starke und schwache Entitäten unterscheiden

Sie haben bereits die 1:1-, 1:n- und m:n-Beziehungen kennengelernt. Vielleicht ist Ihnen ja schon aufgefallen, dass es Entitäten gibt, die Beziehungen eingehen und dabei abhängig von der Existenz einer anderen Entität sind. Als Beispiel betrachten wir noch einmal die 1:n-Beziehung zwischen der Entität *auszubildender* und der Entität *adresse*.

Die Entität *auszubildender* kann eigenständig existieren, unabhängig davon, ob eine weitere Entität eine Beziehung mit ihr eingeht. Bei der Entität *auszubildender* handelt es sich also um eine sogenannte *starke Entität*.

Die Entität *adresse* hingegen steht in absoluter Abhängigkeit von der Entität *auszubildender*. Das Fremdschlüsselattribut *ausid* der Entität *adresse* verweist immer auf das Primärschlüsselattribut *ausid*. Eine Adresse ist also immer von der Existenz eines Auszubildenden abhängig. Derartige Entitäten bezeichnet man als *schwache Entitäten*. Abbildung 7.14 stellt die Verbindung zwischen starken und schwachen Entitäten noch einmal dar.

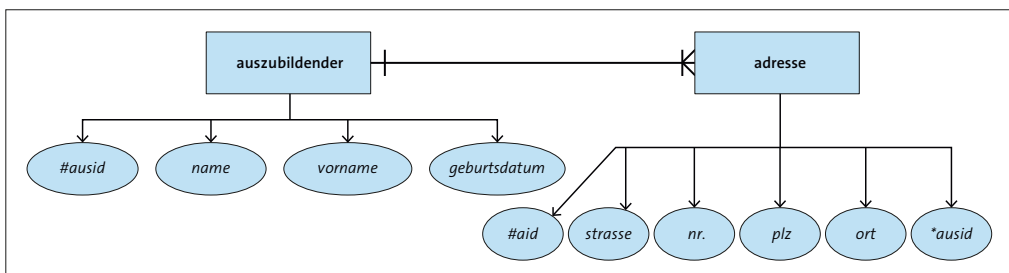


Abbildung 7.14 Beispiele für schwache (»adresse«) und starke Entitäten (»auszubildender«)

Diese Verbindung hat besonders dann große Auswirkungen für Modelle, die in reale Datenbanken überführt werden, wenn Einfüge-, Aktualisierungs- oder Löschoperationen durchgeführt werden sollen.

Um die Auswirkungen von schwachen und starken Entitäten nachvollziehen zu können, betrachten wir zwei Tabellen, die in einer 1:n-Beziehung stehen. Es handelt sich

um die bereits bekannten Tabellen *auszubildender* und *adresse*, die den gleichnamigen Entitäten entsprechen. Die Tabelle *auszubildender* (Tabelle 7.13) wurde aus der starken Entität *auszubildender* abgeleitet. Folglich ist jede Zeile in der Tabelle *auszubildender* unabhängig von der Tabelle *adresse*, die in Beziehung zu ihr steht. Jede Zeile kann also unabhängig davon existieren, ob es eine andere Zeile gibt, die in Beziehung zu dieser Tabelle steht.

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001
2	Klein	Sabine	10.05.2002
3	Lang	Peter	11.03.2001
4	Müller	Ralf	01.04.2001
5	Klein	Sabine	20.05.2003

Tabelle 7.13 Eine Tabelle, die der Entität »auszubildender« entspricht

Als Nächstes betrachten wir die Tabelle *adresse* (Tabelle 7.14), die aus der schwachen Entität *adresse* abgeleitet ist. Mithilfe der Fremdschlüsselspalte *ausid* wird hier auf die Primärschlüsselspalte *ausid* der Tabelle *auszubildender* verwiesen. Das heißt, dass die Zeilen in der Tabelle *adresse* eindeutig auf Zeilen der Tabelle *auszubildender* verweisen und somit eine eindeutige Beziehung zu dieser Tabelle herstellen.

aid	strasse	nr.	plz	ort	ausid
1	Mondstraße	8	50827	Köln	1
2	Sternstraße	10	50127	Bonn	1
3	Sonnenstraße	1	50129	Bonn	2
4	Sternstraße	25	50126	Bonn	2
5	Jupiterstraße	7	50827	Köln	3

Tabelle 7.14 Eine Tabelle, die der Entität »adresse« entspricht

Die Beziehung zwischen einer schwachen und einer starken Entität hat sowohl Auswirkungen auf die starke wie auf die schwache Entität. Folgende Fälle lassen sich beschreiben:

1. In der Tabelle, die eine starke Entität repräsentiert, existiert eine Zeile autonom. Hier ist keine Beziehung zu einer Zeile in einer anderen Tabelle zu beachten. Einfüge-, Aktualisierungs- und Löschooperationen können hier völlig autonom be-

trachtet werden. Es gibt keine Beziehungen zu Zeilen in anderen Tabellen, die wir beachten müssten. Für Aktualisierungs- und Löschanweisungen gilt dies nur, solange keine schwache Entität eine Beziehung zu Zeilen einer starken Entität hat.

2. In einer Tabelle, die eine schwache Entität repräsentiert, existieren Zeilen, die eine Beziehung zu Zeilen herstellen, die eine starke Entität repräsentieren. Diese Beziehung hat unmittelbare Auswirkungen auf Einfüge-, Aktualisierungs- und Löschoperationen:

- Eine Zeile einer Tabelle, die eine starke Entität repräsentiert, kann nicht gelöscht werden, solange eine Beziehung zu einer Zeile einer Tabelle gegeben ist, die eine schwache Entität repräsentiert. Besteht hingegen keine solche Beziehung, so kann die Zeile aus der Tabelle, die einer starken Entität entspricht, einfach gelöscht werden.
- Eine Zeile aus einer Tabelle, die einer schwachen Entität entspricht, darf nur auf gültige Primärschlüsselwerte in den Zeilen der Tabelle verweisen, die eine starke Entität darstellen. Wenn eine neue Zeile eingefügt wird, ist dies unbedingt zu beachten. Die einzige zulässige Ausnahme ist der Fall, dass eine Zeile aus einer Tabelle, die einer schwachen Entität entspricht, keine Beziehung zu einer Zeile einer Tabelle eingeht, die einer starken Entität entspricht. Dies ist etwa der Fall, wenn Sie es mit NULL-Werten in einer Fremdschlüsselspalte zu tun haben.
- Zeilen einer Tabelle, die einer schwachen Entität entspricht, können gelöscht werden, auch wenn eine Beziehung zu den Zeilen einer starken Entität existiert. Merken Sie sich hier, dass die schwache Entität mit den Fremdschlüsselwerten der Besitzer der Beziehung ist. In diesem Fall wird mit dem Löschen der Zeile lediglich die Beziehung zu der Zeile aufgehoben, die der starken Entität entspricht.
- Es muss also sichergestellt werden, dass aus Zeilen von Tabellen, die einer schwachen Entität entsprechen, nur Beziehungen existieren, die auch tatsächlich in den Zeilen der Tabelle, die einer starken Entität entspricht, in Form von Primärschlüsselwerten vorhanden sind.
- Folglich ist es auch nicht zulässig, dass Primärschlüsselwerte aus Zeilen einer Tabelle, die einer starken Entität entspricht, geändert werden, wenn mithilfe eines Fremdschlüsselwerts eine Beziehung zu einer Tabelle existiert, die einer schwachen Entität entspricht. Hier gibt es nur eine Ausnahme; auf sie komme ich später zu sprechen, wenn wir uns kaskadierte Löscho- und Änderungsoperationen ansehen.

Diese Zusammenhänge zwischen starken und schwachen Entitäten werden auch als *Eltern-Kind-Beziehung* oder – nicht ganz so neutral – als *Vater-Sohn-Beziehung* be-

zeichnet. Die Analogie ist einfach: Ein Kind kann nur existieren, wenn Eltern vorhanden sind. Potenzielle Eltern hingegen können auch ohne ein Kind existieren.

Die Auswirkungen, die ich Ihnen soeben in Bezug auf die Beziehungen zwischen starken und schwachen Entitäten dargestellt habe, können wir auch als Anforderungen an ein relationales Datenbanksystem formulieren. Relationale Datenbanksysteme stellen sicher, dass die hier aufgeführten Auswirkungen immer eintreten und dass die Integrität einer Beziehung stets gewährleistet ist. Diese Gewährleistung wird als *referenzielle Integrität* bezeichnet und stellt sicher, dass nur gültige Beziehungen existieren.

Zusammenfassung: Entitäten in Beziehung zueinander bringen

Entitäten können in Beziehung zu anderen Entitäten stehen. Wenn eine Beziehung identifiziert wurde, wird sie zwischen den Entitäten im Diagramm mit einer einfachen Verbindungslinie beschrieben.

Eine Beziehung wiederum ist mit einem Zahlenverhältnis versehen. Einem Mitarbeiter wird zum Beispiel ein Arbeitsvertrag zugeordnet. Einem Mitarbeiter können mehrere Adressen zugeordnet werden. Außerdem ist es zum Beispiel möglich, einem oder mehreren Mitarbeitern einen oder mehrere Fortbildungskurse zuzuordnen.

Hier wird von folgenden Beziehungstypen gesprochen:

- ▶ *1:1-Beziehung*
- ▶ *1:n-Beziehung*
- ▶ *m:n-Beziehung*

Die Beziehungstypen 1:1, 1:n und m:n werden als *Kardinalität* bezeichnet.

Wenn Entitäten in Beziehung zueinander stehen, wird zwischen starken und schwachen Entitäten unterschieden. Eine starke Entität ist unabhängig von der Existenz einer schwachen Entität. Umgekehrt gilt, dass eine schwache Entität abhängig von der Existenz einer starken Entität ist. Eine relationale Datenbank stellt sicher, dass die referenzielle Integrität stets gegeben ist. So wird verhindert, dass nicht gültige Beziehungen erstellt werden.



7.3.5 Die Beziehungstypen »identifying« und »non-identifying«

Es gilt, zwischen zwei Beziehungstypen zu unterscheiden, die als *identifying* und *non-identifying* bezeichnet werden.

▶ Identifizierende Beziehung (identifying)

Eine identifizierende Beziehung in einer Kindtabelle stellt eine besonders starke Beziehung dar. Betrachten wir die Entitäten Auszubildender und Vertrag. Ein Ausbildungsvertrag kann nicht ohne einen Auszubildenden existieren.

Wie können Sie nun sicherstellen, dass ausschließlich Verträge angelegt werden können, die verbindlich einem Auszubildenden zugeordnet sind? Die Lösung ist an dieser Stelle einfach: Um zu garantieren, dass für jeden Vertrag eine gültige Beziehung zu einem Auszubildenden hergestellt wird, wird die Fremdschlüsselspalte `ausid` mit in den Primärschlüssel aufgenommen. Der Primärschlüssel des Vertrags besteht somit aus den Attributen `vid` und `ausid`. Wenn also ein Vertrag angelegt wird, ist die verbindliche Angabe eines Fremdschlüssels für das Attribut `vid` zwingend erforderlich.

► **Nicht identifizierende Beziehung (non-identifying)**

Beziehungen zwischen Eltern-Kind-Entitäten, die nicht identifizierend sind, erlauben die Existenz von Kind-Entitäten unabhängig von Eltern-Entitäten. Für unser Beispiel bedeutet es, dass wir einen Ausbildungsvertrag ohne die Existenz einer Beziehung zu einem Auszubildenden anlegen können. Das Fremdschlüsselattribut wird nicht mit dem Primärschlüsselattribut zu einen ganzheitlichen Primärschlüssel zusammengefasst. In diesem Fall ist es also möglich, Verträge vorab anzulegen und sie gegebenenfalls später mit einer Beziehung zu einem Auszubildenden zu versehen.

7.4 Datenmodelle in der UML-Notation darstellen

In diesem Abschnitt stelle ich Ihnen die neuere *UML-Notation* vor, mit der Sie ebenfalls Datenbankmodelle modellieren können. *UML* steht dabei für *Unified Modeling Language*, also »vereinheitlichte Modellierungssprache«. Sie wurde ursprünglich entwickelt, um Klassendiagramme in objektorientierten Programmiersprachen zu modellieren.

Sehen wir uns als Erstes an, wie Sie mit der UML-Notation eine starke Entität definieren. Als Beispiel nutzen wir hierfür wieder die Entität *auszubildender*.

Die MySQL Workbench enthält einen komfortablen Entity-Relationship-Diagrammeditor. Wenn Sie mit einer MySQL-Datenbank arbeiten, können Sie diesen Diagrammeditor verwenden, um Entity-Relationship-Modelle zu erstellen. Im Rahmen unseres Einstiegs in SQL nutze ich den MySQL-Diagrammeditor, um Ihnen die neue Notation vorzustellen.



Diagrammeditoren

Im *Microsoft SQL Server Management Studio* finden Sie den Diagrammeditor direkt unter dem jeweiligen Datenbankknoten, den Sie gerade verwenden. Der *pgAdmin IV Client* einer PostgreSQL-Datenbank bietet ebenfalls einen Diagrammeditor für das Erstellen von Entity-Relationship-Modelle. Wählen Sie im Menü **Tools** den Eintrag **New ERD PROJECT** aus.

Bei der Darstellung einer Entität aus [Abbildung 7.15](#) handelt sich um ein Rechteck, das in drei Bereiche unterteilt ist.

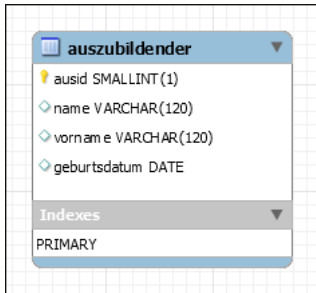


Abbildung 7.15 Die Entität »auszubildender« in UML-Notation

- Im oberen Bereich wird der Name der Entität angegeben.
- Im mittleren Bereich werden die Attribute festgelegt:
 - Den Attributen werden hier auch gleich die Wertebereiche (Datentypen) zugeordnet.
 - Wenn es sich um ein Primärschlüsselattribut handelt, wird das betreffende Attribut durch ein Schlüsselssymbol gekennzeichnet.
 - Wenn ein Attribut **NULL**-Werte zulassen soll, so wird das Attribut mit einer weißen Raute gekennzeichnet.
 - Soll ein Attribut hingegen keine **NULL**-Werte zulassen, so ist das Attribut mit einer farblich ausgefüllten Raute zu kennzeichnen.
- Zu guter Letzt werden im unteren Bereich die Primärschlüssel- bzw. Fremdschlüsselbeziehungen dargestellt.

Die Entität *auszubildender* verfügt in unserem Beispiel nur über eine Primärschlüsseldefinition, die hier durch ein Schlüsselssymbol vor dem Attribut *ausid* gekennzeichnet wurde. In [Abschnitt 7.2.4](#) haben Sie erfahren, dass Sie Attributen einen Wertebereich (Datentyp) zuordnen können. In der UML-Notation einer Entität werden die Datentypen, die einen Wertebereich für ein Attribut festlegen, direkt hinter der Bezeichnung eines Attributs angegeben. Hier werden Attributen folgende Datentypen mit den entsprechenden Wertebereichen zugeordnet:

- *ausid* – **SMALLINT**, ein ganzzahliger Datentyp
- *name* – **VARCHAR(120)**, ein Datentyp für eine Zeichenkette; darf 120 Zeichen lang sein
- *vorname* – **VARCHAR(120)**, ebenfalls ein Datentyp für eine Zeichenkette, die 120 Zeichen lang sein darf
- *geburtsdatum* – **DATE**, ein Datentyp, der ein Datum repräsentiert

Aus dem Modell geht auch hervor, dass bis auf das Primärschlüsselattribut jedes andere Attribut (*name*, *vorname* und *geburtsdatum*) NULL-Werte zulässt. Sie sehen also, dass wir in der UML-Notation sehr übersichtlich eine Menge von Informationen für zu modellierende Entitäten unterbringen können.

In [Abbildung 7.16](#) sehen Sie, wie eine 1:1-Beziehung zwischen den Entitäten *auszubildender* und *ausbildungsvertrag* gemäß der UML-Notation dargestellt wird.

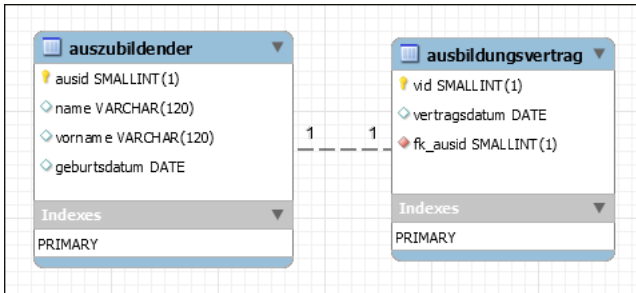


Abbildung 7.16 Eine 1:1-Beziehung zwischen den Entitäten »auszubildender« und »ausbildungsvertrag« in UML-Notation darstellen

Um die Beziehung abzubilden, werden die Entitäten mit einer gestrichelten Linie verbunden. Die 1:1-Kardinalität wird mit der Zahl 1 beim Austritt der Verbindungslinie aus einer Entität und beim Eintritt der Verbindungslinie in die andere Entität einer Beziehung notiert.

Wie sieht es aus, wenn Sie eine 1:n-Beziehung zwischen Entitäten in der UML-Notation darstellen wollen? In [Abbildung 7.17](#) sehen Sie eine gestrichelte Verbindungslinie zwischen den Entitäten *auszubildender* und *adresse*. Die Kardinalität der Beziehung wird wieder am Anfang und am Ende der Beziehungslinie dargestellt. Hier wird die Kardinalität in der Form »1..*« angegeben. Einem Auszubildenden werden in diesem Modell also eine oder mehrere Adressen zugeordnet.

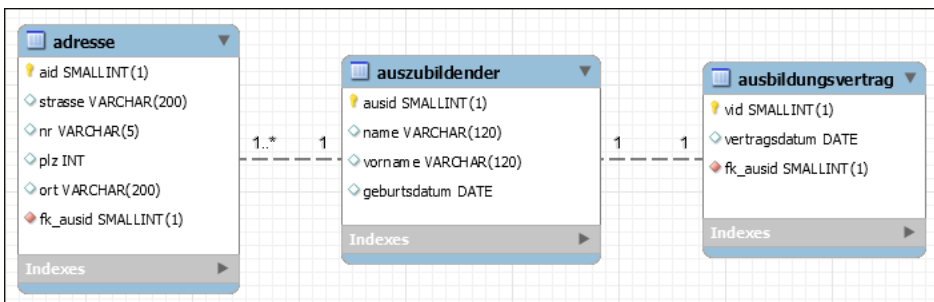


Abbildung 7.17 Eine 1:n-Beziehung zwischen den Entitäten »auszubildender« und »adresse« in der UML-Notation darstellen

Abbildung 7.18 zeigt Ihnen eine m:n-Beziehung in der UML-Notation. Dieses Beispiel haben Sie schon in der Entity-Relationship-Modell-Notation kennengelernt. In der UML-Notation wird sofort erkennbar, dass Sie eine weitere Tabelle benötigen, in der die Primärschlüsselwerte der beiden Tabellen *ausbildungsberuf* und *lehrfach* in Beziehung zueinander gesetzt werden können. Es handelt sich um die Tabelle *beruflehrfach*.

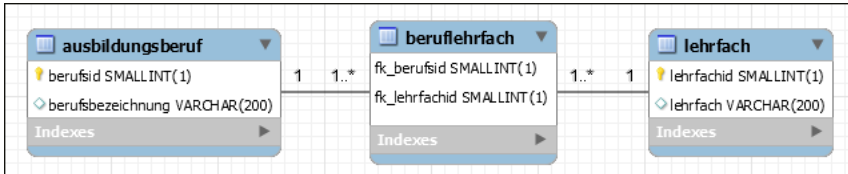


Abbildung 7.18 Eine m:n-Beziehung zwischen den Entitäten »ausbildungsberuf« und »lehrfach« in der UML-Notation darstellen

Zusammenfassung: Die UML-Notation

Entitäten werden in der *UML-Notation* durch Rechtecke dargestellt. Im oberen Bereich des Rechtecks wird der Name einer Entität notiert.

Unterhalb des Namens werden die Attribute aufgelistet, die einer Entität zuzuordnen sind. In dieser Auflistung werden bereits die Wertebereiche (Datentypen) den Attributen zugeordnet.

Wenn ein Attribut verbindlich anzugeben ist, wird es mit einer farblich ausgefüllten vorangestellten Raute versehen.

Handelt es sich bei einem Attribut um ein Primärschlüsselattribut, so wird dies mit einem vorangestellten Schlüsselsymbol gekennzeichnet.

Beziehungen zwischen Entitäten werden auch in der UML-Notation durch eine einfache oder gestrichelte Verbindungslinie dargestellt. Eine durchgezogene Linie steht für eine identifying Beziehung, eine gestrichelte Linie hingegen für eine non-identifying Beziehung.

Die Kardinalität, die für eine Beziehung definiert wird, wird hingegen mit ganzen Zahlen wie »1 zu 1..*« dargestellt (einem Mitarbeiter wird eine oder werden mehrere Adressen zugeordnet).

7.5 Übungen

Übung 1

Jedem Auszubildenden steht Urlaub zu. Modellieren Sie eine Entität *urlaub*, in der die Informationen *beginn urlaub* und *ende urlaub* hinterlegt sind. Die beiden Entitäten *auszubildender* und *urlaub* stehen in Beziehung zueinander. Einem Auszubildenden

können viele Urlaubszeiträume zugeordnet werden. Berücksichtigen Sie bei der Modellierung, dass die Informationen *beginn urlaub* und *ende urlaub* verbindlich gespeichert werden. Außerdem ist die Entität *urlaub* mit einem eindeutigen Schlüsselattribut *urlaubid* auszustatten. In der zu modellierenden Entität *urlaub* soll ein Fremdschlüsselattribut *ausid* definiert werden, mit dem wiederum auf das Primärschlüsselattribut *ausid* der Entität *auszubildender* verwiesen wird.

Beachten Sie dabei, dass Sie nicht unbedingt ein Programm brauchen, um ein E/R-Modell zu entwickeln. Ein Stift und ein Blatt Papier tun es auch.

Übung 2

Überführen Sie das Entity-Relationship-Modell in die UML-Notation. Ergänzen Sie die Attribute um die Wertebereiche, die den Attributen zugeordnet werden sollen.

Lösung zu Übung 1

In [Abbildung 7.19](#) ist die Lösung zu Übung 1 dargestellt.

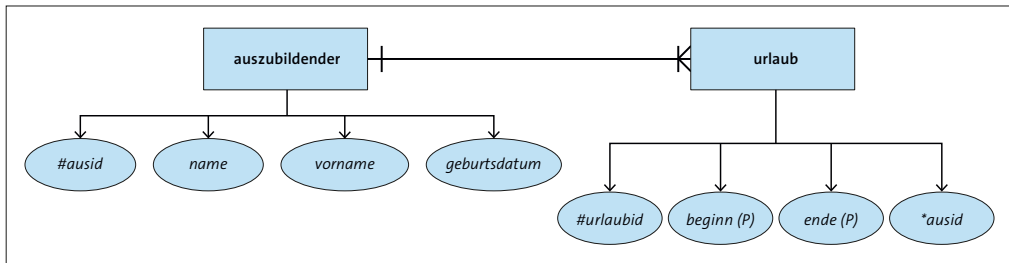


Abbildung 7.19 Ein Entity-Relationship-Modell, das die Beziehung zwischen den Entitäten »auszubildender« und »urlaub« beschreibt

Die Entitäten *auszubildender* und *urlaub* wurden durch eine Linie miteinander verbunden, die die Beziehung darstellt. Die Kardinalität der Beziehung wurde hier entsprechend durch einen vertikalen Strich und die Krähenfußnotation angegeben. Der Entität *urlaub* sind die Attribute *urlaubid*, *beginn*, *ende* und *ausid* zugeordnet. Das Attribut *urlaubid* repräsentiert das Primärschlüsselattribut und wurde mit einem #-Zeichen gekennzeichnet. Das Attribut *ausid* hingegen entspricht dem Fremdschlüsselattribut und wurde hier mit einem *-Zeichen gekennzeichnet.

Zu guter Letzt wurden die Attribute *beginn* und *ende* mit dem Buchstaben P versehen, um sie als Pflichtfeld zu kennzeichnen.

Lösung zu Übung 2

Die Lösung für die Überführung des Entitätenmodells und seine Beziehungen könnte so wie in [Abbildung 7.20](#) aussehen.

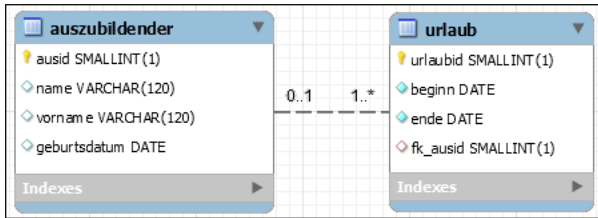


Abbildung 7.20 Überführung des Entity-Relationship-Modells in ein Modell in UML-Notation

Hier sehen Sie ebenfalls, wie die Entitäten *auszubildender* und *urlaub* modellhaft in einer 1:n-Beziehung dargestellt sind. Den Attributen *urlaubid*, *beginn*, *ende* und *fk_ausid* wurden die Wertebereiche **SMALLINT**, **DATE**, **DATE** und **SMALLINT** zugeordnet.

Kapitel 8

Datenmodelle optimieren (Normalisierung)

Ein Datenmodell zu erstellen, ist der erste Schritt, um eine vollständige Übersicht über ein abzubildendes System zu erhalten. Im zweiten Schritt muss sichergestellt werden, dass auch die Qualität des Modells unseren Anforderungen entspricht.

Im Idealfall achten Sie bereits während der Modellierung Ihrer Datenbank darauf, sie zu optimieren, indem Sie Normalisierungen nutzen. Dazu müssen Sie die sogenannten *Normalformen* kennen, deren Ziel es ist, Redundanzen zu vermeiden.

8.1 Redundanzen erkennen

Im Rahmen dieser Einführung in SQL stelle ich Ihnen die ersten drei Normalformen vor, die es Ihnen ermöglichen, anhand von Bedingungen zu prüfen, ob ein Datenmodell Redundanzen aufweist. Es gibt noch weitere Normalformen, die Verwendung finden können, wie z. B. die *Boyce-Codd-Normalform* (BCNF). Auf die BCNF und weitere Normalformen werde ich hier jedoch nicht eingehen.

Sie werden sehen, dass Sie mit den ersten drei Normalformen die Qualität eines Datenmodells hinlänglich prüfen können, indem Sie es auf Redundanzen untersuchen. Bevor wir uns mit der Vermeidung von Redundanzen beschäftigen, müssen wir jedoch zuerst einmal klären, was mit dem Begriff *Redundanz* überhaupt gemeint ist.

8.1.1 Was ist eine Redundanz?

Eine Redundanz ist dann gegeben, wenn die exakt gleiche Information mehrfach vorhanden und somit eigentlich überflüssig ist.

id	berufsbezeichnung	lehrfach
1	Elektroniker	Deutsch
2	Elektroniker	Mathematik

Tabelle 8.1 Redundanz in einer Tabelle erkennen

id	berufsbezeichnung	lehrfach
3	Elektroniker	Integrierte Schaltungstechnik
4	Industriekaufmann	Deutsch
5	Industriekaufmann	Kommunikation
6	Industriekaufmann	Englisch
7	Betriebswirt	Deutsch
8	Betriebswirt	Kommunikation
9	Betriebswirt	Buchhaltung

Tabelle 8.1 Redundanz in einer Tabelle erkennen (Forts.)

Tabelle 8.1 bildet eine Zuordnung von Ausbildungsberufen und Lehrfächern datentechnisch ab. Sie hat nur einen Haken: Sie ist voll von redundanten Informationen. Sehen Sie sich z. B. die Werte der Spalte *berufsbezeichnung* an. Die Werte *Elektroniker*, *Industriekaufmann* und *Betriebswirt* sind jeweils in drei Zeilen enthalten. Hier wird also jeweils dreimal die gleiche Information gespeichert. Die Spalte *lehrfach* enthält ebenfalls einige redundante Werte.

Sie können sich vorstellen, dass die Redundanz von Daten zu einigen Problemen führt: Auf den ersten Blick ist klar, dass dadurch unnötig viel Speicherplatz und Rechenleistung beansprucht werden, um die Daten zu bearbeiten. Aber das ist nur das kleinere Problem: Viel schlimmer sind die sogenannten *Einfüge-, Änderungs- und Löschanomalien*.

Stellen Sie sich vor, Sie würden einen weiteren Beruf in die Tabelle einfügen. Wenn Sie hierzu bereits Lehrfächer kennen, wäre es erforderlich, das Berufsbild mit der Anzahl von Lehrfächern einzufügen, die Sie ihm zugeordnet haben. Hinzu kommt, dass kein Primärschlüsselwert ein Berufsbild eindeutig identifiziert. Die Spalten *berufsbezeichnung* und *lehrfach* sind beide von den Primärschlüsselwerten der Spalte *id* abhängig. Das heißt, dass weder ein Beruf noch ein Lehrfach eindeutig durch einen Schlüsselwert identifiziert werden kann. Und wie sieht es aus, wenn Sie ein neues Lehrfach in den Ausbildungsbetrieb einführen wollen? Welche Berufsbezeichnungen ordnen Sie dem neuen Lehrfach zu?

Sie merken bereits, dass eine Einfügeoperation auch für Lehrfächer sehr schwierig zu bewerkstelligen ist. Zusätzlich haben Sie nie die Möglichkeit, ein Lehrfach anhand eines gültigen Schlüsselwerts zu ermitteln.

Auch das Ändern einer Berufsbezeichnung würde Probleme machen. Jede Berufsbezeichnung kommt in drei Zeilen vor. Welche beabsichtigen wir denn nun zu ändern?

Die Frage kann nicht klar beantwortet werden – wir wissen es einfach nicht. Beispielsweise kommt die Berufsbezeichnung *Elektroniker* dreimal vor. In diesem Fall haben Sie nur eine Möglichkeit: Sie müssen alle Spaltenwerte der Spalte *berufsbezeichnung* *Elektroniker* einzeln ändern. Auch hier macht uns die fehlende Eindeutigkeit schwer zu schaffen. Das gilt natürlich auch für die Werte der Spalte *lehrfach*, in der ebenfalls einige Lehrfächer redundant sind, da sie mehrfach vorkommen.

Wie sieht es aus, wenn Sie ein Berufsbild löschen möchten? Wenn Sie beispielsweise den *Elektroniker* löschen wollen, so werden auch sämtliche Lehrfächer gelöscht, die dem Berufsbild zugeordnet sind. Wenn es Lehrfächer gibt, die ausschließlich diesem Berufsbild zugeordnet sind, so sind sie unwiederbringlich verloren. Umgekehrt sieht das Ganze natürlich genauso aus: Wenn Sie ein Lehrfach löschen, das ausschließlich einer Berufsbezeichnung zugeordnet ist, so geht der Wert der Berufsbezeichnung ebenfalls verloren.

8.1.2 Was bedeutet Normalisierung?

Unter *Normalisierung* versteht man das schrittweise Entfernen von Redundanzen aus einem Datenmodell. Dazu werden mehrfach vorkommende Daten in einer oder mehreren Tabellen bearbeitet. Die Normalisierung stellt also einen Weg dar, Schritt für Schritt redundant vorkommende Daten aus einem Datenmodell und somit auch aus den daraus resultierenden Tabellen zu entfernen.

Im Verlauf dieses Kapitels werde ich Ihnen die ersten drei Normalformen vorstellen. Dabei handelt sich um drei Bedingungen, die aufeinanderfolgend erfüllt sein müssen. Sie helfen uns dabei, Redundanzen zu vermeiden.

Begriffe: Entitäten und Attribute

Bevor wir uns der Normalisierung widmen, ist noch ein kleiner Hinweis angebracht: Im Rahmen der Einführung der Normalformen verwende ich die Begriffe *Entität* und *Attribut*, die Sie bereits bei der Modellierung von Datenbanken in Kapitel 7 kennengelernt haben. Um die Normalformen zu erläutern, benutze ich in diesem Kapitel allerdings Beispiele mit konkreten Tabellen, da die Normalformen so besser verständlich sind. Wenn wir Tabellen betrachten, reden wir eigentlich über Zeilen und Spalten. Da die Normalisierung allerdings während der Datenmodellierung durchgeführt wird, behalte ich hier die Wortwahl *Entität* für die Tabelle und *Attribute* für die Spalten bei.

Zusammenfassung: Redundanz und Normalisierung

Spaltenwerte aus Zeilen bzw. Datensätze einer Tabelle sind *redundant* vorhanden, wenn sie mehrfach gleich vorkommen. Hiervon können eine oder mehrere Spalten einer Tabelle betroffen sein.

Durch redundante Daten entstehen

- ▶ *Einfügeanomalien*,
- ▶ *Aktualisierungsanomalien* und
- ▶ *Löschanomalien*,

die eine konsistente Verwaltung der Daten sehr erschweren oder sogar unmöglich machen.

Neben diesen unangenehmen Effekten wird durch redundante Datenhaltung mehr Speicherplatz verbraucht. Bei großen Datenmengen (»ein paar Millionen Datensätze«) macht sich das durchaus bemerkbar. Redundanz ist also zu vermeiden.

Redundanzen werden mit den drei Normalformen aufgelöst. Diesen Prozess der Auflösung von Redundanzen nennt man *Normalisierung*.

8.2 Die 1. Normalform anwenden

Die 1. *Normalform* ist dann erfüllt, wenn die Attribute einer Entität *atomar* sind und keine mehrfach vorkommenden Werte enthalten. Grundsätzlich wird etwas als *atomar* bezeichnet, wenn es nicht weiter teilbar ist. Im Zusammenhang mit Attributen von Entitäten bedeutet dies, dass ein Attribut ausschließlich für eine Eigenschaft gelten darf.

ausid	auszubildender	adresse
1	Müller Ralf	Mondstraße 8, 50827, Köln
2	Klein Sabine	Sternstraße 10, 50127, Bonn
3	Lang Peter	Sonnenstraße 1, 50126, Bonn
4	Augustus Frank	Jupiterstraße 7, 50827, Köln
5	Müller Ralf	Marsstraße 87, 50827, Köln
6	Klein Sabine	Uranusstraße 36, 50127, Bonn

Tabelle 8.2 Die nicht atomare Tabelle »auszubildender«

In Tabelle 8.2 sehen Sie, dass die Spalte *auszubildender* sich in die zwei Attribute *name* und *vorname* teilen lässt. Auch die Spalte *adresse* ist teilbar, also nicht atomar. Während der Modellierung ist immer darauf zu achten, dass Attribute wie *strasse*, *nr.*, *plz* und *ort* definiert werden, deren Attributwerte nicht mehr weiter teilbar sind. Hinzu

kommt, dass es sich bei den Spaltenbezeichnungen *auszubildender* und *adresse* um Kandidaten für eigene Entitäten handelt. Es würde sich also anbieten, für sie eigene Tabellen zu erstellen. Für dieses Beispiel werden wir dies jedoch nicht umsetzen.

Wie Tabelle 8.2 in der 1. Normalform aussieht, sehen Sie in Tabelle 8.3: Sämtliche Attribute, die in der neuen Tabelle *auszubildender* enthalten sind, sind atomar und somit nicht mehr teilbar.

ausid	name	vorname	strasse	nr.	plz	ort
1	Müller	Ralf	Mondstraße	8	50827	Köln
2	Klein	Sabine	Sternstraße	10	50127	Bonn
3	Lang	Peter	Sonnenstraße	1	50126	Bonn
4	Augustus	Frank	Jupiterstraße	7	50827	Köln
5	Müller	Ralf	Marsstraße	87	50827	Köln
6	Klein	Sabine	Uranusstraße	36	50127	Bonn

Tabelle 8.3 Die Tabelle »auszubildender« in der 1. Normalform

Als Nächstes betrachten wir mit Tabelle 8.4 ein weiteres Beispiel, das sich nicht in der 1. Normalform befindet.

ausid	name	vorname	telefonkontakt
1	Müller	Ralf	022x/1111111, 022x/222222
2	Klein	Sabine	022x/333333, 022x/444444
3	Lang	Peter	022x/555555, 022x/666666
4	Augustus	Frank	022x/777777, 022x/888888

Tabelle 8.4 Eine Tabelle, die sich ebenfalls nicht in der 1. Normalform befindet

In der Spalte *telefonkontakt* werden mehrere Werte angegeben. Es handelt sich also um eine Mehrwertigkeit, die jeweils einem Spaltenwert zugeordnet ist. Auch hier gilt, dass der Spalte *telefon* nur eine Telefonnummer zugeordnet werden darf, da hier ebenfalls nur atomare Werte zulässig sind. Wir kämen ja auch nicht auf die Idee, eine Spalte *auszubildender* mit einer Auflistung aller Auszubildenden zu füllen.

Tabelle 8.4 wurde in Tabelle 8.5 in die 1. Normalform überführt. Jeder Spaltenwert der Spalte *telefonkontakt* ist jetzt einwertig, da es nur einen Eintrag pro Spaltenwert gibt.

ausid	name	vorname	telefonkontakt
1	Müller	Ralf	022x/1111111
2	Müller	Ralf	022x/222222
3	Klein	Sabine	022x/333333
4	Klein	Sabine	022x/444444
5	Lang	Peter	022x/555555
6	Lang	Peter	022x/666666
7	Augustus	Frank	022x/777777
8	Augustus	Frank	022x/888888

Tabelle 8.5 Tabelle 8.4 wurde mit eindeutigen Spaltenwerten in die 1. Normalform überführt.

Ihnen entgeht natürlich nicht, dass die Auflösung der Mehrwertigkeit wiederum zu redundanten Daten führt. Die Auszubildenden sind jetzt jeweils zweimal in der Tabelle enthalten. In den Übungsaufgaben in Abschnitt 8.6 haben Sie Gelegenheit, die entstandene Redundanz aufzulösen.



Zusammenfassung: 1. Normalform

Eine Tabelle befindet sich erst dann in der 1. *Normalform*, wenn die Attribute einer Entität atomar sind. Attribute sind atomar, wenn sie nicht mehr teilbar sind. Eine *Person* ist z. B. in die Attribute *Name*, *Vorname* und *Geburtsdatum* teilbar.

Eine weitere Voraussetzung ist die Einwertigkeit der Attributwerte. Einwertigkeit bedeutet, dass für ein Attribut jeweils nur ein Wert zulässig ist.

8.3 Die 2. Normalform anwenden

Die 2. *Normalform* ist erfüllt, wenn die 1. Normalform gegeben ist und zudem jedes Nicht-Schlüsselattribut von einem eindeutigen Schlüssel abhängt. Jedes Nicht-Schlüsselattribut muss also von einem ganzheitlichen Schlüssel, der aus einem oder mehreren Attributen bestehen kann, ganzheitlich abhängen und nicht von einem Teil eines Gesamtschlüssels.

Die ganzheitliche Abhängigkeit von Attributen von einem ganzheitlichen Schlüssel wird auch als *voll funktionale Abhängigkeit* bezeichnet. Die voll funktionale Abhängigkeit kennen Sie aus der Mathematik: Sie haben bestimmt schon einmal $f(x) = y$ als

Funktion dargestellt, wobei $f(x)$ den Wert auf der y -Achse beschreibt. Jeder y -Wert ist also abhängig von einem x -Wert. Genauso verhält es sich mit den Nicht-Schlüsselattributen, die von einem gesamtheitlichen Schlüsselattribut abhängig sind.

ausid	adressid	name	vorname	strasse	nr.	plz	ort
1	1	Müller	Ralf	Mondstraße	8	50827	Köln
1	2	Müller	Ralf	Sternstraße	10	50127	Bonn
2	1	Lang	Peter	Sonnenstraße	1	50129	Bonn
2	2	Lang	Peter	Sternstraße	25	50126	Bonn

Tabelle 8.6 Eine Tabelle, die sich nicht in der 2. Normalform befindet

Tabelle 8.6. enthält einen Schlüssel, der aus zwei Attributen besteht. Es handelt sich dabei um die Attribute *ausid* (Auszubildender-ID) und *adressid* (Adressen-ID). Diese Gesamtschlüsselkombination ist der Tatsache geschuldet, dass jedem Auszubildenden mehrere Wohnorte zugeordnet werden können.

Stellen wir uns als Nächstes die Frage, welche Attribute voll vom Gesamtschlüssel abhängig sind. Die Attribute *name* und *vorname* sind abhängig vom Attribut *ausid*. Das Attribut *ausid* ist ein Teil des Gesamtschlüssels. Betrachten wir dann die Attribute *strasse*, *nr.*, *plz* und *ort*. Diese Attribute sind unmittelbar abhängig vom Attribut *adressid*. Hierbei handelt es sich allerdings nur um einen Teil des gesamten Schlüssels. Wir können also feststellen, dass es jeweils Attribute gibt, die nur von einem Teil des Gesamtschlüssels abhängig sind und nicht vom Gesamtschlüssel. Die Tabelle befindet sich also nicht in der 2. Normalform.

Noch ein weiterer Punkt wird für Probleme sorgen: Die fehlende Abhängigkeit von Nicht-Schlüsselattributen von einem Gesamtschlüssel wird zu redundanten Daten führen, denn die Informationen zu Auszubildenden, denen mehrere Adressen zugeordnet sind, werden redundant gespeichert. Wie können wir das ändern? Um alle Nicht-Schlüsselattribute in eine eindeutige Abhängigkeit von einem ganzheitlichen Schlüssel zu überführen, ist es erforderlich, die Tabelle in zwei neue Tabellen *auszubildender* und *adresse* aufzuteilen. Tabelle 8.7 enthält daher nur die Informationen über den Namen des Auszubildenden.

ausid	name	vorname
1	Müller	Ralf
2	Lang	Peter

Tabelle 8.7 Die Tabelle »auszubildender«

Exemplarisch sind hier die Auszubildenden Ralf Müller und Peter Lang in jeweils einer Zeile angegeben. Wichtig ist, dass die Tabelle über ein Primärschlüsselattribut *ausid* verfügt und somit die Attribute *name* und *vorname* von diesem ganzen Schlüssel abhängig sind.

adressid	strasse	nr.	plz	ort	ausid
1	Mondstraße	8	50827	Köln	1
2	Sternstraße	10	50127	Bonn	1
3	Sonnenstraße	1	50129	Bonn	2
4	Sternstraße	25	50126	Bonn	2

Tabelle 8.8 Die Tabelle »adresse« mit eindeutiger »adressid«

In Tabelle 8.8 findet sich ein Primärschlüsselattribut, diesmal mit der Bezeichnung *adressid*. Sämtliche Attribute sind jetzt von diesem ganzheitlichen Schlüssel abhängig. Entscheidend ist hier, dass noch ein weiteres Attribut *ausid* hinzugefügt wurde. Dieses Attribut referenziert auf die Spalte *ausid* in der Tabelle *auszubildender* und stellt somit eine Beziehung zwischen einer Adresse und einem Auszubildenden her. In Kapitel 7, »Datenbanken modellieren«, haben Sie diese Form der Beziehungsbildung bereits im Zusammenhang mit einer 1:n-Beziehung kennengelernt.

Die Redundanz aus Tabelle 8.6 ist somit nicht mehr existent. Auch Auszubildende mit mehreren Adressen können nun eindeutig identifiziert werden.



Zusammenfassung: 2. Normalform

Die 2. *Normalform* ist dann erfüllt, wenn alle *Nicht-Schlüsselattribute* von einem *ganzheitlichen Schlüssel* und nicht von einem Teil eines Schlüssels abhängig sind und außerdem die 1. *Normalform* erfüllt ist. Ein ganzheitlicher Schlüssel kann aus einem oder mehreren Attributen bestehen. Alle Nicht-Schlüsselattribute müssen eindeutig von diesem ganzheitlichen Schlüssel abhängig sein. Existieren hingegen Attribute, die nur von einem Teil des Schlüssels (der aus mehreren Attributen besteht) abhängig sind, so ist die 2. *Normalform* nicht erfüllt.

8.4 Die 3. Normalform anwenden

Die 3. *Normalform* ist erfüllt, wenn die ersten beiden Normalformen erfüllt sind und alle Attribute ausschließlich vom Gesamtschlüssel abhängig sind (wobei der Gesamtschlüssel aus einem oder mehreren Attributen bestehen kann). Damit ist gemeint,

dass ein Attribut sich nicht in einer *transitiven Abhängigkeit* von einem Nicht-Schlüsselattribut befinden darf. Eine transitive Abhängigkeit ist also gegeben, wenn ein Attribut auch von einem Nicht-Schlüsselattribut abhängig ist, das wiederum vom Gesamtschlüssel abhängig ist.

Tabelle 8.9 ist identisch mit Tabelle 8.8.

adressid	strasse	nr.	plz	ort	ausid
1	Mondstraße	8	50827	Köln	1
2	Sternstraße	10	50127	Bonn	1
3	Sonnenstraße	1	50129	Bonn	2
4	Sternstraße	25	50126	Bonn	2

Tabelle 8.9 Die Tabelle »adresse« in der 2. Normalform

Sie sehen, dass die drei Attribute *adressid*, *plz* und *ort* weiß hinterlegt sind. Das Attribut *ort* befindet sich in einer transitiven Abhängigkeit vom Schlüsselattribut *adressid* über das Nicht-Schlüsselattribut *plz*.

Hier bestehen also offensichtlich zwei Abhängigkeiten zum Schlüsselattribut *adressid*: Das Attribut *ort* ist direkt von dem Attribut *adressid* abhängig und zusätzlich über das Attribut *plz* transitiv vom Attribut *adressid* abhängig. Eine solche Abhängigkeit lässt sich auch wie folgt beschreiben: $adressid \rightarrow plz \rightarrow ort$.

Um Tabelle 8.9 in die 3. Normalform zu überführen, ist es zunächst erforderlich, das Attribut *ort* aus der Tabelle zu entfernen. Durch das Entfernen dieses Attributs ist die transitive Abhängigkeit aufgehoben, und es werden auch mögliche Redundanzen in Bezug auf den Ort ausgeschlossen.

adressid	strasse	nr.	plz	ausid
1	Mondstraße	8	50827	1
2	Sternstraße	10	50127	1
3	Sonnenstraße	1	50129	2
4	Sternstraße	25	50126	2

Tabelle 8.10 Die Tabelle »adresse« in der 3. Normalform

Um die Orte wieder den Postleitzahlen zuordnen zu können, müssen wir eine weitere Tabelle namens *plzort* einführen:

plz	ort
50827	Köln
50127	Bonn
50129	Bonn
50126	Bonn

Tabelle 8.11 Die Tabelle »plzort«

Tabelle 8.11 enthält neben der Spalte *ort* auch die Primärschlüsselspalte *plz*. Diesem eindeutigen Schlüssel sind die Orte zugeordnet. So stellt die Spalte *plz* aus der Tabelle *adresse* jetzt eine Fremdschlüsselbeziehung zu der Primärschlüsselspalte *plz* der Tabelle *plzort* her, um auf einen Ort zu verweisen, der der Postleitzahl zugeordnet ist.

Jetzt befinden sich die beiden Tabellen *adresse* und *plzort* in der 3. Normalform. Eine weitere transitive Abhängigkeit ist nicht mehr vorhanden.



Zusammenfassung: 3. Normalform

Die 3. Normalform ist erfüllt, wenn kein Nicht-Schlüsselattribut von einem anderen Nicht-Schlüsselattribut transitiv abhängt und zudem die erste und zweite Normalform erfüllt sind. »Transitiv abhängig« bedeutet, dass ein Nicht-Schlüsselattribut von einem anderen Nicht-Schlüsselattribut abhängt.

8.5 Denormalisierung

Die *Denormalisierung* befasst sich mit der Rückführung einer Normalisierung durch die drei Normalformen. Aber warum sollten wir ein Datenmodell, das wir durch die drei Normalformen optimiert und so von Redundanzen befreit haben, nun wieder auflösen, um uns wieder Redundanzen einzuhandeln?

Eine Motivation, ein Datenmodell zu denormalisieren, besteht darin, die Leistungsfähigkeit von Abfragen (also beispielsweise die Abfragedauer) zu erhöhen. Betrachten wir noch einmal kurz die drei Normalformen und welche Auswirkungen eine Denormalisierung hat.

Die 1. Normalform fordert von uns, dass eine Entität atomar aufgebaut ist. Attribute einer Entität dürfen also nicht teilbar sein. Die erste Normalform fordert auch, dass keine Mehrwertigkeit besteht. (Eine Mehrwertigkeit bestünde, wenn beispielsweise mehrere Telefonnummern einem Attribut *telefonkontakte* zugeordnet sind.) Die Nachteile einer Tabelle, die diesen Anforderungen nicht genügt, sind immens: Der

wichtigste Nachteil ist, dass Einfüge-, Änderungs- und Löschanomalien auftreten, die direkt aus der Nichteinhaltung der ersten Normalform erwachsen. Daher empfehle ich Ihnen dringend, die Prüfung auf die 1. Normalform stets durchzuführen und hier nie eine Denormalisierung vorzunehmen.

Die 2. Normalform fordert, dass ein Attribut ausschließlich von einem Gesamtschlüssel und nicht bloß von einem Teil eines Gesamtschlüssels abhängig ist. Die Auswirkungen der Rückführung der 2. Normalform sind in unserem Ausgangsbeispiel, dass sämtliche Informationen, die zum Auszubildenden gespeichert werden, mehrfach vorhanden sind – sie sind also redundant vorhanden. Daher treten auch hier Anomalien auf, wenn wir Einfüge-, Änderungs- und Löschoperationen durchführen. So müssen Sie eine komplette Zeile löschen, wenn Sie eine Adresse eines Auszubildenden mit mehreren Wohnorten entfernen möchten. Diese Löschoperation umfasst dann auch den Auszubildenden selbst. Und was passiert, wenn einem Auszubildenden nur ein Wohnort zugeordnet ist? Dann können Sie die Adresse überhaupt nicht löschen, ohne auch den Auszubildenden, dem die Adresse zugeordnet ist, zu entfernen. Es ist daher also auch nicht ratsam, die 2. Normalform zu denormalisieren.

Die 3. Normalform fordert von uns, dass kein Attribut transitiv von einem Schlüssel abhängig ist, und stellt so ebenfalls sicher, dass wir Redundanzen vermeiden. Die Sicherstellung der 3. Normalform ist nicht unbedingt erforderlich: Die transitive Abhängigkeit eines Attributs von einem Nicht-Schlüsselattribut kann durchaus hingenommen werden, weil sich dadurch die Komplexität eines Datenmodells mindert. Sie sparen sich also eine Schlüsselbeziehung zwischen zwei Tabellen. In [Kapitel 10](#) erfahren Sie, wie Sie Zeilen aus Tabellen verbunden abfragen können. Die Tabelle, deren transitive Abhängigkeit nicht aufgelöst wurde, hat dann natürlich den Vorteil, dass zumindest in diesem Fall keine Zeilen verbunden abgefragt werden müssen und somit die Abfrage schneller durchgeführt werden kann. In [Kapitel 17](#) zeige ich Ihnen, wie Sie Spaltenwerte gruppiert abfragen, um z. B. eine Summe der Spaltenwerte zu bilden. Auch hier kann es sinnvoll sein, eine Spalte einer Tabelle in der transitiven Abhängigkeit zu belassen. Diese Themen werden wir uns zu einem späteren Zeitpunkt ansehen. Wenn es aus fachlicher Sicht erforderlich ist, ein redundanzfreies Datenmodell zu schaffen, sollten Sie die 3. Normalform natürlich umsetzen. Hier kommt es also ganz auf die konkrete Aufgabenstellung an. Sie müssen für jede individuelle Anforderung entscheiden, ob es sinnvoll ist, die 3. Normalform umzusetzen.

Zusammenfassung: Normalformen denormalisieren

Das Auflösen von Normalformen wird als *Denormalisierung* bezeichnet. Eine Denormalisierung sollte nur nach einer intensiven Prüfung der fachlichen Anforderungen durchgeführt werden. Entitäten der 1. oder der 2. Normalform sollten keinesfalls einer Denormalisierung unterzogen werden. Für die 3. Normalform kann hingegen eine Denormalisierung sinnvoll sein. Dies hängt von den konkreten Anforderungen ab.



8.6 Übungen

Übung 1

Tabelle 8.12 enthält Spalten, die nicht atomar sind. Überführen Sie sie in eine Tabelle, die der 1. Normalform entspricht.

ausid	auszubildender	urlaubszeitraum
1	Müller Ralf	2016.08.01 bis 2016.08.25
2	Klein Sabine	2016.04.05 bis 2016.04.20

Tabelle 8.12 Tabelle, die nicht der 1. Normalform entspricht

Übung 2

Tabelle 8.13 enthält Spalten, die nicht von einem ganzheitlichen Schlüssel abhängen. Überführen Sie sie in zwei Tabellen, in denen jede Spalte einer Tabelle eindeutig vom Gesamtschlüssel abhängt. Berücksichtigen Sie, dass zwischen den beiden zu überführenden Tabellen Beziehungen existieren.

ausid	name	vorname	telid	vorwahl	durchwahl
1	Müller	Ralf	2	022x	999999999
2	Klein	Sabine	1	022x	888888888

Tabelle 8.13 Tabelle, die nicht der 2. Normalform entspricht

Übung 3

Tabelle 8.14 ist exakt identisch mit Tabelle 8.17, die in der Lösung zu Übung 2 dargestellt wird. In der Tabelle befindet sich eine Spalte in einer transitiven Abhängigkeit von einer Nicht-Schlüsselspalte. Lösen Sie diese Abhängigkeit auf, indem Sie eine weitere Tabelle einführen.

telid	vorwahl	durchwahl	fk_ausid
1	022x	999999999	2
2	022x	999999999	1

Tabelle 8.14 Tabelle, die nicht der 3. Normalform entspricht

Lösung zu Übung 1

Tabelle 8.15 zeigt die Lösung für die Überführung in die 1. Normalform. Jede Spalte der Tabelle ist atomar, d. h. nicht mehr teilbar.

ausid	name	vorname	beginn	ende
1	Müller	Ralf	2016.08.01	2016.08.25
2	Klein	Sabine	2016.04.05	2016.04.20

Tabelle 8.15 Tabelle, die der 1. Normalform entspricht

Lösung zu Übung 2

Wenn Ihre Lösung in etwa so aussieht wie in Tabelle 8.16 und Tabelle 8.17, dann haben Sie schon einiges richtig gemacht. In der Lösung sehen Sie, dass jede Spalte der beiden Tabellen eindeutig vom Gesamtschlüssel abhängt. Außerdem wurde die Beziehung zwischen den Zeilen der neuen Tabelle *telefon* und *auszubildender* berücksichtigt.

ausid	name	vorname
1	Müller	Ralf
2	Klein	Sabine

Tabelle 8.16 Tabelle »auszubildender«

telid	vorwahl	durchwahl	fk_ausid
1	022x	999999999	2
2	022x	999999999	1

Tabelle 8.17 Tabelle »telefon«

Lösung zu Übung 3

In Tabelle 8.18 und Tabelle 8.19 sehen Sie die Lösung für die Aufhebung der transitiven Abhängigkeit einer Durchwahl von der Vorwahl. Hierzu wurde die Spalte *vorwahl* in einer weiteren, gleichnamigen Tabelle *vorwahl* gespeichert, auf die aus der Tabelle *telefon* referenziert wird. In dieser Lösung sehen Sie sehr gut, dass die Überführung in die 3. Normalform weitere Redundanzen vermeidet.

telid	durchwahl	fk_vorwahlid	fk_ausid
1	999999999	1	2
2	999999999	1	1

Tabelle 8.18 Tabelle »telefon« ohne transitive Abhängigkeit

vorwahlid	vorwahl
1	022x
2	022x

Tabelle 8.19 Tabelle »vorwahl«

Kapitel 9

Datenmodelle in Tabellen überführen

Ein Modell ist eine abstrakte Darstellung der Realität, die es beschreibt. Ein Datenmodell und die darin enthaltenen Entitäten und Beziehungen müssen in Tabellen überführt werden, um das Modell praktisch nutzbar zu machen.

In diesem Kapitel erfahren Sie, wie Sie Tabellen anlegen, die in Beziehung zu anderen Tabellen stehen. Als Grundlage verwenden wir das Datenmodell aus [Kapitel 7](#), »[Datenbanken modellieren](#)«.

Da wir ein neues Datenmodell für die Ausbildungsdatenbank entwickelt haben, zeige ich Ihnen zuerst in [Abschnitt 9.1](#), wie Sie eine neue Datenbank mittels `CREATE DATABASE` anlegen. In [Abschnitt 9.2](#) erfahren Sie dann, wie Sie mit einer `CREATE TABLE`-Anweisung Tabellen erstellen, die zueinander in Beziehung stehen.

9.1 Die Ausbildungsdatenbank anlegen

Es gibt mehrere Wege, eine neue Datenbank anzulegen. Ich zeige Ihnen, wie Sie mit den hier behandelten SQL-Dialekten eine Datenbank anlegen.

Im SQL-Standard wird das Erstellen von Datenbanken nicht vorgegeben. Die Anweisung `CREATE DATABASE` zum Anlegen einer Datenbank wird jedoch von allen hier behandelten Datenbanken unterstützt. Der Datenbank soll bei der Erstellung gleich ein Zeichensatz zugeordnet werden. Dieser Vorgang unterscheidet sich bei den Datenbanken MySQL, MariaDB, PostgreSQL und MS SQL Server leider ein wenig. Wenn Sie nicht mehr genau wissen, was es mit den Zeichensätzen auf sich hat, so schlagen Sie in [Abschnitt 4.1.4](#) nach.

Notwendige Berechtigungen

Natürlich müssen Sie über die entsprechenden Berechtigungen (Administratorenberechtigung) verfügen, um eine neue Datenbank anzulegen. Wenn die Datenbankerstellung partout nicht klappen will, sind fehlende Rechte eine mögliche Fehlerquelle. Wenn Sie dieser Anleitung gefolgt sind, haben Sie jedoch bereits die notwendigen Rechte.



9.1.1 Eine neue Datenbank mit UTF-8-Zeichensatz anlegen (MySQL, MariaDB)

Wenn Sie eine MySQL- oder MariaDB-Datenbank verwenden, beginnen Sie, wie in [Listing 9.1](#) dargestellt, mit der **CREATE DATABASE**-Anweisung. Hinter der Anweisung geben Sie den Namen der zu erstellenden Datenbank an. Nach dem Datenbanknamen notieren Sie die Schlüsselwörter **DEFAULT CHARACTER SET**. Ganz am Ende geben Sie schließlich den zu verwendenden Zeichensatz an. Hier wird der Datenbank ein UTF-8-Zeichensatz zugeordnet.

```
CREATE DATABASE ausbildungsdatenbank
  DEFAULT CHARACTER SET utf8mb4;
```

Listing 9.1 Die Ausbildungsdatenbank anlegen (MySQL, MariaDB)

9.1.2 Eine neue Datenbank mit UTF-8-Zeichensatz anlegen (PostgreSQL)

Wie Sie in [Listing 9.2](#) sehen, beginnt auch die Erstellung einer PostgreSQL-Datenbank mit einer **CREATE DATABASE**-Anweisung und dem Namen der anzulegenden Datenbank. Hinter dem Namen notieren Sie das **ENCODING**, um ihr einen Zeichensatz zuzuordnen. Auf das Schlüsselwort **ENCODING** folgt der Zeichensatz, der von der Datenbank verwendet werden soll. Hier wird der Zeichensatz UTF-8 angegeben.

```
CREATE DATABASE ausbildungsdatenbank ENCODING 'UTF8';
```

Listing 9.2 Die Ausbildungsdatenbank anlegen (PostgreSQL)

9.1.3 Eine neue Datenbank mit Unicode-Zeichensatz anlegen (MS SQL Server)

Auch MS SQL Server beginnt, wie in [Listing 9.3](#) zu sehen ist, mit der **CREATE DATABASE**-Anweisung, um eine neue Datenbank anzulegen. Hier ist es nicht erforderlich, einen Zeichensatz anzugeben, da eine MS SQL Server-Datenbank bereits standardmäßig Unicode verwendet.

```
CREATE DATABASE ausbildungsdatenbank;
```

Listing 9.3 Die Ausbildungsdatenbank anlegen (MS SQL Server)



Zusammenfassung: CREATE DATABASE

Mit einer **CREATE DATABASE**-Anweisung erstellen Sie eine neue Datenbank. Für die hier eingesetzten Datenbanksysteme können Sie folgende **CREATE DATABASE**-Anweisung nutzen:

```
CREATE DATABASE namedb;
```


9.1.4 Übung

Als Nächstes erstellen wir eine Bibliotheksdatenbank für den Ausbildungsbetrieb. Legen Sie dazu eine neue Datenbank mit dem Namen *bibliothek* an. Bitte achten Sie darauf, dass die UTF-8-Kodierung beim Erstellen der Datenbank berücksichtigt wird.

Lösung zur Übung

MySQL und MariaDB:

```
CREATE DATABASE bibliothek  
  DEFAULT CHARACTER SET utf8mb4;
```

Listing 9.4 Eine neue Datenbank anlegen (MySQL, MariaDB)

PostgreSQL:

```
CREATE DATABASE bibliothek ENCODING 'UTF8';
```

Listing 9.5 Eine neue Datenbank anlegen (PostgreSQL)

MS SQL Server:

```
CREATE DATABASE bibliothek;
```

Listing 9.6 Eine neue Datenbank anlegen (MS SQL Server)

9.2 Tabellen mit Beziehungen zu anderen Tabellen erstellen

In diesem Abschnitt erfahren Sie, wie Sie Tabellen inklusive der Beziehungen zu anderen Tabellen anlegen und was Sie dabei beachten müssen. Um uns noch einmal einen Überblick über die zu erstellenden Tabellen zu verschaffen, sehen wir uns in [Abschnitt 9.2.1](#) zunächst ein Datenmodell in der UML-Notation an.

9.2.1 Die Ausbildungsdatenbank im Modell erfassen

[Abbildung 9.1](#) zeigt Ihnen ein Modell unserer Ausbildungsdatenbank in der UML-Notation.

Bis auf die 1:n-Beziehung der Entität *ausbildungsberuf* zur Entität *ausbildungsvertrag* kennen Sie sämtliche Beziehungen und Tabellen bereits aus [Kapitel 7](#), in dem wir uns ausführlich mit der Modellierung einer Ausbildungsdatenbank befasst haben.

In [Kapitel 8](#) habe ich Ihnen einen Unterschied zwischen Beziehungen dargestellt. Wir haben festgestellt, dass es zwei unterschiedliche Varianten von Beziehungen gibt: Zum einen die *Identifying Relationship* und zum anderen die *Non-identifying Rela-*

tionship. In [Abbildung 9.1](#) sind beide Typen enthalten. Beziehungen, die zum Identifying-Relationship-Typ gehören, werden durch eine durchgezogene Linie dargestellt. Beziehungen, die zum Typ Non-identifying-Relationship-Typ gehören, werden hingegen durch eine gestrichelte Linie abgebildet.

Folgende Beziehungen werden wir mithilfe einer `CREATE TABLE`-Anweisung erstellen:

- ▶ *ausbildungsvertrag* → *auszubildender*
- ▶ *ausbildungsvertrag* → *ausbildungsberuf*
- ▶ *adresse* → *auszubildender*
- ▶ *beruflehrfach* → *ausbildungsberuf*
- ▶ *beruflehrfach* → *lehrfach*
- ▶ *mitarbeiterausbildungsbetrieb* → *auszubildender*

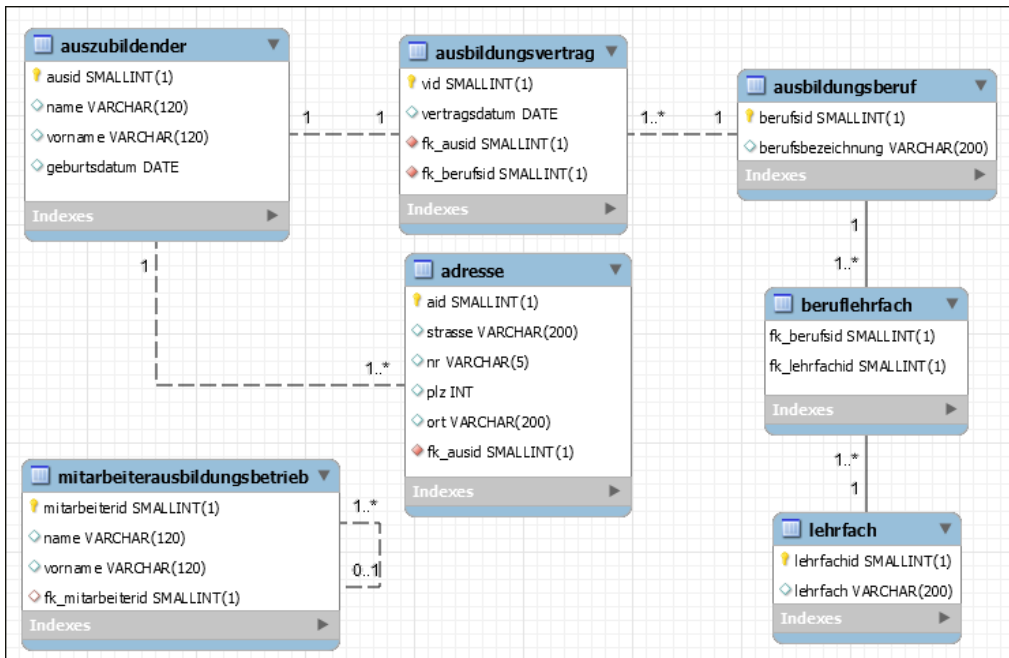


Abbildung 9.1 Die Ausbildungsdatenbank als Modell in der UML-Notation

9.2.2 Tabellen erstellen, die in einer 1:1-Beziehung stehen

Wir beginnen mit der modellierten Beziehung zwischen den Tabellen *auszubildender* und *ausbildungsvertrag*. Bitte achten Sie darauf, dass Sie für die Tabellen, die im Folgenden angelegt werden, die Ausbildungsdatenbank verwenden. Die Bibliotheksdatenbank aus [Listing 9.4](#) bis [Listing 9.6](#) verwenden Sie bitte für die Übungsaufgaben.

Als Erstes formulieren wir die **CREATE TABLE**-Anweisung für die Tabelle *auszubildender*, die eine starke Entität darstellt. Die Tabelle *auszubildender* verweist auf keine Primärschlüsselspalten anderer Tabellen. [Listing 9.7](#) enthält also nichts Neues für Sie. Eine **CREATE TABLE**-Anweisung in dieser Form haben Sie bereits in [Kapitel 4](#), »[Tabellen anlegen \(CREATE TABLE\)](#)«, kennengelernt.

```
CREATE TABLE auszubildender (
  ausid SMALLINT PRIMARY KEY NOT NULL,
  name VARCHAR(120),
  vorname VARCHAR(120),
  geburtsdatum DATE
);
```

Listing 9.7 Die Tabelle »auszubildender« mit einer **CREATE TABLE**-Anweisung anlegen

Als Nächstes formulieren wir die **CREATE TABLE**-Anweisung für die Tabelle *ausbildungsvertrag*. Die Tabelle *ausbildungsvertrag* befindet sich in einer 1:1-Beziehung zu der Tabelle *auszubildender*. Wie Sie diese 1:1-Beziehung in einer **CREATE TABLE**-Anweisung berücksichtigen, sehen Sie in [Listing 9.8](#):

```
CREATE TABLE ausbildungsvertrag (
  vid SMALLINT PRIMARY KEY NOT NULL,
  vertragsdatum DATE,
  fk_ausid SMALLINT,
  CONSTRAINT fk_auszubildender
    FOREIGN KEY (fk_ausid)
      REFERENCES auszubildender (ausid),
  CONSTRAINT uq_fk_ausid
    UNIQUE (fk_ausid)
);
```

Listing 9.8 Die Tabelle »ausbildungsvertrag« anlegen, die in einer 1:1-Beziehung zur Tabelle »auszubildender« steht

In [Abschnitt 4.5](#) haben Sie bereits Tabellen kennengelernt, denen Sie auf Tabellenebene Einschränkungen (**CONSTRAINTs**) zugeordnet haben. Hier formulieren wir in Zeile 5 den **CONSTRAINT** *fk_auszubildender*. Hinter der Bezeichnung folgt mit den Schlüsselwörtern **FOREIGN KEY** die Spalte der Tabelle *ausbildungsvertrag*, die einen fremden Schlüsselwert aufnehmen soll, nämlich den Primärschlüsselwert der Spalte *ausid* der Tabelle *auszubildender*.

Anschließend müssen wir festlegen, auf welche Tabelle und Spalte ausgehend von der Spalte *fk_ausid* verwiesen werden soll. Hierzu notieren Sie hinter der Definition der Fremdschlüsselspalte das Schlüsselwort **REFERENCES**. Damit legen Sie fest, auf welche Tabelle und Spalte die Fremdschlüsselspalte *fk_ausid* verweisen soll: Notieren Sie

hinter dem Schlüsselwort **REFERENCES** den Tabellennamen, gefolgt von einer geschlossenen Klammer, in der Sie die Spalte angeben, auf die die Fremdschlüsselspalte aus der Tabelle *ausbildungsvertrag* verweisen soll. Hier verweisen wir auf die Tabelle *auszubildender* und ihre Primärschlüsselspalte *ausid*, um eine Beziehung zwischen den Tabellen *auszubildender* und *ausbildungsvertrag* zu realisieren.

Die 1:1-Beziehung wird durch den **CONSTRAINT** `uq_fk_ausid` sichergestellt.

9.2.3 Tabellen erstellen, die in einer 1:n-Beziehung stehen

Im Modell aus [Abbildung 9.1](#) ist eine 1:n-Beziehung dargestellt. Es handelt sich um die Beziehung zwischen den Tabellen *auszubildender* und *adresse*. Jetzt fehlt nur noch die Tabelle *adresse*, in der die Beziehung zu der Tabelle *auszubildender* über einen **FOREIGN KEY**-Constraint festgelegt wird.

[Listing 9.9](#) zeigt Ihnen die **CREATE TABLE**-Anweisung für die Tabelle *adresse*. Ihnen fällt sicherlich auf, dass der **FOREIGN KEY**-Constraint genauso aufgebaut ist wie in der Tabelle *ausbildungsvertrag*.

Die Beziehung definieren wir, indem wir der Fremdschlüsselspalte *fk_ausid* einen **FOREIGN KEY**-Constraint zuweisen, der auf die Spalte *ausid* der Tabelle *auszubildender* verweist.

Für die Tabelle *adresse* wurde natürlich auch eine Primärschlüsselspalte festgelegt, und zwar die Spalte *aid* (Adress-ID), die auf Zeilenebene die Primärschlüssel-Einschränkung **PRIMARY KEY** zugeordnet bekommt.

```
CREATE TABLE adresse (
  aid SMALLINT PRIMARY KEY NOT NULL,
  strasse VARCHAR(200),
  nr VARCHAR(5),
  plz INT,
  ort VARCHAR(200),
  fk_ausid SMALLINT,
  CONSTRAINT fk_adresse_auszubildender
    FOREIGN KEY (fk_ausid)
    REFERENCES auszubildender (ausid)
);
```

Listing 9.9 Die Tabelle »adresse« in einer 1:n-Beziehung zu der Tabelle »auszubildender« erstellen

In [Abbildung 9.1](#) sehen Sie noch eine weitere 1:n-Beziehung, nämlich zwischen den Tabellen *ausbildungsberuf* und *ausbildungsvertrag*. Die Tabelle *ausbildungsberuf* stellt eine starke Entität dar. Um eine Beziehung zwischen den Tabellen *ausbildungsberuf* und *ausbildungsvertrag* über eine Fremdschlüsselbeziehung herstellen zu kön-

nen, ist es zunächst erforderlich, die Tabelle *ausbildungsberuf* zu erstellen, da Sie nicht aus der Tabelle *ausbildungsvertrag* mit einem **FOREIGN KEY**-Constraint auf eine Spalte einer Tabelle referenzieren können, die nicht existiert. In [Listing 9.10](#) ist die **CREATE TABLE**-Anweisung zu sehen, mit der wir die zur Beziehungsbildung benötigte Tabelle *ausbildungsberuf* erstellen:

```
CREATE TABLE ausbildungsberuf(
  berufsid SMALLINT PRIMARY KEY NOT NULL,
  berufsbezeichnung VARCHAR(200)
);
```

Listing 9.10 Die Tabelle »ausbildungsberuf« erstellen

Die Tabelle *ausbildungsvertrag* muss jetzt noch um eine weitere Fremdschlüsselspalte erweitert werden, die auf die Primärschlüsselspalte *berufsid* der Tabelle *ausbildungsberuf* verweist. Hierzu müssen wir zunächst noch eine weitere Fremdschlüsselspalte in die Tabelle *ausbildungsvertrag* aufnehmen.

In [Listing 9.11](#) wird die Spalte *fk_berufsid* zur Tabelle *ausbildungsvertrag* hinzugefügt, um auf die Primärschlüsselwerte der Spalte *berufsid* der Tabelle *ausbildungsberuf* zu verweisen. Wenn Sie die Tabelle *ausbildungsvertrag* bereits mit der **CREATE TABLE**-Anweisung aus [Listing 9.8](#) angelegt haben, so bitte ich Sie, zuerst die Tabelle mit einer einfachen **DROP TABLE**-Anweisung zu löschen.

```
CREATE TABLE ausbildungsvertrag (
  vid SMALLINT PRIMARY KEY NOT NULL,
  vertragsdatum DATE,
  fk_ausid SMALLINT,
  fk_berufsid SMALLINT,
  CONSTRAINT fk_auszubildender
    FOREIGN KEY (fk_ausid)
      REFERENCES auszubildender (ausid),
  CONSTRAINT fk_ausbildungsberuf
    FOREIGN KEY (fk_berufsid)
      REFERENCES ausbildungsberuf (berufsid),
  CONSTRAINT uq_fk_ausid
    UNIQUE (fk_ausid));
```

Listing 9.11 Die Tabelle »ausbildungsvertrag« um eine Fremdschlüsselbeziehung zur Tabelle »ausbildungsberuf« erweitern

Außerdem sehen Sie in [Listing 9.11](#) eine weitere **FOREIGN KEY**-Constraint-Definition, die festlegt, dass die hinzugefügte Fremdschlüsselspalte *fk_berufsid* der Tabelle *ausbildungsvertrag* auf die Spalte *berufsid* der Tabelle *ausbildungsberuf* verweist.

9.2.4 Tabellen erstellen, die in einer m:n-Beziehung stehen

In [Abbildung 9.1](#) sehen Sie eine m:n-Beziehung zwischen den Tabellen *ausbildungsberuf* und *lehrfach*. Die m:n-Beziehung zwischen den beiden Tabellen wird über die Schlüsseltabelle *beruflehrfach* hergestellt. In [Abschnitt 7.3](#) haben Sie bereits ein Beispiel für die zu modellierende m:n-Beziehung kennengelernt und dort erfahren, dass Sie die Zuordnung der Schlüsselwerte der Tabellen *ausbildungsberuf* und *lehrfach* über zwei Schlüsselspalten *berufid* und *lehrfachid* in der Tabelle *beruflehrfach* realisieren können. Um die Kombinationen zwischen den Schlüsselwerten in der Tabelle *beruflehrfach* eindeutig zu halten, werden beide Spalten als zusammengesetzter Primärschlüssel definiert.

Die einzelnen Spalten *berufid* und *lehrfachid* der Tabelle *beruflehrfach* stellen wiederum jeweils eine Beziehung zu den Tabellen *ausbildungsberuf* (Primärschlüsselspalte *berufid*) und *lehrfach* (Primärschlüsselspalte *lehrfachid*) her und repräsentieren somit jeweils eine Fremdschlüsselbeziehung.

Für die Tabelle *ausbildungsberuf* haben wir bereits in [Abschnitt 9.2.2](#) eine CREATE TABLE-Anweisung formuliert. Somit fehlen uns noch die CREATE TABLE-Anweisungen zum Erstellen der Tabellen *lehrfach* und *beruflehrfach*. Zunächst muss die Tabelle *lehrfach* angelegt werden, da es sich wie bei der Tabelle *ausbildungsberuf* um eine Tabelle handelt, auf die wir aus der noch zu erstellenden Tabelle *beruflehrfach* referenzieren wollen:

```
CREATE TABLE lehrfach (
    lehrfachid SMALLINT PRIMARY KEY NOT NULL,
    lehrfach VARCHAR(200)
);
```

Listing 9.12 CREATE TABLE-Anweisung zum Erstellen der Tabelle »lehrfach«

Die nächste zu erstellende Tabelle ist die Tabelle *beruflehrfach*. Sie soll sich mit den Fremdschlüsselspalten *fk_berufsid* und *fk_lehrfachid* auf die Primärschlüsselspalten *berufsid* und *lehrfachid* der Tabellen *ausbildungsberuf* und *lehrfach* beziehen. Das heißt für uns, dass wir eine Tabelle mit einem zusammengesetzten Primärschlüssel (eine sogenannte *Identifying Relationship*) und zwei Fremdschlüsselbeziehungen erstellen werden. In [Listing 9.13](#) sehen Sie die CREATE TABLE-Anweisung, die dies leistet:

```
CREATE TABLE beruflehrfach (
    fk_berufsid SMALLINT NOT NULL,
    fk_lehrfachid SMALLINT NOT NULL,
    PRIMARY KEY (fk_berufsid, fk_lehrfachid),
    CONSTRAINT fk_ausbildungsberuf2
    FOREIGN KEY (fk_berufsid)
    REFERENCES ausbildungsberuf (berufsid),
```

```

CONSTRAINT fk_lehrfach
  FOREIGN KEY (fk_lehrfachid)
    REFERENCES lehrfach (lehrfachid)
);

```

Listing 9.13 Die Tabelle »beruflehrfach« und ihre Beziehungen zu den Tabellen »ausbildungsberuf« und »lehrfach« mit einer CREATE TABLE-Anweisung erstellen

Eindeutige Bezeichnungen verwenden

Achten Sie darauf, dass der *Constraint*, der die Beziehung zur Tabelle *ausbildungsberuf* festlegt, den Namen *fk_ausbildungsberuf2* erhält. Diese Namensgebung geschieht mit Rücksicht auf die MySQL- und MariaDB-Datenbank, in der Namen für Constraints nur einmal vergeben werden dürfen. In [Abschnitt 9.2.2](#) haben wir bereits einen Constraint mit dem Namen *fk_ausbildungsberuf* benutzt. In dieser CREATE TABLE-Anweisung dürfen wir den Namen *fk_ausbildungsberuf* für den Constraint der Fremdschlüsseldefinition dann nicht mehr verwenden.

In einer PostgreSQL-Datenbank ist es nicht erforderlich, darauf zu achten, ob der Name eines CONSTRAINTs bereits in einer anderen Tabelle vergeben wurde. Sie könnten also mehrere Tabellen mit gleichen Fremdschlüssel-CONSTRAINT-Namen nutzen.

Eine MS SQL Server-Datenbank hingegen lässt ebenfalls keine mehrfach vorkommenden CONSTRAINT-Bezeichnungen zu. Also achten Sie auch hier darauf, unterschiedliche CONSTRAINT-Bezeichnungen zu vergeben.

9.2.5 Tabellen erstellen, die zu sich selbst in Beziehung stehen

Im Modell aus [Abbildung 9.1](#) ist die Tabelle *mitarbeiterausbildungsbetrieb* enthalten. Die Tabelle steht in einer 1:n-Beziehung zu sich selbst. Die 1:n-Beziehung resultiert aus der Tatsache, dass es sich bei einem Mitarbeiter um einen Vorgesetzten handeln kann, der wiederum in einer 1:n-Beziehung zu seinen Mitarbeitern stehen kann.

In einer CREATE TABLE-Anweisung realisieren wir diese Beziehung, indem wir von der Spalte *fk_mitarbeiterid* auf die Spalte *mitarbeiterid* der Tabelle *mitarbeiterausbildungsbetrieb* verweisen und somit eine Beziehung herstellen, die ausschließlich für die Tabelle *mitarbeiterausbildungsbetrieb* gilt. [Listing 9.14](#) zeigt diese Schritte:

```

CREATE TABLE mitarbeiterausbildungsbetrieb (
  mitarbeiterid SMALLINT PRIMARY KEY NOT NULL,
  name VARCHAR(120),
  vorname VARCHAR(120),
  fk_mitarbeiterid SMALLINT,
  CONSTRAINT fk_mitarbeiterausbildungsbetrieb
    FOREIGN KEY (fk_mitarbeiterid)

```

```
REFERENCES mitarbeiterausbildungsbetrieb (mitarbeiterid)
);
```

Listing 9.14 Die Tabelle »mitarbeiterausbildungsbetrieb« anlegen, die eine Fremdschlüsselbeziehung zu sich selbst eingeht



Zusammenfassung: Beziehungen zwischen Tabellen herstellen

Als Grundlage für Beziehungen ist stets ein Modell heranzuziehen, in dem die Beziehungen zwischen Tabellen festgelegt wurden.

Eine Beziehung wird grundsätzlich über einen **FOREIGN KEY**-Constraint festgelegt.

Eine 1:1-Beziehung zwischen Tabellen wird in der *Kindtabelle* festgelegt, die über einen **FOREIGN KEY**-Constraint auf die Elterntabelle referenziert.

Eine 1:n-Beziehung wird ebenfalls in der *Kindtabelle* über einen **FOREIGN KEY**-Constraint definiert, der auf die *Elterntabelle* referenziert.

Eine m:n-Beziehung zwischen zwei Tabellen wird über eine weitere Tabelle definiert, in der ausschließlich eindeutige Schlüsselpaare gespeichert werden. Diese referenzieren über **FOREIGN KEY**-Constraints auf die jeweiligen Elterntabellen.

Eine 1:1- oder 1:n-Beziehung einer Tabelle, die eine Beziehung mit sich selbst herstellt (*Self Join*), wird ebenfalls über einen **FOREIGN KEY**-Constraint realisiert. Die Tabelle stellt dann gleichzeitig die Eltern- und die Kindtabelle dar. Über eine Spalte, die als **FOREIGN KEY** festgelegt wird, wird auf die Primärschlüsselspalte referenziert und somit eine Beziehung hergestellt.

9.3 Übung

Zur Bibliotheksdatenbank gehören die folgenden vier Tabellen:

- ▶ *verlag*
- ▶ *fachbuch*
- ▶ *fachbereichfachbuch*
- ▶ *fachbereich*

Abbildung 9.2 zeigt das Datenbankmodell in der UML-Notation. Hier sind sämtliche Informationen enthalten, die Sie benötigen, um **CREATE TABLE**-Anweisungen zu formulieren, mit denen Sie die Tabellen in der Datenbank *bibliothek* anlegen. Formulieren Sie für diese Übung vier **CREATE TABLE**-Anweisungen, um die aufgelisteten Tabellen in der Datenbank *bibliothek* zu erstellen.

Wenn Sie die **CREATE TABLE**-Anweisungen formuliert haben, bitte ich Sie, vor der Ausführung der Anweisungen auf die Reihenfolge der Ausführung zu achten. Die Reihenfolge der Ausführung ist von den Beziehungen zwischen den Tabellen abhängig.

Ein kleiner Tipp zur Reihenfolge: Führen Sie zuerst die **CREATE TABLE**-Anweisungen für die Tabellen *verlag* und *fachbereich* aus. Es handelt sich um starke Entitäten, die nicht abhängig von einer Beziehung zu einer anderen Tabelle sind.

Wenn Sie diese beiden Tabellen angelegt haben, ist die Voraussetzung für die Tabelle *fachbuch* bereits gegeben. In der Tabelle ist eine Fremdschlüsselspalte definiert, die die Existenz der Tabelle *verlag* voraussetzt.

Zu guter Letzt legen Sie die Tabelle *fachbereichfachbuch* in der Datenbank an, die aufgrund der Fremdschlüsseldefinitionen wiederum die Existenz der Tabellen *fachbuch* und *fachbereich* voraussetzt.

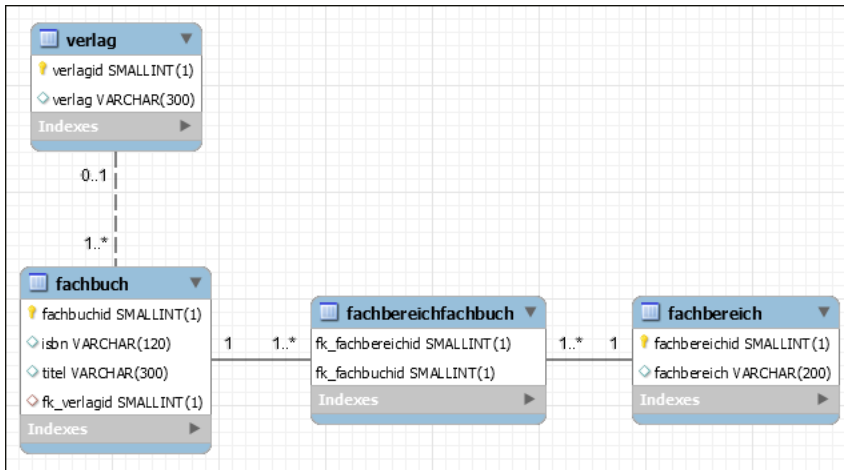


Abbildung 9.2 Die Bibliotheksdatenbank als Modell in der UML-Notation

Lösung zur Übung

```
CREATE TABLE verlag(
    verlagid SMALLINT PRIMARY KEY NOT NULL,
    verlag VARCHAR(300)
);
```

Listing 9.15 Die Tabelle »verlag« mit einer CREATE TABLE-Anweisung erstellen

```
CREATE TABLE fachbereich(
    fachbereichid SMALLINT PRIMARY KEY NOT NULL,
    fachbereich VARCHAR(200)
);
```

Listing 9.16 Die Tabelle »fachbereich« mit einer CREATE TABLE-Anweisung erstellen

```
CREATE TABLE fachbuch(  
    fachbuchid SMALLINT PRIMARY KEY NOT NULL,  
    isbn VARCHAR(120),  
    titel VARCHAR(300),  
    fk_verlagid SMALLINT,  
    CONSTRAINT fk_verlagid_verlag  
        FOREIGN KEY (fk_verlagid)  
        REFERENCES verlag (verlagid)  
);
```

Listing 9.17 Die Tabelle »fachbuch« mit einer CREATE TABLE-Anweisung erstellen

```
CREATE TABLE fachbereichfachbuch (  
    fk_fachbereichid SMALLINT,  
    fk_fachbuchid SMALLINT,  
    PRIMARY KEY (fk_fachbereichid, fk_fachbuchid),  
    CONSTRAINT fk_fachbereichid  
        FOREIGN KEY (fk_fachbereichid)  
        REFERENCES fachbereich (fachbereichid),  
    CONSTRAINT fk_fachbuchid  
        FOREIGN KEY (fk_fachbuchid)  
        REFERENCES fachbuch (fachbuchid)  
);
```

Listing 9.18 Die Tabelle »fachbereichfachbuch« mit einer CREATE TABLE-Anweisung erstellen

9.4 Die referenzielle Integrität verstehen

Relationale Datenbanken stellen die referenzielle Integrität sicher. Die *referenzielle Integrität* gewährleistet, dass Beziehungen zwischen Zeilen von Tabellen gültig sind. Diese Aussage hat es in sich!

In [Abbildung 9.3](#) sehen Sie die Tabellen *auszubildender* und *ausbildungsvertrag*. Die beiden Tabellen stehen in Beziehung zueinander. Es handelt sich um eine 1:1-Beziehung, die über eine Fremdschlüsselbeziehung der Tabelle *ausbildungsvertrag* realisiert wird. Die Fremdschlüsselbeziehung (**FOREIGN KEY**-Constraint) wurde in der **CREATE TABLE**-Anweisung festgelegt. Von der Fremdschlüsselspalte *fk_ausid* der Tabelle *ausbildungsvertrag* wird auf die Primärschlüsselspalte *ausid* der Tabelle *auszubildender* verwiesen.

Zum Zeitpunkt 1 sind keine Zeilen in den Tabellen *auszubildender* und *ausbildungsvertrag* enthalten.

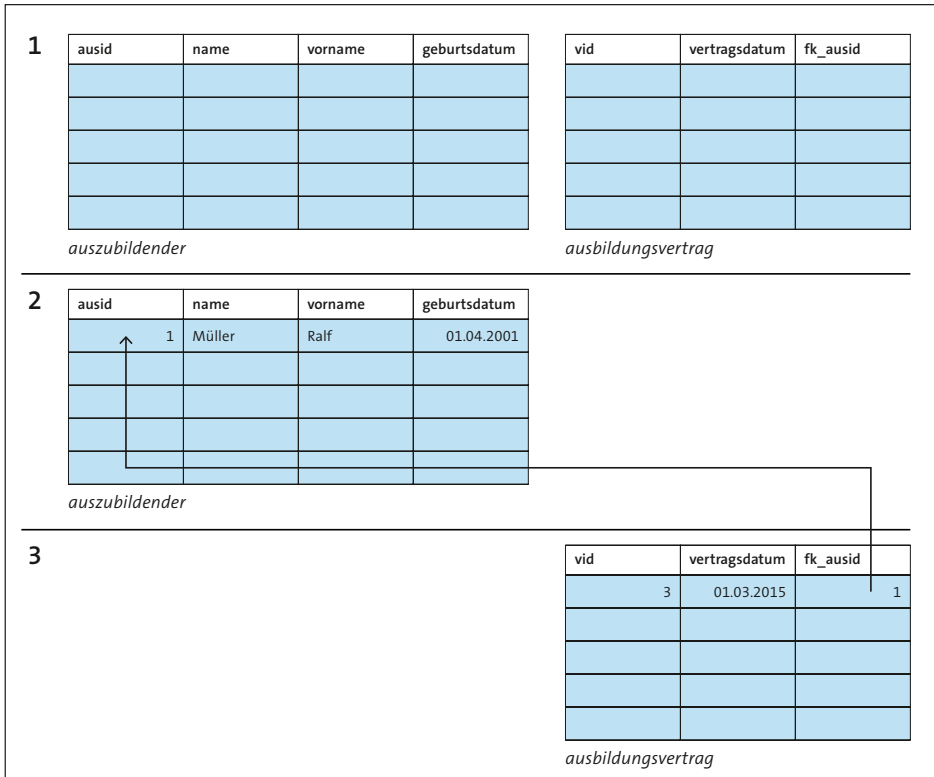


Abbildung 9.3 Zeilen in Tabellen einfügen, die in Beziehung zueinander stehen

Zum Zeitpunkt 2 wird durch eine neue Zeile ein Auszubildender in die Tabelle *auszubildender* eingetragen. Zu diesem Zeitpunkt gibt es noch keinen Ausbildungsvertrag, der auf den Primärschlüsselwert 1 der Spalte *ausid* der Tabelle *auszubildender* verweist. Es existiert also noch keine Beziehung zwischen einer Zeile der Tabelle *ausbildungsvertrag* und einer Zeile der Tabelle *auszubildender*.

Zum Zeitpunkt 3 wird eine neue Zeile in die Tabelle *ausbildungsvertrag* eingefügt, deren Fremdschlüsselwert 1 der Spalte *fk_ausid* auf den Primärschlüsselwert 1 der Spalte *ausid* der Tabelle *auszubildender* referenziert. Zu diesem Zeitpunkt wurde eine Beziehung zwischen diesen beiden Zeilen der Tabellen *auszubildender* und *ausbildungsvertrag* hergestellt.

In [Abbildung 9.4](#) sehen Sie ein Beispiel, das in einer relationalen Datenbank so nicht möglich wäre:

Zum Zeitpunkt 1 sind die Zeilen enthalten, die wir im vorigen Beispiel verwendet haben, um eine gültige Beziehung herzustellen.

Zum Zeitpunkt 2 sehen wir eine Zeile in der Tabelle *ausbildungsvertrag*, die in einer relationalen Datenbank so nicht eingefügt werden könnte, denn sie steht in keiner

gültigen Beziehung zu einer Zeile der Tabelle *auszubildender*. Ohne einen passenden Azubi kann es auch keinen Ausbildungsvertrag geben.

1

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001

auszubildender

vid	vertragsdatum	fk_ausid
3	01.03.2015	1

ausbildungsvertrag

2

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001

auszubildender

vid	vertragsdatum	fk_ausid
3	01.03.2015	1
4	01.03.2015	5

ausbildungsvertrag

←

Abbildung 9.4 Darstellung einer nicht gültigen Beziehung

Die Sicherstellung der referenziellen Integrität einer relationalen Datenbank verhindert, dass in die Tabelle *ausbildungsvertrag* eine Zeile eingefügt wird, die einen Fremdschlüsselwert enthält, der nicht als Primärschlüsselwert in einer Zeile der Tabelle *auszubildender* vorhanden ist. Konkret handelt es sich um den Fremdschlüsselwert 5 der Spalte *fk_ausid* der Tabelle *ausbildungsvertrag*.

Da es keine Zeile in der Tabelle *auszubildender* gibt, die in der Spalte *ausid* einen Primärschlüsselwert 5 enthält, kann diese Zeile aus der Tabelle *ausbildungsvertrag* auch keine gültige Beziehung zu einer Zeile in der Tabelle *auszubildender* herstellen.



Es gibt allerdings eine Ausnahme: NULL-Werte

Wenn die Fremdschlüsselspalte **NULL**-Werte zulässt, ist es möglich, Zeilen in die Tabelle *ausbildungsvertrag* einzufügen. Diese Zeile mit einem **NULL**-Wert in der Spalte *fk_ausid* stünde somit mit keiner Zeile in der Tabelle *auszubildender* in Beziehung. Die **CREATE TABLE**-Anweisung aus [Abschnitt 9.2.2](#) enthält keine **NOT NULL**-Einschränkung. In diesem Fall wäre es also möglich, eine Zeile ohne Fremdschlüsselwert einzufügen. Beachten Sie bitte, dass es sich bei der Beziehung zwischen den Tabellen *auszubildender* und *ausbildungsvertrag* um eine Non-identifying Relationship handelt. Der Fremdschlüsselspalte der Tabelle *ausbildungsvertrag* ist also nicht Bestandteil des Primärschlüssels der Tabelle *ausbildungsvertrag*.

Ist es auch möglich, Primärschlüsselwerte oder Fremdschlüsselwerte aus zwei Tabellen, die in Beziehung zueinander stehen, zu ändern?

Grundsätzlich ist es zulässig, Fremdschlüsselwerte zu ändern. Wenn Sie einen Fremdschlüsselwert ändern, muss der neue Fremdschlüsselwert natürlich auch als Primärschlüsselwert in der Tabelle vorhanden sein, auf die der Fremdschlüssel referenziert. Wie beim Einfügen können Sie auch hier bei einer Aktualisierungsoperation nur auf gültige Primärschlüsselwerte verweisen. Mit der Änderung eines Fremdschlüsselwertes wird also lediglich eine neue gültige Beziehung definiert.

Einen Primärschlüsselwert, auf den sich Fremdschlüsselwerte aus Zeilen einer anderen Tabelle beziehen, können Sie nicht so einfach ändern. Solange sich keine Zeile mit einem Fremdschlüsselwert aus einer anderen Tabelle auf einen zu ändernden Primärschlüsselwert bezieht, ist es möglich, einen Primärschlüsselwert zu ändern. Sobald hier jedoch Beziehungen existieren, ist es nicht mehr möglich, Primärschlüsselwerte zu ändern.

Es gibt nur eine Ausnahme: Sie müssen bereits dann, wenn Sie die Tabelle durch die **CREATE TABLE**-Anweisung erstellen, in der **FOREIGN KEY**-Definition festlegen, dass bei einer Aktualisierung auch der Fremdschlüsselwert der Tabelle geändert werden kann. Eine derart geänderte Fremdschlüsseldefinition muss die **ON UPDATE CASCADE**-Klausel enthalten. Auf die **ON UPDATE CASCADE**-Klausel gehe ich in [Abschnitt 10.3.2](#) ein.

Als Nächstes prüfen wir, ob wir Zeilen aus zwei Tabellen, die in Beziehung zueinander stehen, löschen können. Hierbei handelt es sich um die Zeilen, deren Fremdschlüsselwerte abhängig von Primärschlüsselwerten aus einer anderen Tabelle sind.

1	ausid	name	vorname	geburtsdatum	vid	vertragsdatum	fk_ausid
	1	Müller	Ralf	01.04.2001	3	01.03.2015	1
<i>auszubildender</i>				<i>ausbildungsvertrag</i>			
2	ausid	name	vorname	geburtsdatum	vid	vertragsdatum	fk_ausid
	1	Müller	Ralf	01.04.2001	3	01.03.2015	1
<i>auszubildender</i>				<i>ausbildungsvertrag</i>			

Abbildung 9.5 Eine Zeile aus einer Tabelle löschen, die über einen Fremdschlüsselwert in Beziehung zu einer anderen Tabelle steht

Zunächst betrachten wir dazu wieder zwei Zeitpunkte, die in Abbildung 9.5 dargestellt sind.

Zum Zeitpunkt 1 sind in den Tabellen *auszubildender* und *ausbildungsvertrag* Zeilen enthalten, die unmittelbar in Beziehung zueinander stehen.

Zum Zeitpunkt 2 wurde die Zeile aus der Tabelle *ausbildungsvertrag* entfernt, die mit einem Fremdschlüsselwert auf einen Primärschlüsselwert einer Zeile in der Tabelle *auszubildender* verwies. Sie können jederzeit aus einer Tabelle eine Zeile löschen, die über einen Fremdschlüsselwert von einem Primärschlüsselwert einer anderen Tabelle abhängt.

Jetzt kehren wir den Vorgang um und prüfen, ob es möglich ist, Zeilen mit Primärschlüsselwerten einer Tabelle zu löschen, auf die Zeilen mit Fremdschlüsselwerten einer anderen Tabelle verweisen. Wir betrachten in Abbildung 9.6 wieder zwei Zeitpunkte.

1

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	01.04.2001

auszubildender

vid	vertragsdatum	fk_ausid
3	01.03.2015	1

ausbildungsvertrag

2

ausid	name	vorname	geburtsdatum

auszubildender

vid	vertragsdatum	fk_ausid
3	01.03.2015	1

ausbildungsvertrag

Abbildung 9.6 Eine Zeile aus einer Tabelle löschen, auf die über einen Fremdschlüsselwert aus einer anderen Tabelle referenziert wird

Zum Zeitpunkt 1 existieren jeweils Zeilen in der Tabelle *auszubildender* und *ausbildungsvertrag*, die über einen Schlüsselwert in Beziehung zueinander stehen. In der Tabelle *ausbildungsvertrag* referenziert der Fremdschlüsselwert 1 der Spalte *fk_ausid* auf den Primärschlüsselwert 1 der Spalte *ausid*.

Zum Zeitpunkt 2 wurde aus der Tabelle *auszubildender* die Zeile gelöscht, auf die aus der Tabelle *ausbildungsvertrag* referenziert wird. Die referenzielle Integrität einer relationalen Datenbank stellt sicher, dass genau so ein Löschvorgang nicht möglich ist. Solange Fremdschlüsselwerte aus Zeilen einer Tabelle auf Primärschlüsselwerte einer

anderen Tabelle referenzieren, ist es nicht möglich, die Zeilen zu löschen, zu denen Beziehungen bestehen.

Erst dann, wenn keine Fremdschlüsselwerte mehr auf die Primärschlüsselwerte einer Zeile referenzieren, kann diese Zeile gelöscht werden.

Zusammenfassung: Referenzielle Integrität

Bei Einfüge-, Lösch- und Aktualisierungsanweisungen stellt eine Datenbank stets sicher, dass die referenzielle Integrität gewahrt bleibt:

- ▶ In eine Elterntabelle können Zeilen ohne Rücksicht auf Abhängigkeiten zu einer Kindtabelle eingefügt werden.
- ▶ In einer Kindtabelle müssen einzufügende Zeilen auf die Zeilen einer Elterntabelle referenzieren, zu der sie in Beziehung stehen. Hier gibt es eine Ausnahme: Eine Zeile ohne definierten Fremdschlüsselwert (**NULL**) kann jederzeit eingefügt werden.
- ▶ Die Aktualisierung von Primärschlüsselwerten in einer Elterntabelle setzt voraus, dass entweder keine Beziehung zu einer Zeile in einer Kindtabelle existiert oder dass der Fremdschlüsselwert in der Zeile der Kindtabelle automatisch mit aktualisiert wird.
- ▶ Die Aktualisierung von Fremdschlüsselwerten in einer Kindtabelle ist nur dann möglich, wenn eine andere gültige Beziehung mit einem Fremdschlüsselwert hergestellt wird.
- ▶ Eine Zeile einer Elterntabelle kann nur dann gelöscht werden, wenn keine Zeile aus einer Kindtabelle mehr existiert, die auf die Zeile einer Elterntabelle referenziert. Wenn eine automatische Löschung der Zeilen einer Kindtabelle festgelegt wurde, ist das Löschen einer Zeile einer Elterntabelle auch möglich, wenn Zeilen einer Kindtabelle auf die Zeile der Elterntabelle referenzieren.
- ▶ Eine Zeile einer Kindtabelle, die auf eine Zeile einer Elterntabelle referenziert, können Sie löschen. Mit dem Löschen der Zeile existiert die Beziehung schlicht und ergreifend nicht mehr.



Kapitel 10

Operationen auf Tabellen in Beziehungen anwenden

Ein relationales Datenbanksystem stellt sicher, dass die Beziehungen zwischen den Tabellen auch stets berücksichtigt werden. Wie wir uns dies zunutze machen, sehen Sie in diesem Kapitel.

In diesem Kapitel führen wir das Beispiel aus Kapitel 9, »Datenmodelle in Tabellen überführen«, fort und beschäftigen uns weiter mit der Ausbildungsdatenbank.

10.1 Zeilen in Tabellen einfügen, die in Beziehung zueinander stehen

Um Abhängigkeiten zwischen Tabellen zu beschreiben, verwenden wir auch hier die vereinfachte Ausdrucksweise der Eltern-Kind-Beziehung. Tabellen, die sich über eine *Fremdschlüsseldefinition* in der **CREATE TABLE**-Anweisung in Abhängigkeit zu einer anderen Tabelle befinden, bezeichnen wir als *Kindtabellen*. Tabellen, die sich nicht in einer solchen Abhängigkeit befinden, bezeichnen wir als *Elterntabellen*. In diesem Fall wären also die Tabellen *auszubildender* und *ausbildungsberuf* Beispiele für Elterntabellen, während *adresse* eine Kindtabelle darstellt, die in Beziehung zu einer anderen Tabelle steht.

10.1.1 Zeilen in die Tabelle »auszubildender« einfügen

Als Erstes fügen wir Zeilen in die Tabelle *auszubildender* ein. Die Tabelle *auszubildender* hat keinerlei Abhängigkeiten (*Foreign Key Constraints*), die wir beachten müssen, wenn wir neue Zeilen einfügen. Es ist lediglich darauf zu achten, dass für jeden Auszubildenden ein eindeutiger Primärschlüsselwert in der **INSERT**-Anweisung notiert wird. Listing 10.1 enthält sechs **INSERT**-Anweisungen, mit denen wir sechs neue Auszubildende in die Tabelle *auszubildender* eintragen:

```
INSERT INTO auszubildender (ausid,name,vorname,geburtsdatum)
VALUES (1,'Müller','Ralf','2001-04-01');
INSERT INTO auszubildender (ausid,name,vorname,geburtsdatum)
VALUES (2,'Klein','Sabine','2002-05-10');
```

```
INSERT INTO auszubildender (ausid,name,vorname,geburtsdatum)
VALUES (3,'Lang','Peter','2001-03-11');
INSERT INTO auszubildender (ausid,name,vorname,geburtsdatum)
VALUES (4,'Berg','Frank','2002-07-20');
INSERT INTO auszubildender (ausid,name,vorname,geburtsdatum)
VALUES (5,'Erde','Sabine','2001-01-23');
INSERT INTO auszubildender (ausid,name,vorname,geburtsdatum)
VALUES (6,'Grün','Justus','2001-04-15');
```

Listing 10.1 Zeilen in die Tabelle »auszubildender« einfügen

10.1.2 Zeilen in die Tabelle »ausbildungsberuf« einfügen

Als Nächstes fügen wir Zeilen in die Tabelle *ausbildungsberuf* ein. Die Tabelle *ausbildungsberuf* verfügt ebenfalls über keinerlei Abhängigkeiten (*Foreign Constraints*), die wir beachten müssen. Auch hier ist lediglich darauf zu achten, dass für jeden Ausbildungsberuf ein eindeutiger Primärschlüsselwert in der **INSERT**-Anweisung eingetragen wird. [Listing 10.2](#) enthält sechs **INSERT**-Anweisungen, mit denen wir sechs neue Ausbildungsberufe in die Tabelle *ausbildungsberuf* eintragen:

```
INSERT INTO ausbildungsberuf (berufsid,berufsbezeichnung)
VALUES (1,'Energieelektroniker');
INSERT INTO ausbildungsberuf (berufsid,berufsbezeichnung)
VALUES (2,'Mechatroniker');
INSERT INTO ausbildungsberuf (berufsid,berufsbezeichnung)
VALUES (3,'Buchhalter');
INSERT INTO ausbildungsberuf (berufsid,berufsbezeichnung)
VALUES (4,'Industriekaufmann');
INSERT INTO ausbildungsberuf (berufsid,berufsbezeichnung)
VALUES (5,'Schlosser');
INSERT INTO ausbildungsberuf (berufsid,berufsbezeichnung)
VALUES (6,'Elektriker');
```

Listing 10.2 Zeilen in die Tabelle »ausbildungsberuf« einfügen

10.1.3 Zeilen in die Tabelle »lehrfach« einfügen

Bei der Tabelle *lehrfach* handelt es sich ebenfalls um eine nicht abhängige Tabelle. Wir können also neue Zeilen in die Tabelle *lehrfach* einfügen, ohne auf irgendwelche Abhängigkeiten (*Foreign Key Constraints*) achten zu müssen. Es ist nur wichtig, dass eindeutige Primärschlüsselwerte eingetragen werden. [Listing 10.3](#) enthält zehn **INSERT**-Anweisungen, mit denen wir zehn neue Lehrfächer in die Tabelle *lehrfach* eintragen:

```
INSERT INTO lehrfach (lehrfachid,lehrfach)
VALUES (1,'Mathematik');
```

```

INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (2, 'Buchhaltung 1');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (3, 'Buchhaltung 2');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (4, 'Mechanik Grundlagen 1');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (5, 'Mechanik Grundlagen 2');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (6, 'Englisch');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (7, 'Elektronik Grundlagen 1');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (8, 'Elektronik Grundlagen 2');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (9, 'Rechnungsbearbeitung 1');
INSERT INTO lehrfach (lehrfachid, lehrfach)
VALUES (10, 'Rechnungsbearbeitung 2');

```

Listing 10.3 Zeilen in die Tabelle »lehrfach« einfügen

10.1.4 Zeilen in die Tabelle »adresse« (inklusive der Beziehungen) einfügen

Die Tabelle *adresse* befindet sich in einer 1:n-Abhängigkeit zu der Tabelle *auszubildender*. Neue Zeilen, die wir hier mit einer **INSERT**-Anweisung eintragen, müssen also auf gültige Primärschlüsselwerte in der Tabelle *auszubildender* verweisen. Die sechs **INSERT**-Anweisungen in Listing 10.4 sorgen dafür, dass sechs Adressen in die Tabelle *adresse* eingetragen werden. Die ersten drei **INSERT**-Anweisungen werden mit den Fremdschlüsselwerten 1, 3 und 5 ausgestattet und verweisen somit in der Tabelle *auszubildender* auf die Zeilen mit den Primärschlüsselwerten 1, 3 und 5. Die letzten drei **INSERT**-Anweisungen enthalten keine Fremdschlüsselwerte. Hier wurde anstelle eines Fremdschlüsselwerts der nicht definierte Wert **NULL** eingetragen. Die letzten drei Zeilen verweisen also auf keine Auszubildenden. Es handelt sich stattdessen um Adressen, die nicht zugeordnet sind.

```

/* Mit Zuordnung zu Auszubildenden */
INSERT INTO adresse (aid, strasse, nr, plz, ort, fk_ausid)
VALUES (1, 'Mondstraße', '8', 50827, 'Köln', 1);
INSERT INTO adresse (aid, strasse, nr, plz, ort, fk_ausid)
VALUES (2, 'Sternstraße', '10', 50127, 'Bonn', 3);
INSERT INTO adresse (aid, strasse, nr, plz, ort, fk_ausid)
VALUES (3, 'Sonnenstraße', '1', 50129, 'Bonn', 5);

```

```
/*Ohne Zuordnung zu Auszubildenden*/  
INSERT INTO adresse (aid,strasse,nr,plz,ort,fk_ausid)  
VALUES (4,'Jupiterstraße','11',50827,'Köln',NULL);  
INSERT INTO adresse (aid,strasse,nr,plz,ort,fk_ausid)  
VALUES (5,'Uranusstraße','9',50127,'Bonn',NULL);  
INSERT INTO adresse (aid,strasse,nr,plz,ort,fk_ausid)  
VALUES (6,'Marsstraße','9',50129,'Bonn',NULL);
```

Listing 10.4 Neue Zeilen in die Tabelle »adresse« einfügen

10.1.5 Zeilen in die Tabelle »ausbildungsvertrag« (inklusive der Beziehungen) einfügen

Die Tabelle *ausbildungsvertrag* steht jeweils in einer 1:1-Beziehung zu den Tabellen *auszubildender* und *ausbildungsberuf*. Neue Zeilen, die wir mit einer **INSERT**-Anweisung eintragen, müssen also auf gültige Primärschlüsselwerte der Tabellen *auszubildender* und *ausbildungsberuf* verweisen. Listing 10.5 enthält sechs **INSERT**-Anweisungen. Die letzten beiden Werte, die in der **VALUES**-Klausel enthalten sind, verweisen mit ihren Fremdschlüsselwerten der Spalten *fk_ausid* und *fk_berufsid* auf die Primärschlüsselwerte der Tabellen *auszubildender* (Primärschlüsselspalte: *ausid*) und *ausbildungsberuf* (Primärschlüsselspalte: *berufsid*). Jede Zeile, die wir hier einfügen, enthält also eine Beziehung zu den Zeilen in den Tabellen *auszubildender* und *ausbildungsberuf*.

```
INSERT INTO ausbildungsvertrag (vid,vertragsdatum,fk_ausid,fk_berufsid)  
VALUES (1,'2015-06-01',5,2);  
INSERT INTO ausbildungsvertrag (vid,vertragsdatum,fk_ausid,fk_berufsid)  
VALUES (2,'2015-06-01',4,4);  
INSERT INTO ausbildungsvertrag (vid,vertragsdatum,fk_ausid,fk_berufsid)  
VALUES (3,'2015-06-01',1,3);  
INSERT INTO ausbildungsvertrag (vid,vertragsdatum,fk_ausid,fk_berufsid)  
VALUES (4,'2015-06-01',3,1);  
INSERT INTO ausbildungsvertrag (vid,vertragsdatum,fk_ausid,fk_berufsid)  
VALUES (5,'2015-06-01',6,1);  
INSERT INTO ausbildungsvertrag (vid,vertragsdatum,fk_ausid,fk_berufsid)  
VALUES (6,'2015-06-01',2,5);
```

Listing 10.5 Zeilen in die Tabelle »ausbildungsvertrag« einfügen

10.1.6 Zeilen in die Tabelle »beruflehrfach« (inklusive der Beziehungen) einfügen

Die Tabelle *beruflehrfach* bildet eine m:n-Beziehung zwischen den Tabellen *ausbildungsberuf* und *lehrfach* ab. In der Tabelle *beruflehrfach* werden Kombinationen von

Fremdschlüsselwerten gespeichert, die jeweils auf die Primärschlüsselwerte der Tabellen *ausbildungsberuf* und *lehrfach* verweisen.

Die Fremdschlüsselspalte *fk_berufsid* der Tabelle *beruflehrfach* verweist auf die Primärschlüsselspalte *berufsid* der Tabelle *ausbildungsberuf*. Die Fremdschlüsselspalte *fk_lehrfachid* wiederum verweist auf die Primärschlüsselspalte *lehrfachid* der Tabelle *lehrfach*. In Listing 10.6 sehen Sie zwölf **INSERT**-Anweisungen, mit denen Kombinationen von Fremdschlüsselwerten gespeichert werden, um entsprechende Beziehungen zu den Zeilen der Tabellen *ausbildungsberuf* und *lehrfach* herzustellen:

```
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (1,1);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (1,6);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (1,7);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (1,8);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (4,2);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (4,3);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (4,6);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (4,9);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (6,1);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (6,4);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (6,7);
INSERT INTO beruflehrfach (fk_berufsid,fk_lehrfachid)
VALUES (6,8);
```

Listing 10.6 Zeilen in die Tabelle »beruflehrfach« einfügen

10.1.7 Zeilen in die Tabelle »mitarbeiterausbildungsbetrieb« (inklusive der Beziehungen) einfügen

Die Tabelle *mitarbeiterausbildungsbetrieb* bildet eine 1:n-Beziehung mit sich selbst. Die Tabelle enthält eine Fremdschlüsselspalte *fk_mitarbeiterid*, die auf die Primärschlüsselspalte *mitarbeiterid* referenziert.

Listing 10.7 enthält zehn **INSERT**-Anweisungen, in denen jeweils Werte der Spalte *fk_mitarbeiterid* auf Werte der Spalte *mitarbeiterid* verweisen:

```
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (1,'Müller','Alfred',NULL);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (2,'Ungern','Peter',1);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (3,'Erdenschein','Claudia',1);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (4,'Sternenschein','Ute',1);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (5,'Augustus','Frank',1);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (6,'Erdenfels','Christine',NULL);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (7,'Hoffnung','Ralf',6);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (8,'Freud','Erika',6);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (9,'Bergfels','Diether',6);
INSERT INTO mitarbeiterausbildungsbetrieb (mitarbeiterid,name,vorname,
fk_mitarbeiterid)
VALUES (10,'Lemon','Reinhold',6);
```

Listing 10.7 Zeilen in die Tabelle »mitarbeiterausbildungsbetrieb« einfügen



Zusammenfassung: Zeilen, die in Beziehung stehen, in Tabellen einfügen

Wenn Zeilen in *Kindtabellen*, die in Beziehung zu Zeilen aus *Elterntabellen* stehen, eingefügt werden sollen, gilt es, eine Reihenfolge einzuhalten.

Zuerst werden die Zeilen in eine Elterntabelle eingefügt. Dann werden die Zeilen mit den entsprechenden *Fremdschlüsselwerten* in Kindtabellen eingefügt.

10.1.8 Übungen

Übung 1

In Kapitel 9, »Datenmodelle in Tabellen überführen«, haben Sie Tabellen erstellt, die Informationen für eine Bibliothek verwalten sollen. In dieser Übung sollen für die Tabellen *fachbereich* und *verlag*, die starke Entitäten darstellen, INSERT-Anweisungen formuliert werden, mit denen Sie Zeilen in die Tabellen einfügen. Die Tabellen sind also autark. Sie müssen hier keine Beziehungen zu Zeilen in anderen Tabellen beachten. In Tabelle 10.1 sehen Sie Zeilen, die für die Tabelle *verlag* vorgesehen sind. Tabelle 10.2 enthält wiederum Zeilen, die für die Tabelle *fachbereich* bestimmt sind.

verlagid	verlag
1	Rheinwerk Verlag
2	Elektro Verlag
3	Mechanik Verlag
4	Kaufmann Verlag
5	Medien Verlag

Tabelle 10.1 Zeilen für die Tabelle »verlag«

fachbereichid	fachbereich
1	Elektrotechnik
2	Kaufmann
3	Mechanik
4	Pneumatik

Tabelle 10.2 Zeilen für die Tabelle »fachbereich«

Übung 2

Fügen Sie Zeilen in die Tabelle *fachbuch* ein, die mit einem Fremdschlüsselwert auf Zeilen in der Tabelle *verlag* referenzieren. In Übung 1 haben Sie fünf Verlage in die Tabelle *verlag* eingefügt. Uns stehen also fünf Primärschlüsselwerte aus der Tabelle *verlag* zur Verfügung, die wir als Fremdschlüsselwerte in der Tabelle *fachbuch* verwenden können, je nachdem, von welchem Verlag ein Fachbuch verlegt wird. Beachten Sie, dass die Fremdschlüsselwerte in die Spalte *fk_verlagid* eingefügt werden. Diese

Fremdschlüsselwerte referenzieren auf Zeilen, die Sie in [Tabelle 10.3](#) für **INSERT**-Anweisungen für die Tabelle *verlag* genutzt haben. Die letzten drei Zeilen mit den Primärschlüsselwerten 7, 8 und 9 der Spalte *fachbuchid* verfügen über keine Referenz, die über Fremdschlüsselwerte festgelegt wurde. Tragen Sie hier jeweils **NULL** in die **VALUES**-Klausel der **INSERT**-Anweisung ein.

fachbuchid	isbn	titel	fk_verlagid
1	1235	Mechanik	3
2	9878	Elektrotechnik	2
3	2323	Elektronik	2
4	2254	Pneumatik	3
5	4455	Mathematik Grundlagen 1	2
6	4456	Mathematik Grundlagen 2	2
7	5566	Mengenlehre	NULL
8	7766	Kommunikation 1	NULL
9	7767	Kommunikation 2	NULL

Tabelle 10.3 Zeilen für die Tabelle »fachbuch«

Übung 3

Jetzt fehlt uns noch die Zuordnung eines Fachbuchs zu einem Fachbereich. Diese Zuordnung können Sie mit der Schlüsseltabelle *fachbereichfachbuch* realisieren. Hier müssen jeweils gültige Primärschlüsselwerte oder Kombinationen aus den Tabellen *fachbuch* und *fachbereich* eingetragen werden, um die Beziehung eines Schlüsselpaars zu den Zeilen der Tabellen *fachbuch* bzw. *fachbereich* herzustellen.

In [Tabelle 10.4](#) sehen Sie die Zuordnungen über die Fremdschlüsselwerte eines Fachbuchs zu einem Fachbereich. Formulieren Sie **INSERT**-Anweisungen, mit denen Sie die jeweiligen Zuordnungen in die Tabelle *fachbereichfachbuch* eintragen.

fk_fachbereichid	fk_fachbuchid
3	1
1	3
4	4

Tabelle 10.4 Zeilen für die Tabelle »fachbereichfachbuch«

fk_fachbereichid	fk_fachbuchid
1	5
1	6
1	7
2	8
2	9

Tabelle 10.4 Zeilen für die Tabelle »fachbereichfachbuch« (Forts.)

Lösung zu Übung 1

```

/* Zeilen in die Tabelle verlag einfügen */
INSERT INTO verlag (verlagid,verlag)
VALUES (1, 'Rheinwerk Verlag');
INSERT INTO verlag (verlagid,verlag)
VALUES (2, 'Elektro Verlag');
INSERT INTO verlag (verlagid,verlag)
VALUES (3, 'Mechanik Verlag');
INSERT INTO verlag (verlagid,verlag)
VALUES (4, 'Kaufmann Verlag');
INSERT INTO verlag (verlagid,verlag)
VALUES (5, 'Medien Verlag');

/* Zeilen in die Tabelle fachbereich einfügen */
INSERT INTO fachbereich (fachbereichid,fachbereich)
VALUES (1, 'Elektrotechnik');
INSERT INTO fachbereich (fachbereichid,fachbereich)
VALUES (2, 'Kaufmann');
INSERT INTO fachbereich (fachbereichid,fachbereich)
VALUES (3, 'Mechanik');
INSERT INTO fachbereich (fachbereichid,fachbereich)
VALUES (4, 'Pneumatik');

```

Listing 10.8 INSERT-Anweisungen zum Einfügen von Zeilen in die Tabellen »verlag« und »fachbereich«

Lösung zu Übung 2

```

INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (1, '1235', 'Mechanik',3);

```

```
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (2,'9878','Elektrotechnik',2);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (3,'2323','Elektronik',2);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (4,'2254','Pneumatik',3);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (5,'4455','Mathematik Grundlagen 1',2);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (6,'4456','Mathematik Grundlagen 2',2);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (7,'5566','Mengenlehre',NULL);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (8,'7766','Kommunikation 1',NULL);
INSERT INTO fachbuch (fachbuchid,isbn,titel,fk_verlagid)
VALUES (9,'7767','Kommunikation 2',NULL);
```

Listing 10.9 INSERT-Anweisungen zum Einfügen von Zeilen in die Tabelle »fachbuch«

Lösung zu Übung 3

```
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (3,1);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (1,3);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (4,4);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (1,5);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (1,6);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (1,7);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (2,8);
INSERT INTO fachbereichfachbuch (fk_fachbereichid,fk_fachbuchid)
VALUES (2,9);
```

Listing 10.10 INSERT-Anweisungen zum Einfügen von Zeilen in die Tabelle »fachbereichfachbuch«

10.2 Zeilen aus Tabellen, die in Beziehung stehen, mit JOIN verbunden abfragen

Wir haben Tabellen in Beziehung zueinander gesetzt und Datensätze in Tabellen eingefügt, die berücksichtigen, dass sie in Beziehung zueinander stehen. Jetzt schauen wir uns an, wie Sie Datensätze aus solchen Tabellen abfragen. Daher nutzen wir die Beziehungen zwischen den Tabellen, um **SELECT**-Abfragen zu formulieren, mit denen wir die Datensätze, deren Schlüssel gleich sind, verbunden abfragen. Das heißt, dass Sie in der Spaltenauswahlliste die Wahl haben zwischen den Spalten, die in den verbundenen Tabellen vorkommen.

Es gibt vier unterschiedliche **JOIN**-Arten, mit denen Sie Tabellen abfragen können, deren Zeilen durch Fremdschlüsselbeziehungen verbunden sind: der **INNER JOIN**, der **LEFT OUTER JOIN**, der **RIGHT OUTER JOIN** und der **FULL OUTER JOIN**. Wenn Sie einen **INNER JOIN** verwenden, werden die Zeilen aus Tabellen verbunden abgefragt, wenn Schlüsselwerte übereinstimmen. Ein **LEFT OUTER JOIN** verbindet hingegen alle Zeilen der linken Tabelle mit allen Zeilen der rechten Tabelle. Außerdem bleiben Ihnen beim **LEFT OUTER JOIN** alle Zeilen erhalten, die in der linken Tabelle existent sind, aber in der rechten Tabelle nicht mit Schlüsselwerten verbunden werden können. Für den **RIGHT OUTER JOIN** gilt das Ganze umgekehrt. Ein **FULL OUTER JOIN** verbindet Zeilen mit übereinstimmenden Schlüsselwerten. In die Ergebnistabelle werden auch Zeilen der linken und rechten Tabelle aufgenommen, die keine übereinstimmenden Schlüsselwerte haben. Ein **CROSS JOIN** schließlich verbindet alle Zeilen der beiden Tabellen unabhängig von allen Schlüsselbeziehungen miteinander.

Die Diagramme in [Abbildung 10.1](#) bis [Abbildung 10.5](#) veranschaulichen die Operationen:

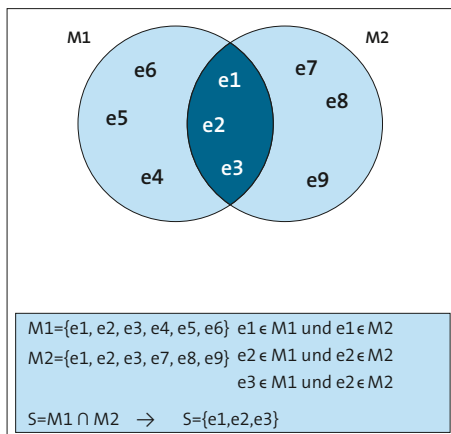


Abbildung 10.1 Ein INNER JOIN

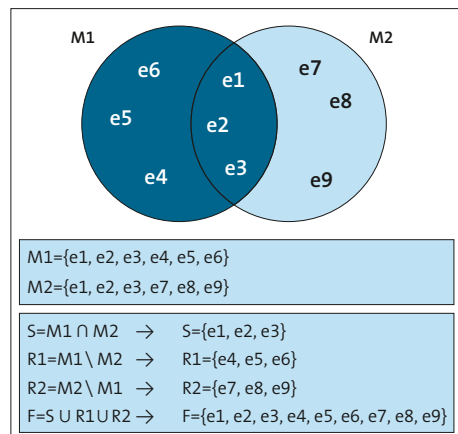


Abbildung 10.2 Ein LEFT OUTER JOIN

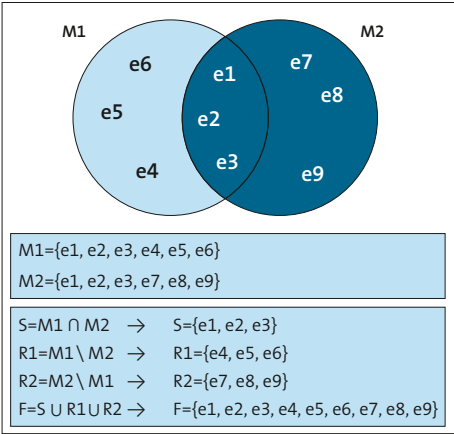


Abbildung 10.3 Ein RIGHT OUTER JOIN

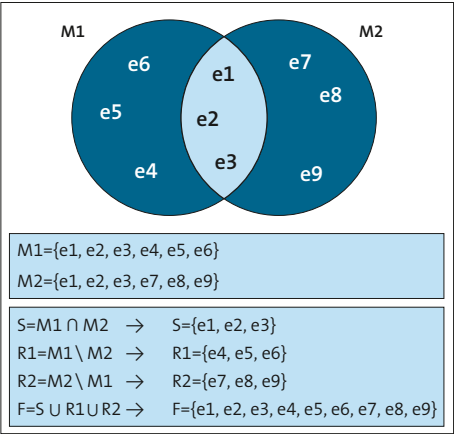


Abbildung 10.4 Ein FULL OUTER JOIN

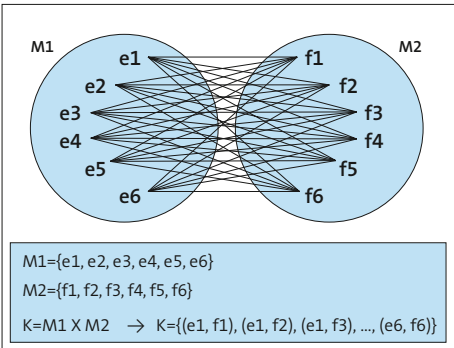


Abbildung 10.5 Ein CROSS JOIN

10.2.1 Zeilen mit einem INNER JOIN verbinden

Ein **INNER JOIN** verbindet Zeilen von Tabellen, indem auf Gleichheit von Primärschlüssel- und Fremdschlüsselwerten geprüft wird. Wie bereits erläutert, stehen die Tabellen *auszubildender* und *adresse* über eine Fremdschlüsselbeziehung in der Tabelle *adresse* zueinander in Beziehung.

In Abbildung 10.6 sehen Sie, dass jeweils die Fremdschlüsselwerte der Spalte *fk_ausid* der Tabelle *adresse* auf gleiche Primärschlüsselwerte der Spalte *ausid* in der Tabelle *auszubildender* verweisen.

Diese Tabellen, die in Beziehung zueinander stehen, wollen wir mit einer **SELECT**-Anweisung abfragen. Wenn wir zwei Tabellen abfragen, stehen uns natürlich auch die Spalten aus zwei Tabellen zur Verfügung. Sie realisieren die Verbindung von Zeilen über identische Schlüsselwerte aus Tabellen mit einem **INNER JOIN**. Ein **INNER JOIN** ver-

bindet Zeilen von Tabellen nur dann, wenn eine Schlüsselbeziehung zwischen einer Eltern- und einer Kindtabelle existiert. Wenn Zeilen in den Tabellen vorhanden sind, die über keine Beziehung verfügen, werden sie also nicht von einem **INNER JOIN** berücksichtigt.

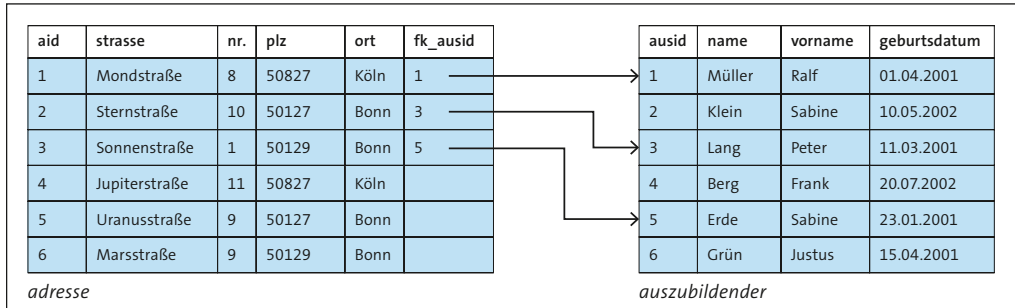


Abbildung 10.6 Zeilen von Tabellen mit Schlüsselwerten verbinden

In [Listing 10.11](#) sehen Sie eine **SELECT**-Abfrage mit einem **INNER JOIN**:

```
SELECT name, vorname, plz, ort
FROM auszubildender INNER JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.11 Die Tabellen »auszubildender« und »adresse« mit einem **INNER JOIN** verbunden abfragen

Hinter dem Schlüsselwort **SELECT** sehen Sie wie gewohnt eine Spaltenauswahlliste. Hier gibt es nur eine Neuerung: Die Spalten *name* und *vorname* fragen wir aus der Tabelle *auszubildender* ab. Die Spalten *plz* und *ort* stammen hingegen aus der Tabelle *adresse*. Es folgen das Schlüsselwort **FROM** und die Tabellenbezeichnung *auszubildender*. Bis hierhin haben Sie fast nichts Neues kennengelernt.

Hinter der Tabellenbezeichnung *auszubildender* folgen jetzt die neuen Schlüsselwörter **INNER JOIN** (innerer Verbund). Hinter der **INNER JOIN**-Klausel notieren Sie die Tabelle, die über einen **INNER JOIN** verbunden abgefragt werden soll. Somit haben Sie schon einmal bekannt gemacht, welche Tabellen verbunden abgefragt werden sollen.

Jetzt müssen wir nur noch angeben, wie die Zeilen der Tabellen miteinander verbunden werden sollen. Die Bedingung, die auf Gleichheit von Schlüsselwerten prüft (Primärschlüssel- und Fremdschlüsselwerte), legen Sie in einer **ON**-Klausel fest.

Das Schlüsselwort **INNER** der **INNER JOIN**-Klausel ist optional anzugeben. In [Listing 10.12](#) sehen Sie eine **SELECT**-Abfrage mit einem **INNER JOIN**, in dem auf das Schlüsselwort **INNER** verzichtet wurde. Diese Abfrage liefert Ihnen exakt das gleiche Ergebnis wie die Abfrage aus [Listing 10.11](#).

```
SELECT name,vorname,plz,ort
FROM auszubildender JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.12 Einen INNER JOIN ohne das optionale Schlüsselwort INNER verwenden

Tabelle 10.5 zeigt Ihnen das Ergebnis der Abfrage:

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Lang	Peter	50127	Bonn
Erde	Sabine	50129	Bonn

Tabelle 10.5 Ergebnistabelle für eine Abfrage mit einem INNER JOIN

Die Zeilen, die jeweils durch eine Referenz der Kindtabelle *adresse* auf Zeilen der Elterntabelle *auszubildender* verweisen, konnten erfolgreich verbunden werden. Sicher fällt Ihnen auf, dass die Datensätze, die über keine Referenz (Fremdschlüsselwert gleich NULL) aus der Tabelle *adresse* verfügen, nicht verbunden werden konnten. Wie Sie Zeilen, die über keine Schlüsselbeziehungen verfügen, mit in die Ergebnisliste aufnehmen, werden Sie in den nächsten Abschnitten erfahren.

10.2.2 Zeilen mit einem LEFT OUTER JOIN verbinden

Ein **LEFT OUTER JOIN** verknüpft ebenfalls Zeilen von Tabellen über eine Fremdschlüsselbeziehung. Zunächst einmal verbindet ein **LEFT OUTER JOIN** alle Zeilen von zwei Tabellen über eine Schlüsselbeziehung. Immer dann, wenn ein Fremdschlüssel aus einer Kindtabelle auf einen Primärschlüssel aus einer Elterntabelle referenziert, werden Zeilen entsprechend verbunden. Das kennen Sie bereits vom **INNER JOIN**.

Ein **LEFT OUTER JOIN** gibt zusätzlich die Zeilen der Tabelle *auszubildender* (die links von der **LEFT OUTER JOIN**-Klausel notiert ist) aus, die nicht über Schlüsselwerte mit der Tabelle *adresse* (die rechts von der **LEFT OUTER JOIN**-Klausel notiert ist) verbunden werden können. Diese Zeilen bleiben uns also erhalten. In diesem Fall bedeutet das, dass die Spaltenwerte der Zeilen der Elterntabelle *auszubildender* ausgegeben werden, auf die keine Referenzierung aus den Zeilen der Kindtabelle *adresse* erfolgt.



LEFT OUTER JOIN und Schlüsselbeziehungen

Beachten Sie Folgendes: Einem **LEFT OUTER JOIN** ist es grundsätzlich egal, ob Sie Fremdschlüsselwerte mit Primärschlüsselwerten verglichen. In der **ON**-Klausel wird

eine Bedingung ausgewertet. Wenn diese wahr ist, werden Zeilen verbunden ausgegeben. Wenn sie nicht wahr ist, wird keine Zeile verbunden abgefragt. Das war es auch schon.

Der **LEFT OUTER JOIN** fragt auch die Zeilen aus der Tabelle links von der **LEFT OUTER JOIN**-Klausel ab, die nicht über eine passende Schlüsselbeziehung mit der rechten Tabelle verbunden werden können. Die Zeilen der Tabelle links von der **LEFT OUTER JOIN**-Klausel sind daher ebenso in der Ergebnistabelle enthalten.

In [Abbildung 10.6](#) sehen Sie das Ergebnis der Abfrage aus [Listing 10.13](#). Darin finden Sie jetzt insgesamt sechs Zeilen. Aus der Tabelle *adresse* wird nicht auf die Zeilen mit den Primärschlüsselwerten 2, 4 und 6 in der Spalte *ausid* referenziert. Diesen drei Auszubildenden kann derzeit noch kein Wohnort zugeordnet werden. In der Tabelle *adresse* sehen Sie also auch keine Zeilen mit Fremdschlüsselwerten, die auf die Zeilen der Tabelle *auszubildender* mit Primärschlüsselwerten verweisen.

Die **SELECT**-Abfrage in [Listing 10.13](#) verbindet Zeilen über einen **LEFT OUTER JOIN**:

```
SELECT name,vorname,plz,ort
FROM auszubildender LEFT OUTER JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.13 Die Tabellen »auszubildender« und »adresse« mit einem **LEFT OUTER JOIN** verbunden abfragen

Die Zeilen aus der linken Elterntabelle *auszubildender*, die nicht mit der Kindtabelle *adresse* verbunden werden können, werden ebenso abgefragt. Anstelle des **INNER JOIN**-Schlüsselworts verwenden Sie jetzt das Schlüsselwort **LEFT OUTER JOIN**, um alle Zeilen der linken Tabelle abzufragen, die über Schlüsselwerte mit der rechten Tabelle verbunden oder eben nicht verbunden werden können.

Das Schlüsselwort **OUTER** der **LEFT OUTER JOIN**-Klausel ist dabei optional anzugeben. In [Listing 10.14](#) sehen Sie eine **SELECT**-Abfrage mit einem **LEFT OUTER JOIN**, in dem auf das Schlüsselwort **OUTER** verzichtet wurde. Diese Abfrage liefert Ihnen exakt das gleiche Ergebnis wie die Abfrage aus [Listing 10.13](#).

```
SELECT name,vorname,plz,ort
FROM auszubildender LEFT JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.14 Einen **LEFT OUTER JOIN** ohne das optionale Schlüsselwort **OUTER** verwenden

[Tabelle 10.6](#) zeigt Ihnen das Ergebnis der Abfrage aus [Listing 10.14](#):

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Klein	Sabine	NULL	NULL
Lang	Peter	50127	Bonn
Berg	Frank	NULL	NULL
Erde	Sabine	50129	Bonn
Grün	Justus	NULL	NULL

Tabelle 10.6 Ergebnistabelle für eine Abfrage mit einem LEFT OUTER JOIN

Sie sehen, dass die Zeilen der Tabelle *auszubildender*, denen über die Fremdschlüsselbeziehungen der Tabelle *adresse* kein Wohnort zugeordnet werden konnte, ebenfalls angezeigt werden. Für diese Zeilen existieren natürlich keine Angaben bzw. Spaltenwerte zum Wohnort.

Wenn ein **LEFT OUTER JOIN** verwendet wird und Zeilen in der linken Tabelle existieren, die nicht über eine Schlüsselbeziehung mit Zeilen der rechten Tabelle verbunden werden können, füllt die Datenbank die Spaltenwerte der rechten Tabelle mit **NULL**-Werten auf. Wie können wir nun die Auszubildenden ermitteln, denen keine Adresse zugeordnet ist? Sie haben eben erfahren, dass die Datenbank unter Verwendung eines **LEFT OUTER JOINs** alle Spaltenwerte von Zeilen der rechten Tabelle, die nicht mit Zeilen aus der linken Tabelle verbunden werden können, mit **NULL**-Werten auffüllt. Mit einer einfachen **WHERE**-Klausel, die auf **NULL**-Werte prüft, können Sie jetzt die Auszubildenden ermitteln, denen keine Adresse zugeordnet ist. Die **WHERE**-Klausel notieren Sie wie in [Listing 10.15](#) angegeben hinter der **ON**-Klausel, um die Spalte *plz* der Tabelle *adresse* auf **NULL** zu prüfen. Welche Spalte der Tabelle *adresse* Sie auf **NULL**-Werte prüfen, ist nicht bedeutend, da ja sämtliche Spaltenwerte der Tabelle *wohntort* mit **NULL**-Werten in der Ergebnistabelle aufgefüllt werden, wenn keine Referenzierung zu einem Auszubildenden bzw. einer Zeile der Tabelle *auszubildender* möglich ist.

```
SELECT name,vorname,plz,ort
FROM auszubildender LEFT OUTER JOIN adresse
ON ausid=fk_ausid
WHERE plz IS NULL;
```

Listing 10.15 Die Tabellen »auszubildender« und »adresse« mit einem LEFT OUTER JOIN verbunden abfragen und nur die Zeilen filtern, denen keine Adresse zugeordnet werden konnte

Tabelle 10.7 zeigt Ihnen das Ergebnis der Abfrage aus Listing 10.15. Gemäß der Bedingung der **WHERE**-Klausel sind jetzt nur noch diejenigen Zeilen der Tabelle *auszubildender* in der Ergebnistabelle enthalten, die nicht über Schlüsselwerte mit einer Adresse verbunden werden konnten.

name	vorname	plz	ort
Klein	Sabine	NULL	NULL
Berg	Frank	NULL	NULL
Grün	Justus	NULL	NULL

Tabelle 10.7 Ergebnistabelle für eine Abfrage mit einem **LEFT OUTER JOIN** und einer Bedingung, die eine Spalte der rechten Tabelle auf **NULL**-Werte prüft

10.2.3 Zeilen mit einem **RIGHT OUTER JOIN** verbinden

Ein **RIGHT OUTER JOIN** verbindet zunächst einmal wie ein **INNER JOIN** oder ein **LEFT OUTER JOIN** alle Zeilen von zwei Tabellen, die über Schlüsselwerte verbunden abgefragt werden können.

Wie sieht es nun aus, wenn wir eine Kindtabelle abfragen wollen, die Zeilen ohne Referenzierung auf die Elterntabelle enthält? In Abschnitt 10.1.4 haben Sie bereits Zeilen in die Tabelle *adresse* eingefügt, die nicht auf Fremdschlüsselwerte, sondern auf Primärschlüsselwerte aus Zeilen der Tabelle *auszubildender* verweisen. Im Beispiel enthalten diese Zeilen **NULL**-Werte. Diese Zeilen der Tabelle *adresse* stehen also schlicht und ergreifend nicht in einer Beziehung zu Zeilen der Tabelle *auszubildender*.

Ein **RIGHT OUTER JOIN** gibt auch die Zeilen der Tabelle *adresse* (die rechts von der **RIGHT OUTER JOIN**-Klausel notiert ist) aus, die nicht über Schlüsselwerte mit der Tabelle *auszubildender* (die links von der **RIGHT OUTER JOIN**-Klausel notiert ist) verbunden werden können. Diese Zeilen bleiben uns also erhalten.

In unserem Fall bedeutet das, dass die Spaltenwerte derjenigen Zeilen der Tabelle *adresse* ausgegeben werden, die über keine Referenzierung zu den Zeilen der Tabelle *auszubildender* verfügen.

RIGHT OUTER JOIN und Schlüsselbeziehungen

Beachten Sie auch bei dieser Beziehungsart Folgendes: Einem **RIGHT OUTER JOIN** ist es ebenfalls grundsätzlich egal, ob Sie Fremdschlüsselwerte mit Primärschlüsselwerten vergleichen. In der **ON**-Klausel wird eine Bedingung ausgewertet; wenn sie wahr ist, werden Zeilen verbunden ausgegeben. Wenn sie nicht wahr ist, wird keine Zeile verbunden zurückgegeben.



Der **RIGHT OUTER JOIN** fragt also auch die Zeilen aus der Tabelle rechts von der **RIGHT OUTER JOIN**-Klausel ab, die nicht über eine passende Schlüsselbeziehung verbunden werden können. Die Zeilen der Tabelle rechts von der **RIGHT OUTER JOIN**-Klausel sind daher ebenso in der Ergebnistabelle enthalten.

Betrachten Sie auch hier [Abbildung 10.6](#). In der Tabelle *adresse* sind Zeilen mit den Primärschlüsselwerten 4, 5 und 6 der Spalte *aid* enthalten, für die keine Fremdschlüsselwerte eingefügt wurden. Hier gibt es also keine Referenzierung auf Primärschlüsselwerte der Tabelle *auszubildender* und somit keine gültigen Beziehungen.

Für den **RIGHT OUTER JOIN** gilt ebenfalls, dass das Schlüsselwort **OUTER** optional anzugeben ist. In [Listing 10.17](#) sehen Sie eine **SELECT**-Abfrage mit einem **RIGHT OUTER JOIN**, in dem auf das Schlüsselwort **OUTER** verzichtet wurde. Diese Abfrage liefert Ihnen exakt das gleiche Ergebnis wie die Abfrage aus [Listing 10.16](#).

```
SELECT name,vorname,plz,ort
FROM auszubildender RIGHT OUTER JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.16 Die Tabellen »auszubildender« und »adresse« mit einem **RIGHT OUTER JOIN** verbunden abfragen

```
SELECT name,vorname,plz,ort
FROM auszubildender RIGHT JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.17 Einen **RIGHT OUTER JOIN** ohne das optionale Schlüsselwort **OUTER** verwenden

Zunächst einmal werden auch hier die Zeilen mit übereinstimmenden Schlüsselwerten verbunden abgefragt. Außerdem werden Zeilen der rechten Tabelle ausgegeben, die keine Einträge (nicht definierte Werte, also **NULL**-Werte) in der Fremdschlüsselspalte enthalten. Die Spalten der linken Tabelle *auszubildender* werden hier wieder mit **NULL**-Werten aufgefüllt.

Das Ergebnis der Abfrage sehen Sie in [Tabelle 10.8](#):

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Lang	Peter	50127	Bonn
Erde	Sabine	50129	Bonn

Tabelle 10.8 Ergebnis für eine Abfrage mit einem **RIGHT OUTER JOIN**

name	vorname	plz	ort
NULL	NULL	50827	Köln
NULL	NULL	50127	Bonn
NULL	NULL	50129	Bonn

Tabelle 10.8 Ergebnis für eine Abfrage mit einem RIGHT OUTER JOIN (Forts.)

Die Zeilen der Tabelle *adresse*, die keine Fremdschlüsselwerte enthalten, werden ebenfalls angezeigt. Wenn ein **RIGHT OUTER JOIN** verwendet wird und die Zeilen der rechten Tabelle nicht über eine Schlüsselbeziehung mit Zeilen der linken Tabelle verbunden werden können, füllt die Datenbank die Spaltenwerte der linken Tabelle wie beim **LEFT OUTER JOIN** mit NULL-Werten auf.

Vielleicht fragen Sie sich jetzt auch, wie Sie ausschließlich die Adressen ermitteln können, die keinem Auszubildenden zugeordnet sind. Wie beim **LEFT OUTER JOIN** prüfen wir beim **RIGHT OUTER JOIN** einfach wieder auf Spaltenwerte aus der linken Tabelle *auszubildender*, deren Zeilen mit keinem Wohnort verbunden werden konnten und somit mit NULL-Werten gefüllt wurden. In der **WHERE**-Klausel in [Listing 10.18](#) werden die Spaltenwerte der Spalte *name* auf NULL geprüft, um nur die Zeilen in der Ergebnistabelle anzuzeigen, die nicht über eine Schlüsselreferenzierung mit Zeilen von Auszubildenden verbunden werden konnten. Sie können hier auch wieder jede beliebige Spalte der Tabelle *auszubildender* auf NULL-Werte prüfen.

```
SELECT name,vorname,plz,ort
FROM auszubildender RIGHT OUTER JOIN adresse
ON ausid=fk_ausid
WHERE name IS NULL;
```

Listing 10.18 Die Tabellen »auszubildender« und »adresse« mit einem RIGHT OUTER JOIN verbunden abfragen und nur die Zeilen ausgeben, die in der rechten Tabelle über keine Schlüsselwerte verfügen

[Tabelle 10.9](#) zeigt Ihnen das Ergebnis der Abfrage aus [Listing 10.18](#).

name	vorname	plz	ort
NULL	NULL	50827	Köln
NULL	NULL	50127	Bonn
NULL	NULL	50129	Bonn

Tabelle 10.9 Ergebnistabelle für eine Abfrage mit einem RIGHT OUTER JOIN und einer Bedingung, die eine Spalte der rechten Tabelle auf NULL-Werte prüft

Gemäß der Bedingung der **WHERE**-Klausel sind jetzt nur noch diejenigen Zeilen der Tabelle *adresse* in der Ergebnistabelle enthalten, die nicht über Schlüsselwerte mit Zeilen der Tabelle *auszubildender* verbunden werden konnten.

10.2.4 Zeilen mit einem FULL OUTER JOIN verbinden

Ein **FULL OUTER JOIN** verbindet die Zeilen aus zwei Tabellen, die über übereinstimmende Schlüsselwerte verfügen.

Mit einem **FULL OUTER JOIN** werden auch die Zeilen ermittelt, die über keine übereinstimmenden Schlüsselwerte der Tabellen verbunden werden können. Ein **FULL OUTER JOIN** ist also eine Mischung aus **INNER JOIN**, **RIGHT OUTER JOIN** und **LEFT OUTER JOIN**.



MySQL und MariaDB unterstützen keinen FULL OUTER JOIN

Die MySQL- und MariaDB-Datenbanken unterstützen keinen **FULL OUTER JOIN** gemäß dem SQL-Standard. Mit den Kenntnissen, die Sie bis jetzt erworben haben, ist es möglich, einen **FULL OUTER JOIN** zu simulieren, um das gleiche Ergebnis zu erhalten. Zum Simulieren eines **FULL OUTER JOINs** werden Sie im Anschluss an diesen Abschnitt die **UNION**-Klausel verwenden.

Falls Sie eine MySQL- oder MariaDB-Datenbank verwenden, sollten Sie diesen Abschnitt dennoch aufmerksam lesen, da ein **FULL OUTER JOIN** nun einmal zum Standard gehört und oft genutzt wird.

Um den **FULL OUTER JOIN** besser zu verstehen, betrachten wir wieder Abbildung 10.6. In der Tabelle *auszubildender* sind Zeilen enthalten, auf die nicht mit einem Fremdschlüsselwert aus der Tabelle *adresse* referenziert wird. In der Tabelle *adresse* wiederum sind Zeilen abgelegt, die keine Fremdschlüsselwerte enthalten. Somit kann hier kein Bezug aus der Tabelle *adresse* zur Tabelle *auszubildender* hergestellt werden. Das kennen Sie bereits aus den Beispielen zum **LEFT OUTER JOIN** und **RIGHT OUTER JOIN**. Ein **FULL OUTER JOIN** stellt eine Kombination aus diesen beiden dar. Wenn Sie also Tabellen mit einem **FULL OUTER JOIN** verbinden, erhalten Sie alle Zeilen mit übereinstimmenden Schlüsselwerten sowie die Zeilen der linken und der rechten Tabelle, in denen Schlüsselwerte nicht referenziert werden können.

In Listing 10.19 sehen Sie eine **SELECT**-Abfrage, die die Zeilen der Tabellen *auszubildender* und *adresse* über einen **FULL OUTER JOIN** verbindet:

```
SELECT name,vorname,plz,ort
FROM auszubildender FULL OUTER JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.19 Mit einem **FULL OUTER JOIN** Zeilen aus Tabellen verbunden abfragen

In [Tabelle 10.10](#) sehen Sie das Ergebnis der **SELECT**-Abfrage aus [Listing 10.19](#). Dort finden Sie Zeilen, die über Schlüsselwerte verbunden wurden. Außerdem sehen Sie Zeilen mit Auszubildenden ohne passende Adresse und gleichzeitig Adressen, denen keine Auszubildenden zugeordnet werden konnten. Für die jeweils fehlenden Werte erhalten wir wieder nicht definierte Werte (**NULL**).

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Klein	Sabine	NULL	NULL
Lang	Peter	50127	Bonn
Berg	Frank	NULL	NULL
Erde	Sabine	50129	Bonn
Grün	Justus	NULL	NULL
NULL	NULL	50129	Bonn
NULL	NULL	50127	Bonn
NULL	NULL	50827	Köln

Tabelle 10.10 Ergebnistabelle für eine Abfrage mit einem **FULL OUTER JOIN**

In den beiden letzten Abschnitten haben wir uns jeweils die Frage gestellt, wie wir die Zeilen eines Auszubildenden ermitteln, die keinem Wohnort zugeordnet sind, oder wie wir die Wohnorte ermitteln, die keinem Auszubildenden zuzuordnen sind. Wir haben eine Antwort auf diese Fragen gefunden, indem wir die betreffenden Zeilen mit einer **WHERE**-Klausel in einer **SELECT**-Abfrage gefiltert haben. Prüfen Sie einfach die Zeilen, in denen keine passende Eltern- oder Kindzeile mit einem Schlüssel verbunden werden kann, auf Gleichheit mit einem **NULL**-Wert.

In [Listing 10.20](#) sehen Sie eine Abfrage, in der die **WHERE**-Klausel jeweils eine Spalte aus den Tabellen *auszubildender* und *adresse* auf Gleichheit mit **NULL** prüft. Wir haben es hier mit zwei Bedingungen zu tun, die mit einem logischen **OR**-Operator verknüpft werden. Das heißt, dass die **WHERE**-Klausel erfüllt ist, wenn eine der Bedingungen erfüllt ist. Oder anders gesagt: Die Bedingung ist wahr, wenn einer der Spaltenwerte der Spalten *name* oder *plz* gleich **NULL** ist.

```
SELECT name,vorname,plz,ort
FROM auszubildender FULL OUTER JOIN adresse
```

```
ON ausid=fk_ausid
where name IS NULL OR plz IS NULL;
```

Listing 10.20 In einem FULL OUTER JOIN eine Spalte der linken und eine Spalte der rechten Tabelle in einer WHERE-Klausel auf NULL prüfen

Tabelle 10.11 zeigt Ihnen das Ergebnis der Abfrage aus Listing 10.20. Die ersten drei Zeilen enthalten ausschließlich die Auszubildenden, denen nicht über passende Schlüsselwerte aus der Tabelle *adresse* Adressen zugeordnet werden können. Die Zeilen 4 bis 6 hingegen enthalten die Adressen, die aufgrund fehlender Schlüsselbeziehungen keinem Auszubildenden in der Tabelle *auszubildender* zugeordnet werden können.

Mit der Ergänzung einer **WHERE**-Klausel, in der jeweils Spaltenwerte der an einer **JOIN**-Verbindung beteiligten Tabellen auf **NULL** geprüft werden, können Sie prüfen, wo keine Beziehungen zwischen den Zeilen der beteiligten Tabellen existieren.

name	vorname	plz	ort
Klein	Sabine	NULL	NULL
Berg	Frank	NULL	NULL
Grün	Justus	NULL	NULL
NULL	NULL	50129	Bonn
NULL	NULL	50127	Bonn
NULL	NULL	50827	Köln

Tabelle 10.11 Ergebnistabelle für eine FULL OUTER JOIN-Abfrage, die mit einer WHERE-Klausel versehen wurde und prüft, ob eine Spalte je Tabelle gleich NULL ist

10.2.5 Einen FULL OUTER JOIN unter MySQL oder MariaDB nachbilden

Den Lesern, die eine MySQL- oder MariaDB-Datenbank verwenden, ist dieser Abschnitt gewidmet, damit sie einen **FULL OUTER JOIN** mithilfe einer **UNION**-Klausel nachbilden können. In Kapitel 5, »Mengenoperationen anwenden«, haben Sie gelernt, wie Sie Schnittmengen, Vereinigungsmengen und Differenzmengen von Zeilen bilden. Dieses Wissen werden wir uns jetzt zunutze machen, um einen **FULL OUTER JOIN** auf einer MySQL- oder MariaDB-Datenbank nachzubilden. Die in Abschnitt 10.2.4 dargestellten Beispiele werden wir in der gleichen Reihenfolge in unserer alternativen Lösung mit der **UNION**-Klausel nutzen, um einen **FULL OUTER JOIN** zu realisieren.

Betrachten Sie hierzu [Listing 10.21](#). Es entspricht [Listing 10.19](#). In dieser **SELECT**-Abfrage haben wir die Zeilen der Tabellen *auszubildender* und *adresse* über einen **FULL OUTER JOIN** verbunden.

```
SELECT name,vorname,plz,ort
FROM auszubildender FULL OUTER JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.21 Mit einem **FULL OUTER JOIN** Zeilen aus Tabellen verbunden abfragen

Um einen **FULL OUTER JOIN** nachzubilden, betrachten Sie die zwei **SELECT**-Abfragen in [Listing 10.22](#), die jeweils einen **LEFT OUTER JOIN** und einen **RIGHT OUTER JOIN** realisieren:

```
SELECT name,vorname,plz,ort
FROM auszubildender LEFT OUTER JOIN adresse
ON ausid=fk_ausid;
```

```
SELECT name,vorname,plz,ort
FROM auszubildender RIGHT OUTER JOIN adresse
ON ausid=fk_ausid;
```

Listing 10.22 **INNER JOIN**, **LEFT OUTER JOIN** und **RIGHT OUTER JOIN**

Die erste Abfrage aus [Listing 10.22](#) verbindet die Zeilen der Tabellen *auszubildender* und *adresse* miteinander, und zwar über einen **LEFT OUTER JOIN**. Das bedeutet, dass alle Zeilen verbunden werden, die über gleiche Schlüsselwerte verfügen, und es werden die Zeilen aus der linken Tabelle mit aufgenommen, für die in der rechten Tabelle keine passenden Schlüsselwerte vorhanden sind.

Die zweite Abfrage aus [Listing 10.22](#) verbindet die Zeilen der Tabellen *auszubildender* und *adresse* über einen **RIGHT OUTER JOIN**. Hier werden also alle Zeilen der rechten Tabelle mit allen Zeilen der linken Tabelle verbunden, die über gleiche Schlüsselwerte verfügen. Außerdem werden die Zeilen der rechten Tabelle mit ausgegeben, in denen keine Fremdschlüsselwerte festgelegt wurden.

Die zwei **SELECT**-Abfragen haben eines gemeinsam: Sie fragen die Spalten *name*, *vorname*, *plz* und *ort* ab. Erinnern Sie sich noch an die **UNION**-Klausel aus [Abschnitt 5.2](#)? Dort haben Sie gelernt, dass für eine Vereinigung von Zeilen von mehreren Tabellen die Spaltenanzahl der einzelnen Abfragen identisch sein muss. Außerdem müssen die Datentypen in der Sequenz der Spaltenangabe gleich sein. Diese Voraussetzung ist hier gegeben.

Sehen wir uns dennoch noch einmal die Ergebnistabellen der zwei **SELECT**-Abfragen an, damit Sie der Lösung auch vertrauen können.

Die erste Abfrage liefert, wie in [Tabelle 10.12](#) gezeigt, die Ergebniszeilen, die aus einem **LEFT OUTER JOIN** für die Zeilen der Tabellen *auszubildender* und *adresse* resultieren:

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Klein	Sabine	NULL	NULL
Lang	Peter	50127	Bonn
Berg	Frank	NULL	NULL
Erde	Sabine	50129	Bonn
Grün	Justus	NULL	NULL

Tabelle 10.12 Ergebnistabelle für eine Abfrage mit einem **LEFT OUTER JOIN**

Die zweite Abfrage liefert Ihnen, wie in [Tabelle 10.13](#) gezeigt, die Ergebniszeilen, die aus einem **RIGHT OUTER JOIN** für die Zeilen der Tabellen *auszubildender* und *adresse* resultieren:

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Lang	Peter	50127	Bonn
Erde	Sabine	50129	Bonn
NULL	NULL	50827	Köln
NULL	NULL	50127	Bonn
NULL	NULL	50129	Bonn

Tabelle 10.13 Ergebnistabelle für eine Abfrage mit einem **RIGHT OUTER JOIN**

Die Übersicht der zwei Ergebnistabellen zeigt Ihnen, dass wir mit einer Vereinigung der Ergebniszeilen aus den zwei Ergebnistabellen einen **FULL OUTER JOIN** in seiner Funktionalität nachbilden können.

In [Listing 10.23](#) sehen Sie die Vereinigung der Ergebniszeilen der zwei Abfragen, die mit einer **UNION**-Klausel realisiert wird. Es werden einfach die Zeilen der Ergebnistabellen der zwei **SELECT**-Abfragen vereinigt, die jeweils mit einer **LEFT OUTER JOIN**-Klausel und einer **RIGHT OUTER JOIN**-Klausel ausgestattet sind.

```
SELECT name,vorname,plz,ort
FROM auszubildender LEFT OUTER JOIN adresse
```



```

ON ausid=fk_ausid
UNION
SELECT name,vorname,plz,ort
FROM auszubildender RIGHT OUTER JOIN adresse
ON ausid=fk_ausid;

```

Listing 10.23 Einen FULL OUTER JOIN mit UNION nachbilden

Wie bei einem **FULL OUTER JOIN** erhalten Sie als Ergebnis eine Tabelle, die die Zeilen enthält, die über Schlüsselwerte (**LEFT** und **RIGHT OUTER JOIN**) verbunden werden konnten. Die Zeilen der linken Tabelle, die mit keinem Schlüsselwert der rechten Tabelle übereinstimmen, und die Zeilen der rechten Tabelle, für die kein Fremdschlüsselwert festgelegt wurde, sind ebenfalls Bestandteil des Ergebnisses.

Die Zeilen, die ausschließlich in der linken oder rechten Tabelle enthalten sind, werden von der Datenbank automatisch mit **NULL**-Werten aufgefüllt. Das Auffüllen mit **NULL**-Werten ist dem **LEFT OUTER JOIN** und dem **RIGHT OUTER JOIN** geschuldet. Der **LEFT** und der **RIGHT OUTER JOIN** liefern uns auch zweimal die Ergebniszeilen, die einem **INNER JOIN** entsprächen. Hier sorgt der **UNION**-Vereinigungsoperator für Eindeutigkeit.

Das Ergebnis sehen Sie in Tabelle 10.14:

name	vorname	plz	ort
Müller	Ralf	50827	Köln
Lang	Peter	50127	Bonn
Erde	Sabine	50129	Bonn
Klein	Sabine	NULL	NULL
Berg	Frank	NULL	NULL
Grün	Justus	NULL	NULL
NULL	NULL	50827	Köln
NULL	NULL	50127	Bonn
NULL	NULL	50129	Bonn

Tabelle 10.14 Einen FULL OUTER JOIN mit UNION nachbilden

Als Nächstes werden wir, wie in den anderen Beispielen, mit einer **WHERE**-Klausel filtern, welche Zeilen der Tabellen *auszubildender* und *adresse* nicht über Schlüsselbeziehungen verbunden werden können.

In [Listing 10.24](#) sehen Sie, dass hinter den einzelnen **SELECT**-Anweisungen mit einer **WHERE**-Klausel jeweils eine Spalte der rechten bzw. linken Tabelle auf **NULL** geprüft wird, um nur die Zeilen zu ermitteln, die nicht über Schlüsselwerte verbunden werden können:

```
SELECT name,vorname,plz,ort
FROM auszubildender LEFT OUTER JOIN adresse
ON ausid=fk_ausid
WHERE plz IS NULL
UNION
SELECT name,vorname,plz,ort
FROM auszubildender RIGHT OUTER JOIN adresse
ON ausid=fk_ausid
WHERE name IS NULL;
```

Listing 10.24 Die Zeilen ermitteln, die nicht über einen JOIN verbunden werden konnten

[Tabelle 10.15](#) zeigt Ihnen das Ergebnis der Abfrage aus [Listing 10.24](#). Die ersten drei Zeilen enthalten – wie im Beispiel aus [Abschnitt 10.2.4](#) – wieder ausschließlich die Auszubildenden, denen nicht über passende Schlüsselwerte aus der Tabelle *adresse* Adressen zugeordnet werden können. Die Zeilen 4 bis 6 hingegen enthalten wieder die Adressen, die aufgrund fehlender Schlüsselbeziehungen keinem Auszubildenden in der Tabelle *auszubildender* zugeordnet werden können.

name	vorname	plz	ort
Klein	Sabine	NULL	NULL
Berg	Frank	NULL	NULL
Grün	Justus	NULL	NULL
NULL	NULL	50827	Köln
NULL	NULL	50127	Bonn
NULL	NULL	50129	Bonn

Tabelle 10.15 Nur die Zeilen ermitteln, die nicht verbunden abgefragt werden konnten

10.2.6 Zeilen mit einem CROSS JOIN verbinden

Ein **CROSS JOIN** verbindet völlig unabhängig von Schlüsselbeziehungen zwischen Eltern- und Kindtabellen die Zeilen einer Tabelle mit allen Zeilen einer anderen Tabelle. Betrachten Sie eine Zeile der linken Tabelle, so wird diese Zeile mit jeder Zeile der rechten Tabelle verbunden.

Mathematisch betrachtet handelt es sich hierbei um ein kartesisches Produkt. Sie können **CROSS JOINs** verwenden, um Testdaten zu generieren oder um alle Zeilen von zwei Tabellen miteinander verbunden auszugeben. Wenn Sie Testdaten generieren wollen, so reicht eine Tabelle mit 20.000 Zeilen aus, die Sie zum Beispiel mit einer Tabelle, die 100 Zeilen enthält, verbunden abfragen. So generieren Sie eine Ergebnistabelle mit Testdaten, die 2.000.000 Zeilen enthält.

Um Ihnen die Funktion eines **CROSS JOINs** möglichst einfach zu erklären, bleiben wir aber bei den Auszubildenden, denen Adressen zugeordnet sind.

Abbildung 10.7 sehen Sie, dass die erste Zeile der Tabelle *adresse* mit allen Zeilen der Tabelle *auszubildender* verbunden wird. Somit gilt auch, dass jede Zeile der rechten Tabelle *auszubildender* mit allen Zeilen der Tabelle *adresse* verbunden ist.

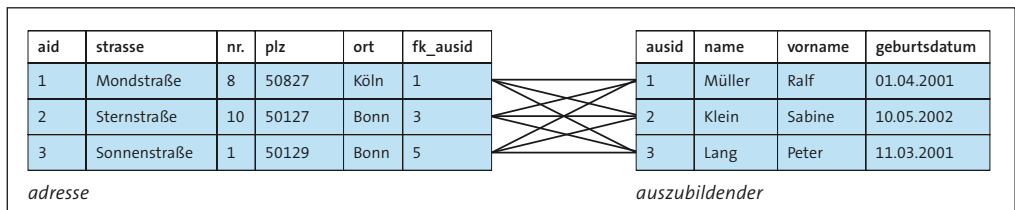


Abbildung 10.7 Zeilen von Tabellen, die über einen CROSS JOIN verbunden abgefragt werden

Abbildung 10.7 gezeigte Beispiel gut, um Sie mit dem **CROSS JOIN** vertraut zu machen.

Um eine **SELECT**-Abfrage über zwei Tabellen mit einem **CROSS JOIN** auszustatten, notieren Sie zwischen den Tabellen *auszubildender* und *adresse* die Schlüsselwörter **CROSS JOIN**. Das war es schon, da ein **CROSS JOIN** ja lediglich alle Zeilen einer Tabelle mit allen Zeilen einer anderen Tabelle verbindet, und das auch umgekehrt. Es wird also keine Bedingung (**ON**-Klausel mit Bedingung) benötigt, die Schlüsselwerte auf Gleichheit prüft. In dieser **SELECT**-Abfrage wurde noch die Spalte *strasse* mit in die Spaltenauswahlliste aufgenommen. So können Sie leichter sehen, dass tatsächlich alle Zeilen einer Tabelle mit allen Zeilen einer anderen Tabelle verbunden werden und umgekehrt.

```
SELECT name,vorname,plz,ort,strasse
FROM auszubildender CROSS JOIN adresse;
```

Listing 10.25 Zeilen aus zwei Tabellen mit einem CROSS JOIN verbunden abfragen

Tabelle 10.16 zeigt Ihnen das Ergebnis der **SELECT**-Abfrage aus Listing 10.25.

name	vorname	plz	ort	strasse
Müller	Ralf	50827	Köln	Mondstraße
Klein	Sabine	50827	Köln	Mondstraße
Lang	Peter	50827	Köln	Mondstraße
Berg	Frank	50827	Köln	Mondstraße
Erde	Sabine	50827	Köln	Mondstraße
Grün	Justus	50827	Köln	Mondstraße
Müller	Ralf	50127	Bonn	Sternstraße
Klein	Sabine	50127	Bonn	Sternstraße
Lang	Peter	50127	Bonn	Sternstraße
Berg	Frank	50127	Bonn	Sternstraße

Tabelle 10.16 Auszug aus der Ergebnistabelle für eine SELECT-Abfrage mit einem CROSS JOIN

Sie sehen, dass jede Zeile der Tabelle *auszubildender* mit jeder Zeile der Tabelle *adresse* ausgegeben wird. Wenn Sie die Abfrage nachvollziehen, werden Sie sehen, dass aus den sechs Zeilen der Tabelle *auszubildender*, in denen die Auszubildenden hinterlegt werden, und den sechs Zeilen der Tabelle *adresse*, in denen Wohnorte gespeichert sind, dank einem **CROSS JOIN** eine Ergebnistabelle mit 36 Zeilen wird.

In [Tabelle 10.16](#) ist lediglich ein Auszug aus der Ergebnistabelle zu sehen. Um zu prüfen, ob es sich tatsächlich um 36 Zeilen handelt, sollten Sie das Beispiel praktisch nachvollziehen.

10.2.7 Zeilen von drei Tabellen mit einem INNER JOIN verbinden

Sie erinnern sich bestimmt, dass ich Ihnen in [Abschnitt 7.4](#) die Ausbildungsdatenbank als Modell in der UML-Notation vorgestellt habe. Hier stehen die Tabellen *ausbildungsberuf*, *beruflehrfach* und *lehrfach* in Beziehung zueinander.

Die Werte der Fremdschlüsselspalten *fk_berufsid* und *fk_lehrfachid* aus der Tabelle *beruflehrfach* zeigen jeweils auf Werte der Primärschlüsselspalten *berufsid* und *lehrfachid* der Tabellen *ausbildungsberuf* und *lehrfach*. Somit sind die optimalen Voraussetzungen gegeben, um mit einem **INNER JOIN** Zeilen aus drei Tabellen zu verbinden und abzufragen.

In [Listing 10.26](#) sehen Sie eine **SELECT**-Abfrage, in der ein **INNER JOIN** über die drei Tabellen *ausbildungsberuf*, *beruflehrfach* und *lehrfach* realisiert wird.

```
SELECT berufsbezeichnung, lehrfach
FROM ausbildungsberuf
INNER JOIN beruflehrfach
ON berufsids=fk_berufsids
INNER JOIN lehrfach
ON fk_lehrfachids=lehrfachids;
```

Listing 10.26 Zeilen von drei Tabellen mit einem **INNER JOIN** verbunden abfragen

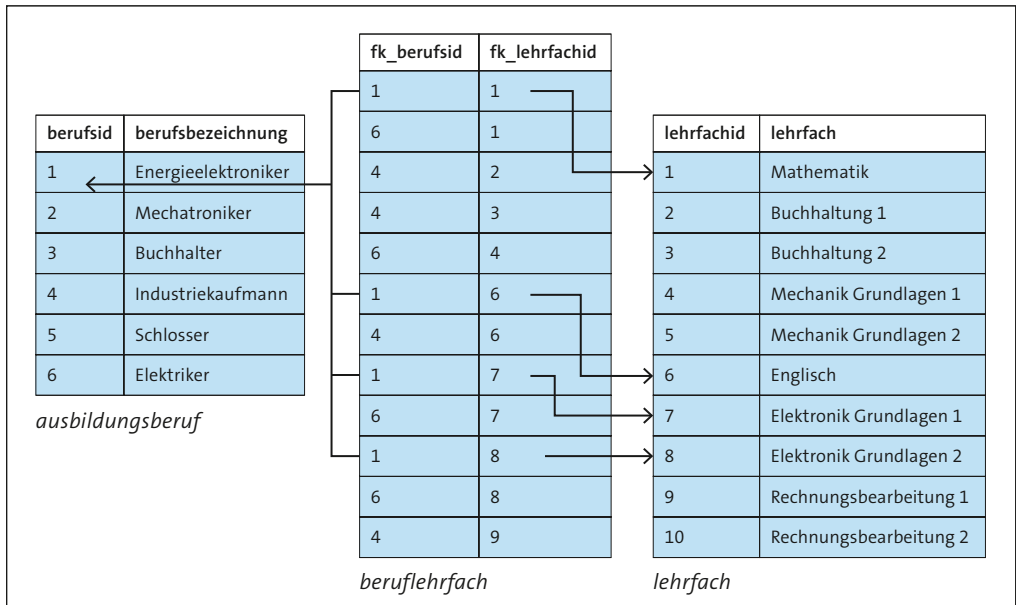


Abbildung 10.8 Drei Tabellen, die zueinander in Beziehung stehen

Vom Grundsatz her habe ich den **INNER JOIN** bereits in [Abschnitt 10.2.1](#) behandelt. Der Unterschied in [Listing 10.26](#) besteht zunächst darin, dass hinter der ersten **ON**-Klausel ein weiterer **INNER JOIN** angegeben wird, der die Tabelle *lehrfach* verbindet. In der Bedingung der **ON**-Klausel wird auf Gleichheit zwischen den Spaltenwerten der Fremdschlüsselspalte *fk_lehrfach* und der Primärschlüsselspalte *lehrfachids* geprüft.

Zunächst wird also mit der ersten **ON**-Klausel auf Gleichheit zwischen den Spaltenwerten der Spalten *berufsids* und *fk_berufsids* geprüft und somit eine Verbindung zwischen den Tabellen *ausbildungsberuf* und *beruflehrfach* geschaffen.

Dann wird in der zweiten **ON**-Klausel auf Gleichheit zwischen den Spaltenwerten der Spalten *fk_lehrfach* und *lehrfachid* geprüft und somit eine weitere Verbindung zwischen den Tabellen *beruflehrfach* und *lehrfach* geschaffen.



Mehrere Tabellen verbinden

Sie können natürlich auch mehr als drei Tabellen mit einem **INNER JOIN** verbunden abfragen. Sie müssen lediglich hinter der letzten **ON**-Klausel eine weitere **INNER JOIN**-Klausel mit der dazugehörigen **ON**-Klausel anfügen.

In Tabelle 10.17 sehen Sie das Ergebnis der **SELECT**-Abfrage aus Listing 10.26.

Die Abfrage liefert uns verbunden über die Schlüsseltabelle *beruflehrfach* die Berufsbezeichnungen und die jeweiligen Lehrfächer als Ergebnistabelle zurück.

berufsbezeichnung	lehrfach
Energieelektroniker	Mathematik
Energieelektroniker	Englisch
Energieelektroniker	Elektronik Grundlagen 1
Energieelektroniker	Elektronik Grundlagen 2
Industriekaufmann	Buchhaltung 1
Industriekaufmann	Buchhaltung 2
Industriekaufmann	Englisch
Industriekaufmann	Rechnungsbearbeitung 1

Tabelle 10.17 Ergebnistabelle für eine verbundene Abfrage aus drei Tabellen

10.2.8 Spalten in einem JOIN über Tabellennamen referenzieren

Bisher haben Sie Zeilen von Tabellen verbunden, deren Spaltenbezeichnungen unterschiedlich sind. Stellen Sie sich vor, Sie formulieren einen **INNER JOIN** über zwei Tabellen, in denen Spalten mit gleichen Spaltennamen vorkommen. Wenn gleichnamige Spaltennamen in der Spaltenauswahlliste einer **SELECT**-Anweisung enthalten sind, kann nicht zwischen den Spalten unterschieden werden. Wenn gleichnamige Spalten in zwei oder mehreren Tabellen vorkommen, kann die Datenbank also nicht unterscheiden, welche Spalte aus welcher Tabelle ausgewählt werden soll.

Das Problem ist allerdings leicht zu lösen: Schreiben Sie die jeweiligen Spaltennamen in der Spaltenauswahlliste in der Notation *tabellenname.spaltenname*. Zuerst notieren Sie also die Tabelle, gefolgt von einem Punkt, und schließlich dem Spaltennamen.

Sehen wir uns als Nächstes ein Beispiel an. In [Abbildung 10.9](#) sehen Sie einen Ausschnitt unseres Datenmodells für eine Ausbildungsdatenbank in der *UML-Notation*.

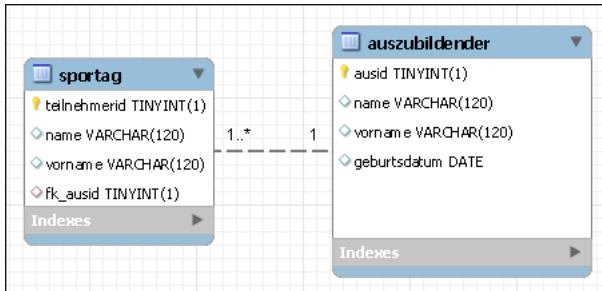


Abbildung 10.9 Tabellen, die identische Spaltennamen enthalten

Das Datenmodell wurde um die Tabelle *sportag* (Sport-Arbeitsgruppe) erweitert. In der Tabelle sind die Spalten *teilnehmerid*, *name*, *vorname* und *fk_ausid* enthalten. Die Spalte *fk_ausid* repräsentiert hier wieder die Fremdschlüsselspalte. Die Spaltennamen *name* und *vorname* kommen in den Tabellen *auszubildender* und *sportag* vor.

Um das Beispiel nachvollziehen zu können, verwenden wir die **CREATE TABLE**-Anweisung aus [Listing 10.27](#), die die Tabelle *sportag* in der *ausbildungsdatenbank* anlegt:

```

CREATE TABLE sportag (
    teilnehmerid SMALLINT PRIMARY KEY NOT NULL,
    name VARCHAR(120),
    vorname VARCHAR(120),
    fk_ausid SMALLINT,
    CONSTRAINT fk_ausid_auszubildender
        FOREIGN KEY (fk_ausid)
        REFERENCES auszubildender (ausid)
);
  
```

Listing 10.27 CREATE TABLE-Anweisung für die Tabelle »sportag«

[Listing 10.28](#) enthält zwei **INSERT**-Anweisungen, die benötigt werden, um zwei Zeilen in die Tabelle *sportag* einzufügen:

```

INSERT INTO sportag (teilnehmerid,name,vorname,fk_ausid)
VALUES (1,'Müller','Ralf',1);

INSERT INTO sportag (teilnehmerid,name,vorname,fk_ausid)
VALUES (2,'Klein','Sabine',2);
  
```

Listing 10.28 Zeilen in die Tabelle »sportag« einfügen

In [Abbildung 10.10](#) sehen Sie die Tabellen *sportag* und *auszubildender*. Sie enthalten beide die Spaltennamen *name* und *vorname*.

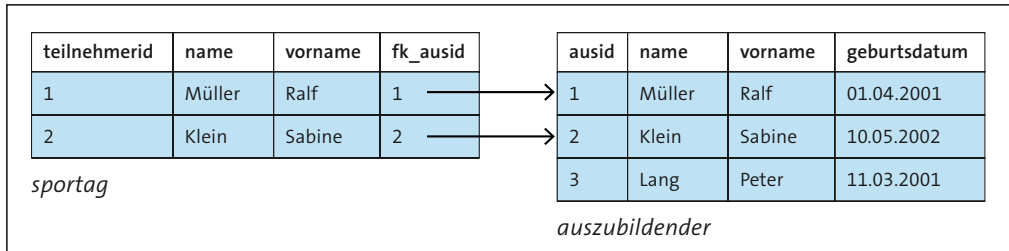


Abbildung 10.10 Tabellen mit gleichen Spaltennamen

Als Nächstes betrachten wir eine **SELECT**-Abfrage mit einem **INNER JOIN**, in dem die Spalten *name* und *vorname* abgefragt werden sollen. Außerdem soll aus der Tabelle *auszubildender* die Spalte *geburtsdatum* abgefragt werden.

```
SELECT name,vorname,geburtsdatum
FROM auszubildender INNER JOIN sportag
ON ausid=fk_ausid;
```

Listing 10.29 Mit einem **INNER JOIN** die Tabellen »auszubildender« und »sportag« verbunden abfragen

Die **SELECT**-Abfrage aus [Listing 10.29](#) führt zu einem Fehler, weil das Datenbanksystem die Spalten *name* und *vorname* nicht eindeutig einer Tabelle zuordnen kann. Die MySQL- und die MariaDB-Datenbank reagieren auf mehrfach gleich vorkommende Spaltennamen mit der Fehlermeldung:

Error Code: 1052. Column 'name' in field list is ambiguous

Die Lösung für dieses Problem sehen Sie in [Listing 10.30](#). Dort werden sämtliche Spalten jeweils über einen Tabellennamen referenziert. Jetzt kann die Datenbank die jeweiligen Spalten den Tabellen zuordnen, und die Mehrdeutigkeit ist aufgehoben.

```
SELECT
    auszubildender.name,
    auszubildender.vorname,
    auszubildender.geburtsdatum
FROM auszubildender INNER JOIN sportag
ON auszubildender.ausid=sportag.fk_ausid;
```

Listing 10.30 Spalten über einen Tabellennamen referenzieren

[Tabelle 10.18](#) zeigt das Ergebnis der Abfrage. Die **SELECT**-Abfrage konnte jetzt problemlos ausgeführt werden.

name	vorname	geburtsdatum
Müller	Ralf	2001-04-01
Klein	Sabine	2002-05-10

Tabelle 10.18 Ergebnistabelle für eine INNER JOIN-Abfrage mit mehrdeutigen Spaltennamen

In diesem Abschnitt haben Sie erfahren, wie Sie Spaltennamen aus zwei oder mehreren Tabellen über Tabellennamen referenzieren. Fällt Ihnen vielleicht auf, dass es noch einen anderen Weg gibt, das Problem der übereinstimmenden Bezeichnungen zu lösen?

Es wäre auch möglich gewesen, ohne die Spalten *name* und *vorname* in der Tabelle *sportag* auszukommen. Über die Fremdschlüsselbeziehung ist ein Zugriff auf die Spalten *name* und *vorname* in der Tabelle *auszubildender* ja bereits gegeben. Durch aufmerksames Nachdenken, welche Spalten überhaupt notwendig sind, hätte sich das Hindernis also auch umgehen lassen. Hier wollte ich Ihnen aber zeigen, wie Sie mit gleichen Spaltennamen umgehen, und deswegen habe ich dieses Beispiel gewählt.

10.2.9 Spalten in einem JOIN über Tabellenaliasse referenzieren

Wenn Sie Tabellen verbunden abfragen, können Sie hinter den Tabellennamen in der SELECT-Anweisung einen Tabellenalias notieren. Mit dem vergebenen Tabellenalias können Sie wiederum auf die Spalten der jeweiligen Tabelle referenzieren. [Listing 10.31](#) ist funktionell identisch mit der SELECT-Abfrage aus [Listing 10.30](#):

```
SELECT a.name,a.vorname,a.geburtsdatum
FROM auszubildender a INNER JOIN sportag s
ON a.ausid=s.fk_ausid;
```

Listing 10.31 Spalten über Tabellenaliasse referenzieren

In [Listing 10.31](#) werden Tabellenaliasse verwendet, die direkt hinter dem Tabellennamen notiert werden. Der Tabelle *auszubildender* ist der Tabellenalias *a* zugeordnet, der Tabelle *sportag* wiederum der Tabellenalias *s*.

Die festgelegten Tabellenaliasse *a* und *s* werden in der Spaltenauswahlliste der SELECT-Anweisung und in der ON-Klausel verwendet. Wenn Sie Spalten über einen Tabellenalias referenzieren, ersparen Sie sich eine Menge Schreibarbeit, und die Abfrage wird übersichtlicher. In der Regel werden Spalten aus Tabellen, die mit einem JOIN verbunden sind, über Tabellenaliasse referenziert.

Das Ergebnis ist exakt identisch mit dem Ergebnis, das aus [Listing 10.31](#) resultiert. Aus dem Grund verzichte ich hier auf eine Ergebnistabelle.



Tabellenaliasse in der ON-Klausel

Ein Tabellenalias kann in einer Spaltenauswahlliste verwendet werden. Ein weiterer Anwendungsfall für Tabellenaliasse ist die **ON**-Klausel, da es dort mehrfach gleich vergebene Spaltennamen geben kann.

In [Kapitel 2](#) haben Sie erfahren, wie Sie Spalten einen Spaltenalias zuordnen. In den Tabellen *auszubildender* und *sportag* sind jeweils die Spalten *name* und *vorname* enthalten. Zwei gleichnamige Spalten, die aus einem **JOIN** hervorgehen, sollten mit Spaltenaliasen abgefragt werden, um die Spalten auseinanderzuhalten, die ja aus unterschiedlichen Tabellen kommen. Sehen wir uns als Nächstes ein Beispiel an, in dem keine Spaltenaliasse verwendet und die gleichnamigen Spalten aus den Tabellen *auszubildender* und *sportag* abgefragt werden:

```
SELECT a.name,a.vorname,a.geburtsdatum,s.name,s.vorname
FROM auszubildender a INNER JOIN sportag s
ON a.ausid=s.fk_ausid;
```

Listing 10.32 Tabellen mit gleichen Spaltennamen mit einem **INNER JOIN** abfragen

[Tabelle 10.19](#) zeigt das Ergebnis der Abfrage aus [Listing 10.32](#).

name	vorname	geburtsdatum	name	vorname
Müller	Ralf	2001-04-01	Müller	Ralf
Klein	Sabine	2002-05-10	Klein	Sabine

Tabelle 10.19 Ergebnistabelle mit gleichen Spaltennamen

Sie sehen, dass Sie die jeweils doppelt vorkommenden Spalten *name* und *vorname* anhand des Namens keiner Tabelle zuordnen können. Hier helfen uns die Spaltenaliasse, die Sie in [Abschnitt 2.7](#) kennengelernt haben.

In [Listing 10.33](#) sehen Sie eine **SELECT**-Abfrage, in der die Spalten der Spaltenauswahlliste jeweils mit einem Spaltenalias versehen wurden:

```
SELECT
a.name AS aus_name,
a.vorname AS aus_vorname,
a.geburtsdatum AS aus_gebdatum,
s.name AS spag_name,
s.vorname AS spag_vorname
```

```
FROM auszubildender a INNER JOIN sporttag s
ON a.ausid=s.fk_ausid;
```

Listing 10.33 Doppelt vorkommende Spalten in einem JOIN mit Spaltenaliassen versehen

Als Resultat erhalten wir eine Ergebnistabelle, in der wir die Spalten eindeutig den Tabellen zuordnen können, aus denen sie stammen.

aus_name	aus_vorname	aus_gebdatum	spag_name	spag_vorname
Müller	Ralf	2001-04-01	Müller	Ralf
Klein	Sabine	2002-05-10	Klein	Sabine

Tabelle 10.20 Ergebnistabelle mit Spaltenaliassen

10.2.10 Zeilen mit einem Self Join verbinden

Bevor wir uns mit dem *Self Join* im Detail befassen, müssen wir klären, wozu er verwendet wird. Mit einem Self Join können Sie hierarchische Strukturen einer Tabelle abbilden, die über die entsprechenden Voraussetzungen verfügt. Um hierarchische Strukturen in einer Tabelle abzubilden, verbinden Sie die Zeilen einer Tabelle mit den Zeilen ebendieser einen Tabelle. Um dies zu realisieren, nutzen Sie eine Fremdschlüsselbeziehung. Der Unterschied ist der, dass der Fremdschlüssel nicht auf einen Primärschlüssel in einer anderen Tabelle verweist, sondern auf den Primärschlüssel, der in der gleichen Tabelle definiert wurde wie der Fremdschlüssel. Sie nutzen also ausschließlich eine Tabelle, um ihre Zeilen wieder miteinander zu verbinden. Eine Tabelle stellt also die Eltern- und die Kindtabelle gleichzeitig dar.

Um einen Self Join besser zu verstehen, ist es gut, die abzufragende Tabelle, deren Zeilen mit einem JOIN verbunden werden sollen, wie zwei eigenständige Tabellen zu betrachten. Wir stellen uns also vor, dass es sich um zwei verschiedene Tabellen handelt. In der ersten Tabelle finden wir eine Primärschlüsselspalte vor. In der zweiten Tabelle befindet sich eine Fremdschlüsselspalte, die wiederum auf die Primärschlüsselspalte der ersten Tabelle verweist.

Der Self Join ist keine SQL-Klausel

Bitte beachten Sie, dass der Self Join keine SQL-Klausel im klassischen Sinne ist. Es handelt sich lediglich um eine Bezeichnung für einen JOIN, der ausschließlich auf eine Tabelle angewendet wird. Einen Self Join können Sie mit einem INNER JOIN, einem LEFT OUTER JOIN, einem RIGHT OUTER JOIN und mit einem FULL OUTER JOIN realisieren.

In Abbildung 10.11 sehen Sie so eine Beziehung, die wir uns vorgestellt haben.

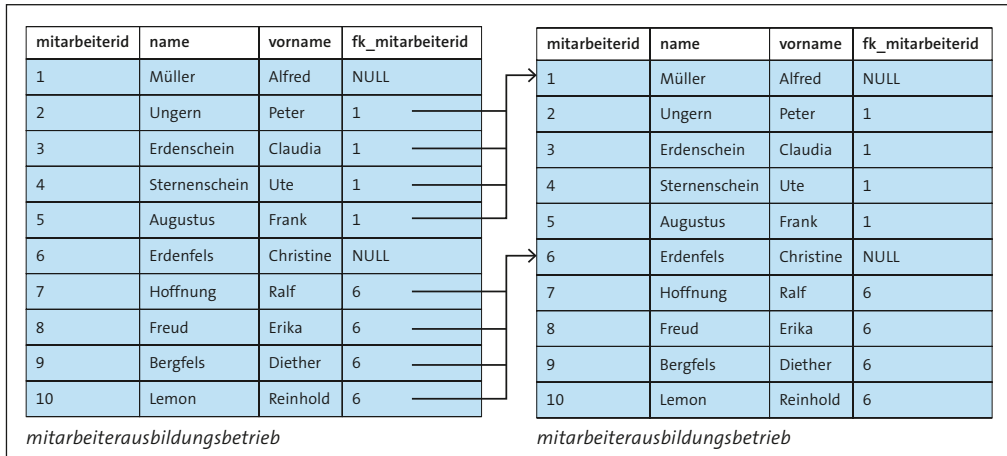


Abbildung 10.11 Eine Tabelle mit Zeilen, die auf Zeilen in der gleichen Tabelle referenzieren

In der linken Tabelle sehen Sie die Primärschlüsselspalte *mitarbeiterid*; in der rechten Tabelle sehen Sie die Fremdschlüsselspalte *fk_mitarbeiterid*. Die Fremdschlüsselwerte der Spalte *fk_mitarbeiterid* der rechten Tabelle verweisen wiederum auf die Primärschlüsselwerte der Spalte *mitarbeiterid*.

Mit diesen Erkenntnissen können wir einen **INNER JOIN** verwenden, der ermittelt, welche Mitarbeiter welchen Vorgesetzten untergeordnet sind. Den Mitarbeitern Alfred Müller und Christine Erdenfels sind in diesem Beispiel gleichzeitig die Rollen des Vorgesetzten und Inhabers zugeordnet, da keine Fremdschlüsselwerte (NULL) für diese Mitarbeiter existieren.

Als Nächstes sehen wir uns [Listing 10.34](#) an:

```
SELECT
    ma1.mitarbeiterid AS vorgesetzterid,
    ma1.name AS vorgesetztername,
    ma2.name AS mitarbeitername,
    ma2.mitarbeiterid AS mitarbeiterid
FROM mitarbeiterausbildungsbetrieb ma1 INNER JOIN
    mitarbeiterausbildungsbetrieb ma2
ON ma1.mitarbeiterid=ma2.fk_mitarbeiterid;
```

Listing 10.34 Eine Tabelle mit sich selbst verbunden abfragen

Hier werden mit einem **INNER JOIN** die Zeilen einer Tabelle mit sich selbst verbunden abgefragt. Daraus ergibt sich die Notwendigkeit, Tabellenaliasse zu verwenden. Ohne die Tabellenaliasse könnte die Datenbank, wie im letzten Abschnitt beschrieben, nicht unterscheiden, welche Spalten zu welcher Tabelle gehören.

Sonst entspricht der **INNER JOIN** den Regeln, die Sie bereits kennengelernt haben.

In [Tabelle 10.21](#) sehen Sie das Ergebnis der Abfrage.

vorgesetzterid	vorgesetztername	mitarbeitername	mitarbeiterid
1	Müller	Ungern	2
1	Müller	Erdenschein	3
1	Müller	Sternenschein	4
1	Müller	Augustus	5
6	Erdenfels	Hoffnung	7
6	Erdenfels	Freud	8
6	Erdenfels	Bergfels	9
6	Erdenfels	Lemon	10

Tabelle 10.21 Ergebnistabelle für einen INNER JOIN, der Zeilen einer Tabelle miteinander verbindet

Die Zeilen der Vorgesetzten (Tabellenalias *ma1*) wurden immer dann mit den Zeilen der Mitarbeiter (Tabellenalias *ma2*) verbunden, wenn die Fremdschlüsselwerte, die auf die Mitarbeiter zeigen, gleich den Primärschlüsselwerten der Vorgesetzten sind.

10.2.11 Zeilen mit einem INNER JOIN ohne Schlüsselvergleich verbinden

Bisher haben Sie **JOIN**-Typen in einer **SELECT**-Abfrage verwendet, um die Zeilen von zwei oder mehreren Tabellen miteinander zu verbinden. Die **JOINS**, die wir bisher formuliert haben, wurden stets über gültige Schlüsselbeziehungen ausgewertet, die bereits in der **CREATE TABLE**-Anweisung festgelegt waren. Es war also immer eine Bedingung vorhanden, die in der **ON**-Klausel genutzt werden konnte, um Spaltenwerte zu vergleichen.

In diesem Abschnitt zeige ich Ihnen, dass Sie einen beliebigen **JOIN**-Typ unabhängig von der Existenz von Fremdschlüsselbeziehungen verwenden können. In der **ON**-Klausel können beliebige Spalten aus zwei Tabellen verglichen werden. Das funktioniert allerdings nur, wenn der Datentyp der zu vergleichenden Spaltenwerte gleich ist oder implizit von der Datenbank konvertiert werden kann. In [Abschnitt 4.3](#) habe ich Ihnen den Unterschied zwischen einer expliziten und einer impliziten Typkonvertierung erklärt. Jetzt ist für Sie wichtig zu wissen, dass in der Bedingung der **ON**-Klausel Spaltenwerte des gleichen Typs ausgewertet werden.

Sehen wir uns als Erstes das Beispiel in [Abbildung 10.12](#) an.

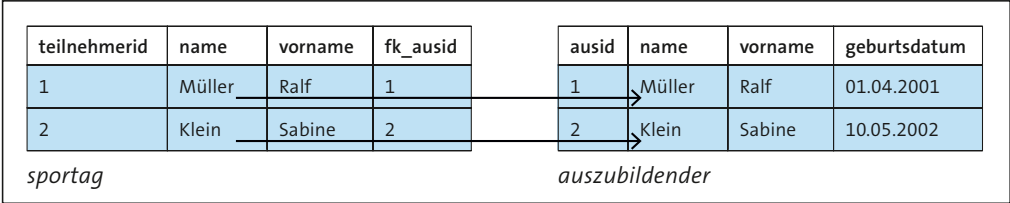


Abbildung 10.12 Verbindungen zwischen Zeilen mit Nicht-Schlüsselspalten realisieren

Hier sehen Sie die bekannten Tabellen *auszubildender* und *sporttag*, in denen jeweils die Spalte *name* enthalten ist. Die Datentypen sind in beiden Tabellen gleich, nämlich **VARCHAR(120)**. Somit hindert uns nichts daran, diese beiden Spalten wie Schlüsselspalten in einer Bedingung einer **ON**-Klausel zu verwenden. Wir könnten also einen **INNER JOIN** formulieren, um die Spalte *name*, die hier in beiden Tabellen vorhanden ist, in einer Bedingung auszuwerten. Auch hier werden die Zeilen der Tabellen nur dann verbunden ausgegeben, wenn die Bedingung in der **ON**-Klausel erfüllt ist.

Kommen wir zu einem konkreten Beispiel:

```
SELECT a.name, a.vorname, a.geburtsdatum, s.teilnehmerid
FROM auszubildender a INNER JOIN sporttag s
ON a.name=s.name;
```

Listing 10.35 Ein **INNER JOIN** mit einer Bedingung, die Spalten in der **ON**-Klausel auswertet, die keine Schlüsselspalten sind

In Listing 10.35 ist eine **SELECT**-Abfrage mit einem **INNER JOIN** zu sehen, die in der **ON**-Klausel die Spalte *name* der beiden Tabellen *auszubildender* und *sporttag* auf Gleichheit prüft.

In Tabelle 10.22 sehen Sie das Ergebnis der **SELECT**-Abfrage aus Listing 10.35. Die Zeilen der Tabellen *auszubildender* und *sporttag* werden immer dann miteinander verbunden und in die Ergebnistabelle mit aufgenommen, wenn die Werte der Spalte *name* der Bedingung in der **ON**-Klausel entsprechen.

name	vorname	geburtsdatum	teilnehmerid
Müller	Ralf	2001-04-01	1
Klein	Sabine	2002-05-10	2

Tabelle 10.22 Ergebnis eines **INNER JOINs**, der ohne Schlüsselspalten gebildet wurde

[*]

Zusammenfassung: Zeilen, die in Beziehung stehen, abfragen

Mit folgenden **JOIN**-Typen können Tabellen verbunden abgefragt werden:

- ▶ **INNER JOIN** – verbindet ausschließlich Zeilen mit gültigen Schlüsselwerten.
- ▶ **LEFT OUTER JOIN** – verbindet Zeilen mit gültigen Schlüsselwerten und gibt Zeilen der linken Tabelle aus, die nicht verbunden abgefragt werden können.
- ▶ **RIGHT OUTER JOIN** – verbindet Zeilen mit gültigen Schlüsselwerten und gibt Zeilen der rechten Tabelle aus, die nicht verbunden abgefragt werden können.
- ▶ **FULL OUTER JOIN** – verbindet Zeilen mit gültigen Schlüsselwerten und gibt die Zeilen der linken und rechten Tabelle aus, die nicht über Schlüsselwerte verbunden werden können.
- ▶ **CROSS JOIN** – verbindet jede Zeile einer Tabelle mit jeder Zeile einer anderen Tabelle und umgekehrt.
- ▶ **Self Join** – verbindet eine Zeile einer Tabelle mit Zeilen der gleichen Tabelle.

Die Verbindung zwischen den Zeilen wird (bis auf den **CROSS JOIN** mit einer Bedingung) stets über Primärschlüsselwerte und darauf referenzierende Fremdschlüsselwerte hergestellt.

Die hier aufgeführten **JOIN**-Typen (bis auf den **CROSS JOIN**) können nicht nur über Schlüsselbeziehungen realisiert werden. Solange die Datentypen der Spalten, die an einem **JOIN** beteiligt sind, identisch sind, kann mit einer Bedingung in einer **ON**-Klausel eine Verbindung hergestellt werden.

10.2.12 Übungen

In den folgenden Übungen werden Sie Zeilen aus zwei oder mehreren Tabellen verbunden abfragen. Um Ihnen die Formulierung der **SELECT**-Abfragen zu erleichtern, sehen Sie in [Abbildung 10.13](#) noch einmal das Datenbankmodell der Bibliotheksdatenbank, das Sie bereits in [Kapitel 9](#) kennengelernt haben.

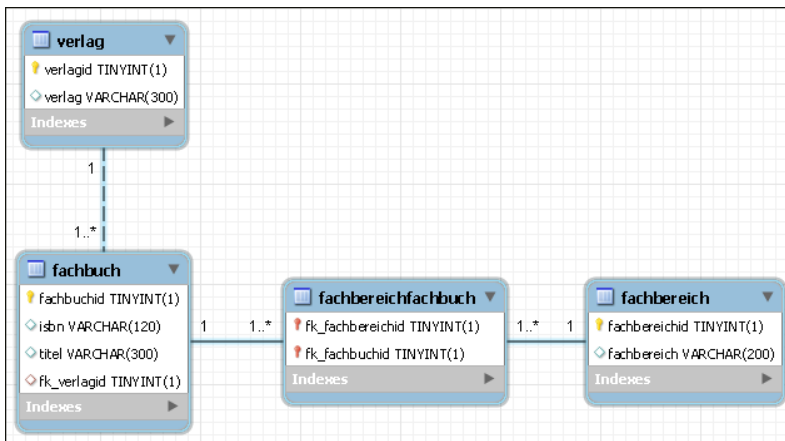


Abbildung 10.13 UML-Modell der Bibliotheksdatenbank

Bevor Sie eine **SELECT**-Anweisung formulieren, in der Sie mit einem **JOIN** die Zeilen von Tabellen verbinden, betrachten Sie zunächst das Modell, um die beteiligten Primärschlüsselspalten und Fremdschlüsselspalten zu identifizieren.

Übung 1

Die Tabellen *fachbuch* und *verlag* stehen in Beziehung zueinander. Die Tabelle *fachbuch* enthält eine Fremdschlüsselspalte *fk_verlagid*, deren Fremdschlüsselwerte auf Primärschlüsselwerte der Spalte *verlagid* in der Tabelle *verlag* referenzieren.

Formulieren Sie eine **SELECT**-Anweisung, in der mit einem **INNER JOIN** die Zeilen der Tabellen *fachbuch* und *verlag* verbunden ausgegeben werden. Hierbei sollen aus der Tabelle *fachbuch* die Spalten *titel*, *isbn* und *fk_verlagid* zurückgeliefert werden. Aus der Tabelle *verlag* sollen die Spalten *verlag* und *verlagid* ausgegeben werden.

Übung 2

In dieser Übung sollen wieder die Zeilen der Tabellen *fachbuch* und *verlag* verbunden ausgegeben werden. In der Ergebnistabelle sollen alle Zeilen enthalten sein, die über Schlüsselwerte verbunden werden können. Außerdem sollen die Zeilen der Tabelle *fachbuch* ausgegeben werden, die über keinen Fremdschlüsselwert zu einem Primärschlüsselwert der Spalte *verlagid* verfügen.

Übung 3

In der letzten Übung haben Sie die Zeilen in einer Ergebnistabelle ausgegeben, die sich über Schlüsselwerte verbinden ließen, sowie die Zeilen der linken Tabelle, die keine Fremdschlüsselwerte enthalten und somit keiner Zeile der rechten Tabelle zugeordnet werden konnten.

In dieser Übung fragen Sie die Zeilen der Tabellen *fachbuch* und *verlag* verbunden ab. In der Ergebnistabelle sollen neben den Zeilen, die sich über Schlüsselwerte verbinden lassen, auch die Zeilen der Tabelle *verlag* enthalten sein, auf die keine Fremdschlüsselwerte aus der Tabelle *fachbuch* verweisen.

Übung 4

In dieser Übung sollen alle Zeilen aus den Tabellen *fachbuch* und *verlag* in einer Ergebnistabelle ausgegeben werden, die sich verbinden lassen oder eben nicht verbinden lassen.

Das heißt also, dass auch die Zeilen der linken Tabellen *fachbuch*, die auf keine Zeilen der rechten Tabelle *verlag* referenzieren, enthalten sein sollen. Oder anders gesagt: Fachbücher, die keinem Verlag zugeordnet sind.

Fehlen dürfen auch nicht die Zeilen aus der rechten Tabelle *verlag*, auf die keine Referenz über Fremdschlüsselwerte aus der linken Tabelle existiert. Das heißt, sämtliche Zeilen der Verlage, denen kein Fachbuch zugeordnet ist, sollen auch enthalten sein.

Falls Sie eine MySQL- oder MariaDB-Datenbank verwenden, müssen Sie beachten, dass der **JOIN**-Typ, der in dieser Übung als Lösung gefordert wird, nicht von einer MySQL- oder MariaDB-Datenbank unterstützt wird. In [Abschnitt 10.2.5](#) habe ich dargestellt, wie Sie alternativ die Funktionalität des geforderten **JOIN**-Typs nachbilden können.

Übung 5

Formulieren Sie eine **SELECT**-Abfrage, in der jede Zeile der Tabelle *fachbuch* mit jeder Zeile der Tabelle *verlag* verbunden ausgegeben wird. Beachten Sie bei dieser Übung, dass hier keine Bedingung erforderlich ist, die auf die Gleichheit von Schlüsselwerten prüft.

Übung 6

Die Zeilen der Tabellen *fachbuch* und *fachbereich* stehen über die Schlüsseltabelle *fachbereichfachbuch* in Beziehung zueinander. In der Tabelle *fachbuch* wird über einen Primärschlüsselwert der Primärschlüsselspalte *fachbuchid* jede Zeile eindeutig identifiziert. Die Tabelle *fachbereich* enthält ebenfalls eine Primärschlüsselspalte *fachbereichid*, mit deren Primärschlüsselwerten jede Zeile eindeutig identifiziert werden kann.

In der Tabelle *fachbereichfachbuch* werden die Primärschlüsselwerte in Beziehung zueinander gesetzt.

Formulieren Sie eine **SELECT**-Abfrage, in der Sie die Zeilen der Tabellen *fachbuch*, *fachbereichfachbuch* und *fachbereich* mit einem **INNER JOIN** verbunden abfragen, um eine Ergebnistabelle zu erhalten. In der Spaltenauswahlliste fordern Sie aus der Tabelle *fachbuch* die Spalte *titel* und aus der Tabelle *fachbereich* die gleichnamige Spalte *fachbereich* an.

Übung 7

Eine Bibliothek für Fachbücher ist natürlich nur sinnvoll, wenn die Fachbücher auch ausgeliehen werden können. In [Abbildung 10.14](#) sehen Sie, dass dem Datenmodell eine weitere Tabelle, *ausleihe*, hinzugefügt wurde.

Der Primärschlüssel der Tabelle setzt sich aus den Spalten *ausleihid* und *exemplarid* zusammen. Der Verleihzeitraum wird durch die Spalten *von* und *bis* abgebildet.

Um für diese Übung ein Beispiel zu erhalten, ist die Modellierung der Tabelle bewusst schlicht gehalten. Die Tabelle *ausleihe* enthält die Spalten *isbn* und *titel*, die auch in

der Tabelle *fachbuch* enthalten sind und somit redundant vorkommen. Die Spalte *fachbuchid* stellt eine Fremdschlüsselspalte dar, die wiederum auf die Primärschlüsselspalte *fachbuchid* der Tabelle *fachbuch* referenziert.

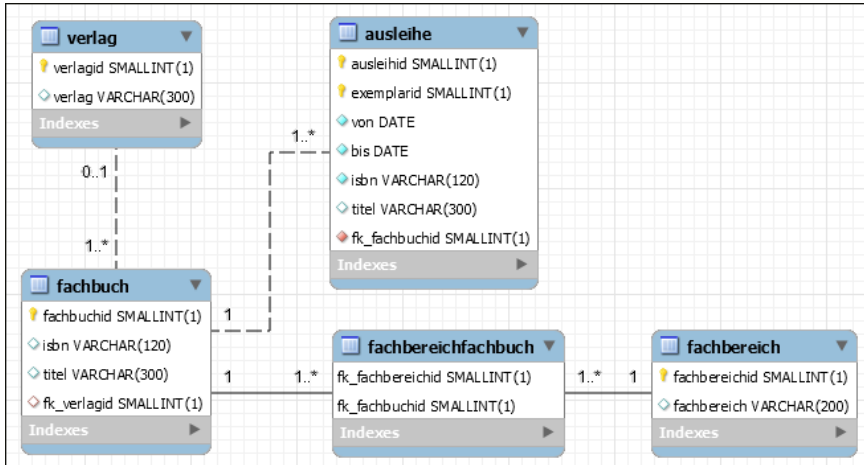


Abbildung 10.14 Das Datenmodell wurde um die Tabelle »ausleihe« erweitert.

Kommen wir jetzt zu der eigentlichen Übung: Die Spaltennamen *isbn*, *titel* und *fachbuchid* sind in beiden Tabellen identisch. Wenn Sie eine Abfrage mit einem **INNER JOIN** formulieren, wird die Datenbank die Spaltennamen nicht eindeutig einer Tabelle zuzuordnen können. Aus diesem Grund soll in dieser Übung ein **INNER JOIN** zwischen den Tabellen *ausleihe* und *fachbuch* formuliert werden, wobei Sie die Spalten der Tabellen jeweils über die Tabellennamen ansprechen.

Nehmen Sie aus der Tabelle *fachbuch* die Spalten *isbn* und *titel* mit in die Spaltenauswahlliste auf. Fragen Sie des Weiteren die gleichnamigen Spalten *isbn* und *titel* aus der Tabelle *ausleihe* über die Spaltenauswahlliste ab. Außerdem sollen die Spalten *von* und *bis* der Tabelle *ausleihe* enthalten sein.

In [Listing 10.36](#) ist die **CREATE TABLE**-Anweisung aufgeführt, die Sie benötigen, um die Tabelle *ausleihe* zu erstellen.

```
CREATE TABLE ausleihe (
    ausleihid SMALLINT, exemplarid SMALLINT,
    von DATE NOT NULL,
    bis DATE NOT NULL,
    isbn VARCHAR(120) NOT NULL,
    titel VARCHAR(300),
    fk_fachbuchid SMALLINT NOT NULL,
    PRIMARY KEY (ausleihid, exemplarid),
    CONSTRAINT fk_fachbuchid_fachbuch
```

```

FOREIGN KEY (fk_fachbuchid)
REFERENCES fachbuch (fachbuchid)
);

```

Listing 10.36 Die Tabelle »ausleihe« mit einer CREATE TABLE-Anweisung erstellen

Um eine Ergebnistabelle zu erhalten, fügen wir zunächst zwei Zeilen in die Tabelle *ausleihe* ein. [Listing 10.37](#) enthält zwei **INSERT**-Anweisungen, mit denen Sie Zeilen in die Tabelle *ausleihe* einfügen können:

```

INSERT INTO ausleihe (ausleihid,exemplarid,von,bis,isbn,titel,fk_fachbuchid)
VALUES (1,1,'2015-04-01','2015-05-01','9878','Elektrotechnik',2);

INSERT INTO ausleihe (ausleihid,exemplarid,von,bis,isbn,titel,fk_fachbuchid)
VALUES (2,3,'2015-05-10','2015-05-20','9878','Elektrotechnik',2);

```

Listing 10.37 Zeilen in die Tabelle »ausleihe« einfügen

Übung 8

In Übung 7 haben Sie einen **INNER JOIN** über die Tabellen *fachbuch* und *ausleihe* formuliert und die Spalten jeweils über die Tabellennamen referenziert. Verwenden Sie in dieser Übung Tabellenaliasse, um die jeweiligen Spalten der Tabellen *fachbuch* und *ausleihe* anzusprechen.

Übung 9

In [Abschnitt 10.2.10](#) haben Sie erfahren, dass Sie Zeilen aus einer Tabelle mit sich selbst verbunden abfragen können. Betrachten Sie hierzu [Tabelle 10.23](#), in der Fachbereiche in einer hierarchischen Struktur dargestellt werden.

fachbereichid	fachbereich	fk_fachbereich
1	Elektrotechnik	NULL
2	Kommunikationselektronik	1
3	Energieelektronik	1
4	Kaufmann	NULL
5	Industriekaufmann	4
6	Einzelhandelskaufmann	4
7	Mechanik	NULL

Tabelle 10.23 Zeilen, die auf Zeilen in der gleichen Tabelle referenzieren

fachbereichid	fachbereich	fk_fachbereich
8	Industriemechaniker	7
9	Feinmechaniker	7

Tabelle 10.23 Zeilen, die auf Zeilen in der gleichen Tabelle referenzieren (Forts.)

In [Listing 10.38](#) sehen Sie eine **CREATE TABLE**-Anweisung, mit der Sie die Tabelle *fachbereichshierarchie* erstellen:

```
CREATE TABLE fachbereichshierarchie (
    fachbereichid SMALLINT PRIMARY KEY,
    fachbereich VARCHAR(200),
    fk_fachbereich SMALLINT,
    CONSTRAINT fk_fachbereich_hierarchie
        FOREIGN KEY (fk_fachbereich)
        REFERENCES fachbereichshierarchie (fachbereichid)
);
```

Listing 10.38 Eine Tabelle erstellen, die mit einer Fremdschlüsselspalte auf die eigene Primärschlüsselspalte referenziert

Auch hier müssen Sie zuerst Zeilen einfügen, um sie anschließend mit einem **INNER JOIN** verbunden abfragen zu können. In [Listing 10.39](#) sind die **INSERT**-Anweisungen aufgeführt, mit denen Sie die benötigten Zeilen in die Tabelle *fachbereichshierarchie* einfügen:

```
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (1,'Elektrotechnik',NULL);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (2,'Kommunikationselektronik',1);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (3,'Energieelektronik',1);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (4,'Kaufmann',NULL);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (5,'Industrie Kaufmann',4);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (6,'Einzelhandelskaufmann',4);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (7,'Mechanik',NULL);
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (8,'Industriemechaniker',7);
```

```
INSERT INTO fachbereichshierarchie (fachbereichid,fachbereich,fk_fachbereich)
VALUES (9,'Feinmechaniker',7);
```

Listing 10.39 Zeilen in die Tabelle »fachbereichshierarchie« einfügen

Wenn Sie die Zeilen in die Tabelle *fachbereichshierarchie* eingefügt haben, formulieren Sie eine **SELECT**-Anweisung, in der Sie mit einem **INNER JOIN** die Zeilen der Tabelle *fachbereichshierarchie* wiederum mit Zeilen der Tabelle *fachbereichshierarchie* verbunden abfragen. Vergessen Sie nicht, die Tabellenaliasse zu verwenden. In die Spaltenauswahlliste nehmen Sie entsprechend pro Tabellenalias jeweils die Spalte *fachbereich* mit auf, um die Hierarchie der Fachbereiche abzubilden. Der Übersicht halber ordnen Sie der Spalte *fachbereich* jeweils einen sprechenden Spaltenalias zu.

Übung 10

In Abschnitt 10.2.11 haben Sie einen **INNER JOIN** kennengelernt, der keine Fremdschlüsselbeziehungen nutzt, um diese in einer **ON**-Klausel auszuwerten. In der Bedingung der **ON**-Klausel haben Sie Spalten aus zwei Tabellen verwendet, denen der gleiche Wertebereich (Datentyp) zugeordnet ist. Nutzen Sie in dieser Übung die Tabelle *ausleihe*, die wir in der Lösung zu Übung 7 erstellt haben, um diese Tabelle mit Zeilen der Tabelle *fachbuch* zu verbinden. Setzen Sie diesmal allerdings nicht die Fremdschlüsselbeziehungen ein. Nutzen Sie in der **ON**-Klausel die Spalte *isbn*, die in beiden Tabellen vorkommt, um eine Bedingung zu formulieren, die auf Gleichheit der Spaltenwerte prüft. Verwenden Sie auch hier Tabellenaliasse, um auf die jeweiligen Spalten der Tabellen zu referenzieren. In der Spaltenauswahlliste wählen Sie die Spalten *isbn* und *titel* der Tabelle *fachbuch* und die Spalten *von* und *bis* der Tabelle *ausleihe*.

Lösung zu Übung 1

```
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch
INNER JOIN verlag
ON fk_verlagid=verlagid;
```

Listing 10.40 Mit einem **INNER JOIN** Zeilen von zwei Tabellen verbunden abfragen

Lösung zu Übung 2

```
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch
LEFT OUTER JOIN verlag
ON fk_verlagid=verlagid;
```

Listing 10.41 Mit einem **LEFT OUTER JOIN** auch diejenigen Spaltenwerte der linken Tabelle ausgeben, die nicht mit Zeilen der rechten Tabelle verbunden werden konnten

Lösung zu Übung 3

```
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch  
RIGHT OUTER JOIN verlag  
ON fk_verlagid=verlagid;
```

Listing 10.42 Mit einem RIGHT OUTER JOIN auch diejenigen Spaltenwerte der rechten Tabelle ausgeben, auf die keine Zeilen der linken Tabelle referenzieren

Lösung zu Übung 4

```
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch  
FULL OUTER JOIN verlag  
ON fk_verlagid=verlagid;
```

Listing 10.43 Mit einem FULL OUTER JOIN alle Zeilen verbinden, die sich über Schlüsselwerte verbinden lassen, und diejenigen Zeilen der beteiligten Tabellen ausgeben, die sich nicht über Schlüsselwerte verbinden lassen

```
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch  
LEFT OUTER JOIN verlag  
ON fk_verlagid=verlagid  
UNION  
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch  
RIGHT OUTER JOIN verlag  
ON fk_verlagid=verlagid;
```

Listing 10.44 Einen FULL OUTER JOIN unter MySQL und MariaDB nachbilden

Lösung zu Übung 5

```
SELECT titel,isbn,fk_verlagid,verlag,verlagid FROM fachbuch  
CROSS JOIN verlag;
```

Listing 10.45 Alle Zeilen einer Tabelle mit sämtlichen Zeilen einer anderen Tabelle verbinden und umgekehrt

Lösung zu Übung 6

```
SELECT titel,fachbereich FROM fachbuch  
INNER JOIN  
fachbereichfachbuch  
ON fachbuchid=fk_fachbuchid
```

INNER JOIN

fachbereich

ON fk_fachbereichid=fachbereichid;

Listing 10.46 Einen INNER JOIN über drei Tabellen anwenden

Lösung zu Übung 7

SELECT

```
ausleihe.isbn,  
ausleihe.titel,  
fachbuch.isbn,  
fachbuch.titel,  
ausleihe.von,  
ausleihe.bis
```

FROM ausleihe INNER JOIN fachbuch

ON ausleihe.fk_fachbuchid=fachbuch.fachbuchid;

Listing 10.47 Die Spalten in einer SELECT-Abfrage mit dem Tabellennamen referenzieren

Lösung zu Übung 8

SELECT a.isbn,a.titel,f.isbn,f.titel,a.von,a.bis

FROM ausleihe a INNER JOIN fachbuch f

ON a.fk_fachbuchid=f.fachbuchid;

Listing 10.48 Die Spalten in einer SELECT-Abfrage mit Tabellenaliasen referenzieren

Lösung zu Übung 9

SELECT

f1.fachbereich AS bereich_uber,

f2.fachbereich AS bereich_unter

from fachbereichhierarchie f1

INNER JOIN fachbereichhierarchie f2

ON f1.fachbereichid=f2.fk_fachbereich;

Listing 10.49 Eine Tabelle mit einem INNER JOIN mit sich selbst verbunden abfragen

Lösung zu Übung 10

SELECT f.isbn,f.titel,a.von,a.bis

FROM fachbuch f INNER JOIN ausleihe a

ON f.isbn=a.isbn;

Listing 10.50 Zeilen von Tabellen über Nicht-Schlüsselwerte verbinden

10.3 Beziehungen (Schlüsselbeziehungen) ändern

Der Status einer Beziehung kann sich ändern. Das gilt auch für die Zeilen einer Kindtabelle, die mit einem Fremdschlüsselwert auf einen Primärschlüsselwert einer Zeile einer Elterntabelle verweisen. Hier gibt es nur eine klare Einschränkung, und zwar, dass nur gültige Beziehungen zwischen Zeilen existieren dürfen. In [Abschnitt 9.4](#) haben Sie die *referenzielle Integrität* kennengelernt, über die eine Datenbank sicherstellt, dass sie keine ungültigen Beziehungen zulässt. Bei der Beziehungsauswahl zwischen den Zeilen einer Kindtabelle und den Zeilen einer Elterntabelle müssen Sie also stets darauf achten, dass sie auch gültig ist.

In [Abschnitt 10.3.1](#) erfahren Sie, wie Sie Beziehungen zwischen Zeilen aus Eltern- und Kindtabellen ändern.

10.3.1 Beziehungen aus Zeilen aus einer Kindtabelle ändern

Beziehungen zu ändern, ist sehr einfach. Sie ändern einen Fremdschlüsselwert einer Kindtabelle in einen anderen Primärschlüsselwert einer Elterntabelle. Die Zeile in der Kindtabelle referenziert dann auf die andere Zeile der Elterntabelle.

Sehen wir uns hierzu ein Beispiel an. In [Abbildung 10.15](#) sehen Sie die Tabellen *auszubildender* und *adresse*.

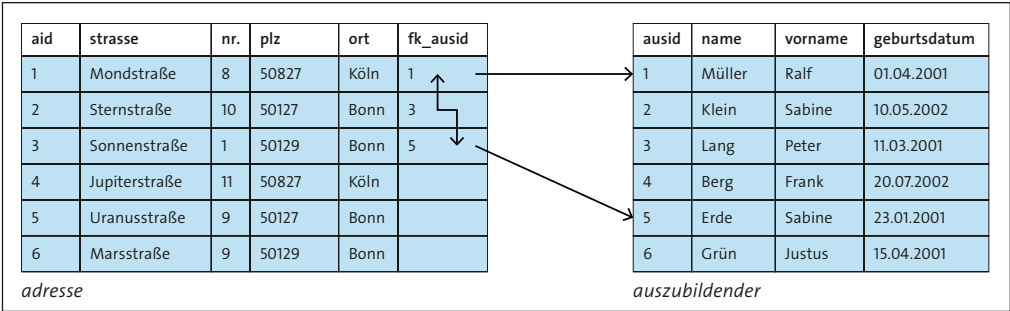


Abbildung 10.15 Beziehungen zwischen Zeilen in Kind- und Elterntabellen ändern

Die Zeilen der Tabellen stehen über eine Fremdschlüsselspalte in der Tabelle *adresse* in Beziehung zueinander. Auf die Auszubildende Sabine Erde mit dem Primärschlüsselwert 5 (der Spalte *ausid*) referenziert der Fremdschlüsselwert 5 der Spalte *fk_ausid* der Tabelle *adresse*. Dem Auszubildenden Ralf Müller mit dem Primärschlüsselwert 1 (der Spalte *ausid*) wird ebenfalls über einen Fremdschlüsselwert 1 aus der Tabelle *adresse* eine Adresse zugeordnet. Die beiden Auszubildenden tauschen nun die Wohnungen. Die Beziehungen der Adressen zu den Auszubildenden sind somit nicht mehr richtig. Die alten Beziehungen müssen also verändert werden, um die aktuellen Adressen den Auszubildenden zuordnen zu können.

Dazu verwenden Sie eine **UPDATE**-Anweisung wie in [Listing 10.51](#), um die Fremdschlüsselwerte der Spalte *fk_ausid* entsprechend zu aktualisieren:

```
UPDATE adresse SET fk_ausid=5 WHERE aid=1;
UPDATE adresse SET fk_ausid=1 WHERE aid=3;
```

Listing 10.51 Beziehungen zwischen Tabellen mit einer UPDATE-Anweisung ändern

Es sind also zwei **UPDATE**-Anweisungen vorhanden, die in der Tabelle *adresse* jeweils den Wert der Spalte *fk_ausid* auf die Fremdschlüsselwerte 5 bzw. 1 ändern. Wichtig ist die **WHERE**-Klausel, die jeweils für die **UPDATE**-Anweisungen genutzt wird, um sicherzustellen, dass der Fremdschlüsselwert der richtigen Adresszeile geändert wird. Es muss daher die Primärschlüsselspalte *aid* auf Gleichheit mit den jeweiligen Primärschlüsselwerten geprüft werden, um die richtigen Zeilen aus der Tabelle *adresse* mit der **UPDATE**-Anweisung zu aktualisieren.

Als Nächstes überprüfen wir mit einem einfachen **INNER JOIN**, ob die Beziehungen der Adressen zu den Auszubildenden tatsächlich geändert wurden. Mit der Abfrage aus [Listing 10.52](#) können wir die Beziehungsänderung nachvollziehen:

```
SELECT
    aus.ausid,
    aus.name,
    aus.vorname,
    adr.strasse,
    adr.nr,
    adr.plz,
    adr.ort,
    adr.aid
FROM auszubildender aus INNER JOIN adresse adr
ON aus.ausid=adr.fk_ausid;
```

Listing 10.52 Mit einem INNER JOIN überprüfen, ob die Beziehungen zwischen den Zeilen der Tabellen »adresse« und »auszubildender« geändert wurden

Die Ergebnistabelle zeigt, dass die Beziehung der Adresse verändert worden ist (siehe [Tabelle 10.24](#)).

ausid	name	vorname	strasse	nr	plz	ort	aid
1	Müller	Ralf	Sonnenstraße	1	50129	Bonn	3
3	Lang	Peter	Sternstraße	10	50127	Bonn	2
5	Erde	Sabine	Mondstraße	8	50827	Köln	1

Tabelle 10.24 Ergebnis der Beziehungsänderung

Dem Auszubildenden Ralf Müller ist jetzt die Adresse mit dem Primärschlüsselwert 3 der Spalte *aid* der Tabelle *adresse* zugeordnet. Der Auszubildenden Sabine Erde hingegen wurde jetzt die Adresse mit dem Primärschlüsselwert 1 der Spalte *aid* der Tabelle *adresse* zugeordnet. Somit haben wir bestätigt, dass die Beziehungsänderung erfolgreich durchgeführt wurde.

10.3.2 Beziehungen aus Zeilen einer Elterntabelle ändern (ON UPDATE CASCADE)

In [Abschnitt 10.3.1](#) haben Sie Beziehungen geändert, bei denen die Schlüsselwerte von Zeilen einer Kindtabelle auf existierende Primärschlüsselwerte verwiesen. In diesem Abschnitt erfahren Sie, wie Sie Primärschlüsselwerte in einer Elterntabelle ändern. Diese Änderung muss dann an den Zeilen in der Kindtabelle, die auf diesen Primärschlüsselwert referenzieren, nachvollzogen werden, um nicht die referenzielle Integrität zu verletzen.

Sie haben die folgenden zwei Möglichkeiten, Primärschlüsselwerte aus Elterntabellen zu ändern, die auf die Zeilen aus Kindtabellen referenzieren.

Eine Möglichkeit besteht darin, die Fremdschlüsselwerte, die auf die zu ändernden Primärschlüssel der Elterntabelle zeigen, in der Kindtabelle auf **NULL** zu setzen. Somit wäre die Beziehung der betroffenen Zeilen aufgehoben. Beachten Sie, dass ein Primärschlüsselwert in einer Elterntabelle erst dann verändert werden kann, wenn kein Fremdschlüsselwert aus einer Kindtabelle auf den zu ändernden Primärschlüsselwert referenziert. Hierbei handelt es sich um die in [Abschnitt 9.4](#) beschriebene referenzielle Integrität, die von einem relationalen Datenbanksystem garantiert wird. Die referenzielle Integrität stellt sicher, dass ein Primärschlüsselwert nicht verändert werden kann, solange Fremdschlüsselwerte in Beziehung zu ihm stehen.

In der Praxis wird so eine Änderung eher selten durchgeführt. Stellen Sie sich ein kompliziertes Datenmodell vor, in dem aus unterschiedlichen Kindtabellen Fremdschlüsselwerte auf einen Primärschlüsselwert in einer Elterntabelle referenzieren. Sämtliche Änderungen des Primärschlüsselwertes müssten an jeder Stelle nachvollzogen werden.

Wenn absehbar ist, dass sich Primärschlüsselwerte ändern, ist es daher sinnvoll, bereits während der Tabellendefinition einer Kindtabelle mit einer **CREATE TABLE**-Anweisung für diesen Fall vorzusorgen. In der Kindtabelle muss hierzu die Fremdschlüsseleinschränkung (**CONSTRAINT**) um die **ON UPDATE CASCADE**-Klausel erweitert werden.

Die **ON UPDATE CASCADE**-Klausel bewirkt für jede Änderung eines Primärschlüsselwertes in einer Elterntabelle die automatische Änderung der Fremdschlüsselwerte in der Kindtabelle. Das ist ziemlich praktisch, wenn Sie tatsächlich in die Situation kommen, dass Primärschlüsselwerte geändert werden müssen. Dies könnte dann der Fall

sein, wenn sich der Primärschlüsselwert eines Auszubildenden verändert, weil sich z. B. die Personalnummer ändert.

Wir legen die Tabellen *auszubildender* und *adresse* unter den Namen *auszubildender_neu* und *adresse_neu* an, um die bestehenden Tabellen und ihre Datensätze zu erhalten. Erstellen wir also zuerst eine neue Tabelle *auszubildender_neu*. Die **CREATE TABLE**-Anweisung aus [Listing 10.53](#) enthält nichts Neues. Wie wir bereits in der Modellierungsphase festgestellt haben, handelt es sich um eine starke Entität. Das bedeutet, dass die Tabelle *auszubildender_neu* zu keiner anderen Tabelle in einer Abhängigkeit steht.

```
CREATE TABLE auszubildender_neu (  
    ausid SMALLINT PRIMARY KEY NOT NULL,  
    name VARCHAR(120),  
    vorname VARCHAR(120),  
    geburtsdatum DATE  
);
```

Listing 10.53 Die Tabelle »auszubildender_neu« mit einer **CREATE TABLE**-Anweisung anlegen

Anschließend fügen wir mit den **INSERT**-Anweisungen aus [Listing 10.54](#) neue Zeilen in die Tabelle *auszubildender_neu* ein:

```
INSERT INTO auszubildender_neu (ausid,name,vorname,geburtsdatum)  
    VALUES (1,'Müller','Ralf','2001-04-01');  
  
INSERT INTO auszubildender_neu (ausid,name,vorname,geburtsdatum)  
    VALUES (2,'Klein','Sabine','2002-05-10');  
  
INSERT INTO auszubildender_neu (ausid,name,vorname,geburtsdatum)  
    VALUES (3,'Lang','Peter','2001-03-11');  
  
INSERT INTO auszubildender_neu (ausid,name,vorname,geburtsdatum)  
    VALUES (4,'Berg','Frank','2002-07-20');  
  
INSERT INTO auszubildender_neu (ausid,name,vorname,geburtsdatum)  
    VALUES (5,'Erde','Sabine','2001-01-23');  
  
INSERT INTO auszubildender_neu (ausid,name,vorname,geburtsdatum)  
    VALUES (6,'Grün','Justus','2001-04-15');
```

Listing 10.54 Zeilen in die Tabelle »auszubildender_neu« einfügen

Die **ON UPDATE CASCADE**-Klausel sehen Sie in [Listing 10.55](#):

```
CREATE TABLE adresse_neu (  
    aid SMALLINT PRIMARY KEY NOT NULL,  
    strasse VARCHAR(200),  
    nr VARCHAR(5),  
    plz INT,  
    ort VARCHAR(200),  
    fk_ausid SMALLINT,  
    CONSTRAINT fk_adresse_auszubildender_neu  
        FOREIGN KEY (fk_ausid)  
        REFERENCES auszubildender_neu (ausid)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE  
);
```

Listing 10.55 Eine Fremdschlüsseldefinition um eine **ON UPDATE CASCADE**-Klausel ergänzen

Die **ON UPDATE CASCADE**-Klausel geben Sie hinter der **REFERENCES**-Klausel an, in der Sie festlegen, auf welche Tabelle und Primärschlüsselspalte Sie referenzieren. Durch diese Ergänzung legen Sie bereits während der Tabellenerstellung fest, dass die Fremdschlüsselwerte in der Tabelle *adresse_neu* automatisch angepasst werden, wenn sich die Primärschlüsselwerte in Zeilen der Tabelle *auszubildender_neu* verändern. Direkt hinter der **ON UPDATE CASCADE**-Klausel habe ich hier noch die **ON DELETE CASCADE**-Klausel notiert, um auch gleich auf den nächsten Abschnitt vorbereitet zu sein.

Als Nächstes müssen wir wieder Zeilen in die Tabelle *adresse_neu* einfügen. Hierzu nutzen wir die **INSERT**-Anweisungen aus [Listing 10.56](#):

```
/* Mit Zuordnung zu Auszubildenden */  
INSERT INTO adresse_neu (aid,strasse,nr,plz,ort,fk_ausid)  
    VALUES (1,'Mondstraße','8',50827,'Köln',1);  
  
INSERT INTO adresse_neu (aid,strasse,nr,plz,ort,fk_ausid)  
    VALUES (2,'Sternstraße','10',50127,'Bonn',3);  
  
INSERT INTO adresse_neu (aid,strasse,nr,plz,ort,fk_ausid)  
    VALUES (3,'Sonnenstraße','1',50129,'Bonn',5);
```

Listing 10.56 Zeilen in die Tabelle »adresse_neu« einfügen

In [Tabelle 10.25](#) werden lediglich die Zeilen der Auszubildenden dargestellt, denen eine Adresse zugeordnet ist. In der Spalte *ausid* sehen Sie, dass die drei Primärschlüsselwerte 1, 3 und 5 enthalten sind:

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	2001-04-01
3	Lang	Peter	2001-03-11
5	Erde	Sabine	2001-01-23

Tabelle 10.25 Auszubildende, denen eine Adresse zugeordnet ist

[Tabelle 10.26](#) enthält drei Zeilen, die in der Spalte *fk_ausid* Fremdschlüsselwerte enthalten, die auf die drei Primärschlüsselwerte 1, 3 und 5 der Tabelle *auszubildender* referenzieren:

aid	strasse	nr	plz	ort	fk_ausid
1	Mondstraße	8	50827	Köln	1
2	Sternstraße	10	50127	Bonn	3
3	Sonnenstraße	1	50129	Bonn	5

Tabelle 10.26 Adressen, die Auszubildenden zugeordnet sind

Die Primärschlüsselwerte 1, 3 und 5 der Tabelle *auszubildender_neu* aktualisieren wir im nächsten Schritt mit den **UPDATE**-Anweisungen aus [Listing 10.57](#) auf die Werte 10, 30 und 50, um die automatische Änderung der Fremdschlüsselwerte in der Tabelle *adresse_neu* nachzuvollziehen:

```
UPDATE auszubildender_neu SET ausid=10 WHERE ausid=1;
UPDATE auszubildender_neu SET ausid=30 WHERE ausid=3;
UPDATE auszubildender_neu SET ausid=50 WHERE ausid=5;
```

Listing 10.57 Primärschlüsselwerte mit einer UPDATE-Anweisung ändern

In [Tabelle 10.27](#) sind die aktualisierten Zeilen (mit geänderten Primärschlüsselwerten) dargestellt:

ausid	name	vorname	geburtsdatum
10	Müller	Ralf	2001-04-01
30	Lang	Peter	2001-03-11
50	Erde	Sabine	2001-01-23

Tabelle 10.27 Die Tabelle »auszubildender_neu« nach der Aktualisierung der Primärschlüsselwerte

Die **ON UPDATE CASCADE**-Klausel, die in der **CREATE TABLE**-Anweisung aus [Listing 10.55](#) für die Fremdschlüsselbeziehung der Tabelle *adresse_neu* festgelegt wurde, stellt sicher, dass die Fremdschlüsselwerte in der Tabelle *adresse_neu* ebenfalls geändert werden.

In [Tabelle 10.28](#) sehen Sie das Ergebnis der automatischen Aktualisierung der Fremdschlüsselwerte in der Spalte *fk_ausid*.

aid	strasse	nr	plz	ort	fk_ausid
1	Mondstraße	8	50827	Köln	10
2	Sternstraße	10	50127	Bonn	30
3	Sonnenstraße	1	50129	Bonn	50

Tabelle 10.28 Die Tabelle »adresse_neu« nach der Aktualisierung der Primärschlüsselwerte der Spalte »ausid« der Tabelle »auszubildender_neu«



Zusammenfassung: Schlüsselbeziehungen ändern

Beziehungen aus Zeilen in Kindtabellen zu Zeilen in Elterntabellen können Sie verändern, indem Sie aus einer Zeile einer Kindtabelle mit einem anderen Fremdschlüsselwert auf einen anderen gültigen Primärschlüsselwert verweisen.

Primärschlüsselwerte aus Elterntabellen können nur verändert werden, wenn in der Kindtabelle während der Tabellenerstellung eine **CASCADE UPDATE**-Klausel verwendet wurde.

10.3.3 Übungen

Übung 1

In Übung 7 aus [Abschnitt 10.2.12](#) haben Sie die Tabelle *ausleihe* angelegt und zwei Datensätze eingefügt. In [Tabelle 10.29](#) sehen Sie einen Auszug der verfügbaren Spalten der Tabelle *ausleihe*. Beide Ausleihvorgänge beziehen sich auf den Fremdschlüssel-

wert 2 der Spalte *fachbuchid* der Tabelle *fachbuch*. In [Tabelle 10.30](#) sehen Sie ebenfalls einen Auszug des Spaltenvorrats der Tabelle *fachbuch*. Hier sind zwei Zeilen enthalten, als Erstes die Zeile mit dem Primärschlüsselwert 2 der Spalte *fachbuchid*, auf den ein Fremdschlüsselwert 2 der Spalte *fk_fachbuchid* der Tabelle *ausleihe* referenziert. Hierbei handelt es sich also um eine gültige Beziehung zwischen einer Kind- und einer Elterntabelle.

Ändern Sie die Beziehungen aus der Tabelle *ausleihe*, die jeweils auf ein Fachbuch mit dem Primärschlüsselwert 2 der Tabelle *fachbuch* referenzieren. Stellen Sie jeweils aus der Tabelle *ausleihe* eine Beziehung zu der Zeile mit dem Primärschlüsselwert 3 der Primärschlüsselspalte *fachbuchid* der Tabelle *fachbuch* her, um die Ausleihvorgänge zu korrigieren.

Vergessen Sie nicht, dass Sie mit einer Prüfabfrage die Änderung der Schlüsselwerte nachvollziehen können.

ausleihid	exemplarid	isbn	titel	fk_fachbuchid
1	1	9878	Elektrotechnik	2
2	3	9878	Elektrotechnik	2

Tabelle 10.29 Auszug der Spalten aus der Tabelle »ausleihe«

fachbuchid	isbn	titel
2	9878	Elektrotechnik
3	2323	Elektronik

Tabelle 10.30 Auszug der Spalten der Tabelle »fachbuch«

Übung 2

In dieser Übung ändern Sie die Beziehung zu Zeilen aus einer Kindtabelle von einer Elterntabelle ausgehend. Sie müssen also die Primärschlüsselwerte ändern, auf die sich Zeilen aus einer Kindtabelle beziehen. Der Fremdschlüsselwert soll automatisch in der Kindtabelle angepasst werden, wenn in der Elterntabelle ein Primärschlüsselwert geändert wird.

Zunächst benötigen Sie dazu eine Elterntabelle mit Zeilen, auf die Sie aus einer Kindtabelle referenzieren, sowie eine Kindtabelle, aus der Sie auf die Elterntabelle referenzieren. Für diese Übung verwenden wir die Tabellen *ausgemustert* und *grund* (siehe [Abbildung 10.16](#)).

Erstellen Sie diese beiden Tabellen zuerst. Achten Sie darauf, dass Sie für die Fremdschlüsseldefinition eine **ON UPDATE**-Klausel mit einfügen.

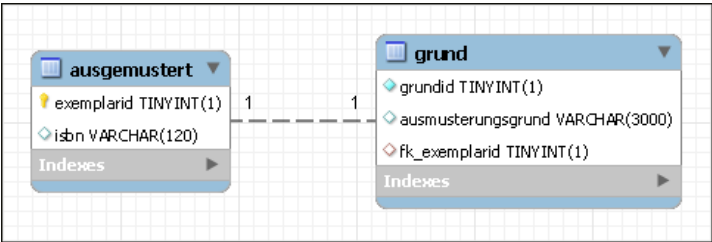


Abbildung 10.16 UML-Modell für die Tabellen »ausgemustert« und »grund« und ihre Beziehung zueinander

Als Nächstes müssen Sie Zeilen in die neu erstellten Tabellen einfügen, um Primärschlüsselwerte zu ändern, auf die aus der Kindtabelle *grund* referenziert wird. Fügen Sie daher zuerst die Zeilen aus [Tabelle 10.31](#) mit **INSERT**-Anweisungen in die Tabelle *ausgemustert* ein. Somit verfügen Sie über Zeilen in der Elterntabelle *ausgemustert* und können jetzt Zeilen in die Tabelle *grund* einfügen, die auf die Primärschlüsselwerte der Tabelle *ausgemustert* verweisen.

[Tabelle 10.32](#) enthält ebenfalls vier Zeilen, die Sie mit **INSERT**-Anweisungen in die Tabelle *grund* einfügen.

exemplarid	isbn
28	8976
29	8976
5	3456
6	3456

Tabelle 10.31 Datensätze für die Tabelle »ausgemustert«

grundid	ausmusterungsgrund	fk_exemplarid
1	beschädigt	28
2	verschmutzt	29
3	veraltet	5
4	veraltet	6

Tabelle 10.32 Datensätze für die Tabelle »grund«

Wenn Sie hier angekommen sind, haben Sie schon sehr viel erreicht. Ändern Sie im nächsten Schritt einen Primärschlüsselwert (z. B. den Wert 28 in 30) in der Eltern-

tabelle *ausgemustert*, auf die sich eine Zeile (z. B. mit dem Primärschlüsselwert 1 in der Tabelle *grund*) in der Kindtabelle bezieht. Verwenden Sie die Anweisung **UPDATE**, um die Aktualisierung durchzuführen. Achten Sie darauf, eine **WHERE**-Klausel mit einer entsprechenden Bedingung zu verwenden.

Gleich im Anschluss formulieren Sie Prüfabfragen, um die Änderung des Primärschlüsselwertes in der Tabelle *ausgemustert* in der Tabelle *grund* nachzuvollziehen.

Lösung zu Übung 1

```
/*
Die Beziehungen aus einer Kindtabelle mit einer UPDATE-Anweisung ändern
*/
UPDATE ausleihe SET fk_fachbuchid=3 WHERE fk_fachbuchid=2;

/*
Die Änderung mit einem INNER JOIN nachvollziehen

Achtung, mit der UPDATE-Anweisung wurden nur die Schlüsselbeziehungen der
Kindtabelle "ausleihe" geändert.
Die Spaltenwerte der Spalte "isbn" der Tabelle "ausleihe" müssten ggf.
auch noch aktualisiert werden.
*/
SELECT a.isbn,a.titel,f.isbn,f.titel,a.von,a.bis
FROM ausleihe a INNER JOIN fachbuch f
ON a.fk_fachbuchid=f.fachbuchid;
```

Listing 10.58 Beziehungen aus Zeilen von Kindtabellen zu Zeilen in Elterntabellen ändern

Lösung zu Übung 2

Als Erstes legen Sie die Tabelle *ausgemustert* an und fügen Zeilen in die Tabelle *ausgemustert* ein, wie in [Listing 10.59](#) und [Listing 10.60](#) dargestellt.

```
CREATE TABLE ausgemustert (
    exemplarid SMALLINT NOT NULL,
    isbn VARCHAR(120),
    PRIMARY KEY (exemplarid)
);
```

Listing 10.59 Die Elterntabelle »ausgemustert« mit einer CREATE TABLE-Anweisung anlegen

Im nächsten Schritt werden vier Zeilen in die Tabelle *ausgemustert* eingefügt:

```
INSERT INTO ausgemustert (exemplarid,isbn) VALUES (28,'8976');
INSERT INTO ausgemustert (exemplarid,isbn) VALUES (29,'8976');
INSERT INTO ausgemustert (exemplarid,isbn) VALUES (5,'3456');
INSERT INTO ausgemustert (exemplarid,isbn) VALUES (6,'3456');
```

Listing 10.60 Datensätze in die Tabelle »ausgemustert« einfügen

Als Nächstes legen Sie die Kindtabelle *grund* mit einer **CREATE TABLE**-Anweisung wie in [Listing 10.61](#) dargestellt an. In der Fremdschlüsseldefinition verwenden Sie die **ON UPDATE CASCADE**-Klausel, um die Fremdschlüsselwerte der Spalte *fk_exemplarid* der Tabelle *grund* automatisch anzupassen, wenn Primärschlüsselwerte in der Tabelle *ausgemustert* geändert werden.

Um die automatische kaskadierte Änderung von Fremdschlüsselwerten nachzuvollziehen, benötigen wir auch hier Datensätze. In [Listing 10.62](#) sind vier **INSERT**-Anweisungen aufgeführt, die in Beziehung zu Zeilen der Tabelle *ausgemustert* stehen:

```
CREATE TABLE grund (
    grundid SMALLINT NOT NULL,
    ausmusterungsgrund VARCHAR(3000),
    fk_exemplarid SMALLINT, PRIMARY KEY (grundid),
    CONSTRAINT fk_exemplarid
        FOREIGN KEY (fk_exemplarid)
            REFERENCES ausgemustert (exemplarid)
            ON UPDATE CASCADE
);
```

Listing 10.61 Die Kindtabelle »grund« mit einer **CREATE TABLE**-Anweisung anlegen

```
INSERT INTO grund (grundid,ausmusterungsgrund,fk_exemplarid) VALUES
(1,'beschädigt',28);
INSERT INTO grund (grundid,ausmusterungsgrund,fk_exemplarid) VALUES
(2,'verschmutzt',29);
INSERT INTO grund (grundid,ausmusterungsgrund,fk_exemplarid) VALUES
(3,'veraltet',5);
INSERT INTO grund (grundid,ausmusterungsgrund,fk_exemplarid) VALUES
(4,'veraltet',6);
```

Listing 10.62 Datensätze in die Tabelle »grund« einfügen

Die **ON UPDATE CASCADE**-Klausel, die Sie für die Fremdschlüsseldefinition der Tabelle *grund* verwendet haben, greift automatisch bei jeder Aktualisierungsanweisung, die für Primärschlüsselwerte der Tabelle *ausgemustert* angewendet wird. In [Listing 10.63](#) sehen Sie eine Lösung, mit der Sie einen Primärschlüsselwert der Spalte *exemplarid* der Tabelle *ausgemustert* in den Wert 30 ändern. Mit der **WHERE**-Klausel wird sicherge-

stellt, dass ausschließlich der Primärschlüsselwert der Zeile mit dem Primärschlüsselwert 28 in 30 verändert wird.

```
UPDATE ausgemustert SET exemplarid=30 WHERE exemplarid=28;
```

Listing 10.63 Einen Primärschlüsselwert in einer Elterntabelle aktualisieren

Um zu überprüfen, ob die Änderung des Primärschlüsselwerts in den Tabellen *ausgemustert* und *grund* erfolgt ist, verwenden Sie zwei **SELECT**-Anweisungen, wie sie in [Listing 10.64](#) dargestellt sind:

```
SELECT * FROM ausgemustert;
SELECT * FROM grund;
```

Listing 10.64 Prüfabfragen verwenden, um die Aktualisierung nachzuvollziehen

10.4 Beziehungen (Schlüsselbeziehungen) aufheben oder löschen

In diesem Abschnitt erfahren Sie, wie Sie Beziehungen aufheben oder löschen, die aus Zeilen einer Kindtabelle hervorgehen. Das könnte dann erforderlich werden, wenn ein Auszubildender umzieht und die Beziehung zwischen der alten Adresse und dem Auszubildenden nicht mehr benötigt wird. Das setzt natürlich voraus, dass Sie bei einer Adressänderung einen neuen Datensatz in die Tabelle *adresse* einfügen. Wie gehen wir mit dem nun überflüssig gewordenen Adresseintrag um? In [Abschnitt 10.4.1](#) und [Abschnitt 10.4.2](#) zeige ich Ihnen zwei Möglichkeiten, eine Beziehung aufzuheben oder zu löschen.

In [Abschnitt 10.4.3](#) zeige ich Ihnen dann noch, wie Sie Zeilen aus einer Elterntabelle löschen, auf die Zeilen aus einer Kindtabelle mit Fremdschlüsselwerten referenzieren. Das wird dann erforderlich sein, wenn zwei Jahre nach Abschluss der Ausbildung die Daten der Auszubildenden archiviert und aus der aktuellen Tabelle entfernt werden.

Wie Sie sich denken können, stellt bei den beschriebenen Methoden zum Aufheben oder Löschen von Beziehungen auch hier die referenzielle Integrität sicher, dass keine bestehenden Beziehungen verletzt werden. Im nächsten Abschnitt beginnen wir damit, Beziehungen aus Kindtabellen zu entfernen, ohne gleich die ganzen Zeilen löschen zu müssen.

10.4.1 Zeilen aus Kindtabellen auf NULL setzen

Beziehungen zwischen Zeilen aus Kindtabellen, deren Fremdschlüsselwerte auf Primärschlüsselwerte aus Zeilen einer Elterntabelle verweisen, können Sie sehr einfach aufheben. Sie müssen lediglich den Fremdschlüsselwert in der betroffenen Zeile der Kindtabelle auf **NULL** setzen. Das setzt allerdings voraus, dass in der **CREATE TABLE**-

Anweisung der Kindtabelle für die Fremdschlüsselspalte kein **NOT NULL**-Constraint festgelegt wurde. Wenn ein **NOT NULL**-Constraint verwendet wird, können Sie bestehende Fremdschlüsselwerte dieser Spalte nicht auf einen **NULL**-Wert setzen, um somit eine Beziehung aufzuheben. Fremdschlüsselwerte können auch nicht auf **NULL** gesetzt werden, wenn sie Bestandteil eines Primärschlüssels sind. Denken Sie an den identifizierenden Beziehungstyp (Identifying Relationship), bei dem genau dies der Fall ist.

Wir nutzen wieder die Tabellen *auszubildender_neu* und *adresse_neu*, die wir in [Abschnitt 10.3.2](#) angelegt haben. Mit einem **INNER JOIN** prüfen wir dabei zunächst wie in [Listing 10.65](#) gezeigt, welche Adressen welchen Auszubildenden zugeordnet sind. Der Übersicht halber wählen wir aus der Tabelle *adresse_neu* nur die Spalten *strasse* und *nr* aus:

SELECT

```
aus.ausid,  
aus.name,  
aus.vorname,  
adr.aid,  
adr.strasse,  
adr.nr,  
adr.fk_ausid
```

FROM

```
auszubildender_neu aus INNER JOIN adresse_neu adr  
ON aus.ausid=adr.fk_ausid;
```

Listing 10.65 Die Tabellen »auszubildender_neu« und »adresse_neu« mit einem **INNER JOIN** verbunden abfragen

In [Tabelle 10.33](#) sehen Sie das Ergebnis der Abfrage aus [Listing 10.65](#):

ausid	name	vorname	aid	strasse	nr	fk_ausid
10	Müller	Ralf	1	Mondstraße	8	10
30	Lang	Peter	2	Sternstraße	10	30
50	Erde	Sabine	3	Sonnenstraße	1	50

Tabelle 10.33 Ergebnistabelle für eine mit einem **INNER JOIN** verbundene Abfrage auf die Tabellen »auszubildender_neu« und »adresse_neu«

Dem Auszubildenden Ralf Müller mit dem Primärschlüsselwert 10 in der Spalte *ausid* ist eine Adresse mit dem Primärschlüsselwert 1 in der Spalte *aid* zugeordnet. Die Fremdschlüsselspalte *fk_ausid* mit dem Wert 10 referenziert auf den Primärschlüsselwert 10 in der Spalte *ausid*. Die Beziehung zu der Adresse soll als Nächstes aufgehoben

werden, sodass keine Beziehung mehr zwischen dem Auszubildenden Ralf Müller und einer Adresse besteht.

Um das zu erreichen, ordnen wir dem Fremdschlüsselwert der Spalte *fk_ausid* einen **NULL**-Wert zu. Die Adresse, deren Beziehung zum Auszubildenden Ralf Müller aufgehoben werden soll, hat in der Spalte *aid* den Primärschlüsselwert 1. Somit haben wir sämtliche Angaben, um mit einer **UPDATE**-Anweisung die Beziehung zum Auszubildenden Ralf Müller aufzuheben:

```
UPDATE adresse_neu SET fk_ausid=NULL WHERE aid=1;
```

Listing 10.66 Eine Beziehung aufheben, indem der Fremdschlüsselwert auf **NULL** gesetzt wird

Um das Ergebnis der Aufhebung der Beziehung zu überprüfen, ändern wir die Abfrage aus [Listing 10.65](#). Diesmal sollen alle Zeilen der Tabellen *auszubildender_neu* und *adresse_neu* abgefragt werden, die mit einer Übereinstimmung der Schlüsselwerte verbunden werden können. Außerdem sollen die Auszubildenden ausgegeben werden, denen keine Adresse zugeordnet werden kann. Sie denken bestimmt schon an den **LEFT OUTER JOIN**, den ich in [Abschnitt 10.2.2](#) erläutert habe. Richtig, in [Listing 10.67](#) sehen Sie die veränderte Abfrage, die jetzt mit einem **LEFT OUTER JOIN** die Zeilen der Tabellen *auszubildender_neu* und *adresse_neu* verbunden abfragt:

SELECT

```
aus.ausid,  
aus.name,  
aus.vorname,  
adr.aid,  
adr.strasse,  
adr.nr,  
adr.fk_ausid
```

FROM

```
auszubildender_neu aus LEFT OUTER JOIN adresse_neu adr  
ON aus.ausid=adr.fk_ausid;
```

Listing 10.67 Auszubildende mit und ohne eine Beziehung zu einer Zeile aus der Tabelle »adresse_neu« ausgegeben

In [Tabelle 10.34](#) sehen Sie das Ergebnis der Abfrage. Dem Auszubildenden Ralf Müller ist, wie in der letzten Zeile ersichtlich, keine Adresse mehr zugeordnet. Die Beziehung der Zeile mit der Adresse wurde also aufgehoben.

ausid	name	vorname	aid	strasse	nr	fk_ausid
30	Lang	Peter	2	Sternstraße	10	30
50	Erde	Sabine	3	Sonnenstraße	1	50
2	Klein	Sabine	NULL	NULL	NULL	NULL
4	Berg	Frank	NULL	NULL	NULL	NULL
6	Grün	Justus	NULL	NULL	NULL	NULL
10	Müller	Ralf	NULL	NULL	NULL	NULL

Tabelle 10.34 Ergebnistabelle, nachdem eine Adressbeziehung zum Auszubildenden Ralf Müller aufgehoben wurde

10.4.2 Zeilen aus Kindtabellen löschen

Die Beziehung zu einer Zeile aus einer Kindtabelle wird natürlich aufgehoben, wenn die Zeile einfach gelöscht wird. Voraussetzung dafür ist allerdings, dass aus keiner dritten Tabelle mehr eine Beziehung zu den Zeilen in der Kindtabelle existiert. In [Tabelle 10.35](#) sind noch zwei Adressen enthalten, die mit Fremdschlüsselwerten auf Auszubildende referenzieren.

aid	strasse	nr	plz	ort	fk_ausid
2	Sternstraße	10	50127	Bonn	30
3	Sonnenstraße	1	50129	Bonn	50

Tabelle 10.35 Adressen, die noch in Beziehung zu Auszubildenden stehen

[Tabelle 10.36](#) zeigt Ihnen, dass es sich um die Auszubildenden Peter Lang und Sabine Erde handelt.

ausid	name	vorname	geburtsdatum
30	Lang	Peter	2001-03-11
50	Erde	Sabine	2001-01-23

Tabelle 10.36 Auszubildende, denen mit einer Fremdschlüsselbeziehung eine Adresse aus der Tabelle »adresse_neu« zugeordnet ist

Als Nächstes löschen wir die Adresse mit dem Fremdschlüsselwert 30 der Spalte *fk_ausid* der Tabelle *adresse_neu*. Der Adresse, die gelöscht werden soll, ist in der

Spalte *aid* der Primärschlüsselwert 2 zugeordnet. In [Listing 10.68](#) sehen Sie die **DELETE**-Anweisung mit der benötigten **WHERE**-Klausel, die die Zeile löscht und so die Beziehung zu dem Auszubildenden mit dem Primärschlüsselwert 30 der Spalte *ausid* aufhebt:

```
DELETE FROM adresse_neu WHERE aid=2;
```

Listing 10.68 Eine Adresse löschen, die in Beziehung zu einer Zeile der Tabelle »auszubildender_neu« steht

In [Tabelle 10.37](#) sehen Sie das Ergebnis der **DELETE**-Anweisung. Hier ist nur noch dem Auszubildenden mit dem Primärschlüsselwert 3 eine Adresse zugeordnet.

aid	strasse	nr	plz	ort	fk_ausid
3	Sonnenstraße	1	50129	Bonn	50

Tabelle 10.37 Ergebnistabelle nach dem Löschen einer Kindzeile

In diesem Abschnitt haben Sie gelernt, wie Sie Zeilen, die in Beziehung zu Elterntabellen stehen, löschen, um so die Beziehung zu einer Zeile einer Elterntabelle aufzuheben. Im nächsten Abschnitt erfahren Sie, wie Sie Zeilen aus Elterntabellen löschen, auf die noch aus Zeilen von Kindtabellen referenziert wird.

10.4.3 Zeilen aus Elterntabellen löschen

Beziehungen zwischen Schlüsselwerten aus Zeilen in Elterntabellen, auf deren Primärschlüssel über einen Fremdschlüssel aus einer Kindtabelle referenziert wird, können Sie ebenfalls aufheben. Dies bedingt allerdings, dass in der Kindtabelle während der Tabellenerstellung für die Fremdschlüsseldefinition eine **ON DELETE CASCADE**-Klausel festgelegt wurde. Dann haben Sie die Möglichkeit, eine Zeile aus einer Elterntabelle zu entfernen, ohne zuvor die Zeilen aus einer Kindtabelle zu löschen, die auf die Zeilen der Elterntabelle referenzieren. Die Zeilen aus der Kindtabelle, die auf die Zeilen der Elterntabelle referenzieren, werden dann automatisch ebenfalls entfernt.

In der **CREATE TABLE**-Anweisung aus [Listing 10.55](#) haben wir bereits die **ON DELETE CASCADE**-Klausel berücksichtigt, als Sie die **ON UPDATE CASCADE**-Klausel kennengelernt haben. Es sind daher alle Voraussetzungen gegeben, um eine Zeile aus der Elterntabelle *auszubildender_neu* zu löschen und dabei sicherzustellen, dass automatisch alle Kindzeilen aus der Tabelle *adresse_neu* ebenfalls gelöscht werden.

In [Tabelle 10.38](#) sehen Sie einen Auszug der derzeit gespeicherten Auszubildenden in der Tabelle *auszubildende_neu*.

ausid	name	vorname	geburtsdatum
30	Lang	Peter	2001-03-11
50	Erde	Sabine	2001-01-23

Tabelle 10.38 Auszubildende in der Tabelle »auszubildender_neu«

In [Tabelle 10.39](#) sehen Sie eine Zeile, die mit dem Fremdschlüsselwert 50 der Spalte *fk_ausid* auf den Primärschlüsselwert 50 der Spalte *ausid* in der Tabelle *auszubildender_neu* referenziert. Hier besteht also eine Beziehung zwischen einer Adresse und einem Auszubildenden.

aid	strasse	nr	plz	ort	fk_ausid
1	Mondstraße	8	50827	Köln	NULL
3	Sonnenstraße	1	50129	Bonn	50

Tabelle 10.39 Adressen, die den Auszubildenden zugeordnet sind

Da wir mit der **ON DELETE CASCADE**-Klausel vorgesorgt haben, können wir jetzt mit einer einfachen **DELETE**-Anweisung die Zeile der Auszubildenden Sabine Erde löschen. Gleichzeitig wird automatisch die dazugehörige Adresse mit dem Primärschlüsselwert 3 der Spalte *aid* gelöscht.

```
DELETE FROM auszubildender_neu WHERE ausid=50;
```

Listing 10.69 Einen Auszubildenden löschen, dem eine Adresse zugeordnet ist

Das Ergebnis der Löschung sehen Sie in [Tabelle 10.40](#) und [Tabelle 10.41](#):

ausid	name	vorname	geburtsdatum
30	Lang	Peter	2001-03-11

Tabelle 10.40 Die Zeile der Auszubildenden mit dem Primärschlüsselwert »50« in der Spalte »ausid« wurde gelöscht.

aid	Strasse	nr	plz	ort	fk_ausid
1	Mondstraße	8	50827	Köln	NULL

Tabelle 10.41 Die Adresszeile, die auf die Adresse referenzierte, wurde ebenfalls gelöscht.

Zuletzt müssen wir uns anschauen, was passiert, wenn Sie versuchen, eine Elternzeile aus einer Tabelle zu löschen, auf die aus einer Zeile einer Kindtabelle referenziert wird und deren Fremdschlüsseldefinition nicht mit einer **ON DELETE CASCADE**-Klausel versehen wurde. Die Frage lässt sich einfach beantworten: Die Sicherstellung der referenziellen Integrität würde so lange das Löschen einer Zeile in einer Elterntabelle verhindern, bis keine Zeilen mehr aus einer Kindtabelle auf diese Zeile referenzieren.

Die Sicherstellung der referenziellen Integrität durch eine relationale Datenbank demonstriere ich Ihnen anhand eines einfachen Beispiels. Hierzu eignen sich die Tabellen *adresse* und *auszubildender* optimal, da hier für die Fremdschlüsseldefinition weder eine **ON UPDATE CASCADE**-Klausel noch eine **ON DELETE CASCADE**-Klausel festgelegt wurde, wobei in diesem Fall die fehlende **ON DELETE CASCADE**-Klausel wichtig ist.

Die Tabelle *adresse* referenziert mit der Fremdschlüsselspalte *fk_ausid* auf die Primärschlüsselspalte *ausid*. Solange also Adressen auf Auszubildende referenzieren, muss die referenzielle Integrität durch die Datenbank sicherstellen, dass aus der Elterntabelle keine Zeilen gelöscht werden können, solange darauf referenzierende Zeilen in der Kindtabelle vorhanden sind.

Die relevanten Beispielzeilen sehen Sie in [Tabelle 10.42](#) und [Tabelle 10.43](#):

ausid	name	vorname	geburtsdatum
1	Müller	Ralf	2001-04-01
3	Lang	Peter	2001-03-11
5	Erde	Sabine	2001-01-23

Tabelle 10.42 Zeilen von Auszubildenden, denen eine Adresse zugeordnet ist

aid	strasse	nr	plz	ort	fk_ausid
1	Mondstraße	8	50827	Köln	1
2	Sternstraße	10	50127	Bonn	3
3	Sonnenstraße	1	50129	Bonn	5

Tabelle 10.43 Zeilen von Adressen, denen Auszubildende zugeordnet sind

Probieren wir nun, ob die Datenbank die referenzielle Integrität sicherstellt. Zunächst versuchen wir, mit einer **DELETE**-Anweisung den Auszubildenden Peter Lang mit dem Primärschlüsselwert 3 der Spalte *ausid* zu löschen. Aus der Tabelle *adresse* referenziert die Zeile des Auszubildenden Peter Lang mit dem Fremdschlüsselwert 3 der

Spalte *fk_ausid* auf den Primärschlüsselwert 3 der Spalte *ausid* in der Tabelle *auszubildender*.

In [Listing 10.70](#) sehen Sie die **DELETE**-Anweisung, mit der geprüft wird, ob eine Zeile aus einer Elterntabelle gelöscht werden kann oder ob noch Zeilen aus Kindtabellen auf sie referenzieren:

```
DELETE FROM auszubildender WHERE ausid=3;
```

Listing 10.70 Ein Versuch, einen Auszubildenden zu löschen

Die Datenbank stellt die referenzielle Integrität und somit die Gültigkeit der Beziehungen sicher und reagiert mit einer Fehlermeldung.

```
Error Code: 1451. Cannot delete or update a parent row: a foreign key
constraint fails (`ausbildungsdatenbank`.`adresse`, CONSTRAINT
`fk_adresse_auszubildender` FOREIGN KEY (`fk_ausid`) REFERENCES
`auszubildender` (`ausid`))
```

Wenn in der Kindtabelle die Fremdschlüsseldefinition keine **ON DELETE CASCADE**-Klausel enthält, stellt die Datenbank stets sicher, dass Zeilen aus Elterntabellen nicht gelöscht werden können, solange Zeilen aus Kindtabellen auf sie referenzieren.



Zusammenfassung: Schlüsselbeziehungen aufheben

Eine Beziehung aus einer Zeile einer Kindtabelle wird aufgehoben, indem der Fremdschlüsselwert, der auf einen Primärschlüsselwert in einer Elterntabelle verweist, auf nicht definiert (**NULL**) gesetzt wird.

Wenn eine Zeile einer Kindtabelle, die in Beziehung zu einer Zeile einer Elterntabelle steht, gelöscht wird, ist die Beziehung ebenfalls nicht mehr existent.

Eine Zeile in einer Elterntabelle, die in Beziehung zu Zeilen einer Kindtabelle steht, kann nur dann gelöscht werden, wenn bei der Erstellung der Kindtabelle die **ON DELETE CASCADE**-Klausel für den **FOREIGN KEY**-Constraint angegeben wurde.

10.4.4 Übungen

Übung 1

In Übung 2 aus [Abschnitt 10.3.3](#) haben Sie die Tabellen *ausgemustert* und *grund* erstellt. Jetzt nutzen wir diese Tabellen, um eine Beziehung einer Zeile einer Kindtabelle zu einer Zeile einer Elterntabelle über einen Fremdschlüsselwert aufzuheben.

Setzen Sie dazu den Fremdschlüsselwert 30, der in [Tabelle 10.44](#) in der Spalte *fk_exemplarid* dargestellt ist, mit einer **UPDATE**-Anweisung auf **NULL**.

grundid	ausmusterungsgrund	fk_exemplarid
1	beschädigt	30
2	verschmutzt	29
3	veraltet	5
4	veraltet	6

Tabelle 10.44 Die Tabelle »grund«

Wenn Sie in der Tabelle *grund* den Fremdschlüsselwert von 30 in **NULL** geändert haben, formulieren Sie eine **SELECT**-Anweisung mit einem **INNER JOIN**, der die Tabellen *ausgemustert* und *grund* verbunden abfragt. So wird bestätigt, dass keine Beziehung mehr zu einer Zeile mit dem Primärschlüsselwert 30 der Tabelle *ausgemustert* existiert. Wählen Sie aus der Tabelle *ausgemustert* die Spalten *exemplarid* und *isbn* für die Spaltenauswahlliste aus. Aus der Tabelle *grund* soll die Spalte *ausmusterungsgrund* in die Spaltenauswahlliste aufgenommen werden.

Übung 2

Löschen Sie mit einer **DELETE**-Anweisung eine Zeile aus der Kindtabelle *grund*, die in Beziehung zu einer Zeile der Elterntabelle *ausgemustert* steht. Als konkretes Beispiel können Sie die Zeile mit dem Fremdschlüsselwert 29 der Spalte *fk_exemplarid* der Tabelle *grund* entfernen, die auf den Primärschlüsselwert 29 in der Tabelle *ausgemustert* referenziert. Die Bedingung in der **WHERE**-Klausel der **DELETE**-Anweisung muss entsprechend formuliert sein.

Im Anschluss fragen Sie mit einer **SELECT**-Abfrage die Zeilen der Tabellen *ausgemustert* und *grund* verbunden ab, um zu bestätigen, dass die Beziehung nicht mehr existiert. Sie können hier natürlich die gleiche Abfrage verwenden wie in Übung 1.

Übung 3

In dieser Übung legen Sie eine Kindtabelle an, in der automatisch sämtliche Kindzeilen gelöscht werden, wenn eine in Beziehung stehende Elternzeile gelöscht wird. Um die Voraussetzungen zu schaffen, erstellen Sie eine Kindtabelle, in der Sie einer Fremdschlüsseldefinition eine **ON DELETE**-Klausel zuordnen, die festlegt, was zu tun ist, wenn eine Zeile in der Elterntabelle gelöscht wird.

Wie in den vorherigen Übungen nutzen Sie hier wieder die Tabellen *ausgemustert* und *grund*. Die **CREATE TABLE**-Anweisung für die Tabelle *grund* aus [Listing 10.61](#) in [Abschnitt 10.3.3](#) muss hierzu an der passenden Stelle um die **ON DELETE**-Klausel ergänzt

werden. Zuvor müssen Sie die bisher verwendete Tabelle *grund* mit einer **DROP TABLE**-Anweisung entfernen, wie in [Listing 10.71](#) gezeigt:

```
DROP TABLE grund;
```

Listing 10.71 Die Tabelle »grund« mit einer DROP TABLE-Anweisung entfernen

Im nächsten Schritt müssen Sie die Kindtabelle *grund* wieder mit Datensätzen versorgen, um Beziehungen zu den Zeilen in der Elterntabelle *ausgemustert* zu schaffen. Die hierzu benötigten **INSERT**-Anweisungen gebe ich Ihnen in [Listing 10.72](#) vor:

```
INSERT INTO grund (grundid, ausmusterungsgrund, fk_exemplarid)
VALUES (1, 'beschädigt', 30);
INSERT INTO grund (grundid, ausmusterungsgrund, fk_exemplarid)
VALUES (2, 'verschmutzt', 29);
INSERT INTO grund (grundid, ausmusterungsgrund, fk_exemplarid)
VALUES (3, 'veraltet', 5);
INSERT INTO grund (grundid, ausmusterungsgrund, fk_exemplarid)
VALUES (4, 'veraltet', 6);
```

Listing 10.72 Zeilen in die Tabelle »grund« einfügen, die in Beziehung zu Zeilen in der Tabelle »ausgesondert« stehen

Ich gehe davon aus, dass in die Tabelle *grund* Zeilen eingefügt worden sind. Jetzt können Sie eine Zeile in der Elterntabelle *ausgemustert* löschen, obwohl eine Zeile aus der Kindtabelle *grund* auf diese Zeile referenziert. Wenn eine Zeile aus der Elterntabelle *ausgemustert* gelöscht wird, werden alle Zeilen aus der Kindtabelle *grund*, die auf den Primärschlüsselwert der Elterntabelle *ausgemustert* referenzieren, ebenfalls gelöscht. In [Listing 10.73](#) sehen Sie die **DELETE**-Anweisung, die Sie verwenden können:

```
DELETE FROM ausgemustert WHERE exemplarid=30;
```

Listing 10.73 Eine Zeile einer Elterntabelle löschen

Gleich im Anschluss nutzen Sie einfache **SELECT**-Anweisungen, um das Löschen der Zeilen in beiden Tabellen zu überprüfen.

Lösung zu Übung 1

```
/* Eine Beziehung einer Zeile einer Kindtabelle aufheben, indem ein
Fremdschlüsselwert auf NULL gesetzt wird. */
UPDATE grund SET fk_exemplarid=NULL WHERE fk_exemplarid=30;
```

```
/* Eine Prüfabfrage mit einem INNER JOIN formulieren, um zu bestätigen,  
dass die Beziehung nicht mehr existiert. */
```

```
SELECT  
    a.exemplarid,  
    a.isbn,  
    g.ausmusterungsgrund  
FROM  ausgemustert a  
      INNER JOIN grund g  
ON a.exemplarid=g.fk_exemplarid;
```

Listing 10.74 Eine Beziehung aufheben, indem ein Fremdschlüsselwert auf NULL gesetzt wird

Lösung zu Übung 2

```
DELETE FROM grund WHERE fk_exemplarid=29;
```

Listing 10.75 Eine Zeile aus einer Kindtabelle löschen, die in Beziehung zu einer Zeile einer Elterntabelle steht

Lösung zu Übung 3

```
CREATE TABLE grund (  
    grundid SMALLINT NOT NULL,  
    ausmusterungsgrund VARCHAR(3000),  
    fk_exemplarid SMALLINT,  
    PRIMARY KEY (grundid),  
    CONSTRAINT fk_exemplarid  
        FOREIGN KEY (fk_exemplarid)  
            REFERENCES ausgemustert (exemplarid)  
            ON UPDATE CASCADE ON DELETE CASCADE  
);
```

Listing 10.76 Eine Fremdschlüsseldefinition um eine ON DELETE CASCADE-Klausel in einer CREATE TABLE-Anweisung erweitern

Kapitel 11

Transaktionen

Die Transaktionssicherheit, die relationale Datenbanksysteme gewährleisten, gehört zu den Kernfunktionalitäten und bietet uns z. B. die Sicherheit, dass eine oder mehrere datentechnische Operationen wie INSERT, UPDATE und DELETE durchgeführt werden. Welche Sicherheiten Ihnen die Transaktionskontrolle noch bietet, erfahren Sie in diesem Kapitel.

Bevor wir uns mit Transaktionen auseinandersetzen, betrachten wir zunächst einige Szenarien, die Ihnen verdeutlichen werden, warum es so wichtig ist, dass Datenbankoperationen wie **INSERT**, **UPDATE** und **DELETE** verbindlich durchgeführt werden.

Das erste Szenario betrifft schlicht und ergreifend Einflüsse, die wir nicht in der Hand haben. Stellen Sie sich vor, eine Sicherung geht kaputt und Ihr Datenbankserver wird plötzlich nicht mehr mit Strom versorgt. Oder was wäre, wenn es durch die Installation einiger Sicherheitsupdates zu einem Absturz des Betriebssystems käme? Diese Vorstellungen sind eigentlich nicht beunruhigend. Sie ersetzen die Sicherung oder deinstallieren die fehlerhaften Updates, und schon nimmt der Datenbankserver seinen Betrieb wieder auf.

Fehler an der Datenbank oder am Datenbankserver sind jedoch nicht die einzigen Störungsquellen. Eine Datenbank steht zentral zur Verfügung. Daher kann auch der Rechner ausfallen, von dem aus Sie die Operationen ausführen. Stellen Sie sich vor, Sie arbeiten gerade von Ihrem entfernten Rechner mit einer MySQL Workbench auf einem MySQL-Datenbankserver und das Netzkabel wird aus Unachtsamkeit getrennt.

Auch wenn Sie solche Fehler sicherlich einfach beheben können, bleibt eine große Frage bestehen:

Was passiert mit den Operationen, die vor den Störungen oder mitten in einem Störfall durchgeführt wurden?

Wurden die Updates für die Rechnungen vor einer Störung noch durchgeführt oder nicht? Was ist mit den neuen Kunden, die mit **INSERT**-Operationen in eine Tabelle eingefügt wurden?

In [Abschnitt 11.1](#) sehen wir uns an, welche Anforderungen an eine relationale Datenbank zu stellen sind. Hier erfahren Sie auch, wie mit Operationen umgegangen werden soll, wenn eine Störung auftritt.

11.1 Forderungen an relationale Datenbanksysteme

Wir fordern, dass eine relationale Datenbank sicherstellt, dass Operationen ganz oder gar nicht durchgeführt werden. Technische Störungen würden uns dann nicht mehr allzu sehr beunruhigen. Warum das so ist, erkläre ich Ihnen im Folgenden.



Operationen sollen atomar in der Ausführung sein

Operationen wie **INSERT**-, **UPDATE**- oder **DELETE**-Anweisungen sollen ganz oder gar nicht ausgeführt werden. Wenn z. B. mehrere **INSERT**-Anweisungen hintereinander auf eine Tabelle ausgeführt werden sollen und nach der Hälfte der Einfügeanweisungen eine Störung eintritt, soll der ursprüngliche Zustand wiederhergestellt werden. Es gilt also die Forderung, dass nicht ein Teil von Operationen ausgeführt und ein anderer Teil aufgrund einer Störung nicht ausgeführt wird. In der Gesamtheit der Durchführung ist eine Folge von Operationen also unteilbar (*atomar*) durchzuführen.

Die Störszenarien, die ich Ihnen dargestellt habe, führen uns zu einer weiteren Forderung, die auf den Erklärungen aus [Kapitel 9](#), »Datenmodelle in Tabellen überführen«, beruht. Dort haben Sie gelernt, dass Fremdschlüsselwerte einer Kindtabelle auf Primärschlüsselwerte einer Elterntabelle referenzieren können. Wenn eine Störung auftritt, erwarten wir einen definierten Zustand der Schlüsselbeziehungen. Konkret darf der Fremdschlüsselwert in einer Kindtabelle entweder **NULL** sein, oder er referenziert mit einem Wert auf einen Primärschlüsselwert. Beide Zustände sind erlaubt, sofern die Fremdschlüsselspalte **NULL**-Werte zulässt. Wenn dies nicht der Fall ist, sollte die Datenbank auf jeden Fall gewährleisten, dass immer ein konkreter Fremdschlüsselwert einer Kindtabelle auf einen Primärschlüsselwert einer Elterntabelle verweist. Dieses Verhalten sollte auch bei Störungen garantiert werden.



Der Datenbestand soll konsistent sein

Wenn Operationen wie **INSERT**- oder **UPDATE**-Anweisungen auf einer Datenbank durchgeführt werden, muss sichergestellt sein, dass der Datenbestand konsistent bleibt. Das betrifft insbesondere Schlüsselbeziehungen zwischen Tabellen bzw. die referenzielle Integrität. Eine Störung darf nicht dazu führen, dass Beziehungen zwischen Zeilen in einem nicht definierten Zustand vorliegen. Es gilt also die Forderung, dass eine Datenbank stets die *Konsistenz* der Daten sicherzustellen hat.

Eine relationale Datenbank arbeitet nach dem *Client-Server-Prinzip*. Eine Datenbank steht den Clients (den Nutzern der Datenbank) also zentral zur Verfügung. Wenn auf einer Datenbank also z. B. 800 Nutzer gleichzeitig arbeiten, ist es durchaus möglich, dass vier Nutzer gleichzeitig auf einen Datensatz eines Kunden zugreifen.

Stellen Sie sich vor, dass zwei Nutzer den Datensatz ändern, während die anderen beiden den Datensatz lesen. Hier ist sicherzustellen, dass die Operationen isoliert voneinander durchgeführt werden.

Operationen sollen isoliert voneinander durchgeführt werden

Eine *relationale Datenbank* verwaltet Daten zentral. Eine relationale Datenbank arbeitet im sogenannten *Multiuser-Betrieb*. Das heißt, dass mehrere Nutzer gleichzeitig auf die gleichen Datensätze zugreifen und diese gegebenenfalls ändern oder auch löschen können. Operationen, die gleichzeitig von unterschiedlichen Nutzern an einem Datensatz ausgeführt werden, dürfen sich nicht gegenseitig beeinflussen. Hier gilt die Forderung, diese Operationen stets *isoliert* voneinander durchzuführen.

Stellen Sie sich als Nächstes vor, eine Datenbank würde die zu verwaltenden Daten lediglich im Arbeitsspeicher eines Servers ablegen. Wenn es zu einem Stromausfall käme, wären alle Daten verloren, da der Arbeitsspeicher eines Rechners flüchtig ist. Es ist also sicherzustellen, dass die zu verwaltenden Daten dauerhaft gespeichert werden.

Daten in einer Datenbank sollen dauerhaft gespeichert werden

Unabhängig von jeglichen Störszenarien gilt für eine relationale Datenbank die Forderung, die zu verwaltenden Daten *dauerhaft* zu speichern.

Eine relationale Datenbank sollte also folgende Forderungen erfüllen:

- ▶ Operationen werden *atomar* durchgeführt.
- ▶ Die *Konsistenz* der Daten ist stets gegeben.
- ▶ Operationen werden *isoliert* voneinander ausgeführt.
- ▶ Informationen werden *dauerhaft* gespeichert.

Diese Forderungen werden in relationalen Datenbanken durch das *Transaktionsprinzip* umgesetzt. Die Forderungen werden in Kurzform auch als *ACID-Prinzip* bezeichnet; die Abkürzung resultiert aus den englischen Begriffen *atomicity*, *consistency*, *isolation* und *durability*.

11.2 Transaktionen verstehen

In diesem Abschnitt widmen wir uns dem Thema der Anwendung von Transaktionen. Mit Transaktionen erfüllen relationale Datenbanksysteme die Forderungen, die wir in [Abschnitt 11.1](#) gestellt haben.



Transaktionen

Transaktionen fassen einzelne Operationen zu einer gesamtheitlichen Operation zusammen, die entweder ganz oder gar nicht ausgeführt wird. Transaktionen überführen eine Datenbank bei der Durchführung von Operationen bzw. Einfüge-, Änderungs- oder Löschanweisungen aus einem konsistenten Zustand in einen konsistenten Zustand. Transaktionen stellen also gemäß dem ACID-Prinzip sicher, dass

- ▶ eine oder mehrere Operationen ganz oder gar nicht durchgeführt werden (*atomicity*),
- ▶ die referenzielle Integrität der Daten stets erhalten bleibt und die Daten aus einem konsistenten Zustand in einen konsistenten Zustand überführt werden (*consistency*),
- ▶ konkurrierende Zugriffe auf gleiche Datensätze stets isoliert voneinander stattfinden (*isolation*) und dass
- ▶ Daten dauerhaft gespeichert werden (*durability*).

11.2.1 Allgemeiner Aufbau einer Transaktion

Der allgemeine Aufbau einer Transaktion ist einfach zu verstehen. Eine Transaktion hat einen Startpunkt und einen Endpunkt. Zwischen dem Startpunkt und dem Endpunkt können SQL-Anweisungen wie **INSERT**, **UPDATE**, **DELETE** oder auch **SELECT** ausgeführt werden. Eine Transaktion stellt sicher, dass alle Operationen innerhalb des Start- und Endpunkts vollständig ausgeführt werden. Eine **SELECT**-Anweisung spielt hier natürlich eher eine untergeordnete Rolle, da ja lediglich Datensätze abgefragt, aber keine neuen Datensätze eingefügt, geändert oder gelöscht werden.

Manchmal sagen Abbildungen einfach mehr als tausend Worte. Betrachten Sie also [Abbildung 11.1](#).

Hier sehen Sie einen definierten Startpunkt und einen definierten Endpunkt einer Transaktion. Dazwischen befinden sich **INSERT**-Anweisungen. Wenn diese Einfügeoperationen innerhalb eines Zeitraums zwischen dem Start- und Endpunkt durchgeführt werden, sichert uns eine Datenbank die ACID-Eigenschaften zu.

Den Start- und Endpunkt einer Transaktion teilen Sie über bestimmte SQL-Anweisungen einer Datenbank mit. Heutige SQL-Clients wie die MySQL Workbench, HeidiSQL-Client, das Microsoft SQL Server Management Studio oder der pgAdmin einer

PostgreSQL-Datenbank führen die Transaktionskontrolle *implizit* aus. Das heißt, bei den hier verwendeten Datenbanken müssen Sie nicht mehr zwangsläufig angeben, wann Sie eine Transaktion beginnen bzw. verbindlich abschließen wollen. Natürlich besteht weiterhin die Möglichkeit, eine Transaktion mit einer expliziten Angabe zu beginnen bzw. abzuschließen.

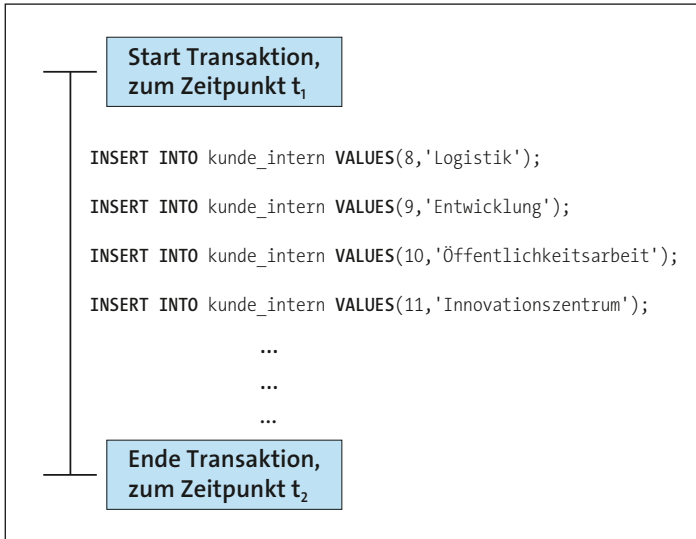


Abbildung 11.1 Start und Ende einer Transaktion

11.2.2 Einen atomaren Datenzustand mit Transaktionen sicherstellen

Sehen wir uns nun an, wie Transaktionen sicherstellen, dass mehrere Operationen ganz oder gar nicht durchgeführt werden. Transaktionen gewährleisten nicht nur, dass der Datenbestand in dramatischen Szenarien wie bei einem Stromausfall oder einem Serverausfall immer aus einem konsistenten Zustand in einen konsistenten Zustand überführt wird, sondern bieten auch die Möglichkeit an, eine Transaktion, solange sie noch nicht verbindlich abgeschlossen wurde, wieder rückgängig zu machen.

Listing 11.1. zeigt Ihnen, wie Sie eine Transaktion beginnen und abschließen:

```

BEGIN;
INSERT INTO kunde_intern VALUES(8,'Logistik');
INSERT INTO kunde_intern VALUES(9,'Entwicklung');
INSERT INTO kunde_intern VALUES(10,'Öffentlichkeitsarbeit');
INSERT INTO kunde_intern VALUES(11,'Innovationszentrum');
COMMIT;

```

Listing 11.1 SQL-Anweisungen explizit in einer Transaktion ausführen

Ein Transaktionsblock, der eine oder mehrere SQL-Anweisungen enthalten kann, beginnt mit dem Schlüsselwort **BEGIN**. Hinter dem Schlüsselwort **BEGIN** folgen die SQL-Anweisungen. In diesem Fall handelt es sich um vier **INSERT**-Anweisungen, die zusätzliche Kostenstellen in die Tabelle *kunde_intern* eintragen sollen. Der Transaktionsblock wird mit dem Schlüsselwort **COMMIT** hinter der letzten **INSERT**-Anweisung abgeschlossen. Wenn Sie diese Sequenz von SQL-Anweisungen ausführen, sichert Ihnen die Datenbank zu, dass die ACID-Eigenschaften gewährleistet sind.

Diese Zusicherung der ACID-Eigenschaften gilt nur dann, wenn wirklich alle Anweisungen ausgeführt wurden. Hierzu zählen die **BEGIN**-Anweisung, mit der Sie eine Transaktion starten, die Einfüge-, Änderungs- oder Löschoperationen und die **COMMIT**-Anweisung, mit der Sie eine Transaktion verbindlich abschließen. Sollte es also nach der Ausführung der ersten zwei **INSERT**-Anweisungen zu einem Problem kommen und die **COMMIT**-Anweisung somit nicht ausgeführt werden können, so ist sichergestellt, dass der Zustand vor dem Beginn der Transaktion nach wie vor gegeben ist, da die Transaktion ja nicht abgeschlossen werden konnte. Mit »vor dem Beginn der Transaktion« ist gemeint, dass sich die Datenbank wieder in dem konsistenten Zustand befindet, der vor Beginn der Transaktion gegeben war. Hier gilt also das Ganz-oder-gar-nicht-Prinzip der Atomarität von Transaktionen.



Hinweis für die Nutzer einer MS SQL Server-Datenbank

Eine MS SQL Server-Datenbank beginnt eine Transaktion mit **BEGIN TRANSACTION** anstelle des Schlüsselwortes **BEGIN**.

11.2.3 Transaktionen mit **ROLLBACK** rückgängig machen

Wenn Sie eine Transaktion starten und SQL-Anweisungen wie **INSERT**, **UPDATE** und **DELETE** ausführen, haben Sie, solange Sie eine Transaktion mit **COMMIT** noch nicht verbindlich abgeschlossen haben, immer noch die Möglichkeit, alle Anweisungen vom Start des Transaktionsblocks bis hin zur zuletzt ausgeführten SQL-Anweisung rückgängig zu machen. Hierzu nutzen Sie die **ROLLBACK**-Anweisung.

Der Datenbestand befindet sich nach der Ausführung einer **ROLLBACK**-Anweisung wieder in dem Zustand, den er hatte, bevor der Transaktionsblock begonnen wurde.

Sehen wir uns hierzu eine Sequenz von SQL-Anweisungen an. Wir beginnen mit einer einfachen **SELECT**-Abfrage in [Listing 11.2](#):

```
SELECT * FROM kunde_intern;
```

Listing 11.2 Alle internen Kunden abfragen

Die Tabelle *kunde_intern* enthält derzeit die Kostenstellen 1 bis 11.

kostenstelle	abteilung
1	Produktion Sechskant 15mm
2	Produktion Inbusschraube 8mm
3	Instandsetzung Elektrik
4	Instandsetzung Pneumatik
5	Einkauf
6	Controlling
7	Qualitätsmanagement
8	Logistik
9	Entwicklung
10	Öffentlichkeitsarbeit
11	Innovationszentrum

Tabelle 11.1 Ergebnis der Abfrage aus [Listing 11.1](#)

Im nächsten Schritt beginnen wir, wie in [Listing 11.3](#) angegeben, mit dem Schlüsselwort **BEGIN** bzw. **BEGIN TRANSACTION** (MS SQL Server-Datenbank) einen Transaktionsblock. Gleich im Anschluss folgen vier **INSERT**-Anweisungen, die hier zu Demonstrationzwecken Kostenstellen in die Tabelle *kunde_intern* einfügen, die schlicht und ergreifend unsinnig sind.

```
BEGIN;
INSERT INTO kunde_intern VALUES(12, 'Kost 1');
INSERT INTO kunde_intern VALUES(13, 'Kost 2');
INSERT INTO kunde_intern VALUES(14, 'Kost 3');
INSERT INTO kunde_intern VALUES(15, 'Kost 4');
```

Listing 11.3 Einen Transaktionsblock beginnen und vier **INSERT**-Anweisungen ausführen

Nachdem Sie diese Anweisungen durchgeführt haben, überprüfen wir wieder mit der **SELECT**-Abfrage aus [Listing 11.2](#) den Inhalt der Tabelle *kunde_intern* und erhalten das Ergebnis aus [Tabelle 11.2](#). Die vier Datensätze mit den unsinnigen Kostenstellen wurden erfolgreich eingefügt.

kostenstelle	abteilung
1	Produktion Sechskant 15mm
2	Produktion Inbusschraube 8mm
3	Instandsetzung Elektrik
4	Instandsetzung Pneumatik
5	Einkauf
6	Controlling
7	Qualitätsmanagement
8	Logistik
9	Entwicklung
10	Öffentlichkeitsarbeit
11	Innovationszentrum
12	Kost 1
13	Kost 2
14	Kost 3
15	Kost 4

Tabelle 11.2 Ergebnistabelle nach vier INSERT-Anweisungen, die sinnige Kostenstellen in die Tabelle »kunde_intern« eingetragen haben

Beachten Sie, dass wir bis jetzt noch keine **COMMIT**-Anweisung durchgeführt haben. Da der Transaktionsblock noch nicht mit **COMMIT** abgeschlossen wurde, können wir ihn mit einer einfachen **ROLLBACK**-Anweisung wieder zurückrollen. Die Datenbank stellt dann exakt den Zustand wieder her, der dem der letzten verbindlichen **COMMIT**-Anweisung entspricht.

Um eine Transaktion zurückzurollen, notieren Sie lediglich die Anweisung **ROLLBACK** gefolgt von einem Semikolon und führen die Anweisung aus:

ROLLBACK;

Listing 11.4 SQL-Anweisungen bis zum letzten verbindlichen **COMMIT** wieder zurückrollen

Wenn Sie die **ROLLBACK**-Anweisung hinter dem Transaktionsblock aus [Listing 11.3](#) ausführen, werden die vier **INSERT**-Anweisungen wieder rückgängig gemacht, und Sie er-

halten bei der Ausführung der **SELECT**-Anweisung aus [Listing 11.2](#) wieder die Ergebnistabelle, wie sie in [Tabelle 11.1](#) dargestellt ist.

11.2.4 Operationen mit Transaktionen isoliert ausführen

Eine relationale Datenbank wird als Mehrbenutzersystem in einer Client-Server-Landschaft eingesetzt. Rein theoretisch könnten auf einen Datensatz Hunderte Nutzer gleichzeitig mit einer **SELECT**-Abfrage zugreifen. Solange diese Nutzer alle nur lesend auf einen Datensatz zugreifen, besteht kein Grund zur Sorge.

Aber wie sieht es aus, wenn zwanzig Nutzer den gleichen Datensatz ändern und weitere zwanzig Nutzer den Datensatz löschen? In einem solchen Fall handelt es sich um konkurrierende Zugriffe auf den einen Datensatz. Sie merken wahrscheinlich schon intuitiv, dass dies gewisse Verwirrungen auslösen wird.

Sehen wir uns in [Abbildung 11.2](#) einen weiteren Transaktionsblock an, in dem zwei Nutzer zwei Transaktionen durchführen.

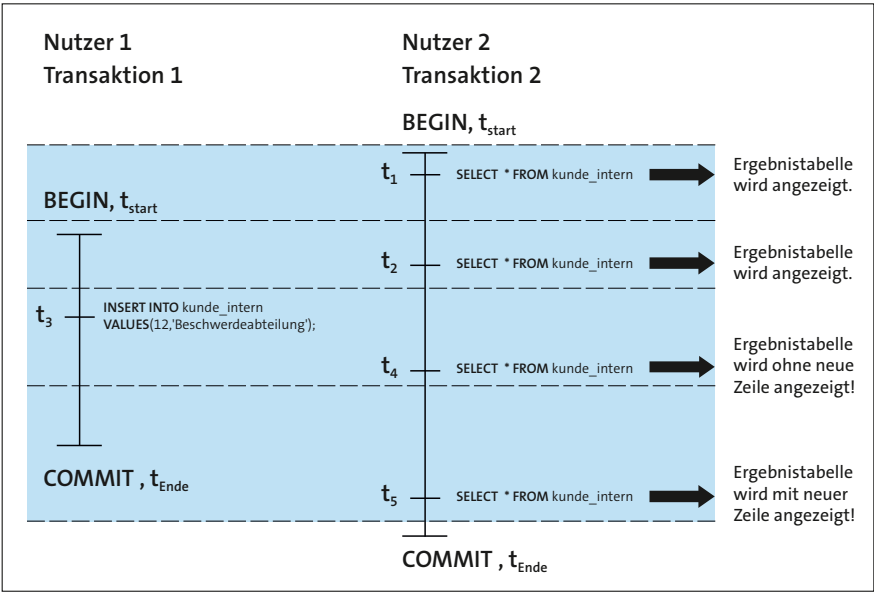


Abbildung 11.2 Transaktionen isoliert betrachten

Transaktion 1 ist Nutzer 1 zuzuordnen. Transaktion 2 ist hingegen Nutzer 2 zuzuordnen. Jeweils zu einem Zeitpunkt t_{start} wird die Transaktion des jeweiligen Nutzers mit einer **BEGIN**-Anweisung gestartet. Die beiden Transaktionen enden zum Zeitpunkt t_{ende} .

Von Interesse ist an dieser Stelle, was zwischen den Zeitpunkten des Beginns und des Endes der jeweiligen Transaktion passiert.

Wir beginnen die Betrachtung zum Zeitpunkt t_1 . Innerhalb von Transaktion 2 führt Nutzer 2 eine **SELECT**-Abfrage auf die Tabelle *kunde_intern* aus. Folglich erhält Nutzer 2 eine Ergebnistabelle.

Vor dem Zeitpunkt t_2 beginnt Nutzer 1 Transaktion 1. Ab diesem Zeitpunkt betrachten wir zwei Transaktionen, die parallel von zwei Nutzern verwendet werden. Exakt zum Zeitpunkt t_2 tätigt Nutzer 2 noch einmal eine Abfrage auf die Tabelle *kunde_intern*. Als Ergebnis erhält Nutzer 2 die gleiche Ergebnistabelle wie zuvor.

Einschränkend muss ich hier bemerken, dass wir in diesem Beispiel nur von zwei Nutzern und ihren Transaktionen ausgehen. Wir gehen also davon aus, dass in dem hier betrachteten Zeitintervall keine weiteren Nutzer manipulativ auf die Tabelle *kunde_intern* wirken.

Jetzt wird es spannend: Zum Zeitpunkt t_3 fügt Nutzer 1, der der Transaktion 1 zugeordnet ist, mit einer **INSERT**-Anweisung den internen Kunden *beschwerdeabteilung* in die Tabelle *kunde_intern* ein.

Nutzer 2 führt zum Zeitpunkt t_4 eine weitere **SELECT**-Abfrage auf die Tabelle *kunde_intern* aus. Zu diesem Zeitpunkt wurde von Nutzer 1 bereits der neue Datensatz eingefügt. Nutzer 2 erhält eine Ergebnistabelle ohne den neuen Datensatz. Dies liegt daran, dass die beiden Transaktionen isoliert voneinander durchgeführt werden und bis zu diesem Zeitpunkt Nutzer 1 seine Transaktion noch nicht mit einer **COMMIT**-Anweisung verbindlich bestätigt hat.

Zum Zeitpunkt t_5 führt Nutzer 2 abermals eine **SELECT**-Abfrage auf die Tabelle *kunde_intern* durch. Zu diesem Zeitpunkt hat Nutzer 1 Transaktion 1 bereits verbindlich abgeschlossen. Nutzer 2 erhält eine Ergebnistabelle zurück, die den neuen Datensatz enthält, den Nutzer 1 eingefügt hat.

In diesem Abschnitt haben Sie gelernt, dass Transaktionen isoliert voneinander durchgeführt werden. Im SQL-Standard werden vier Isolationsebenen unterschieden:

- ▶ *Read uncommitted*: Die Ergebnisse von Einfüge- und Aktualisierungsoperationen können auch ohne eine verbindliche abschließende **COMMIT**-Anweisung in einer parallel ablaufenden Transaktion gelesen werden.
- ▶ *Read committed*: Die Ergebnisse von Einfüge- und Aktualisierungsoperationen können nur nach einer verbindlich abschließenden **COMMIT**-Anweisung in einer parallel ablaufenden Transaktion gelesen werden.
- ▶ *Repeatable read*: Innerhalb einer Transaktion ist sichergestellt, dass ein gelesener Datensatz stets wiederholbar ist, auch dann, wenn er innerhalb einer anderen Transaktion verändert wird.
- ▶ *Serializable*: Transaktionen werden sequenziell abgearbeitet. Die Isolationsebene *Serializable* stellt die höchste Isolationsstufe dar.

Unterschiede zwischen den Datenbanksystemen

Leider setzen die verschiedenen Datenbanksysteme den SQL-Standard unterschiedlich um:

- ▶ MySQL und MariaDB stellen alle vier Isolationsebenen zur Verfügung.
- ▶ PostgreSQL behandelt die Isolationsebene *Read uncommitted* wie die Isolations-ebene *Read committed*.
- ▶ MS SQL Server bietet noch eine weitere Isolationsebene an: *Snapshot*: ein schreibgeschütztes, statisches Abbild der Datenbank.

Die Isolationsebenen von Transaktionen sind also je nach verwendeter Datenbank unterschiedlich implementiert. Wenn Sie Isolationsebenen von Transaktionen vertiefen wollen, empfehle ich Ihnen, die Dokumentation des Datenbanksystems zu lesen, das Sie gerade verwenden.

Zusammenfassung: Transaktionen

Transaktionen stellen sicher, dass Operationen wie **INSERT**, **UPDATE** oder **DELETE** gemäß den ACID-Eigenschaften selbst unter widrigen Umständen ganz oder gar nicht durchgeführt werden. Dies bedeutet, dass die *Konsistenz* von Datensätzen nicht gefährdet ist, dass Operationen *atomar* ausgeführt werden, *isoliert* voneinander ablaufen und dass die Ergebnisse von Operationen *dauerhaft gespeichert* werden.

Hierzu werden die Anweisungen **BEGIN** und **COMMIT** verwendet, um einen in sich geschlossenen Transaktionsblock zu definieren, der die beschriebenen Eigenschaften sicherstellt.

11.3 Übungen

Übung 1

Welche vier Eigenschaften werden unter der Abkürzung ACID zusammengefasst?

Übung 2

Wie ist ein Transaktionsblock aufgebaut, in dem eine oder mehrere SQL-Anweisungen zusammengefasst verbindlich ausgeführt werden?

Übung 3

Mit welcher Anweisung machen Sie die Operationen, die Sie in einem Transaktionsblock seit seinem Beginn durchgeführt haben, wieder rückgängig?

Übung 4

Benennen Sie die vier Isolationsstufen, die Sie für Transaktionen anwenden können (vorausgesetzt, die jeweilige Datenbank unterstützt die Isolationsstufe).

Lösung zu Übung 1

1. Dass eine oder mehrere Operationen ganz oder gar nicht ausgeführt werden.
2. Dass die Konsistenz der Daten gewährleistet ist. Hiermit ist insbesondere die referenzielle Integrität von Datensätzen gemeint.
3. die isolierte Ausführung von Transaktionen
4. die dauerhafte Speicherung von Datensätzen nach einer abgeschlossenen Transaktion

Lösung zu Übung 2

`BEGIN`

...

...

`COMMIT;`

Listing 11.5 Transaktionsblock für eine MySQL-, eine MariaDB- und eine PostgreSQL-Datenbank

`BEGIN TRANSACTION`

...

...

`COMMIT;`

Listing 11.6 Transaktionsblock für eine MS SQL Server-Datenbank

Lösung zu Übung 3

`ROLLBACK;`

Listing 11.7 INSERT-, UPDATE- und DELETE-Anweisungen innerhalb einer Transaktion rückgängig machen

Lösung zu Übung 4

- ▶ *Read uncommitted*
- ▶ *Read committed*
- ▶ *Repeatable read*
- ▶ *Serializable*

Kapitel 12

Tabellenstrukturen verändern

In diesem Kapitel lernen Sie, wie sich Datenstrukturen oder Einschränkungen von Tabellen nachträglich ändern lassen.

Stellen Sie sich vor, Sie haben mühsam ein Datenmodell erstellt und das Datenmodell (die Entitäten und ihre Beziehungen) in eine Datenbank überführt. Nun fällt Ihnen auf, dass Sie nicht nur einen ungünstigen Namen für eine Tabelle gewählt, sondern sogar eine Spalte vergessen haben.

Keine Panik, glücklicherweise können Sie die Bezeichnung einer Tabelle ebenso wie Spaltenbezeichnungen noch ändern. Genauso leicht können Sie Spalten zu einer Tabelle hinzufügen oder entfernen und Tabelleneinschränkungen (CONSTRAINTs) ändern oder hinzufügen. Sogar die Beziehungen zwischen Tabellen und ihren Zeilen können Sie nachträglich verändern.

12.1 Eine Tabelle umbenennen

Wir haben eine Tabelle namens *infokunde* (siehe [Abbildung 12.1](#)). Nun wollen wir diese Bezeichnung in *kundeinfo* ändern.

Das Umbenennen einer Tabelle muss für die hier behandelten Datenbanken unterschiedlich gelöst werden.

Zunächst schauen wir uns die Umsetzung für eine MySQL- und MariaDB-Datenbank an. In [Listing 12.1](#) sehen Sie, wie Sie eine (wirklich nur eine) Tabelle umbenennen können:

```
ALTER TABLE infokunde RENAME kundeinfo;
```

Listing 12.1 Eine Tabelle umbenennen (MySQL-, MariaDB-Variante 1)

Sie verwenden dazu eine **ALTER TABLE**-Anweisung. **ALTER TABLE** heißt in etwa »verändere eine Tabelle«. Dahinter folgt der Name der Tabelle, die Sie umbenennen wollen, gefolgt von dem Schlüsselwort **RENAME** (dt. »umbenennen«). Hinter **RENAME** geben Sie den neuen Tabellennamen, *kundeinfo*, an.

Eine weitere Möglichkeit, wie Sie (mindestens) eine Tabelle umbenennen können, sehen Sie in [Listing 12.2](#):

`RENAME TABLE infokunde TO kundeinfo;`

Listing 12.2 Eine Tabelle umbenennen (MySQL-, MariaDB-Variante 2)

Diese Anweisung beginnt mit den Schlüsselwörtern **RENAME TABLE**, gefolgt von dem Namen der Tabelle, die umbenannt werden soll; hier: *infokunde*. Zu guter Letzt folgen das Schlüsselwort **TO** und der neue Tabellename, *kundeninfo*.

Die **RENAME**-Anweisung bietet Ihnen die Möglichkeit, gleich mehrere Tabellen in einem Schritt umzubenennen. Hierfür setzen Sie hinter den Befehl aus [Listing 12.2](#) anstelle des Semikolons ein Komma und geben den Namen der nächsten umzubenennenden Tabelle ein, wieder gefolgt von **TO** und ihrem neuen Namen. Sie können nach diesem Muster beliebig viele Tabellen umbenennen. Getrennt wird die Aufzählung durch Kommas, zum Schluss setzen Sie ein Semikolon:

```
RENAME TABLE altername1 TO neuername1,  
altername2 TO neuername2,  
altername3 TO neuername3,  
...;
```

Sie können das Ergebnis der Umbenennung mit einer einfachen **SELECT**-Anweisung überprüfen:

`SELECT infoid,informationkunde FROM kundeinfo;`

Listing 12.3 Das Ergebnis einer Umbenennung mit einer SELECT-Anweisung überprüfen

infoid	informationkunde
1	Sehr guter Kunde!
2	Guter Kunde!
3	Nicht so guter Kunde!

Tabelle 12.1 Prüfung der Umbenennung der Tabelle »infokunde« in »kundeinfo«

Wenn Sie eine PostgreSQL-Datenbank verwenden, hat die **ALTER TABLE**-Anweisung aus [Listing 12.4](#) durchaus Ähnlichkeit mit der aus [Listing 12.1](#). Der kleine Unterschied besteht darin, dass die PostgreSQL-Datenbank zusätzlich das Schlüsselwort **TO** hinter **RENAME** erwartet.

`ALTER TABLE infokunde RENAME TO kundeinfo;`

Listing 12.4 Eine Tabelle umbenennen (PostgreSQL)

Eine MS SQL Server-Datenbank verwendet eine Prozedur, um eine Tabelle umzubenennen. Um eine Prozedur auszuführen, verwenden Sie die Anweisung `EXEC`, gefolgt vom Prozedurnamen. In unserem Fall handelt es sich um die Prozedur `SP_RENAME` (siehe [Listing 12.5](#)). Sie ist für das Umbenennen von Datenbankobjekten wie Tabellen oder Spalten zuständig. Hinter der Prozedur folgt in einfachen Hochkommas das Datenbankschema (hier *dbo*), dann ein Punkt als Trennzeichen und danach die Tabelle (*infokunde*), die Sie umbenennen wollen. Nach einem Komma folgt die neue Tabellenbezeichnung (*kundeinfo*).

```
EXEC SP_RENAME 'dbo.infokunde',kundeinfo;
```

Listing 12.5 Eine Tabelle umbenennen (MS SQL Server)

12.2 Spalten einer Tabelle ändern

In diesem Abschnitt erfahren Sie, wie Sie eine Spalte umbenennen, den Datentyp einer Spalte ändern, eine `NOT NULL`-Einschränkung hinzufügen und wie Sie einige weitere Einschränkungen zu einer Spalte hinzufügen oder auch von ihr entfernen können.

Auch das Umbenennen einer Spalte muss für jede hier behandelte Datenbank unterschiedlich gelöst werden.

Betrachten wir zunächst [Abbildung 12.1](#), bevor wir damit beginnen, die Tabellenstruktur bzw. Einschränkungen der Tabelle zu ändern. Sie sehen die Entität *infokunde*. Hier ist bei der Modellierung einiges schiefgelaufen. Daher werden wir ein paar Korrekturen vornehmen.

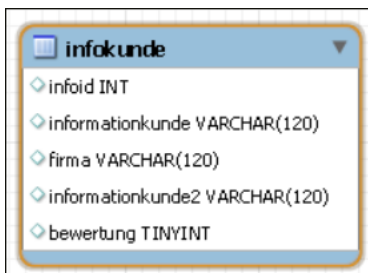


Abbildung 12.1 Die Struktur der Tabelle »infokunde«

12.2.1 Eine Spalte umbenennen

Die MySQL- und die MariaDB-Datenbank bieten Ihnen zwei Varianten zum Umbenennen einer Spalte. In beiden Varianten benennen wir die Spalte *informationkunde*

in *information* um. Verwenden Sie bitte die zu Ihrer Version passende Anweisung, um eine Spalte umzubenennen.

In [Listing 12.6](#) sehen Sie die Variante, die Sie zum Umbenennen einer Spalte in einer MySQL-Datenbank bis zur Version 8 anwenden können:

```
ALTER TABLE kundeinfo CHANGE informationkunde information VARCHAR(120);
```

Listing 12.6 Eine Spalte umbenennen (MySQL vor 8.x)

Eingeleitet wird die Anweisung mit der **ALTER TABLE**-Klausel. Es folgt der Name der Tabelle, in der eine Spalte umbenannt werden soll. Gleich im Anschluss folgt das Schlüsselwort **CHANGE**. Durch ein Leerzeichen getrennt, geben Sie die Spalte an, die umbenannt werden soll. Nach einem weiteren Leerzeichen folgt schließlich der neue Spaltenname. Abschließend müssen Sie noch den Datentyp der Spalte angeben.

Wenn Sie eine MariaDB-Datenbank einsetzen, können Sie diese ältere Notation ebenfalls verwenden. Die Notation bleibt uns also in aktuellen Versionen von MySQL und MariaDB erhalten.

[Listing 12.7](#) zeigt, wie Sie in MySQL ab Version 8.x oder für eine PostgreSQL- oder MariaDB-Datenbank eine Spalte umbenennen:

```
ALTER TABLE kundeinfo RENAME COLUMN informationkunde TO information;
```

Listing 12.7 Eine Spalte umbenennen (ab MySQL 8.x, MariaDB und in PostgreSQL)

Hinter der **ALTER TABLE**-Anweisung folgt auch hier der Name der Tabelle, in der Sie eine Spalte umbenennen wollen. Nach dem Tabellennamen müssen Sie jedoch die Schlüsselwörter **RENAME COLUMN** (Spalte umbenennen) angeben. Als Nächstes folgt die Spalte, die umbenannt werden soll. Hinter der Spaltenbezeichnung notieren Sie diesmal die Anweisung **TO**, gefolgt von dem neuen Spaltennamen. Einen Datentyp müssen Sie hier bei einer Anweisung, die für die Umbenennung einer Spalte zuständig ist, glücklicherweise nicht angeben. Wenn Sie beabsichtigen, eine Spalte umzubenennen, ist es also empfehlenswert, die neue Notation zu nutzen.

Für eine MS SQL Server-Datenbank verwenden Sie wie beim Umbenennen einer Tabelle die Prozedur **SP_RENAME**. Hinter der Prozedur folgt wieder in einfachen Hochkommas das Datenbankschema (*dbo*) mit der durch einen Punkt getrennten Tabelle *kundeinfo*. Anschließend folgt die Spalte, die Sie umbenennen wollen, hier *informationkunde*. Getrennt durch ein Komma, folgt schließlich der neue Spaltenname *information*:

```
EXEC SP_RENAME 'dbo.kundeinfo.informationkunde',information;
```

Listing 12.8 Eine Spalte umbenennen (MS SQL Server)

Wenn die Umbenennung erfolgreich war, können Sie zur Prüfung die Spalte *information* mit der **SELECT**-Anweisung aus [Listing 12.9](#) abfragen:

```
SELECT infoid,information FROM kundeinfo;
```

Listing 12.9 Eine umbenannte Spalte abfragen

12.2.2 Den Datentyp einer Spalte ändern

In [Abschnitt 4.1](#) haben Sie erfahren, dass jeder Spalte in einer Tabelle ein Datentyp zugeordnet werden muss. Es kann vorkommen, dass der Datentyp einer Spalte geändert werden muss. Das kann z. B. dann notwendig sein, wenn der Wertebereich eines Datentyps einer Spalte nicht mehr ausreichend ist. In [Abbildung 12.1](#) sehen Sie, dass der Spalte *bewertung* der Datentyp **TINYINT** zugeordnet ist.

Der Wertebereich des Datentyps **TINYINT** einer MySQL- oder MariaDB-Datenbank geht von 0 bis 255 bzw. mit Vorzeichen von -128 bis 127. In unserem Beispiel soll nun die Granularität der Bewertung differenzierter bzw. feiner ausgestaltet werden. Jetzt soll der Datentyp **SMALLINT** (**UNSIGNED** bis 65.535 bzw. mit Vorzeichen **SIGNED** von -32.768 bis 32.767) der Spalte *bewertung* zugeordnet werden, um den größeren Wertebereich des Datentyps verwenden zu können.

Wertebereiche der Datentypen (PostgreSQL und MS SQL Server)

Für eine PostgreSQL- oder MS SQL Server-Datenbank müssen Sie beachten, dass die Datentypen zwar gleiche Bezeichnungen besitzen, je nach verwendeter Datenbank aber unterschiedliche Wertebereiche aufweisen können:

MS SQL Server: **TINYINT** → **SMALLINT**

PostgreSQL: **SMALLINT** → **INTEGER**

In [Listing 12.10](#) zeige ich Ihnen, wie Sie den Datentyp der Spalte *bewertung* von **TINYINT** in **SMALLINT** ändern. Hinter dem Tabellennamen *kundeinfo* wird hier das Schlüsselwort **MODIFY** verwendet, um der **ALTER TABLE**-Klausel mitzuteilen, dass eine Spalte verändert werden soll. Hinter dem Schlüsselwort **MODIFY** geben wir den Spaltennamen an, dessen Datentyp wir ändern wollen. Hier tragen wir also den Spaltennamen *bewertung* ein, gefolgt von dem Datentyp, der der Spalte zugeordnet werden soll.

```
ALTER TABLE kundeinfo MODIFY bewertung SMALLINT UNSIGNED;
```

Listing 12.10 Den Datentyp einer Spalte ändern (MySQL, MariaDB)

Falls Sie eine PostgreSQL-Datenbank verwenden, nutzen Sie die **ALTER TABLE**-Anweisung aus [Listing 12.11](#). Hinter der **ALTER TABLE**-Klausel folgt hier eine **ALTER COLUMN**-

Anweisung, hinter der wiederum der Spaltenname *bewertung* steht. Auf den Spaltennamen folgt das Schlüsselwort **TYPE** und schließlich der neue Datentyp **INTEGER**, den wir der Spalte *bewertung* zuordnen:

```
ALTER TABLE kundeinfo ALTER COLUMN bewertung TYPE INTEGER;
```

Listing 12.11 Den Datentyp einer Spalte ändern (PostgreSQL)

Die Syntax aus [Listing 12.12](#) nutzen Sie, wenn Sie eine MS SQL Server-Datenbank verwenden. Hinter dem Tabellennamen *kundeinfo* der **ALTER TABLE**-Anweisung notieren Sie die **ALTER COLUMN**-Anweisung, gefolgt vom Namen der Spalte (hier: *bewertung*), deren Datentyp Sie ändern wollen. Hinter dem Spaltennamen geben Sie einfach den neuen Datentyp an, der der Spalte zugeordnet werden soll.

```
ALTER TABLE kundeinfo ALTER COLUMN bewertung INTEGER;
```

Listing 12.12 Den Datentyp einer Spalte ändern (MS SQL Server)

In [Listing 12.13](#) prüfen wir mit einer **UPDATE**-Anweisung, ob sich der ganzzahlige Wertebereich der Spalte *bewertung* für die MySQL- und MariaDB-Datenbank geändert hat, und weisen einem Spaltenwert den Wert 50000 bzw. 2147000000 (PostgreSQL und MS SQL Server) zu:

```
/* MySQL und MariaDB*/
UPDATE kundeinfo SET bewertung=50000 WHERE infoid=3;
/* PostgreSQL und MS SQL Server*/
UPDATE kundeinfo SET bewertung=2147000000 WHERE infoid=3;
```

Listing 12.13 Einen größeren Wertebereich durch die Zuweisung eines größeren Wertes überprüfen

Überprüfen wir auch diese Änderung wie in [Listing 12.14](#) angegeben:

```
SELECT infoid,bewertung FROM kundeinfo;
```

Listing 12.14 Abfrage zum Überprüfen des größeren Wertebereichs

In [Tabelle 12.2](#) sehen Sie das Ergebnis der Prüfabfrage. Zeile 3 der Spalte *bewertung* wurde mit dem Wert 50000 aktualisiert. Falls Sie eine PostgreSQL- oder eine MS SQL Server-Datenbank verwenden, wird Ihnen hier der Wert 2147000000 angezeigt.

infoid	bewertung
1	5

Tabelle 12.2 Ergebnis der Abfrage zur Überprüfung eines größeren Wertebereichs

inford	bewertung
2	4
3	50000

Tabelle 12.2 Ergebnis der Abfrage zur Überprüfung eines größeren Wertebereichs (Forts.)

12.2.3 Eine Spalte als Primärschlüsselspalte definieren

In [Abschnitt 4.4.2](#) haben Sie eine erste **CREATE TABLE**-Anweisung formuliert und dabei eine Primärschlüsselspalte festgelegt. Jetzt wollen wir eine Tabelle, in der kein Primärschlüssel definiert ist, nachträglich mit einem Primärschlüssel ausstatten. Die Tabellenstruktur aus [Abbildung 12.1](#) besitzt keine Primärschlüsseleinschränkung. Glücklicherweise ist die Spalte *inford* des Datentyps **INT** dazu geeignet, sie in einen Primärschlüssel umzuwandeln.

In [Listing 12.15](#) wird die **ADD CONSTRAINT**-Anweisung auf die **ALTER TABLE**-Klausel angewendet. Geben Sie zunächst hinter **ADD CONSTRAINT** den Namen der Einschränkung, *pk_kundeinfo_inford*, an. Hinter diesem Namen notieren Sie als Nächstes den Typ der Einschränkung, der hier **PRIMARY KEY** lautet. Anschließend übergeben Sie in einfachen Klammern den Spaltennamen, der als Primärschlüsselspalte festgelegt werden soll (hier: *inford*).

```
ALTER TABLE kundeinfo ADD CONSTRAINT pk_kundeinfo_inford PRIMARY KEY(inford);
```

Listing 12.15 Eine Spalte als Primärschlüsselspalte festlegen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Als Nächstes überprüfen wir, ob die Spalte *inford* der Tabelle *kundeinfo* tatsächlich als Primärschlüsselspalte definiert worden ist. In [Listing 12.16](#) verwenden wir hierfür eine **INSERT**-Anweisung mit einem Primärschlüsselwert, der bereits vergeben worden ist. Wenn Sie diese Anweisung ausführen, erhalten Sie eine Fehlermeldung aufgrund einer Primärschlüsselverletzung. Daran erkennen Sie, dass die Spalte *inford* erfolgreich in eine Primärschlüsselspalte umgewandelt wurde.

```
INSERT INTO kundeinfo (inford,information)
VALUES (3, 'Standardkunde');
```

Listing 12.16 Überprüfen, ob eine Primärschlüsselverletzung auftritt

Achten Sie auf Spalten, die bereits NULL-Werte enthalten

Wenn Sie einer Spalte nachträglich eine **PRIMARY KEY**-Einschränkung zuweisen möchten, darf die Spalte keine **NULL**-Werte enthalten, da Sie ihr ansonsten keine **PRIMARY KEY**-Einschränkung zuordnen können.





Eindeutigkeit der bereits enthaltenden Werte sicher stellen

Wenn eine Spalte in eine Primärschlüsselspalte geändert werden soll, ist es erforderlich, dass die Spaltenwerte eindeutig sind. Duplikate sind hier also nicht erlaubt.

Sie können einer Primärschlüsselspalte die Eigenschaft eines Primärschlüssels auch wieder entziehen. Allerdings müssen Sie dabei auf Unterschiede zwischen den hier behandelten Datenbanken achten.

In [Listing 12.17](#) ist eine **ALTER TABLE**-Anweisung dargestellt, mit der wir für eine MySQL- oder MariaDB-Datenbank eine Primärschlüsseinschränkung entfernen können. Hinter der **ALTER TABLE**-Anweisung notieren Sie wie gewohnt den Tabellennamen. Ergänzen Sie im Anschluss eine **DROP PRIMARY KEY**-Anweisung, um die Primärschlüsselspalte der Tabelle *kundeinfo* zu entfernen.

```
ALTER TABLE kundeinfo DROP PRIMARY KEY;
```

Listing 12.17 Die Primary-Key-Einschränkung entfernen (MySQL, MariaDB)

Wenn Sie eine PostgreSQL- oder eine MS SQL Server-Datenbank verwenden, nutzen Sie die Syntax aus [Listing 12.18](#). Hier notieren Sie hinter der **ALTER TABLE**-Klausel die **DROP CONSTRAINT**-Anweisung, gefolgt vom Namen der Einschränkung *pk_kundeinfo_infoid*, um die Primärschlüsseinschränkung wieder zu entfernen:

```
ALTER TABLE kundeinfo DROP CONSTRAINT pk_kundeinfo_infoid;
```

Listing 12.18 Die Primary-Key-Einschränkung entfernen (PostgreSQL, MS SQL Server)

12.2.4 Einer Spalte eine NOT NULL-Einschränkung zuordnen

In [Abschnitt 4.5.1](#) haben Sie bereits, im Zusammenhang mit einer **CREATE TABLE**-Klausel, eine **NOT NULL**-Einschränkung definiert. Jetzt wollen wir nachträglich eine Spalte mit einer **NOT NULL**-Einschränkung ausstatten.

Die Spalte *firma* der Tabelle *kundeinfo* ermöglicht es, **NULL**-Werte einzufügen. Unsere Absicht ist jetzt, die Spalte mit einer **NOT NULL**-Einschränkung zu versehen. Auch in diesem Fall gilt, dass die hier behandelten Datenbanken das Hinzufügen einer **NOT NULL**-Einschränkung jeweils etwas anders realisieren.



Bedingung für eine nachträgliche NOT NULL-Einschränkung

Wenn einer Spalte nachträglich eine **NOT NULL**-Einschränkung zugordnet wird, dürfen keine nicht definierten Spaltenwerte (**NULL**) in der Spalte enthalten sein. Sind **NULL**-Werte in einer bestehenden Spalte enthalten, wird der Versuch einer nachträglich hinzuzufügenden **NOT NULL**-Einschränkung mit einer Fehlermeldung quittiert.

Für eine MySQL- oder MariaDB-Datenbank verwenden Sie die in [Listing 12.19](#) angegebene **ALTER TABLE**-Anweisung in Kombination mit der **MODIFY**-Anweisung. Hinter der **MODIFY**-Anweisung notieren Sie den Namen der Spalte, der Sie die **NOT NULL**-Einschränkung zuordnen möchten, in unserem Fall *firma*. Dann folgt der Datentyp der Spalte und zu guter Letzt die **NOT NULL**-Einschränkung:

```
ALTER TABLE kundeinfo MODIFY firma VARCHAR(120) NOT NULL;
```

Listing 12.19 Einer Spalte eine NOT NULL-Einschränkung zuordnen (MySQL, MariaDB)

Wenn Sie eine PostgreSQL-Datenbank verwenden, notieren Sie hinter der **ALTER TABLE**-Klausel eine **ALTER COLUMN**-Anweisung. Dahinter folgt der Name der Spalte, der Sie eine **NOT NULL**-Einschränkung zuordnen möchten (hier: *firma*). Abschließend fügen Sie die Anweisung **SET NOT NULL** hinzu:

```
ALTER TABLE kundeinfo ALTER COLUMN firma SET NOT NULL;
```

Listing 12.20 Einer Spalte eine NOT NULL-Einschränkung zuordnen (PostgreSQL)

Sollten Sie eine MS SQL Server-Datenbank nutzen, verwenden Sie die **ALTER TABLE**-Klausel aus [Listing 12.21](#). Nach dem Tabellennamen folgt die **ALTER COLUMN**-Anweisung, gefolgt vom Spaltennamen. Nach dem Spaltennamen notieren Sie den Datentyp **VARCHAR(120)** und schließlich die **NOT NULL**-Einschränkung:

```
ALTER TABLE kundeinfo ALTER COLUMN firma VARCHAR(120) NOT NULL;
```

Listing 12.21 Einer Spalte eine NOT NULL-Einschränkung zuordnen (MS SQL Server)

Als Nächstes überprüfen wir, ob die Zuordnung funktioniert hat, indem wir eine **INSERT**-Anweisung mit einem **NULL**-Wert wie in [Listing 12.22](#) ausführen:

```
INSERT INTO kundeinfo (infoid,information,firma,bewertung)
VALUES(4, 'Guter Kunde!',NULL,3);
```

Listing 12.22 In eine Spalte mit einer NOT NULL-Einschränkung einen NULL-Wert eintragen

Wenn Sie eine Fehlermeldung erhalten, wissen Sie, dass die **NOT NULL**-Einschränkung erfolgreich der Spalte *firma* zugewiesen wurde.

12.2.5 Einer Spalte eine NULL-Einschränkung zuordnen

Eine **NOT NULL**-Einschränkung wieder in eine **NULL**-Einschränkung zu ändern, ist recht einfach. Wie Sie sehen, unterscheidet sich [Listing 12.23](#) (MySQL, MariaDB) kaum von [Listing 12.19](#). Hinter dem Datentyp geben Sie hier lediglich eine **NULL**-Einschränkung (anstelle der **NOT NULL**-Einschränkung) an, um wieder **NULL**-Werte für die Spalte *firma* zuzulassen:

```
ALTER TABLE kundeinfo MODIFY firma VARCHAR(120) NULL;
```

Listing 12.23 Einer Spalte eine NULL-Einschränkung zuordnen (MySQL, MariaDB)

Wenn Sie eine PostgreSQL-Datenbank verwenden, notieren Sie hinter dem Spaltennamen *firma* anstelle der SET NOT NULL-Anweisung die DROP NOT NULL-Anweisung:

```
ALTER TABLE kundeinfo ALTER COLUMN firma DROP NOT NULL;
```

Listing 12.24 Einer Spalte eine NULL-Einschränkung zuordnen (PostgreSQL)

Auch für MS SQL Server nutzen Sie einfach die ALTER TABLE-Klausel aus [Listing 12.25](#) und setzen hinter den Datentyp VARCHAR(120) einen NULL-Constraint anstelle der NOT NULL-Einschränkung:

```
ALTER TABLE kundeinfo ALTER COLUMN firma VARCHAR(120) NULL;
```

Listing 12.25 Einer Spalte eine NULL-Einschränkung zuordnen (MS SQL Server)

Als Nächstes überprüfen wir mit der INSERT-Anweisung aus [Listing 12.22](#), ob Sie jetzt in der Lage sind, der Spalte *firma* einen NULL-Wert zuzuweisen. Wenn die Anweisung erfolgreich ausgeführt wurde, sollte die INSERT-Anweisung ohne Fehlermeldung ausgeführt werden.

12.2.6 Einer Spalte einen Standardwert (DEFAULT VALUE) zuordnen

Existierende Spalten einer Tabelle können nachträglich mit einem Standardwert versehen werden.

In [Abbildung 12.2](#) ist die Tabelle *dsgvo* dargestellt. Die Spalte *status* verfügt derzeit über keine Standardwertzuordnung. Als Standardwert käme für die Spalte *status* der Wert nicht zugestimmt infrage. Dieser Wert könnte immer dann eingefügt werden, wenn der Status noch unbekannt ist und mit einer INSERT-Anweisung lediglich Werte der Spalten *dsgvoid* und *firma* in eine Zeile eingetragen werden.



Abbildung 12.2 Die Spalte »status« der Tabelle »dsgvo« verfügt über keinen Standardwert.

Betrachten wir als Nächstes die Zeilen in der Tabelle *dsgvo*. Hierzu nutzen wir die **SELECT**-Anweisung aus [Listing 12.26](#):

```
SELECT * FROM dsgvo;
```

Listing 12.26 Die Zeilen der Tabelle »dsgvo« abfragen

Das Ergebnis sehen Sie in [Tabelle 12.3](#): Offensichtlich sind drei Zeilen enthalten. Die Spalte *status* enthält derzeit keine Werte.

dsgvoid	firma	status
1	Schrauben Meier GmbH	NULL
2	Schrauben Peter GmbH	NULL
3	Schrauben Klein GmbH	NULL

Tabelle 12.3 Die Zeilen der Tabelle »dsgvo«

Wenn Sie eine MySQL-, MariaDB- bzw. eine PostgreSQL-Datenbank verwenden, gehen Sie wie in [Listing 12.27](#) vor. Hinter dem Spaltennamen der **ALTER COLUMN**-Anweisung folgt die **SET DEFAULT**-Anweisung, gefolgt von dem Literal nicht zugestimmt, das als Standardwert in die Tabelle eingetragen werden soll:

```
ALTER TABLE dsgvo
  ALTER COLUMN status SET DEFAULT 'nicht zugestimmt';
```

Listing 12.27 Einer Spalte einen Standardwert zuweisen (MySQL, MariaDB, PostgreSQL)

Für die MS SQL Server-Datenbank sieht das etwas anders aus. Hier nutzen Sie die **ADD CONSTRAINT**-Anweisung aus [Listing 12.28](#), gefolgt von dem Namen für die Einschränkung. Dahinter notieren Sie das Schlüsselwort **DEFAULT**, gefolgt von dem Wert, der als Standardwert festgelegt werden soll. Jetzt müssen Sie nur noch angeben, für welche Spalte der Standardwert angewendet werden soll. und zwar mit dem Schlüsselwort **FOR**, gefolgt vom Namen dieser Spalte:

```
ALTER TABLE dsgvo
  ADD CONSTRAINT df_dsgvo_status DEFAULT 'nicht zugestimmt' FOR status;
```

Listing 12.28 Einer Spalte einen Standardwert zuweisen (MS SQL Server)

Die folgende **INSERT**-Anweisung trägt lediglich einen gültigen Primärschlüsselwert in die Spalte *dsgvoid* der Tabelle *dsgvo* ein:

```
INSERT INTO dsgvo (dsgvoid) VALUES(4);
```

Listing 12.29 Einen Datensatz mit nur einem Spaltenwert einfügen

Mit einer einfachen **SELECT**-Anweisung wie in [Listing 12.26](#) überprüfen wir, ob tatsächlich ein Standardwert in die Spalte *status* eingetragen wurde.

In [Tabelle 12.4](#) sehen Sie das Ergebnis der **SELECT**-Abfrage. In der Spalte *dsgvoid* mit dem Primärschlüsselwert 4 wurde der Spalte *status* automatisch der Standardwert nicht zugestimmt zugewiesen.

dsgvoid	firma	status
1	Schrauben Meier GmbH	
2	Schrauben Peter GmbH	
3	Schrauben Klein GmbH	
4		nicht zugestimmt

Tabelle 12.4 Ergebniszeilen nach dem Einfügen einer Zeile, in der ausschließlich der Spalte »dsgvoid« ein Wert zugeordnet wurde

In [Listing 12.30](#) zeige ich Ihnen, wie Sie für eine MySQL-, MariaDB- und PostgreSQL-Datenbank eine Standardwertzuordnung einer Spalte wieder entfernen können. Hierzu notieren Sie lediglich hinter dem Spaltennamen der **ALTER COLUMN**-Anweisung die **DROP DEFAULT**-Anweisung:

```
ALTER TABLE dsgvo
  ALTER COLUMN status DROP DEFAULT;
```

Listing 12.30 Eine Standardwertzuordnung wieder entfernen
(MySQL, MariaDB, PostgreSQL)

Wenn Sie eine MS SQL Server-Datenbank verwenden, notieren Sie hinter der **ALTER TABLE**-Anweisung die **DROP CONSTRAINT**-Anweisung, gefolgt vom Namen der Einschränkung. In [Listing 12.31](#) haben wir der Einschränkung den Namen *df_dsgvo_status* zugeordnet.

```
ALTER TABLE dsgvo DROP CONSTRAINT df_dsgvo_status;
```

Listing 12.31 Eine Standardwertzuordnung wieder entfernen (MS SQL Server)

Probieren Sie nun mit einer **INSERT**-Anweisung wie in [Listing 12.29](#), ob die Standardwertzuweisung nun aufgehoben ist. Verwenden Sie hierfür einen gültigen Primärschlüsselwert (5). Die MySQL-Datenbank ließ aufgrund eines Bugs (Community-

Version 8.0.38) die **INSERT**-Anweisung zum Zeitpunkt der Veröffentlichung dieses Buches nicht zu und reagierte mit einer Fehlermeldung:

```
Field 'information' doesn't have a default value
```

Die MariaDB-Datenbank liefert bei dem Versuch, die **INSERT**-Anweisung auszuführen, exakt die gleiche Fehlermeldung. Die PostgreSQL-Datenbank und die MS SQL-Datenbank verhalten sich hingegen erwartungsgemäß.

Mit der **SELECT**-Abfrage aus [Listing 12.26](#) können Sie überprüfen, ob der Spalte *status* tatsächlich kein Standardwert zugeordnet wurde. Wenn Sie die Anweisung ausführen, erhalten Sie von der PostgreSQL- bzw. MS SQL Server-Datenbank ein Ergebnis, in dem die Zeile eingefügt wurde und der Spalte *status* kein Standardwert mehr zugeordnet ist.

Achtung bei UNIQUE und PRIMARY KEY

Wenn eine Spalte einer **UNIQUE**-Einschränkung unterliegt, kann ihr kein Standardwert zugewiesen werden.

Das Gleiche gilt für eine Primärschlüsselspalte, in der Werte ebenfalls eindeutig (verschieden) sein müssen.



12.2.7 Einer Spalte eine UNIQUE-Einschränkung zuordnen

Damit eine Firma nur einmal bewertet wird, stattdessen wir die Spalte *firma* der Tabelle *kundeninfo* mit einer **UNIQUE**-Einschränkung aus.

[Listing 12.32](#) zeigt, wie eine **ADD CONSTRAINT**-Anweisung im Zusammenhang mit einer **ALTER TABLE**-Anweisung verwendet wird. Hinter dem Tabellennamen *kundeinfo* folgt die **ADD CONSTRAINT**-Anweisung, wiederum gefolgt von einem Namen für die Einschränkung. Hier wurde der Name *uq_kundeninfo_bewertung* gewählt. Hinter dem Namen der Einschränkung notieren Sie das Schlüsselwort **UNIQUE**, gefolgt vom Spaltennamen, der von einer einfachen Klammer umschlossen ist.

```
ALTER TABLE kundeninfo ADD CONSTRAINT uq_kundeninfo_firma
UNIQUE (firma);
```

Listing 12.32 Einer Spalte eine **UNIQUE**-Einschränkung zuordnen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Achtung, MS SQL Server betrachtet NULL-Werte ebenfalls eindeutig

Wenn Sie eine MS SQL Server-Datenbank verwenden, sollten Sie beachten, dass **NULL**-Werte ebenfalls als eindeutige Werte angenommen werden.





Eindeutigkeit der Werte sicher stellen

Wenn Sie einer Spalte nachträglich eine **UNIQUE**-Einschränkung zuweisen, muss sichergestellt sein, dass sie keine Werte enthält, die mehrfach vorkommen. Duplikate sind hier also nicht erlaubt. Wenn mehrfach gleiche Werte enthalten sind, wird eine nachträglich vorgenommene **UNIQUE**-Einschränkung mit einer Fehlermeldung quittiert.

Im nächsten Schritt überprüfen wir, ob die **UNIQUE**-Einschränkung ihre Aufgabe wahrnimmt und tatsächlich nur eindeutige Werte zulässt. Die Spalte *firma* der Tabelle *kundeinfo* enthält bereits einen Wert *Schrauben Klein GmbH*. Dieser Wert dürfte sich mittels einer **INSERT**-Anweisung also nicht einfügen lassen. Um die **UNIQUE**-Einschränkung zu prüfen, fügen Sie einfach die **INSERT**-Anweisung aus [Listing 12.33](#) aus:

```
INSERT INTO kundeinfo (infoid,firma)
VALUES (5, 'Schrauben Klein GmbH');
```

Listing 12.33 Die **UNIQUE**-Klausel durch eine **INSERT**-Anweisung mit einem bereits vorhandenen Wert prüfen

Wenn Sie die **INSERT**-Anweisung ausgeführt haben, liefert Ihnen die Datenbank eine Fehlermeldung, die darauf hinweist, dass eine **UNIQUE**-Einschränkung verletzt wurde:

```
Error Code: 1062. Duplicate entry 'Schrauben Klein GmbH' for key
'uq_kundeinfo_bewertung'
```

Wenn Sie den Wert für die Spalte *firma* in *Schrauben Karl* ändern, lässt sich die neue Zeile jedoch einfügen.

[Listing 12.34](#) zeigt Ihnen, wie Sie die **UNIQUE**-Einschränkung entfernen können. Hinter der **ALTER TABLE**-Klausel notieren Sie hierzu lediglich die **DROP CONSTRAINT**-Anweisung, gefolgt vom Namen der Einschränkung, die entfernt werden soll (hier: *uq_kundeinfo_firma*).

```
ALTER TABLE kundeinfo DROP CONSTRAINT uq_kundeinfo_firma;
```

Listing 12.34 Einen Index entfernen, der einer Spalte zugeordnet ist (MySQL, MariaDB, PostgreSQL, MS SQL Server)

12.2.8 Eine Spalte mit einer **CHECK**-Einschränkung versehen

In [Abschnitt 12.2.2](#) haben Sie einen Wert in die Spalte *bewertung* eingetragen, der prüft, ob der Wertebereich einer Spalte tatsächlich größer geworden ist. Mit der **UPDATE**-Anweisung aus [Listing 12.35](#) machen wir diese Änderung rückgängig, um der Spalte *bewertung* eine einfache **CHECK**-Einschränkung zuzuordnen:


```
UPDATE kundeinfo SET bewertung=1 WHERE infoid=3;
```

Listing 12.35 Den Wert einer Zeile für eine CHECK-Einschränkung anpassen

[Listing 12.36](#) fügt mit einer **ALTER TABLE**-Anweisung und der **ADD CONSTRAINT**-Anweisung die Einschränkung *kundeinfo_bewertung_check* zu der Tabelle *kundeinfo* hinzu. Hinter dem Namen folgt die eigentliche **CHECK**-Einschränkung. Hier wird geprüft, ob die Spaltenwerte der Spalte *bewertung* zwischen 1 und 5 sind.

```
ALTER TABLE kundeinfo
ADD CONSTRAINT kundeinfo_bewertung_check
CHECK(bewertung=1 OR bewertung=2 OR bewertung=3 OR bewertung=4 OR
bewertung=5);
```

Listing 12.36 Einer Tabelle eine CHECK-Einschränkung zuordnen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Jetzt haben wir die Spalte *bewertung* mit einer **CHECK**-Einschränkung versehen, die ausschließlich die Werte 1, 2, 3, 4 und 5 zulässt. Außerdem sind nicht definierte Werte (**NULL**) erlaubt. Diese Einschränkung überprüfen wir in [Listing 12.37](#) mit einer einfachen **INSERT**-Anweisung, in der die Einschränkung verletzt wird. Hierfür versuchen wir, den Spaltenwert 6 in die Spalte *bewertung* einzutragen:

```
INSERT INTO kundeinfo (infoid, firma, bewertung)
VALUES (5, 'Schrauben Karl GmbH', 6);
```

Listing 12.37 Eine CHECK-Einschränkung mit einer INSERT-Anweisung prüfen

Die Datenbank weist die neue Zeile zurück, weil sie gegen die Einschränkung *kundeinfo_bewertung_check* verstößt:

```
FEHLER: neue Zeile für Relation "kundeinfo" verletzt Check-Constraint
"kundeinfo_bewertung_check"
```

Die **CHECK**-Einschränkung entfernen Sie wieder mit einer **ALTER TABLE**-Anweisung, wie aus [Listing 12.38](#) hervorgeht. Hinter **ALTER TABLE** wird wieder die **DROP CONSTRAINT**-Anweisung ausgeführt. Notieren Sie hinter der **DROP CONSTRAINT**-Anweisung einfach den Namen der entsprechenden Einschränkung, in unserem Fall also *kundeinfo_bewertung_check*.

```
ALTER TABLE kundeinfo DROP CONSTRAINT kundeinfo_bewertung_check;
```

Listing 12.38 CHECK-Einschränkung entfernen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Auch hier überprüfen wir mit der **INSERT**-Anweisung aus [Listing 12.37](#), ob die **CHECK**-Einschränkung auch tatsächlich nicht mehr vorhanden ist. Wenn Sie die Anweisung ausgeführt haben, wird die neue Zeile ohne Fehlermeldung in die Tabelle *kundeinfo* eingefügt.

12.3 Spalten hinzufügen und entfernen

12.3.1 Einer Tabelle eine Spalte hinzufügen

Sie können eine Tabelle mit der **ALTER TABLE**-Anweisung um eine oder mehrere Spalten ergänzen.

Als Beispiel soll die Tabelle *kundeinfo* um die Spalte *datuminfo* ergänzt werden, damit wir das Datum, an dem die Information eingetragen wurde, speichern können.

Hierfür leiten wir in [Listing 12.39](#) mit **ALTER TABLE**, gefolgt vom Tabellennamen *kundeinfo*, die Anweisung ein. Es folgt der Befehl **ADD COLUMN**, der für das Hinzufügen von Spalten zuständig ist. Dahinter geben Sie den gewünschten Spaltennamen (hier *datuminfo*) und den Datentyp (hier **DATE**) an, der der Spalte zugeordnet werden soll.

```
ALTER TABLE kundeinfo ADD COLUMN datuminfo DATE;
```

Listing 12.39 Eine Spalte hinzufügen (MySQL, MariaDB, PostgreSQL)

Die **ALTER TABLE**-Klausel, mit der Sie in einer MS SQL Server-Datenbank eine Spalte hinzufügen, ist in [Listing 12.40](#) dargestellt. Hinter dem Tabellennamen der **ALTER TABLE**-Klausel notieren Sie lediglich die **ADD**-Anweisung, um die Spalte *datuminfo* des Typs **DATE** zu der Tabelle *kundeinfo* hinzuzufügen:

```
ALTER TABLE kundeinfo ADD datuminfo DATE;
```

Listing 12.40 Eine Spalte hinzufügen (MS SQL Server)

Wenn Sie die Anweisung aus [Listing 12.39](#) ausgeführt haben, prüfen Sie mit einer kleinen Abfrage wie in [Listing 12.41](#), ob die Spalte *datuminfo* auch tatsächlich zu der Tabelle *kundeinfo* hinzugefügt worden ist:

```
SELECT infoid,information,datuminfo FROM kundeinfo;
```

Listing 12.41 Prüfabfrage, um festzustellen, ob eine Spalte hinzugefügt wurde

In [Tabelle 12.5](#) werden die ersten drei Zeilen der Abfrage dargestellt. Die Spalte *datuminfo* wurde erfolgreich zu der Tabelle *kundeinfo* hinzugefügt. Da für die neue Spalte keine Werte bekannt sind, sind die Spaltenwerte **NULL**.

inford	information	datuminfo
1	Sehr guter Kunde!	NULL
2	Guter Kunde!	NULL
3	Nicht so guter Kunde!	NULL

Tabelle 12.5 Die ersten drei Zeilen der Prüfabfrage aus [Listing 12.41](#)

Spalten mit Einschränkungen hinzufügen

Wenn Sie eine neue Spalte zu einer Tabelle hinzufügen, können Sie sie mit sämtlichen Einschränkungen versehen, die hier aufgezeigt wurden. Die vorgestellte **ALTER TABLE**-Anweisung zum Hinzufügen einer Spalte müssen Sie gemäß den Anforderungen einfach nur erweitern. Sie können z. B. eine Spalte hinzufügen, die mit einer **PRIMARY KEY**-Einschränkung versehen ist. Sämtliche Einschränkungen, die Sie hier kennengelernt haben, können Sie verwenden, wenn Sie eine neue Spalte zu einer Tabelle hinzufügen.

12.3.2 Eine Spalte aus einer Tabelle entfernen

Wenn Sie eine Spalte aus einer Tabelle entfernen möchten, verwenden Sie ebenfalls die **ALTER TABLE**-Anweisung, gefolgt von einem Tabellennamen. Es folgt die **DROP COLUMN**-Anweisung, gefolgt von dem Namen der Spalte, die Sie zu löschen beabsichtigen. Wir wollen die Spalte *informationkunde2* löschen und notieren sie folglich hinter der **DROP COLUMN**-Anweisung:

```
ALTER TABLE kundeinfo DROP COLUMN informationkunde2;
```

Listing 12.42 Eine Spalte aus einer Tabelle entfernen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Auch hier prüfen wir, ob die Spalte *informationkunde2* auch tatsächlich aus der Tabelle *kundeinfo* entfernt wurde:

```
SELECT * FROM kundeinfo;
```

Listing 12.43 Prüfabfrage, die ermittelt, ob die Spalte »informationkunde2« gelöscht wurde

Als Ergebnis erhalten wir eine Tabelle, in der die Spalte *informationkunde2* nicht mehr enthalten ist. In [Tabelle 12.6](#) sind lediglich die ersten drei Zeilen der Ergebnistabelle dargestellt.

inford	information	firma	bewertung	datuminfo
1	Sehr guter Kunde!	Schrauben Meier GmbH	5	NULL
2	Guter Kunde!	Schrauben Peter GmbH	4	NULL
3	Nicht so guter Kunde!	Schrauben Klein GmbH	1	NULL

Tabelle 12.6 Ergebnis der Prüfabfrage



Achtung! Primärschlüsselspalte mit DROP COLUMN entfernen

Mit der **DROP COLUMN**-Anweisung der **ALTER TABLE**-Klausel können Sie auch eine Primärschlüsselspalte entfernen. Die Datenbanksysteme gehen davon aus, dass Sie auch **DROP COLUMN** meinen, wenn Sie **DROP COLUMN** verwenden.

12.4 Beziehungen zwischen Tabellen herstellen und entfernen

In diesem Abschnitt zeige ich Ihnen, wie Sie vorgehen, wenn Sie eine Beziehung zwischen Tabellen nachträglich herstellen wollen.

In [Abbildung 12.3](#) sind die Tabellen *premiumkunde* und *anschrift* dargestellt, die keine Beziehung zueinander haben. Wir möchten nun eine 1:n-Beziehung zwischen den Tabellen *premiumkunde* und *anschrift* abbilden. Es fehlt also eine Beziehung zwischen der Spalte *fk_kundeid* der Tabelle *anschrift* und der Primärschlüsselspalte *kundeid* der Tabelle *premiumkunde*.

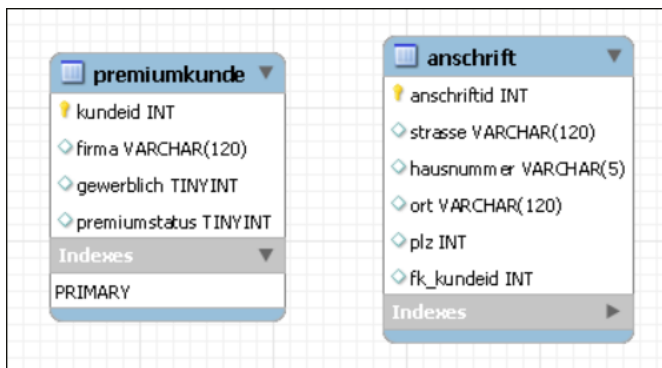


Abbildung 12.3 Tabellen ohne Beziehung

Tabelle 12.7 zeigt die Tabelle *premiumkunde*. Jeder Premiumkunde wird über einen eindeutigen Primärschlüsselwert der Spalte *kundeid* identifiziert.

kundeid	firma	gewerblich	premiumstatus
1	Schrauben Karl	1	6
2	Schrauben Meier	1	4
3	Schrauben Peters	1	1

Tabelle 12.7 Zeilen der Tabelle »premiumkunde«

In Tabelle 12.8 sehen Sie die Adressen, die jeweils den Premiumkunden zugeordnet sind. Hier existiert auch schon eine Spalte mit der Bezeichnung *fk_kundeid*, deren Werte auf die Primärschlüsselspalte *kundeid* der Tabelle *premiumkunde* verweisen. Im Grunde handelt es sich also schon um eine Fremdschlüsselspalte mit gültigen Werten. Wenn Sie sich die Fremdschlüsselwerte ansehen, werden Sie feststellen, dass jedem Premiumkunden drei Anschriften zugeordnet sind.

anschriftid	strasse	hausnummer	ort	plz	fk_kundeid
1	Mondstraße	8	Köln	50000	1
2	Saturnstraße	10	Koblenz	50100	1
3	Venusstraße	5	Bonn	50200	1
4	Marsstraße	7	Düsseldorf	50150	2
5	Saturnstraße	3	Aachen	50158	2
6	Uranusstraße	6	Hamburg	12000	2
7	Plutostraße	1	Bremen	50222	3
8	Merkurstraße	8	Bonn	50150	3
9	Jupiterstraße	6	Düsseldorf	50266	3

Tabelle 12.8 Zeilen der Tabelle »anschrift«

Die Tabellen *premiumkunde* und *anschrift* könnten Sie dazu verleiten, anzunehmen, dass doch alles in Ordnung ist und eine Fremdschlüsselbeziehung zwischen den Tabellen über die Spalten *kundeid* und *fk_kundeid* abgebildet ist.

Wie können wir prüfen, ob eine gültige Beziehung zwischen den Tabellen vorhanden ist? In [Abschnitt 10.4](#) haben Sie erfahren, dass es nicht möglich ist, eine Zeile aus einer

Elterntabelle zu entfernen, wenn noch Zeilen einer Kindtabelle auf sie referenzieren. Sichergestellt wird dies durch die Eigenschaft der referenziellen Integrität. Würden wir also versuchen, eine Zeile der Tabelle *premiumkunde* zu löschen, auf die Fremdschlüsselwerte aus der Tabelle *anschrift* verweisen, müsste uns die Datenbank dies mit einer Fehlermeldung quittieren. Ausgenommen sind hier natürlich die Beziehungen, die mit einer **CASCADE** festgelegt wurden.

Wenn Sie die **DELETE**-Anweisung aus [Listing 12.44](#) ausführen, werden Sie feststellen, dass die Zeile gelöscht werden kann. Hier erfolgt also keine Überwachung der referenziellen Integrität:

```
DELETE FROM premiumkunde WHERE kundeid=1;
```

Listing 12.44 Eine Zeile aus einer potenziellen Elterntabelle löschen

Da wir ja jetzt eine Zeile aus einer potenziellen Elterntabelle gelöscht haben, ist es erforderlich, auch die Zeilen der potenziellen Kindtabelle zu entfernen. Dieser Schritt ist notwendig, weil wir beabsichtigen, eine gültige Beziehung zwischen den Tabellen *premiumkunde* und *anschrift* herzustellen. [Listing 12.45](#) zeigt die dazu erforderliche **DELETE**-Anweisung:

```
DELETE FROM anschrift WHERE fk_kundeid=1;
```

Listing 12.45 Zeilen aus einer potenziellen Kindtabelle entfernen

[Listing 12.46](#) zeigt, wie eine Beziehung zwischen den Tabellen *premiumkunde* und *anschrift* hergestellt werden kann. Hierzu verwenden wir die **ADD CONSTRAINT**-Anweisung der **ALTER TABLE**-Klausel. Sie wird hinter dem Tabellennamen *anschrift* notiert. Hinter dem Namen der Einschränkung (*fk_premiumkunde_kundeid*) folgen hier die Schlüsselwörter **FOREIGN KEY**, gefolgt von dem Spaltennamen (*fk_kundeid*) der Tabelle (*anschrift*), der in einfachen Klammern steht. Als Nächstes müssen wir der Datenbank noch mitteilen, auf welche Primärschlüsselspalte die Spaltenwerte der Spalte *fk_kundeid* verweisen sollen. Hierzu notieren wir hinter dieser Angabe **REFERENCES**, um den Bezug zur Primärschlüsselspalte *kundeid* der Tabelle *premiumkunde* herzustellen.

```
ALTER TABLE anschrift ADD CONSTRAINT fk_premiumkunde_kundeid  
FOREIGN KEY (fk_kundeid) REFERENCES premiumkunde (kundeid);
```

Listing 12.46 Eine Fremdschlüsselbeziehung zu einer Tabelle hinzufügen
(MySQL, MariaDB, PostgreSQL, MS SQL Server)

Im Anschluss probieren wir abermals aus, eine Zeile aus der Elterntabelle (*premiumkunde*) zu entfernen, auf die Zeilen aus der Kindtabelle (*anschrift*) referenzieren. Hierzu ändern wir in [Listing 12.44](#) lediglich den Wert der Spalte *kundeid* in 2 (die Be-

dingung der **WHERE**-Klausel) und führen die Anweisung aus. Wenn Sie die **DELETE**-Anweisung ausgeführt haben, erhalten Sie folgende Fehlermeldung:

Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails

Die Fehlermeldung weist Sie darauf hin, dass Sie keine Elternzeile löschen können, weil dadurch eine Fremdschlüsselbeziehung verletzt wird.

Wenn Sie beabsichtigen, eine Fremdschlüsselbeziehung zu entfernen, so verwenden Sie bitte die Anweisung aus [Listing 12.47](#). Hier geben Sie hinter der **ALTER TABLE**-Anweisung die Anweisung **DROP CONSTRAINT** an, gefolgt vom Namen der Einschränkung:

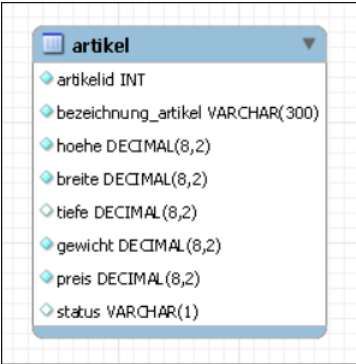
```
ALTER TABLE anschrift DROP CONSTRAINT fk_premiumkunde_kundeid;
```

Listing 12.47 Eine FOREIGN KEY-Einschränkung entfernen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

12.5 Übungen

Übung 1

Die Tabelle *artikel* aus [Abbildung 12.4](#) wurde leider nicht ganz richtig umgesetzt, so dass einige Änderungen erforderlich sind. Ändern Sie bitte den Tabellennamen von *artikel* in *artikelaktuell*. Bitte achten Sie jeweils auf die spezifische Version für die von Ihnen verwendete Datenbank.



The image shows a screenshot of a database table structure for a table named 'artikel'. The table has the following columns:

Column Name	Data Type
artikelid	INT
bezeichnung_artikel	VARCHAR(300)
hoehe	DECIMAL(8,2)
breite	DECIMAL(8,2)
tiefe	DECIMAL(8,2)
gewicht	DECIMAL(8,2)
preis	DECIMAL(8,2)
status	VARCHAR(1)

Abbildung 12.4 Die Tabelle »Artikel«

Übung 2

Als Nächstes soll die Spalte *bezeichnung_artikel* der Tabelle *artikelaktuell* in *bezeichnung* umbenannt werden. Achten Sie bitte auch hier auf die jeweilige Lösungsvariante der von Ihnen verwendeten Datenbank.

Übung 3

Bei der Definition der Spalte *status* der Tabelle *artikelaktuell* ist auch etwas schiefgelaufen. Bitte ändern Sie den Datentyp der Spalte *status* von `VARCHAR(1)` in `VARCHAR(300)`.

Übung 4

Die Tabelle *artikelaktuell* verfügt über keinen Primärschlüssel. Als idealer Kandidat kommt die Spalte *artikelid* infrage. Erweitern Sie die Spalte um eine Primärschlüsseleinschränkung.

Übung 5

Entfernen Sie die Primärschlüsseleinschränkung der Tabelle *artikelinfo* wieder. Wenn Sie eine PostgreSQL- oder MS SQL Server-Datenbank verwenden, beachten Sie bitte, dass Sie dazu die Bezeichnung (*pk_artikel_id*) der Primärschlüsseleinschränkung angeben müssen.

Übung 6

Die Spalte *tiefe* der Tabelle *artikelaktuell* lässt `NULL`-Werte zu. Bitte fügen Sie eine Einschränkung hinzu, die sicherstellt, dass keine `NULL`-Werte in die Spalte *tiefe* eingetragen werden können.

Übung 7

In Übung 6 haben Sie der Spalte *tiefe* der Tabelle *artikelaktuell* eine `NOT NULL`-Einschränkung zugeordnet. Bitte entfernen Sie die `NOT NULL`-Einschränkung, indem Sie der Spalte *tiefe* wieder eine `NULL`-Einschränkung zuordnen.

Übung 8

Der Spalte *preis* der Tabelle *artikelaktuell* ist derzeit kein Standardwert zugeordnet. Der Standardwert für die Spalte *preis* soll grundsätzlich mit 0,00 € belegt sein. Ändern Sie die Spalte so, dass dies stets gewährleistet ist.

Übung 9

Entfernen Sie den Standardwert, den Sie in Übung 8 der Spalte *preis* der Tabelle *artikelaktuell* zugeordnet haben.

Übung 10

Erweitern Sie die Spalte *bezeichnung* der Tabelle *artikelaktuell* um eine **UNIQUE**-Einschränkung, um sicherzustellen, dass ein Artikel stets nur einmal in der Tabelle aufgeführt ist.

Übung 11

Entfernen Sie die in Übung 10 zugeordnete **UNIQUE**-Einschränkung der Spalte *bezeichnung* der Tabelle *artikelaktuell*.

Übung 12

Die Spalte *preis* der Tabelle *artikelaktuell* soll lediglich Preise zulassen, die kleiner als 1.000 € sind. Ergänzen Sie die Tabelle *artikelaktuell* um eine **CHECK**-Einschränkung, die dies sicherstellt.

Übung 13

Entfernen Sie die zuvor angelegte **CHECK**-Einschränkung der Spalte *preis* der Tabelle *artikelaktuell*.

Übung 14

Als die Tabelle *artikelaktuell* erstellt wurde, wurde die Spalte *kommentar* vergessen. Fügen Sie die Spalte *kommentar* zur Tabelle *artikelaktuell* hinzu. Als Datentyp wählen Sie **VARCHAR(300)** aus.

Übung 15

Die in Übung 14 hinzugefügte Spalte *kommentar* wird doch nicht benötigt. Entfernen Sie die Spalte *kommentar* aus der Tabelle *artikelaktuell*.

Übung 16

Abbildung 12.5 zeigt die Tabellen *artikelinfo* und *positionartikel*.

Eine oder mehrere Bestellpositionen sollen jeweils einem Artikel zugeordnet werden. Bitte erstellen Sie eine 1:n-Beziehung zwischen den Tabellen *artikelinfo* und *positionartikel*, um die referenzielle Integrität herzustellen. Verwenden Sie für diese Übung die Spalte *fk_artikelinfo_artikelid* der Tabelle *positionartikel* und die Spalte *artikelid* der Tabelle *artikelinfo*, um die Beziehung zwischen den Tabellen herzustellen.

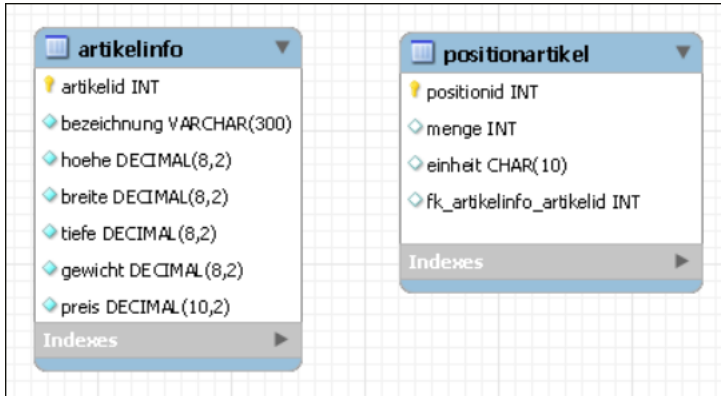


Abbildung 12.5 Die Tabellen »artikelinfo« und »positionartikel« stehen nicht in Beziehung zueinander.

Übung 17

Entfernen Sie die zuvor in Übung 16 erstellte Beziehung zwischen den Tabellen *artikelinfo* und *bestellposition*.

Lösung zu Übung 1

```
ALTER TABLE artikel RENAME artikelaktuell;
```

Listing 12.48 Die Tabelle »artikel« in »artikelaktuell« umbenennen (MySQL, MariaDB)

```
ALTER TABLE artikel RENAME TO artikelaktuell;
```

Listing 12.49 Die Tabelle »artikel« in »artikelaktuell« umbenennen (PostgreSQL)

```
EXEC SP_RENAME 'dbo.artikel', artikelaktuell;
```

Listing 12.50 Die Tabelle »artikel« in »artikelaktuell« umbenennen (MS SQL Server)

Lösung zu Übung 2

```
ALTER TABLE artikelaktuell CHANGE bezeichnung_artikel bezeichnung
VARCHAR(120);
```

Listing 12.51 Die Spalte »bezeichnung_artikel« der Tabelle »artikelaktuell« in »bezeichnung« umbenennen (MySQL, MariaDB)

```
ALTER TABLE artikelaktuell RENAME COLUMN bezeichnung_artikel TO bezeichnung;
```

Listing 12.52 Die Spalte »bezeichnung_artikel« der Tabelle »artikelaktuell« in »bezeichnung« umbenennen (PostgreSQL)

```
EXEC SP_RENAME 'dbo.artikelaktuell.bezeichnung_artikel',bezeichnung;
```

Listing 12.53 Die Spalte »bezeichnung_artikel« der Tabelle »artikelaktuell« in »bezeichnung« umbenennen (MS SQL Server)

Lösung zu Übung 3

```
ALTER TABLE artikelaktuell MODIFY status VARCHAR(300);
```

Listing 12.54 Den Datentyp der Spalte »status« von VARCHAR(1) in VARCHAR(300) ändern (MySQL, MariaDB)

```
ALTER TABLE artikelaktuell ALTER COLUMN status TYPE VARCHAR(300);
```

Listing 12.55 Den Datentyp der Spalte »status« von VARCHAR(1) in VARCHAR(300) ändern (PostgreSQL)

```
ALTER TABLE artikelaktuell ALTER COLUMN status VARCHAR(300);
```

Listing 12.56 Den Datentyp der Spalte »status« von VARCHAR(1) in VARCHAR(300) ändern (MS SQL Server)

Lösung zu Übung 4

```
ALTER TABLE artikelaktuell ADD CONSTRAINT pk_artikel_id PRIMARY  
KEY(artikelid);
```

Listing 12.57 Die Spalte »artikelid« als Primärschlüssel festlegen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 5

```
ALTER TABLE artikelaktuell DROP PRIMARY KEY;
```

Listing 12.58 Die Primärschlüsseleinschränkung der Tabelle »artikelaktuell« entfernen (MySQL, MariaDB)

```
ALTER TABLE artikelaktuell DROP CONSTRAINT pk_artikel_id;
```

Listing 12.59 Die Primärschlüsseleinschränkung der Tabelle »artikelaktuell« entfernen (PostgreSQL, MS SQL Server)

Lösung zu Übung 6

```
ALTER TABLE artikelaktuell MODIFY tiefe DECIMAL(8,2) NOT NULL;
```

Listing 12.60 Der Spalte »tiefe« der Tabelle »artikelaktuell« eine NOT NULL-Einschränkung zuordnen (MySQL, MariaDB)

```
ALTER TABLE artikelaktuell ALTER COLUMN tiefe SET NOT NULL;
```

Listing 12.61 Der Spalte »tiefe« der Tabelle »artikelaktuell« eine NOT NULL-Einschränkung zuordnen (PostgreSQL)

```
ALTER TABLE artikelaktuell ALTER COLUMN tiefe DECIMAL(8,2) NOT NULL;
```

Listing 12.62 Der Spalte »tiefe« der Tabelle »artikelaktuell« eine NOT NULL-Einschränkung zuordnen (MS SQL Server)

Lösung zu Übung 7

```
ALTER TABLE artikelaktuell MODIFY tiefe DECIMAL(8,2) NULL;
```

Listing 12.63 Der Spalte »tiefe« der Tabelle »artikelaktuell« eine NULL-Einschränkung zuordnen (MySQL, MariaDB)

```
ALTER TABLE artikelaktuell ALTER COLUMN tiefe DROP NOT NULL;
```

Listing 12.64 Der Spalte »tiefe« der Tabelle »artikelaktuell« eine NULL-Einschränkung zuordnen (PostgreSQL)

```
ALTER TABLE artikelaktuell ALTER COLUMN tiefe DECIMAL(8,2) NULL;
```

Listing 12.65 Der Spalte »tiefe« der Tabelle »artikelaktuell« eine NULL-Einschränkung zuordnen (MS SQL Server)

Lösung zu Übung 8

```
ALTER TABLE artikelaktuell  
  ALTER COLUMN preis SET DEFAULT 0.0;
```

Listing 12.66 Der Spalte »preis« den Wert »0.0« als Standardwert zuordnen (MySQL, MariaDB, PostgreSQL)

```
ALTER TABLE artikelaktuell ADD CONSTRAINT df_wert_preis DEFAULT 0.0 FOR preis;
```

Listing 12.67 Der Spalte »preis« den Wert »0.0« als Standardwert zuordnen (MS SQL Server)

Lösung zu Übung 9

```
ALTER TABLE artikelaktuell  
  ALTER COLUMN preis DROP DEFAULT;
```

Listing 12.68 Die Standardwerteinschränkung der Spalte »preis« der Tabelle »artikelaktuell« entfernen (MySQL, MariaDB, PostgreSQL)

```
ALTER TABLE artikelaktuell DROP CONSTRAINT df_wert_preis;
```

Listing 12.69 Die Standardwerteinschränkung der Spalte »preis« der Tabelle »artikelaktuell« entfernen (MS SQL Server)

Lösung zu Übung 10

```
ALTER TABLE artikelaktuell ADD CONSTRAINT artikelaktuell_bezeichnung_unique  
UNIQUE (bezeichnung);
```

Listing 12.70 Eine UNIQUE-Einschränkung der Spalte »bezeichnung« zuordnen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 11

```
ALTER TABLE artikelaktuell DROP CONSTRAINT artikelaktuell_bezeichnung_unique;
```

Listing 12.71 Die UNIQUE-Einschränkung der Spalte »bezeichnung« entfernen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 12

```
ALTER TABLE artikelaktuell  
ADD CONSTRAINT artikelaktuell_preis_check CHECK(preis<1000);
```

Listing 12.72 Die Spalte »preis« der Tabelle »artikelaktuell« um eine Einschränkung erweitern, die sicherstellt, dass der Wert kleiner als 1.000 € ist (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 13

```
ALTER TABLE artikelaktuell DROP CONSTRAINT artikelaktuell_preis_check;
```

Listing 12.73 Die CONSTRAINT-Einschränkung der Spalte »preis« entfernen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 14

```
ALTER TABLE artikelaktuell ADD COLUMN kommentar VARCHAR(300);
```

Listing 12.74 Die Tabelle »artikelaktuell« um die Spalte »kommentar« erweitern (MySQL, MariaDB, PostgreSQL)

```
ALTER TABLE artikelaktuell ADD kommentar VARCHAR(300);
```

Listing 12.75 Die Tabelle »artikelaktuell« um die Spalte »kommentar« erweitern (MS SQL Server)

Lösung zu Übung 15

```
ALTER TABLE artikelaktuell DROP COLUMN kommentar;
```

Listing 12.76 Die Spalte »kommentar« aus der Tabelle »artikelaktuell« entfernen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 16

```
ALTER TABLE positionartikel ADD CONSTRAINT fk_artikelinfo_artikelid FOREIGN  
KEY (fk_artikelinfo_artikelid) REFERENCES artikelinfo (artikelid);
```

Listing 12.77 Eine 1:n-Beziehung zwischen den Tabellen »artikelinfo« und »positionartikel« herstellen (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Lösung zu Übung 17

```
ALTER TABLE positionartikel DROP CONSTRAINT fk_artikelinfo_artikelid;
```

Listing 12.78 Die Beziehung zwischen den Tabellen »artikelinfo« und »positionartikel« aufheben (MySQL, MariaDB, PostgreSQL, MS SQL Server)

Kapitel 13

Mit SQL rechnen

*Jetzt wird es mathematisch. Mit SQL können Sie auch rechnen!
Denn was wäre eine Datenbank ohne die Möglichkeit, diese Daten
auch weiterzuverarbeiten?*

Für die Rechenoperationen *Addition*, *Subtraktion*, *Multiplikation* und *Division* bietet SQL die Operatoren +, -, *, / und % an (siehe Tabelle 13.1). Bei dem %-Operator handelt es sich um den *Restwertoperator (Modulo-Operator)*. Berechnungen können Sie an vielen Stellen durchführen, beispielsweise in der Spaltenauswahlliste, in einer **ORDER BY**-Klausel, in einer **WHERE**-Klausel und an anderen Orten einer **SELECT**-Anweisung.

Operator	Funktion	Beispiel
+	Addition	4+4=8
-	Subtraktion	5-1=4
*	Multiplikation	2*3=6
/	Division	6/3=2
%	Restwert (Modulo)	5%2=1

Tabelle 13.1 Mathematische Operatoren in SQL

Für diese Operatoren gelten die allgemein bekannten Vorrangregeln. Stellen Sie sich vor, Sie führen folgende Rechenoperation durch: $5+3*2$. Sie kommen hier schnell zum Ergebnis 11. Eine Multiplikation oder Division ist immer vor einer Addition oder Subtraktion durchzuführen, da die *Assoziationsregel* gilt – oder ganz einfach: »Punktrechnung geht vor Strichrechnung.« Sie können den Vorrang der Rechenoperationen durch Klammersetzung jedoch verändern. Schreiben Sie beispielsweise $(5+3)*2$, so würde die Datenbank 16 als Ergebnis zurückliefern.

Bevor wir uns mit Berechnungen in der Spaltenauswahlliste befassen, müssen wir uns noch die Ausdrücke (engl. *expressions*) ansehen. Zur Auswahl stehen Ihnen folgende Ausdrücke:

- ▶ Werte
- ▶ Konstanten
- ▶ Variablen
- ▶ Operationen
- ▶ Berechnungen
- ▶ Vergleiche

Ein Ausdruck führt letztendlich immer zu einem Ergebnis und kann überall da verwendet werden, wo ein Wert stehen kann. Er wird nämlich von der Datenbank ausgewertet (das Fachwort ist *evaluiert*) und liefert dann einen Wert zurück. Ein Ausdruck kann daher in der Spaltenauswahlliste vorkommen oder in einer **WHERE**- oder **ORDER BY**-Klausel verwendet werden. Später werden Sie sehen, dass so ein Ausdruck auch noch an anderen Stellen eingesetzt werden kann.



Ausdrücke (Expressions)

In SQL werden sogenannte *Ausdrücke* verwendet. Hierbei kann es sich um Rechenoperationen oder um Bedingungen handeln, die ausgewertet werden. Ausdrücken ist gemein, dass sie ausgewertet werden und zu einem Ergebnis führen.

Ausdrücke können überall da verwendet werden, wo Werte erwartet werden. Daher können Sie Ausdrücke auch in zahlreichen anderen SQL-Anweisungen (z. B. in **INSERT** und **UPDATE**) einsetzen.

13.1 Spaltenwerte addieren

In [Listing 13.1](#) werden in der Spaltenauswahlliste der **SELECT**-Anweisung die Spalten *bonus* und *monatslohn* addiert. Wenn Sie eine Addition von Spaltenwerten wünschen, notieren Sie einfach die Spaltenbezeichnungen, die Sie addieren wollen, und dazwischen einen **+**-Operator. Natürlich werden hier keine Spalten addiert, sondern die Werte, die in den jeweiligen Spalten enthalten sind. Im vorigen Abschnitt haben Sie erfahren, dass Rechenoperationen ausgewertet werden und zu einem Ergebnis führen. Für diesen berechneten Spaltenwert gibt es keine definierte Spaltenbezeichnung, sodass sich für uns hier wieder die Gelegenheit ergibt, Spaltenaliasse zu verwenden. Dem Ausdruck zur Berechnung der Summe aus den Werten der Spalten *bonus* und *monatslohn* wird hier der Spaltenalias *summe* zugeordnet.

SELECT

```
name,
vorname,
```



```
bonus,  
monatslohn,  
bonus + monatslohn AS summe  
FROM  
mitarbeiter;
```

Listing 13.1 Die Spaltenwerte »bonus« und »monatslohn« addieren

In [Tabelle 13.2](#) sehen Sie, dass die Datenbank die Werte der Spalten *bonus* und *monatslohn* für uns addiert und als berechnete Spalte mit dem Spaltenalias *summe* zur Ergebnistabelle hinzugefügt hat.

name	vorname	bonus	monatslohn	summe
Müller	Ralf	500.00	4000.00	4500.00
Schneider	Petra	500.00	4000.00	4500.00
Klein	Thomas	500.00	4000.00	4500.00
Lang	Ute	280.50	2800.00	3080.50

Tabelle 13.2 Ergebnis einer Addition der Spaltenwerte aus [Listing 13.1](#)

Sie können natürlich auch mehr als zwei Summanden verwenden. In [Listing 13.2](#) wird zusätzlich ein konstanter Wert 1000 addiert:

```
SELECT  
name,  
vorname,  
bonus,  
monatslohn,  
1000 AS wert1000,  
bonus + monatslohn + 1000 AS summe  
FROM  
mitarbeiter;
```

Listing 13.2 Addition der Spalten »bonus«, »monatslohn« und des numerischen Werts »1000«

In [Tabelle 13.3](#) sehen Sie, dass die Spaltenwerte der Spalten *bonus* und *monatslohn* und der Wert 1000 addiert wurden und dass das Ergebnis der Addition in der Spalte *summe* ausgegeben wird.

name	vorname	bonus	monatslohn	wert1000	summe
Müller	Ralf	500.00	4000.00	1000	5500.00
Schneider	Petra	500.00	4000.00	1000	5500.00
Klein	Thomas	500.00	4000.00	1000	5500.00
Lang	Ute	280.50	2800.00	1000	4080.50

Tabelle 13.3 Ergebnis für die Addition von drei Summanden aus [Listing 13.2](#)

13.2 Spaltenwerte subtrahieren

Selbstverständlich ist eine Subtraktion in der Spaltenauswahlliste genauso einfach wie die Addition. Um Spaltenwerte voneinander zu subtrahieren, verwenden wir einfach das Minuszeichen, wie in [Listing 13.3](#) dargestellt. Von den Spaltenwerten der Spalte *bonus* werden hier einfach die Spaltenwerte der Spalte *monatslohn* subtrahiert.

```
SELECT
    name,
    vorname,
    bonus,
    monatslohn,
    bonus - monatslohn AS differenz
FROM
    mitarbeiter;
```

Listing 13.3 Die Spalte »monatslohn« von der Spalte »bonus« subtrahieren

[Tabelle 13.4](#) zeigt das Ergebnis der Subtraktion in der Spalte *differenz*. Eine Subtraktion kann natürlich auch einen negativen Wert ergeben.

name	vorname	bonus	monatslohn	differenz
Müller	Ralf	500.00	4000.00	-3500.00
Schneider	Petra	500.00	4000.00	-3500.00
Klein	Thomas	500.00	4000.00	-3500.00
Lang	Ute	280.50	2800.00	-2519.50

Tabelle 13.4 Ergebnis für die Subtraktion der Spaltenwerte aus [Listing 13.3](#)

13.3 Spaltenwerte multiplizieren

In [Listing 13.4](#) sehen Sie eine Multiplikation von Spaltenwerten in der Spaltenauswahlliste. Die **SELECT**-Anweisung fordert die Datenbank auf, die Werte der Spalten *stundenpromonat* und *stundenlohn* miteinander zu multiplizieren und unter dem Spaltenalias *gehalt* als Ergebnis zurückzuliefern.

SELECT

```
name,
vorname,
stundenpromonat,
stundenlohn,
stundenpromonat*stundenlohn AS gehalt
```

FROM

```
mitarbeiter;
```

Listing 13.4 Spaltenwerte miteinander multiplizieren

In [Tabelle 13.5](#) sehen Sie das Ergebnis der Multiplikation aus den Spaltenwerten der Spalten *stundenpromonat* und *stundenlohn*, das von der Datenbank unter dem Spaltenalias *gehalt* zurückgeliefert wird.

Name	vorname	stundenpromonat	stundenlohn	gehalt
Müller	Ralf	160	20.80	3328.00
Schneider	Petra	180	28.50	5130.00
Klein	Thomas	165	20.80	3432.00
Lang	Ute	170	20.00	3400.00

Tabelle 13.5 Die Spalte »gehalt« bildet das Ergebnis für die Multiplikation der Spalten »stundenlohn« und »stundenpromonat«.

13.4 Spaltenwerte dividieren

Eine Division berechnen Sie genauso einfach wie eine Multiplikation. Für die Division verwenden Sie lediglich den Divisionsoperator. In [Listing 13.5](#) sehen Sie in der Spaltenauswahlliste eine Division, in der die Spaltenwerte der Spalten *monatslohn* und *stundenpromonat* dividiert werden und dem Ergebnis der Berechnung der Spaltenalias *stundenlohn* zugeordnet wird.

```
SELECT
    name,
    vorname,
    monatslohn,
    stundenpromonat,
    monatslohn / stundenpromonat AS stundenlohn
FROM
    mitarbeiter;
```

Listing 13.5 Spaltenwerte dividieren

Das Ergebnis dieser Berechnung, das Sie in [Tabelle 13.6](#) sehen, enthält nicht nur einen ganzzahligen Anteil, sondern auch einen Fließkommaanteil.

Name	vorname	monatslohn	stundenpromonat	stundenlohn
Müller	Ralf	4000.00	160	25.000000
Schneider	Petra	4000.00	180	22.222222
Klein	Thomas	4000.00	165	24.242424
Lang	Ute	2800.00	170	16.470588

Tabelle 13.6 Ergebnistabelle für eine Division von Spaltenwerten

13.5 Den Restwert einer Division von Spaltenwerten berechnen

Im nächsten Beispiel berechnen wir den *Restwert* einer Division mit dem *Modulo-Operator*. Ein Restwert ist dann vorhanden, wenn bei einer Division ganzer Zahlen der Zähler nicht vollständig durch den Nenner teilbar ist. In [Listing 13.6](#) sehen Sie in der Spaltenauswahlliste lediglich einen Ausdruck, der einen Restwert einer Division aus dem Divisor 5 und dem Dividend 2 ermittelt. Um Ihnen die Funktion des Modulo-Operators vorzustellen, nutze ich hier die kürzeste aller **SELECT**-Abfragen. Hinter dem Schlüsselwort folgt ein Ausdruck, der zu einem Ergebnis führt. Abgeschlossen wird das Ganze mit einem Semikolon. Wenn Sie also nur die Ergebnisse eines Ausdrucks ermitteln wollen, ist es nicht erforderlich, eine Tabelle anzugeben.

```
SELECT 5%2 AS rest;
```

Listing 13.6 Einen Restwert berechnen

In [Tabelle 13.7](#) sehen Sie das Ergebnis der **SELECT**-Abfrage aus [Listing 13.6](#).

Rest
1

Tabelle 13.7 Ergebnistabelle der Restwertberechnung aus [Listing 13.6](#)

Da wir in dem Ausdruck den Restwert aus zwei konstanten Werten ermitteln, erhalten wir für jede Zeile das gleiche Ergebnis. Wenn Sie den Wert 5 durch den Wert 2 teilen, passt der Wert 2 zweimal in den Wert 5. Wenn Sie das Ergebnis 2 mit dem Divisor multiplizieren, ergibt sich ein Wert 4; wenn Sie diesen vom Ausgangswert abziehen, erhalten Sie den Restwert. Als Restwert bleibt also 1 übrig.

13.6 Nach dem Ergebnis einer Berechnung filtern

Bisher haben Sie Bedingungen in der **WHERE**-Klausel verwendet, in denen Sie Spaltenwerte mit konstanten Werten verglichen haben, wobei die unterschiedlichsten Vergleichsoperatoren zum Einsatz kamen.

In [Listing 13.7](#) multiplizieren wir zuerst die Werte der Spalten *stundenlohn* und *stundenpromonat*. Das Ergebnis vergleichen wir mithilfe des Kleiner-als-Operators **<** mit den Spaltenwerten der Spalte *monatslohn* und ermitteln somit alle Mitarbeiter, die gemäß der Multiplikation aus *stundenlohn* und *stundenpromonat* weniger verdienen als den hinterlegten Wert der Spalte *monatslohn*:

```
SELECT
    name,
    vorname,
    stundenlohn*stundenpromonat AS monatslohnberechnet,
    monatslohn
FROM
    mitarbeiter
WHERE
    stundenlohn*stundenpromonat<monatslohn;
```

Listing 13.7 Nach einem berechneten Wert mit einer **WHERE**-Klausel filtern

In [Tabelle 13.8](#) sehen Sie, dass zwei Datensätze herausgefiltert wurden, in denen der berechnete Wert des Monatslohns kleiner ist als der Monatslohn, der in der Spalte *monatslohn* hinterlegt ist.

name	vorname	monatslohnberechnet	monatslohn
Müller	Ralf	3328.00	4000.00
Klein	Thomas	3432.00	4000.00

Tabelle 13.8 Ergebnistabelle für die WHERE-Klausel mit den berechneten Filterwerten aus [Listing 13.7](#)

13.7 Nach dem Ergebnis einer Berechnung sortieren lassen

Die in [Abschnitt 13.2](#) durchgeführte Subtraktion nutzen wir jetzt, um eine absteigende Sortierung mit der **ORDER BY**-Klausel zu realisieren. In [Listing 13.8](#) wird die **SELECT**-Anweisung um die **ORDER BY**-Klausel erweitert:

```
SELECT
    name,
    vorname,
    bonus,
    monatslohn,
    bonus - monatslohn AS sonderbonus
FROM
    mitarbeiter
ORDER BY
    bonus-monatslohn
/* sonderbonus */
DESC;
```

Listing 13.8 Nach einem berechneten Ergebnis einer Subtraktion absteigend sortieren lassen

Mit dem Schlüsselwort **DESC** wird für das Ergebnis der Subtraktion *bonus-monatslohn* eine absteigend sortierte Ergebnistabelle angefordert. Beachten Sie, dass Sie in der **ORDER BY**-Klausel anstelle des Ausdrucks der Subtraktion auch den Spaltenalias *sonderbonus* verwenden könnten, der in der Spaltenauswahlliste dem Ausdruck der Subtraktion zugeordnet wurde.

In [Tabelle 13.9](#) sehen Sie eine Liste, die gemäß den Ergebniswerten der Subtraktion absteigend sortiert wurde. In der **ORDER BY**-Klausel kann also nicht nur nach existierenden Spalten einer bestehenden Tabelle sortiert werden, sondern auch nach Ergebnissen von mathematischen Ausdrücken.

Name	vorname	bonus	monatslohn	sonderbonus
Oberscheidt	Michael	200000.00	1800.00	198200.00
Mey	Lisa	50000.00	6000.00	44000.00
Jule	Stefanie	10000.00	6000.00	4000.00
Sternental	Marlene	10000.00	7500.00	2500.00

Tabelle 13.9 Ergebnistabelle, die nach dem Ergebnis der Subtraktion mittels DESC absteigend sortiert wurde



Zusammenfassung: Berechnungen in der SELECT-Anweisung durchführen

In einer **SELECT**-Anweisung können Sie mithilfe der bekannten Rechenoperatoren + (Addition), - (Subtraktion), * (Multiplikation), / (Division) und % (Modulo) Berechnungen in der Spaltenauswahlliste durchführen.

Außerdem ist es möglich, Ergebnisse in einer **ORDER BY**-Klausel als Sortierkriterium zu verwenden oder aber nach Ergebnissen in einer Bedingung einer **WHERE**-Klausel Zeilen zu filtern.

13.8 Übungen

Tabelle 13.10 dient als Basis für die Übungen, in denen wir einfache Berechnungen durchführen werden. Sie enthält die Spalten *id*, *jahr*, *q1*, *q2*, *q3*, *q4*, *miete*, *nebkosten*, *personal* und *invest*.

id	Jahr	q1	q2	q3	q4	miete	nebkosten	personal	invest
1	2000	223000	148217	210000	26783	400000	100000	500000	5000000
2	2001	275000	150716	222893	199054	400500	122000	550000	500000
3	2002	298000	210898	318055	320780	401600	130000	600000	800000
4	2003	270456	201888	343567	301298	4025000	140000	650000	900000

Tabelle 13.10 Die Tabelle »umsatzausgaben«

Übung 1

Formulieren Sie eine **SELECT**-Anweisung, in der Sie die Spalten *jahr*, *q1*, *q2*, *q3* und *q4* abfragen und die Summe aus den Spalten *q1* bis *q4* bilden. Ordnen Sie diesem Ausdruck den Alias *summequartale* zu.

Übung 2

In dieser Übung soll der Spaltenwert der Spalte *personal* von den Spaltenwerten der Spalte *invest* in der Spaltenauswahlliste subtrahiert werden. Fragen Sie außerdem die Spalte *jahr* über die Spaltenauswahlliste mit ab.

Übung 3

In dieser Übung ermitteln Sie den Gewinn des Unternehmens. Hierzu fragen Sie zunächst die Summe der Umsätze ab, von der Sie die Kosten subtrahieren. Addieren Sie dazu die Umsätze der Quartale 1 bis 4; hinter der letzten Addition folgt die Subtraktion der Spaltenwerte *miete*, *nebenkosten*, *personal* und *invest*.

Übung 4

Formulieren Sie anhand der Gesamtkosten für das Jahr 2003 eine Abfrage, in der die Kosten für die nächsten 10 Jahre hochgerechnet werden. Summieren Sie zuerst sämtliche anfallenden Kosten, und multiplizieren Sie sie mit dem Wert 10. Dem Ausdruck ordnen Sie in der Spaltenauswahlliste den Alias *nebenkosten10jahre* zu. Welcher potenzielle Fehler lauert hier?

Übung 5

Berechnen Sie die Summe sämtlicher Kosten, und dividieren Sie die sich ergebende Summe durch 4, um die Kosten je Quartal zu berechnen. Dem Ausdruck ordnen Sie den Alias *kostenquartal* zu.

Übung 6

In dieser Übung benötigen Sie das *kostenquartal* aus Übung 5. Zusätzlich sollen die Zeilen absteigend nach dem Ergebnis des Ausdrucks sortiert ausgegeben werden.

Lösung zu Übung 1

SELECT

```
jahr,  
q1,  
q2,  
q3,  
q4,  
q1+q2+q3+q4 AS summequartale
```

FROM

```
umsatzausgaben;
```

Listing 13.9 SELECT-Abfrage mit einer Addition von Spaltenwerten

Lösung zu Übung 2

SELECT

jahr,
invest-personal **AS** differenz

FROM

umsatzausgaben;

Listing 13.10 SELECT-Abfrage mit einer Subtraktion von Spaltenwerten

Lösung zu Übung 3

SELECT

q1,
q2,
q3,
q4,
q1+q2+q3+q4-miete-nebkosten-personal-invest **AS** gewinn

FROM

umsatzausgaben;

Listing 13.11 SELECT-Abfrage mit einer Addition und einer Subtraktion

Lösung zu Übung 4

Denken Sie daran, die Addition mit Klammern von der anschließenden Multiplikation zu trennen:

SELECT

(miete+nebkosten+personal+invest) * 10
AS nebenkosten10jahre

FROM

umsatzausgaben

WHERE

jahr=2003;

Listing 13.12 SELECT-Abfrage mit einer Multiplikation und einer WHERE-Klausel

Lösung zu Übung 5

SELECT

(
miete+
nebkosten+
personal+
invest

```
) / 4 AS kostenquartal  
FROM  
umsatzausgaben;
```

Listing 13.13 SELECT-Abfrage mit einer Division

Lösung zu Übung 6

```
SELECT  
(  
    miete+  
    nebkosten+  
    personal+  
    invest  
) / 4 AS kostenquartal  
FROM  
umsatzausgaben  
ORDER BY  
kostenquartal DESC;
```

Listing 13.14 SELECT-Abfrage mit einer Division und einer ORDER BY-Klausel

Kapitel 14

Skalarfunktionen anwenden

SQL bietet Ihnen zahlreiche Funktionen für die unterschiedlichsten Aufgaben. Hier erfahren Sie, wie Skalarfunktionen grundsätzlich verwendet werden und wie Sie Textwerte bearbeiten oder mathematische Operationen auf Spaltenwerte bzw. ein Literal anwenden.

In diesem Kapitel stelle ich Ihnen *Skalarfunktionen* vor, die Sie auf Spaltenwerte einer Zeile anwenden können. Funktionen sind kleine Programme, die bestimmte Aufgaben erfüllen. Wenn etwa Spaltenwerte einer Spalte zu Beginn führende Leerzeichen enthalten, wird eine **WHERE**-Klausel eine Bedingung nicht richtig auswerten können. Die Lösung wäre, eine Skalarfunktion zu verwenden, die die führenden Leerzeichen aus Spaltenwerten entfernt. Da Skalarfunktionen auf Spalten und deren Werte angewendet werden, können Sie sie auch in der **ORDER BY**-Klausel und natürlich auch in der Spaltenauswahlliste einsetzen.

Skalarfunktionen können also überall angewendet werden, wo Spaltenwerte bzw. Ausdrücke stehen können. Stellen Sie sich eine Skalarfunktion als ein Programm vor, das Spaltenwerte übergeben bekommt und Ihnen exakt einen Wert als Ergebnis zurückliefert. Vielleicht kennen Sie Begriff »Skalar« ja noch aus der Mathematik: Ein Skalar besteht immer nur aus einem Wert.

Es gibt zahlreiche Funktionen, und zwar für folgende Anwendungsgebiete:

- ▶ numerische Skalarfunktionen
- ▶ Zeichenketten-Skalarfunktionen
- ▶ Datumsskalarfunktionen
- ▶ Konvertierungsfunktionen

Datumsfunktionen werde ich in [Kapitel 16](#), »Mit Zeit und Datum arbeiten«, behandeln. Und wenn Sie selbst Funktionen und Prozeduren entwickeln möchten, bieten Ihnen [Kapitel 21](#) und [Kapitel 22](#) einen kurze Überblick.

In [Abschnitt 14.1](#) lernen Sie Funktionen kennen, die Sie auf Textwerte anwenden können. In [Abschnitt 14.2](#) erkläre ich Ihnen Funktionen, die auf Zahlenwerte anwendbar sind. Abschließend sehen wir uns dann in [Abschnitt 14.3](#) noch die verschachtelten Funktionsaufrufe an.

Sie merken schon, dass ich hier das Wort »Funktion« für den etwas sperrigeren Begriff »Skalarfunktion« verwende. In der Praxis ist diese Kurzform gebräuchlicher und reicht in den aller meisten Fällen aus.

14.1 Funktionen für Textwerte

In Tabelle 14.1 liste ich Ihnen fünf Skalarfunktionen auf, mit denen Sie Textwerte einer Spalte oder eines Literals nach Ihren Wünschen modifizieren. Es handelt sich um eine kleine Auswahl, die ich bewusst für den Einstieg in das Thema Funktionen gewählt habe.

Skalarfunktion	Funktionsbeschreibung
LOWER (spaltenname)	Der Funktion LOWER übergeben Sie einen Spaltennamen oder ein Textliteral. Sie wandelt dann sämtliche Buchstaben der Spaltenwerte in Kleinbuchstaben um und gibt Ihnen diesen Wert zurück.
UPPER (spaltenname)	Der Funktion UPPER übergeben Sie einen Spaltennamen oder ein Literal. Sie wandelt sämtliche Zeichen der Spaltenwerte in Großbuchstaben um und gibt uns diesen Wert zurück.
TRIM (spaltenname)	Der Funktion TRIM übergeben Sie einen Spaltennamen oder ein Literal. Sie entfernt sämtliche Leerzeichen und Tabulatoren vor und hinter den Spaltenwerten und gibt das Ergebnis als Wert zurück.
SUBSTRING (spaltenname FROM x FOR y)	Der Funktion SUBSTRING übergeben Sie einen Spaltennamen oder ein Textliteral, gefolgt von dem Schlüsselwort FROM und einem ganzzahligen numerischen Wert, der die Startposition x festlegt. Danach folgt das Schlüsselwort FOR und schließlich ein ganzzahliger numerischer Wert y, der die Länge der zu extrahierenden Zeichenkette festlegt. Diese Funktion werden Sie häufig in Ihrer praktischen Arbeit anwenden, um Texte aus Spaltenwerten zu extrahieren.
CONCAT (spaltenname1, spaltenname2, ...) (spaltenname1 spaltenname2 ...) (spaltenname1 + spaltenname2 + ...)	Die MySQL- bzw. MariaDB-Funktion CONCAT fügt eine beliebige Anzahl von Spaltenwerten zusammen. Die Anzahl der als Parameter übergebenen Spaltennamen ist variabel. Im SQL-Standard wird der <i>Concatination Operator</i> verwendet, um Spaltenwerte aneinanderzufügen. MS SQL Server verwendet den ++Operator, um Spaltenwerte zusammenzufügen.

Tabelle 14.1 Spezialisten, die Textwerte modifizieren

14.1.1 Zeichenkette in Kleinbuchstaben umwandeln (LOWER)

Mit der Skalarfunktion **LOWER** wandeln Sie Spaltenwerte, die einen Text darstellen, in Kleinbuchstaben um. Hierzu übergeben Sie der Funktion einfach den Spaltennamen als Parameter.

In diesem Beispiel sollen die Spaltenwerte der Spalte *artikelbezeichnung* der Tabelle *produkt* in Kleinschreibung ausgegeben werden. Die Funktion **LOWER** führt in der Spaltenauswahlliste zu einem Ergebnis. Es handelt sich also um einen Ausdruck. Wir ordnen diesem Ausdruck einen Spaltenalias *artikelbezeichnungklein* zu. Um den Unterschied zwischen dem Spaltenwert der Spalte *artikelbezeichnung* und dem Ergebnis der **LOWER**-Funktion sichtbar zu machen, nehmen wir in [Listing 14.1](#) auch die Spalte *artikelbezeichnung* in die Spaltenauswahlliste mit auf.

SELECT

```
artikelbezeichnung,  
  LOWER(artikelbezeichnung) AS artikelbezeichnungklein
```

FROM

```
produkt;
```

Listing 14.1 Anwendung der LOWER-Funktion in einer Spaltenauswahlliste

artikelbezeichnung	artikelbezeichnungklein
Schraube 1	schraube 1
Schraube 2	schraube 2
Schraube 3	schraube 3
Schraube 4	schraube 4

Tabelle 14.2 Ergebnis des Funktionsaufrufs LOWER in [Listing 14.1](#)

Der erste Buchstabe einer jeden Artikelbezeichnung wurde durch den Aufruf der Funktion **LOWER** mit dem Parameter *artikelbezeichnung* in einen Kleinbuchstaben umgewandelt. Wenn innerhalb der Zeichenketten der Spaltenwerte weitere Großbuchstaben vorgekommen wären, hätte die Funktion **LOWER** diese ebenfalls in Kleinbuchstaben umgewandelt.

14.1.2 Spaltenwerte in Großbuchstaben umwandeln (UPPER)

Die Funktion **UPPER** wandelt sämtliche Buchstaben innerhalb einer Zeichenkette in Großbuchstaben um. In [Listing 14.2](#) übergeben wir der Funktion **UPPER** wieder den Spaltennamen *artikelbezeichnung*, um die Spaltenwerte der Spalte *artikelbezeich-*

nung in Großbuchstaben zurückzuerhalten. Um das Ergebnis wieder mit dem ursprünglichen Spaltenwert vergleichen zu können, nehmen wir wieder die Spalte *artikelbezeichnung* in die Spaltenauswahlliste mit auf.

```
SELECT
    artikelbezeichnung,
    UPPER(artikelbezeichnung) AS artikelbezeichnunggroß
FROM
    produkt;
```

Listing 14.2 Anwendung der UPPER-Funktion in einer Spaltenauswahlliste

Sämtliche Spaltenwerte der Spalte *artikelbezeichnung* wurden in Tabelle 14.3 durch die Funktion `UPPER` in Großbuchstaben umgewandelt.

artikelbezeichnung	artikelbezeichnunggroß
Schraube 1	SCHRAUBE 1
Schraube 2	SCHRAUBE 2
Schraube 3	SCHRAUBE 3
Schraube 4	SCHRAUBE 4

Tabelle 14.3 Ergebnis des Funktionsaufrufs `UPPER` in Listing 14.2

14.1.3 Spaltenwerte von führenden und endenden Leerzeichen befreien (TRIM)

Bei Spaltenwerten, die einen Text repräsentieren, werden Sie häufig auf führende oder endende Leerzeichen treffen. Solche überflüssigen Leerzeichen stören und führen schlimmstenfalls zu Problemen bei anderen Operationen. Wenn Sie etwa mit einer `WHERE`-Klausel beabsichtigen, bestimmte Zeilen in einer Ergebnisliste herauszufiltern, verfälschen Leerzeichen das Resultat, da »Ergebnis« nun einmal nicht das Gleiche ist wie »Ergebnis«.

Unsere Tabelle *produkt* enthält keine führenden oder endenden Leerzeichen in den Zeichenketten. Um Ihnen die Funktion `TRIM` vorzustellen, stelle ich Ihnen eine weitere Tabelle bereit, in der ebenfalls die Artikel des Unternehmens hinterlegt sind. Es handelt sich um die Tabelle *produktleerzeichen*. Mit der einfachen `SELECT`-Abfrage aus Listing 14.3 machen wir uns ein Bild davon, worin sich die Spaltenwerte der Spalte *artikelbezeichnung* aus den Tabellen *produkt* und *produktleerzeichen* unterscheiden.

```
SELECT artikelbezeichnung FROM produktleerzeichen;
```

Listing 14.3 Die Spalte »artikelbezeichnung« der Tabelle »produktleerzeichen« abfragen

Tabelle 14.4 enthält das Ergebnis der Abfrage aus Listing 14.3. Sie sehen, dass die Werte der Spalte *artikelbezeichnung* jeweils führende und endende Leerzeichen oder sogar beides enthalten.

artikelbezeichnung
.....Schraube 1
Schraube 2.....
.....Schraube 3.....
Schraube 4

Tabelle 14.4 Ergebnis der Spaltenabfrage »artikelbezeichnung« aus der Tabelle »produktleerzeichen«

Die Funktion **TRIM** entfernt führende und endende Leerzeichen vor und hinter Spaltenwerten. In Listing 14.4 wenden wir die Funktion **TRIM** an, um die Spaltenwerte der Spalte *artikelbezeichnung* von führenden und endenden Leerzeichen zu bereinigen. Die Funktion **TRIM** erwartet für eine MySQL-, MariaDB-, PostgreSQL- und MS SQL Server-Datenbank (Version 2022) zwei Parameter. Zum einen müssen Sie festlegen, welches Zeichen (z. B. ein Leerzeichen) entfernt werden soll, und zum anderen müssen Sie die Spalte angeben, aus der ein führendes, endendes oder umschließendes (**LEADING**, **TRAILING**, **BOTH**) Zeichen entfernt werden soll.

Eingeleitet wird der Funktionsaufruf **TRIM** mit einem der Schlüsselwörter **LEADING**, **TRAILING** oder **BOTH**. Als Erstes geben wir also an, ob führende, endende oder umschließende Zeichen entfernt werden sollen. Es folgt, von einfachen Hochkommas umschlossen, das Zeichen (hier ein Leerzeichen), das gelöscht werden soll. Hinter dem Zeichen geben Sie mit **FROM** an, aus welcher Spalte Sie die Zeichen entfernen wollen.

Wenn Sie eine MS SQL Server-Datenbank ab Version 2017 und vor Version 2022 einsetzen, können Sie ausschließlich die **TRIM**-Variante nutzen, die umschließende Zeichen entfernt. **LEADING**, **TRAILING** oder **BOTH** sind hier nicht in der Syntax vorgesehen. In den älteren Versionen war es also nicht möglich, die Funktion auf führende oder endende Zeichen anzuwenden.

In Listing 14.4 werden in der ersten Abfrage führende und endende Leerzeichen der Werte in der Spalte *artikelbezeichnung* entfernt. Diese Abfrage können Sie für eine MySQL-, MariaDB-, PostgreSQL- und MS SQL Server-Datenbank ab Version 2022 verwenden. Die zweite Abfrage zeigt Ihnen, wie Sie ausschließlich umschließende Leerzeichen in einer MS SQL Server-Datenbank löschen, deren Version kleiner als 2022 und größer als/gleich 2017 ist.

```

/* MySQL, MariaDB, PostgreSQL, MS SQL Server seit Version 2022 */
SELECT
  TRIM(BOTH ' ' FROM artikelbezeichnung) AS artikelohneleerzeichen
FROM
  produktleerzeichen;

/*
MS SQL Server kleiner als Version 2022 und größer als/gleich Version 2017
*/

SELECT
  TRIM(' ' FROM artikelbezeichnung) AS artikelohneleerzeichen
FROM
  produktleerzeichen;

```

Listing 14.4 Verwendung der Skalarfunktion TRIM in der Spaltenauswahlliste

Tabelle 14.5 zeigt Ihnen das Ergebnis der Abfragen aus Listing 14.4: Sämtliche führenden und endenden Leerzeichen wurden aus den Spaltenwerten der Spalte *artikelbezeichnung* entfernt, und das Resultat wird unter dem Spaltenalias *artikelohneleerzeichen* in der Ergebnistabelle angezeigt.

artikelohneleerzeichen
Schraube 1
Schraube 2
Schraube 3
Schraube 4

Tabelle 14.5 Ergebnistabelle für den Funktionsaufruf TRIM



TRIM auf andere Zeichen anwenden

Die führenden, endenden bzw. umschließenden Zeichen werden in Hochkommas angegeben. Hier können Sie jedes beliebige Zeichen bzw. eine Zeichenkette notieren. Die MySQL- und die MariaDB-Datenbank entfernen eine führende, endende bzw. umschließende Zeichenkette.

Die PostgreSQL- und die MS SQL Server-Datenbank entfernt hingegen sämtliche führenden, endenden bzw. umschließenden Zeichen, die in Hochkommas angegeben werden.



TRIM standardmäßig aufrufen

Die Funktion **TRIM** kann auch ausschließlich mit einem Parameter (eine Spaltenangabe) angewendet werden. Der Standardaufruf wird meistens angewendet, wenn ausschließlich Leerzeichen entfernt werden sollen.

Mit der hier vorgestellten Syntax gehen Sie immer auf Nummer sicher und legen fest, welche führenden, endenden oder umschließenden Zeichen entfernt werden sollen.

14.1.4 Text aus Spaltenwerten extrahieren (SUBSTRING)

In der Praxis müssen Sie häufig Textwerte aus einer Spalte extrahieren. Diese Textextrakte können Sie dann in einer **WHERE**-Klausel auswerten oder in einer **ORDER BY**-Klausel zum Sortieren verwenden. Zu Beginn nutzen wir die Extraktionsfunktion **SUBSTRING** in einer Spaltenauswahlliste. In der MS SQL Server-Datenbank gibt es eine **SUBSTRING**-Version mit einer anderen Bezeichnung, die aber exakt das Gleiche leistet.

Beginnen wir damit, einen Text aus Spaltenwerten zu extrahieren. In diesem Beispiel soll der Text von Position 1 bis Position 8 der Spalte *artikelbezeichnung* aus der Tabelle *produkt* extrahiert werden. In [Listing 14.5](#) wird die **SUBSTRING**-Funktion in der Spaltenauswahlliste aufgerufen:

SELECT

```
    artikelbezeichnung,
    SUBSTRING(artikelbezeichnung FROM 1 FOR 8) AS extrakt1bis8
/*
    MS SQL Server-Version
    SUBSTRING(artikelbezeichnung,1,8) AS extrakt1bis8
*/
```

FROM

```
    produkt;
```

Listing 14.5 SELECT-Abfrage mit einer SUBSTRING-Skalarfunktion in der Spaltenauswahlliste

Betrachten wir jetzt die Informationen einschließlich des Spaltenwerts, die wir der Funktion übergeben: Hinter der öffnenden Klammer schreiben Sie zuerst die Spalte, die einen Textwert enthält, und das Schlüsselwort **FROM**. Danach folgt die Startposition, die der Funktion mitteilt, von welcher Stelle aus extrahiert werden soll. Hier übergeben wir den Wert 1 für die Startposition der Extraktion. Das Schlüsselwort **FOR** gibt die Anzahl der Zeichen ab der Startposition an. Der Wert 8 gibt hier also an, wie viele Zeichen ab der Startposition 1 extrahiert werden sollen. Kurz zusammengefasst: Wir übergeben der Funktion **SUBSTRING** den Spaltenwert, aus dem wir einen Text

extrahieren möchten, sowie die Startposition und die Anzahl der Zeichen des zu extrahierenden Textes.

Falls Sie eine Microsoft SQL Server-Datenbank verwenden, können Sie den Funktionsaufruf innerhalb der Kommentarzeichen verwenden. Die Handhabung der Funktion ist ebenfalls einfach: Sie übergeben die Spalte als ersten Parameter. Hinter einem Komma folgt die Startposition des zu extrahierenden Textes. Zuletzt übergeben Sie hinter einem Komma die Länge des zu extrahierenden Textes. Das war es schon. Um den eigentlichen Textspaltenwert mit unserem Extrakt zu vergleichen, nehmen wir die Werte der Spalte *artikelbezeichnung* mit in die Auswahlliste auf.

In [Tabelle 14.6](#) wurde mittels **SUBSTRING** der Text der Spalte *artikelbezeichnung* von Position 1 bis zum 8 Zeichen von der Position 1 aus gesehen extrahiert.

artikelbezeichnung	extrakt1bis8
Schraube 1	Schraube
Schraube 2	Schraube
Schraube 3	Schraube
Schraube 4	Schraube

Tabelle 14.6 Ergebnis der Anwendung der SUBSTRING-Funktion

Als Nächstes werden wir die extrahierte Zeichenkette von Position 1 bis 8 der Spalte *artikelbezeichnung* in einer **WHERE**-Klausel verwenden, um nur die Artikel zu filtern, deren Zeichenkette von Position 1 aus betrachtet 8 Zeichen lang ist und dem Wert Schraube entspricht.

Zwar sind in der Tabelle *produkt* noch keine weiteren Artikel außer Schrauben enthalten, aber es könnte ja gut sein, dass wir unser Produktportfolio in der Zukunft um Muttern erweitern werden.

SELECT

```
    artikelbezeichnung,  
    SUBSTRING(artikelbezeichnung FROM 1 FOR 8) AS extrakt1bis8  
FROM  
    produkt  
WHERE  
    SUBSTRING(artikelbezeichnung FROM 1 FOR 8)='Schraube';
```

Listing 14.6 SELECT-Abfrage mit einer SUBSTRING-Funktion in der WHERE-Klausel

Wenn Sie die **SELECT**-Abfrage aus [Listing 14.6](#) ausgeführt haben, erhalten Sie, wie in [Tabelle 14.7](#) dargestellt, ausschließlich die Ergebniszeilen, bei denen die Werte der Spalte *artikelbezeichnung* gleich *Schraube* sind.

artikelbezeichnung	extrakt1bis8
Schraube 1	Schraube
Schraube 2	Schraube
Schraube 3	Schraube
Schraube 4	Schraube

Tabelle 14.7 Ergebnis einer Verwendung der SUBSTRING-Funktion in einer WHERE-Klausel

14.1.5 Textspaltenwerte verkettet ausgeben

SQL bietet Ihnen die Möglichkeit, Spaltenwerte aneinanderzufügen. Sie können auch Literale an Spaltenwerte anfügen. In [Tabelle 14.8](#) sehen Sie ein Beispiel dazu.

artikelbezeichnung	kategorie	legierung	produktinfo
Schraube 1	Schrauben	Zink/Eisen	Schraube 1 Schrauben Zink/Eisen
Schraube 2	Schrauben	Kupfer/Eisen	Schraube 2 Schrauben Kupfer/Eisen
Schraube 3	Schrauben	Nickel/Eisen	Schraube 3 Schrauben Nickel/Eisen
Schraube 4	Schrauben	Gold/Eisen	Schraube 4 Schrauben Gold/Eisen

Tabelle 14.8 Spaltenwerte miteinander verketteten

In [Tabelle 14.8](#) werden die Spaltenwerte der Spalten *artikelbezeichnung*, *kategorie* und *legierung* in einer Zeile zusammengefügt, und das Ergebnis des Zusammenfügens wird in einer weiteren Spalte namens *produktinfo* ausgegeben. Die Spaltenwerte dieser Spalten wurden also zu einem Spaltenwert verbunden. Sie sehen auch, dass zwischen den einzelnen Spaltenwerten Leerzeichen eingefügt wurden. Bevor wir uns ansehen, wie Sie Spaltenwerte und Literale verketteten, betrachten wir, wie die hier behandelten Datenbanken die Verkettung bzw. Verbindung von Spaltenwerten und Literalen umsetzen.



Unterschiede zwischen den Datenbanken

Der SQL-Standard sieht für das Verbinden bzw. Verketteten von Spaltenwerten und Literalen den Verkettungsoperator `||` (doppeltes Pipe-Zeichen) vor. Gemäß dem Standard wird also ein Operator zum Aneinanderfügen von Spaltenwerten oder Literalen verwendet und keine Skalarfunktion. Die Datenbank hält sich an diesen Standard und nutzt den `||`-Operator.

MySQL und MariaDB bieten stattdessen für das Verbinden von Spaltenwerten die Funktion **CONCAT** an, der Sie beliebig viele Spaltennamen oder Literale als Parameter übergeben können, um diese verkettet auszugeben.

Die MS SQL Server-Datenbank sieht das Ganze noch einmal anders und liefert Ihnen für die Verkettung von Spaltenwerten und Literalen den `+`-Operator. Die Verkettung von Spaltenwerten zeige ich Ihnen jeweils in den Abschnitten »MySQL und MariaDB«, »PostgreSQL« und »MS SQL Server« für die hier behandelten Datenbanken.

MySQL und MariaDB

In [Listing 14.7](#) zeige ich Ihnen, wie Sie das Ergebnis aus [Tabelle 14.8](#) mit einer MySQL- und MariaDB-Datenbank nachbilden:

SELECT

```
artikelbezeichnung,
kategorie,
legierung,
CONCAT(artikelbezeichnung, ' ', kategorie, ' ', legierung)
AS produktinfo
```

FROM

```
produkt;
```

Listing 14.7 Spaltenwerte und Literale in einer MySQL- und MariaDB-Datenbank verketteten

Zum Verketteten von Spaltenwerten und Literalen verwenden Sie die **CONCAT**-Funktion und übergeben die Spaltenwerte und Literale in der Reihenfolge, in der sie miteinander verkettet werden sollen. In diesem Beispiel werden fünf Parameter übergeben: Die drei Spaltennamen *artikelbezeichnung*, *kategorie* und *legierung* werden durch zwei Zeichenkettenliterals getrennt, die jeweils ein Leerzeichen enthalten.

Der Funktion ordnen wir den Spaltenalias *produktinfo* zu, um über einen sprechenden Spaltenbezeichner zu verfügen.

PostgreSQL

In [Listing 14.8](#) zeige ich Ihnen, wie Sie das Ergebnis aus [Tabelle 14.8](#) mit einer PostgreSQL-Datenbank erzielen:

SELECT

```

    artikelbezeichnung,
    kategorie,
    legierung,
    artikelbezeichnung || ' ' || kategorie || ' ' || legierung
    AS produktinfo

```

FROM

```

    produkt;

```

Listing 14.8 Spaltenwerte und Literale in einer PostgreSQL-Datenbank verketten

Zum Verketten von Spaltenwerten und Literalen verwenden Sie hier gemäß dem SQL-Standard den doppelten *Pipe-Operator* (||) und verketten die Spaltenwerte und Literale einfach, indem Sie den Operator zwischen den zu verkettenden Spaltenwerten bzw. Literalen notieren. In diesem Beispiel notieren Sie also die Spaltennamen *artikelbezeichnung*, *kategorie* und *legierung* sowie die Leerzeichen getrennt durch die doppelten Pipe-Zeichen.

Dem daraus resultierenden Ausdruck ordnen wir den Spaltenalias *produktinfo* zu, um über einen sprechenden Spaltenbezeichner zu verfügen.

MS SQL Server

In [Listing 14.9](#) sehen Sie, wie Sie das Ergebnis aus [Tabelle 14.8](#) mit einer MS SQL Server-Datenbank erzielen:

SELECT

```

    artikelbezeichnung,
    kategorie,
    legierung,
    artikelbezeichnung + ' ' + kategorie + ' ' + legierung
    AS produktinfo

```

FROM

```

    produkt;

```

Listing 14.9 Spaltenwerte und Literale in einer MS SQL Server-Datenbank verketten

Zum Verketten von Spaltenwerten und Literalen notieren Sie den Plus-Operator (+) zwischen den zu verkettenden Spaltenwerten bzw. Literalen. In diesem Beispiel geben Sie also die Spaltennamen *artikelbezeichnung*, *kategorie* und *legierung* an sowie die Leerzeichen, jeweils getrennt durch einen +-Operator.

Dem daraus resultierenden Ausdruck ordnen wir den Spaltenalias *produktinfo* zu, um über einen sprechenden Spaltenbezeichner zu verfügen.

14.1.6 Übungen

Übung 1

In dieser Übung geben Sie die Spaltenwerte der Spalte *legierung* aus der Tabelle *produkt* in der Spaltenauswahlliste jeweils in Kleinschreibung und Großschreibung aus. Ordnen Sie den Funktionen die Spaltenaliasse *legierungklein* und *legierunggross* zu.

Übung 2

Formulieren Sie eine **SELECT**-Abfrage, in der die Spaltenwerte der Spalte *legierung* der Tabelle *produktleerzeichen* mit und ohne führende bzw. endende Leerzeichen abgefragt werden.

Übung 3

In dieser Übung soll ein Extrakt aus den Werten der Spalte *artikelbezeichnung* der Tabelle *produkt* mit einer **SELECT**-Anweisung abgefragt werden. Das Extrakt soll das zehnte Zeichen aus den Spaltenwerten der Spalte *artikelbezeichnung* enthalten, um die numerischen Werte der jeweiligen Produkte zu ermitteln.

Falls Sie eine MS SQL Server-Datenbank nutzen, sollten Sie die vorgestellte Variante zur Lösung dieser Aufgabe verwenden.

Übung 4

Nutzen Sie die in Übung 3 erstellte **SELECT**-Abfrage, und erweitern Sie diese um eine **WHERE**-Klausel, in der ausschließlich die Produkte gefiltert werden, deren Extrakt des ganzzahligen numerischen Wertes der *artikelbezeichnung* gleich 1 oder 2 ist. Beachten Sie an dieser Stelle, dass Sie zwar numerische Werte extrahieren, diese aber als Zeichenkette von der Funktion **SUBSTRING** zurückgegeben werden.

Übung 5

In dieser Übung sollen die Werte der Spalten *artikelbezeichnung* und *legierung* der Tabelle *produktleerzeichen* miteinander verkettet werden. Als Trennzeichen soll jeweils ein _ (Unterstrich) verwendet werden. Um die führenden Leerzeichen der Werte der Spalte *artikelbezeichnung* sichtbar zu machen, verketten Sie vor der Spalte *artikelbezeichnung* ebenfalls einen Unterstrich mit der Spalte *artikelbezeichnung*. Das Gleiche gilt für die endenden Leerzeichen der Werte der Spalte *legierung*. Verketteten Sie hierzu hinter dem Spaltennamen *legierung* einen Unterstrich mit der Spalte *legierung*.

Lösung zu Übung 1

SELECT

UPPER(*legierung*) **AS** *legierunggross*,

```

    LOWER(legierung) AS legierungklein
FROM
    produkt;

```

Listing 14.10 Die Funktionen UPPER und LOWER in einer Spaltenauswahlliste verwenden

Lösung zu Übung 2

```

/* MySQL-, MariaDB, PostgreSQL- und MS SQL Server-Variante 2017/2022 */
SELECT
    legierung,
    /*
        MS SQL Server ab 2017
        TRIM(' ' FROM legierung) AS ohneleerzeichen

    */
    /* MySQL, PostgreSQL, MS SQL Server ab Version 2022 */
    TRIM(BOTH ' ' FROM legierung) AS ohneleerzeichen

FROM
    produktleerzeichen;

```

Listing 14.11 Die Funktion TRIM nutzen, um führende und endende Leerzeichen aus Spaltenwerten zu entfernen

Lösung zu Übung 3

```

/* Lösung für eine MySQL-, MariaDB- und PostgreSQL-Datenbank */
SELECT
    SUBSTRING(artikelbezeichnung FROM 10 FOR 10) AS extrakt10bis10
FROM
    produkt;

/* Lösung für eine MS SQL Server-Datenbank */
SELECT
    SUBSTRING(artikelbezeichnung,10,1) AS extrakt10bis10
FROM
    produkt;

```

Listing 14.12 Die Funktion SUBSTRING nutzen, um Zeichenketten aus Spaltenwerten zu extrahieren

Lösung zu Übung 4

```
/* Lösung für eine MySQL-, MariaDB- und PostgreSQL-Datenbank */
SELECT
    SUBSTRING(artikelbezeichnung FROM 10 FOR 10) AS extrakt10bis10
FROM
    produkt
WHERE
    SUBSTRING(artikelbezeichnung FROM 10 FOR 10) IN ('1','2');

/* Lösung für eine MS SQL Server-Datenbank */
SELECT
    SUBSTRING(artikelbezeichnung,10,1) AS extrakt10bis10
FROM
    produkt
WHERE
    SUBSTRING(artikelbezeichnung,10,1) IN ('1','2');
```

Listing 14.13 Die Funktion SUBSTRING in einer Bedingung einer WHERE-Klausel nutzen

Lösung zu Übung 5

```
/* MySQL- und MariaDB-Variante */
SELECT CONCAT('-',artikelbezeichnung,'-',legierung,'-')
FROM produktleerzeichen;

/* PostgreSQL-Variante */
SELECT '-'||artikelbezeichnung||'-'||legierung||'-'
FROM produktleerzeichen;

/* MS SQL Server-Variante */
SELECT '-'+artikelbezeichnung+'-'+legierung+'-'
FROM produktleerzeichen;
```

Listing 14.14 Zeichenketten aneinanderfügen

14.2 Funktionen für Zahlenwerte

Gemäß dem SQL-Standard gibt es einige Skalarfunktionen für numerische Werte. In Tabelle 14.9 stelle ich Ihnen eine Auswahl von Skalarfunktionen vor, die numerische Werte als Ergebnis zurückliefern.

Skalarfunktion	Funktionsbeschreibung
<code>CHAR_LENGTH (spaltenname)</code>	<code>CHAR_LENGTH</code> gibt Ihnen die Länge einer Zeichenkette zurück.
<code>POSITION (zkette1 IN zkette2)</code>	Liefert Ihnen die numerische Position einer Zeichenkette. Im Beispiel wird die Position der Zeichenkette <code>zkette1</code> in einer anderen Zeichenkette, <code>zkette2</code> , ermittelt.
<code>POWER (Basis, Exponent)</code>	Die Skalarfunktion <code>POWER</code> liefert Ihnen zu einer Basis und einem Exponenten als Ergebnis die Potenz zurück. Mit einer Basis 10 und einem Exponenten 2 ergäbe diese Berechnung 100.
<code>SQRT (spaltenname)</code>	Die Skalarfunktion <code>SQRT</code> liefert Ihnen zu einem numerischen Wert die Quadratwurzel als Ergebnis zurück. Übergeben Sie den Wert 9, so erhalten Sie als Ergebnis den Wert 3.

Tabelle 14.9 Die am häufigsten verwendeten Spezialisten für numerische Werte

14.2.1 Die Länge einer Zeichenkette ermitteln (`CHAR_LENGTH`, `LEN`)

Mit der Funktion `CHAR_LENGTH(spaltenname)` ermitteln Sie die Länge eines Spaltenwerts. Die Funktion `CHAR_LENGTH` wird von der PostgreSQL-, der MySQL- und der MariaDB-Datenbank gemäß dem Standard unterstützt. Microsoft SQL Server kennt die Funktion nicht. Anstelle dieser Funktion wird die Funktion `LEN(spaltenname)` angeboten, die ebenfalls die Länge von Textspaltenwerten ermittelt. Sehen wir uns die Verwendung der Funktionen anhand eines Beispiels an.

Hierzu soll die Länge der Spaltenwerte für die Spalte *legierung* der Tabelle *produkt* ermittelt werden. Um das Ergebnis der Funktion zu verifizieren, nehmen wir die Spalte *legierung* mit in die Spaltenauswahlliste auf.

```
SELECT
    legierung,
    CHAR_LENGTH(legierung) AS laengezeichenkette
/* LEN(legierung) AS laengezeichenkette MS SQL Server */
FROM
    produkt;
```

Listing 14.15 Anwendung der Skalarfunktion »CHAR_LENGTH(spaltenname)« in einer Spaltenauswahlliste



Unterschiede zwischen den Datenbanken

Falls Sie mit einer Microsoft SQL Server-Datenbank arbeiten, verwenden Sie die Funktion `LEN` anstelle der Funktion `CHAR_LENGTH`, wie innerhalb der Kommentarzeichen angegeben.

Die Datenbank liefert Ihnen, wie in [Tabelle 14.10](#) dargestellt, eine Ergebnisliste zurück, die jeweils die Länge der Werte der Spalte *legierung* unter dem Spaltenalias *laengezeichenkette* enthält.

Legierung	laengezeichenkette
Zink/Eisen	10
Kupfer/Eisen	12
Nickel/Eisen	12
Gold/Eisen	10

Tabelle 14.10 Ergebnis der Funktion `CHAR_LENGTH`

14.2.2 Die Startposition einer Zeichenkette in einem Textwert ermitteln (`POSITION`, `CHARINDEX`)

Die SQL-Funktion `POSITION`(text **IN** spaltenname) liefert Ihnen für einen von Ihnen vorgegebenen Text (text) die Startposition in einem weiteren Text (spaltenname), der als Spaltenwert enthalten ist, als ganze Zahl zurück.

Die MySQL-, MariaDB- und die PostgreSQL-Datenbank bieten die Funktion `POSITION` an, wie es der SQL-Standard vorsieht. Die MS SQL Server-Datenbank hingegen stellt Ihnen anstelle der Funktion `POSITION` die Funktion `CHARINDEX`(text, spaltenname) zur Verfügung, die ebenfalls die Position einer Zeichenkette (text) in Spaltenwerten ermittelt.

An welcher Stelle kommt das Wort Eisen in den Spaltenwerten der Spalte *legierung* vor? Um diese Frage zu beantworten, nutzen wir die Skalarfunktion `POSITION` und ermitteln die Startposition der Zeichenkette Eisen.

Falls Sie mit MS SQL Server arbeiten, müssen Sie die Funktion `CHARINDEX` so nutzen, wie sie im Listingkommentar dargestellt ist.

SELECT

```

    legierung,
    POSITION('Eisen' IN legierung) AS startpos
/* MS SQL Server
    CHARINDEX('Eisen', legierung) AS startpos */

```

FROM

```
produkt;
```

Listing 14.16 SELECT-Abfrage, die mit der Skalarfunktion POSITION die Startposition eines Textwertes ermittelt

Als Ergebnis erhalten Sie von der Datenbank [Tabelle 14.11](#). In dieser Liste finden Sie sämtliche Legierungen, die das Wort `Eisen` enthalten, sowie seine Startposition, bezogen auf die Spaltenwerte der Spalte `legierung`.

Legierung	startpos
Zink/Eisen	6
Kupfer/Eisen	8
Nickel/Eisen	8
Gold/Eisen	6

Tabelle 14.11 Ergebnis der Skalarfunktion POSITION

Sie verfügen jetzt über die Information, an welcher Position das Wort `Eisen` in den Werten der Spalte `artikelbezeichnung` beginnt. Durch einfaches Zählen finden Sie heraus, dass es fünf Zeichen lang ist. Sie ahnen vielleicht bereits, dass wir zu einem späteren Zeitpunkt die **SUBSTRING**-Funktion verwenden werden, um das Wort `Eisen` aus einer Zeichenkette herauszuschneiden. An dieser Stelle brauchen Sie noch etwas Geduld; in [Abschnitt 14.3](#) kommen wir darauf im Zusammenhang mit verschachtelten Funktionsaufrufen zurück.

14.2.3 Potenzen berechnen (POWER)

Sie erinnern sich: *Potenzen* sind nichts anderes als eine andere Schreibweise für eine Multiplikation einer Zahl mit sich selbst. Eine Zahl können Sie theoretisch beliebig oft in der Form $10 \times 10 \times 10 \times 10 \dots$ mit sich selbst multiplizieren. Stellen Sie sich vor, Sie würden die Zahl 10 vierunddreißigmal mit sich selbst multiplizieren. Das ist ganz schön viel Schreibarbeit und fehlerträchtig. In der Mathematik hat man dafür eine schöne Lösung gefunden. Die Zahl, die mit sich selbst multipliziert wird, wird *Basis* genannt. Die Anzahl, wie oft die Multiplikation stattfinden soll, heißt *Exponent*. Wenn wir die Zahl 10 vierunddreißigmal mit sich selbst multiplizieren wollen, schreiben wir also: 10^{34} .

Das Wort »Potenz« stammt aus dem Lateinischen und bedeutet so viel wie »Macht«. Die Funktion folgt der englischen Übersetzung und heißt **POWER**. Der Aufruf der Funk-

tion ist relativ einfach: Sie übergeben ihr lediglich die Basis und den Exponenten in der Form `POWER(Basis,Exponent)`. Daraufhin berechnet die Funktion die Potenz und gibt den numerischen Wert zurück.

Sehen wir uns [Listing 14.17](#) als Beispiel an:

```
SELECT
    artikelbezeichnung,
    POWER(100,3) AS potenz
FROM
    produkt;
```

Listing 14.17 Berechnung einer Potenz in der Spaltenauswahlliste mit der Funktion `POWER`

Wir wenden den Exponenten 3 auf die Basis 100 an. Um die Berechnung auszuführen, rufen wir in der Spaltenauswahlliste einfach die Skalarfunktion `POWER` auf und übergeben ihr den Wert 100 als Basis und den Wert 3 als Exponenten.

[Tabelle 14.12](#) zeigt Ihnen das Ergebnis der Berechnung. Zu der Basis 100 und dem Exponenten 3 hat die Funktion `POWER` das Ergebnis 1000000 zurückgeliefert.

artikelbezeichnung	potenz
Schraube 1	1000000
Schraube 2	1000000
Schraube 3	1000000
Schraube 4	1000000

Tabelle 14.12 Ergebnis der Funktion `POWER`

14.2.4 Eine Quadratwurzel berechnen (SQRT)

Befassen wir uns jetzt mit der SQL-Funktion, die eine einfache Quadratwurzel berechnet: Die Funktion `SQRT(spaltenname)` erhält einen Zahlenwert als Parameter, kalkuliert die Quadratwurzel und liefert Ihnen das Ergebnis zurück. Der Methodenname `SQRT` steht für *square root* (dt. Quadratwurzel). In [Listing 14.18](#) berechnen wir mit der Funktion `SQRT` die Wurzel aus 9. Dem Ergebnis der Berechnung ordnen wir den Spaltenalias *wurzel* zu.

```
SELECT
    artikelbezeichnung,
    SQRT(9) AS wurzel
```

FROM

```
produkt;
```

Listing 14.18 SELECT-Abfrage mit einem Funktionsaufruf SQRT, der die Quadratwurzel einer Zahl berechnet

artikelbezeichnung	wurzel
Schraube 1	3
Schraube 2	3
Schraube 3	3
Schraube 4	3

Tabelle 14.13 Ergebnis der Wurzelberechnung, die dem Spaltenalias »wurzel« zugeordnet ist.

14.2.5 Übungen

Übung 1

Formulieren Sie eine **SELECT**-Abfrage, in der Sie die Spaltenwerte der Spalte *artikelbezeichnung* der Tabelle *produktleerzeichen* abfragen. Ermitteln Sie zusätzlich in der Spaltenauswahlliste die Länge der Spaltenwerte der Spalte *artikelbezeichnung*.

Welche Beobachtung machen Sie, wenn Sie die Ergebnistabelle der zu formulierenden Abfrage zurückerhalten?

Übung 2

Ermitteln Sie mit einer **SELECT**-Abfrage die Position des Zeichens *n* in den Werten der Spalte *kategorie* der Tabelle *produkt*.

Übung 3

Fragen Sie mit einer **SELECT**-Abfrage die Spalte *artikelbezeichnung* der Tabelle *produkt* ab. Außerdem soll in der Spaltenauswahlliste die Potenz 5^3 berechnet werden.

Übung 4

In dieser Übung soll wieder die Spalte *artikelbezeichnung* der Tabelle *produkt* abgefragt werden. Berechnen Sie diesmal anstelle einer Potenz die Wurzel aus dem Wert 64 in der Spaltenauswahlliste.

Lösung zu Übung 1

```
/* MySQL-, MariaDB- und PostgreSQL-Variante */  
SELECT artikelbezeichnung, CHAR_LENGTH(artikelbezeichnung)  
FROM produktleerzeichen;
```

```
/* MS SQL Server-Variante */  
SELECT artikelbezeichnung, LEN(artikelbezeichnung)  
FROM produktleerzeichen;
```

Listing 14.19 Die Länge von Spaltenwerten ermitteln, die Textwerte enthalten

In dieser Übung können Sie beobachten, dass die Anzahl der Zeichen in der Zeichenkette länger ist als der eigentliche Spaltenwert. In den Spaltenwerten der Spalte *artikelbezeichnung* finden sich führende und endende Leerzeichen.

Lösung zu Übung 2

```
/* MySQL-, MariaDB- und PostgreSQL-Variante*/  
SELECT POSITION('n' IN kategorie) AS pos_zeichen FROM produkt;
```

```
/* MS SQL Server */  
SELECT CHARINDEX('n', kategorie) AS pos_zeichen FROM produkt;
```

Listing 14.20 Die Position eines Zeichens in Spaltenwerten ermitteln

Lösung zu Übung 3

```
SELECT artikelbezeichnung, POWER(5,3) FROM produkt;
```

Listing 14.21 Eine Potenz berechnen

Lösung zu Übung 4

```
SELECT artikelbezeichnung, SQRT(64) FROM produkt;
```

Listing 14.22 Eine Wurzel berechnen

14.3 Verschachtelte Funktionsaufrufe

Jetzt kommen wir, wie zuvor versprochen, zu den verschachtelten Funktionsaufrufen in einer Spaltenauswahlliste. Ein verschachtelter Funktionsaufruf ist nicht kompliziert, obwohl er auf den ersten Blick etwas Verwirrung stiften könnte.

Sehen wir uns das Prinzip eines verschachtelten Funktionsaufrufs in [Listing 14.23](#) an:

```
FUNKTION1(FUNKTION2(FUNKTION3(parameter1)))
```

Listing 14.23 Syntax für einen verschachtelten Funktionsaufruf

Richten Sie Ihren Blick auf die Funktion **FUNKTION3**. Sie übergeben der Funktion einen Wert `parameter1`, der ein Spaltenwert oder ein Literal sein könnte. Wandern Sie nun mit Ihren Augen ein kleines bisschen nach links. Sie treffen hier auf eine offene Klammer, und vor der Klammer sehen Sie **FUNKTION2**. Diese Funktion **FUNKTION2** erhält als Parameter das Ergebnis von **FUNKTION3** und wendet die in **FUNKTION2** enthaltene Logik oder Arithmetik auf diesen Wert an. Wenn Sie Ihren Blick dann noch ein bisschen weiter nach links wandern lassen, begegnet Ihnen wieder eine offene Klammer. Vor der Klammer sehen Sie **FUNKTION1**, die wiederum das Ergebnis von **FUNKTION2** als Parameter übergeben bekommt. Grundsätzlich sind verschachtelte Funktionen immer von innen nach außen zu betrachten und auch so bei der Formulierung von SQL-Anweisungen zu berücksichtigen.

Kennen Sie diese kleinen russischen Matroschka-Puppen? Sie öffnen eine, und darin enthalten ist eine weitere. Sie öffnen diese weitere Figur, und darin findet sich wieder eine. Übertragen auf unsere Funktionen, befindet sich im Innersten dieser verschachtelten »Puppe« ein Spezialist. Der verrichtet seine Arbeit und übergibt das Ergebnis der übergeordneten Puppe, die wiederum auf eine andere Tätigkeit spezialisiert ist und das Ergebnis der inneren Puppe als Grundlage für die weitere Funktionalität verwenden kann.

Die in [Listing 14.24](#) dargestellte Verschachtelung wendet dieses Prinzip an, indem **FUNKTION1** als Parameter die Ergebnisse von **FUNKTION2** und **FUNKTION3** übergeben werden. Es werden also zuerst **FUNKTION2** und **FUNKTION3** ausgeführt und die Ergebnisse als Parameter an **FUNKTION1** übergeben.

```
FUNKTION1(FUNKTION2(parameter1), FUNKTION3(parameter2))
```

Listing 14.24 Verschachtelung von Funktionen und deren Ergebniswerte als Grundlage für einen Funktionsaufruf

Da ich mit Ihnen das Wort `Eisen` aus den Spaltenwerten der Spalte *legierung* extrahieren möchte, zeige ich Ihnen jetzt einen etwas konkreteren Fall, in dem wir die Platzhalter-Schreibweise **FUNKTION1** bis **FUNKTION3** verlassen und die konkrete Funktion **SUBSTRING** betrachten.

Wir wollen den Wert `Eisen` aus der Spalte *legierung* der Tabelle *produkt* extrahieren. In der Spaltenauswahlliste nehmen wir hierzu die Spalte *artikelbezeichnung* und einen etwas komplexeren Funktionsaufruf mit auf, den Sie in [Listing 14.25](#) (MySQL, MariaDB und PostgreSQL) und [Listing 14.26](#) (MS SQL Server) sehen:

SELECT

```
    artikelbezeichnung,  
    legierung,  
    SUBSTRING(legierung  
    FROM  
        POSITION('Eisen' IN legierung)  
    FOR  
        CHAR_LENGTH('Eisen')  
    ) AS eisen  
FROM  
    produkt;
```

Listing 14.25 SELECT-Abfrage mit einem verschachtelten Funktionsaufruf in der Parameterliste einer Skalarfunktion

Damit sich der lange Funktionsaufruf im Buch besser lesen lässt, habe ich ihn aufgeteilt. Er beginnt in der vierten Zeile mit der geöffneten Klammer hinter **SUBSTRING** und endet in Zeile 9 mit einer geschlossenen Klammer. Solche verschachtelten Funktionsaufrufe werden immer von innen nach außen ausgewertet. Das gilt auch für die Funktion **SUBSTRING**(spaltenname **FROM** startpos **FOR** endpos), die die Parameter spaltenname, startpos und endpos benötigt, um eine Zeichenkette aus Spaltenwerten zu extrahieren. Für den Parameter startpos übergeben wir hier keine Zahl, die eine Startposition festlegt. Stattdessen rufen wir die Funktion **POS**(zeichenkette **IN** spaltenname) auf, um die Startposition zu ermitteln. In diesem Beispiel ermitteln wir die Startposition des Wortes *Eisen* in den Spaltenwerten der Spalte *legierung*.

Hinter dem Schlüsselwort **FOR** ist die Angabe erforderlich, bis zu welcher Position extrahiert werden soll. Auch hier nutzen wir einen Funktionsaufruf, um an die Information zu kommen. Konkret rufen wir die Funktion **CHAR_LENGTH**(zeichenkettenwert) mit einem Parameter auf, der in diesem Beispiel die zu extrahierende Zeichenkette *Eisen* enthält.

Die Funktionen **POS** und **CHAR_LENGTH** und ihre Ergebnisse werden also zuerst ausgewertet. Die Ergebnisse der Funktionen können somit der Funktion **SUBSTRING** als Parameter übergeben werden, um die Zeichenkette *Eisen* aus den Spaltenwerten der Spalte *legierung* zu extrahieren.

Falls Sie mit MS SQL Server arbeiten, sollten Sie sich [Listing 14.26](#) genauer ansehen:

SELECT

```
    artikelbezeichnung,  
    legierung,  
    SUBSTRING(  
        legierung,
```



```

PATINDEX('%Eisen%', legierung),
LEN('Eisen')
) AS eisen
FROM
produkt;

```

Listing 14.26 SELECT-Abfrage mit einem verschachtelten Funktionsaufruf in der Parameterliste einer Skalarfunktion für eine MS SQL Server-Datenbank

Das Prinzip des Aufrufs bleibt gleich: Zuerst müssen von innen heraus die zu übergebenden Parameter ermittelt werden, um die Funktion aufzurufen. Die Funktion unterscheidet sich jedoch dadurch, dass die Parameter durch eine kommaseparierte Liste der Funktion übergeben werden.

Die Ergebnisliste in [Tabelle 14.14](#) enthält jetzt eine Spalte namens *eisen*, die ausschließlich den extrahierten Textwert *Eisen* aus den Werten der Spalte *legierung* enthält.

artikelbezeichnung	legierung	eisen
Schraube 1	Zink/Eisen	Eisen
Schraube 2	Kupfer/Eisen	Eisen
Schraube 3	Nickel/Eisen	Eisen
Schraube 4	Gold/Eisen	Eisen

Tabelle 14.14 Ergebnis für einen verschachtelten Funktionsaufruf

Zusammenfassung: Skalarfunktionen

Funktionen, die einen Wert als Ergebnis einer Verarbeitung zurückliefern, werden als *Skalarfunktionen* bezeichnet. Bei der Verarbeitung kann es sich um Berechnungen, Textoperationen oder Datums- und Zeitoperationen handeln.

Skalarfunktionen werden auf Zeilenebene angewendet. Das heißt, Sie können alle Spalten, die in einer Zeile vorkommen, auch als potenzielle Kandidaten für Parameterangaben von Skalarfunktionen betrachten.

Als Parameter werden je nach Skalarfunktion und ihrer Verwendung auch Literale (Zahlen, Text, Datums- und Zeitangaben) verwendet.

Es gibt auch zahlreiche Skalarfunktionen, die keinen Parameter übergeben bekommen. Hierzu zählt z. B. die Skalarfunktion `SESSION_USER`. Sie liefert lediglich den Nutzer zurück, der gerade in einer gültigen Sitzung aktiv ist.



Es gibt zahlreiche Funktionen, und viele unterscheiden sich in ihrer Schreibweise und Anwendung in den unterschiedlichen Datenbanksystemen. Nutzen Sie die Dokumentation und weiterführende Literatur, wenn Sie sich genauer über den Funktionsschatz des jeweils genutzten Datenbanksystems informieren wollen.

14.3.1 Übungen

Übung 1

Erklären Sie kurz, in welche Richtung verschachtelte Skalarfunktionen aufgerufen werden.

Übung 2

In dieser Übung sollen Sie die Spaltenwerte der Spalte *artikelbezeichnung* in Großbuchstaben ausgeben. Stellen Sie sicher, dass die Spaltenwerte der Spalte ohne führende und endende Leerzeichen in Großbuchstaben umgewandelt werden.

Lösung zu Übung 1

Verschachtelte Skalarfunktionen werden stets von innen nach außen aufgerufen. Die äußeren Funktionen zur Weiterverarbeitung sind auf die Ergebnisse der inneren Funktionen angewiesen, auf deren Werte ihre Funktionalität angewendet werden soll.

Lösung zu Übung 2

```
/* MySQL-, MariaDB, PostgreSQL und MS SQL Server-Variante 2022*/
```

```
SELECT
```

```
    UPPER(TRIM(BOTH ' ' FROM artikelbezeichnung)) AS artbezgross
```

```
FROM
```

```
    produktleerzeichen;
```

```
/* MS SQL Server-Variante ab 2017*/
```

```
SELECT
```

```
    UPPER(TRIM(' ' FROM artikelbezeichnung)) AS artbezgross,
```

```
FROM
```

```
    produktleerzeichen;
```

Listing 14.27 Funktionen verschachtelt aufrufen

Kapitel 15

Bedingungslogik

Entscheidungen müssen Sie ständig treffen: Wenn eine Bedingung erfüllt ist, soll Fall A eintreten; wenn nicht, dann Fall B. Schauen wir uns hier an, wie Sie solche Entscheidungen in SQL nutzen können.

Alle Programmiersprachen verfügen über die Möglichkeit, logische Zweige darzustellen. Dabei erfolgt zunächst die Auswertung einer Bedingung, auf die dann reagiert wird.

15.1 Die CASE-Klausel

In SQL gibt es für solche Operationen die **CASE**-Klausel, mit der Sie Bedingungen auswerten können und Ausdrücke abhängig von der Auswertung der Bedingung zurückbekommen.

In [Kapitel 2](#) haben Sie im Zusammenhang mit der **WHERE**-Klausel bereits zahlreiche Bedingungen kennengelernt, die erfüllt sein müssen, damit es eine Zeile in die Ergebnistabelle schafft.

Eine Bedingung kann wahr oder falsch sein. Sie können festlegen, welche weiteren Schritte je nach dem Ergebnis der Auswertung einer Bedingung jeweils vorgenommen werden sollen.

Nehmen wir an, zwei Zahlen sollen dividiert werden. Eigentlich kein Problem, wenn es nicht den Fall gäbe, dass eine der beiden null (eine echte 0) sein könnte. Dann wäre die Division natürlich unmöglich. Wir müssen vorher also prüfen, ob die Zahl gleich null ist, und für diesen Fall eine besondere Vorkehrung treffen. So etwas in einer Programmierlogik darzustellen, erscheint viel schwieriger, als es wirklich ist. Sehen Sie sich als Erklärung [Abbildung 15.1](#) an.

Im oberen Teil der Abbildung sehen Sie ein Dreieck mit der Spitze nach unten. Hier ist die auszuwertende Bedingung notiert. In diesem Fall prüfen wir die Variable `Zahl` auf Gleichheit mit dem Wert 0. Wenn die Aussage der Bedingung wahr ist, verzweigen Sie auf die linke Seite der Spitze des Dreiecks. Sie müssen nun aufpassen, denn hier droht eine Division durch null. Daher setzen Sie die `Zahl` auf den Wert 1, um einen Wert, der einer Variablen `x` zugewiesen ist, durch den Wert der `Zahl` zu dividieren.

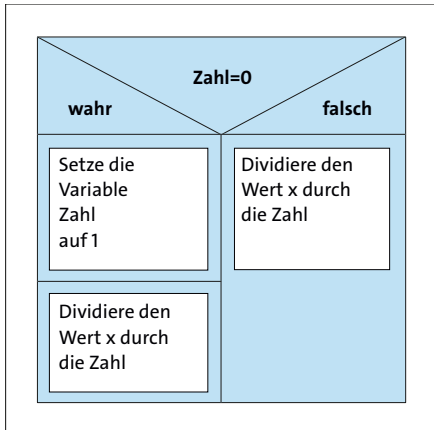


Abbildung 15.1 Auswertung einer Bedingung und Verzweigung in Anweisungsblöcke

Ist die Aussage der Bedingung `Zahl=0` falsch, so verzweigen Sie auf die rechte Seite des Dreiecks. Da der Wert ungleich null ist, ist es nicht erforderlich, eine Wertzuweisung vorzunehmen, und die Variable `x` kann ohne Bedenken durch die `Zahl` dividiert werden.

Wenn Sie eine Bedingung auswerten, gibt es also zwei Zustände – nämlich wahr und falsch –, die ausgewertet werden können. In SQL steht uns für die Auswertung von Bedingungen die **CASE**-Klausel zur Verfügung. In der **CASE**-Klausel verwenden wir die Wörter **WHEN** für die Bedingung, **THEN** für den ersten Ausdruck und **ELSE** für den zweiten Ausdruck, um ein Ergebnis einer Bedingung auszuwerten und entsprechend zu verzweigen. Vielleicht interessiert Sie ja auch der Diagrammtyp, in dem die Darstellung der Kontrollstruktur beschrieben wurde. Es handelt sich um das Nassi-Shneiderman-Diagramm. Es wird *Struktogramm* genannt und ist in der DIN 66261 beschrieben.

15.2 Bedingungslogik in einer Spaltenauswahlliste einer SELECT-Anweisung anwenden

Die **CASE**-Klausel werden wir anhand eines Beispiels nachvollziehen. In diesem Beispiel werden wir die Tabelle *mitarbeiter* abfragen, um zu ermitteln, welche Mitarbeiter noch aktiv im Unternehmen tätig sind und wer uns verlassen hat.

Hierzu sollen die Spaltenwerte der Spalte *austrittsdatum* ausgewertet werden. Bei einem vorhandenen Austrittsdatum wird anstelle des Datums der Text *ausgeschieden* zurückgeliefert, bei einem fehlenden Austrittsdatum der Text *aktiver Mitarbeiter*. Wir haben es hier also mit zwei bedingten Anweisungen zu tun.

In der Spalte *austrittsdatum* der Tabelle *mitarbeiter* gibt es zwei mögliche Fälle: Entweder ist der Wert nicht definiert (NULL), oder es ist ein Austrittsdatum eingetragen. Hier können wir also prüfen, ob der Wert der Spalte *austrittsdatum* NULL ist. Wenn dies der Fall ist, handelt es sich um einen aktiven Mitarbeiter. Wenn hier hingegen ein Datum eingetragen ist, handelt es sich um einen Mitarbeiter, der nicht mehr für das Unternehmen tätig ist.

In [Listing 15.1](#) zeige ich Ihnen, wie Sie diese Anforderung realisieren können:

```
SELECT
  name,
  vorname,
  austrittsdatum,
  CASE
    WHEN
      austrittsdatum IS NULL
    THEN
      'aktiver Mitarbeiter'
    ELSE
      'ausgeschieden'
  END AS mitarbeiterstatus
FROM
  mitarbeiter;
```

Listing 15.1 Anwendung eines CASE-Ausdrucks in einer Spaltenauswahlliste

Hinter der Spalte *vorname* in der Spaltenauswahlliste finden Sie den Beginn des CASE-Ausdrucks. Innerhalb des CASE-Ausdrucks ist eine Bedingung formuliert, die im WHEN-Zweig prüft, ob das Austrittsdatum NULL ist. Wenn dies der Fall ist, wird im THEN-Zweig der Wert *aktiver Mitarbeiter* als Spaltenwert zurückgegeben. Sollte diese Bedingung nicht zutreffen, da der Wert nicht NULL ist, so kommt der ELSE-Zweig zum Zuge. Dieser gibt dann als Spaltenwert in der Ergebnisliste *ausgeschieden* zurück. Schließlich wird der CASE-Ausdruck mit dem Schlüsselwort END beendet und mit einem Spaltenalias *mitarbeiterstatus* versehen.

In [Tabelle 15.1](#) sehen Sie das Ergebnis der Abfrage aus [Listing 15.1](#).

Name	vorname	austrittsdatum	mitarbeiterstatus
Müller	Ralf	NULL	aktiver Mitarbeiter
Schneider	Petra	2000-07-31	ausgeschieden

Tabelle 15.1 Bedingte Ausgabe von Spaltenwerten mit CASE

Name	vorname	austrittsdatum	mitarbeiterstatus
Klein	Thomas	NULL	aktiver Mitarbeiter
Lang	Ute	NULL	aktiver Mitarbeiter

Tabelle 15.1 Bedingte Ausgabe von Spaltenwerten mit CASE (Forts.)

Gemäß der Logik im CASE-Ausdruck wird je nach Ergebnis der Auswertung der Bedingung entweder `aktiver Mitarbeiter` oder `ausgeschieden` in der Ergebnisliste unter dem Spaltenalias `mitarbeiterstatus` ausgegeben.

15.3 Bedingungslogik in einer ORDER BY-Klausel anwenden

In diesem Beispiel beabsichtigen wir mit einer Auswahl von Spaltenwerten, die in einer Spalte vorkommen können, eine Sortierreihenfolge festzulegen.

In aufsteigender Sortierung sollen zuerst sämtliche Mitarbeiter gelistet werden, die an einem Abfindungsprogramm teilgenommen haben. Im Anschluss sollen die Zeilen derjenigen Mitarbeiter aufgelistet werden, die sich zwecks Veränderung entschieden haben, das Unternehmen zu verlassen. Zu guter Letzt sollen die Zeilen der Mitarbeiter in der Ergebnisliste angezeigt werden, die sich bereits im Ruhestand befinden. Aktiv im Unternehmen beschäftigte Mitarbeiter sollen nicht in der Liste vorkommen.

Betrachten wir die SELECT-Abfrage aus [Listing 15.2](#):

```
SELECT
    name,
    vorname,
    austrittsgrund
FROM
    mitarbeiter
WHERE
    austrittsgrund IS NOT NULL
ORDER BY
    CASE
        WHEN
            austrittsgrund='ruhestand'
        THEN
            3
        WHEN
            austrittsgrund='veränderung'
        THEN
            2
```

```
WHEN
    austrittsgrund='abfindungsprogramm'
THEN
    1
END
;
```

Listing 15.2 SELECT-Abfrage mit einem CASE-Ausdruck in der ORDER BY-Klausel

Die Betrachtung beginnen wir hinter der **WHERE**-Klausel, da bis dahin alles bekannt sein sollte. Sie dient offensichtlich dazu, nur die Mitarbeiter in die Liste aufzunehmen, die nicht mehr im Unternehmen tätig sind. Dies wird realisiert, indem die Spaltenwerte der Spalte *austrittsgrund* auf nicht **NULL** geprüft werden.

Hinter der **WHERE**-Klausel folgt die **ORDER BY**-Klausel. Die **ORDER BY**-Klausel benötigt ein Sortierkriterium. Ab hier betreten Sie Neuland. Hier steht nicht nur eine Spalte, deren Wert zur Sortierung verwendet wurde, sondern ein **CASE**-Ausdruck, der unsere Sortierkriterien auswertet. In [Tabelle 15.2](#) finden Sie die Kriterien noch einmal auf einen Blick so, wie wir sie in der Abfrage verwenden werden. Hinter den Bedingungen sehen Sie eine Spalte *Ergebnis*, die der Bedingung einen bestimmten Wert zuordnet.

Bedingung	Ergebnis
austrittsgrund=ruhestand	3
austrittsgrund=veränderung	2
austrittsgrund=abfindungsprogramm	1

Tabelle 15.2 Ergebnisse der Bedingungen des CASE-Ausdrucks

Die Bedingungen im **CASE**-Ausdruck liefern uns ganze Zahlen, die wir für eine numerische Sortierung verwenden können. Die **ORDER BY**-Klausel erhält für jede Zeile des **CASE**-Ausdrucks eine ganze Zahl als Sortierkriterium zurück und liefert somit die Basis für die Sortierung.

In [Tabelle 15.3](#) sehen Sie das Ergebnis, das Ihnen die Datenbank zurückliefert.

name	vorname	austrittsgrund
Haier	Peter	abfindungsprogramm
Mey	Tanja	veränderung
Lempe	Dirk	veränderung

Tabelle 15.3 Ergebnistabelle mit einer eigens festgelegten Sortierreihenfolge gemäß dem Ergebnis der Auswertung von Bedingungen in einer CASE-Klausel

name	vorname	austrittsgrund
Klarfeld	Simone	veränderung
Groß	Vera	veränderung
Schneider	Petra	ruhestand
Blume	Siegmar	ruhestand

Tabelle 15.3 Ergebnistabelle mit einer eigens festgelegten Sortierreihenfolge gemäß dem Ergebnis der Auswertung von Bedingungen in einer CASE-Klausel (Forts.)

Wir können also die vorgegebene Sortierreihenfolge verändern und Datensätze so sortieren, wie wir möchten. Dazu ordnen wir bestimmten Spaltenwerten ganze Zahlen als Sortierkriterium zu und werten diese mit **CASE** aus. Sie sehen bereits hier, dass Sie **CASE**-Ausdrücke in vielfältigen Anwendungsszenarien einsetzen können.



Zusammenfassung: Anweisungen abhängig von der Auswertung von Bedingungen ausführen

Die **CASE**-Klausel ist in SQL dazu vorgesehen, Ausdrücke abhängig von Bedingungen zurückzuliefern.

Die Auswertung einer Bedingung kann **wahr** oder **falsch** sein. Von diesen beiden Zuständen hängen Verzweigungsmöglichkeiten ab, die genutzt werden können, um bestimmte Ausdrücke zurückzuerhalten.

Wenn also eine Bedingung erfüllt ist, wird ein bestimmter Wert oder ein Ausdruck als Ergebnis zurückgegeben. Ist eine Bedingung nicht erfüllt, so wird ein anderer Wert oder Ausdruck als Ergebnis zurückgeliefert.

Im Englischen wird eine derartige Entscheidungsfindung mit den Wörtern **WHEN** (»wenn«), **THEN** (»dann«) und **ELSE** (»sonst«) abgebildet.

Die Ausführung einer bedingten Anweisung beginnt in SQL mit dem Schlüsselwort **CASE**. Hinter dem **CASE**-Schlüsselwort werden die Bedingungen (**WHEN bedingung**) formuliert. Hinter den Bedingungen folgen schließlich die bedingten Anweisungen (**THEN ausdruck**) und (**ELSE ausdruck**). Eine **CASE**-Klausel wird immer mit dem Schlüsselwort **END** abgeschlossen.

15.4 Übungen

Übung 1

Die Abteilung Unternehmensentwicklung hat die Aufgabe, die Quartalszahlen auszuwerten. Die Quartalszahlen werden in der Tabelle *umsatz* gespeichert und bewegen

sich im Bereich zwischen 6 und 12 Millionen. Die Tabelle verfügt über die Spalten *quartal* und *umsatz*, die beide in der Ergebnisliste enthalten sein sollen. Sie werden nun gebeten, eine Auswertungsspalte *statusumsatz* in einer Ergebnisliste mit abzubilden, die den Umsatz auswertet und mit einem Ampelsystem darstellt:

- ▶ Umsatz < 6.000.000 → rot
- ▶ Umsatz < 9.000.000 → gelb
- ▶ Umsatz < 12.000.000 → grün

Die farbliche Zuordnung soll mithilfe der Wörter *rot*, *gelb* und *grün* erfolgen.

Übung 2

Die Tabelle *mitarbeiter* enthält weibliche und männliche Mitarbeiter. Die Spalte *geschlecht* enthält die Spaltenwerte *m* für männlich und *w* für weiblich. Formulieren Sie eine *SELECT*-Abfrage, in der die Spalten *name* und *vorname* in der Spaltenauswahlliste enthalten sind. Bevor dies geschieht, soll durch eine *CASE*-Klausel überprüft werden, ob der Spaltenwert der Spalte *geschlecht* gleich *w* oder *m* ist. Sollte der Spaltenwert gleich *w* sein, ist die Anrede *Frau* zu verwenden. Ist der Spaltenwert hingegen gleich *m*, muss die Anrede *Herr* genutzt werden. Vergessen Sie bitte nicht, einen Spaltenalias *anrede* hinter der *CASE*-Klausel zu notieren.

Lösung zu Übung 1

```
SELECT
    quartal,
    umsatz,
    CASE
        WHEN umsatz<6000000 THEN 'rot'
        WHEN umsatz<9000000 THEN 'gelb'
        WHEN umsatz<12000000 THEN 'grün'
    END AS statusumsatz
FROM
    umsatz;
```

Listing 15.3 Lösung zu Übung 1

Lösung zu Übung 2

```
SELECT
    CASE
        WHEN geschlecht='w'
        THEN 'Frau'
        WHEN geschlecht='m'
```

```
    THEN 'Herr'  
END AS anrede,  
name,  
vorname  
FROM  
mitarbeiter;
```

Listing 15.4 Lösung zu Übung 2

Kapitel 16

Mit Zeit und Datum arbeiten

Der Umgang mit Zeit und Datum ist ein weltumspannendes Thema. Denken Sie nur an die unterschiedlichen Zeitzonen und die Zeitverschiebung, die daraus resultiert. Datumsangaben werden je nach Region in unterschiedlicher Art und Weise formatiert. Sie können natürlich auch einzelne Datumsbestandteile extrahieren, um sie für verschiedene Anwendungsfälle zu verwenden.

In dieser Einleitung geht es um Zeit und Datum. Im Verlauf des Kapitels behandle ich Datumsformate, Datumsextrakte und Datumsoperationen, z. B. wie Sie Tage zu einem bestehenden Datum hinzufügen. Bewusst habe ich nur eine kleine Auswahl von Skalarfunktionen zusammengestellt, die Ihnen hilfreich sein werden.

Wie Jahre, Monate, Tage und Stunden gemessen werden, war schon immer eine komplizierte Frage. Unser heutiges Kalendersystem entstand unter Papst Gregor XIII. und basiert auf dem julianischen Kalender. Wenn wir uns hier mit Datumsangaben befassen, gehen wir vom gregorianischen Kalender aus, der seit 1582 verwendet wird. Auch mit der Uhrzeit sieht es nicht einfach aus: Zwar werden Sie mir sicherlich zustimmen, dass ein Tag 24 Stunden hat. Aber woher wissen wir, dass es in Deutschland 13 Uhr und in Großbritannien 12 Uhr ist? Dazu gibt es mehr 24 Zeitzonen, die die Welt untergliedern. Der Wertebereich dieser Aufteilung reicht von +12 bis -12. Diese Aufteilung resultiert aus der koordinierten Weltzeit, der sogenannten UTC (*Universal Time Coordinated*). Der Londoner Stadtteil Greenwich wurde als Zentrum bestimmt und liegt in der Zone 0. Daher wird UTC in der Praxis meist mit der Zeitzone GMT (*Greenwich Mean Time*) gleichgesetzt. Die Zeitzone 0 ist der Referenzpunkt und wird angewendet, um Zeitangaben auszutauschen.

Um an die aktuelle Ortszeit zu kommen, müssen Sie im Rahmen des Wertebereiches Werte addieren oder subtrahieren. In Deutschland und in vielen anderen Ländern kommen (derzeit noch) die Sommer- und die Winterzeit hinzu. Sie merken schon: Sich mit Zeit und Datum zu befassen, ist keineswegs trivial.

16.1 Datumsformate

Neben der komplizierten Frage, wie man Zeit misst, gibt es noch die ebenso schwere Frage, wie man sie darstellt. Es wäre ja auch zu einfach, wenn man sich auf ein weltweit gültiges Datumsformat einigen würde. Stattdessen gibt es eine Fülle von gängigen Datumsformaten, denn das Datumsformat der Norm ISO 8601 hat sich im Alltag noch nicht durchgesetzt. Sehen wir uns also einfach einmal ein paar Datumsformate an:

- ▶ 20.08.1970 – das in Deutschland gebräuchliche Format ([Abschnitt 16.3](#))
- ▶ 1970-08-20 – das Datumsformat gemäß ISO 8601
- ▶ 08-20-1970 – das in den USA übliche Datumsformat

Und Sie ahnen es wahrscheinlich schon: Jede Datenbank organisiert ihre Datentypen etwas anders. Das gilt natürlich auch für den Datentyp **DATE** der MySQL-Datenbank, deren Entwickler sich am internationalen Standard der ISO 8601 orientieren. Die hier behandelten Datenbanken verfügen über Formatierungsfunktionen, die Sie individuell je nach Anforderung auf Datumswerte anwenden können.

16.2 Skalarfunktionen für Zeit- und Datumsangaben in SQL nutzen

Für die Arbeit mit Zeit- und Datumsangaben sieht der SQL-Standard Skalarfunktionen vor. Zu diesen zählen:

- ▶ **CURRENT_DATE** gibt das aktuelle Datum zurück.
- ▶ **CURRENT_TIME** gibt die aktuelle Uhrzeit zurück.
- ▶ **CURRENT_TIMESTAMP** gibt den aktuellen Zeitstempel zurück.

Diese Skalarfunktionen können Sie überall dort anwenden, wo Ausdrücke stehen können, also in der Spaltenauswahlliste einer **SELECT**-Abfrage, in einer Wertzuweisung einer **SET**-Klausel, in einer **UPDATE**-Anweisung und so weiter. Ihnen fällt sicherlich auf, dass hier keine Parameter übergeben werden. Das ist hier auch nicht notwendig, da Sie Skalarfunktionen nutzen, die lediglich aktuelle Zeit- und Datumsinformationen zurückliefern. Als Beispiel zeige ich Ihnen in [Listing 16.1](#), wie Sie Zeit- und Datumsfunktionen innerhalb der kürzesten Form einer **SELECT**-Abfrage verwenden können:

```
SELECT CURRENT_DATE;  
SELECT CURRENT_TIME;  
SELECT CURRENT_TIMESTAMP;
```

Listing 16.1 Skalarfunktionen zur Ermittlung von Zeit- und Datumsangaben gemäß dem SQL-Standard



Unterschiede zwischen den Datenbanken

MS SQL Server unterstützt lediglich die `CURRENT_TIMESTAMP`-Skalarfunktion gemäß dem SQL-Standard.

16.2.1 Datum, Zeit und Zeitstempel vom Datenbankserver ermitteln lassen

Entsprechend der Beispieldarstellung aus [Listing 16.1](#) werden Sie jetzt das aktuelle Datum bei Ihrer Datenbank erfragen. Beachten Sie, dass dies nur auf MySQL-, MariaDB- und PostgreSQL-Datenbanken funktioniert.

```
SELECT CURRENT_DATE AS aktuellesdatum;
```

Listing 16.2 Das aktuelle Datum beim Datenbankserver abfragen

Die Datenbank lässt nicht auf sich warten und liefert Ihnen, wie in [Tabelle 16.1](#) dargestellt, das aktuelle Datum gemäß der Norm ISO 8601 zurück.

aktuellesdatum
2024-11-03

Tabelle 16.1 Aktuelles Datum nach ISO 8601

Wie in der Syntaxdarstellung aus [Listing 16.1](#) ersichtlich, können wir auch die aktuelle Zeit bei der Datenbank erfragen. Auch dies probieren wir jetzt aus. Hierzu verwenden Sie die `SELECT`-Anweisung aus [Listing 16.3](#). Beachten Sie auch hier, dass diese Anweisung nur auf einer MySQL-, MariaDB- und PostgreSQL-Datenbank funktioniert.

```
SELECT CURRENT_TIME aktuellezeit;
```

Listing 16.3 Die aktuelle Zeit bei der Datenbank abfragen

aktuellezeit
11:22:55

Tabelle 16.2 Aktuelle Zeit als Ausgabe von [Listing 16.3](#)

Kommen wir zur letzten in [Listing 16.1](#) aufgeführten Skalarfunktion, dem Zeitstempel. Die `CURRENT_TIMESTAMP`-Skalarfunktion funktioniert auf allen vier in diesem Buch besprochenen Datenbanken. Wir nutzen in [Listing 16.4](#) ebenfalls die reduzierte Ver-

sion der `SELECT`-Anweisung, um den aktuellen Zeitstempel von der Datenbank abzufragen:

```
SELECT CURRENT_TIMESTAMP AS aktuellerzeitstempel;
```

Listing 16.4 Den aktuellen Zeitstempel bei der Datenbank erfragen

aktuellerzeitstempel
2024-11-03 11:26:53

Tabelle 16.3 Aktueller Zeitstempel aus [Listing 16.4](#)

16.2.2 Ergebnislisten mit einem Berichtsdatum versehen

Im nächsten Beispiel ermitteln wir sämtliche Mitarbeiter, die im Unternehmen beschäftigt sind, anhand der Informationen *mitarbeiterid*, *name*, *vorname*, *gebdatum* und *eintrittsdatum*. Außerdem soll das Datum der Datenerhebung in der Ergebnisliste enthalten sein. Um diese Anforderung zu erfüllen, fragen Sie die Tabelle *mitarbeiter* ab und ergänzen die Abfrage um eine Spalte *reportdatum*, die mit der Skalarfunktion `CURRENT_TIMESTAMP` realisiert wird:

```
SELECT
  CURRENT_TIMESTAMP AS reportdatum,
  mitarbeiterid,
  name,
  vorname,
  gebdatum,
  eintrittsdatum
FROM
  mitarbeiter;
```

Listing 16.5 Ergebnisliste mit einem Reportdatum versehen

Die Datenbank liefert Ihnen eine Ergebnisliste mit den angeforderten Spalten und dem Berichtsdatum, das mit der Skalarfunktion `CURRENT_TIMESTAMP` angefordert wurde.

reportdatum	mitarbeiterid	name	vorname	gebdatum	eintrittsdatum
2024-04-03 11:32:11	1	Müller	Ralf	1970-12-20	1990-01-13
2024-04-03 11:32:11	2	Schneider	Petra	1965-03-07	1995-08-27

Tabelle 16.4 Ergebnis zu [Listing 16.5](#)

reportdatum	mitarbeiterid	name	vorname	gebdatum	eintrittsdatum
2024-04-03 11:32:11	3	Klein	Thomas	1970-12-20	1990-01-13
2024-04-03 11:32:11	4	Lang	Ute	1985-07-19	2005-06-07

Tabelle 16.4 Ergebnis zu Listing 16.5

16.2.3 Übungen

Übung 1

Welche Norm beschreibt, wie ein Datum zu notieren ist?

Übung 2

Formulieren Sie einen `SELECT`-Befehl, in dem Sie die Spalten *name*, *vorname*, *urlaubstage* und *urlaubgenommen* der Tabelle *mitarbeiter* abfragen. Geben Sie bitte zusätzlich in jeder Zeile den aktuellen Zeitstempel unter dem Spaltenalias *datumstandurlaub* aus.

Lösung zu Übung 1

Die Norm ISO 8601.

Lösung zu Übung 2

SELECT

```
name,
vorname,
urlaubstage,
urlaubgenommen,
CURRENT_TIMESTAMP AS datumstandurlaub
```

FROM

```
mitarbeiter;
```

Listing 16.6 Lösung zu Übung 2

16.3 Zeit- und Datumsangaben formatieren

Manchmal möchte man ein Datum in einem anderen Format als ISO 8601 darstellen. Der SQL-Standard macht hierzu leider keine Vorgaben und ist uns daher keine Hilfe. Allein bei unserer kleinen Auswahl von Datenbanksystemen wie MS SQL Server, PostgreSQL, MariaDB oder MySQL finden sich drei Funktionen, die Sie für die Forma-

tierung von Datumsangaben verwenden können. Allen gemeinsam ist, dass sie mit sogenannten *Patterns* (engl. für »Muster«) auf einen Datumswert angewendet werden. Leider haben die Muster nicht bei jedem Datenbanksystem die gleiche Bedeutung.

Diese Tatsache macht es notwendig, das Thema »Formatierung von Datumsangaben« auf drei Abschnitte zu verteilen. In [Abschnitt 16.3.1](#) beschreibe ich für diejenigen unter Ihnen, die eine MySQL- oder MariaDB-Datenbank verwenden, wie Sie Datumsangaben formatieren. [Abschnitt 16.3.2](#) ist den Lesern und Leserinnen gewidmet, die eine PostgreSQL-Datenbank einsetzen. Wenn Sie mit einer MS SQL Server-Datenbank arbeiten, machen Sie sich in [Abschnitt 16.3.3](#) mit Datumsformatierungen vertraut.

16.3.1 Datumsformatierung unter MySQL und MariaDB (DATE_FORMAT)

In [Listing 16.7](#) sehen Sie die Syntax der Datumsformatierung für eine MySQL- oder eine MariaDB-Datenbank:

`DATE_FORMAT(datum, format)`

Listing 16.7 Syntax zur Formatierung von Datumswerten in einer MySQL-Datenbank

Die Skalarfunktion beginnt mit dem Funktionsnamen `DATE_FORMAT`, gefolgt von einer offenen Klammer und einem Spalten- oder Literalwert, der wiederum einem Datum entsprechen muss. In der Methodenparameterliste folgt, durch ein Komma getrennt, das Datumsformat, in dem der Datumswert ausgegeben werden soll. Abgeschlossen wird die Skalarfunktion wie üblich durch eine geschlossene Klammer.

Das Datumsformat setzt sich aus unterschiedlichen Abkürzungen zusammen, mit denen Sie ein Muster für das Datum bestimmen können. Als Nächstes nutzen wir ein Datumsmuster, um für die Auswertung aus [Listing 16.5](#) eine Ausgabe in der hierzu-lande üblichen Datumsformatierung `dd.mm.yyyy` zu erzielen.

Das Kürzel `dd` steht hier für den Tag (*day*), `mm` für den Monat (*month*) und `yyyy` für das Jahr (*year*). Das Datumsformat für die Formatierungsfunktion in MySQL oder MariaDB würde dann aus den Formatmustern zusammengesetzt, die Sie in [Tabelle 16.5](#) sehen. Ein Formatmuster beginnt in einer MySQL- und MariaDB-Datenbank immer mit einem Prozentzeichen, gefolgt vom eigentlichen Muster.

Formatmuster	Beschreibung
%d	00 bis 31 – der Tag eines Monats, numerisch zweistellig

Tabelle 16.5 Eine kleine Auswahl der Formatmuster einer MySQL- und MariaDB-Datenbank

Formatmuster	Beschreibung
%m	00 bis 12 – der Monat eines Jahres, numerisch zweistellig
%Y	Steht für eine vierstellige Jahresangabe.

Tabelle 16.5 Eine kleine Auswahl der Formatmuster einer MySQL- und MariaDB-Datenbank (Forts.)

Aus diesen drei Formatmustern können Sie ein Datumsmuster erstellen, das die hierzulande verwendete Schreibweise für ein Datum liefert. Hierzu schreiben Sie zur Formatierung eines Datums die Formatmuster in der gewünschten Reihenfolge hintereinander.

Bestimmt fällt Ihnen auf, dass ich zwischen den Formatmustern jeweils einen Punkt platziert habe, wie er in der hierzulande gebräuchlichen Schreibweise verwendet wird. Sie könnten auch beliebige andere Zeichen oder Zeichenketten einsetzen.

Achten Sie bei der Verwendung von Datumsformatmustern immer auf die Groß- und Kleinschreibung, da je nach Schreibweise eine andere Formatierung zugeordnet sein kann. Mit den in [Tabelle 16.6](#) angegebenen Mustern sollten Sie alle wichtigen Daten darstellen können.

Muster	Beschreibung
%a	abgekürzter Wochentag wie Mo., Di., Mi. ...
%b	abgekürzter Monat wie Jan., Feb. ...
%c	numerische Monatsangabe von 0 bis 12
%D	Tag des Monats mit englischer Erweiterung: 1st, 2nd ...
%d	Tag des Monats numerisch von 00 bis 31
%e	Tag des Monats numerisch von 0 bis 31
%f	Mikrosekunden
%H	Stunde eines Tages von 00 bis 23
%h	Stunde eines Tages von 01 bis 12
%i	Minute, numerisch von 00 bis 59
%j	Tag eines Jahres von 001 bis 366
%k	Stunde von 0 bis 23

Tabelle 16.6 Gebräuchliche Formatmuster einer MySQL- und MariaDB-Datenbank

Muster	Beschreibung
%l	Stunde von 1 bis 12
%M	Monatsname von Januar bis Dezember
%m	Monat, numerisch von 00 bis 12
%p	Vormittagsangabe AM oder Nachmittagsangabe PM
%r	Zeit gemäß der 12-Stunden-Notation, gefolgt von AM oder PM
%S	Sekunden von 00 bis 59
%T	Zeitangabe im Format hh:mm:ss
%U	Wochenangabe von 00 bis 53 mit einem Wochenbeginn am Sonntag
%u	Wochenangabe von 00 bis 53 mit einem Wochenbeginn am Montag
%W	Wochentagsbezeichnung von Sonntag bis Samstag
%Y	numerische Jahresangabe vierstellig
%y	numerische Jahresangabe zweistellig

Tabelle 16.6 Gebräuchliche Formatmuster einer MySQL- und MariaDB-Datenbank (Forts.)

Schauen wir uns nun den Gebrauch dieser Muster an. Erweitern Sie dazu die **SELECT**-Abfrage aus [Listing 16.5](#) mit der **DATE_FORMAT**-Skalarfunktion, um das Datum gemäß dem Format `dd.mm.yyyy` auszugeben. In [Listing 16.8](#) werden sämtliche Formatangaben der **DATE_FORMAT**-Skalarfunktion mit dem Datumsmuster `%d.%m.%Y` übergeben:

```
SELECT
  DATE_FORMAT(CURRENT_TIMESTAMP, '%d.%m.%Y') AS reportdatum,
  mitarbeiterid,
  name,
  vorname,
  DATE_FORMAT(gebdatum, '%d.%m.%Y') AS gebdatum,
  DATE_FORMAT(eintrittsdatum, '%d.%m.%Y') AS eintrittsdatum
FROM
  mitarbeiter;
```

Listing 16.8 Datumsangaben formatiert in einer SELECT-Anweisung abfragen

Als Ergebnis erhalten Sie, wie in [Tabelle 16.7](#) gezeigt, eine Liste sämtlicher Mitarbeiter mit deren Geburtsdatum und Eintrittsdatum sowie dem Erstellungsdatum der Auswertung. Die Datumsangaben sind nach Ihren Vorgaben formatiert.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03.04.2024	1	Müller	Ralf	20.12.1970	13.01.1990
03.04.2024	2	Schneider	Petra	07.03.1965	27.08.1995
03.04.2024	3	Klein	Thomas	20.12.1970	13.01.1990
03.04.2024	4	Lang	Ute	19.07.1985	07.06.2005

Tabelle 16.7 Datumsangaben gemäß der Datumsformatierung »dd.mm.yyyy« ausgeben

Die Formatierung gemäß der hierzulande üblichen Schreibweise ist uns schon mal gut gelungen. Als Nächstes sollen die Monatsangaben in der Ergebnistabelle ausgeschrieben dargestellt werden. Um diese Anforderung zu erfüllen, benötigen Sie ein weiteres Formatmuster. Ein ausgeschriebener Monatsname wird mit dem Formatmuster %M umgesetzt. Um die Monatsangaben ausgeschrieben zu erhalten, ändern Sie [Listing 16.8](#) geringfügig ab und ersetzen jeweils das Muster %m durch das Muster %M. Zudem entfernen Sie den Punkt hinter der Monatsbezeichnung und ergänzen ein Leerzeichen:

SELECT

```
DATE_FORMAT(CURRENT_TIMESTAMP, '%d. %M %Y') AS reportdatum,
mitarbeiterid,
name,
vorname,
DATE_FORMAT(gebdatum, '%d. %M %Y') AS gebdatum,
DATE_FORMAT(eintrittsdatum, '%d. %M %Y') AS eintrittsdatum
```

FROM

```
mitarbeiter;
```

Listing 16.9 Datumsangaben formatiert in einer SELECT-Anweisung mit voll ausgeschriebenem Monat

Als Ergebnis erhalten wir [Tabelle 16.8](#), in der sämtliche Datumsangaben mit einem ausgeschriebenen Monatsnamen erscheinen.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	1	Müller	Ralf	20. December 1970	13. January 1990

Tabelle 16.8 Die Monatsangaben in einer Ergebnistabelle ausgeschrieben darstellen

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	2	Schneider	Petra	07. March 1965	27. August 1995
03. April 2024	3	Klein	Thomas	20. December 1970	13. January 1990
03. April 2024	4	Lang	Ute	19. July 1985	07. June 2005

Tabelle 16.8 Die Monatsangaben in einer Ergebnistabelle ausgeschrieben darstellen (Forts.)

Ihnen ist sicherlich aufgefallen, dass die Monatsangaben in [Tabelle 16.8](#) in englischer Sprache ausgegeben wurden. Im nächsten Schritt wollen wir die Monatsangaben übersetzt sehen. Hierzu müssen Sie eine Umgebungsvariable Ihrer MySQL- bzw. MariaDB-Datenbank auf `de_DE` setzen.

So ein Kürzel sagt der MySQL- bzw. MariaDB-Datenbank, dass sie die deutsche Sprache (`de`) gemäß den in Deutschland üblichen Regeln (`DE`) verwenden soll – und nicht etwa die Sprache Deutsch nach den Konventionen der Schweiz. Diese Kürzelkombination besteht immer aus einer Abkürzung für die Sprache, gefolgt von einem Unterstrich und der Abkürzung für das Land, in dem die Sprache gesprochen wird. Die Umgebungsvariable setzen Sie durch den Befehl in [Listing 16.10](#):

SET `LC_TIME_NAMES=de_DE;`

Listing 16.10 Umgebungsvariable für die deutsche Zeit einstellen

Mit dem Kommando `SET` sagen Sie der Datenbank, dass Sie einen Wert für eine Systemvariable setzen möchten. In der Systemvariablen `LC_TIME_NAMES` werden die Informationen über die Sprachausgaben der Datenbank gespeichert. Dieser Variablen ordnen wir die Sprach- und die Länderkennung `de_DE` zu.

Gleich im Anschluss führen wir noch einmal die in [Listing 16.9](#) dargestellte `SELECT`-Anweisung aus.

Endlich! Als Ergebnis liefern Ihnen die MySQL- und die MariaDB-Datenbank die Datumsangaben jetzt in deutscher Lokalisierung: Die Spalten mit Datumsangaben enthalten die Bezeichnungen der Monate in deutscher Sprache.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	1	Müller	Ralf	20. Dezember 1970	13. Januar 1990
03. April 2024	2	Schneider	Petra	07. März 1965	27. August 1995
03. April 2024	3	Klein	Thomas	20. Dezember 1970	13. Januar 1990
03. April 2024	4	Lang	Ute	19. Juli 1985	07. Juni 2005

Tabelle 16.9 Durch die Länderkennung »de_DE« werden die Monatsnamen auf Deutsch ausgegeben.

16.3.2 Datumsformatierung unter PostgreSQL (TO_CHAR)

Zunächst zeige ich Ihnen, mit welcher Syntax Sie mit einer Skalarfunktion in einer PostgreSQL-Datenbank Datumsangaben formatieren. Eingeleitet wird die Skalarfunktion mit `TO_CHAR`, gefolgt von einer offenen Klammer und den Funktionsparametern, die den Datumswert und das Datumsformat kommasepariert an die Funktion übergeben. Die Parameterübergabe endet wie üblich mit einer schließenden Klammer.

`TO_CHAR(date, format)`

Listing 16.11 Syntax zur Formatierung von Datumswerten in einer PostgreSQL-Datenbank

Das Datumsformat setzt sich aus unterschiedlichen Abkürzungen zusammen, mit denen Sie die Zusammensetzung eines Datums bestimmen können. Im nächsten Beispiel soll die Auswertung aus [Listing 16.5](#) in einer hierzulande üblichen Datumsformatierung gemäß dem Format `dd.mm.yyyy` ausgegeben werden. Das Kürzel `dd` steht hier für den Tag (*day*), das Kürzel `mm` für den Monat (*month*) und das Kürzel `yyyy` für das Jahr (*year*). In einer PostgreSQL-Datenbank würde dann aus den Datumsfragmentstellvertretern, die Sie in [Tabelle 16.10](#) sehen, ein Datumsformatmuster erstellt, das dem hierzulande gebräuchlichen Datumsformat entspricht.

Formatmuster	Beschreibung
DD	Tag eines Monats von 1 bis 31

Tabelle 16.10 Eine kleine Auswahl der Datumsformatmuster einer PostgreSQL-Datenbank

Formatmuster	Beschreibung
MM	Monat eines Jahres von 1 bis 12
YYYY	Jahresangabe vierstellig

Tabelle 16.10 Eine kleine Auswahl der Datumsformatmuster einer PostgreSQL-Datenbank (Forts.)

Zur Formatierung eines Datums schreiben Sie die Formatmuster in der gewünschten Reihenfolge hintereinander. Bestimmt fällt Ihnen auf, dass ich jeweils zwischen den Formatmustern einen Punkt platziert habe, wie es hierzulande üblich ist. Sie können auch beliebige andere Zeichen oder Zeichenketten einsetzen. Achten Sie dabei immer auf Groß- und Kleinschreibung, da hier jeweils ein anderes Datumsformat zugeordnet sein kann.

Die gebräuchlichsten Formatmuster einer PostgreSQL-Datenbank finden Sie in [Tabelle 16.11](#). Beachten Sie bitte, dass es sich um PostgreSQL-spezifische Muster handelt, die Sie nicht in anderen Datenbanken einsetzen können. Es gibt leider keine einheitliche Vorgehensweise für die Spezifikation von Datumsformatmustern.

Muster	Beschreibung
HH	Stunde des Tages von 01 bis 12
HH24	Stunde des Tages von 00 bis 23
MI	Minute von 00 bis 59
SS	Sekunde von 00 bis 59
AM oder PM	Vormittagsangabe (AM) bzw. Nachmittagsangabe (PM)
YYYY	vierstellige Jahresangabe
YYY	die letzten drei Ziffern eines Jahres
YY	die letzten zwei Ziffern einer Jahreszahl
Y	die letzte Ziffer einer Jahreszahl
MONTH	ausgeschriebene Monatsangabe in Großbuchstaben
month	ausgeschriebene Monatsangabe in Kleinbuchstaben
MON	abgekürzte Schreibweise einer Monatsangabe in Großbuchstaben

Tabelle 16.11 Gebräuchliche Formatmuster einer PostgreSQL-Datenbank

Muster	Beschreibung
Mon	abgekürzte Schreibweise einer Monatsangabe, beginnend mit einem Großbuchstaben
mon	abgekürzte Schreibweise einer Monatsangabe in Kleinbuchstaben
MM	numerische Monatsangabe von 01 bis 12
DAY	Bezeichnung des Tages in Großbuchstaben
Day	Bezeichnung des Tages mit einem beginnenden Großbuchstaben
day	Bezeichnung des Tages in Kleinbuchstaben
DY	abgekürzte Schreibweise des Tages in Großbuchstaben
Dy	abgekürzte Schreibweise des Tages beginnend mit einem Großbuchstaben
dy	abgekürzte Schreibweise des Tages in Kleinbuchstaben
DDD	Tag des Jahres von 001 bis 366
DD	Tag des Monats von 01 bis 31
D	Tag der Woche von 1 bis 7
W	Woche des Monats von 1 bis 5
WW	Woche des Jahres von 1 bis 53
Q	Quartal des Jahres
TM Prefix	Gibt den Übersetzungsmodus für das Formatmuster gemäß der gesetzten LC_TIME an und wird vor ein Formatmusterfragment gesetzt. Zum Beispiel besagt TM_Month, dass das Formatmuster Month gemäß dem Wert der LC_TIME-Variablen ausgegeben werden soll.

Tabelle 16.11 Gebräuchliche Formatmuster einer PostgreSQL-Datenbank (Forts.)

Erweitern wir jetzt die **SELECT**-Abfrage aus [Listing 16.5](#), um das gewünschte Datumsformat zu erhalten:

SELECT

```
TO_CHAR(CURRENT_TIMESTAMP, 'DD.MM.YYYY') AS reportdatum,
mitarbeiterid,
name,
vorname,
```

```
TO_CHAR(gebdatum, 'DD.MM.YYYY') AS gebdatum,  
TO_CHAR(eintrittsdatum, 'DD.MM.YYYY') AS eintrittsdatum  
FROM  
mitarbeiter;
```

Listing 16.12 Datumsangaben mit der Funktion TO_CHAR formatiert in einer SELECT-Anweisung abfragen

Als Ergebnisliste erhalten Sie von der PostgreSQL-Datenbank sämtliche Datumsangaben in der Spaltenauswahlliste Ihrer SELECT-Anweisung gemäß dem Format, das Sie in der Datumsformatierung mit dem Muster DD.MM.YYYY festgelegt haben.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03.04.2024	1	Müller	Ralf	20.12.1970	13.01.1990
03.04.2024	2	Schneider	Petra	07.03.1965	27.08.1995
03.04.2024	3	Klein	Thomas	20.12.1970	13.01.1990
03.04.2024	4	Lang	Ute	19.07.1985	07.06.2005

Tabelle 16.12 Datumsangaben gemäß der Datumsformatierung »dd.mm.yyyy«

Die Datumsformatierung ist uns also gelungen. Als Nächstes soll die Monatsbezeichnung in der Datumsausgabe ausgeschrieben dargestellt werden.

Um diese Anforderung zu erfüllen, benötigen Sie ein weiteres Formatmuster. Ein ausgeschriebener Monatsname wird bei einer PostgreSQL-Datenbank über das Formatmuster Month angefordert. Wir ändern das Datumsmuster in [Listing 16.12](#) geringfügig ab: Wir ersetzen jeweils das Muster MM durch Month und den Punkt hinter der Monatsbezeichnung durch ein Leerzeichen. Hinter dem Punkt eines Tages ergänzen wir ebenfalls ein Leerzeichen, sodass folgendes Datumsmuster entsteht: DD. Month YYYY.

SELECT

```
TO_CHAR(CURRENT_TIMESTAMP, 'DD. Month YYYY') AS reportdatum,  
mitarbeiterid,  
name,  
vorname,  
TO_CHAR(gebdatum, 'DD. Month YYYY') AS gebdatum,  
TO_CHAR(eintrittsdatum, 'DD. Month YYYY') AS eintrittsdatum  
FROM  
mitarbeiter;
```

Listing 16.13 Datumsangaben formatiert in einer SELECT-Anweisung mit voll ausgeschriebenen Monatsnamen ausgeben

Als Ergebnisliste erhalten Sie [Tabelle 16.13](#), in der die Monatsangaben voll ausgeschrieben sind.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	1	Müller	Ralf	20. December 1970	13. January 1990
03. April 2024	2	Schneider	Petra	07. March 1965	27. August 1995
03. April 2012	3	Klein	Thomas	20. December 1970	13. January 1990
03. April 2024	4	Lang	Ute	19. July 1985	07. June 2005

Tabelle 16.13 Die Monatsangaben ausgeschrieben darstellen

Das Ergebnis der letzten Auswertung in [Tabelle 16.13](#) sieht schon ganz gut aus. Die Monatsangaben werden jedoch noch in englischer Sprache ausgegeben. Damit sie auf Deutsch erscheinen, müssen Sie eine Umgebungsvariable Ihrer PostgreSQL-Datenbank auf `German_Germany` setzen. Die Umgebungsvariable sagt der PostgreSQL-Datenbank, dass sie die deutsche Sprache gemäß den in Deutschland üblichen Gepflogenheiten soll – und nicht etwa Schweizerdeutsch. Die Umgebungsvariable setzen Sie mit dem Kommando `SET`, wie in [Listing 16.14](#) zu sehen:

```
SET LC_TIME=German_Germany;
```

Listing 16.14 Lokale Datumsformatierung in der PostgreSQL-Datenbank einstellen

Die Systemvariable lautet hier `LC_TIME`, also lokale Zeit. Dieser Variablen ordnen wir die Sprach- und Länderkennung `German_Germany` zu. Nachdem Sie die lokale Zeiteinstellung vorgenommen haben, müssen Sie im Datumsformatmuster auch angeben, dass diese lokale Zeiteinstellung verwendet werden soll. Das erreichen Sie, indem Sie `TM` (*Translation Mode*) vor dem Datumsformat notieren. Die folgende `SELECT`-Anweisung verwendet ein Datumsmuster, das den Translation Mode anwendet:

SELECT

```
TO_CHAR(CURRENT_TIMESTAMP, 'DD. TMonth YYYY') AS reportdatum,
mitarbeiterid,
name,
vorname,
TO_CHAR(gebdatum, 'DD. TMonth YYYY') AS gebdatum,
TO_CHAR(eintrittsdatum, 'DD. TMonth YYYY') AS eintrittsdatum
```

FROM

mitarbeiter;

Listing 16.15 Datumsangaben formatiert in einer SELECT-Anweisung mit voll ausgeschriebenem Monat in derjenigen Landessprache ausgeben, die in der Umgebungsvariablen »LC_TIMES« festgelegt wurde

Die PostgreSQL-Datenbank liefert Ihnen, wie in Tabelle 16.14 gezeigt, ein Ergebnis, in dem die Monatsangaben in deutscher Sprache und in ausgeschriebener Form dargestellt werden.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	1	Müller	Ralf	20. Dezember 1970	13. Januar 1990
03. April 2024	2	Schneider	Petra	07. März 1965	27. August 1995
03. April 2024	3	Klein	Thomas	20. Dezember 1970	13. Januar 1990
03. April 2024	4	Lang	Ute	19. Juli 1985	07. Juni 2005

Tabelle 16.14 Monatsangaben gemäß der Lokalisierung in deutscher Sprache

16.3.3 Datumsformatierung unter MS SQL Server (FORMAT)

Die Syntax der Datumsformatierung für eine MS SQL Server-Datenbank sehen Sie in Listing 16.16:

FORMAT(wert,format[,lokalisierung])

Listing 16.16 Syntax zur Formatierung von Datumswerten in einer MS SQL Server-Datenbank

Der Aufruf beginnt mit dem Namen der Funktion, der schlicht und ergreifend **FORMAT** lautet, gefolgt von einer offenen Klammer und einem Spalten- oder Literalwert, der einem Datum entspricht. In der Methodenparameterliste folgt nun durch ein Komma getrennt das Datumsformat. Optional können Sie Sprache und Land als Parameter in der Form de_DE angeben. Beendet wird der Aufruf wie üblich durch eine schließende Klammer.

Das Datumsformat setzt sich aus unterschiedlichen Abkürzungen zusammen, mit denen Sie die Zusammensetzung eines Datums bestimmen. Eine vollständige Auflistung der verfügbaren Optionen finden Sie in Tabelle 16.15.

Muster	Beschreibung
d	der Tag eines Monats von 01 bis 31
ddd	abgekürzter Tag des Monats von Mon. bis Son.
dddd	die vollständige Bezeichnung eines Wochentages von Montag bis Sonntag
h	Stunde eines Tages von 1 bis 12
H	Stunde des Tages von 0 bis 23
HH	Stunde des Tages von 01 bis 23
m	die Minute einer Stunde von 0 bis 59
mm	die Minute einer Stunde von 00 bis 59
MMM	der abgekürzte Monatsname eines Jahres von Jan. bis Dez.
MMMM	die voll ausgeschriebene Monatsbezeichnung eines Jahres von Januar bis Dezember
s	die Sekunde einer Minute von 0 bis 59
ss	die Sekunde einer Minute von 00 bis 59
tt	Vormittagsangabe AM und Nachmittagsangabe PM gemäß englischer Notation
y	Jahresangabe von 0 bis 99
yy	Jahresangabe von 00 bis 99
yyyy	Jahresangabe vierstellig

Tabelle 16.15 Gebräuchliche Formatmuster einer MS SQL Server-Datenbank

Bringen wir nun unser Beispiel aus [Listing 16.5](#) in die in Deutschland übliche Datumsformatierung gemäß dem Format `dd.mm.yyyy`. Das Kürzel `dd` steht hier für den Tag (*day*), das Kürzel `mm` für den Monat (*month*) und das Kürzel `yyyy` für das Jahr (*year*). Das Datumsformat für die Formatierungsfunktion innerhalb einer MS SQL Server-Datenbank wird mithilfe der Formatmuster aus [Tabelle 16.16](#) zusammengesetzt.

Muster	Beschreibung
dd	Tag des Monats von 01 bis 31

Tabelle 16.16 Eine kleine Auswahl der Datumsformatmuster einer MS SQL Server-Datenbank

Muster	Beschreibung
MM	Monat des Jahres von 01 bis 12
yyyy	Jahreszahl vierstellig

Tabelle 16.16 Eine kleine Auswahl der Datumsformatmuster einer MS SQL Server-Datenbank (Forts.)

Diese drei Muster schreiben wir hintereinander und erhalten so `dd.MM.yyyy`. Bestimmt fällt Ihnen auf, dass ich zwischen den Formatmustern jeweils einen Punkt platziert habe, wie es im deutschsprachigen Raum üblich ist. Sie könnten auch beliebige andere Zeichen oder Zeichenketten einsetzen – das nur als Hinweis am Rande. Achten Sie immer auf die Groß- und Kleinschreibung der Formatmuster, da damit jeweils eine andere Formatierung zugeordnet werden kann.

Erweitern wir jetzt die `SELECT`-Abfrage aus [Listing 16.5](#), und verwenden wir die `FORMAT`-Funktion, um ein Datum formatiert auszugeben. In [Listing 16.17](#) werden sämtliche Datumsangaben der `FORMAT`-Funktion mit dem Datumsmuster `dd.MM.yyyy` und dem aktuellen Zeitstempel übergeben:

```
SELECT
    FORMAT(CURRENT_TIMESTAMP, 'dd.MM.yyyy') AS reportdatum,
    mitarbeiterid,
    name,
    vorname,
    FORMAT(gebdatum, 'dd.MM.yyyy') AS gebdatum,
    FORMAT(eintrittsdatum, 'dd.MM.yyyy') AS eintrittsdatum
FROM
    mitarbeiter;
```

Listing 16.17 Datumsangaben formatiert in einer `SELECT`-Anweisung abfragen

Als Ergebnisliste erhalten Sie [Tabelle 16.17](#) zurück, die uns sämtliche Datensätze aller Mitarbeiter mit der von Ihnen abgefragten Spaltenauswahlliste liefert. Die Datumsangaben der Spalten `reportdatum`, `gebdatum` und `eintrittsdatum` wurden im von uns festgelegten Datumsformat zurückgeliefert.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03.04.2024	1	Müller	Ralf	20.12.1970	13.01.1990
03.04.2024	2	Schneider	Petra	07.03.1965	27.08.1995

Tabelle 16.17 Datumsangaben gemäß der Datumsformatierung »dd.MM.yyyy«

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03.04.2024	3	Klein	Thomas	20.12.1970	13.01.1990
03.04.2024	4	Lang	Ute	19.07.1985	07.06.2005

Tabelle 16.17 Datumsangaben gemäß der Datumsformatierung »dd.MM.yyyy« (Forts.)

Das Ergebnis aus [Tabelle 16.17](#) sieht schon mal gut aus. Als Nächstes soll das Datum voll ausgeschrieben in den Ergebnistabellen enthalten sein. Um diese Anforderung zu erfüllen, benötigen Sie ein weiteres Formatmuster. Ein ausgeschriebener Monatsname wird bei einer MS SQL Server-Datenbank über das Muster `MMMM` dargestellt. Um diese Anforderung zu realisieren, ändern wir [Listing 16.17](#) geringfügig ab: Wir ersetzen jeweils das Muster `MM` durch `MMMM`, entfernen den Punkt hinter der Monatsbezeichnung und ergänzen an seiner Stelle ein Leerzeichen. Hinter dem Punkt eines Tages und eines Monats ergänzen wir ebenfalls ein Leerzeichen, sodass folgendes Datumsmuster entsteht: `dd. MMMM yyyy`.

SELECT

```
FORMAT(CURRENT_TIMESTAMP, 'dd. MMMM yyyy') AS reportdatum,
mitarbeiterid,
name,
vorname,
FORMAT(gebdatum, 'dd. MMMM yyyy') AS gebdatum,
FORMAT(eintrittsdatum, 'dd. MMMM yyyy') AS eintrittsdatum
```

FROM

```
mitarbeiter;
```

Listing 16.18 Datumsangaben formatiert in einer SELECT-Anweisung mit voll ausgeschriebenen Monatsnamen ausgeben

Die MS SQL Server-Datenbank liefert Ihnen nun wie in [Tabelle 16.18](#) die ausgeschriebenen Monatsbezeichnungen zurück.

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	1	Müller	Ralf	20. Dezember 1970	13. Januar 1990
03. April 2024	2	Schneider	Petra	07. März 1965	27. August 1995
03. April 2024	3	Klein	Thomas	20. Dezember 1970	13. Januar 1990

Tabelle 16.18 Monatsangaben in deutscher Sprache ausgeben

reportdatum	mid	name	vorname	gebdatum	eintrittsdatum
03. April 2024	4	Lang	Ute	19. Juli 1985	07. Juni 2005

Tabelle 16.18 Monatsangaben in deutscher Sprache ausgeben (Forts.)

16.3.4 Übungen

Übung 1

Nutzen Sie die Kurzform einer **SELECT**-Abfrage, um mit einem aktuellen Zeitstempel die Uhrzeit in der Form 13:00:00 auszugeben. Nutzen Sie jeweils die Formatmuster, die für Ihre Datenbank Verwendung finden.

Übung 2

Formulieren Sie eine Abfrage, in der die Spalten *name*, *vorname*, *gebdatum* und *eintrittsdatum* der Tabelle *mitarbeiter* enthalten sind. Das Geburtsdatum soll hier als 01. August 1990 ausgegeben werden. Das Eintrittsdatum wiederum soll gemäß folgendem Beispiel dargestellt werden: 01.04.89.

Achten Sie bei dieser Übung darauf, welches Datenbanksystem Sie verwenden. Bei einer MySQL- oder MariaDB-Datenbank bietet sich zur Lösung dieser Übung die Skalarfunktion **DATE_FORMAT** an. Bei einer PostgreSQL-Datenbank greifen Sie zur Lösung dieser Übung zur Skalarfunktion **TO_CHAR**. Falls Sie mit einer MS SQL Server-Datenbank arbeiten, führt die **FORMAT**-Skalarfunktion zur Lösung dieser Übung. Achten Sie auch auf die Formatmuster, die Ihnen zur Verfügung stehen.

Lösung zu Übung 1

```
/* MySQL- und MariaDB-Variante */
SELECT
    DATE_FORMAT(CURRENT_TIMESTAMP, '%H: %i %S')
    AS aktuelleuhrzeit;

/* PostgreSQL-Variante */
SELECT
    TO_CHAR(CURRENT_TIMESTAMP, 'HH24:MI:SS')
    AS aktuelleuhrzeit;

/* MS SQL Server-Variante */
SELECT
    FORMAT(CURRENT_TIMESTAMP, 'HH:mm:ss')
    AS aktuelleuhrzeit;
```

Listing 16.19 Die aktuelle Uhrzeit formatiert ausgeben

Lösung zu Übung 2

```

/* MySQL- und MariaDB-Variante */
SELECT
    name,
    vorname,
    DATE_FORMAT(gebdatum, '%d %M %Y'),
    DATE_FORMAT(eintrittsdatum, '%d.%m.%y')
FROM
    mitarbeiter;

/* PostgreSQL-Variante */
SELECT
    name,
    vorname,
    TO_CHAR(gebdatum, 'DD TMonth YYYY'),
    TO_CHAR(eintrittsdatum, 'DD.MM.YY')
FROM
    mitarbeiter;

/* MS SQL Server-Variante */
SELECT
    name,
    vorname,
    FORMAT(gebdatum, 'dd MMMM yyyy'),
    FORMAT(eintrittsdatum, 'dd.MM.yy')
FROM
    mitarbeiter;

```

Listing 16.20 Datumsangaben formatiert ausgeben für MySQL, MariaDB, PostgreSQL und MS SQL Server

16.4 Datumsangaben extrahieren (EXTRACT)

In diesem Abschnitt zeige ich Ihnen, wie Sie Bestandteile eines Datums extrahieren. SQL sieht für diese Aufgabe eine Skalarfunktion mit der Bezeichnung **EXTRACT** vor:

EXTRACT(teileeinesdatums **FROM** datum)

Listing 16.21 Syntax für die Anwendung der SQL-EXTRACT-Skalarfunktion

Diese Funktion bekommt zwei Parameter übergeben. Der erste Parameter, *teileeinesdatums*, gibt an, welchen Teil eines Datums Sie extrahieren möchten. Dies kann der Tag eines Monats, ein Monat eines Jahres oder aber auch das Jahr selbst sein.

Als zweiten Parameter geben Sie hinter dem Schlüsselwort **FROM** mit `datum` den Wert desjenigen Datums an, aus dem ein Teil extrahiert werden soll. Die SQL-konforme Skalarfunktion wird von der MySQL-, MariaDB- und PostgreSQL-Datenbank unterstützt.

In [Tabelle 16.19](#) und [Tabelle 16.20](#) finden Sie die gebräuchlichsten Datumsbestandteile, über die Sie mit der **EXTRACT**-Skalarfunktion Datumsangaben extrahieren. [Tabelle 16.19](#) gilt für die MySQL- bzw. MariaDB-Datenbank, [Tabelle 16.20](#) für die PostgreSQL-Datenbank.

Datumsextrakt	Beschreibung
SECOND	die Sekunde einer Minute
MINUTE	die Minute einer Stunde
HOURL	die Stunde eines Tages
DAY	der Tag eines Monats
WEEK	die Woche eines Jahres
MONTH	der Monat eines Jahres
QUARTER	das Quartal eines Jahres
YEAR	das Jahr
MINUTE_SECOND	die Sekunde einer Minute
HOURL_MINUTE	die Minute einer Stunde
DAY_HOUR	die Stunde eines Tages

Tabelle 16.19 Extraktionsmuster für Datumsangaben für eine MySQL- und eine MariaDB-Datenbank

Datumsextrakt	Beschreibung
day	der Tag eines Monats
month	der Monat eines Jahres
year	das Jahr
hour	die Stunde eines Tages

Tabelle 16.20 Extraktionsmuster für Datumsangaben für eine PostgreSQL-Datenbank

Datumsextrakt	Beschreibung
minute	die Minute einer Stunde
second	die Sekunde einer Minute
quarter	das Quartal eines Jahres
week	die Woche eines Jahres

Tabelle 16.20 Extraktionsmuster für Datumsangaben für eine PostgreSQL-Datenbank (Forts.)

Beide Datenbanken verwenden zwar gemäß dem SQL-Standard die **EXTRACT**-Funktion, um Datumsbestandteile zu extrahieren, bieten jedoch unterschiedliche Extraktionsmuster an.

Die MS SQL Server-Datenbank fordert einen anderen Aufruf mit der Bezeichnung **DATEPART**. Als Parameter bekommt diese Skalarfunktion ebenfalls mitgeteilt, welcher Datumsanteil eines Datums extrahiert werden soll, sowie – durch ein Komma getrennt – das Datum, auf das die Funktion angewendet werden soll:

DATEPART(teileeinesdatums,datum)

Listing 16.22 Syntax für die Anwendung der MS SQL Server-spezifischen **DATEPART**-Skalarfunktion zur Extraktion von Datumsbestandteilen

Im nächsten Beispiel betrachten wir eine **SELECT**-Anweisung, in der sämtliche Mitarbeiter mit ihrem Namen, Vornamen, Eintrittsdatum, Austrittsdatum und (falls im Datenbestand vorhanden) Geburtsdatum abgefragt werden. Bei den bis jetzt aufgeführten Datumsangaben sollen lediglich die Jahreszahlen ausgegeben werden. Auf die Spalte *reportdatum* soll hier verzichtet werden. Falls Sie mit einer MS SQL Server-Datenbank arbeiten, finden Sie ein Beispiel in [Listing 16.24](#). Die SQL-Standard-konforme Lösung ist in [Listing 16.23](#) dargestellt:

```
SELECT
    name,
    vorname,
    EXTRACT(year FROM gebdatum) AS gebdatumjahr,
    EXTRACT(year FROM eintrittsdatum) AS eintrittsdatumjahr
FROM
    mitarbeiter;
```

Listing 16.23 Jahresangabe eines Datums in einer SELECT-Abfrage extrahieren

Als Ergebnis erhalten Sie [Tabelle 16.21](#) zurück, in der die MySQL-Datenbank durch die Skalarfunktion `EXTRACT` die Jahreszahlen extrahiert hat.

name	vorname	gebdatumjahr	eintrittsdatumjahr
Müller	Ralf	1970	1990
Schneider	Petra	1965	1995
Klein	Thomas	1970	1990
Lang	Ute	1985	2005

Tabelle 16.21 Jahresangaben aus einem Datum extrahieren

Falls Sie eine MS SQL Server-Datenbank nutzen, modifizieren Sie die Abfrage aus [Listing 16.23](#) ein wenig und verwenden anstelle der ANSI-Skalarfunktion `EXTRACT` die MS SQL Server-Variante `DATEPART`, wie in [Listing 16.24](#) gezeigt:

```
SELECT
    name,
    vorname,
    DATEPART(year,gebdatum) AS gebdatumjahr,
    DATEPART(year,eintrittsdatum) AS eintrittsdatumjahr
FROM
    mitarbeiter;
```

Listing 16.24 Jahresangabe eines Datums in einer SELECT-Abfrage auf einem MS SQL Server-Datenbanksystem extrahieren

In [Tabelle 16.22](#) sehen Sie das Ergebnis der Abfrage. Die Datenbank liefert Ihnen die extrahierten Datumsangaben für die Spalten *gebdatum* und *eintrittsdatum* zurück.

name	vorname	gebdatumjahr	eintrittsdatumjahr
Müller	Ralf	1970	1990
Schneider	Petra	1965	1995
Klein	Thomas	1970	1990
Lang	Ute	1985	2005

Tabelle 16.22 Jahresangaben aus einem Datum extrahieren

In [Tabelle 16.23](#) finden Sie die gebräuchlichsten Datumsbestandteile für die **DATEPART**-Skalarfunktion der MS SQL Server-Datenbank, mit denen Sie einen Datumswert extrahieren können.

Datumsextrakt	Beschreibung
year	das Jahr eines Datums
quarter	das Quartal eines Jahres
month	der Monat eines Jahres
dayofyear	der Tag eines Jahres
day	der Tag eines Monats
week	die Woche eines Jahres
hour	die Stunde eines Tages
minute	die Minute einer Stunde
second	die Sekunde einer Minute

Tabelle 16.23 Extraktionsmuster für Datumsangaben für eine MS SQL Server-Datenbank

16.4.1 Übungen

Übung 1

Bitte extrahieren Sie aus den Spaltenwerten der Spalte *austrittsdatum* der Tabelle *mitarbeiter* lediglich die Jahresangabe. In die Spaltenauswahlliste nehmen Sie die Spalten *name* und *vorname* mit auf, um das Austrittsjahr einem Mitarbeiter zuzuordnen zu können.

Übung 2

In dieser Übung ermitteln Sie, in welchen Monaten Bonuszahlungen ausgezahlt wurden. Die Tabelle *bonus* enthält eine Spalte mit der Bezeichnung *auszahlungsdatum*. Diese können Sie nutzen, um die Monatsangabe aus den Spaltenwerten des Auszahlungsdatums zu extrahieren. In der Ergebnisliste sollen auch die Spalten *mitarbeiter-id* und *bonuszahlung* enthalten sein.

Wenn Sie mit MySQL-, MariaDB- oder PostgreSQL-Datenbanken arbeiten, können Sie die ANSI-SQL-konforme Skalarfunktion **EXTRACT** verwenden, um diese Übung durchzuführen. Arbeiten Sie mit einer MS SQL Server-Datenbank, dann verwenden Sie die **DATEPART**-Skalarfunktion.

Lösung zu Übung 1

```
/* MySQL-, MariaDB- und PostgreSQL-Variante */
SELECT
    name,
    vorname,
    EXTRACT(YEAR FROM austrittsdatum)
FROM
    mitarbeiter;

/* MS SQL Server-Variante */
SELECT
    name,
    vorname,
    DATEPART(year, austrittsdatum)
FROM
    mitarbeiter;
```

Listing 16.25 Jahresangabe aus einem Datumswert extrahieren

Lösung zu Übung 2

```
/* MySQL-, MariaDB und PostgreSQL-Variante */
SELECT
    mitarbeiterid,
    bonuszahlung,
    EXTRACT(month FROM auszahlungsdatum)
FROM
    bonus;

/* MS SQL Server-Variante */
SELECT
    mitarbeiterid,
    bonuszahlung,
    DATEPART(month, auszahlungsdatum)
FROM
    bonus;
```

Listing 16.26 Monatsangabe aus einem Datumswert extrahieren

16.5 Mit Datumsangaben rechnen

Egal, ob es sich um eine Sekunde, eine Minute oder eine Stunde handelt, jede Zeiteinheit hat einen Beginn und ein Ende. Die kleinste Zeiteinheit, die wir betrachten, ist die

Sekunde. Es folgt eine Minute, die 60 Sekunden enthält. Eine Stunde dauert wiederum 60 Minuten. Es gibt natürlich noch weitere Zeitintervalle. Und auch wenn wir mit Datumsangaben rechnen, bezieht sich die Berechnung immer auf Intervalle.

Stellen wir uns vor, wir würden einen Urlaub planen. Nach unserer Vorstellung beginnt der Urlaub am 2. Januar 2024 und endet am 10. Januar 2024. Jetzt möchten wir den Urlaub um drei Tage verlängern.

In [Abbildung 16.1](#) sehen Sie zwei Intervalle. Das erste geht vom 2. Januar bis zum 10. Januar. Das zweite Intervall ist drei Tage lang und stellt somit unsere Urlaubsverlängerung dar. Schließlich werden diese beiden Intervalle durch eine Addition zusammengefügt, und der Urlaubszeitraum verlängert sich um drei Tage bis zum 13. Januar.

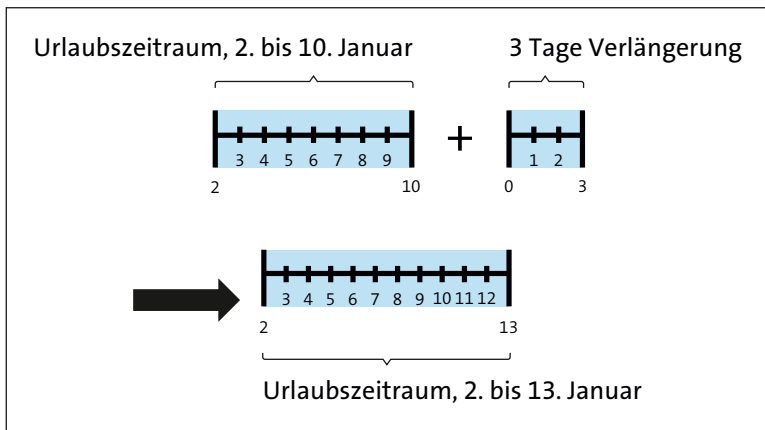


Abbildung 16.1 Intervallbetrachtungen

Wie lässt sich so etwas mit SQL realisieren? Wieder machen es uns die hier behandelten Datenbanken etwas schwer, denn in jeder funktioniert die Berechnung anders: Arbeiten Sie mit einer MySQL- oder MariaDB-Datenbank, dann sind Sie in [Abschnitt 16.5.1](#) gut aufgehoben. Falls Sie eine PostgreSQL-Datenbank verwenden, dann sind Sie in [Abschnitt 16.5.2](#) richtig. Sie nutzen eine MS SQL Server-Datenbank? Für Sie ist [Abschnitt 16.5.3](#).

16.5.1 Mit Datumswerten rechnen unter MySQL und MariaDB

Als Erstes betrachten wir die Syntax in [Listing 16.27](#), die Sie verwenden können, um zu einem Datumswert ein Intervall zu addieren oder davon zu subtrahieren:

```
/* Intervall zu einem Datum addieren*/
SELECT DATE_ADD(datum, INTERVAL zahl datumseinheit)
```

```
/* Intervall von einem Datum subtrahieren*/  
SELECT DATE_SUB(datevalue,INTERVAL zahl datumseinheit)
```

Listing 16.27 Ein Intervall zu einem Datum hinzufügen bzw. von ihm abziehen

Hierzu nutzen wir wieder unsere Kurzform für eine **SELECT**-Anweisung. Wie aus der Syntax hervorgeht, wird für die Addition eines Intervalls zu einem Datum die Skalarfunktion **DATE_ADD** verwendet. Möchten Sie hingegen ein Intervall subtrahieren, so verwenden Sie die Skalarfunktion **DATE_SUB**.

Sehen wir uns die Parameter, die wir den Funktionen übergeben, genauer an. Die Parameterlisten der beiden Funktionen sind identisch. Das macht es uns etwas einfacher. Zuerst übergeben wir das Datum, zu dem ein Intervall addiert oder von dem ein Intervall subtrahiert wird. Es folgt ein Komma, und schließlich übergeben wir den Funktionen den zweiten Parameter. Der zweite Parameter besteht aus dem Schlüsselwort **INTERVAL**, gefolgt von einer Zahl und der Datumseinheit, die für die Zahl verwendet werden soll.

Folgende Datumseinheiten können Sie für den Parameter *datumseinheit* einer MySQL- bzw. einer MariaDB-Datenbank verwenden:

Datumseinheit	Parameter
SECOND	Einheit für Sekunden
MINUTE	Einheit für Minuten
HOURL	Einheit für Stunden
DAY	Einheit für Tage
WEEK	Einheit für Wochen
MONTH	Einheit für Monate
QUARTER	Einheit für Quartale
YEAR	Einheit für Jahre

Tabelle 16.24 Datumseinheiten zum Addieren und Subtrahieren von Intervallen

Jetzt habe ich Sie mit den Werkzeugen ausgestattet, mit denen Sie ein kleines Beispiel bearbeiten können. Wir addieren zum Datumswert 2024-03-01 zehn Tage und nutzen wieder die Kurzform einer **SELECT**-Anweisung. In [Tabelle 16.25](#) sehen Sie das Ergebnis.

```
SELECT DATE_ADD('2024-03-01',INTERVAL 10 DAY) AS datumaddition;
```

Listing 16.28 Ein Intervall zu einem Datumswert addieren

datumaddition
2024-03-11

Tabelle 16.25 Addition eines Intervalls zu einem Datum

16.5.2 Mit Datumswerten rechnen unter PostgreSQL

Als Erstes betrachten wir die Syntax in [Listing 16.29](#), die Sie verwenden können, um Intervalle zu Datumswerten zu addieren oder von ihnen zu subtrahieren. Hierzu nutzen wir wieder unsere Kurzform für eine **SELECT**-Anweisung.

```
/* Intervall zu einem Datum addieren */
SELECT DATE 'jahr-monat-tag' + INTERVAL 'x datumseinheit'

/* Intervall von einem Datum subtrahieren */
SELECT DATE 'jahr-monat-tag' - INTERVAL 'x datumseinheit'
```

Listing 16.29 Ein Intervall zu einem Datum hinzufügen bzw. von ihm abziehen

Hinter dem **SELECT**-Schlüsselwort sehen Sie zunächst den Datentyp **DATE**, der vor einem Datumsliteral steht. Die PostgreSQL-Datenbank stellt so sicher, dass es sich bei dem in Hochkommas eingeschlossenen Literal tatsächlich um ein Datum handelt und nicht um eine Zeichenkette.

In der Syntax verwende ich stellvertretend für ein Datumsliteral die Zeichenfolge `jahr-monat-tag`. Mit einem einfachen Plusoperator signalisieren wir der Datenbank, dass wir einen Zeitraum hinzufügen wollen. Hinter dem Pluszeichen notieren wir hierzu das Schlüsselwort **INTERVAL**, um der Datenbank mitzuteilen, dass es sich um ein zeitliches Intervall handelt, das wir zum Datumswert addieren werden.

Was uns jetzt noch fehlt, ist der konkrete Zeitraum, den wir dem Literal hinzufügen wollen. Diesen Zeitraum notieren wir ebenfalls in einfachen Hochkommas. Der Zeitraum besteht wiederum aus zwei Komponenten: aus einem ganzzahligen numerischen Wert und aus der dazugehörigen Einheit. Bei der Einheit kann es sich um Tage, Monate, Jahre etc. handeln.

In [Tabelle 16.26](#) finden Sie eine Auswahl der Datumseinheiten, die Sie für den Parameter *datumseinheit* einer PostgreSQL-Datenbank verwenden können.

Datumseinheit	Parameter
SECOND	Einheit für Sekunden
MINUTE	Einheit für Minuten

Tabelle 16.26 Datumseinheiten zum Addieren und Subtrahieren von Intervallen

Datumseinheit	Parameter
hour	Einheit für Stunden
day	Einheit für Tage
month	Einheit für Monate
year	Einheit für Jahre

Tabelle 16.26 Datumseinheiten zum Addieren und Subtrahieren von Intervallen (Forts.)

Nun kennen Sie alles, um ein kleines Beispiel zu bearbeiten. Wir addieren zum Datums-
wert 2024-03-01 zehn Tage und nutzen wieder die Kurzform einer **SELECT**-Anweisung:

```
SELECT DATE '2024-03-01' + INTERVAL '10 DAY' AS datumaddition;
```

Listing 16.30 Ein Intervall zu einem Datumswert addieren

In Tabelle 16.27 sehen Sie das Ergebnis unserer Addition. Zum Datumswert 2024-03-
01 wurde ein Intervall mit 10 Tagen addiert.

datumaddition
2024-03-11 00:00:00

Tabelle 16.27 Addition eines Intervalls zu einem Datum

16.5.3 Mit Datumswerten rechnen unter MS SQL Server

Als Erstes betrachten wir die Syntax in Listing 16.31, die Sie verwenden können, um In-
tervalle zu Datumswerten zu addieren oder davon zu subtrahieren. Hierzu nutzen
wir wieder unsere Kurzform für eine **SELECT**-Anweisung.

```
/* Intervall zu einem Datum addieren */  
SELECT DATEADD(datumseinheit,zahl,datum)  
  
/* Intervall von einem Datum subtrahieren */  
SELECT DATEADD(datumseinheit,-zahl,datum)
```

Listing 16.31 Ein Intervall zu einem Datum hinzufügen bzw. von ihm abziehen

Wie aus der Syntax hervorgeht, wird für die Addition eines Intervalls zu einem Da-
tum die Skalarfunktion **DATEADD** verwendet. Möchten Sie ein Intervall subtrahieren, so
verwenden Sie ebenfalls die Skalarfunktion **DATEADD**.

Sehen wir uns die Parameter, die wir den Funktionen übergeben, genauer an. Zuerst übergeben wir die Einheit des Wertes, den wir hinzufügen oder abziehen wollen. Es folgt ein Komma, und schließlich übergeben wir der Funktion den zweiten Parameter. Hierbei handelt es sich um einen ganzzahligen numerischen vorzeichenbehafteten Wert. Sie subtrahieren also ein Intervall, indem Sie schlicht und ergreifend ein negatives Vorzeichen für das zu subtrahierende Intervall verwenden. Als letzten Parameter übergeben wir den Funktionen schließlich den Datumswert, dem wir einen Intervallwert hinzuzufügen werden bzw. von dem wir subtrahieren.

Tabelle 16.28 zeigt eine Auswahl der Datumseinheiten, die Sie für den Parameter `datumseinheit` (`dateunit`) einer MS SQL Server-Datenbank verwenden können:

Datumseinheit	Parameter
SECOND	Einheit für Sekunden
MINUTE	Einheit für Minuten
HOURL	Einheit für Stunden
DAY	Einheit für Tage
WEEK	Einheit für Wochen
MONTH	Einheit für Monate
QUARTER	Einheit für Quartale
YEAR	Einheit für Jahre

Tabelle 16.28 Datumseinheiten zum Addieren und Subtrahieren von Intervallen

Mit diesem Wissen können Sie nun ein kleines Beispiel bearbeiten. Wir addieren zum Datumswert 2024-03-01 zehn Tage und nutzen wieder die Kurzform einer **SELECT**-Anweisung:

```
SELECT DATEADD(day,10,'2024-03-01T00:00:00.000') AS datumaddition;
```

Listing 16.32 Ein Intervall zu einem Datumswert addieren

Vielleicht wundern Sie sich darüber, dass hinter dem Datum ein T angegeben wurde, auf das eine Zeitangabe folgt. Das T bedeutet Time und gibt eben an, dass nun eine Zeitangabe folgt. Das ist erforderlich, wenn Sie der Funktion **DATEADD** ein Datum gemäß dem Format der Norm ISO 8601 übergeben.

In Tabelle 16.29 sehen Sie das Ergebnis unserer Addition. Zum Datumswert 2024-03-01 wurde ein Intervall mit 10 Tagen addiert.

datumaddition
2024-03-11 00:00:00.000

Tabelle 16.29 Addition eines Intervalls zu einem Datum



Zusammenfassung: Mit Zeit und Datum arbeiten

SQL bietet einige Skalarfunktionen an, die es ermöglichen, Zeit- und Datumsangaben zu formatieren und zu extrahieren. Auch eine Addition oder Subtraktion mit Datums- bzw. Zeitintervallen ist möglich. Die Umsetzung ist je nach verwendetem Datenbanksystem verschieden. Zur Ermittlung von Zeit- und Datumswerten stehen folgende Skalarfunktionen für die hier besprochenen Datenbanken zur Verfügung:

- ▶ **CURRENT_DATE** – MySQL, MariaDB und PostgreSQL
- ▶ **CURRENT_TIME** – MySQL, MariaDB und PostgreSQL
- ▶ **CURRENT_TIMESTAMP** – MySQL, MariaDB, PostgreSQL und MS SQL Server

Für die Formatierung von Zeit- und Datumsangaben wird die folgende Auswahl von Skalarfunktionen angeboten:

- ▶ **DATE_FORMAT** – MySQL, MariaDB
- ▶ **TO_CHAR** – PostgreSQL
- ▶ **FORMAT** – MS SQL Server

Für die Extraktion von Zeit- und Datumsangaben stehen folgende Skalarfunktionen zur Verfügung:

- ▶ **EXTRACT** – MySQL, MariaDB und PostgreSQL
- ▶ **DATEPART** – MS SQL Server

Berechnungen können mit folgenden Skalarfunktionen bzw. Operatoren durchgeführt werden:

- ▶ **DATE_ADD** oder **DATE_SUB** – MySQL, MariaDB
- ▶ **+|- INTERVAL** – PostgreSQL
- ▶ **DATEADD** – MS SQL Server

Es gibt noch deutlich mehr Funktionen, die Sie in der Dokumentation zu Ihrer Datenbank finden.

16.5.4 Übungen

Übung 1

Ermitteln Sie in der Kurzform einer **SELECT**-Anweisung das Datum, das aus einer Addition von 10 Monaten zum Datumswert 2024.04.01 resultiert.

Übung 2

Ermitteln Sie in der Kurzform einer **SELECT**-Anweisung das Datum, das aus einer Subtraktion von 10 Monaten vom Datumswert 2024.04.01 resultiert.

Lösung zu Übung 1

```
/* MySQL- und MariaDB Variante */
SELECT
    DATE_ADD('2024-04-01',INTERVAL 10 MONTH) AS additionmonate;

/* PostgreSQL-Variante */
SELECT
    DATE '2024-04-01' + INTERVAL '10 MONTH' AS additionmonate;

/* MS SQL Server-Variante */
SELECT
    DATEADD(MONTH,10,'2024-04-01T00:00:00.000')
    AS additionmonate;
```

Listing 16.33 Monate zu einem Datum addieren

Lösung zu Übung 2

```
/* MySQL- und MariaDB-Variante */
SELECT
    DATE_SUB('2024-04-01',INTERVAL 10 MONTH)
    AS subtraktionmonate;

/* PostgreSQL-Variante */
SELECT
    DATE '2024-04-01' - INTERVAL '10 MONTH' AS subtraktionmonate;

/* MS SQL Server-Variante */
SELECT
    DATEADD(MONTH,-10,'2024-04-01T00:00:00.000')
    AS subtraktionmonate;
```

Listing 16.34 Monate von einem Datum subtrahieren

Kapitel 17

Spaltenwerte gruppieren (GROUP BY)

Gruppierungen und Funktionen, die Sie auf gruppierte Spaltenwerte anwenden können, ermöglichen vielfältige Berechnungen und Auswertungen mithilfe von *SELECT*-Anweisungen. Über eine Gruppe von Spaltenwerten können Sie z. B. Maximalwerte, Minimalwerte, Summen, Durchschnittswerte und die Anzahl der Zeilen einer Gruppe ermitteln.

SQL bietet Ihnen die **GROUP BY**-Klausel, die gleiche Werte einer Spalte zu Gruppen zusammenfasst. Betrachten Sie hierzu Abbildung 17.1.

Gruppieren von Datensätzen nach der Abteilung

name	vorname	abteilung
Nolte	Reinhard	vertrieb
Jule	Stefanie	vertrieb
Petersen	Hans	vertrieb
Stein	Rolf	controlling
Groß	Vera	controlling
Funke	Andreas	controlling
Maier	Tim	verkauf
Winter	Elise	verkauf
Sommer	Werner	verkauf
Lempe	Dirk	personal
Müller	Iris	personal
Ludwig	Heinz	personal

Gruppe vertrieb

Gruppe controlling

Gruppe verkauf

Gruppe personal

Abbildung 17.1 Die Anzahl der Mitarbeiter pro Abteilung ermitteln

Hier sind jeweils einer Auswahl von Mitarbeitern Abteilungen zugeordnet. Die Werte in der Spalte *abteilung* eignen sich hervorragend, um vier Gruppierungen zu bilden. Dies sind die Abteilungen Personal, Vertrieb, Controlling und Verkauf. Die Gruppierungen wiederum enthalten Zeilen, über die Sie eine Aggregatfunktion ausführen können. Mit einer Aggregatfunktion können Sie über eine Gruppe von Spaltenwerten

die Anzahl der Mitarbeiter ermitteln, die in dieser Gruppe enthalten sind. Als Ergebnis würden Sie dann beispielsweise eine Anzahl von drei Mitarbeitern für die Gruppe Vertrieb zurückerhalten. In tabellarischer Form sähe dies so aus wie in [Tabelle 17.1](#):

Gruppierung nach Abteilung	Anzahl der Mitarbeiter
Personal	2
Vertrieb	3
Controlling	2
Verkauf	3

Tabelle 17.1 Gruppierung über die Spalte »abteilung« mit einer Auswertung der Anzahl der Mitarbeiter pro Gruppierung

In [Tabelle 17.1](#) wurden die Mitarbeiter pro Gruppe gezählt, und diese gruppenspezifische Information wurde jeweils hinter der Gruppe notiert. Somit haben wir hier eine klassische *Aggregationsoperation* durchgeführt. Der Begriff »Aggregation« stammt aus dem Lateinischen und besagt so viel wie »Anhäufung«. Bezogen auf Tabellen werden gleichartige Werte zu Gruppen zusammengefasst und mit Aggregationsoperationen ausgewertet.

Die fünf wichtigsten Aggregatfunktionen, die Sie auf Gruppierungen von Spaltenwerten einer oder mehrerer Spalten anwenden können, sind:

- ▶ **COUNT**(spaltenwert) zählt die Spaltenwerte einer Gruppe.
- ▶ **SUM**(spaltenwert) addiert numerische Spaltenwerte einer Gruppe.
- ▶ **MIN**(spaltenwert) ermittelt den minimalen Wert einer Gruppe.
- ▶ **MAX**(spaltenwert) ermittelt den maximalen Wert einer Gruppe.
- ▶ **AVG**(spaltenwert) ermittelt den arithmetischen Durchschnittswert einer Gruppe.

Zusammenfassend können wir also feststellen, dass Gruppierungen und Aggregatfunktionen, mit denen sich Gruppierungen auswerten lassen, in der Regel im Zusammenspiel zu betrachten sind. Wie Sie Gruppierungen in SQL realisieren, schauen wir uns in [Abschnitt 17.1](#) an. Dort ermitteln wir die Anzahl der Bonuszahlungen pro Mitarbeiter mithilfe einer **GROUP BY**-Klausel und der **COUNT**-Funktion.

17.1 Die Aggregatfunktion COUNT anwenden

Bevor wir uns ans Gruppieren und die Anwendung von Aggregatfunktionen begeben, stelle ich Ihnen kurz einen Auszug der Tabelle *bonus* vor, auf der Sie im Rahmen dieses Kapitels Gruppierungen ausführen werden.

bonusid	bonuszahlung	mitarbeiterid	datum
47	9000.00	1	2018-03-01
10	1800.00	1	2018-02-01
56	11700.00	1	2018-09-01
11	3800.00	8	2018-03-01
2	7000.00	8	2018-01-01
30	3000.00	8	2018-08-01
49	2500.00	12	2018-09-01
31	4440.00	12	2018-01-01
58	17800.00	12	2018-08-01

Tabelle 17.2 Auszug aus der Tabelle »bonus«

In [Tabelle 17.2](#) finden Sie die Struktur und Beispieldatensätze, die in der Tabelle *bonus* enthalten sind. Diese verfügt über die Spalten *bonusid*, *bonuszahlung*, *mitarbeiterid* und *datum*. In der Tabelle werden Mitarbeitern Bonuszahlungen zugeordnet. Jeder Mitarbeiter kann mehrere Bonuszahlungen zu unterschiedlichen Zeitpunkten bekommen. Die Spaltenwerte der Spalte *mitarbeiterid* eignen sich also sehr gut, um Gruppen zu bilden und über andere Spaltenwerte zu aggregieren.

In [Listing 17.1](#) sehen Sie eine **SELECT**-Anweisung, in der nach den Spaltenwerten der Spalte *mitarbeiterid* gruppiert wird:

```
SELECT
    mitarbeiterid,
    COUNT(bonusid) AS anzahlbonuszahlungen
FROM
    bonus
GROUP BY
    mitarbeiterid;
```

Listing 17.1 Zählen der Bonuszahlungen pro Mitarbeiter

Hinter dem Tabellennamen der **FROM**-Klausel geben Sie die **GROUP BY**-Klausel an. Danach folgt die Bezeichnung der Spalte, deren Werte gruppiert werden sollen. Eine Gruppierung allein ist natürlich nicht sinnvoll. In der Spaltenauswahlliste sehen Sie die Funktion **COUNT**(spaltenname), die über die Werte der Spalte *bonusid* zählt und für jede Gruppierung über die Werte der Spalte *mitarbeiterid* die Anzahl der Werte der Spalte *bonusid* ermittelt.

Wie in [Tabelle 17.3](#) dargestellt, liefert Ihnen die Funktion **COUNT** die Anzahl der Bonuszahlungen pro Mitarbeiter zurück und gruppiert sie nach der Spalte *mitarbeiterid*.

mitarbeiterid	anzahlbonuszahlungen
1	7
8	7
10	1
12	7

Tabelle 17.3 Die Anzahl der Bonuszahlungen pro Mitarbeiter ermitteln

Im nächsten Beispiel wollen wir die Gesamtanzahl der Mitarbeiter, die Bonuszahlungen erhalten haben, eindeutig ermitteln. Betrachten wir in [Tabelle 17.4](#) zunächst noch einmal einen Ausschnitt der Zeilen, die in der Tabelle *bonus* vorkommen.

bonusid	bonuszahlung	mitarbeiterid	auszahlungsdatum
29	8000.00	1	2018-12-01
19	1000.00	1	2018-04-01
1	5000.00	1	2018-08-01
11	3800.00	8	2018-03-01
2	7000.00	8	2018-01-01
30	3000.00	8	2018-08-01
68	NULL	10	2020-10-11
62	45000000.00	10	2019-02-01
71	2000000.00	10	2019-04-10

Tabelle 17.4 Auszug aus der Tabelle »bonus«

Für uns sind hier insbesondere die Werte der Spalte *mitarbeiterid* von Interesse. In der Spalte sehen Sie die Werte 1, 8 und 10, die jeweils einem Mitarbeiter zugeordnet sind. Diese Mitarbeiter erhielten jeweils drei Bonuszahlungen. Wenn Sie die Aggregatfunktion **COUNT** auf die Spaltenwerte der Spalte *mitarbeiterid* anwenden, würden Sie alle Werte zählen, also auch die mehrfach vorkommenden Werte, wie das hier der Fall ist.

Wie können wir mit der Aggregatfunktion **COUNT** die Anzahl der Mitarbeiter eindeutig ermitteln, die jemals Bonuszahlungen erhalten haben? Die Antwort auf diese Frage finden Sie durch einen Vergleich. Zunächst ermitteln wir mit [Listing 17.2](#) die Anzahl der Werte, die in der Spalte *mitarbeiterid* vorkommen. Diese Ermittlung findet unabhängig davon statt, wie oft eine Mitarbeiter-ID in der Spalte *mitarbeiterid* vorkommt. Hierzu lassen wir die **GROUP BY**-Klausel weg und betrachten somit alle Zeilen der Tabelle als eine Gruppierung.

SELECT

COUNT(mitarbeiterid) **AS** anzahl derspaltenwerte

FROM

bonus;

Listing 17.2 Die Anzahl von Spaltenwerten mit der COUNT-Funktion ermitteln

Als Ergebnis erhalten wir [Tabelle 17.5](#) zurück, in der die Anzahl der Spaltenwerte aufgelistet wird, die in der Spalte *mitarbeiterid* enthalten sind. Hier sind auch die mehrfach vorkommenden Spaltenwerte enthalten.

anzahl derspaltenwerte
65

Tabelle 17.5 Die Anzahl der Werte einer Spalte ermitteln

Um die Anzahl der Mitarbeiter, die eine Bonuszahlung erhalten haben, eindeutig zu ermitteln, nutzen wir die **COUNT**-Aggregatfunktion mit dem Schlüsselwort **DISTINCT**. Mit dem Schlüsselwort **DISTINCT** teilen wir der Funktion **COUNT** mit, dass sie ausschließlich die eindeutigen Spaltenwerte zählen soll, die in der Spalte *mitarbeiterid* enthalten sind. Mehrfach vorkommende gleiche Spaltenwerte in der Spalte *mitarbeiterid* werden bei Verwendung der **DISTINCT**-Klausel während der Zählung also nicht berücksichtigt. In [Listing 17.3](#) sehen Sie die Anwendung der **DISTINCT**-Klausel in der Funktion **COUNT**:

SELECT

COUNT(**DISTINCT** mitarbeiterid) **AS** anzahl derspaltenwerte

FROM

bonus;

Listing 17.3 Die Anzahl von Spaltenwerten mit der COUNT-Funktion eindeutig ermitteln

Als Ergebnis erhalten Sie die eindeutige Anzahl der Mitarbeiter, die jemals eine Bonuszahlung erhalten haben, aufgelistet wie in [Tabelle 17.6](#). Es gibt also nur 11 Mitarbeiter, die Bonuszahlungen erhalten haben.

anzahlderspaltenwerte
11

Tabelle 17.6 Die eindeutige Anzahl von Werten einer Spalte ermitteln

Die Aggregatfunktion **COUNT** bietet uns noch eine weitere Funktionalität. Wenn Sie ihr als Parameter einen * (Asterisk) übergeben, zählt die Funktion **COUNT** lediglich die Anzahl der Zeilen innerhalb einer Gruppe. Das heißt, der Zählvorgang wird unabhängig von irgendwelchen Spaltenwerten durchgeführt. In [Listing 17.4](#) sehen Sie Anwendung des Parameters *:

```
SELECT
    mitarbeiterid,
    COUNT(*) AS anzahlrderzeilenprogruppe
FROM
    bonus
GROUP BY
    mitarbeiterid;
```

Listing 17.4 Die Anzahl der Zeilen einer Gruppe mit der COUNT(*)-Funktion ermitteln

Die **SELECT**-Abfrage gruppiert gemäß den Spaltenwerten der Spalte *mitarbeiterid*. Der Aufruf der Aggregatfunktion **COUNT(*)** liefert Ihnen als Ergebnis jeweils die Anzahl der Zeilen pro Gruppe.

In [Tabelle 17.7](#) sehen Sie das Ergebnis für die **SELECT**-Abfrage aus [Listing 17.4](#). Betrachten Sie also die Mitarbeiter-ID 1, so erhalten Sie den Wert 7 zurück, der Ihnen angibt, dass die Gruppe für die Mitarbeiter-ID 1 insgesamt sieben Zeilen enthält.

mitarbeiterid	anzahlderzeilenprogruppe
1	7
8	7
10	5
12	7

Tabelle 17.7 Ergebnis für die Ermittlung der Anzahl der Zeilen pro Gruppe

17.1.1 Übungen

Übung 1

In der Tabelle *kreditinstitut_auszug* sind sämtliche Kreditinstitute und deren Bankleitzahlen erfasst. Jeder Bankleitzahl können mehrere Kreditinstitute zugeordnet sein. Bilden Sie eine Gruppierung über die Bankleitzahlen der Kreditinstitute. Ermitteln Sie, wie viele Kreditinstitute einer Bankleitzahl zugeordnet sind.

Übung 2

In dieser Übung ermitteln Sie die Anzahl der verwalteten Kreditinstitute in der Tabelle *kreditinstitut_auszug*. Ein kleiner Tipp: Sie benötigen hier keine GROUP BY-Klausel, um die entsprechende Aggregatfunktion auszuführen.

Lösung zu Übung 1

```
SELECT
    bankleitzahl,
    COUNT(bankid) AS anzahlinstituteproblz
FROM
    kreditinstitut_auszug
GROUP BY
    bankleitzahl;
```

Listing 17.5 Eine weitere Gruppierung über Spaltenwerte bilden und diese Werte zählen

Lösung zu Übung 2

```
SELECT
    COUNT(*) AS anzahlzeilen
FROM
    kreditinstitut_auszug;
```

Listing 17.6 Die Anzahl sämtlicher Zeilen einer Tabelle ermitteln

17.2 Die Aggregatfunktion SUM anwenden

Da es sich hier um ein besonderes Programm der Bonuszahlungen handelt, sind wir auch daran interessiert, welchen Gesamtbetrag die einzelnen Mitarbeiter an Bonuszahlungen erhalten haben. Es liegt die Idee nahe, einfach über die Werte der Spalte *mitarbeiterid* zu gruppieren und eine Summe über die Werte der Spalte *bonuszahlung* der Tabelle *bonus* zu bilden. Zur Summenbildung über eine Spalte wenden Sie die Aggregatfunktion **SUM** an:

```
SELECT
    mitarbeiterid,
    SUM(bonuszahlung) AS summebonuszahlungen
FROM
    bonus
GROUP BY
    mitarbeiterid;
```

Listing 17.7 Summieren der Bonuszahlungen pro Mitarbeiter

In [Tabelle 17.8](#) sehen Sie das Ergebnis, das Ihnen die Datenbank zurückliefert. Sie hat gemäß Ihrer **SELECT**-Anweisung die Summe der Bonuszahlungen pro Mitarbeiter berechnet. Diese Berechnung erfolgte auf Basis der Gruppierung, die anhand der Werte in der Spalte *mitarbeiterid* vorgenommen wurde.

mitarbeiterid	summebonuszahlungen
1	38500.00
8	33200.00
10	45000000.00
12	42440.00

Tabelle 17.8 Ergebnis für eine Summenbildung über Spaltenwerte einer Gruppierung

In [Abschnitt 17.3](#) interessieren wir uns für die durchschnittlich geleisteten Bonuszahlungen pro Mitarbeiter. Aber zunächst haben Sie Gelegenheit, die Summenfunktion **SUM** praktisch anzuwenden.

17.2.1 Übungen

Übung 1

Ermitteln Sie die Summe der Auszahlungen für Bonuszahlungen, die jeweils an einem Tag ausgeschüttet wurden. Nutzen Sie die Werte der Spalte *auszahlungsdatum*, um entsprechende Gruppierungen zu bilden und über die Werte der Spalte *bonuszahlung* zu summieren.

Übung 2

Ermitteln Sie die Summe aller ausgezahlten Bonuszahlungen. Denken Sie daran, dass hier alle Zeilen der Tabelle *bonus* berücksichtigt werden müssen.

Lösung zu Übung 1

SELECT

SUM(bonuszahlung) AS bonuszahlungjedatum,
auszahlungsdatum

FROM

bonus

GROUP BY

auszahlungsdatum;

Listing 17.8 Über eine Gruppierung von Datumswerten summieren

Lösung zu Übung 2

SELECT

SUM(bonuszahlung) AS summeallerbonuszahlungen

FROM

bonus;

Listing 17.9 Die Summe aller Bonuszahlungen

17.3 Die Aggregatfunktion AVG anwenden

In diesem Beispiel werden wir die durchschnittlich geleisteten Bonuszahlungen pro Mitarbeiter mit der Aggregatfunktion **AVG** ermitteln. Die Bezeichnung der Funktion **AVG** stammt von dem englischen Begriff *average* (dt. »Durchschnitt«). Sehen wir uns in [Listing 17.10](#) die Anwendung dieser Funktion an:

SELECT

mitarbeiterid,
AVG(bonuszahlung) AS durchschnittbonuszahlungen

FROM

bonus

GROUP BY

mitarbeiterid;

Listing 17.10 Ermittlung der durchschnittlichen Bonuszahlungen pro Mitarbeiter

Die **SELECT**-Anweisung fragt die Tabelle *bonus* ab. Über die Spaltenwerte der Spalte *mitarbeiterid* bilden wir mit der **GROUP BY**-Klausel Gruppierungen. Bis hierhin ist alles bekannt. In der Spaltenauswahlliste finden Sie die Spalte *mitarbeiterid* und den Aufruf der Aggregatfunktion **AVG**, die für die Werte der Spalte *bonuszahlung* die durch-

schnittlich gezahlte Bonuszahlung bezogen auf einen Mitarbeiter berechnet und als Ergebnis zurückliefert.

Als Ergebnisliste erhalten Sie von der Datenbank eine Liste der durchschnittlich gezahlten Bonuszahlungen pro Mitarbeiter.

mitarbeiterid	durchschnittbonuszahlungen
1	5500.000000
8	4742.857143
10	45000000.000000
12	6062.857143

Tabelle 17.9 Ergebnis für eine Durchschnittsberechnung über Spaltenwerte einer Gruppierung



Die Aussagekraft von arithmetischen Mittelwerten

Da Sie vermutlich sehr häufig Mittelwerte auf Gruppierungen berechnen werden, sollten Sie dort besonders aufpassen. Der *arithmetische* Mittelwert ist sehr empfindlich gegenüber Ausreißern. Was ist damit gemeint? Die Spalte *bonuszahlung* enthält Werte zwischen 1.000 € und 45.000.000 €. Die meisten Werte liegen zwischen 1.000 € und 25.000 €. Drei Werte liegen im Bereich zwischen 2.000.000 € und 45.000.000 €. Wenn Sie nun mit der Funktion **AVG** einen arithmetischen Mittelwert über alle Bonuszahlungen ermitteln, wird das Ergebnis aufgrund der Ausreißer ungenau. Abhilfe könnte ein *gewichteter* Mittelwert schaffen, um ein Ergebnis zu erhalten, dass die gewünschten Informationen besser erfasst.

Grundsätzlich gilt bei allen statistischen Rechnungen, dass Sie genau hinsehen sollten, ob Ihre Ergebnisse auch das beschreiben, was Sie möchten. Fehler schleichen sich hier erfahrungsgemäß schnell ein.

17.3.1 Übungen

Übung 1

Ermitteln Sie in dieser Übung aus der Tabelle *bonus* die durchschnittlich gezahlten Bonuszahlungen, die jeweils an einem Tag ausgeschüttet wurden.

Übung 2

Ermitteln Sie den Durchschnitt aller geleisteten Bonuszahlungen, die in der Tabelle *bonus* enthalten sind.

Lösung zu Übung 1

```
SELECT
    auszahlungsdatum,
    AVG(bonuszahlung) AS zahlungprozahltag
FROM
    bonus
GROUP BY
    auszahlungsdatum;
```

Listing 17.11 Durchschnittliche Auszahlungen für Gruppierungen ermitteln

Lösung zu Übung 2

```
SELECT
    AVG(bonuszahlung) AS durchschnittbonuszahlungen
FROM
    bonus;
```

Listing 17.12 Den Gesamtdurchschnitt von Bonuszahlungen ermitteln

17.4 Die Aggregatfunktion MAX anwenden

Jetzt soll der maximale Betrag der ausgezahlten Bonuszahlungen pro Mitarbeiter ermittelt werden. Hierzu wenden wir die Aggregatfunktion **MAX** an, um aus den Werten der Spalte *bonuszahlung* über die Gruppierung der Spalte *mitarbeiterid* jeweils den Maximalwert der gezahlten Bonusleistungen je Mitarbeiter zu ermitteln:

```
SELECT
    mitarbeiterid,
    MAX(bonuszahlung) AS maxbonuszahlungen
FROM
    bonus
GROUP BY
    mitarbeiterid;
```

Listing 17.13 Ermittlung der maximalen Bonuszahlung pro Mitarbeiter

Die Datenbank liefert Ihnen eine Ergebnisliste wie in [Tabelle 17.10](#) zurück, aus der auf einen Blick hervorgeht, dass der Mitarbeiter mit der Mitarbeiter-ID 10 der Spitzenreiter ist. Aus den geleisteten Bonuszahlungen wurde hier mit der Aggregatfunktion **MAX** jeweils der Maximalwert der Bonuszahlungen für jeden Mitarbeiter ermittelt.

mitarbeiterid	maxbonuszahlungen
1	11700.00
8	14800.00
10	45000000.00
12	17800.00

Tabelle 17.10 Ergebnis für eine Maximalberechnung über Spaltenwerte einer Gruppierung

Neben der Aggregatfunktion **MAX** können Sie natürlich auch die Aggregatfunktion **MIN** ausprobieren, die Ihnen in diesem Fall die Bonuszahlung mit dem geringsten Wert pro Mitarbeiter ermitteln würde.

17.4.1 Übungen

Übung 1

Ermitteln Sie für die Tabelle *bonus* die maximal geleistete Bonuszahlung jeweils für einen Auszahlungstag.

Übung 2

Ermitteln Sie in dieser Übung den Wert der höchsten ausgezahlten Bonuszahlung.

Lösung zu Übung 1

SELECT

auszahlungsdatum, **MAX**(bonuszahlung) **AS** auszahlungmaxprotag

FROM

bonus

GROUP BY

auszahlungsdatum;

Listing 17.14 Maximale Zahlung pro Auszahlungsdatum ermitteln

Lösung zu Übung 2

SELECT

MAX(bonuszahlung) **AS** auszahlungmax

FROM

bonus;

Listing 17.15 Den Maximalwert der jemals ausgezahlten Bonuszahlungen ermitteln

17.5 NULL-Werte berücksichtigen

Es gibt zwei Stellen, an denen NULL-Werte für das Thema Gruppierungen eine Rolle spielen: erstens bei den Gruppen, die durch eine **GROUP BY**-Klausel gebildet werden, zweitens beim Aufruf von Aggregatfunktionen.

Betrachten wir als Erstes das Thema der Gruppierung selbst. Wenn nach den Werten einer Spalte gruppiert wird und hier NULL-Werte enthalten sind, so wird eine Gruppierung über diese Gruppe von Spaltenwerten gebildet. In [Tabelle 17.11](#) sehen Sie einen Auszug der Zeilen, die in der Tabelle *bonus* enthalten sind.

bonusid	bonuszahlung	mitarbeiterid	auszahlungsdatum
63	10000.00	NULL	2019-02-01
64	5000.00	NULL	2019-02-01
65	20000.00	NULL	2019-02-01
66	25000.00	NULL	2019-02-01
67	18000.00	NULL	2013-01-21

Tabelle 17.11 Auszug aus der Tabelle »bonus«

Es existieren also fünf NULL-Werte in der Spalte *mitarbeiterid*. In diesem Fall würde eine Gruppierung von NULL-Werten gebildet, über die Aggregatfunktionen auf andere Spalten angewendet werden können. Eine solche Anwendung wäre zwar möglich, aber nicht sehr sinnvoll, da keinerlei Aussage darüber erfolgt, welcher Gruppe von Spaltenwerten die Aggregation zugeordnet werden kann.

Die **SELECT**-Abfrage in [Listing 17.16](#) gruppiert nach den NULL-Werten der Spalte *mitarbeiterid*:

```
SELECT
    mitarbeiterid,
    SUM(bonuszahlung) AS summebonuszahlungen
FROM
    bonus
WHERE
    mitarbeiterid IS NULL
GROUP BY
    mitarbeiterid;
```

Listing 17.16 Eine Gruppe über Spaltenwerte bilden, die NULL sind

Als Aggregatfunktion wenden wir hier die **SUM**-Funktion auf die Spalte *bonuszahlung* an, um die Summe der Bonuszahlungen zu ermitteln, die sich aus der Gruppierung von **NULL**-Werten der Spalte *mitarbeiterid* ergibt.

Der Übersichtlichkeit halber nutzen wir in dieser Abfrage eine **WHERE**-Klausel, die ausschließlich die Zeilen filtert, deren Werte für die Spalte *mitarbeiterid* gleich **NULL** sind. In [Tabelle 17.12](#) sehen Sie das Ergebnis der Abfrage aus [Listing 17.16](#).

mitarbeiterid	summebonuszahlungen
NULL	78000.00

Tabelle 17.12 Gruppierung über Spaltenwerte, die **NULL** sind

Als Nächstes betrachten wir eine Aggregation über Spaltenwerte, die **NULL** sind. In [Tabelle 17.13](#) sehen Sie wieder einen Auszug der Zeilen, die in der Bonustabelle enthalten sind.

Wir haben bereits über die Werte der Spalte *bonuszahlung* mit der Aggregatfunktion **SUM** Summen von Gruppierungen gebildet. Wie sieht es aus, wenn wir eine Gruppierung über die Werte der Spalte *mitarbeiterid* bilden und in dieser Summe auch **NULL**-Werte enthalten sind?

bonusid	bonuszahlung	mitarbeiterid	auszahlungsdatum
62	45000000.00	10	2019-02-01
68	NULL	10	2020-10-11
69	NULL	10	2011-03-11
70	NULL	10	2018-02-20
71	2000000.00	10	2019-04-10

Tabelle 17.13 Auszug aus der Tabelle »bonus«

Die **SELECT**-Anweisung aus [Listing 17.17](#) gruppiert wieder über die Werte der Spalte *mitarbeiterid*:

```
SELECT
    SUM(bonuszahlung) AS summebonuszahlung,
    mitarbeiterid
FROM bonus
WHERE
    mitarbeiterid=10
```

GROUP BY

```
mitarbeiterid;
```

Listing 17.17 Über Spaltenwerte, die NULL sind, Aggregatfunktionen anwenden

In der Spaltenauswahlliste geben Sie zum einen die gruppierten Spaltenwerte und zum anderen das Ergebnis der Aggregatfunktion **SUM** aus. Um ausschließlich die Zeilen des Mitarbeiters mit der Mitarbeiter-ID 10 zu erhalten, nutzen wir eine **WHERE**-Klausel. **NULL**-Werte sind in diesem Fall Teil der Spaltenwerte, über die wir aggregieren.

In [Tabelle 17.14](#) sehen Sie das Ergebnis der Aggregatfunktion **SUM**. Die Funktion **SUM** ignoriert die **NULL**-Werte bei der Summenberechnung und summiert lediglich die beiden Werte 45000000 und 2000000.

summebonuszahlung	mitarbeiterid
47000000.00	10

Tabelle 17.14 Ergebnis für eine Aggregation über Spaltenwerte, die auch **NULL**-Werte enthalten

Grundsätzlich ist es gut, dass die Aggregatfunktionen **NULL**-Werte einfach ignorieren. Betrachten wir dennoch die Aggregatfunktionen **COUNT**, **SUM**, **MIN**, **MAX** und **AVG** kurz im Detail, um die Auswirkungen dieses Verhaltens zu erkennen:

- ▶ Die Funktion **COUNT** zählt nur die Werte, die nicht **NULL** sind, wenn Sie ihr eine Spalte übergeben, über die aggregiert werden soll. Dieses Verhalten kann, muss aber nicht erwünscht sein. Alternativ können Sie sich jedoch mit **COUNT(*)** behelfen, um alle Zeilen einer Gruppierung zu zählen.
- ▶ Die Funktion **SUM** bildet einfach eine Summe über die zu aggregierende Spalte. Dabei treten keine negativen Effekte auf, die das Ergebnis beeinflussen können.
- ▶ Wenn Sie einen Minimalwert oder Maximalwert mit den Funktionen **MIN** oder **MAX** ermitteln wollen, sind durch das Ignorieren der **NULL**-Werte ebenfalls keine negativen Auswirkungen zu erkennen.
- ▶ Wie sieht es aus, wenn Sie mit der **AVG**-Funktion über Spaltenwerte aggregieren, um den Durchschnittswert zu ermitteln? Die **AVG**-Funktion ignoriert wie die anderen Aggregatfunktionen **NULL**-Werte. Kommen also **NULL**-Werte in einer aggregierten Spalte vor, so werden sie nicht in den Durchschnittswert einfließen.

Um sich die Problematik zu verdeutlichen, betrachten Sie noch einmal [Tabelle 17.14](#). Hier sind lediglich die Bonuszahlungen des Mitarbeiters mit der Mitarbeiter-ID 10 aufgeführt. In der Spalte *bonuszahlung* aus [Tabelle 17.13](#) gibt es drei Fälle, denen an-

stelle eines konkreten Wertes jeweils ein **NULL**-Wert zugeordnet ist. Diese drei **NULL**-Werte würden bei der Berechnung der durchschnittlichen Bonuszahlungen für den Mitarbeiter mit der Mitarbeiter-ID 10 nicht berücksichtigt. Somit erhalten wir also kein korrektes Ergebnis für die Durchschnittsberechnung der Bonuszahlungen.

Abhilfe schafft die Verwendung der **COALESCE**-Skalarfunktion. Ihr können wir eine beliebige Anzahl von Parametern übergeben. Zurückgegeben wird der erste Nicht-**NULL**-Wert. Wenn also der Wert der Spalte *bonuszahlung* **NULL** ist, wird der Wert 0 (der erste Nicht-**NULL**-Wert) von der Skalarfunktion **COALESCE** zurückgegeben. Ist der Wert der Spalte *bonuszahlung* nicht **NULL**, so entspricht dieser Wert dem ersten Nicht-**NULL**-Wert und wird von der Skalarfunktion **COALESCE** zurückgeliefert. In [Listing 17.18](#) sehen Sie eine Möglichkeit, mit der Funktion **AVG** einen Durchschnittswert zu berechnen, der dem tatsächlichen Durchschnitt entspricht:

```
SELECT
  AVG(COALESCE(bonuszahlung,0)) AS mittelwertbonuszahlung,
  mitarbeiterid
FROM bonus
WHERE
  mitarbeiterid=10
GROUP BY
  mitarbeiterid;
```

Listing 17.18 Korrektur für eine Durchschnittsberechnung mit der Funktion **AVG**

In [Tabelle 17.15](#) sehen Sie den von der Funktion **AVG** ermittelten Durchschnittswert.

summebonuszahlung	mitarbeiterid
9400000	10

Tabelle 17.15 Ergebnis für einen Durchschnittswert mit Korrektur

Durch die Korrektur erhalten Sie jetzt einen Wert von 9400000 zurück. Würden Sie die Funktion ohne die Korrektur mittels der **COALESCE**-Skalarfunktion den Durchschnittswert der Bonuszahlungen berechnen lassen, so würden Sie den fehlerhaften Wert 23500000 zurückerhalten.



Achtung bei AVG

Aggregatfunktionen ignorieren **NULL**-Werte, auf die sie aggregieren. Das kann z. B. bei der Verwendung der **AVG**-Funktion zu unerwünschten Ergebnissen führen. Mit der **COALESCE**-Skalarfunktion lassen sich diese Probleme korrigieren.

17.5.1 Übungen

Übung 1

Wenn Sie mit einer **GROUP BY**-Klausel nach einer Spalte gruppieren und diese Spalte **NULL**-Werte enthält, wie wirken sich die **NULL**-Werte auf das Ergebnis der Gruppierung aus?

Übung 2

Wie verhalten sich Funktionen wie **SUM**, **AVG**, **MAX**, **MIN** und **COUNT**, wenn die Spaltenwerte, über die sie aggregieren, **NULL**-Werte enthalten?

Lösung zu Übung 1

Die Spaltenwerte, die **NULL** sind, werden zu einer Gruppe zusammengefasst.

Lösung zu Übung 2

Wenn mit Aggregatfunktionen über Spaltenwerte aggregiert wird und diese **NULL**-Werte umfassen, so werden die **NULL**-Werte von den Aggregatfunktionen ignoriert.

17.6 Nach aggregierten Werten einer Gruppierung filtern (HAVING)

SQL bietet Ihnen mit der **HAVING**-Klausel in Bezug auf aggregierte Werte eine spezielle Filterfunktion an. Sie können die **HAVING**-Klausel verwenden, um einen aggregierten Durchschnittswert mit einem Vergleichsoperator gegen ein Literal zu prüfen. Sehen wir uns die Verwendung der **HAVING**-Klausel anhand des Beispiels aus [Listing 17.19](#) genauer an:

SELECT

bankleitzahl, **COUNT**(bankleitzahl) **AS** anzahlkreditinstitute

FROM

kreditinstitut_auszug

GROUP BY

bankleitzahl

HAVING

COUNT(bankleitzahl) = 1;

Listing 17.19 Nach gruppierten Werten filtern

Mit der **GROUP BY**-Klausel bilden wir hier eine Gruppierung über die Spaltenwerte der Spalte *bankleitzahl*. Hinter der **GROUP BY**-Klausel folgt die **HAVING**-Klausel, die für das Filtern von gruppierten Werten zuständig ist. Hinter der **HAVING**-Klausel notieren Sie

eine Aggregatfunktion, die auf die Gruppierung angewendet wird. Den Wert wiederum, der sich aus der Aggregatfunktion ergibt, können Sie mithilfe eines Vergleichsoperators mit einem Literal oder aber dem Ergebnis einer weiteren Aggregation vergleichen. Hier wird mittels der **COUNT**-Funktion in einer Bedingung geprüft, ob der Wert, den die **COUNT**-Funktion zurückliefert, gleich 1 ist.

In diesem Beispiel ermitteln wir sämtliche Bankleitzahlen, denen nur ein Kreditinstitut zugeordnet ist. Das Ergebnis sehen Sie auszugsweise in [Tabelle 17.16](#).

bankleitzahl	anzahlkreditinstitute
10000000	1
10010010	1
10010111	1
10010222	1

Tabelle 17.16 Ein Aggregationsergebnis für eine Bedingung verwenden, die mit der **HAVING**-Klausel ausgewertet wird

Sie können eine **WHERE**-Klausel nicht zum Filtern der aggregierten Ergebnisse einer Gruppierung verwenden. Dies liegt daran, dass eine **WHERE**-Klausel nur auf Zeilenebene angewendet werden kann. Eine **WHERE**-Klausel wird also stets vor einer **GROUP BY**-Klausel ausgewertet. Eine Aggregatfunktion kann also innerhalb der **WHERE**-Klausel auf kein Ergebnis einer Gruppierung zurückgreifen, weil es schlicht und ergreifend noch nicht existiert.

Auf Zeilenebene hingegen können Sie die **WHERE**-Klausel gemeinsam mit einer **GROUP BY**-Klausel verwenden, um vor der Gruppierung mit der **GROUP BY**-Klausel die Zeilen einer Tabelle gemäß einer Bedingung zu filtern.

Schauen Sie sich hingegen die **SELECT**-Anweisung aus [Listing 17.20](#) an:

```
SELECT
    bankleitzahl, COUNT(bankleitzahl) AS anzahlkreditinstitute
FROM
    kreditinstitut_auszug
WHERE COUNT(bankleitzahl) = 1
GROUP BY
    bankleitzahl;
```

Listing 17.20 Versuch, die **WHERE**-Klausel zum Filtern von aggregierten Werten zu nutzen

Wenn Sie versuchen, diesen Aufruf auszuführen, erhalten Sie die folgende Fehlermeldung zurück, die Sie darauf hinweist, dass es nicht möglich ist, Aggregatfunktionen zum Filtern auf Zeilenebene zu verwenden:

Error Code: 1111. Invalid use of group function

Entsprechend sind sämtliche Funktionen, die auf Gruppierungen angewendet werden, nicht in der **WHERE**-Klausel zulässig, da diese ausschließlich auf Zeilenebene filtert. Dazu gehören die **SUM**-, **MIN**-, **MAX**-, **COUNT**- und **AVG**-Funktionen.

17.6.1 Übungen

Übung 1

Ermitteln Sie in der Tabelle *kreditinstitut_auszug* alle Bankleitzahlen, denen mehr als ein Kreditinstitut zugeordnet ist.

Übung 2

Warum können Sie in einer **WHERE**-Klausel keine Aggregatfunktionen zum Filtern verwenden?

Lösung zu Übung 1

SELECT

bankleitzahl, **COUNT**(bankleitzahl) **AS** anzahlkreditinstitute

FROM

kreditinstitut_auszug

GROUP BY

bankleitzahl

HAVING

COUNT(bankleitzahl) > 1;

Listing 17.21 Auflistung aller Bankleitzahlen

Lösung zu Übung 2

Mit einer **WHERE**-Klausel können nur auf Zeilenebene Datensätze gefiltert werden. Die **WHERE**-Klausel wird stets vor einer **GROUP BY**-Klausel auf Zeilenebene ausgeführt. Die Verwendung einer Aggregatfunktion in einer **WHERE**-Klausel führt zu einer Fehlermeldung.

17.7 Nach zwei oder mehr Spalten gruppieren

Bisher haben Sie Gruppierungen kennengelernt, die auf die Spaltenwerte einer Spalte angewendet wurden. Die Anzahl der zu gruppierenden Spalten hängt natürlich davon ab, was Sie vorhaben. Je nach Anforderung können Sie über zwei, drei oder mehr Spalten gruppieren. Im nächsten Beispiel werden wir eine Gruppierung über die Spaltenwerte der Spalten *ort* und *bezeichnung* bilden. Pro Ort soll hier auch die Anzahl der jeweiligen Kreditinstitute als Information hinterlegt sein. In [Tabelle 17.17](#) werden die Kreditinstitute gezeigt, die mehrfach an einem Ort vertreten sind. Das Kreditinstitut Commerzbank CC ist z. B. zweimal in Dortmund ansässig. Die HKB Bank ist hingegen zweimal in Frankfurt am Main zu finden.

ort	bezeichnung
Dortmund	Commerzbank CC
Dortmund	Commerzbank CC
Frankfurt am Main	HKB Bank
Frankfurt am Main	HKB Bank
Erbach, Donau	Raiffeisenbank Ehingen-Hochsträß
Erbach, Donau	Raiffeisenbank Ehingen-Hochsträß

Tabelle 17.17 Kreditinstitute, die mehrfach in einem Ort vertreten sind

Formulieren wir als Nächstes eine `SELECT`-Anweisung, die über zwei Spalten und deren Spaltenwerte Gruppierungen bildet. Diese Gruppierungen werden im Anschluss daran durch die Aggregatfunktion `COUNT` verwendet. In [Listing 17.22](#) sehen Sie, dass wir hinter der `GROUP BY`-Klausel nach den Spaltenwerten der Spalten *ort* und *bezeichnung* gruppieren.

```
SELECT
  ort,
  bezeichnung,
  COUNT(*) AS anzahlinstitute
FROM
  kreditinstitut_auszug
GROUP BY
  ort,bezeichnung;
```

Listing 17.22 Nach zwei Spalten gruppieren

Die zu gruppierenden Spalten werden kommasepariert hinter der **GROUP BY**-Klausel angegeben. In der Spaltenauswahlliste sind die Spalten *ort*, *bezeichnung* und die Aggregatfunktion **COUNT** enthalten, die uns die Anzahl der Zeilen pro Gruppierung zurückliefert.

Das Ergebnis dieser Abfrage sehen Sie auszugsweise in Tabelle 17.18.

ort	bezeichnung	anzahlinstitute
Dortmund	Commerzbank CC	2
Frankfurt am Main	HKB Bank	2
Erbach, Donau	Raiffeisenbank Ehingen-Hochsträß	2

Tabelle 17.18 Ergebnis für eine Gruppierung über die Spaltenwerte von zwei Spalten

Es handelt sich um eine Liste von gruppierten Spaltenwerten, die über die Spalten *ort* und *bezeichnung* gebildet wurden. Pro Ort und Bezeichnung wird mit der Aggregatfunktion **COUNT** die Anzahl von Kreditinstituten gleichen Namens ermittelt.

Zusammenfassung: Spaltenwerte in einer Abfrage gruppieren



Die **GROUP BY**-Klausel schafft in SQL die Möglichkeit, Werte einer Spalte zu einer Gruppe zusammenzufassen. Das Zusammenfassen von Spaltenwerten kann auch als *Verdichtung* bezeichnet werden. Eine Gruppierung kann aus einer oder mehreren Spalten bestehen. In der Spaltenauswahlliste sind dann nur die Spalten zulässig, nach denen gruppiert wurde. Außerdem können in der Spaltenauswahlliste Aggregatfunktionen enthalten sein, die beispielsweise eine Summe über eine Spalte bilden, die nicht den Gruppierungsspalten angehört.

Folgende Auswahl von Aggregatfunktionen steht zur Aggregation von Spaltenwerten zur Verfügung:

- ▶ **MAX**
- ▶ **MIN**
- ▶ **SUM**
- ▶ **AVG**
- ▶ **COUNT**

Je nach verwendeter Datenbank können Sie auch andere Aggregatfunktionen nutzen. Schauen Sie sich dazu die Dokumentation Ihrer Datenbank an.

Die **GROUP BY**-Klausel bietet mit der **HAVING**-Klausel eine Möglichkeit, nach gruppierten Werten zu filtern. Ähnlich wie in einer **WHERE**-Klausel werden hier Vergleichsoperatoren verwendet, um Ergebnisse einer Operation, die auf eine Gruppierung angewendet wurde, mit anderen Werten zu vergleichen.

Wenn eine **WHERE**-Klausel Anwendung finden soll, ist sie immer vor der **GROUP BY**-Klausel zu notieren, da die **WHERE**-Klausel auf Zeilenebene Informationen filtert und vor der **GROUP BY**-Klausel ausgeführt wird.

Die **WHERE**-Klausel kann nicht zum Filtern von aggregierten Werten genutzt werden, da sie ausschließlich auf Zeilenebene arbeitet. Die **WHERE**-Klausel wird vor der **GROUP BY**-Klausel ausgeführt, sodass das Ergebnis einer Aggregation, die auf einer Gruppierung basiert, zu dem Zeitpunkt der Auswertung der **WHERE**-Klausel also nicht zur Verfügung steht.

17.7.1 Übungen

Übung 1

In der Tabelle *arbeitszeit_auszug* sind die Arbeitszeiten eines jeden Mitarbeiters pro Tag hinterlegt. Diese Informationen sind in den Spalten *jahr*, *monat*, *arbeitstag* und *anzahlstunden* gegliedert. Erstellen Sie einen Bericht, der auswertet, wie viele Stunden jeder Mitarbeiter pro Monat gearbeitet hat. Diese Auswertung bezieht sich immer auf das jeweilige Jahr. Formulieren Sie eine **SELECT**-Anweisung, in der Sie nach den Spalten *jahr*, *monat* und *mitarbeiterid* gruppieren und eine Summe über die Spaltenwerte *anzahlstunden* bilden, um die abgeleisteten Stunden je Mitarbeiter und Monat zu ermitteln. In die Spaltenauswahlliste übernehmen Sie die Spalten *jahr*, *monat*, *mitarbeiterid* und die Aggregatfunktion **SUM**, die über die Spaltenwerte der Spalte *anzahlstunden* aggregiert. Damit der Bericht leichter lesbar ist, nehmen Sie eine aufsteigende Sortierung gemäß den Spalten *jahr*, *monat* und *mitarbeiterid* vor.

Übung 2

Nutzen Sie Ihr Ergebnis aus Übung 1, und erweitern Sie die Abfrage um eine Filterfunktion, die auf den aggregierten Wert der Spalte *anzahlstunden* der Tabelle *arbeitszeit* Anwendung findet. Es sollen nur die aggregierten Summenergebnisse in den Bericht mit aufgenommen werden, die größer als 176 Stunden sind.

Lösung zu Übung 1

SELECT

```
jahr,
monat,
mitarbeiterid,
SUM(anzahlstunden) AS sumimestundenmonat
```

FROM

```
arbeitszeit_auszug
```

GROUP BY

jahr,monat,mitarbeiterid

ORDER BY

jahr,monat,mitarbeiterid;

Listing 17.23 Eine Gruppierung über drei Spalten bilden und die Spaltenwerte einer Spalte summieren

Lösung zu Übung 2

SELECT

jahr,
monat,
mitarbeiterid,
SUM(anzahlstunden) **AS** sumimestundenmonat

FROM

arbeitszeit_auszug

GROUP BY

jahr,monat,mitarbeiterid

HAVING

SUM(anzahlstunden)>176

ORDER BY

jahr,monat,mitarbeiterid;

Listing 17.24 Aggregierte Werte einer Gruppierung filtern

Kapitel 18

Mächtiges Werkzeug: Die Unterabfragen (Subqueries)

Ihrer Kreativität sind keine Grenzen gesetzt, wenn Sie Unterabfragen in SQL verwenden. Unterabfragen können Sie überall dort einsetzen, wo Ausdrücke genutzt werden können. Sie werden jetzt lernen, wie Sie Unterabfragen in Abfragen formulieren und diese in Beziehung zueinander setzen.

Unterabfragen, auch *Subqueries* genannt, bieten in SQL eine hohe Flexibilität bei der Lösungsfindung für komplexe Anforderungen. Aber klären wir zunächst, womit wir es überhaupt zu tun haben.

Unterabfragen sind **SELECT**-Abfragen, die innerhalb einer übergeordneten **SELECT**-Abfrage Verwendung finden. Sie verwenden also innerhalb einer **SELECT**-Anweisung eine weitere **SELECT**-Abfrage. Solche Subqueries werden angewendet, wenn Sie innerhalb einer Abfrage Informationen oder Datensätze benötigen, die Ihnen eine andere **SELECT**-Abfrage liefern kann.

Die Anwendung von Unterabfragen ist nicht auf die **SELECT**-Anweisung beschränkt: Unterabfragen können Sie auch in **UPDATE**-, **DELETE**- oder **INSERT**-Anweisungen einsetzen. Im Rahmen dieses Einstiegs in SQL befassen wir uns mit Unterabfragen, die in übergeordneten Abfragen Verwendung finden.

Unterabfragen werden in zwei Kategorien unterteilt. In die erste Kategorie fallen die Unterabfragen, die nicht in Beziehung zur übergeordneten Abfrage stehen. Diese Unterabfragen werden als *nicht korrelierende Abfragen* bezeichnet. Zur zweiten Kategorie von Unterabfragen zählen diejenigen, die in Beziehung zu einer übergeordneten Abfrage stehen. Wie Sie nicht korrelierende und korrelierende Unterabfragen verwenden, zeige ich Ihnen in den folgenden Abschnitten.

In [Abschnitt 18.1](#) geht es zunächst um die Unterabfragen, die in einer Korrelation zueinander stehen. In [Abschnitt 18.2](#) lernen Sie dann Unterabfragen kennen, die nicht in Korrelation zur übergeordneten Abfrage stehen. In [Abschnitt 18.3](#) wenden wir die Vergleichsoperatoren **ANY**, **SOME** und **ALL** auf Unterabfragen an, und im letzten Abschnitt prüfen wir mit **EXISTS**, ob eine Unterabfrage Zeilen zurückliefert.

18.1 Unterabfragen, die in Korrelation zueinander stehen

In diesem Abschnitt werden wir Unterabfragen behandeln, die in Korrelation zu einer übergeordneten Abfrage stehen.

Was heißt in diesem Zusammenhang *Korrelation*? Mit Korrelation ist gemeint, dass Spaltenwerte in einer Unterabfrage mit Spaltenwerten der übergeordneten Abfrage verglichen werden. Der Vergleich von Spaltenwerten erfolgt also, indem Spaltenwerte der Unterabfrage in Beziehung zu Spaltenwerten der übergeordneten Abfrage gesetzt werden. Sehen wir uns dazu ein Beispiel an.

Historisch bedingt existiert in der Tabelle *mitarbeiter* eine Spalte *bonus*, in der Zahlungen an die Mitarbeiter hinterlegt sind. Sehen wir uns zunächst die **SELECT**-Abfrage in [Listing 18.1](#) an, mit der wir die Spalten *mitarbeiterid*, *name*, *vorname* und *bonus* aus der Tabelle *mitarbeiter* abfragen:

```
SELECT
    mitarbeiterid,
    name,
    vorname,
    bonus
FROM
    mitarbeiter
WHERE
    mitarbeiterid IN (38,45);
```

Listing 18.1 Bonuszahlungen ermitteln, die in der Tabelle »mitarbeiter« hinterlegt sind

Eingeschränkt wird die **SELECT**-Abfrage durch eine **WHERE**-Klausel, in der mit einem **IN**-Operator geprüft wird, ob die Werte der Spalte *mitarbeiterid* in einer kommaseparierten Liste von numerischen Literalen enthalten sind. In diesem Fall handelt es sich um die Mitarbeiter-IDs 38 und 45.

Als Ergebnis erhalten Sie für die **SELECT**-Abfrage aus [Listing 18.1](#) die Mitarbeiter mit entsprechenden IDs zurück, wie Sie in [Tabelle 18.1](#) sehen.

mitarbeiterid	name	vorname	bonus
38	Mey	Lisa	50000.00
45	Sternental	Marlene	10000.00

Tabelle 18.1 Bonuszahlungen der Mitarbeiter mit den IDs 38 und 45

Der Mitarbeiterin Lisa Mey ist also eine Bonuszahlung in Höhe von 50.000 € zugeordnet, während die Mitarbeiterin Marlene Sternental 10.000 € als Bonus erhalten

hat. Dieses Ergebnis werden wir nutzen, um das Ergebnis der **SELECT**-Abfrage aus [Listing 18.2](#) nachvollziehen zu können:

SELECT

m.mitarbeiterid,m.name,m.vorname,m.bonus

FROM

mitarbeiter m

WHERE

m.bonus>(

SELECT MAX(bonuszahlung) **FROM** bonus b

WHERE m.mitarbeiterid=b.mitarbeiterid

GROUP BY mitarbeiterid

);

Listing 18.2 Korrelierende Unterabfragen in einer **WHERE**-Klausel verwenden

Betrachten wir als Erstes die Unterabfrage, die in der **WHERE**-Klausel angewendet wird. Sie gruppiert nach den Spaltenwerten der Spalte *mitarbeiterid* der Tabelle *bonus* und ermittelt pro Gruppierung den Maximalwert der gezahlten Boni.

Kommen wir jetzt zum spannenden Teil, nämlich zu der **WHERE**-Klausel in der Unterabfrage. Hier wird jeweils nur die Zeile gefiltert, in der die Spaltenwerte der Spalte *mitarbeiterid* der Tabelle *mitarbeiter* und der Tabelle *bonus* gleich sind. Die Bedingung vergleicht also die Werte der Spalte *mitarbeiterid* der übergeordneten Abfrage mit den Werten der Spalte *mitarbeiterid* der Unterabfrage. Der Bezug der beiden Spalten zu den Tabellen wird über Tabellenaliasse realisiert.

Sehen wir uns als Nächstes die **WHERE**-Klausel der übergeordneten Abfrage an. Diese Klausel prüft, ob die Werte der Spalte *bonus*, die ja in der Tabelle *mitarbeiter* vorhanden sind, größer sind als das Ergebnis der Unterabfrage. Die **WHERE**-Klausel der übergeordneten Abfrage vergleicht also in der Bedingung exakt einen Spaltenwert mit exakt einem Spaltenwert aus der Unterabfrage. Demzufolge darf die Unterabfrage an dieser Stelle nur einen einzigen Wert zurückgeben. Da wir in der Unterabfrage nach den Werten der Spalte *mitarbeiterid* gruppieren und über die Gruppe pro Mitarbeiter den Maximalwert der Spalte *bonuszahlung* ermitteln, ist dies sichergestellt.

Jetzt stellen wir uns natürlich die Frage, wie sichergestellt ist, dass in der **WHERE**-Klausel der Wert der Spalte *m.bonus* der übergeordneten Abfrage mit dem richtigen Spaltenwert der Unterabfrage verglichen wird. Unser Ziel ist ja, jeweils für einen Mitarbeiter die Bonuszahlungen aus den Tabellen *mitarbeiter* und *bonus* zu vergleichen. Hierzu sehen wir uns die **WHERE**-Klausel der Unterabfrage in [Listing 18.3](#) genauer an:

WHERE m.mitarbeiterid=b.mitarbeiterid

Listing 18.3 **WHERE**-Klausel zum Verbinden von Zeilen

Der Alias *m* ist der Tabelle *mitarbeiter* der übergeordneten Abfrage zugeordnet. Der Alias *b* ist der Tabelle *bonus* der Unterabfrage zugeordnet. Mit den beiden Tabellenaliasen *m* und *b* können wir in der **WHERE**-Klausel der Unterabfrage die Werte der Spalten *mitarbeiterid*, die ja in beiden Tabellen vorkommen, auf Gleichheit prüfen. Wenn die Werte der Spalten *mitarbeiterid* gleich sind, werden schon jetzt alle Zeilen der Tabelle *bonus* auf die Zeilen der Mitarbeiter reduziert, die die gleiche Mitarbeiter-ID wie in der Abfrage haben.

Um das besser zu verstehen, stellen Sie sich einfach vor, dass die Datenbank zunächst eine Zeile aus der übergeordneten Abfrage ermittelt. Der Wert der Spalte *mitarbeiterid* ist somit schon einmal bekannt.

Im Anschluss schnappt sich die Datenbank die Zeilen der Unterabfrage, in denen der Wert der Spalte *mitarbeiterid* der übergeordneten Abfrage enthalten ist, und vergleicht sie in der **WHERE**-Klausel mit den Werten der Spalte *mitarbeiterid* der Unterabfrage.



Aufruf von korrelierenden Unterabfragen

Korrelierende Unterabfragen werden für jede Zeile der übergeordneten Abfrage ausgeführt. Je nachdem, wie viele Zeilen eine Tabelle enthält, die in einer übergeordneten **SELECT**-Anweisung abgefragt wird, kann dies sehr viel Rechenleistung und Arbeitsspeicher in Anspruch nehmen.

In [Abschnitt 18.2](#) erfahren Sie, wie Sie Unterabfragen formulieren, die nicht korrelieren. Nicht korrelierende Abfragen können alternativ verwendet werden, um z. B. zunächst eine Aggregatfunktion auf eine Gruppierung anzuwenden und die daraus resultierenden Zeilen der Ergebnistabelle mit einem **JOIN** zu verbinden.

Dann wird gruppiert und das Ergebnis zurückgeliefert. Das Ergebnis der Gruppierung wird dann in der **WHERE**-Klausel der übergeordneten Abfrage ausgewertet, um zu ermitteln, ob in der Tabelle *mitarbeiter* für einen Mitarbeiter Bonuszahlungen enthalten sind, die größer sind als die maximale Bonuszahlung für einen Mitarbeiter, der in der Tabelle *bonus* ermittelt werden konnte. In [Tabelle 18.2](#) sehen Sie das Ergebnis der Abfrage aus [Listing 18.2](#).

mitarbeiterid	name	vorname	bonus
38	Mey	Lisa	50000.00
45	Sternental	Marlene	10000.00

Tabelle 18.2 Ergebnis einer Abfrage, die in Korrelation zu einer Unterabfrage steht

Es handelt sich exakt um das gleiche Ergebnis wie in [Tabelle 18.1](#). Die Mitarbeiterinnen Lisa Mey und Marlene Sternental gehören zu denjenigen Mitarbeitern, die in der

Vergangenheit eine höhere Bonuszahlung erhalten haben als die aktuell in der Tabelle *bonus* verwalteten Bonuszahlungen.

Unterabfragen, die in Korrelation zu einer übergeordneten Abfrage stehen, sind nicht trivial. Sehen wir uns daher in [Listing 18.4](#) ein weiteres Beispiel an:

```
SELECT
(
  SELECT anrede FROM anrede a
  WHERE m.geschlecht=a.geschlecht
) AS anrede,
name,
vorname
FROM mitarbeiter m;
```

Listing 18.4 Eine korrelierte Unterabfrage in der Spaltenauswahlliste

In der Spaltenauswahlliste sehen Sie zunächst eine Unterabfrage. Außerdem werden in der Spaltenauswahlliste die Spalten *name* und *vorname* abgefragt. Die Tabelle *geschlecht* enthält die Spalten *anrede* und *geschlecht*. Die Tabelle enthält zwei Zeilen. Die eine Zeile ordnet der Abkürzung *m* die Anrede Herr zu, und die andere ordnet der Abkürzung *w* die Anrede Frau zu. In der Tabelle *mitarbeiter* wiederum ist die Spalte *geschlecht* ebenfalls vorhanden. Hier sind den Mitarbeitern ebenfalls die Buchstaben *m* und *w* je nach Geschlecht zugeordnet.

In der Spaltenauswahlliste der Unterabfrage fragen Sie die Spaltenwerte der Spalte *anrede* ab. Um auch hier die richtige Anrede abzufragen, prüfen Sie zuvor in einer **WHERE**-Klausel, ob die Werte der Spalten *geschlecht* der übergeordneten Abfrage denen der Spalte *geschlecht* der Unterabfrage entsprechen.

Die korrelierende Gleichheitsprüfung realisieren Sie wieder über Tabellenaliasse. Mit der Bedingung *m.geschlecht=a.geschlecht* verbinden Sie also die Zeilen der Unterabfrage mit den Zeilen der übergeordneten Abfrage.

Die Unterabfrage liefert für jede Zeile der übergeordneten Abfrage nur einen Wert zurück. Alles andere (z. B. mehrere Zeilen oder mehrere Spalten) ist in der Spaltenauswahlliste auch nicht zulässig.

Das Ergebnis der Abfrage sehen Sie in [Tabelle 18.3](#).

anrede	name	vorname
Herr	Müller	Ralf
Frau	Schneider	Petra

Tabelle 18.3 Ergebnis für eine Abfrage mit einer Unterabfrage in der Spaltenauswahlliste

anrede	name	vorname
Herr	Klein	Thomas
Frau	Lang	Ute

Tabelle 18.3 Ergebnis für eine Abfrage mit einer Unterabfrage in der Spaltenauswahlliste

Die korrelierende Unterabfrage gibt jeweils die Anrede zurück. Sie hängt vom Ergebnis der korrelierten Bedingung in der **WHERE**-Klausel ab. Sie sehen also, dass Sie auch in einer Spaltenauswahlliste eine korrelierende Unterabfrage nutzen können.

In der Praxis würden Sie wohl einen einfachen **INNER JOIN**-Aufruf verwenden, um die Zeilen der beiden Tabellen *anrede* und *mitarbeiter* miteinander zu verbinden. Aber viele Wege führen ja nach Rom, und Sie sollten üben, wie Sie korrelierende Unterabfragen benutzen.



Zusammenfassung: Korrelierende Unterabfragen

Unterabfragen stehen in Korrelation zu einer übergeordneten Abfrage, wenn Spaltenwerte einer Unterabfrage mit Spaltenwerten einer übergeordneten Abfrage verglichen werden, um eine Beziehung zwischen den Abfragen herzustellen. Ein Vergleich dieser Art wird in der Regel in der **WHERE**-Klausel vorgenommen.

Um Spalten aus Abfragen und Unterabfragen vergleichen zu können, ist es sinnvoll, Tabellenaliasse zu verwenden. Alternativ können Sie natürlich auch die Tabellennamen verwenden, um die jeweiligen Spalten einer Tabelle eindeutig zu identifizieren.

Korrelierende Unterabfragen werden für jede Zeile der übergeordneten Abfrage ausgeführt. Das kann allerdings zu Problemen führen, wenn Sie sehr große Datenbestände abfragen.

18.1.1 Übungen

Übung 1

In dieser Übung sollen die Namen und Vornamen der Mitarbeiter ermittelt werden, die mindestens eine Sachprämie erhalten haben. In [Tabelle 18.4](#) sehen Sie die Struktur und Beispieldatensätze der Tabelle *sachpraemie*.

Formulieren Sie eine **SELECT**-Abfrage, in der die Spalten *name* und *vorname* der Tabelle *mitarbeiter* abgefragt werden.

Ergänzen Sie die **SELECT**-Anweisung um eine **WHERE**-Klausel, die in der Bedingung das Ergebnis einer korrelierten Unterabfrage auswertet. Sie soll die Bedingung auswerten, dass die Anzahl der Sachprämien größer als null ist. So listen Sie nur die Mitarbeiter auf, die mindestens eine Sachprämie erhalten haben.

praemieid	praemie	grund	preis	mitarbeiterid
1	Goldene Uhr Typ 1	40 Jahre	500.00	5
4	Goldene Uhr Typ 1	40 Jahre	500.00	10
7	Goldene Uhr Typ 4	10 Jahre	178.00	8

Tabelle 18.4 Die Tabelle »sachpraemie«

Die Korrelation zwischen der Unterabfrage und der übergeordneten Abfrage stellen Sie über die Spalte *mitarbeiterid* her, die in beiden Tabellen vorhanden ist. Ein kleiner Hinweis an dieser Stelle sei erlaubt: Für die Lösung benötigen Sie keine **GROUP BY**-Klausel in der Unterabfrage.

Übung 2

Formulieren Sie eine **SELECT**-Anweisung, in der Sie die Spalten *name* und *vorname* aus der Tabelle *mitarbeiter* abfragen.

In der Spaltenauswahlliste soll auch die Summe der Preise der Sachprämien enthalten sein, bezogen auf den jeweiligen Mitarbeiter. Um diese Anforderung umzusetzen, formulieren Sie eine Unterabfrage in der Spaltenauswahlliste. In der Unterabfrage ermitteln Sie die Summe der Preise anhand einer Gruppierung der Spalte *mitarbeiterid*.

In der Unterabfrage stellen Sie vor der Gruppierung mit einer **WHERE**-Klausel sicher, dass vor der Aggregation nur die Sachprämien ermittelt werden, deren Werte für die Spalten *mitarbeiterid* aus der übergeordneten Abfrage und der Unterabfrage gleich sind.

Lösung zu Übung 1

SELECT

name,
vorname

FROM

mitarbeiter m

WHERE

0<(

SELECT

COUNT(*)

FROM

sachpraemie s

```
WHERE
    m.mitarbeiterid=s.mitarbeiterid
);
```

Listing 18.5 In einer WHERE-Klausel das Ergebnis einer Unterabfrage korrelierend für einen Vergleich verwenden

Lösung zu Übung 2

```
SELECT
    name,
    vorname,
    (
        SELECT
            SUM(preis)
        FROM
            sachpraemie s
        WHERE
            s.mitarbeiterid=b.mitarbeiterid
        GROUP BY mitarbeiterid
    ) AS summepreise
FROM
    mitarbeiter b;
```

Listing 18.6 Eine Unterabfrage korrelierend in der Spaltenauswahlliste anwenden

18.2 Unterabfragen, die nicht in Korrelation zueinander stehen

In Kapitel 17, »Spaltenwerte gruppieren (GROUP BY)«, haben Sie erfahren, wie Sie Gruppierungen bilden. In Listing 18.7 werden alle Mitarbeiter ermittelt, die jemals eine Bonuszahlung erhalten haben. Hierzu wird nach den Werten der Spalte *mitarbeiterid* gruppiert, um die Werte der Spalte *mitarbeiterid* der Tabelle *bonus* eindeutig zurückzuerhalten:

```
SELECT
    mitarbeiterid
FROM
    bonus
GROUP BY
    mitarbeiterid;
```

Listing 18.7 Ermittlung der maximalen Bonuszahlung pro Mitarbeiter

Die Abfrage aus [Listing 18.7](#) liefert Ihnen ein Ergebnis zurück, wie es auszugsweise in [Tabelle 18.5](#) dargestellt ist. Beachten Sie, dass die Abfrage eine Spalte mit mehreren Werten zurückliefert.

mitarbeiterid
1
8
10
12

Tabelle 18.5 Ermittlung der Schlüsselwerte der Mitarbeiter, die Bonuszahlungen erhalten haben

Das Ergebnis aus [Tabelle 18.5](#) ist nicht sehr aussagekräftig. Hier sind lediglich die Mitarbeiter-IDs enthalten. Um diese Information anzureichern, formulieren wir eine **SELECT**-Anweisung, in der die Namen und Vornamen der Tabelle *mitarbeiter* abgefragt werden. Um an die Mitarbeiter zu kommen, die eine Bonuszahlung erhalten haben, ergänzen wir die **SELECT**-Anweisung um eine **WHERE**-Klausel, in der die Abfrage aus [Listing 18.7](#) als Unterabfrage Verwendung findet.

Betrachten wir also [Listing 18.8](#):

```
SELECT name,vorname FROM mitarbeiter
WHERE
  mitarbeiterid IN
  (
    SELECT mitarbeiterid FROM bonus
    GROUP BY
      mitarbeiterid
  );
```

Listing 18.8 Eine Unterabfrage in einer WHERE-Klausel verwenden

In der **WHERE**-Klausel wenden wir den **IN**-Operator an, um die Werte der Spalte *mitarbeiterid* auf eine Mitgliedschaft in einer Ergebnismenge von Werten zu prüfen. Den **IN**-Operator haben Sie bereits in [Abschnitt 2.2](#) kennengelernt. Sie haben allerdings auf Mitgliedschaft in einer kommaseparierten Liste von Werten geprüft, die von einer Klammer umschlossen war. In diesem Fall befindet sich innerhalb einer Klammer eine weitere **SELECT**-Abfrage, mit der wir die Werte der Spalte *mitarbeiterid* der Tabelle *bonus* abfragen. In der Unterabfrage wird in der **GROUP BY**-Klausel genau nach diesen Spaltenwerten gruppiert, sodass Sie sie eindeutig zurückerhalten. Das heißt in die-

sem Fall, dass der **IN**-Operator die Werte der Spalte *mitarbeiterid* der übergeordneten Abfrage auf Mitgliedschaft in den Spaltenwerten prüft, die uns die Unterabfrage zurückliefert.

An diesem Beispiel sehen Sie, dass eine Unterabfrage eine Spalte mit mehreren Werten als Ergebnis zurückliefern kann. Außerdem ist die Unterabfrage von einer Klammer umschlossen. In diesem Fall ist das auch notwendig, da wir mit einem **IN**-Operator auf Mitgliedschaft in einer Menge von Spaltenwerten prüfen. Unterabfragen können wie gewöhnliche Abfragen folgende Ergebnisse zurückgeben:

- ▶ einen Wert einer Spalte (oder einen Wert in einer Zeile)
- ▶ mehrere Werte einer Spalte (siehe Beispiel)
- ▶ Zeilen und Spalten (eine Ergebnistabelle)

Dies ist wichtig, weil Sie je nachdem, in welchem Zusammenhang Sie eine Unterabfrage verwenden, wissen müssen, welche Ergebnisse die übergeordnete Abfrage auswertet.

Unterabfragen, die einen Spaltenwert einer Zeile zurückgeben, werden als *Skalarabfragen* bezeichnet.



Aufruf von nicht korrelierenden Unterabfragen

Nicht korrelierende Unterabfragen werden nur einmal aufgerufen. Die übergeordnete Abfrage verwendet die Ergebnistabelle zum Beispiel, um mit einem **IN**-Operator auf Mengenmitgliedschaft zu prüfen.

Eine nicht korrelierende Abfrage ist also bei Weitem nicht so rechenintensiv wie eine korrelierende Abfrage.

Das Ergebnis der Abfrage ist auszugsweise in Tabelle 18.6 zu sehen.

name	vorname
Müller	Ralf
Müller	Iris
Klarfeld	Simone
Funke	Doris

Tabelle 18.6 Mitarbeiter, die eine Bonuszahlung erhalten haben

In der Ergebnistabelle sind nur die Mitarbeiter enthalten, die die Mitgliedschaftsprüfung der **IN**-Klausel bestanden haben. Sie erhalten also aus der Tabelle *mitarbeiter* die Angestellten, die eine Bonuszahlung erhalten haben.

Das Ergebnis ist schon aussagekräftiger. Wie sieht es aus, wenn Sie jeweils den Maximalwert einer Bonuszahlung pro Mitarbeiter ermitteln wollen? Unser Ziel ist es also, die Informationen der Mitarbeiter und die Aggregation des Maximalwerts einer Gruppierung auszugeben.

Um an diese Informationen zu kommen, verbinden wir die Zeilen der Tabelle *mitarbeiter* mit dem Ergebnis einer Unterabfrage. Sehen wir uns dazu [Listing 18.9](#) an:

```
SELECT m.name,m.vorname,b.maxbonuszahlung
FROM mitarbeiter m
INNER JOIN
(
  SELECT mitarbeiterid,MAX(bonuszahlung) AS maxbonuszahlung
  FROM bonus
  GROUP BY mitarbeiterid
) b
ON m.mitarbeiterid=b.mitarbeiterid;
```

Listing 18.9 Tabellen mit den Ergebniszeilen einer Unterabfrage verbinden (INNER JOIN)

Hinter der **INNER JOIN**-Klausel wird also anstelle einer Tabelle eine Unterabfrage notiert, die in einfachen Klammern steht. Sehen wir uns als Erstes an, welche Tabelle wie abgefragt wird. In der Unterabfrage wird die Tabelle *bonus* verwendet, um nach den Werten der Spalte *mitarbeiterid* zu gruppieren und die Werte der Spalte *mitarbeiterid* sowie das Ergebnis der Aggregatfunktion **MAX** bezogen auf die Gruppierung zurückzuliefern. Hinter der schließenden Klammer wird der Unterabfrage ein Alias zugeordnet.

Beachten Sie, dass die Unterabfrage Zeilen mit zwei Spalten und mehreren Zeilen zurückliefert. In diesem Beispiel haben wir nur die Spalte *b.maxbonuszahlung* in die Spaltenauswahlliste mit aufgenommen. Die Spalte *b.mitarbeiterid* wird hier in der **ON**-Klausel mit der Spalte *m.mitarbeiterid* verglichen, um die Zeilen bei Übereinstimmung miteinander zu verbinden.

In [Tabelle 18.7](#) sehen Sie das Ergebnis der Abfrage aus [Listing 18.9](#). Die Zeilen der Tabelle *mitarbeiter* wurden über die Schlüsselwerte der Spalte *mitarbeiterid* mit den Zeilen der Unterabfrage verbunden ausgegeben.

name	vorname	maxbonuszahlung
Müller	Ralf	11700.00
Müller	Iris	14800.00

Tabelle 18.7 Unterabfrage mit einem JOIN verbinden

name	vorname	maxbonuszahlung
Klarfeld	Simone	45000000.00
Funke	Doris	17800.00

Tabelle 18.7 Unterabfrage mit einem JOIN verbinden (Forts.)



Zusammenfassung: Nicht korrelierende Unterabfragen

Unterabfragen, die nicht in Beziehung zu einer übergeordneten Abfrage stehen, werden als *nicht korrelierende Unterabfragen* bezeichnet.

Diese Art von Unterabfragen wird sehr häufig verwendet, um in einer **WHERE**-Klausel einer übergeordneten Abfrage auf das Ergebnis einer Unterabfrage zu prüfen, aus der ein oder mehrere Spaltenwerte zurückgeliefert werden, die das Ergebnis einer Gruppierung sind.

Ein weiterer Anwendungsbereich von nicht korrelierenden Abfragen ist das Verbinden von Zeilen einer Tabelle mit den Ergebniszeilen einer Unterabfrage. Auch in diesem Fall enthält die Unterabfrage häufig Resultate aus Gruppierungen, die mit einem **JOIN** um weitere Informationen angereichert werden können.

Es wird zwischen folgenden Unterabfragen unterschieden:

- ▶ Abfragen, die einen Spaltenwert zurückliefern
- ▶ Abfragen, die mehrere Werte einer Spalte zurückliefern
- ▶ Abfragen, die mehrere Spaltenwerte aus mehreren Zeilen zurückliefern

Diese Differenzierung ist notwendig, um das Ergebnis von Unterabfragen bestimmen zu können.

Wird der **IN**-Operator in einer **WHERE**-Klausel einer übergeordneten Abfrage verwendet, so ist es zulässig, dass eine Unterabfrage einen oder mehrere Spaltenwerte zurückliefert.

Wird eine Unterabfrage innerhalb eines **JOINS** genutzt, so ist es zulässig, dass sie mehrere Spalten und mehrere Zeilen als Ergebnis zurückliefert.

Wird hingegen in einer **WHERE**-Klausel einer übergeordneten Abfrage mit einer Unterabfrage auf einen Spaltenwert einer der übergeordneten Abfragen geprüft, so darf die Unterabfrage nur einen Spaltenwert zurückgeben.

Eine nicht korrelierende Abfrage wird nur einmal ausgeführt. Die Ergebnistabelle kann dann von übergeordneten Abfragen verwendet werden. Nicht korrelierende Abfragen sind also bei Weitem nicht so leistungshungrig wie korrelierende Abfragen.

18.2.1 Übungen

Übung 1

In der Tabelle *sachpraemie* sind ausgewählten Mitarbeitern Sachprämien zugeordnet. In Tabelle 18.8 sehen Sie die relevanten Informationen.

praemieid	praemie	grund	preis	mitarbeiterid
1	Goldene Uhr Typ 1	40 Jahre	500.00	5
2	Goldene Uhr Typ 2	30 Jahre	300.00	5
3	Goldene Uhr Typ 3	20 Jahre	200.00	5
4	Goldene Uhr Typ 1	40 Jahre	500.00	10

Tabelle 18.8 Prämien, die Mitarbeitern zugeordnet sind

Einem Mitarbeiter können mehrere Sachprämien zugeordnet sein. In einer Abfrage sollen die Namen und Vornamen der Mitarbeiter ermittelt werden, die eine oder mehrere Sachprämien erhalten haben.

Bilden Sie zunächst in einer ersten **SELECT**-Abfrage eine Gruppierung über die Spaltenwerte der Spalte *mitarbeiterid* der Tabelle *sachpraemie*, und geben Sie die eindeutigen Spaltenwerte der Spalte *mitarbeiterid* gemäß der Gruppierung in der Spaltenauswahlliste aus.

Im nächsten Schritt formulieren Sie eine übergeordnete **SELECT**-Anweisung, um die Spalten *name* und *vorname* der Tabelle *mitarbeiter* auszuwählen. Prüfen Sie im Anschluss in einer **WHERE**-Klausel, ob die Werte der Spalte *mitarbeiterid* der Tabelle *mitarbeiter* mit denen der ersten Abfrage übereinstimmen.

Übung 2

In dieser Übung soll in einer ersten **SELECT**-Abfrage die Summe der Preise ermittelt werden, die jeweils einem Mitarbeiter in Form von Sachprämien übergeben wurden. Gruppieren Sie hierzu über die Werte der Spalte *mitarbeiterid* der Tabelle *sachpraemie*, und geben Sie in der Spaltenauswahlliste die Summe aus, die sich durch die Gruppierungen der Werte der Spalte *preis* ergibt. In der Spaltenauswahlliste sollen auch die Werte der Spalte *mitarbeiterid* enthalten sein, die zur Gruppierung verwendet werden.

In einer übergeordneten Abfrage sollen die Namen und Vornamen der Mitarbeiter ermittelt werden. Fragen Sie hierzu die Tabelle *mitarbeiter* ab, und verbinden Sie die Zeilen der Tabelle *mitarbeiter* mit den Ergebniszeilen der ersten **SELECT**-Abfrage. Somit können Sie die berechnete Summe der ersten Abfrage mit in die Spaltenauswahlliste der übergeordneten Abfrage aufnehmen.

Lösung zu Übung 1

SELECT

name,vorname

FROM

mitarbeiter

WHERE

mitarbeiterid **IN**(

SELECT mitarbeiterid

FROM sachpraemie

GROUP BY mitarbeiterid

);

Listing 18.10 Gruppierung der Mitarbeiterprämien

Lösung zu Übung 2

SELECT

m.name,

m.vorname,

p.summepreise

FROM

mitarbeiter m

INNER JOIN

(

SELECT

SUM(preis) **AS** summepreise,

mitarbeiterid

FROM

sachpraemie

GROUP BY

mitarbeiterid

) p

ON

m.mitarbeiterid=p.mitarbeiterid;

Listing 18.11 Summe der Preise

18.3 Vergleichsoperatoren auf Unterabfragen mit ANY, SOME und ALL anwenden

In Kapitel 2, »Los geht's: Die Grundfunktionen der Tabellenabfrage (SELECT)«, haben Sie im Zusammenhang mit der **WHERE**-Klausel bereits einige Vergleichsoperatoren wie

=, <>, <= und >= kennengelernt. Diese Vergleichsoperatoren können Sie auch auf Ergebnismengen von Werten einer Spalte einer Unterabfrage anwenden. Hinter dem Vergleichsoperator notieren Sie hierzu eines der Schlüsselwörter **ANY**, **SOME** oder **ALL**.

ANY, SOME und ALL

Der Spaltenwert in einer übergeordneten Abfrage wird mit einem Vergleichsoperator gegen eine Menge von Werten einer Spalte einer Unterabfrage geprüft.

Die Unterabfrage darf allerdings nur eine Spalte mit einem oder mehreren Werten zurückliefern.

Sehen Sie sich dazu in [Listing 18.12](#) ein Beispiel an, das ausschließlich die **WHERE**-Klausel unter Verwendung der Erweiterungen **ANY**, **SOME** und **ALL** betrachtet:

WHERE

```
spalte1 (=|!=|<|>|<=|>=)
      (ANY|SOME|ALL) (Unterabfrage, eine Spalte mit einem oder mehreren Werten)
```

Listing 18.12 Syntax zur Anwendung von Vergleichsoperatoren mit ANY und SOME

Sie sehen, dass unterschiedliche Vergleichsoperatoren eingesetzt werden, um die Spaltenwerte von *spalte1* mit dem Ergebnis einer Unterabfrage zu vergleichen. Hinter den Vergleichsoperatoren notieren Sie **ANY**, **SOME** oder **ALL**, um auf das Ergebnis der Unterabfrage zu prüfen.

Verwenden Sie hinter einem Vergleichsoperator **SOME** oder **ANY**, so muss die Auswertung der Bedingung gegenüber der Menge von Spaltenwerten einer Spalte mindestens einmal erfüllt sein. Die Bedingung kann natürlich auch öfter zutreffen. Die beiden Schlüsselwörter **ANY** und **SOME** sind in ihrer Bedeutung identisch. Das Ergebnis der Auswertung der Bedingung ist dann wahr, wenn die Bedingung mindestens einmal erfüllt ist.

ANY und SOME

ANY und **SOME** überprüfen, ob eine Bedingung, die auf das Ergebnis einer Unterabfrage angewendet wird, wahr ist. Die Bedingung, die einen Spaltenwert der übergeordneten Abfrage gegen eine Menge von Spaltenwerten der Unterabfrage prüft, muss lediglich einmal zutreffen, um erfüllt zu sein. **ANY** und **SOME** können synonym verwendet werden.

Soll das Ergebnis einer Unterabfrage auf gleich (=) **ANY** oder **SOME** überprüft werden, so kann auch stets der **IN**-Operator verwendet werden. Auch er prüft, ob Spaltenwerte in einer Spalte enthalten sind.

NOT IN kann hingegen alternativ für die Prüfung auf ungleich (<>) **ALL** verwendet werden.

Das probieren wir gleich einmal aus. Hier verwenden wir hinter dem Vergleichsoperator das Schlüsselwort **SOME**, um zu prüfen, ob eine Bonuszahlung mindestens einmal mit einem Spaltenwert der Spalte *umsatz* übereinstimmt. Hierzu vergleichen wir in einer **WHERE**-Klausel die Spaltenwerte der Spalte *bonus* der Tabelle *mitarbeiter* in einer übergeordneten Abfrage auf Gleichheit mit einer Menge von Werten der Spalte *umsatz* aus der gleichnamigen Tabelle *umsatz*, die von einer Unterabfrage zurückgeliefert werden:

SELECT

```
    mitarbeiterid,  
    bonuszahlung
```

FROM

```
    bonus
```

WHERE

```
    bonuszahlung = SOME (  
        SELECT umsatz FROM umsatz  
    );
```

Listing 18.13 Der Einsatz von **SOME** zum Vergleich von Spaltenwerten aus Unterabfragen

Die Datenbank liefert Ihnen eine leere Ergebnistabelle zurück, wie in [Tabelle 18.9](#) zu sehen ist. Keine Bonuszahlung eines Mitarbeiters ist identisch mit einem Umsatz in der Tabelle *umsatz*.

mitarbeiterid	bonuszahlung

Tabelle 18.9 Bedingungen mit dem **SOME**-Operator auf Ergebnisse von Unterabfragen prüfen

Als Nächstes prüfen wir, ob den Mitarbeitern Bonuszahlungen ausgezahlt wurden, die größer sind als sämtliche Quartalsumsätze. Hierzu verwenden wir den **ALL**-Operator hinter dem Vergleichsoperator der Bedingung. So wird geprüft, ob die Bedingung für alle Spaltenwerte einer Unterabfrage gilt. Die Aussage der Bedingung in der **WHERE**-Klausel ist also nur dann wahr, wenn der Vergleich für alle Spaltenwerte der Unterabfrage wahr ist.

Sehen wir uns also [Listing 18.14](#) an:

SELECT

```
    bonuszahlung,  
    mitarbeiterid
```

FROM

```
    bonus
```

WHERE

```
bonuszahlung > ALL (
  SELECT umsatz FROM umsatz
);
```

Listing 18.14 Ermittlung der Zeilen, deren Spaltenwerte größer sind als alle Spaltenwerte der Unterabfrage

Zunächst betrachten wir die Unterabfrage. Sie wählt die Spalte *umsatz* der gleichnamigen Tabelle *umsatz* aus. Jetzt sehen wir uns die **WHERE**-Klausel der übergeordneten Abfrage an.

Tabelle 18.10 zeigt das Ergebnis der Abfrage.

bonuszahlung	mitarbeiterid
30000000.00	23
45000000.00	10

Tabelle 18.10 Den ALL-Operator für eine Bedingung anwenden, die auf Spaltenwerte einer Unterabfrage angewendet wird

Jeder Spaltenwert der übergeordneten Abfrage wurde mit der Bedingung größer als (>) gegen alle Werte der Spalte *umsatz* geprüft. Lediglich zwei Zeilen, in denen die Bonuszahlungen größer sind als sämtliche Umsatzzahlen, haben es in die Ergebnisliste geschafft.

Zusammenfassung: Mit ANY, SOME und ALL Vergleiche mit Ergebnistabellen aus Unterabfragen durchführen



ANY, SOME und ALL werden verwendet, um Bedingungen zu formulieren, die auf Ergebnisse von Unterabfragen angewendet werden.

ANY und SOME sind in ihrer Funktion identisch und prüfen jeweils einen Spaltenwert einer übergeordneten Abfrage gegen Spaltenwerte einer Unterabfrage. Die Bedingung muss nur einmal zutreffen, um wahr zu sein.

Eine Bedingung, die mit ALL gegen Spaltenwerte einer Unterabfrage prüft, muss für alle Spaltenwerte der Unterabfrage erfüllt sein, um wahr zu sein.

18.3.1 Übungen

Übung 1

Für welche Arten von Vergleichen werden SOME und ANY verwendet?

Übung 2

Beschreiben Sie den Unterschied zwischen **SOME** und **ANY**.

Übung 3

Für welche Art von Vergleich wird der **ALL**-Operator verwendet?

Übung 4

In dieser Übung schauen Sie sich die Steuerklassen an, die den Mitarbeitern in der Tabelle *mitarbeiter* zugeordnet sind. Das Ziel dieser Übung ist es, die Mitarbeiter zu ermitteln, deren Steuerklasse nicht in der Tabelle *steuerklasse* hinterlegt ist.

Hierzu nutzen Sie die in Tabelle 18.11 dargestellte Tabelle *steuerklasse*. In der Spalte *steuerklassezahl* sind sechs Steuerklassen enthalten.

Formulieren Sie eine **SELECT**-Abfrage, in der Sie prüfen, welche Werte der Spalte *steuerklasse* der Tabelle *mitarbeiter* nicht mit den Werten der Spalte *steuerklassezahl* der Tabelle *steuerklasse* übereinstimmen.

steuerklasseid	steuerklasse	steuerklassezahl
1	Steuerklasse 1	1
2	Steuerklasse 2	2
3	Steuerklasse 3	3
4	Steuerklasse 4	4
5	Steuerklasse 5	5
6	Steuerklasse 6	6

Tabelle 18.11 Die Tabelle »steuerklasse«

Lösung zu Übung 1

SOME und **ANY** vergleichen einen Spaltenwert einer übergeordneten Abfrage mit einer Menge von Werten einer Spalte, die aus einer Unterabfrage stammen. Dazu wird einer der bekannten Vergleichsoperatoren eingesetzt. Die Bedingung muss nur einmal auf die Menge der Spaltenwerte der Unterabfrage zutreffen, damit sie wahr ist.

Lösung zu Übung 2

Es gibt keinen Unterschied. Die beiden Schlüsselwörter sind äquivalent zu verwenden.

Lösung zu Übung 3

Der ALL-Operator wird angewendet, wenn die Bedingung für alle Spaltenwerte der Unterabfrage zutreffen muss, um erfüllt zu sein.

Lösung zu Übung 4

SELECT

name,
vorname,
steuerklasse

FROM

mitarbeiter

WHERE

steuerklasse <> ALL (
SELECT steuerklassezahl FROM steuerklasse
);

Listing 18.15 Die Werte einer Spalte einer Unterabfrage mit ALL auf Ungleichheit prüfen

18.4 Auf die Existenz von Ergebniszeilen aus Unterabfragen prüfen (EXISTS)

Sehen wir uns einen weiteren Vergleichsoperator an, der in einer WHERE-Klausel einer übergeordneten Abfrage verwendet werden kann. Es handelt sich um den EXISTS-Vergleichsoperator, dessen Syntax Sie in [Listing 18.16](#) sehen:

WHERE EXISTS (zeileeinerunterabfrage)

Listing 18.16 Mit dieser Syntax prüfen Sie, ob Ergebniszeilen einer Unterabfrage existieren.

Wenn Sie diesen Operator verwenden, brauchen Sie keine Spalte der übergeordneten Abfrage anzugeben, da hier keine Werte verglichen werden. Es wird lediglich geprüft, ob die Unterabfrage Zeilen zurückliefert.

Wenn das der Fall ist, dann ist die Bedingung wahr. Sehen wir uns also in [Listing 18.17](#) an, wie Sie den EXISTS-Vergleichsoperator in einer WHERE-Klausel verwenden können. Ihnen fällt sicherlich als Erstes auf, dass hinter der WHERE-Klausel keine Spalte angegeben ist, deren Werte auf eine Bedingung geprüft werden. Direkt hinter der WHERE-Klausel folgt der EXISTS-Operator, der auf Vorhandensein von Ergebniszeilen einer Unterabfrage prüft.

Sehen wir uns an, wie wir diesen Vergleichsoperator in der Praxis nutzen können. In [Listing 18.17](#) soll geprüft werden, ob einem Mitarbeiter jemals eine Bonuszahlung ausbezahlt wurde:

```
SELECT
    name,
    vorname
FROM
    mitarbeiter m
WHERE
    EXISTS(
        SELECT mitarbeiterid FROM bonus b
        WHERE b.mitarbeiterid=m.mitarbeiterid);
```

Listing 18.17 In einer Unterabfrage auf Existenz von Ergebniszeilen prüfen

Hierzu nutzen wir eine Abfrage, in der die Spalten *name* und *vorname* der Tabelle *mitarbeiter* ermittelt werden. Mit einer **WHERE**-Klausel soll hier geprüft werden, ob Zeilen in der Tabelle *bonus* zum jeweiligen Mitarbeiter existieren. Hier wenden wir unseren **EXISTS**-Vergleichsoperator an, mit dem wir in einer Unterabfrage prüfen, ob Datensätze existieren.

Um dies zu realisieren, setzen wir die Werte der Spalte *mitarbeiterid* der Tabelle *bonus* in Korrelation zu den übergeordneten Werten der Spalte *mitarbeiterid* der Tabelle *mitarbeiter*. Wenn die **WHERE**-Klausel der Unterabfrage in den jeweils korrelierenden Zeilen der übergeordneten Abfrage fündig wird, ist die Bedingung des **EXISTS**-Vergleichsoperators erfüllt. Er prüft nur, ob Ergebniszeilen zurückgeliefert werden.

In der Ergebnisliste liefert Ihnen die Datenbank, wie in [Tabelle 18.12](#) ersichtlich, sämtliche Mitarbeiter, denen in der Unterabfrage eine Bonuszahlung zugeordnet werden konnte.

name	vorname
Müller	Ralf
Müller	Iris
Klarfeld	Simone
Funke	Doris

Tabelle 18.12 Ergebnis einer Prüfung auf Ergebniszeilen einer Unterabfrage mit **EXISTS**

Optional können Sie den **EXISTS**-Operator auch mit dem vorangestellten Schlüsselwort **NOT** verwenden, um zu erreichen, dass die Bedingung in der **WHERE**-Klausel nur dann wahr ist, wenn keine Zeilen aus der Unterabfrage zurückgeliefert werden.

Bleiben wir also bei unserem Beispiel. Jetzt interessieren uns die Mitarbeiter, die noch nie eine Bonuszahlung erhalten haben. Hierzu verändern wir die **SELECT**-Abfrage aus

Listing 18.17 nur minimal in der **WHERE**-Klausel, indem wir einen **NOT EXISTS**-Vergleichsoperator auf das Ergebnis der Unterabfrage anwenden:

```
SELECT
    name,
    vorname
FROM
    mitarbeiter m
WHERE
    NOT EXISTS(
        SELECT mitarbeiterid FROM bonus b
        WHERE b.mitarbeiterid=m.mitarbeiterid);
```

Listing 18.18 In einer Unterabfrage auf Nichtexistenz von Ergebniszeilen prüfen

Tabelle 18.13 zeigt Ihnen einen Auszug des Resultats der **SELECT**-Abfrage aus Listing 18.18.

name	vorname
Schneider	Petra
Klein	Thomas
Lang	Ute
Eisenhof	Frank

Tabelle 18.13 Ergebnis für eine Prüfung auf die Nichtexistenz von Ergebniszeilen einer Unterabfrage

Die Datenbank liefert Ihnen nur die Zeilen zurück, für die in der Bedingung der **WHERE**-Klausel der Unterabfrage keine korrelierenden Werte gefunden werden konnten. Die Unterabfrage liefert uns dann keine Ergebniszeile zurück. Die Bedingung des **NOT EXISTS**-Vergleichsoperators ist in diesen Fällen wahr, und wir erhalten nur die Zeilen der Mitarbeiter, die noch keine Bonuszahlung erhalten haben.

Zusammenfassung: Auf die Existenz von Zeilen prüfen

Mit dem **EXISTS**-Vergleichsoperator wird in einer **WHERE**-Klausel einer übergeordneten Abfrage geprüft, ob eine Unterabfrage eine Zeile zurückgibt.

Die Anzahl der Spalten, die in diesem Fall von der Unterabfrage zurückgeliefert werden, ist nicht von Bedeutung. Es wird nur ausgewertet, ob eine Zeile zurückgegeben wird.

Der Inhalt der Zeile ist ebenfalls bedeutungslos. Der **EXISTS**-Operator prüft ausschließlich auf die Existenz von Zeilen.



18.4.1 Übungen

Übung 1

Beschreiben Sie kurz, welche Art von Vergleich der **EXISTS**-Operator auf eine Unterabfrage in einer **WHERE**-Klausel anwendet.

Übung 2

Sie haben bereits in Übung 4 aus [Abschnitt 18.3](#) die Tabelle *steuerklasse* kennengelernt. Das Ziel dieser Übung ist mit demjenigen aus Übung 4 identisch: Wir wollen wieder die Mitarbeiter ermitteln, denen in der Tabelle *mitarbeiter* eine Steuerklasse zugeordnet ist und die nicht in der Tabelle *steuerklasse* vorhanden sind.

In diesem Fall soll der **EXISTS**-Vergleichsoperator verwendet werden, um das Ziel zu erreichen. Beachten Sie, dass in beiden Tabellen, *mitarbeiter* und *steuerklasse*, die Spalten *steuerklasse* und *steuerklassezahl* enthalten sind. Diese Spalten eignen sich, um die Werte der Spalte *steuerklasse* der Tabelle *mitarbeiter* mit den Werten der Spalte *steuerklassezahl* der Tabelle *steuerklasse* korrelierend in der Unterabfrage zu vergleichen, um anschließend in der übergeordneten Abfrage auf die Nichtexistenz von Ergebniszeilen zu prüfen.

Lösung zu Übung 1

Der **EXISTS**-Operator prüft für eine übergeordnete Abfrage, ob eine Unterabfrage eine Zeile zurückliefert. Wenn die Unterabfrage eine Zeile zurückliefert, ist die Bedingung erfüllt.

Lösung zu Übung 2

```
SELECT
    name,
    vorname,
    steuerklasse
FROM
    mitarbeiter m
WHERE
    NOT EXISTS (
        SELECT
            steuerklassezahl
        FROM
            steuerklasse s
        WHERE s.steuerklassezahl=m.steuerklasse
    );
```

Listing 18.19 Auf Nichtexistenz von Ergebniszeilen in einer Unterabfrage prüfen

Kapitel 19

Views: Abfragen in virtuellen Tabellen speichern

Views bieten die Möglichkeit, komplexe SQL-Abfragen zu kapseln, um die Abfrage vor dem Anwender zu verbergen. Mit Views können Sie also dem Anwender eine vereinfachte Sicht auf komplexe Abfragen zur Verfügung stellen.

SQL bietet Ihnen mit *Sichten* (Views) eine ziemlich einfache Möglichkeit, **SELECT**-Abfragen dauerhaft in einer Datenbank zu hinterlegen. Eine View speichert eine **SELECT**-Abfrage innerhalb der Datenbank unter einem Namen. Eine solche View können Sie unter Ihrem Namen wie eine gewöhnliche Tabelle mit einer **SELECT**-Anweisung abfragen. Wenn Sie eine View abfragen, wird die in der View definierte **SELECT**-Anweisung ausgeführt.

Es gibt zwei Gründe dafür, Views zu verwenden:

- ▶ Sie haben mit Views die Möglichkeit, komplexe SQL-Abfragen dauerhaft in der Datenbank zu speichern und auf die Views wie auf eine Tabelle mit einer **SELECT**-Anweisung zuzugreifen. Das erleichtert Ihnen den Umgang mit Ihrem Datenbestand.
- ▶ Ein komplexes Datenmodell und die beteiligten Tabellen (z. B. Tabellen einer Personalabteilung) sollen in der Regel nicht jedem zugänglich gemacht werden. Mit Views können Sie komplexe **SELECT**-Anweisungen (gegebenenfalls mit vielen **JOINS**) kapseln und die tatsächlich abgefragten Tabellen verbergen.

Views

Mithilfe einer View können **SELECT**-Abfragen unter einem Namen dauerhaft in einer Datenbank hinterlegt werden. Die gespeicherte View kann wie eine gewöhnliche Tabelle mit einer **SELECT**-Anweisung abgefragt werden. Wenn Sie eine View abfragen, wird die **SELECT**-Anweisung ausgeführt, die bei der Erstellung der View definiert wurde.

Abschnitt 19.1 zeigt Ihnen, wie Sie eine einfache View erstellen. In Abschnitt 19.2 lernen Sie, wie Sie Views erzeugen, die eine von Ihnen festgelegte Sortierung bereitstellen. In Abschnitt 19.3 erfahren Sie, in welchem Rahmen Sie **INSERT**-, **UPDATE**- und **DELETE**-Anweisungen auf Views anwenden können.

19.1 Einfache Views anlegen

In [Listing 19.1](#) sehen Sie ein Beispiel für die Erstellung von Views bzw. Sichten:

```
CREATE VIEW v_mitarbeiter
AS SELECT mitarbeiterid,name,vorname FROM mitarbeiter;
```

Listing 19.1 Syntax für die Speicherung von SELECT-Abfragen

Das Listing beginnt mit dem Schlüsselwort **CREATE**, gefolgt von dem Schlüsselwort **VIEW**. Mit der Wortfolge **CREATE VIEW** teilen Sie der Datenbank mit, dass Sie eine View erstellen wollen. Hinter das Schlüsselwort **VIEW** schreiben Sie einen Namen, den Sie frei wählen können. Er sollte allerdings den Inhalt der View möglichst sprechend wiedergeben: Sie wollen ja sicherlich später noch leicht verstehen, was in dieser View enthalten ist, oder?



Bezeichnungen müssen eindeutig sein

Wählen Sie hier einen Namen, der noch nicht als Tabellename vergeben ist. Wenn Sie einer View einen Namen zuordnen, der bereits als Tabellename in der Datenbank existiert, kommt es zu einer Fehlermeldung.

Nun folgt das Schlüsselwort **AS**, gefolgt von der **SELECT**-Abfrage, die Sie als View speichern möchten. In [Listing 19.1](#) ermittelt die **SELECT**-Abfrage nur die Spalten *mitarbeiterid*, *name* und *vorname* der Tabelle *mitarbeiter*.

Achten Sie bei der Formulierung einer **SELECT**-Anweisung, die Sie in einer View hinterlegen wollen, darauf, dass Sie stets eindeutige Spaltennamen verwenden. Die Spaltennamen in einer View müssen unterschiedlich sein: Wenn Sie gleiche Spaltenbezeichnungen in der Definition der **SELECT**-Anweisung einer View verwenden, kommt es zu einer Fehlermeldung. Kurz zusammengefasst können Sie die Anweisung wie folgt beschreiben: »Erzeuge eine Sicht mit dem Namen *v_mitarbeiter*, die aus der folgenden **SELECT**-Abfrage besteht.«

Bei der Namenvergabe für Views hat es sich eingebürgert, Namen, die einer View zugeordnet werden, mit dem Präfix *v_* zu beginnen, um die Views von den Tabellennamen unterscheiden zu können.

In [Listing 19.2](#) sehen Sie eine **SELECT**-Anweisung, in der die Spalten *mitarbeiterid*, *name* und *vorname* der View *v_mitarbeiter* abgefragt werden:

```
SELECT mitarbeiterid, name, vorname FROM v_mitarbeiter;
```

Listing 19.2 Eine View mit einer SELECT-Anweisung abfragen

Es handelt sich um eine gewöhnliche **SELECT**-Anweisung, mit der wir anstelle einer Tabelle die View mit dem Namen `v_mitarbeiter` abfragen.

Wenn Sie die **SELECT**-Anweisung aus [Listing 19.2](#) ausführen, erhalten Sie ein Ergebnis, wie es in [Tabelle 19.1](#) dargestellt ist.

mitarbeiterid	Name	vorname
1	Müller	Ralf
2	Schneider	Petra
3	Klein	Thomas
4	Lang	Ute

Tabelle 19.1 Ergebnis einer **SELECT**-Abfrage auf die View »v_mitarbeiter«

Was passiert, wenn sich die Daten ändern, die von einer gespeicherten View abgefragt werden? Sehen wir uns dazu ein Beispiel an: Ändern wir einfach in der Tabelle *mitarbeiter* den Namen des Mitarbeiters Ralf Müller in den Wert Müller 2. Hierzu verwenden wir die **UPDATE**-Anweisung aus [Listing 19.3](#):

```
UPDATE mitarbeiter
SET name='Müller 2'
WHERE mitarbeiterid=1;
```

Listing 19.3 Daten in der Tabelle ändern, auf die eine View zugreift

Nach der Aktualisierung der Zeile des Mitarbeiters Ralf Müller prüfen Sie, ob die View `v_mitarbeiter` die Änderung zur Kenntnis nimmt. Hierzu nutzen Sie die **SELECT**-Anweisung aus [Listing 19.4](#):

```
SELECT * FROM v_mitarbeiter;
```

Listing 19.4 Abfrage der View »v_mitarbeiter«

Als Ergebnis erhalten Sie [Tabelle 19.2](#).

mitarbeiterid	name	vorname
1	Müller 2	Ralf
2	Schneider	Petra

Tabelle 19.2 Ergebnis einer Abfrage auf eine View nach einer Aktualisierung der Tabelle, die in der View abgefragt wird

mitarbeiterid	name	vorname
3	Klein	Thomas
4	Lang	Ute

Tabelle 19.2 Ergebnis einer Abfrage auf eine View nach einer Aktualisierung der Tabelle, die in der View abgefragt wird (Forts.)

Die Änderung, die durch die **UPDATE**-Anweisung durchgeführt wurde, ist sofort sichtbar. Die **SELECT**-Abfrage auf die View *v_mitarbeiter* bewirkt also die erneute Ausführung der **SELECT**-Anweisung, die während der View-Erstellung gespeichert wurde.

Im Beispiel aus [Listing 19.5](#) zeige ich Ihnen, wie Sie eine View erstellen, deren **SELECT**-Anweisung zwei Tabellen über einen **INNER JOIN** abfragt. Die Vorgehensweise zur Erstellung einer View bleibt die gleiche. Hier ändert sich lediglich die **SELECT**-Anweisung hinter dem **AS**-Schlüsselwort der **CREATE VIEW**-Anweisung:

```
CREATE VIEW v_mitarbeiterkontakt
AS
SELECT
    m.mitarbeiterid,
    m.name,
    m.vorname,
    k.festnetz,
    k.mobilnetz
FROM
    mitarbeiter m INNER JOIN kontakt k
ON
    m.mitarbeiterid=k.mitarbeiterid;
```

Listing 19.5 Eine View mit einer **SELECT**-Anweisung anlegen, die über einen **INNER JOIN** die Zeilen von zwei Tabellen verbindet

Als Nächstes werden wir die View *v_mitarbeiterkontakt* aus [Listing 19.5](#) mit einer **SELECT**-Anweisung abfragen, wie sie in [Listing 19.6](#) dargestellt ist:

```
SELECT * FROM v_mitarbeiterkontakt;
```

Listing 19.6 Eine View abfragen, deren **SELECT**-Abfrage mehrere Tabellen über einen **JOIN** verbindet

Als Ergebnis für die Abfrage aus [Listing 19.6](#) erhalten Sie [Tabelle 19.3](#). In der **SELECT**-Anweisung haben Sie mit dem *****-Auswahloperator sämtliche Spalten der View *v_mitarbeiterkontakt* ausgewählt. Die Ergebnistabelle enthält also die durch den **JOIN** verbundenen Zeilen und Spalten der Tabellen *mitarbeiter* und *kontakt*.

mitarbeiterid	name	vorname	festnetz	mobilnetz
2	Schneider	Petra	0123/1111111	4566/000000000
4	Lang	Ute	8977/999999	2345/5555555
6	Lupin	Anja	2345/333333	3456/6666666
8	Müller	Iris	3456/111111111	2345/9999999

Tabelle 19.3 Ergebnis einer SELECT-Abfrage, die auf eine View ausgeführt wird, die zwei Tabellen mit einem INNER JOIN verbindet

Zusammenfassung: Views anlegen

Views werden mit der **CREATE VIEW**-Anweisung angelegt.

Die Namen der zu erstellenden Views dürfen nicht identisch mit bestehenden Tabellennamen sein.

Es ist ratsam, Views mit einem Präfix wie **v_** zu versehen, um sie von Tabellen unterscheiden zu können.

In der zugrunde liegenden **SELECT**-Anweisung einer View dürfen keine gleichen Spaltennamen verwendet werden. Diese Gefahr besteht insbesondere, wenn **JOINS** benutzt werden.

19.1.1 Übungen

Übung 1

Worauf müssen Sie achten, wenn Sie einer View einen Namen geben?

Übung 2

Was ist bei der Auswahl der Spalten der **SELECT**-Abfrage zu beachten, die für die Erstellung einer View verwendet wird?

Übung 3

Wie wirkt es sich aus, wenn sich die Spaltenwerte in der Tabelle der **SELECT**-Abfrage ändern, die der View zugrunde liegt?

Übung 4

Erstellen Sie eine View mit der Bezeichnung **v_urlaub**. Für die Erstellung der View verwenden Sie eine **SELECT**-Abfrage auf die Tabelle *mitarbeiter*, in der die Spalten *name*,

vorname, *urlaubstage* und *urlaubgenommen* aus der Tabelle *mitarbeiter* abgefragt werden.

Übung 5

Erstellen Sie eine View mit der Bezeichnung *v_praemie*. Die zugrunde liegende **SELECT**-Abfrage soll die Zeilen der Tabellen *mitarbeiter* und *sachpraemie* über einen **INNER JOIN** verbinden. In die Spaltenauswahlliste sollen folgende Spalten aufgenommen werden: *name*, *vorname*, *praemie* und *grund*.

Lösung zu Übung 1

Der Name einer zu erstellenden View darf nicht bereits als Tabellennamen vergeben sein.

Lösung zu Übung 2

Die Spaltenauswahlliste der **SELECT**-Abfrage, die für die Erstellung einer View genutzt wird, darf keine mehrfach vorkommenden gleichen Spaltennamen enthalten.

Lösung zu Übung 3

Wenn eine View abgefragt wird, wird die dahinter liegende **SELECT**-Anweisung ausgeführt, und somit werden in einer View stets die Spaltenwerte angezeigt, die in der dahinter liegenden Tabelle aktuell sind.

Lösung zu Übung 4

```
CREATE VIEW v_urlaub AS
SELECT
    name,
    vorname,
    urlaubstage,
    urlaubgenommen
FROM mitarbeiter;
```

Listing 19.7 Eine einfache View erstellen, deren **SELECT**-Abfrage eine Tabelle abfragt

Lösung zu Übung 5

```
CREATE VIEW v_praemie AS
SELECT
    m.name,
    m.vorname,
    s.praemie,
```



```

s.grund
FROM mitarbeiter m INNER JOIN sachpraemie s
ON m.mitarbeiterid=s.mitarbeiterid;

```

Listing 19.8 Eine View anlegen, deren SELECT-Abfrage zwei Tabellen mit einem INNER JOIN zugrunde liegen

19.2 Views und ORDER BY

Wenn Sie eine View erstellen, in der eine SELECT-Anweisung festgelegt wird, besteht die Möglichkeit, eine ORDER BY-Klausel zu verwenden.

Unterschiede zwischen den Datenbanken

Leider wird auch hier der SQL-Standard von den unterschiedlichen Datenbanken nicht gleich umgesetzt. MySQL-, MariaDB- und PostgreSQL-Datenbanken ermöglichen es Ihnen ohne Probleme, die ORDER BY-Klausel in der SELECT-Abfrage einer CREATE VIEW-Anweisung zu verwenden.

Wenn Sie eine MS SQL Server-Datenbank verwenden, nutzen Sie die TOP 100 PERCENT-Klausel vor der Spaltenauswahlliste, um eine ORDER BY-Klausel in einer SELECT-Abfrage einer CREATE VIEW-Anweisung verwenden zu können. Ohne die TOP 100 PERCENT-Klausel würde das Anlegen der View auf der Datenbank zu einem Fehler führen. Die Anlage einer View mit einer ORDER BY-Klausel funktioniert also unter den dargestellten Bedingungen. Und wenn Sie sich wundern, wenn Sie eine Abfrage auf eine erstellte View machen: Die Sortierung wird aufgrund der TOP-Klausel, die vorrangig ausgeführt wird, nicht berücksichtigt!

Sehen wir uns ein konkretes Beispiel für eine CREATE VIEW-Anweisung an, in deren SELECT-Anweisung eine ORDER BY-Klausel Verwendung findet:

```

CREATE VIEW v_mitarbeitersortiert
AS
SELECT
  TOP 100 PERCENT /* MS SQL Server */
  mitarbeiterid,
  name,
  vorname
FROM
  mitarbeiter
ORDER BY name;

```

Listing 19.9 SELECT-Abfrage einer View mit einer ORDER BY-Klausel verwenden

Mit einer einfachen Abfrage, wie sie in [Listing 19.10](#) dargestellt ist, überprüfen Sie im nächsten Schritt, ob die Sortierung, die in der **SELECT**-Abfrage der **CREATE VIEW**-Anweisung festgelegt wurde, auch tatsächlich ihre Dienste verrichtet:

```
SELECT mitarbeiterid,name,vorname FROM v_mitarbeitersortiert;
```

Listing 19.10 Abfrage einer sortierten View

In [Tabelle 19.4](#) sehen Sie das Ergebnis der **SELECT**-Abfrage aus [Listing 19.10](#). Die Ergebniszeilen der Abfrage wurden gemäß den Werten der Spalte *name* sortiert und zurückgegeben.

mitarbeiterid	name	vorname
27	Blume	Siegmar
44	Bücher	Herbert
43	Dorsten	Klaus
5	Eisenhof	Frank

Tabelle 19.4 Ergebnis einer Abfrage einer View, bei deren Erstellung eine **ORDER BY**-Klausel auf die **SELECT**-Anweisung angewendet wurde

Was passiert eigentlich, wenn eine **SELECT**-Abfrage auf eine View mit einer eigenen **ORDER BY**-Klausel ausgestattet ist? Die **ORDER BY**-Klausel, die während der Erstellung der View *v_mitarbeitersortiert* verwendet wurde, gibt die Zeilen sortiert zurück. Diese Sortierung erfolgt nach den Werten der Spalte *name* der Tabelle *mitarbeiter*.

In [Listing 19.11](#) stattdessen wir eine **SELECT**-Anweisung, die die View *v_mitarbeitersortiert* abfragt, mit einer eigenständigen **ORDER BY**-Klausel aus, um die Zeilen nach den Werten der Spalte *vorname* der View *v_mitarbeitersortiert* zu sortieren und auszugeben:

```
SELECT mitarbeiterid,name,vorname
FROM v_mitarbeitersortiert
ORDER BY vorname;
```

Listing 19.11 Sortierte Abfrage einer sortierten View

Als Ergebnis erhalten wir [Tabelle 19.5](#) zurück.

mitarbeiterid	name	vorname
23	Funke	Andreas

Tabelle 19.5 Ergebnis für eine **SELECT**-Abfrage, die auf eine View zugreift und über eine **ORDER BY**-Klausel verfügt

mitarbeiterid	name	vorname
6	Lupin	Anja
20	Klein	Bärbel
36	Ludowig	Carmen

Tabelle 19.5 Ergebnis für eine SELECT-Abfrage, die auf eine View zugreift und über eine ORDER BY-Klausel verfügt (Forts.)

Die Mitarbeiter wurden nach der Spalte *vorname* sortiert. Die Sortierung der zugrunde liegenden SELECT-Anweisung der View *v_mitarbeitersortiert* wird durch die ORDER BY-Klausel in einer SELECT-Abfrage übersteuert, die die Zeilen der View *v_mitarbeitersortiert* abfragt.

Zusammenfassung: Views mit einer ORDER BY-Klausel in der zugrunde liegenden SELECT-Anweisung

Die einer View zugrunde liegende SELECT-Anweisung kann eine ORDER BY-Klausel enthalten. Wenn die View mithilfe einer SELECT-Anweisung abgefragt wird, werden die Zeilen in der Ergebnistabelle gemäß den Spaltenangaben der ORDER BY-Klausel sortiert zurückgegeben.

Wird die View mit einer Sortierung durch eine SELECT-Abfrage abgefragt, die ebenfalls eine ORDER BY-Klausel enthält, so wird die Sortierung der SELECT-Abfrage auf die Ergebnistabelle angewendet.



19.2.1 Übungen

Übung 1

Erstellen Sie eine View mit der Bezeichnung *v_mitarbeiterbonus*. Die zugrunde liegende SELECT-Abfrage soll durch einen JOIN die Zeilen der Tabellen *mitarbeiter* und *bonus* miteinander verbunden ausgeben. Außerdem soll die View mit einer Sortierung ausgestattet sein, die dafür sorgt, dass die Zeilen aufsteigend sortiert nach den Werten der Spalte *bonus* ausgegeben werden. In die Spaltenauswahlliste der SELECT-Anweisung nehmen Sie die Spalten *name*, *vorname* und *bonus* auf.

Übung 2

Wenn Sie mit einer SELECT-Anweisung eine View abfragen, deren zugrunde liegende SELECT-Anweisung über eine Sortierung verfügt, wie wirkt sich dann eine ORDER BY-Klausel auf das Ergebnis der SELECT-Anweisung aus, mit der Sie die bereits sortierte View abfragen?

Lösung zu Übung 1

```
CREATE VIEW v_mitarbeiterbonus AS
SELECT
    TOP 100 PERCENT /* MS SQL Server */
    m.name,
    m.vorname,
    b.bonuszahlung
FROM mitarbeiter m INNER JOIN bonus b
ON m.mitarbeiterid=b.mitarbeiterid
ORDER BY b.bonuszahlung;
```

Listing 19.12 Eine View mit einer zugrunde liegenden SELECT-Anweisung erstellen, die mit einer ORDER BY-Klausel ausgestattet ist

Lösung zu Übung 2

Die **ORDER BY**-Klausel der **SELECT**-Anweisung, die auf eine bereits sortierte View angewendet wird, übersteuert die Sortierung der View.

19.3 INSERT, UPDATE und DELETE auf Views anwenden

Wenn Sie eine View angelegt haben, können Sie unter anderem neue Zeilen einfügen, Werte aus bestehenden Zeilen ändern oder auch Zeilen löschen. Vielleicht ist das etwas schwierig nachzuvollziehen, da eine View ja lediglich das Ergebnis einer Abfrage zurückliefert.

19.3.1 Eine INSERT-Anweisung auf Views anwenden

Der View *v_mitarbeiter* liegt eine **SELECT**-Abfrage zugrunde, in der die Spalten *mitarbeiterid*, *name* und *vorname* der Tabelle *mitarbeiter* abgefragt werden.

Wenn Sie eine neue Zeile in eine View einfügen wollen, müssen Sie sicherstellen, dass die View auch die Primärschlüsselspalte abfragt, um die neue Zeile mit einem eindeutigen Primärschlüsselwert versehen zu können. Das gilt nicht, wenn eine Tabelle eine Autinkrement-Spalte als Primärschlüsselspalte verwendet.

Beachten Sie auch Folgendes: Wenn ich schreibe, dass wir Zeilen in Views einfügen, dann ist damit stets gemeint, dass wir Zeilen mit einer **INSERT**-Anweisung über eine View in eine Tabelle einfügen. Um die Formulierung einheitlich zu halten, verwende ich hier dennoch stets die Formulierung »in eine View einfügen«.

Die View `v_mitarbeiter` enthält die Primärschlüsselspalte `mitarbeiterid`, die in der Spaltenauswahlliste der zugrunde liegenden `SELECT`-Anweisung enthalten ist. [Listing 19.13](#) zeigt eine `INSERT`-Anweisung, die einen neuen Mitarbeiter, Josef Ebert, in die View `v_mitarbeiter` einfügt:

```
INSERT INTO v_mitarbeiter VALUES (55, 'Ebert', 'Josef');
```

Listing 19.13 Eine neue Zeile in eine View einfügen

Nach dem Einfügen fragen Sie mit einer einfachen `SELECT`-Abfrage wie in [Listing 19.14](#) die View `v_mitarbeiter` ab, um zu prüfen, ob die neue Zeile eingefügt wurde:

```
SELECT * FROM v_mitarbeiter;
```

Listing 19.14 Prüfabfrage für die View »v_mitarbeiter« nach einem Einfügevorgang

Das Ergebnis der Abfrage aus [Listing 19.14](#) sehen Sie in [Tabelle 19.6](#).

mitarbeiterid	name	vorname
48	Bücher	Ilse
49	Dorsten	Stephanie
50	Eisenhof	Linda
55	Ebert	Josef

Tabelle 19.6 Ergebnistabelle für eine Prüfabfrage auf die View »v_mitarbeiter«

Die neue Zeile wurde in die View `v_mitarbeiter` eingefügt: In der letzten Zeile des Auszugs der Ergebnistabelle sehen Sie eine neue Zeile mit dem neuen Mitarbeiter Josef Ebert.

Als Nächstes interessiert uns, ob die neue Zeile auch in der Tabelle `mitarbeiter` enthalten ist. Hierzu führen wir die `SELECT`-Anweisung aus [Listing 19.15](#) aus:

```
SELECT * FROM mitarbeiter;
```

Listing 19.15 Prüfabfrage für die Tabelle »mitarbeiter« nach einem Einfügevorgang, der über die View »v_mitarbeiter« realisiert wurde

Das Ergebnis der `SELECT`-Abfrage aus [Listing 19.15](#) sehen Sie auszugsweise dargestellt in [Tabelle 19.7](#). Die letzte Zeile enthält die zuvor in die View `v_mitarbeiter` eingefügte Zeile mit dem Mitarbeiter Josef Ebert.

mitarbeiterid	name	vorname	gebdatum	eintrittsdatum
48	Bücher	Ilse	1955-01-11	1975-06-07
49	Dorsten	Stephanie	1968-10-21	1995-06-19
50	Eisenhof	Linda	1958-07-15	1970-08-07
55	Ebert	Josef	NULL	NULL

Tabelle 19.7 Prüfabfrage für die Tabelle »mitarbeiter«, die der View »v_mitarbeiter« zugrunde liegt

Als Nächstes betrachten wir die View *v_mitarbeiterkontakt*, die wir mit [Listing 19.5](#) angelegt haben.



Unterschiede zwischen den Datenbanken

Wenn Sie eine MySQL-, MariaDB- oder eine MS SQL Server-Datenbank verwenden, können Sie dieses Beispiel auch ohne weitere Anweisungen praktisch nachvollziehen. Die PostgreSQL-Datenbank lässt einen Einfügevorgang in eine View, deren Abfrage auf mehrere Tabellen zugreift, standardmäßig nicht zu. INSERT-Anweisungen auf eine View erfordern eine RULE (Regel). Für unseren konkreten Fall sieht die notwendige RULE so aus:

```
CREATE RULE rule_insert_v_mitarbeiterkontakt
AS ON INSERT TO v_mitarbeiterkontakt
DO INSTEAD
INSERT INTO mitarbeiter (mitarbeiterid,name,vorname)
VALUES (NEW.mitarbeiterid,NEW.name,NEW.vorname);
```

Listing 19.16 Eine Regel (RULE) für eine INSERT-Operation festlegen

Wenn Sie diese Regel erstellt haben, lassen sich über die View *v_mitarbeiterkontakt* Zeilen in die Tabelle *mitarbeiter* einfügen. Weitere Details entnehmen Sie bitte der PostgreSQL-Dokumentation.

Der View liegen die Tabellen *mitarbeiter* und *kontakt* zugrunde. In [Listing 19.17](#) sehen Sie eine INSERT-Anweisung, mit der eine weitere Zeile in die View *v_mitarbeiterkontakt* eingefügt werden soll:

```
INSERT INTO v_mitarbeiterkontakt (mitarbeiterid,name,vorname)
VALUES (56, 'Albers', 'Daniel');
```

Listing 19.17 Eine neue Zeile in eine View einfügen

Die Abfrage zum Anlegen der View *v_mitarbeiterkontakt* verwendet einen **INNER JOIN**, in dessen **ON**-Klausel die Spalten *mitarbeiterid* der Tabellen *mitarbeiter* und *kontakt* auf Gleichheit geprüft werden, um die Zeilen der beiden Tabellen zu verbinden. Ebenfalls wichtig ist hier, dass wir in der **INSERT**-Anweisung aus [Listing 19.17](#) ausschließlich die Spaltenwerte für die Tabelle *mitarbeiter* in die View *v_mitarbeiterkontakt* einfügen.

Jetzt prüfen wir, ob der neue Mitarbeiter Daniel Albers auch tatsächlich in die View eingefügt wurde. Hierzu nutzen wir die **SELECT**-Anweisung aus [Listing 19.18](#):

```
SELECT * FROM v_mitarbeiterkontakt;
```

Listing 19.18 Prüfabfrage für die View »v_mitarbeiterkontakt« nach einem Einfügevorgang

In [Tabelle 19.8](#) sehen Sie das Ergebnis der Prüfabfrage aus [Listing 19.18](#).

mitarbeiterid	name	vorname	festnetz	mobilnetz
40	Kanis	Herbert	789012/ 444444444444	456789/ 55555555555
42	Oberscheidt	Michael	56789/ 2222222222	7890123/ 444444444444
44	Bücher	Herbert	11111111/ 777777777777	55555/ 444444444444
46	Heinrich	Hans	00000000/ 11111111	77777/ 999999999

Tabelle 19.8 Ergebnis für eine Prüfabfrage auf die View »v_mitarbeiterkontakt«

Der neue Mitarbeiter, der mit der **INSERT**-Anweisung aus [Listing 19.17](#) eingefügt wurde, ist in der View *v_mitarbeiterkontakt* nicht enthalten.

Dies liegt daran, dass Sie mit der **INSERT**-Anweisung nur die Spaltenwerte angegeben haben, mit denen eine neue Zeile in die Tabelle *mitarbeiter* eingefügt werden kann. Aufgrund des **INNER JOINs** der zugrunde liegenden Abfrage der View *v_mitarbeiterkontakt* können also keine Zeilen der Tabelle *kontakt* dem neuen Mitarbeiter in der **ON**-Klausel zugeordnet werden. Es sind ja schlicht und ergreifend noch keine Kontaktdaten für diesen Mitarbeiter in der Tabelle *kontakt* hinterlegt.

Eine weitere einfache Prüfabfrage auf die Tabelle *mitarbeiter*, wie wir sie bereits in [Listing 19.15](#) verwendet haben, zeigt, dass die Zeile des neuen Mitarbeiters tatsächlich in die Tabelle *mitarbeiter* eingefügt wurde. Die letzte Zeile von [Tabelle 19.9](#) enthält den neuen Mitarbeiter Daniel Albers.

mitarbeiterid	name	vorname	gebdatum	eintrittsdatum
49	Dorsten	Stephanie	1968-10-21	1995-06-19
50	Eisenhof	Linda	1958-07-15	1970-08-07
55	Ebert	Josef	NULL	NULL
56	Albers	Daniel	NULL	NULL

Tabelle 19.9 Ergebnis für eine Prüfabfrage auf eine Tabelle, in die durch eine View eine neue Zeile eingefügt wurde

Als Nächstes versuchen wir, eine neue Zeile in die View `v_mitarbeiterkontakt` einzufügen, in der wir auch die Telefonangaben berücksichtigen:

```
INSERT INTO v_mitarbeiterkontakt
(mitarbeiterid,name,vorname,festnetz,mobilnetz)
VALUES
(57,'Stern','Julian','0000/1234567','0000/234456');
```

Listing 19.19 Werte für eine View einfügen, der zwei Tabellen zugrunde liegen

Wenn Sie versuchen, diese **INSERT**-Anweisung auszuführen, erhalten Sie eine Fehlermeldung. Beispielsweise meldet die MySQL-Datenbank:

Error Code: 1393. Can not modify more than one base table through a join view

Inhaltlich erhalten Sie von einer MySQL-, MariaDB-, PostgreSQL- bzw. einer MS SQL Server-Datenbank die gleiche Fehlermeldung. In eine View kann keine Zeile eingefügt werden, die zwei Tabellen gleichzeitig mit neuen Datensätzen befüllen würde. Der Hintergrund ist, dass die Datenbank aus einer **INSERT**-Anweisung zwei **INSERT**-Anweisungen generieren müsste.

19.3.2 Eine UPDATE-Anweisung auf Views anwenden

In [Listing 19.3](#) aus [Abschnitt 19.1](#) haben Sie bereits erfahren, dass Sie eine View, der eine Tabelle zugrunde liegt, mit einer **UPDATE**-Anweisung aktualisieren können.

Im folgenden Beispiel werden wir prüfen, ob Sie auch eine View aktualisieren können, der mehr als eine Tabelle zugrunde liegt, wie das z. B. bei einem **JOIN** die Regel ist. Eine Ausnahme bildet natürlich ein Self Join; diesen werden wir hier allerdings nicht betrachten.



Unterschiede zwischen den Datenbanken

Falls Sie eine MySQL-, MariaDB- oder eine MS SQL Server-Datenbank verwenden, können Sie dieses Beispiel wieder ohne zusätzliche Anweisungen praktisch nachvollziehen.

Die PostgreSQL-Datenbank hingegen lässt auch keine Aktualisierung für eine View zu, die aus mehreren Tabellen besteht. Sehen wir uns kurz an, wie Sie mit einer Regel Zeilen einer Tabelle abfragen, die einer View zugrunde liegt, die mehrere Tabellen über einen **JOIN** abfragt:

```
CREATE RULE rule_update_v_mitarbeiterkontakt
AS ON UPDATE TO v_mitarbeiterkontakt
DO INSTEAD
UPDATE mitarbeiter
SET name = NEW.name
WHERE mitarbeiterid = NEW.mitarbeiterid;
```

Listing 19.20 Eine Regel (RULE) für eine UPDATE-Operation festlegen

Wenn Sie diese Regel erstellt haben, lassen sich über die View *v_mitarbeiterkontakt* Zeilen der Tabelle *mitarbeiter* aktualisieren. Weitere Details entnehmen Sie bitte der PostgreSQL-Dokumentation

In [Listing 19.21](#) sehen Sie eine **UPDATE**-Anweisung, mit der Sie den Wert der Spalte *name* der View *v_mitarbeiterkontakt* mit dem neuen Spaltenwert *Winter* versehen:

```
UPDATE v_mitarbeiterkontakt SET name='Winter'
WHERE mitarbeiterid=35;
```

Listing 19.21 Eine View mit einer UPDATE-Anweisung aktualisieren, der zwei Tabellen (INNER JOIN) zugrunde liegen

Um sicherzustellen, dass ausschließlich die richtige Zeile aktualisiert wird, nutzen Sie eine **WHERE**-Klausel, die prüft, ob die Spalte *mitarbeiterid* gleich dem Wert 35 ist.

Als Nächstes fragen Sie die View *v_mitarbeiterkontakt* ab, um zu prüfen, ob die Aktualisierung stattgefunden hat. Für diese Aufgabe nutzen Sie die **SELECT**-Abfrage aus [Listing 19.22](#), die mit einer **WHERE**-Klausel versehen ist, um nur die Zeile abzufragen, die dem betroffenen Mitarbeiter zugeordnet ist:

```
SELECT * FROM v_mitarbeiterkontakt WHERE mitarbeiterid=35;
```

Listing 19.22 Prüfabfrage auf die View »v_mitarbeiterkontakt« mit einer WHERE-Klausel, die ausschließlich den Mitarbeiter mit dem Wert »35« in der Spalte »mitarbeiterid« ermittelt

Als Ergebnis der **SELECT**-Abfrage aus [Listing 19.22](#) erhalten Sie eine Zeile zurück. [Tabelle 19.10](#) zeigt Ihnen, dass der Name des Mitarbeiters geändert wurde.

mitarbeiterid	name	vorname	festnetz	mobilnetz
35	Winter	Martin	34567/ 777777777	345678/ 6666666666

Tabelle 19.10 Ergebnis der Prüfabfrage für eine Zeile, die mittels einer View aktualisiert wurde

Die tatsächliche Änderung des Wertes wurde in der Tabelle *mitarbeiter* vorgenommen. Wenn Sie die View *v_mitarbeiterkontakt* mit einer **SELECT**-Anweisung abfragen, wird die für die View definierte **SELECT**-Anweisung wie bisher auch ausgeführt und somit die aktuelle Zeile des Mitarbeiters mit dem Wert 35 für die Spalte *mitarbeiterid* zurückgegeben.



INNER JOIN und mehrere Tabellen

In einer View, die aus mehreren Tabellen (**INNER JOIN**) besteht, kann immer nur eine der am **INNER JOIN** beteiligten Tabellen aktualisiert werden. Achten Sie also darauf, dass Sie die **SET**-Klausel einer **UPDATE**-Anweisung, die Sie auf eine View anwenden, immer nur auf die Spalten einer Tabelle ausführen.

19.3.3 Eine DELETE-Anweisung auf Views anwenden

Wenn Sie versuchen, aus einer View, die über einen **JOIN** auf mehrere Tabellen zugreift, eine Zeile zu löschen, kann die Datenbank nicht ermitteln, welche Zeile aus welcher Tabelle gelöscht werden soll. Diese Information fehlt einfach, wenn Sie versuchen, Zeilen einer View zu löschen, deren **SELECT**-Abfrage einen oder mehrere **JOINS** enthält. Hinzu kommt, dass es sich bei Zeilen aus Tabellen, die mit einem **JOIN** verbunden werden, mit hoher Wahrscheinlichkeit um in Beziehung stehende Zeilen handelt, Stichwort *referenzielle Integrität*. Es ist also nicht möglich, Zeilen aus Views zu löschen, deren zugrunde liegende **SELECT**-Abfrage über einen oder mehrere **JOINS** auf eine oder mehrere Tabellen zugreift.

Greift die **SELECT**-Anweisung einer View nur auf eine Tabelle zu, so ist es möglich, Zeilen mit einer **DELETE**-Anweisung aus einer View bzw. der dahinter liegenden Tabelle zu löschen.

Wir werden jetzt versuchen, eine Zeile aus der View *v_mitarbeiter* zu löschen, die wir aus [Tabelle 19.6](#) kennen. Die **SELECT**-Abfrage der View *v_mitarbeiter* fragt lediglich die Spalten *mitarbeiterid*, *name* und *vorname* der Tabelle *mitarbeiter* ab. Sehen wir uns

noch einmal die ersten vier Zeilen der View `v_mitarbeiter` an. Die Mitarbeiterin Petra Schneider scheidet aus dem Unternehmen aus. Ihr ist die Mitarbeiter-ID 2 zugeordnet.

mitarbeiterid	name	vorname
1	Müller	Ralf
2	Schneider	Petra
3	Klein	Thomas
4	Lang	Ute

Tabelle 19.11 Die ersten vier Zeilen der View »v_mitarbeiter«

In [Listing 19.23](#) sehen Sie eine `DELETE`-Anweisung, die gemäß der Einschränkung der `WHERE`-Klausel den Mitarbeiter mit der ID 2 aus der View `v_mitarbeiter` zu löschen versucht:

```
DELETE FROM v_mitarbeiter WHERE mitarbeiterid=2;
```

Listing 19.23 Eine `DELETE`-Anweisung auf eine View anwenden

Mit der Abfrage aus [Listing 19.24](#) überprüfen wir das Ergebnis der Löschoperation aus [Listing 19.23](#):

```
SELECT * FROM v_mitarbeiter WHERE mitarbeiterid<4;
```

Listing 19.24 Mit einer `SELECT`-Anweisung die Mitarbeiter abfragen, deren Wert in der Spalte »mitarbeiterid« kleiner als 4 ist

Hierzu ermitteln wir in der `WHERE`-Klausel der `SELECT`-Anweisung nur die Mitarbeiter, deren Spaltenwerte der Spalte `mitarbeiterid` kleiner als 4 sind. In [Tabelle 19.12](#) sehen Sie das Ergebnis der `SELECT`-Abfrage aus [Listing 19.24](#).

mitarbeiterid	name	vorname
1	Müller	Ralf
3	Klein	Thomas

Tabelle 19.12 Ergebnis für eine Abfrage, die nur die Mitarbeiter ermittelt, deren Spaltenwerte der Spalte »mitarbeiterid« kleiner als 4 sind

Die Mitarbeiterin mit der ID 2 ist nicht mehr in der Ergebnisliste der Abfrage auf die View `v_mitarbeiter` enthalten.

Bis jetzt haben Sie nur gesehen, dass die Mitarbeiterin mit der ID 2 nicht mehr in der View bzw. der darunter liegenden Tabelle enthalten ist. Das setzt natürlich voraus, dass die Zeile der Mitarbeiterin auch erfolgreich aus der Tabelle *mitarbeiter* gelöscht wurde, auf die ja die View *v_mitarbeiter* zugreift. Ein kurzer Blick mit einer **SELECT**-Abfrage auf die Tabelle *mitarbeiter* bestätigt uns, was wir schon wissen:

```
SELECT * FROM mitarbeiter WHERE mitarbeiterid<4;
```

Listing 19.25 Mit einer **SELECT**-Anweisung die Mitarbeiter abfragen, deren Wert in der Spalte »mitarbeiterid« kleiner als 4 ist

19.3.4 Views, auf die keine **INSERT**-, **DELETE**- oder **UPDATE**-Anweisung angewendet werden kann

Je nachdem, wie die **SELECT**-Anweisung für eine View formuliert wurde, sind Anweisungen wie **INSERT**, **UPDATE** und **DELETE** nicht möglich, und zwar immer dann, wenn die **SELECT**-Abfrage eine der folgenden Klauseln oder Aggregatfunktionen verwendet:

- ▶ **DISTINCT**
- ▶ Aggregatfunktionen: **SUM**, **MIN**, **MAX**, **AVG**, **COUNT**
- ▶ **GROUP BY**
- ▶ **HAVING**
- ▶ **UNION**
- ▶ **UNION ALL**
- ▶ **LEFT OUTER JOIN** – nur Zeilen einer Tabelle können geändert werden.
- ▶ **RIGHT OUTER JOIN** – nur Zeilen einer Tabelle können geändert werden.
- ▶ **FULL OUTER JOIN** – nur Zeilen einer Tabelle können geändert werden.
- ▶ Unterabfrage in der Spaltenauswahlliste
- ▶ Ausdruck in der Spaltenauswahlliste: Wenn Sie in einer **SELECT**-Anweisung einer View einen Ausdruck wie z. B. *spalte1+100* in der Spaltenauswahlliste verwenden, so können Sie diesen berechneten Wert auch nicht mit einer **UPDATE**-Anweisung aktualisieren oder einen neuen Datensatz mit einer **INSERT**-Anweisung einfügen.

Das gilt auch für Aufrufe von Skalarfunktionen. Hier können Sie das Ergebnis der Skalarfunktion nicht aktualisieren, da es sich ja ebenfalls um das Ergebnis eines Ausdrucks handelt und somit kein Bezug zu einem Spaltenwert hergestellt werden kann.



Zusammenfassung: **INSERT, **UPDATE** und **DELETE** auf Views anwenden**

Sämtliche hier im Zusammenhang mit Views aufgeführten Operationen wenden **INSERT**-, **UPDATE**- und **DELETE**-Anweisungen auf die Tabellen an, die in der zugrunde liegenden **SELECT**-Abfrage der View enthalten sind.

Eine **INSERT**-Anweisung kann ohne Weiteres auf Views verwendet werden, die nur eine Tabelle abfragen. Achten Sie dabei darauf, dass in der zugrunde liegenden **SELECT**-Anweisung auch die Primärschlüsselspalte abgefragt wird. Wenn in der Tabelle der zugrunde liegenden **SELECT**-Anweisung Einschränkungen mit **NOT NULL** definiert sind, sind diese ebenfalls beim Einfügen von Zeilen über eine View zu berücksichtigen.

Wenn Zeilen in eine View eingefügt werden sollen, der eine **SELECT**-Anweisung mit einem **JOIN** zugrunde liegt, so kann nur in eine der zugrunde liegenden Tabellen eine neue Zeile eingefügt werden. Bei einer PostgreSQL-Datenbank ist eine **RULE** erforderlich, um neue Zeilen in eine View einzufügen, deren **SELECT**-Abfrage einem **INNER JOIN** zugrunde liegt.

Views mit einem **SELECT**-Befehl, der nur eine Tabelle abfragt, können mit einer **UPDATE**-Anweisung aktualisiert werden. Views, die mehrere Tabellen abfragen, können ebenfalls mit einer **UPDATE**-Anweisung aktualisiert werden.

Eine **DELETE**-Anweisung kann nur auf Views ausgeführt werden, deren zugrunde liegende **SELECT**-Abfrage eine Tabelle abfragt. Liegt ein **JOIN** zugrunde, können die Zeilen nicht gelöscht werden.

INSERT-, **UPDATE**- und **DELETE**-Anweisungen können nicht auf eine View ausgeführt werden, wenn sie folgende Klauseln oder Aggregatfunktionen enthalten: **DISTINCT**, **SUM**, **MIN**, **MAX**, **AVG**, **COUNT**, **GROUP BY**, **HAVING**, **HAVING ALL**, **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, **FULL OUTER JOIN**, **UNION** und Unterabfragen in der Spaltenauswahl-liste.

INSERT- und **UPDATE**-Anweisungen können auch nicht auf Spalten angewendet werden, die einen Ausdruck enthalten. Solche Ausdrücke sind beispielsweise: **Spaltenwert1 + 2**, **TRIM(Spaltenname)**.

19.3.5 Übungen

Übung 1

Was ist zu beachten, wenn Sie Zeilen in eine View einfügen, deren Tabelle, auf die die **SELECT**-Abfrage zugreift, keine Autoinkrement-Spalte als Schlüsselspalte aufweist?

Übung 2

Erstellen Sie die in [Listing 19.26](#) angegebene View `v_mitarbeiterkrankenkasse`:

```
CREATE VIEW v_mitarbeiterkrankenkasse AS
SELECT
    mitarbeiterid,
    name,
```

```
    vorname,  
    krankenversicherung  
FROM  
    mitarbeiter;
```

Listing 19.26 Eine View »mitarbeiterkrankenkasse« anlegen

Im Anschluss soll eine neue Zeile in die View eingefügt werden. Der neue Mitarbeiter Oliver Bergstein ist Mitglied bei der Krankenkasse MH PLUS. Beachten Sie, dass Sie einen neuen Primärschlüsselwert in die **INSERT**-Anweisung eingeben müssen.

Übung 3

Was müssen Sie beachten, wenn Sie neue Zeilen in Views einfügen, denen eine **SELECT**-Abfrage mit einem **INNER JOIN** zugrunde liegt?

Übung 4

Der Nachname der Mitarbeiterin Anja Lupin mit der Mitarbeiter-ID 6 hat sich in Lang geändert. Aktualisieren Sie die Zeile über die View *v_mitarbeiter*.

Übung 5

Was müssen Sie beachten, wenn Sie durch eine **UPDATE**-Anweisung eine Zeile einer View aktualisieren wollen, deren zugrunde liegende **SELECT**-Abfrage mit einem **INNER JOIN** die Zeilen von zwei Tabellen miteinander verbindet? Falls Sie eine PostgreSQL-Datenbank nutzen, beachten Sie bitte, dass die Formulierung von **RULEs** erforderlich ist.

Übung 6

Die Mitarbeiterin Anja Lang mit der Mitarbeiter-ID 6 scheidet aus dem Unternehmen aus. Löschen Sie die Zeile, indem Sie die View *v_mitarbeiter* nutzen.

Übung 7

Unter welchen Umständen können Sie für eine View keine **INSERT**-, **UPDATE**- oder **DELETE**-Anweisung durchführen?

Lösung zu Übung 1

Es muss sichergestellt sein, dass die View auch die Primärschlüsselspalte in der zugrunde liegenden **SELECT**-Abfrage enthält, um eine neue Zeile mit einem eindeutigen Schlüsselwert mithilfe einer **INSERT**-Anweisung einfügen zu können.

Lösung zu Übung 2

```
INSERT INTO v_mitarbeiterkrankenkasse  
  (mitarbeiterid,name,vorname,krankenversicherung)  
VALUES  
  (57,'Bergstein','Oliver','MH Plus');
```

Listing 19.27 Einen Datensatz in eine View einfügen

Lösung zu Übung 3

Die Tabelle, der Sie eine neue Zeile hinzufügen wollen, muss zu den Tabellen gehören, die am **INNER JOIN** beteiligt sind. Des Weiteren ist sicherzustellen, dass die Primärschlüsselspalte mit der zugrunde liegenden **SELECT**-Anweisung abgefragt wird. Wenn Sie eine PostgreSQL-Datenbank verwenden, müssen Sie eine **RULE** erstellen.

Lösung zu Übung 4

```
UPDATE v_mitarbeiter SET name='Lang'  
WHERE mitarbeiterid=6;
```

Listing 19.28 Eine Zeile einer View mit UPDATE aktualisieren

Lösung zu Übung 5

Wenn Sie eine View aktualisieren wollen, deren **SELECT**-Abfrage die Zeilen von zwei Tabellen über einen **INNER JOIN** verbindet, ist zu beachten, dass Sie nur die Zeilen einer der beteiligten Tabellen mit der **UPDATE**-Anweisung aktualisieren können. Auch hier ist der Primärschlüssel in der zugrunde liegenden Abfrage der View erforderlich.

Lösung zu Übung 6

```
DELETE FROM v_mitarbeiter  
WHERE mitarbeiterid=6;
```

Listing 19.29 Eine Zeile einer View löschen

Lösung zu Übung 7

Die Anweisungen **INSERT**, **UPDATE** und **DELETE** können nicht auf eine View angewendet werden, wenn die zugrunde liegende **SELECT**-Anweisung folgende Klauseln oder Aggregationen enthält:

- ▶ **DISTINCT**
- ▶ Aggregatfunktionen: **SUM**, **MIN**, **MAX**, **AVG**, **COUNT**
- ▶ **GROUP BY**

- ▶ **HAVING**
- ▶ **UNION**
- ▶ **UNION ALL**
- ▶ **LEFT OUTER JOIN** – nur Zeilen einer Tabelle können geändert werden.
- ▶ **RIGHT OUTER JOIN** – nur Zeilen einer Tabelle können geändert werden.
- ▶ **FULL OUTER JOIN** – nur Zeilen einer Tabelle können geändert werden.
- ▶ Unterabfragen in der Spaltenauswahlliste
- ▶ Skalarfunktionen
- ▶ berechnete Ausdrücke

19.4 Views entfernen oder ersetzen

Eine bestehende View aus einer Datenbank zu entfernen, ist sehr einfach. Sie leiten Ihre SQL-Anweisung mit dem Schlüsselwort **DROP** ein, gefolgt von dem Wort **VIEW**, damit die Datenbank weiß, dass sie eine View entfernen soll, und schließen den Befehl mit der Bezeichnung der View, die entfernt werden soll. [Listing 19.30](#) zeigt Ihnen beispielhaft, wie Sie die View `v_mitarbeiter` löschen:

```
DROP VIEW v_mitarbeiter;
```

Listing 19.30 Löschen einer View

Falls Sie eine bestehende View ersetzen möchten, dann ist das auch recht einfach, sofern Sie eine MySQL-, MariaDB- oder PostgreSQL-Datenbank verwenden. Die **CREATE VIEW**-Anweisung haben Sie genutzt, um Views anzulegen. Um eine View zu ersetzen, ergänzen Sie lediglich hinter dem Schlüsselwort **CREATE** die Wörter **OR REPLACE**. Die **CREATE OR REPLACE**-Anweisung steht leider nicht auf einer MS SQL Server-Datenbank zur Verfügung. Hier bleibt also nur die Möglichkeit, eine bestehende View zu löschen und mit der Anweisung **CREATE VIEW** neu zu erstellen.

```
/* Eine bestehende View ersetzen oder eine nicht bestehende erzeugen */  
CREATE OR REPLACE VIEW viewname AS selectquery;
```

Listing 19.31 Syntax zum Löschen oder Ersetzen einer View



Zusammenfassung: Eine View löschen oder ersetzen

Mit der **DROP VIEW**-Anweisung löschen Sie eine View. Diese Anweisung kann für die MySQL-, MariaDB-, PostgreSQL- und MS SQL Server-Datenbank verwendet werden. Mit der **CREATE OR REPLACE VIEW**-Anweisung ersetzen Sie eine bestehende View. Diese Anweisung kann für die MySQL-, MariaDB- und PostgreSQL-Datenbank genutzt werden.

In diesem Abschnitt haben Sie erfahren, wie Sie Views löschen oder bereits bestehende Views ersetzen. Im nächsten Kapitel zeige ich Ihnen, wie Sie für eine Tabelle einen Index festlegen, um z. B. die Ausführung von `SELECT`-Anweisungen mit `WHERE`-Klauseln zu beschleunigen und somit die Ergebnistabelle schneller von der Datenbank zurückzuerhalten.

19.4.1 Übungen

Übung 1

Löschen Sie nun die View `v_mitarbeiterkrankenkasse`, die Sie in Übung 2 aus [Abschnitt 19.3.5](#) erstellt haben.

Übung 2

Ersetzen Sie die in [Abschnitt 19.1](#) angelegte View `v_mitarbeiterkontakt` durch eine gleichnamige View, die auch die E-Mail-Adressen der Mitarbeiter enthält. Diese Übung können Sie nur bearbeiten, wenn Sie eine MySQL-, MariaDB- oder eine PostgreSQL-Datenbank verwenden.

Lösung zu Übung 1

```
DROP VIEW v_mitarbeiterkrankenkasse;
```

Listing 19.32 Eine View löschen

Lösung zu Übung 2

```
CREATE OR REPLACE VIEW v_mitarbeiterkontakt
AS
SELECT
    m.mitarbeiterid,
    m.name,
    m.vorname,
    k.festnetz,
    k.mobilnetz,
    k.email
FROM
    mitarbeiter m INNER JOIN kontakt k
ON
    m.mitarbeiterid=k.mitarbeiterid;
```

Listing 19.33 Eine View verändern

Kapitel 20

Abfragen mit einem Index optimieren

Abfragen auf Tabellen, die sehr viele Zeilen enthalten, nehmen schon mal etwas Zeit in Anspruch. Das betrifft insbesondere Abfragen mit einer WHERE-Klausel, mit der gezielt anhand eines Kriteriums nach Zeilen gesucht wird. Aber auch JOINS, die Zeilen aus Tabellen verbunden abfragen, können lange dauern. Glücklicherweise gibt SQL Ihnen Werkzeuge an die Hand, mit denen Sie das Abfragen optimieren können.

Eine **SELECT**-Abfrage kann lange dauern, wenn Sie sehr viele Datensätze in einer Tabelle speichern und dann in einer **WHERE**-Klausel Filterkriterien auf Spalten anwenden, die keiner Primärschlüsselspalte entsprechen. Das gilt auch, wenn Sie Zeilen aus Tabellen über **JOINS** verbinden oder eine **ORDER BY**-Klausel verwenden.

Hier hilft uns ein Index, den wir für eine oder mehrere Spalten einer Tabelle anlegen können, um mit großen Datenaufkommen performant zu arbeiten.

20.1 Einführung

Falls Sie sich fragen, ob das Ausführen eines Index kompliziert ist, kann ich Sie beruhigen: Sie alle kennen bereits das Verfahren, in einem Telefonbuch mithilfe eines Nachnamens nach einer Person zu suchen. Ein Telefonbuch ist in der Regel alphabetisch sortiert. Hier haben wir es mit einem Index zu tun, der es Ihnen ermöglicht, sehr schnell gemäß dem Kriterium des Nachnamens nach einer Telefonnummer zu suchen. Oder betrachten wir in Abbildung 20.1 zwei Adressbücher, die mit und ohne Index ausgestattet sind.

Das erste Adressbuch wird ohne Index genutzt. In ihm schreiben Sie einfach ohne ein Ordnungskriterium die Adress- und Kontaktdaten Ihres Freundeskreises auf. Wenn Sie nach einem bestimmten Kontakt suchen, werden Sie ein wenig Zeit aufwenden müssen, da Sie alle Seiten des Adressbuchs durchblättern müssen, um den besagten Kontakt zu finden. Sie können natürlich Glück haben, wenn sich der gesuchte Kontakt auf der ersten oder zweiten Seite befindet. Wenn Sie Pech haben, ist er auf der

letzten notiert. Und wenn Sie etwa wissen möchten, wie viele Namen Ihrer Freunde mit dem Buchstaben *N* anfangen, müssen Sie auf jeden Fall das ganze Buch durchgehen.

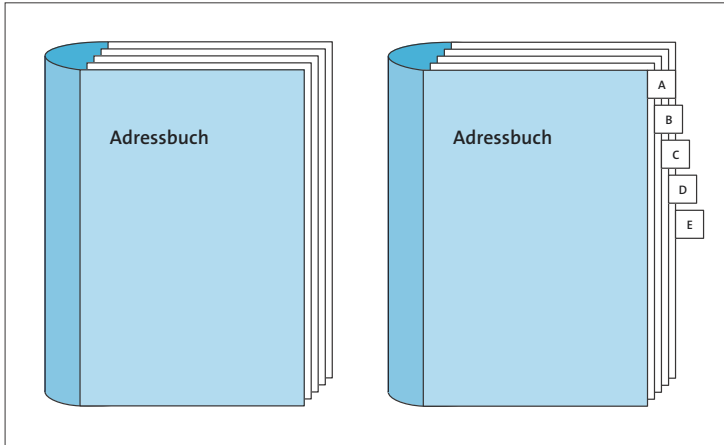


Abbildung 20.1 Adressbuch ohne und mit Index nutzen

Verwenden Sie hingegen ein Adressbuch mit einem alphabetischen Index, wie er in Abbildung 20.1 rechts dargestellt ist, so werden Sie den gesuchten Kontakt viel schneller finden. Ein Index in einer Datenbank macht es ganz ähnlich wie wir, wenn wir nach einer bestimmten Information suchen. Datenbanken verfügen zwar über keine Marker wie in einem Adressbuch, aber über Tabellen, in denen Indexe zu Tabellen hinterlegt werden.

Es gibt unterschiedliche Indextypen, die Sie im Zusammenhang mit Datenbanken verwenden können. Im Rahmen unseres Einstiegs betrachten wir nur den am häufigsten genutzten Indextyp, den sogenannten *B-Tree-Index*. Ein B-Tree-Index ist wie das Verzeichnis unseres Adressbuchs organisiert. Es handelt sich um eine baumartige Struktur.

Abbildung 20.2 zeigt Ihnen den Index in einer Baumstruktur. Die Wurzel des Baums ist mit der Bezeichnung gleichzusetzen. In diesem Fall handelt es sich um einen *Adressbuchindex*. Unterhalb der Wurzel sehen Sie die Abzweigungen *A*, *B*, *C* und *D*. Unterhalb dieser Buchstaben finden Sie wiederum Personen, deren Nachnamen mit den jeweiligen Buchstaben beginnen.

SQL bietet die Möglichkeit, eine oder mehrere Spalten von Tabellen mit einem Index zu versehen, um die Abfragegeschwindigkeit z. B. bei gefilterten Abfragen oder auch bei gruppierten Abfragen zu erhöhen.

Auch wenn Sie Tabellen mit einem **JOIN** verbunden abfragen, ist es sinnvoll, einen Index zu nutzen. Wie in einer **WHERE**-Klausel wird auch in der **ON**-Klausel eine Bedingung ausgewertet, die Schlüsselwerte auf Übereinstimmung vergleicht.

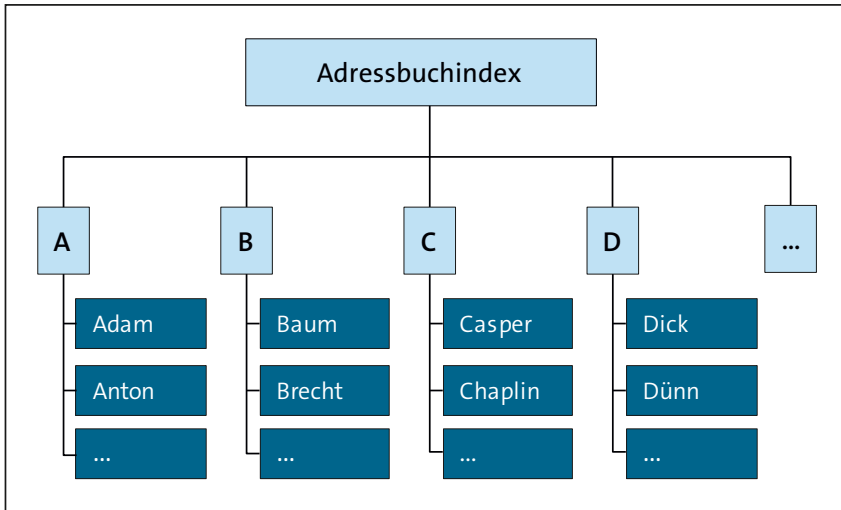


Abbildung 20.2 Der Adressbuchindex in einer Baumstruktur dargestellt

Eigentlich kennen Sie schon längst indexierte Spalten von Tabellen: nämlich die Primärschlüsselspalten, die eine Zeile einer Tabelle eindeutig identifizieren. Eine Primärschlüsselspalte stellt also nicht nur sicher, dass die Spaltenwerte eindeutig sind, sondern legt automatisch einen Index an.

Achtung! Index ist nicht standardkonform

Der SQL-Standard kennt keinen Index. Die Erstellung eines Index kann also von Datenbanksystem zu Datenbanksystem stark abweichen. Ich kann Ihnen daher nur einen ersten Überblick über die Funktionsweise liefern. Über den genauen Einsatz informiert Sie die Dokumentation Ihrer Datenbank.



Für dieses Kapitel benötigen wir Tabellen mit möglichst vielen Zeilen.

Die Tabellen *kreditinstitut* und *arbeitszeit* sind bereits Teil des SQL-Skripts, das Sie verwendet haben, um die Übungsdatenbank anzulegen. Jetzt müssen wir noch die benötigten Zeilen einfügen.

In den Ordnern der jeweiligen Datenbanken befinden sich folgende Skripte, die Sie bitte nacheinander in der MySQL Workbench öffnen und ausführen:

- ▶ *importsript_kreditinstitut.sql*
- ▶ *importsript_arbeitszeit.sql*

Je nach Leistungsfähigkeit Ihres Computers kann es ein paar Minuten dauern. Wenn Sie eine MariaDB-, MS SQL Server- oder PostgreSQL-Datenbank verwenden, führen Sie bitte analog die gleichen Schritte im SQL Server Management Studio, in HeidiSQL bzw. in pgAdmin aus.

20.2 Syntax: Index erstellen

Einen einfachen Index erstellen Sie mit der Anweisung **CREATE INDEX**:

CREATE INDEX

indexname

ON

tabellenname (spaltenname1,spaltenname2,...);

Listing 20.1 Index mit erlaubten mehrfach vorkommenden Einträgen für eine Spalte einer Tabelle

Sie beginnen mit dem bereits bekannten Schlüsselwort **CREATE**, gefolgt vom Schlüsselwort **INDEX** und schließlich einer Indexbezeichnung, die hier *indexname* lautet. Es folgen das Schlüsselwort **ON** und der Name der Tabelle, die Sie mit einem Index ausstatten möchten. In einfachen Klammern geben Sie die Spalten an, die durch das Datenbanksystem indexiert werden sollen. Diese Form des Index erlaubt mehrfach vorkommende gleiche Werte innerhalb der Spalten, die Sie als Index für eine Tabelle definiert haben. Die in der Syntax beschriebene Vorgehensweise zum Erstellen eines Index können Sie für alle wichtigen Datenbanken verwenden.

Es gibt auch eine Möglichkeit, einen eindeutigen Index für eine oder mehrere Spalten festzulegen. Die Spaltenwerte innerhalb des Index müssen dabei eindeutig sein. Die Syntax zur Erstellung eines eindeutigen Index, die Sie in [Listing 20.2](#) sehen, gilt für alle vier hier behandelten Datenbanken:

CREATE UNIQUE INDEX

indexname

ON

tabellenname (spaltenname1,spaltenname2,...);

Listing 20.2 Index mit eindeutigen Einträgen für eine Spalte einer Tabelle erstellen

Um Ihnen zu zeigen, welche Leistungssteigerung bei einer Abfrage das Anlegen eines Index bringt, benötigen wir eine Tabelle mit einer großen Anzahl von Zeilen. Die Tabelle existiert bereits unter dem Namen *kreditinstitutneu*. In dieser Tabelle sind allerdings noch keine Zeilen vorhanden, weil Sie sonst zu Beginn beim Anlegen der Datenbank eine längere Zeit benötigt hätten.

20.3 Eine Tabelle mit vielen Zeilen generieren

Sie haben in [Kapitel 18](#) Unterabfragen (Subqueries) kennengelernt. Jetzt werden wir eine Unterabfrage nutzen, um Zeilen in die Tabelle *kreditinstitutneu* einzufügen. Schauen Sie sich dazu [Listing 20.3](#) an:

INSERT INTO

```
kreditinstitutneu (bankid,bankleitzahl,bezeichnung,plz,ort)
```

SELECT

```
TOP 3000000 /* MS SQL Server */
```

```
k1.bankid,  
k1.bankleitzahl,  
k1.bezeichnung,  
k1.plz,  
k1.ort
```

FROM

```
kreditinstitut k1 CROSS JOIN kreditinstitut k2
```

```
LIMIT 3000000; /* MySQL, MariaDB */
```

```
FETCH FIRST 3000000 ROWS ONLY; /* PostgreSQL */
```

Listing 20.3 3.000.000 Zeilen mit einem CROSS JOIN in die Tabelle »kreditinstitutneu« einfügen

Sie beginnen die **INSERT**-Anweisung wie gewohnt mit der **INSERT INTO**-Klausel, gefolgt von einer kommaseparierten Liste von Spalten, in die Sie Spaltenwerte einfügen möchten.

Hinter der Spaltenauswahlliste folgt eine **SELECT**-Abfrage. Diese **SELECT**-Abfrage enthält in der Spaltenauswahlliste exakt die gleichen Spalten in der Reihenfolge, wie sie in der kommaseparierten Liste der **INSERT INTO**-Klausel angegeben wurden. Es müssen also die Reihenfolge und die Spalten der Spaltenauswahlliste der **SELECT**-Abfrage und der **INSERT INTO**-Klausel übereinstimmen. Ohne die Begrenzung würde diese Abfrage versuchen, 700.000.000 Zeilen in die Tabelle *kreditinstitutneu* einzufügen. Hier fehlen also nur noch 300.000.000 Zeilen bis zu einer Milliarde Zeilen. Das würde für Ihren Rechner eine sehr, sehr große Herausforderung darstellen.

Um die Anforderung etwas geringer zu halten, sehen Sie in der **INSERT**-Anweisung in Listing 20.3 eine **TOP**-, **LIMIT**- und **FIRST ROWS ONLY**-Variante, die jeweils um einen Kommentar ergänzt wird. Diese Varianten dienen dazu, die Zeilen zu begrenzen, die von der Unterabfrage zurückgegeben werden. Falls erforderlich, können Sie natürlich die Begrenzung erweitern, um die Leistungssteigerung von z. B. **SELECT**-Abfragen mit **WHERE**-Klauseln noch besser erkennen zu können. Meine Empfehlung lautet an dieser Stelle jedoch, Ihrem Rechner nicht zu viel zuzumuten und maximal 5.000.000 Zeilen in die Tabelle *kreditinstitutneu* einzufügen.

Ein Einfügevorgang mit einer sehr großen Anzahl von Zeilen wird Ihren Rechner stark einschränken oder gar zum Absturz bringen. Beachten Sie diesbezüglich bitte meine Empfehlung. Ich gehe an dieser Stelle davon aus, dass der Rechner, den Sie verwenden, eine gewisse Leistungsfähigkeit aufweist. Sollten Sie mit sehr alter Hardware arbeiten, reduzieren Sie die Anzahl der Zeilen nochmals.

Wenn Sie die Zeilen mit der **INSERT**-Anweisung aus [Listing 20.3](#) eingefügt haben, können wir zu [Abschnitt 20.4](#) übergehen.



MySQL Workbench: Maximale Abfragedauer einstellen

Die maximale Abfragedauer einer Abfrage ist in der MySQL Workbench auf 3 Sekunden eingestellt. Für unser Beispiel ist das zu wenig. Um diesen Wert anzupassen, klicken Sie im Menü auf **EDIT** und dann auf **PREFERENCES**. In den Einstellungen der MySQL Workbench finden Sie links in der Baumstruktur den Eintrag **SQL EDITOR**. Im Abschnitt **MySQL SESSION** müssen Sie den Eintrag **DBMS CONNECTION TIMEOUT INTERVAL** auf einen höheren Wert stellen. Wenn Sie auf Nummer sicher gehen wollen, tragen Sie dort 60 Sekunden ein und klicken auf **OK**.

20.4 Einen Index für eine Tabelle anlegen

Bevor wir die Tabelle *kreditinstitutneu* mit einem Index ausstatten, werden wir eine Abfrage formulieren, in der wir die Datensätze eines Kreditinstituts mit der Bankleitzahl 37050198 ermitteln.

Achtung: Da wir die Zeilen aus der Abfrage aus [Listing 20.3](#) nur ausschnittsweise eingefügt haben, kann es sein, dass diese Bankleitzahl nicht in der Tabelle *kreditinstitutneu* vorhanden ist. Prüfen Sie daher zunächst, ob diese Bankleitzahl überhaupt existiert. Falls nicht, führen Sie mit einer **SELECT**-Abfrage ohne **WHERE**-Klausel eine Abfrage durch, in der die ersten zehn Zeilen zurückgegeben werden. Notieren Sie eine dieser Bankleitzahlen, und verwenden Sie sie in den folgenden Beispielen weiter.

Kommen wir nun aber zur Abfrage in [Listing 20.4](#):

```
SELECT
    *
FROM
    kreditinstitutneu
WHERE
    bankleitzahl=37050198;
```

Listing 20.4 Gefilterte Abfrage auf die Tabelle »kreditinstitutneu« ohne Index

Als Ergebnis liefert Ihnen die Datenbank den Datensatz zurück, den Sie in [Tabelle 20.1](#) sehen. Diese Abfrage benötigt auf einer MySQL-Datenbank bei der ersten Durchführung einige Sekunden, um das Ergebnis zu präsentieren.

bankid	bankleitzahl	bezeichnung	plz	ort
4360	37050198	Sparkasse KölnBonn	50667	Köln

Tabelle 20.1 Ergebnis einer SELECT-Abfrage, die in einer WHERE-Klausel auf Gleichheit mit der Bankleitzahl 37050198 prüft

Wenn Sie die Abfrage aus [Listing 20.4](#) erneut ausführen, erscheint das Ergebnis jedoch sofort. Der Grund dafür ist, dass die MySQL-Datenbank die Abfrage sowie die dazugehörige Ergebnistabelle in einem *Cache* speichert. Führen Sie exakt dieselbe Abfrage mehrmals durch, wird einfach das Ergebnis aus dem Cache geholt und angezeigt. Das geht natürlich sehr schnell. Grundsätzlich ist dieses Verhalten sehr zu begrüßen. In diesem Beispiel sorgt es jedoch dafür, dass ich Ihnen die Vorteile eines Index nicht sehr gut darstellen kann.

Um dennoch die Vorteile eines Index deutlich zu machen, verwenden wir hier ausnahmsweise eine für die MySQL- bzw. MariaDB-Datenbank leicht modifizierte Abfrage. Vor dem *-Auswahloperator notieren wir hierzu `SQL_NO_CACHE`, um auf den internen Cache bei der Abfrage zu verzichten:

```
SELECT
    SQL_NO_CACHE *
FROM
    kreditinstitutneu
WHERE
    bankleitzahl=37050198;
```

Listing 20.5 Gefilterte Abfrage auf die Tabelle »kreditinstitutneu« ohne Cache

Die MySQL- und die MariaDB-Datenbank benötigen für diese Abfrage ein paar Sekunden – jedenfalls genug, um Ihnen die Vorteile eines Index zu erläutern.

Die PostgreSQL-Datenbank benötigt konstant etwa eine Sekunde, um die Abfrage aus [Listing 20.4](#) durchzuführen. Das reicht uns an dieser Stelle ebenfalls aus.

Wenn Sie eine MS SQL Server-Datenbank verwenden, so führen Sie stets vor der Ausführung einer Abfrage die Anweisung `DBCC DROPCLEANBUFFERS` aus [Listing 20.6](#) aus, um den Puffer bzw. den Cache zu leeren:

```
DBCC DROPCLEANBUFFERS;
```

Listing 20.6 Den Puffer einer MS SQL Server-Datenbank bereinigen

Die Datenbank benötigt, wenn der Puffer geleert wurde, für die Abfrage aus [Listing 20.4](#) ebenfalls ein paar Sekunden. Auch das reicht aus, um Ihnen die Vorteile eines Index auf einer Tabelle aufzuzeigen.

Jetzt legen Sie mit der **CREATE INDEX**-Anweisung aus [Listing 20.7](#) einen Index auf die Spalte *bankleitzahl* der Tabelle *kreditinstitutneu* an. Das Anlegen des Index dauert auf meinem Rechner für eine MySQL-Datenbank etwa 30 Sekunden.

CREATE INDEX

```
index_bankleitzahl  
ON  
kreditinstitutneu (bankleitzahl);
```

Listing 20.7 Ein Index wird auf die Spalte »bankleitzahl« der Tabelle »kreditinstitutneu« definiert.

Gleich im Anschluss daran führen Sie die **SELECT**-Abfrage noch einmal aus. Prüfen Sie zunächst, ob Sie den Cache Ihrer Datenbank ausschalten müssen. Falls Sie eine PostgreSQL-Datenbank verwenden, müssen Sie nichts weiter unternehmen, um das Ergebnis replizierbar zu machen. Wenn Sie eine MS SQL Server-Datenbank einsetzen, müssen Sie die Anweisung aus [Listing 20.6](#) vor der Ausführung der **SELECT**-Anweisung aus [Listing 20.4](#) ausführen, um ein replizierbares Ergebnis zu erhalten. Wer eine MySQL- oder MariaDB-Datenbank nutzt, nutzt den leicht modifizierten Aufruf aus [Listing 20.5](#).

Für alle drei Datenbanken gilt, dass Sie nun deutlich schneller an das Ergebnis der Abfragen gelangen.

20.5 Einen Index über mehrere Spalten anlegen

Wie bereits in der Syntax aus [Listing 20.1](#) und [Listing 20.2](#) dargestellt, können Sie einen Index auch über mehrere Spalten einer Tabelle anlegen. Im Folgenden erstellen wir für die Tabelle *arbeitszeit* einen Index. Dieser soll die Spalten *jahr*, *monat*, *tag* und *mitarbeiterid* indexieren.

Wenn Sie einen Index über die Werte der Spalten *jahr*, *monat*, *tag* und *mitarbeiterid* anlegen, wird der Index in einer baumartigen Struktur angelegt. Die Werte aus der Spalte *jahr* verweisen auf Werte der Spalte *monat*, die Werte der Spalte *monat* wiederum verweisen auf die Werte der Spalte *tag*. Um die Monate für ein bestimmtes Jahr in einer **WHERE**-Klausel zu ermitteln, sucht die Datenbank im Index zunächst nach dem betreffenden Jahr. Wenn dieser Wert gefunden wurde, sucht sie nach dem Spaltenwert des Monats, der durch die Indexstruktur dem Jahr zugewiesen ist.

Über mehrere Spalten lassen sich durch eine derartige Indexstruktur sehr schnell Daten abfragen.

In unserem Beispiel nutzen wir einen eindeutigen Index, den wir mit dem Schlüsselwort **UNIQUE** festlegen. Wenn Sie einen eindeutigen Index nutzen, muss sichergestellt

sein, dass die Spaltenwerte, über die er erstellt wird, nicht mehrfach vorkommen. Existieren in einer Spalte, über die ein eindeutiger Index angelegt werden soll, Spaltenwerte mehrfach, so kann kein eindeutiger Index angelegt werden.

Die Syntax des Aufrufs sehen Sie in [Listing 20.8](#):

```
CREATE UNIQUE INDEX index_arbeitszeit
ON arbeitszeit (jahr,monat,tag,mitarbeiterid);
```

Listing 20.8 Einen eindeutigen Index über mehrere Spalten erstellen

UNIQUE oder einen mehrdeutigen Index verwenden?

Welcher Index auf Spalten einer Tabelle angewendet wird, hängt davon ab, ob die jeweiligen Spalten gleiche Spaltenwerte enthalten oder nicht. Ob das der Fall sein darf, hängt davon ab, wie Sie eine Tabelle modelliert haben. Bei einer Tabelle, die produktiv genutzt wird, müssen Sie also zuerst überprüfen, ob die Spaltenwerte einer Spalte oder auch mehrerer Spalten in Kombination miteinander gleiche Werte enthalten können.

Um zu prüfen, ob die Spalten einer produktiven Tabelle nur eindeutige Spaltenwerte enthalten, können Sie eine einfache Abfrage nutzen, in der Sie eine **GROUP BY**-Klausel in Kombination mit einer **HAVING**-Klausel für die betreffende Spalte anwenden:

```
SELECT spalte FROM tabelle GROUP BY spalte
HAVING COUNT(spalte)>1;
```

Diese Prüfabfrage können Sie verwenden, um produktiv genutzte Daten zu analysieren. Das Ergebnis der Analyse wird benutzt, um abzuleiten, dass ein eindeutiger (**UNIQUE**) oder mehrdeutiger Index erstellt werden kann.

Beachten Sie an dieser Stelle das Wort »kann«: Entscheidend ist, wie eine Spalte im Modell definiert und letztendlich mit der **CREATE TABLE**-Anweisung umgesetzt wurde. Bevor Sie einen eindeutigen oder mehrdeutigen Index anlegen, müssen Sie also immer prüfen, wie die jeweiligen Spalten im Modell festgelegt wurden.

Wenn eine oder mehrere Spalten in einer **CREATE TABLE**-Anweisung mit einer **UNIQUE**-Einschränkung versehen wurden, kann ein **UNIQUE**-Index auf Spalten angewendet werden. In [Abschnitt 4.5.2](#) haben Sie die **UNIQUE**-Einschränkung im Zusammenhang mit der **CREATE TABLE**-Anweisung bereits kennengelernt. Sie stellt sicher, dass nur Zeilen eingefügt werden dürfen, wenn die Spaltenwerte einer Spalte, für die eine **UNIQUE**-Einschränkung festgelegt wurde, eindeutig sind.

Natürlich können Sie den Index auch mit einer **SELECT**-Abfrage nutzen. In [Listing 20.9](#) sehen Sie eine **SELECT**-Abfrage mit einer **WHERE**-Klausel:



```
SELECT
*
FROM
    arbeitszeit
WHERE
    jahr=2012
    AND
    monat=7
    AND
    tag=2
    AND
    mitarbeiterid=4;
```

Listing 20.9 In einer SELECT-Abfrage einen Index über mehrere Spalten nutzen

In der **WHERE**-Klausel werden mehrere Bedingungen mit einem **AND**-Operator logisch verknüpft. Die Spalten, die hier ausgewertet werden, wurden exakt in der Reihenfolge auch als Index hinterlegt.

Als Ergebnis liefert Ihnen die Datenbank exakt eine Zeile zurück, die Sie in Tabelle 20.2 sehen.

arbeitszeit	arbeitstag	jahr	monat	tag	anzahlstunden	mitarbeiterid
3	2012-07-02	2012	7	2	8	4

Tabelle 20.2 Ergebnis für eine SELECT-Abfrage, deren WHERE-Klausel auf einen Index zugreift

Egal, welche der hier behandelten Datenbanken Sie verwenden – sie nutzen alle den zuvor angelegten Index, um die Abfrage durchzuführen.

Die Verwendung des Index durch eine Datenbank funktioniert wie folgt: Die Datenbank sucht im Index zuerst nach Merkmalen, die dem Jahr 2012 entsprechen. Damit hat sie schon ihre erste Eingrenzung vorgenommen. Innerhalb dieser Ergebnisliste aus dem Index wird jetzt geprüft, welche Datensätze mit `monat=7` hinterlegt sind. Es folgen schließlich der Tag und die Mitarbeiter-ID, die innerhalb des Index ausgewertet werden.

Sie können sich so eine Auswertung eines Index wie einen Baum vorstellen. Stellen Sie sich einfach vor, Ihr Ziel ist einer der vielen abzweigenden Äste. Sie springen vom Stamm zu immer kleiner und feiner verzweigenden Ästen, bis Sie am Ziel angelangt sind.

20.6 Den Index einer Tabelle löschen

Wenn Sie bestimmte **SELECT**-Abfragen nicht mehr benötigen, die auf einen Index zugreifen, so können Sie ihn auch wieder mit einer **DROP INDEX**-Anweisung löschen.

Index löschen

Bevor Sie einen Index löschen, stellen Sie stets sicher, dass er in keiner Abfrage mehr verwendet wird.

Wenn Sie einen Index anlegen, wird Speicherplatz für die Indextabelle benötigt. Sollte ein Index nicht mehr benötigt werden, ist es daher tatsächlich ratsam, ihn wieder zu löschen. In [Listing 20.10](#) sehen Sie die Syntax, mit der Sie einen Index für die hier behandelten Datenbanken löschen können:

```
/* MySQL, MariaDB und MS SQL Server */
DROP INDEX indexname ON tabellenname;
/* PostgreSQL */
DROP INDEX indexname;
```

Listing 20.10 Syntax zum Löschen eines Index

Den zuvor erstellten Index mit der Bezeichnung *index_bankleitzahl* werden wir gemäß der Syntax aus [Listing 20.10](#) nun wieder löschen. Zuerst zeige ich Ihnen, wie Sie den Index aus einer MySQL-, MariaDB- oder MS SQL Server-Datenbank wieder entfernen:

```
DROP INDEX index_bankleitzahl ON kreditinstitutneu;
```

Listing 20.11 Einen Index in einer MySQL-, MariaDB oder MS SQL Server-Datenbank löschen

Falls Sie eine PostgreSQL-Datenbank verwenden, nutzen Sie hingegen die Anweisung aus [Listing 20.12](#), um einen Index zu entfernen:

```
DROP INDEX index_bankleitzahl;
```

Listing 20.12 Einen Index auf einer PostgreSQL-Datenbank löschen

20.7 Fremdschlüsselspalten indexieren

Wenn Tabellen in Beziehung zu anderen Tabellen stehen, ist es sinnvoll, bereits während der Tabellenerstellung die Fremdschlüsselspalten, die wiederum auf Primärschlüsselspalten referenzieren, mit einem Index auszustatten. Eine MySQL- und MariaDB-Datenbank erlaubt es, Spalten, die in einer **CREATE TABLE**-Anweisung definiert werden, auf Tabellenebene direkt mit einem Index auszustatten.



In einer CREATE TABLE-Anweisung für Spalten einen Index festlegen

MySQL-, MariaDB- und MS SQL Server-Datenbanken erlauben es bereits während der Tabellenerstellung, ausgewählte Spalten mit einem Index zu versehen. Die PostgreSQL-Datenbank verfügt nicht über diese Option. Hier müssen Sie nach der Tabellenerstellung für ausgewählte Spalten (z. B. Fremdschlüsselspalten) einen Index festlegen.

Wenn der Index nicht bereits während der Tabellenerstellung mit einer **CREATE TABLE**-Anweisung erstellt werden soll, können Sie die Anweisungen **CREATE TABLE** und **CREATE INDEX** einfach hintereinander ausführen.

In Kapitel 9, »Datenmodelle in Tabellen überführen«, haben Sie Tabellen mit **CREATE TABLE**-Anweisungen erstellt, die in Beziehung zueinander stehen. Wie Sie Zeilen aus Tabellen mit einem **INNER JOIN** verbunden abfragen, haben Sie in Abschnitt 10.2 erfahren. Wenn Beziehungen zwischen Tabellen existieren und Sie eine **SELECT**-Abfrage mit einem **INNER JOIN** auf die Tabellen ausführen, werden in der **ON**-Klausel die beteiligten Schlüsselwerte (Primär- und Fremdschlüsselwerte) verglichen. Auch hier gilt also, dass die Datenbank Spaltenwerte einer Tabelle jeweils mit Spaltenwerten einer anderen Tabelle vergleichen muss, um die Zeilen der beteiligten Tabellen zu verbinden. Als Beispiel sehen wir uns zwei Tabellen an, in der Bestellungen von Kunden verwaltet werden.

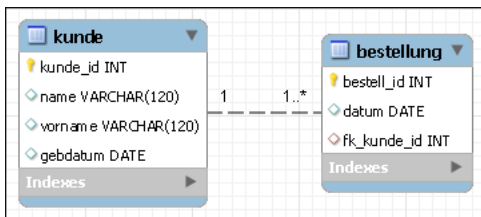


Abbildung 20.3 Die Tabellen »kunde« und »bestellung«

Im Modell sehen Sie, dass die Spalte `fk_kunde_id` aus der Tabelle `bestellung` auf die Spalte `kunde_id` der Tabelle `kunde` referenziert. Wenn die Tabelle `bestellung` mehrere Millionen Zeilen enthält, die allesamt auf Tausende von Kunden referenzieren, kann eine Abfrage mit einem **JOIN** länger dauern. Die Spalte `kunde_id` der Tabelle `kunde` ist nicht kritisch. Die Spalte wurde im Modell als Primärschlüsselspalte definiert. Für eine Primärschlüsselspalte einer Tabelle wird automatisch ein Index erstellt.

Problematisch ist an dieser Stelle die Spalte `fk_kunde_id` der Tabelle `bestellung`, deren Fremdschlüsselwerte auf die Primärschlüsselwerte der Tabelle `kunde` referenzieren. Diese Fremdschlüsselspalte `fk_kunde_id` ist nicht mit einem Index versehen. Eine Abfrage wie in Listing 20.13 mit einer **ON**-Klausel, in der die Spaltenwerte der Spalte `fk_kunde_id` der Tabelle `bestellung` mit den Spaltenwerten der Spalte `kunde_id` der

Tabelle *kunde* auf Gleichheit geprüft werden, könnte also schon einmal etwas länger dauern.

```
SELECT k.name, b.bestell_id FROM kunde k
INNER JOIN bestellung b
ON k.kunde_id=b.fk_kunde_id;
```

Listing 20.13 Tabellen mit einem JOIN verbunden abfragen

Als Nächstes sehen wir uns die erforderlichen **CREATE TABLE**-Anweisungen an, mit denen wir die Tabellen *kunde* und *bestellung* auf der Datenbank anlegen. Das meiste kennen Sie bereits.

Listing 20.14 hält für Sie also nichts Neues bereit. Es handelt sich um eine **CREATE TABLE**-Anweisung, mit der wir die Tabelle *kunde* erstellen, die eine starke Entität repräsentiert. Erwähnenswert ist hier lediglich, dass die Primärschlüsseleinschränkung für die Spalte *kunde_id* auf Tabellenebene festgelegt wurde.

```
CREATE TABLE kunde (
    kunde_id INT NOT NULL,
    name VARCHAR(120),
    vorname CHAR(120),
    gebdatum DATE,
    PRIMARY KEY (kunde_id)
);
```

Listing 20.14 Die Tabelle »kunde« wird mit einer **CREATE TABLE**-Anweisung erstellt.

In Listing 20.15 gibt es hingegen etwas Neues: In Zeile 6 der **CREATE TABLE**-Anweisung für die Tabelle *bestellung* sehen Sie, dass hier mit dem Schlüsselwort **INDEX** ein Index festgelegt wird. Hinter dem Schlüsselwort **INDEX** wird der Name des zu erstellenden Index angegeben. Für den zu erstellenden Index habe ich hier den Namen *fk_kunde_id_index* gewählt. Zu guter Letzt geben Sie in Klammern die Spalte an, für die ein Index erstellt werden soll. Hier handelt es sich um die Spalte *fk_kunde_id*, die als Fremdschlüsselspalte auf die Spalte *kunde_id* der Tabelle *kunde* referenziert.

Wenn Sie eine PostgreSQL-Datenbank nutzen, können Sie leider keinen Index in der **CREATE TABLE**-Anweisung definieren. Hier müssen Sie zunächst die Tabelle anlegen und dann mit einem Index ausstatten.

```
CREATE TABLE bestellung (
    bestell_id INT,
    datum DATE,
    fk_kunde_id INT,
    PRIMARY KEY (bestell_id),
    INDEX fk_kunde_id_index (fk_kunde_id),
```

```

CONSTRAINT fk_kunde_id FOREIGN KEY (fk_kunde_id)
REFERENCES kunde (kunde_id)
);

```

Listing 20.15 Die Tabelle »bestellung« mit einer **CREATE TABLE**-Anweisung erstellen

Wenn Sie in der **CREATE TABLE**-Anweisung keinen Index festlegen wollen, können Sie alternativ nach der **CREATE TABLE**-Anweisung eine **CREATE INDEX**-Anweisung aufrufen, um für die Fremdschlüsselspalte einen Index festzulegen. [Listing 20.16](#) enthält eine **CREATE INDEX**-Anweisung, um für die Spalte *fk_kunde_id* einen Index zu erstellen.

```

CREATE INDEX
  fk_kunde_id_index
ON
  bestellung (fk_kunde_id);

```

Listing 20.16 Explizit einen Index für eine Fremdschlüsselspalte festlegen



Zusammenfassung: Spalten von Tabellen indexieren

Die Performance von **SELECT**-Abfragen können Sie durch die Indexierung von Spalten erheblich steigern. Alle Anweisungen, die in Kombination mit einer **WHERE**-Klausel arbeiten, profitieren von einem solchen Index.

Wird ein **JOIN** beliebigen Typs (bis auf den **CROSS JOIN**) verwendet, so ist es sinnvoll, die Fremdschlüsselspalte, die auf die Primärschlüsselspalte referenziert, zu indexieren. In der **ON**-Klausel eines **JOINS** wird wie in einer **WHERE**-Klausel eine Bedingung auf Gleichheit von Primärschlüsselwerten und Fremdschlüsselwerten ausgewertet. Eine Primärschlüsselspalte wird automatisch indiziert. Die Fremdschlüsselspalte sollte entweder bereits während der Tabellenerstellung mit einer **CREATE TABLE**-Anweisung (wenn dies von dem Datenbanksystem unterstützt wird) oder manuell mit einem **CREATE INDEX**-Befehl ausgestattet werden, um die Geschwindigkeit mit einem **JOIN** zu erhöhen.

Ein Index wird mit der **CREATE INDEX**-Anweisung erstellt. Einer Tabelle wird dann eine Struktur (eine weitere Tabelle) zugeordnet, die den Index enthält. Ein Index kann über eine oder mehrere Spalten einer Tabelle gebildet werden.

Die hier betrachteten Indexarten bieten die Möglichkeiten, einen Index über Spalten mit eindeutigen Spaltenwerten oder über Spalten mit nicht eindeutigen Spaltenwerten anzulegen. Ein eindeutiger Index wird mit dem Schlüsselwort **UNIQUE** angelegt.

Wenn ein Index nicht mehr erforderlich ist, weil z. B. eine **SELECT**-Abfrage, die auf einen Index zugreift, keine Verwendung mehr findet, so können Sie ihn mit der Anweisung **DROP INDEX** entfernen.

20.8 Übungen

Übung 1

In dieser Übung ist es unser Ziel, die Abfragegeschwindigkeit für die Arbeitszeiten der Mitarbeiter zu erhöhen. Um die Anforderung zu erfüllen, legen Sie für die Spalte *mitarbeiterid* der Tabelle *arbeitszeit* einen einfachen Index mit der Bezeichnung *index_arbeitszeit_mitarbeiter* an.

Übung 2

Die folgende Anforderung hat zum Ziel, die Zeit für die Abfrage der Kreditinstitute zu minimieren. Diese Abfrage verwendet in der *WHERE*-Klausel die Spalten *ort* und *plz* und prüft diese auf Gleichheit mit einem Ort bzw. einer Postleitzahl. Die beiden Bedingungen müssen beide erfüllt sein, damit es die Datensätze in die Ergebnisliste schaffen.

Im [Listing 20.17](#) sehen Sie die Abfrage, deren Abfragezeit zu optimieren ist:

```
SELECT * FROM kreditinstitutneu WHERE ort='Berlin' AND plz=10789;
```

Listing 20.17 SELECT-Abfrage, deren Abfragezeit zu optimieren ist

Beachten Sie auch in dieser Übung, dass es gegebenenfalls erforderlich ist, einen Ort und eine Postleitzahl aus der Tabelle *kreditinstitutneu* zu ermitteln. Nutzen Sie auch hier eine *SELECT*-Anweisung, in der Sie die Anzahl der zurückgegebenen Zeilen auf 10 eingrenzen.

Um das Ziel zu erreichen, soll ein Index über die Spalten *ort* und *plz* der Tabelle *kreditinstitutneu* erstellt werden. Der Index soll den Namen *index_plzort* erhalten.

Übung 3

Entfernen Sie die soeben erstellten Indexe *index_arbeitszeit* und *index_plzort* wieder.

Übung 4

In [Abbildung 20.4](#) wurde das Modell aus [Abbildung 20.3](#) um eine weitere Tabelle erweitert. Es handelt sich um die Tabelle *bestellposition*. Die Tabelle *bestellposition* enthält wiederum eine Fremdschlüsselspalte *fk_bestellung_id*, die auf die Primärschlüsselspalte *bestell_id* der Tabelle *bestellung* referenziert.

In [Abschnitt 20.7](#) haben Sie erfahren, dass es sinnvoll ist, Fremdschlüsselspalten einer Tabelle direkt in der *CREATE TABLE*-Anweisung mit einem Index auszustatten, wenn Ihr Datenbanksystem (MySQL, MariaDB und MS SQL Server) es erlaubt. Erstellen Sie in dieser Übung abhängig von Ihrem verwendeten Datenbanksystem die Tabelle *bestellposition*, wie sie im Modell aus [Abbildung 20.4](#) dargestellt ist. Stellen Sie sicher,

dass Sie die Fremdschlüsselspalte mit einem Index versehen. Wenn Sie PostgreSQL verwenden, müssen Sie zunächst die Tabelle erstellen, bevor Sie einen Index für die Spalte *fk_bestellung_id* anlegen.

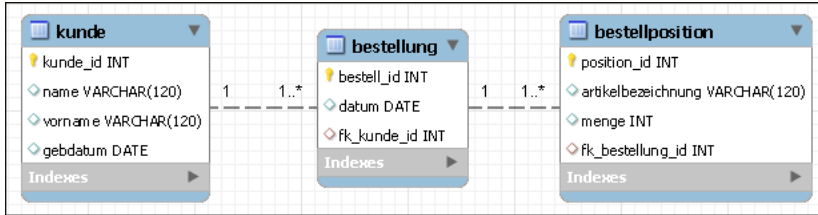


Abbildung 20.4 Die Tabelle »bestellung« in Beziehung zur Tabelle »bestellposition« setzen

Lösung zu Übung 1

```
CREATE INDEX
    index_arbeitszeit_mitarbeiter
ON
    arbeitszeit (mitarbeiterid);
```

Listing 20.18 Einen einfachen Index anlegen

Lösung zu Übung 2

```
CREATE INDEX
    index_plzort
ON
    kreditinstitutneu (ort,plz);
```

Listing 20.19 Einen Index über mehrere Spalten anlegen

Lösung zu Übung 3

```
DROP INDEX index_arbeitszeit_mitarbeiter ON arbeitszeit;
DROP INDEX index_plzort ON kreditinstitutneu;
```

Listing 20.20 Einen Index entfernen (MySQL, MariaDB und MS SQL Server)

```
DROP INDEX index_arbeitszeit_mitarbeiter;
DROP INDEX index_plzort;
```

Listing 20.21 Einen Index entfernen (PostgreSQL)

Lösung zu Übung 4

```
CREATE TABLE bestellposition(
    position_id INTEGER,
    artikelbezeichnung VARCHAR(120),
    menge INTEGER,
    fk_bestellung_id INTEGER,
    PRIMARY KEY (position_id),
    INDEX fk_bestellung_id_index (fk_bestellung_id),
    CONSTRAINT fk_bestellung_id FOREIGN KEY (fk_bestellung_id)
        REFERENCES bestellung (bestell_id)
);
```

Listing 20.22 Eine Fremdschlüsselspalte einer Tabelle während der Tabellenerstellung mit einem Index versehen (MySQL, MariaDB und MS SQL Server)

```
CREATE TABLE bestellposition(
    position_id INTEGER,
    artikelbezeichnung VARCHAR(120),
    menge INTEGER,
    fk_bestellung_id INTEGER,
    PRIMARY KEY (position_id),
    CONSTRAINT fk_bestellung_id FOREIGN KEY (fk_bestellung_id)
        REFERENCES bestellung (bestell_id)
);
```

```
CREATE INDEX
    fk_bestellung_id_index
ON
    bestellposition (fk_bestellung_id);
```

Listing 20.23 Eine Fremdschlüsselspalte nach der Tabellenerstellung mit einem Index ausstatten (PostgreSQL)

Kapitel 21

Skalarfunktionen entwickeln

Sie haben bereits zahlreiche Funktionen verwendet. Hierbei handelte es sich allerdings um Funktionen, die vom jeweiligen Datenbanksystem bereitgestellt werden. In diesem Kapitel erfahren Sie, wie Sie eigene Funktionen erstellen und in einer SQL-Anweisung verwenden können.

Funktionen kennen Sie bereits aus [Kapitel 4](#), als wir mit `CAST` Typen konvertiert haben, aus [Kapitel 14](#), wo wir mit Texten und Zahlen gearbeitet haben, und aus [Kapitel 16](#), in dem es um die Datumsfunktionen ging.

Die Funktionen, die Ihnen die Datenbanksysteme bereitstellen, sind zwar hilfreich und tausendfach erprobt, aber vielleicht haben Sie sich schon die Frage gestellt, wie Sie eigene Funktionen entwickeln können, die genau das tun, was Sie brauchen.

Als kleiner Auffrischer: In Funktionen werden wiederkehrende Aufgaben gebündelt, die sonst umständlich jedes Mal per Hand ausgeführt werden müssten. Als Beispiel nutzen wir die Funktion `SQRT`, die eine Quadratwurzel einer Zahl berechnet. [Listing 21.1](#) zeigt Ihnen anhand einer einfachen `SELECT`-Abfrage, wie Sie die Funktion verwenden.

```
SELECT SQRT(9);
```

Listing 21.1 Die Quadratwurzel einer Zahl wird mit der Funktion `SQRT` ermittelt.

Das Ergebnis lautet wie erwartet 3. Halten wir uns noch einmal kurz vor Augen, wie wir die Funktion angewendet haben: Übergeben haben wir der Funktion einen Zahlenwert, wir haben es also mit einem einzigen Parameter zu tun. Die Funktion bearbeitet den Wert (sie ermittelt die Quadratwurzel) und liefert schließlich das Ergebnis zurück.

Es brauchen allerdings nicht alle Funktionen einen Parameter. Bei der Ermittlung eines Zeitstempels mit der Funktion `CURRENT_TIMESTAMP` ist beispielsweise kein Parameter erforderlich. Andere Funktionen erfordern je nach Anwendungsfall wiederum mehrere Parameter.

Nach der Bearbeitung einer Skalarfunktion wird immer ein Wert zurückgegeben. Sehen wir uns im nächsten Schritt noch einmal kurz an, wie Funktionen aufgebaut sind.

In [Listing 21.2](#) sind zwei Varianten von Skalarfunktionen aufgeführt, nämlich mit und ohne Parameter.

```
rückgabewert funktionname1();  
rückgabewert funktionname2(Parameter1, Parameter2,...);
```

Listing 21.2 Grundlegender Aufbau von Funktionen

- ▶ Variante 1 zeigt eine Funktion mit der Bezeichnung `funktionname1()`, die keine Parameter (also eine leere Parameterliste) übergeben bekommt. Diese Funktion liefert lediglich einen Wert zurück, der als *Rückgabewert* bezeichnet wird.
- ▶ In Variante 2 enthält die klammerumschlossene Parameterliste der Funktion `funktionname2` auch Parameter mit den Bezeichnungen `Parameter1`, `Parameter2` und so weiter. Auch diese Funktion liefert einen Rückgabewert zurück.

Rückgabewerte und Parameter einer Funktion sind Datentypen zugeordnet. Diese haben Sie bereits z. B. in [Kapitel 14](#) kennengelernt. Wenn eine Funktion Werte berechnen soll, könnte den Parametern z. B. der Datentyp `NUMERIC` oder `INTEGER` zugeordnet sein. Der berechnete Rückgabewert würde ebenfalls mit einem numerischen Datentyp definiert.

In den nächsten Abschnitten 21.1 und 21.2 sehen wir uns an, wie Sie eigene Funktionen entwickeln und aufrufen können.

21.1 Funktionen mit einem Rückgabewert erstellen

In diesem Abschnitt erfahren Sie, wie Sie Funktionen entwickeln, die ohne Parameter aufgerufen werden und lediglich einen Wert zurückliefern. Sehen wir uns anhand eines Beispiels an, wie der strukturelle Aufbau einer Anweisung `CREATE FUNCTION` aussieht, die einen Wert zurückgibt.

```
CREATE FUNCTION funktion() RETURNS datentyp  
BEGIN  
    RETURN wert;  
END
```

Listing 21.3 Pseudosyntax zum Erstellen einer Funktion mit Rückgabewert

Die Funktion erstellen Sie einleitend mit der Anweisung `CREATE FUNCTION`. Es folgen der Funktionsname und eine leere Parameterliste, die durch eine einfache Umklammerung angegeben wird. Achten Sie bitte darauf, keine Funktionsnamen zu vergeben, die bereits in den eingebauten Funktionen verwendet werden.

Als Nächstes geben Sie mit **RETURN** den Datentyp des Rückgabewertes der Funktion an. Dann folgt ein Funktionskörper (der *Body*), der wiederum mit **BEGIN** und **END** umschlossen wird.

Innerhalb des Funktionskörpers können Sie Anweisungen durchführen, um letztendlich einen Rückgabewert zu erhalten. Wichtig ist hierbei, dass es sich tatsächlich um nur *einen* Wert handelt.

Mit der Anweisung **RETURN wert** geben Sie einen Wert zurück, der z. B. das Ergebnis einer Berechnung sein könnte, die innerhalb des Bodys durchgeführt wurde.

Die in [Listing 21.3](#) gezeigte Grundform zeigt den kleinsten gemeinsamen Nenner, der für die hier behandelten Datenbanken gilt. Wie Sie bereits ahnen, gibt es je nach genutzter Datenbank Abweichungen vom Standard.

Schauen wir uns das an einem Beispiel an. Unser Ziel ist es, eine Funktion zu erstellen, die lediglich die Zeichenkette 'Hallo ' zurückliefert. Diese Funktionalität könnten Sie natürlich auch mit anderen Operatoren bzw. Funktionen erreichen, aber ich möchte Ihnen anhand eines einfachen Beispiels zeigen, wie einfach Sie eigene Funktionen erstellen können. Die jeweils erforderlichen Anpassungen stelle ich Ihnen in den folgenden Abschnitten für die hier behandelten Datenbanken vor.

21.1.1 MySQL und MariaDB

Innerhalb einer Funktion können zahlreiche SQL-Anweisungen bzw. -Ausdrücke Verwendung finden. Bisher haben Sie Anweisungen immer mit einem Semikolon abgeschlossen. Das Erstellen einer Funktion mit der MySQL Workbench erfordert vor der eigentlichen Anweisung zum Erstellen einer Funktion die Aufhebung des Anweisungstrennzeichens, um innerhalb des Funktionskörpers das Semikolon weiterhin als Anweisungstrennzeichen verwenden zu können.

In [Listing 21.4](#) definieren wir also zuerst ein Trennzeichen, in diesem Fall die Zeichen **\$\$**. Es folgt die **CREATE FUNCTION**-Anweisung, die Sie in [Listing 21.3](#) ja bereits kennengelernt haben, gefolgt vom Funktionsnamen `hallokollege()`. Als Rückgabebetyp habe ich **CHAR(6)** ausgewählt.

Hinter dem Rückgabebetyp der Funktion geben Sie an, dass die Funktion deterministisch (**DETERMINISTIC**) ist. Eine Funktion wird als *deterministisch* bezeichnet, wenn sie abhängig von den Eingabeparametern (falls vorhanden) immer das gleiche Ergebnis zurückliefert. Dies gilt auch, wenn keine Eingabeparameter vorhanden sind. Hier gibt es zwei Varianten: Entweder wird die Funktion als deterministisch deklariert oder als deterministisch und ausschließlich lesend. Geben Sie an dieser Stelle nicht an, ob eine Funktion deterministisch bzw. deterministisch und ausschließlich lesend ist, erhalten Sie beim Ausführen der **CREATE FUNCTION**-Anweisung eine Fehlermeldung.

```
DELIMITER $$
CREATE FUNCTION hallokollege() RETURNS CHAR(6) DETERMINISTIC
BEGIN
    RETURN 'Hallo ';
END$$
DELIMITER ;
```

Listing 21.4 , Eine Funktion mit einem Rückgabewert anlegen (MySQL und MariaDB)

Der Funktionskörper wird von den Anweisungen `BEGIN` und `END` umschlossen. Innerhalb des Funktionskörpers wird mit der Anweisung `RETURN` eine einfache Zeichenkette zurückgegeben, nämlich `'Hallo'` gefolgt von einem Leerzeichen. Hinter der Anweisung `END` nutzen wir das zuvor vereinbarte Anweisungstrennzeichen `$$`, um die Anweisung `CREATE FUNCTION` abzuschließen.

Zum Schluss müssen Sie das Anweisungstrennzeichen mit `DELIMITER ;` wieder auf ein Semikolon setzen.

Um die Funktion zu erstellen, führen Sie die in [Listing 21.4](#) dargestellten Anweisungen komplett in der MySQL Workbench bzw. im HeidiSQL-Client (MariaDB) aus. Markieren Sie hierzu jeweils den Anweisungsblock

Die erstellte Funktion wollen wir natürlich auch verwenden. Hierzu formulieren wir eine einfache `SELECT`-Abfrage auf die Tabelle *mitarbeiter*. In der Spaltenauswahlliste ist als erster Funktionsaufruf `hallokollege()` enthalten, gefolgt von der Spalte *vorname*.

```
SELECT hallokollege() as begruessung,vorname FROM mitarbeiter;
```

Listing 21.5 Die Funktion »hallo()« in einer Abfrage verwenden

[Tabelle 21.1](#) zeigt das Ergebnis.

begruessung	vorname
Hallo	Ralf
Hallo	Petra
Hallo	Thomas

Tabelle 21.1 Ergebnis des Funktionsaufrufs

Glückwunsch, Sie selbst eine Funktion für die *uebungsdatenbank* angelegt. In [Abschnitt 21.2](#) sehen wir uns an, wie Sie Funktionen erstellen, die Parameter übergeben bekommen, die im Funktionskörper zur weiteren Verarbeitung verwendet werden können.

21.1.2 PostgreSQL

Listing 21.6 zeigt auf, wie Sie in einem aktuellen PostgreSQL-System eine Skalarfunktion erstellen. Mit den Schlüsselwörtern **CREATE FUNCTION**, die Sie bereits in Listing 21.3 kennengelernt haben, leiten Sie die Anweisung ein. Es folgt der Funktionsname `hallokollege()`, wiederum gefolgt von einer leeren einfachen Umklammerung. Auch hier wird der Datentyp des Rückgabewerts auf **CHAR(6)** festgelegt.

```
CREATE FUNCTION hallokollege () RETURNS CHAR(6)
BEGIN ATOMIC
    RETURN 'Hallo ';
END;
```

Listing 21.6 Eine Funktion mit einem Rückgabewert anlegen (PostgreSQL)

Der Funktionskörper wird von den Anweisungen **BEGIN ATOMIC** und **END** umschlossen. Innerhalb des Funktionskörpers wird mit der Anweisung **RETURN** eine einfache Zeichenkette zurückgegeben. Hier handelt es sich um die Zeichenkette 'Hallo ' gefolgt von einem Leerzeichen. Bitte beachten Sie an dieser Stelle, dass die Umschließung des Funktionskörpers mit **BEGIN ATOMIC** und **END** ab PostgreSQL 14 eingeführt wurde, um dem SQL-Standard zu entsprechen.

Skalarfunktionen in älteren PostgreSQL-Versionen

Wenn Sie eine PostgreSQL-Datenbank in einer Version kleiner als 14 verwenden, findet folgendes Beispiel Anwendung, um die gleiche Funktionalität zu realisieren wie in Listing 21.6. Die eigentliche Funktion wird hier als Zeichenkettenkonstante hinter dem **AS**-Schlüsselwort definiert. Die Zeichenkettenkonstante (Inhalt des Funktionskörpers) wiederum wird von einer **\$\$**-Zeichenkette umschlossen, was als *Quoting* bezeichnet wird. Die **\$\$**-Zeichen legen also ähnlich wie **BEGIN** und **END** den Anfang und das Ende eines Funktionskörpers fest. Zu guter Letzt notieren Sie mit dem Schlüsselwort **LANGUAGE SQL** die innerhalb des Funktionskörpers verwendete Sprache. Hier kommt die Sprache **SQL** zum Zuge.

```
CREATE FUNCTION hallokollege () RETURNS CHAR(6)
AS $$ SELECT 'Hallo '; $$
LANGUAGE SQL
```

Listing 21.7 PostgreSQL < Version 14, eine Funktion mit einem Rückgabewert anlegen (Sprache SQL)

Alternativ können Sie hier auch die PostgreSQL-spezifische Sprache **PLPGSQL** verwenden, um näher an unser Beispiel aus Listing 21.3 zu kommen. An dieser Stelle ist das Schlüsselwort **ATOMIC** hinter der **BEGIN**-Anweisung nicht erforderlich. Hier wird ebenfalls ein Quoting in Form von **\$\$** verwendet, um die eigentliche Zeichenkettenkonstante zu umschließen.



```
CREATE FUNCTION hallokollege() RETURNS CHAR(6) AS $$
BEGIN
    RETURN 'Hallo';
END;
$$ LANGUAGE plpgsql;
```

Listing 21.8 PostgreSQL < Version 14, eine Funktion mit einem Rückgabewert anlegen (Sprache PLPGSQL)

Die für PostgreSQL erstellte Funktion werden wir im nächsten Schritt einfach mal ausprobieren. Hierzu formulieren wir eine einfache Abfrage, in der wir die Funktion gefolgt von der Spalte *vorname* aus der Tabelle *mitarbeiter* mit einer SELECT-Anweisung aufrufen.

```
SELECT hallokollege() AS begruessung,vorname FROM mitarbeiter;
```

Listing 21.9 Die Funktion hallokollege() in einer Abfrage verwenden

Auch hier wird den Treffern aus der Tabelle wieder die passende Zeichenkette zugeordnet.

begruessung	vorname
Hallo	Ralf
Hallo	Petra
Hallo	Thomas

Tabelle 21.2 Ergebnis des Funktionsaufrufs

21.1.3 MS SQL Server

Wenn Sie mit MS SQL Server arbeiten, beginnen Sie die Anweisung ebenfalls wie in Listing 21.3 mit **CREATE FUNCTION**. Es folgen der Funktionsname und eine einfache Umklammerung der leeren Parameterliste. Schließlich geben Sie den Rückgabebetyp **CHAR(6)** mit der **RETURNS**-Klausel an.

```
CREATE FUNCTION hallokollege () RETURNS CHAR(6)
BEGIN
    RETURN 'Hallo ';
END;
```

Listing 21.10 Eine Funktion mit einem Rückgabewert anlegen (MS SQL Server)

Ähnlich wie bei den anderen Datenbanksystemen wird der Funktionskörper auch mit den Anweisungen **BEGIN** und **END** umschlossen. Innerhalb des Funktionskörpers ist eine **RETURN**-Anweisung dafür zuständig, eine einfache Zeichenkette zurückgeben.

Auch diese Funktion probieren wir einfach mal aus. Wir nutzen eine einfache Abfrage, in der wir die Funktion gefolgt von der Spalte *vorname* aus der Tabelle *mitarbeiter* mit einer **SELECT**-Anweisung abfragen.

```
SELECT hallokollege () AS begruessung,vorname FROM mitarbeiter;
```

Listing 21.11 Die Funktion »hallokollege()« in einer Abfrage verwenden

begruessung	vorname
Hallo	Ralf
Hallo	Petra
Hallo	Thomas

21.2 Funktionen mit Parameter entwickeln

In diesem Abschnitt zeige ich Ihnen, wie Sie Funktionen entwickeln, denen Sie Parameter übergeben, um diese im Funktionskörper zu verarbeiten. Ihr Aufbau ist ganz ähnlich wie der Aufbau von Funktionen, die keine Parameter übergeben bekommen. In [Listing 21.12](#) sehen Sie den Unterschied sofort: Innerhalb der Klammer werden Parameter übergeben.

Auch die Aufgabe dieser Beispielfunktion ist schnell erklärt: Diese Parameter werden innerhalb des Funktionskörpers addiert, und das Ergebnis wird als Rückgabewert von der Funktion ausgegeben.

```
CREATE FUNCTION funktion(p1 DATENTYP, p2 DATENTYP) RETURNS DATENTYP
BEGIN
    RETURN p1+p2;
END
```

Listing 21.12 Syntax zum Erstellen einer Funktion mit Rückgabewert und Parameterübergabe

In den nächsten Abschnitten zeige ich Ihnen, wie Sie Funktionen mit Parameterübergabe für die hier behandelten Datenbanken entwickeln. Unser Ziel ist, eine Funktion zu schreiben, die zwei Parameter übergeben bekommt: einerseits den bestehenden Urlaubsanspruch eines Mitarbeiters und andererseits den bereits genommenen Ur-

laub. Hierbei handelt es sich jeweils um ganzzahlige Werte. Gefragt ist an dieser Stelle der noch zur Verfügung stehende Urlaubsanspruch.

21.2.1 MySQL und MariaDB

Wie Parameter verwendet werden, betrachten wir anhand der zu erstellenden Funktion `urlaubverfuegbar(u1,u2)`.

Parameter werden innerhalb der klammerumschlossenen Parameterliste festgelegt. Wenn Sie mehr als einen Parameter übergeben, notieren Sie sie innerhalb der Parameterliste, separiert mit Kommas. Zu einem Parameter gehören ein Parametername und der Datentyp.

In [Listing 21.13](#) werden zwei Parameter in der Parameterliste festgelegt:

- Der Parameter `u1`, der dem aktuellen Urlaubsanspruch entspricht. Diesem Parameter wurde durch ein Leerzeichen separiert der Datentyp `SMALLINT` zugeordnet.
- Durch ein Komma getrennt wird der zweite Parameter definiert, der Parameter `u2`, der den bereits genommenen Urlaub repräsentiert. Auch hier wurde der Datentyp `SMALLINT` zugewiesen.

Im Funktionskörper wird die Differenz zwischen `u1` und `u2` gebildet und mit dem Schlüsselwort `RETURN` als Ergebnis der Skalarfunktion zurückgegeben.

```
DELIMITER $$
CREATE FUNCTION urlaubverfuegbar(u1 SMALLINT,u2 SMALLINT) RETURNS SMALLINT
DETERMINISTIC
BEGIN
    /* u1, aktueller Urlaubsanspruch */
    /* u2, Urlaub genommen */
    RETURN u1 - u2;
END$$
DELIMITER ;
```

Listing 21.13 Eine Funktion mit Parameterübergabe anlegen

In [Listing 21.14](#) probieren wir die soeben erstellte Funktion in einer einfachen `SELECT`-Abfrage aus. Der Funktion werden die Werte der Spalten `urlaubstage` und `urlaubgenommen` übergeben, die Spalten `name` und `vorname` der Mitarbeiter werden abgefragt.

```
SELECT
    urlaubverfuegbar(urlaubstage,urlaubgenommen) urlaub,
    name,
```

```

    vorname
FROM
    mitarbeiter;

```

Listing 21.14 Die Funktion »urlaubverfuegbar« in einer Abfrage verwenden

In [Tabelle 21.3](#) sehen Sie das Ergebnis der Abfrage aus [Listing 21.14](#). In der Spalte *urlaub* sind die noch verfügbaren Urlaubstage angegeben.

urlaub	name	vorname
2	Klein	Thomas
5	Lang	Ute
10	Eisenhof	Frank

Tabelle 21.3 Ergebnis des Funktionsaufrufs »urlaubverfuegbar«

21.2.2 PostgreSQL

Wenn Sie eine PostgreSQL-Datenbank verwenden, sieht das Beispiel etwas anders aus: Wir betrachten die zu erstellende Funktion `urlaubverfuegbar(u1,u2)`. Die Parameter `u1` und `u2` werden innerhalb der klammerumschlossenen Parameterliste definiert. Wenn Sie mehr als einen Parameter übergeben, sind die Parameter kommagetrennt innerhalb der Klammer zu notieren. Eine Parameterdefinition besteht aus einem Parameternamen und dem dazugehörigen Datentyp.

In [Listing 21.15](#) werden zwei Parameter festgelegt:

- Der Parameter `u1` steht für den aktuellen Urlaubsanspruch. Ihm ist der Datentyp `SMALLINT` zugeordnet.
- Durch ein Komma getrennt wird der zweite Parameter `u2` angegeben. Er repräsentiert den genommenen Urlaub. Auch hier wurde der Datentyp `SMALLINT` dem Parameter zugewiesen.

Innerhalb des Funktionskörpers wird die Differenz zwischen `u1` und `u2` gebildet und das Ergebnis mit dem Schlüsselwort `RETURN` zurückgegeben.

```

CREATE FUNCTION urlaubverfuegbar(u1 SMALLINT,u2 SMALLINT) RETURNS SMALLINT
BEGIN ATOMIC
    RETURN u1-u2;
END;

```

Listing 21.15 Eine Funktion mit Parameterübergabe anlegen

Die soeben erstellte Funktion rufen wir in einer einfachen **SELECT**-Abfrage auf. Die Spalten *urlaubstage* und *urlaubgenommen* werden der Funktion übergeben. In der Spaltenauswahlliste fragen wir auch die Spalten *name* und *vorname* der Tabelle *mitarbeiter* ab.

```
SELECT
    urlaubverfuegbar(urlaubstage,urlaubgenommen) urlaub,
    name,
    vorname
FROM
    mitarbeiter;
```

Listing 21.16 Die Funktion »urlaubverfuegbar« in einer Abfrage verwenden

In [Tabelle 21.4](#) sehen Sie das Ergebnis der Abfrage aus [Listing 21.6](#). Die Spalte *urlaub* enthält die berechneten noch verfügbaren Urlaubstage.

urlaub	name	vorname
10	Müller	Ralf
5	Lang	Ute
2	Klein	Thomas

Tabelle 21.4 Ergebnis des Funktionsaufrufs »urlaubverfuegbar«

21.2.3 MS SQL Server

Wenn Sie eine MS SQL Server-Datenbank verwenden, dann sind Sie hier genau richtig. Wieder geht es um die Funktion `urlaubverfuegbar(u1,u2)`. Innerhalb der umschlossenen Parameterliste werden die Parameter *u1* und *u2* festgelegt. Vor dem Parameternamen wird ein **@**-Zeichen notiert. Die Parameterdefinitionen werden innerhalb der Klammer durch Kommas separiert angegeben. Eine Parameterdefinition wiederum besteht aus einem Parameternamen und dem dazugehörigen Datentyp.

In [Listing 21.17](#) werden zwei Parameter festgelegt:

- Der bestehende Urlaubsanspruch wird durch den Parameter *u1* definiert. Hier wurde der Datentyp **SMALLINT** zugeordnet.
- Der zweite Parameter, *u2* (genommener Urlaub), wird durch ein Komma separiert angegeben. Auch dem Parameter *u2* wird der Datentyp **SMALLINT** zugeordnet.

Innerhalb des Funktionskörpers ermitteln wir die Differenz der übergebenen Parameter *@u1* und *@u2*. Das Ergebnis der Differenz wird mit dem Schlüsselwort **RETURN** zurückgegeben.

```
CREATE FUNCTION urlaubverfuegbar(@u1 SMALLINT,@u2 SMALLINT) RETURNS INT
BEGIN
    RETURN @u1-@u2;
END;
```

Listing 21.17 Eine Funktion mit Parameterübergabe anlegen

Nachdem Sie die Funktion angelegt haben, verwenden wir sie in einer einfachen **SELECT**-Abfrage. Sie übergeben die Werte der Spalten *urlaubstage* und *urlaubgenommen*. In der Spaltenauswahlliste fragen wir außerdem die Spalten *name* und *vorname* der Tabelle *mitarbeiter* ab.

Beachten Sie beim Aufruf der erstellten Funktion eine Eigenheit, die MS SQL Server betrifft: Es ist erforderlich, das Standardschema *dbo* vor dem eigentlichen Funktionsaufruf durch einen Punkt separiert mit anzugeben, da die Funktion sonst nicht gefunden wird.

```
SELECT
    dbo.urlaubverfuegbar(urlaubstage,urlaubgenommen) urlaub,
    name,
    vorname
FROM
    mitarbeiter;
```

Listing 21.18 Die Funktion »urlaubverfuegbar« in einer Abfrage verwenden

Tabelle 21.5 zeigt das Ergebnis der Abfrage aus Listing 21.16. In der Spalte *urlaub* sind die noch verfügbaren Urlaubstage angegeben.

urlaub	name	vorname
2	Klein	Thomas
5	Lang	Ute
10	Eisenhof	Frank

Tabelle 21.5 Ergebnis des Funktionsaufrufs »urlaubverfuegbar«

21.3 Funktionen entfernen

Gegebenenfalls möchten Sie eine Funktion, die Sie selbst erstellt haben, entfernen, wenn sie nicht mehr benötigt wird. SQL bietet die Anweisung **DROP FUNCTION** an. Die Anweisung ist in der Anwendung für die hier behandelten Datenbanken MySQL,

MariaDB und PostgreSQL gleich. Wenn Sie eine MS SQL Server-Datenbank verwenden, vergessen Sie nicht, das verwendete Schema mit anzugeben.

In [Listing 21.19](#) zeige ich Ihnen, wie Sie die von Ihnen erstellte Funktion `hallokollege()` wieder löschen. Dies ist denkbar einfach: Eingeleitet wird die Anweisung mit `DROP FUNCTION`, gefolgt vom Namen (`hallokollege`) der zu löschenden Funktion.

```
/* MySQL, MariaDB, PostgreSQL */  
DROP FUNCTION hallokollege;  
/* MS SQL Server */  
DROP FUNCTION dbo.hallokollege;
```

Listing 21.19 Eine Funktion entfernen

21.4 Übungen

Übung 1

Erstellen Sie eine Skalarfunktion, die den Namen der aktuell verwendeten Datenbank ermittelt und als Zeichenkette zurückgibt. Ordnen Sie der neuen Funktion den Namen `db_aktuell()` zu. Die neue Funktion bekommt keine Parameter übergeben.

Nutzen Sie folgende Skalarfunktionen, die von den jeweiligen Datenbanksystemen bereitgestellt werden, und verwenden Sie den jeweiligen Rückgabewert der Funktionen als Rückgabewert für Ihre Funktion mit der Bezeichnung `db_aktuell`:

- ▶ MySQL und MariaDB: `database()`
- ▶ PostgreSQL: `current_database()`
- ▶ MS SQL Server: `db_name()`

Übung 2

Erstellen Sie eine Funktion mit dem Namen `erhoeheWertUmProzent(wert,prozent)`. Der erste Parameter gibt einen numerischen Wert an. Der zweite Parameter bestimmt den Prozentwert, um den der erste Parameter zu erhöhen ist.

Dem Parameter und dem Rückgabewert ist der Einfachheit halber der Datentyp `DECIMAL(9,2)` zuzuordnen. Diese Vereinfachung ist erforderlich, um je nach verwendeter Datenbank auf eine Typkonvertierung zu verzichten. Wenn die Skalarfunktion erfolgreich erstellt wurde, können wir die Funktion mit einer einfachen `SELECT`-Anweisung ausprobieren, die lediglich das Ergebnis der Funktion abfragt.

Übung 3

Die Skalarfunktion `erhoeheWertUmProzent` wird nicht mehr benötigt. Entfernen Sie sie mit der `DROP FUNCTION`-Anweisung.

Lösung zu Übung 1

```
DELIMITER $$
CREATE FUNCTION db_aktuell() RETURNS VARCHAR(40) DETERMINISTIC
BEGIN
    RETURN database();
END$$
DELIMITER ;
```

Listing 21.20 MySQL und MariaDB

```
CREATE FUNCTION db_aktuell() RETURNS VARCHAR(40)
BEGIN ATOMIC
    RETURN current_database();
END;
```

Listing 21.21 PostgreSQL

```
CREATE FUNCTION db_aktuell() RETURNS VARCHAR(40)
BEGIN
    RETURN db_name();
END;
```

Listing 21.22 MS SQL Server

Mit einer einfachen SELECT-Anweisung können wir die Skalarfunktion testen.

```
SELECT db_aktuell();
```

Listing 21.23 Die Skalarfunktion »db_aktuell()« prüfen

Lösung zu Übung 2

```
DELIMITER $$
CREATE FUNCTION erhoeheWertUmProzent(w1 DECIMAL(8,2),w2 DECIMAL(8,2)) RETURNS
DECIMAL(8,2) DETERMINISTIC
BEGIN
    /* w1, Wert, der erhöht werden soll*/
    /* w2, ganzzahliger Prozentwert, um den erhöht werden soll */
    RETURN w1+(w1/100*w2);
END$$
DELIMITER ;
```

Listing 21.24 MySQL und MariaDB

```
CREATE FUNCTION erhoeheWertUmProzent(w1 DECIMAL(8,2),w2 DECIMAL(8,2)) RETURNS
DECIMAL(8,2)
BEGIN ATOMIC
/* w1, Wert, der erhöht werden soll*/
/* w2, ganzzahliger Prozentwert, um den erhöht werden soll */
    RETURN w1+(w1/100*w2);
END
```

Listing 21.25 PostgreSQL

```
CREATE FUNCTION erhoeheWertUmProzent(@w1 DECIMAL(8,2),@w2 DECIMAL(8,2))
RETURNS DECIMAL(8,2)
BEGIN
/* w1, Wert, der erhöht werden soll*/
/* w2, ganzzahliger Prozentwert, um den erhöht werden soll */

    RETURN @w1+(@w1/100*@w2);
END;
```

Um die Skalarfunktion zu testen, verwenden Sie die einfachste aller SELECT-Abfragen und rufen die Funktion wie folgt auf:

```
/* MySQL, MariaDB, PostgreSQL */
SELECT erhoeheWertUmProzent(100,20);

/* MS SQL Server */
SELECT dbo.erhoeheWertUmProzent(100,20);
```

Listing 21.26 Prüfabfrage für die Skalarfunktion »erhoeheWertUmProzent(100,20)«

Lösung zu Übung 3

```
/* MySQL, MariaDB, PostgreSQL */
DROP FUNCTION erhoeheWertUmProzent;

/* MS SQL Server */
DROP FUNCTION dbo.erhoeheWertUmProzent;
```

Listing 21.27 Skalarfunktion entfernen

Kapitel 22

Prozeduren entwickeln

Sie müssen oftmals die gleichen Aufgaben erledigen und immer wieder gleichartige Daten einfügen oder aktualisieren? Mit Prozeduren sparen Sie Zeit und Arbeit.

Prozeduren ermöglichen es Ihnen zum Beispiel, eine große Anzahl von Datensätzen auf einmal zu aktualisieren oder einzufügen, berechnete Resultate in Tabellen zu schreiben, komplexe Berichte zu erstellen und, wenn erforderlich, Datenbereinigungen durchzuführen.

Eine Prozedur ähnelt einer Funktion in einer Programmiersprache: Sie wird einmal erstellt und kann dann immer wieder genutzt werden. Eine Prozedur besteht ähnlich wie eine Skalarfunktion aus einem Namen und einer klammerumschlossenen Parameterliste, die auch leer sein kann, wenn keine Parameter benötigt werden.

22.1 Prozeduren ausführen

Die Syntax für das Ausführen einer Prozedur ist einfach: Hinter der Anweisung **CALL** notieren Sie den Namen der Prozedur, die ausgeführt soll. Je nach Anwendungszweck folgt eine klammerumschlossene Liste mit Parametern bzw. ohne Parameter (siehe [Listing 21.2](#)). Der Aufruf einer Prozedur erfolgt für die Datenbanksysteme MySQL, MariaDB und PostgreSQL mit der Anweisung **CALL**.

Eine Ausnahme bildet MS SQL Server. Dort nutzen Sie die Anweisung **EXEC**, gefolgt vom Prozedurnamen, um eine Prozedur auszuführen. Gegebenenfalls vorhandene Parameter werden, durch ein Leerzeichen vom Prozedurnamen getrennt, kommagetrennt angegeben.

```
/* MySQL, MariaDB, PostgreSQL */  
CALL prozedurname();  
CALL prozedurname(p1,p2,...);  
/* MS SQL Server */  
EXEC prozedurname p1,p2,...;
```

Listing 22.1 Prozeduren aufrufen

22.2 Prozeduren anlegen

Sehen wir uns im nächsten Schritt anhand einer Pseudosyntax an, wie Sie eine Prozedur erstellen. Diese Darstellung soll Ihnen nur kurz vor Augen führen, wie eine **CREATE PROCEDURE**-Anweisung aufgebaut ist. [Listing 22.2](#) wird mit der Anweisung **CREATE PROCEDURE** eingeleitet. Es folgen der Name der Prozedur und eine leere, klammerumschlossene Parameterliste.

Als Nächstes sehen Sie den Prozedurkörper, der von den Schlüsselwörtern **BEGIN** und **END** umschlossen wird. Innerhalb des Prozedurkörpers können Sie z. B. SQL-Anweisungen ausführen. Eine Prozedur hat keinen Rückgabewert.

```
CREATE PROCEDURE prozedurname()  
BEGIN  
    /* SQL Anweisungen */  
END;
```

Listing 22.2 Pseudosyntax einer CREATE PROCEDURE-Anweisung

Kommen wir nun zu einem einfachen konkreten Beispiel, in dem Sie eine Prozedur anwenden, um mehrere Datensätze in eine Tabelle zu schreiben. Hierbei handelt es sich um eine Batch-Verarbeitung (Stapelverarbeitung), in der mehrere **INSERT**-Anweisungen innerhalb des Prozedurkörpers ausgeführt werden, um Zeilen in die Tabelle *rohstoff* einzufügen. Die Tabelle enthält die Spalten *id*, *bezeichnung* und *eigenschaft*. Sie Tabelle ist in der Übungsdatenbank enthalten, Sie können das Beispiel also sofort ausprobieren.

Leider unterscheiden sich die Datenbanksysteme etwas darin, wie sie mit Prozeduren umgehen. Wenn Sie eine MySQL- oder MariaDB-Datenbank verwenden, sehen Sie sich [Abschnitt 22.2.1](#) an; um PostgreSQL geht es in [Abschnitt 22.2.2](#), um die MS SQL Server-Datenbank in [Abschnitt 22.2.3](#).

22.2.1 Prozeduren anlegen (MySQL und MariaDB)

[Listing 22.3](#) zeigt, wie Sie eine Prozedur mit der Bezeichnung `insert_rohstoff` für eine MySQL- oder MariaDB-Datenbank anlegen.

```
DELIMITER //  
CREATE PROCEDURE insert_rohstoff ()  
BEGIN  
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)  
        VALUES (1,'Edel-Stahl','Korrosionsbeständig');  
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)  
        VALUES (2,'Kohlenstoff-Stahl','Festigkeit');  
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)
```

```
VALUES (3, 'Legierter-Stahl', 'Korrosionsbeständig');
END //
DELIMITER ;
```

Listing 22.3 Eine Prozedur für eine Stapelverarbeitung anlegen

Als Erstes müssen wir wieder das Anweisungstrennzeichen Semikolon (;) mit der Anweisung **DELIMITER** durch **//** ersetzen, um das Semikolon innerhalb des Prozedurkörpers verwenden zu können.

Eingeleitet wird die Anweisung mit **CREATE PROCEDURE**. Es folgt schließlich der Prozedurname, der hier `insert_rohstoff` lautet. Der Prozedurkörper wird mit den Schlüsselwörtern **BEGIN** und **END** umschlossen. Innerhalb des Prozedurkörpers werden drei **INSERT**-Anweisungen hintereinander ausgeführt. Hinter dem Schlüsselwort **END** schließen wir die **CREATE PROCEDURE**-Anweisung mit dem zuvor vereinbarten Anweisungstrennzeichen **//** ab. Zu guter Letzt setzen wir das Anweisungstrennzeichen mit **DELIMITER ;** wieder auf ein Semikolon zurück. Führen Sie [Listing 22.3](#) in der MySQL Workbench oder im HeidiSQL-Editor einfach mal aus.

Wenn Sie die **CREATE PROCEDURE**-Anweisung aus [Listing 22.3](#) ausgeführt haben, können Sie die soeben erstellte Prozedur direkt testen. In [Listing 22.4](#) rufen Sie die Prozedur `insert_rohstoff` mit der Anweisung **CALL** auf.

```
CALL insert_rohstoff();
```

Listing 22.4 Eine Prozedur mit CALL ausführen

Wenn alles gut gegangen ist, befinden sich nach der Ausführung drei Zeilen mit den entsprechenden Rohstoffen in der Tabelle *rohstoff*, was wir mit der einfachen **SELECT**-Abfrage aus [Listing 22.5](#) überprüfen.

```
SELECT * FROM rohstoff;
```

Listing 22.5 Das Ergebnis der Ausführung überprüfen

[Tabelle 22.1](#) zeigt das Ergebnis der Abfrage aus [Listing 22.5](#). Durch den Aufruf der Prozedur wurden die drei **INSERT**-Anweisungen ausgeführt, die im Prozedurkörper enthalten sind.

id	bezeichnung	eigenschaft
1	Edel-Stahl	Korrosionsbeständig
2	Kohlenstoff-Stahl	Festigkeit
3	Legierter-Stahl	Korrosionsbeständig

Tabelle 22.1 Ergebnis der abgefragten Tabelle »rohstoff«

22.2.2 Prozeduren anlegen (PostgreSQL)

In [Listing 22.6](#) sehen Sie, wie Sie für eine PostgreSQL-Datenbank eine Prozedur anlegen. Wir beginnen mit der Anweisung **CREATE PROCEDURE**. Dahinter folgt der Prozedurname `insert_rohstoff`. Jetzt müssen Sie nur noch die Sprache angeben, die innerhalb des Prozedurkörpers verwendet wird. In diesem Fall geben wir SQL als eingesetzte Sprache vor. Alternativ könnten Sie auch PLPGSQL angeben – das ist eine Spracherweiterung, die nur in PostgreSQL genutzt werden kann. Der Prozedurkörper wird mit den Schlüsselwörtern **BEGIN ATOMIC** und **END** umschlossen. Im Prozedurkörper sind drei **INSERT**-Anweisungen angegeben, die hintereinander ausgeführt werden. Führen Sie als Nächstes [Listing 22.6](#) im pgAdmin-Editor aus.

```
CREATE PROCEDURE insert_rohstoff()  
LANGUAGE SQL  
BEGIN ATOMIC  
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)  
        VALUES (1,'Edel-Stahl','Korisionsbeständig');  
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)  
        VALUES (2,'Kohlenstoff-Stahl','Festigkeit');  
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)  
        VALUES (3,'Legierter-Stahl','Korisionsbeständig');  
END;
```

Listing 22.6 Eine Prozedur für eine Stapelverarbeitung anlegen

Nach der Ausführung der **CREATE PROCEDURE**-Anweisung aus [Listing 22.6](#) rufen wir die soeben erstellte Prozedur `insert_rohstoff`, wie in [Listing 22.7](#) dargestellt, mit der Anweisung **CALL** auf.

```
CALL insert_rohstoff();
```

Listing 22.7 Eine Prozedur mit CALL ausführen

Im letzten Schritt überprüfen wir mit einer einfachen **SELECT**-Abfrage, ob die Tabelle *rohstoff* tatsächlich drei neue Zeilen enthält (siehe [Listing 22.8](#).)

```
SELECT * FROM rohstoff;
```

Listing 22.8 Das Ergebnis der Ausführung überprüfen

In [Tabelle 22.2](#) sehen Sie das Ergebnis der Abfrage aus [Listing 22.8](#). Durch den Aufruf der Prozedur `insert_rohstoff` wurden drei neue Zeilen in die Tabelle *rohstoff* eingefügt.

id	bezeichnung	eigenschaft
1	Edel-Stahl	Korrosionsbeständig
2	Kohlenstoff-Stahl	Festigkeit
3	Legierter-Stahl	Korrosionsbeständig

Tabelle 22.2 Ergebnis der Abfrage der Tabelle »rohstoff«

22.2.3 Prozeduren anlegen (MS SQL Server)

Wenn Sie eine Prozedur in einer MS SQL Server-Datenbank anlegen möchten, betrachten Sie [Listing 22.9](#). Hinter der **CREATE PROCEDURE**-Anweisung geben Sie den Prozedurnamen `insert_rohstoff` an. Sie müssen hier tatsächlich keine leere umklammerte Parameterliste übergeben, da in einer MS SQL Server-Datenbank keine Klammerung für die Parameterliste vorgesehen ist.

Der Prozedurkörper beginnt mit dem Schlüsselwort **BEGIN** und endet mit dem Schlüsselwort **END**. Im Beispiel sind drei **INSERT**-Anweisungen im Prozedurkörper angegeben; sie werden nacheinander ausgeführt. Um die Prozedur anzulegen, führen Sie die **CREATE PROCEDURE**-Anweisung aus [Listing 22.9](#) im SQL Server Management Studio aus.

```
CREATE PROCEDURE insert_rohstoff
AS
BEGIN
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)
    VALUES (1,'Edel-Stahl','Korrosionsbeständig');
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)
    VALUES (2,'Kohlenstoff-Stahl','Festigkeit');
    INSERT INTO rohstoff (id,bezeichnung,eigenschaft)
    VALUES (3,'Legierter-Stahl','Korrosionsbeständig');
END;
```

Listing 22.9 Eine Prozedur für eine Stapelverarbeitung anlegen

Wenn Sie die **CREATE PROCEDURE**-Anweisung aus [Listing 22.9](#) ausgeführt haben, rufen Sie die Prozedur `insert_rohstoff` mit der Anweisung **CALL** wie in [Listing 22.10](#) gezeigt auf. Wenn alles geklappt hat, befinden sich nach dem Aufruf der Prozedur drei neue Zeilen in der Tabelle *rohstoff*.

```
EXEC insert_rohstoff();
```

Listing 22.10 Eine Prozedur mit CALL ausführen

Mit einer **SELECT**-Anweisung prüfen wir als Nächstes, ob die Tabelle *rohstoff* tatsächlich drei neue Zeilen enthält. Führen Sie hierzu einfach die **SELECT**-Anweisung aus [Listing 22.11](#) aus.

```
SELECT * FROM rohstoff;
```

Listing 22.11 Das Ergebnis der Ausführung überprüfen

Wenn Sie die **SELECT**-Anfrage aus [Listing 22.11](#) ausgeführt haben, werden Ihnen, wenn alles funktioniert hat, die durch die Prozedur eingefügten Zeilen als Ergebnis angezeigt.

id	bezeichnung	eigenschaft
1	Edel-Stahl	Korrosionsbeständig
2	Kohlenstoff-Stahl	Festigkeit
3	Legierter-Stahl	Korrosionsbeständig

Tabelle 22.3 Ergebnis der abgefragten Tabelle »rohstoff«

22.3 Übung

Übung 1

Entwickeln Sie eine Prozedur mit dem Namen `clear_rohstoff`, in der sämtliche Zeilen der Tabelle *rohstoff* gelöscht werden. Verwenden Sie zum Löschen der Zeilen innerhalb des Prozedurkörpers eine **DELETE**-Anweisung. Wenn Sie die Prozedur erstellt haben, rufen Sie sie auf und prüfen Sie, ob sie die hier gestellte Anforderung auch erfüllt.

Lösung

```
/* MySQL und MariaDB*/
DELIMITER //
CREATE PROCEDURE clear_rohstoff()
BEGIN
    DELETE FROM rohstoff;
END //
DELIMITER ;
```

```
CALL clear_rohstoff();
```

```
/* PostgreSQL */
CREATE PROCEDURE clear_rohstoff()
```



```
LANGUAGE SQL
BEGIN ATOMIC
    DELETE FROM rohstoff;
END;

CALL clear_rohstoff();

/* MS SQL Server */
CREATE PROCEDURE clear_rohstoff
AS
BEGIN
    DELETE FROM rohstoff;
END;

EXEC clear_rohstoff;
```

Listing 22.12 Lösung zur Übung

Index

_ (Platzhalter)	71
% (Platzhalter)	71
.....	472
1:1-Beziehung	289
1:n-Beziehung	291

A

Abfrage	
<i>korrelierende</i>	554
<i>nicht korrelierende</i>	560
<i>Spalte</i>	47
Abhängigkeit	
<i>transitive</i>	317
<i>voll funktionale</i>	314
ACID-Prinzip	413
Administratorberechtigung	36
Aggregationsoperation	530
Algebra	
<i>boolesche</i>	91
Alias	127
ALL	232, 239, 566, 567
AND (Logik)	91
AND NOT (Logik)	98
Änderungsanomalie	310
Anforderungskatalog	279
ANSI-SQL-Standard	25
ANY	566, 567
AS	128
ASC	114
Asterisk	49
Atomar	312
Attribut	281
<i>atomares</i>	312
<i>optionales</i>	286
<i>Pflichtattribut</i>	286
<i>Wertebereich</i>	284
Ausdruck	452
<i>auswerten</i>	452
<i>evaluieren</i>	452
Ausfallsicherheit	411
Autoinkrement-Schlüssel	194
AVG	537

B

Basic Latin	173
Bedingungen verknüpfen	
<i>AND NOT-Operator</i>	99
<i>AND-Operator</i>	92
<i>Kombinationen</i>	101
<i>OR NOT-Operator</i>	100
<i>OR-Operator</i>	96
Bedingungslogik	487
BEGIN	416
Benutzer	
<i>anlegen</i>	260
<i>entfernen</i>	261
Berechtigungen	
<i>entziehen (REVOKE)</i>	270
<i>Rollen</i>	270, 271
<i>vergeben (GRANT)</i>	268
<i>verwalten</i>	268
BETWEEN	67
Beziehung	388
<i>1:1</i>	326
<i>1:n</i>	328
<i>aufheben</i>	399
<i>identifizierende</i>	301
<i>m:n</i>	330
<i>nicht identifizierende</i>	302
<i>Verhältnismäßigkeit</i>	289
<i>zwischen Objekten</i>	280
Binärsystem	166
Boolesche Algebra	91
Boyce-Codd-Normalform → BCNF	
B-Tree-Index	600

C

Cache	605
CASE	487
case-insensitive	49
CAST	179
CHAR	175
CHAR_LENGTH	477
CHARINDEX	478
CHECK	207, 221
Codd, Edgar F.	23
Collation	112
COMMIT	416

CONSTRAINT	199, 215, 327
COUNT	530, 531
<i>DISTINCT</i>	533
CREATE DATABASE	323
CREATE INDEX	606
CREATE TABLE	165, 187, 188
CREATE VIEW	576
CROSS JOIN	366
CRUD	22
CURRENT_DATE	496
CURRENT_TIME	496
CURRENT_TIMESTAMP	496

D

DATE_ADD	522
DATE_FORMAT	500
DATE_SUB	522
DATEADD (MS SQL Server)	524
Datenbank	
<i>anlegen</i>	323
<i>anlegen, UTF-8</i>	324
<i>modellieren</i>	279
Datenbankabfragesprache	21
Datenbankbenutzer	
<i>Administrator</i>	36
Datenbankmodell	279
Datenbanksystem	24
Datenmodell	323
Datensätze	22
Datentyp	
<i>CHAR</i>	175
<i>DATE</i>	496
<i>Datum und Zeit</i>	172
<i>DECIMAL</i>	171
<i>Fixkomma</i>	171
<i>Fließkomma</i>	171
<i>floating point</i>	171
<i>ganze Zahlen</i>	169
<i>INTEGER</i>	170
<i>konvertieren</i>	179
<i>NUMERIC</i>	171
<i>rationale Zahlen</i>	171
<i>SIGNED</i>	170
<i>TINYINT</i>	165
<i>Überblick</i>	165
<i>umwandeln</i>	179
<i>UNSIGNED</i>	170
<i>VARCHAR</i>	175
<i>Zeichenketten</i>	173
DATEPART	517

Datum	172
<i>Berichtsdatum</i>	498
<i>einfügen</i>	497
<i>formatieren</i>	499
Datumsangabe	
<i>berechnen</i>	520
<i>berechnen (MS SQL Server)</i>	524
<i>berechnen (MySQL und MariaDB)</i>	521
<i>berechnen (PostgreSQL)</i>	523
<i>extrahieren</i>	515
<i>formatieren (MS SQL Server)</i>	510
<i>formatieren (MySQL und MariaDB)</i>	500
<i>formatieren (PostgreSQL)</i>	505
Datumsformat UTC	495
Datumswert	60
DBCC DROPCLEANBUFFERS	605
DECIMAL	171
DEFAULT	205
DEFAULT CHARACTER	324
DELETE	156, 268
<i>View</i>	590
Denormalisierung	318
DESC	114
DETERMINISTIC	619
Differenzmenge	229, 235, 250
Disjunktion	94
Disjunktionsfalle	104
DISTINCT	131, 132, 237, 533
Domain	284
Drei-Werte-Logik	81
DROP INDEX	609
DROP TABLE	191
DROP VIEW	596
Dualsystem → Binärsystem	

E

Einfügeanomalie	310
Einschränkungen	
<i>Spalten</i>	199
<i>Tabelle</i>	214
ELSE	488, 492
END	489
Entität	
<i>1:1-Beziehung</i>	289
<i>1:n-Beziehung</i>	291
<i>Attribute</i>	281
<i>Beziehungen</i>	288
<i>Beziehungen zwischen Entitäten</i>	287
<i>Definition</i>	280
<i>Fremdschlüssel</i>	290

Entität (Forts.)	
<i>identifizieren</i>	280
<i>in Beziehung</i>	288
<i>Kardinalität</i>	289
<i>m:n-Beziehung</i>	293
<i>Pflichtattribut</i>	286
<i>Schlüsselattribut</i>	281
<i>Schlüsselattribute</i>	282
<i>schwache</i>	298
<i>Selbstverweis</i>	296
<i>starke</i>	298
Entity-Relationship-Diagrammeditor	302
Entity-Relationship-Modell	280
<i>Krähenfußnotation</i>	291
Entity-Relationship-Notation	280
Ergebnis	
<i>filtern</i>	457
<i>sortieren</i>	458
Ergebniszeilen einschränken	109
EXCEPT	235, 250
Existenzprüfung	571
EXISTS	571
Explizite Typkonvertierung	180
Expression	451
EXTRACT	515

F

FETCH	109, 110
FOR	469
FOREIGN KEY	327
Foreign Key	290
FORMAT	510
Fremdschlüsseldefinition	341
Fremdschlüsselspalte	327
FROM	48, 469
FULL OUTER JOIN	360, 362
Funktion	463
<i>CONCAT</i>	464
<i>LOWER</i>	464
<i>SUBSTRING</i>	464
<i>Textwerte</i>	464
<i>TRIM</i>	464
<i>UPPER</i>	464
<i>verschachtelte</i>	482

G

GRANT	268, 271
GROUP BY	529, 531
<i>AVG</i>	537

COUNT	531
HAVING	545
MAX	539
MIN	540
<i>nach zwei oder mehr Spalten</i>	
<i>gruppieren</i>	548
NULL-Werte	541
SUM	535

H

HAVING	545
HeidiSQL	263
Hochkommata	55

I

Identifying	301
Identifying Relationship	330
Implizite Typkonvertierung	181
IN	76
Index	599
<i>B-Tree-Index</i>	600
<i>CREATE INDEX</i>	606
<i>DBCC DROPCLEANBUFFERS</i>	605
<i>DROP INDEX</i>	609
<i>eindeutiger</i>	606
<i>erstellen</i>	602
<i>Fremdschlüsselspalten indexieren</i>	609
<i>löschen</i>	609
<i>MS SQL Server</i>	605
<i>MySQL</i>	605
<i>Spalten</i>	606
<i>SQL_NO_CACHE</i>	605
<i>Tabelle</i>	604
<i>UNIQUE</i>	606
information_schema	140
INNER JOIN	351, 352, 558
<i>mehrere Tabellen</i>	368
<i>Schlüsselvergleich</i>	377
INSERT	135, 268
<i>View</i>	584
Installation	28
International Organization for Standardization → ISO	
INTERSECT	233, 245
INTERVAL	523
INTO	136
IS NOT NULL	83
IS NULL	83
ISO	24, 25

ISO 8601	57, 60, 172, 496
ISO/IEC 9075-1	25

J

JOIN

<i>FULL OUTER JOIN</i>	351
<i>gleichnamige Spalten</i>	370
<i>INNER JOIN</i>	351
<i>LEFT OUTER JOIN</i>	351
<i>ohne Schlüsselvergleich verbinden</i>	377
<i>ON</i>	353
<i>RIGHT OUTER JOIN</i>	351
<i>SELF JOIN</i>	375
<i>Tabellenaliasse</i>	373
Joker	71

K

Kardinalität	289, 301
Kommentar	42
Konjunktion	91
Konsistenz	412
Konstante	
<i>abfragen</i>	125
<i>Übersicht</i>	124
Konvertierungsfunktionen	179
Korrelation	554
Krähenfußnotation	291
Künstlicher Schlüssel → Schlüsselattribut	

L

Latin 1 Supplement	173
Leerzeichen filtern	466
<i>LEFT OUTER JOIN</i>	351, 354
<i>LIKE</i>	71
<i>LIMIT</i>	109
Logische Verknüpfung	91
Löschanomalie	310
<i>LOWER</i>	465

M

m:n-Beziehung	293
MariaDB	25
Mathematischer Operator	451
<i>Ergebnis filtern</i>	457
<i>Ergebnis sortieren</i>	458
<i>Potenz</i>	479
<i>Quadratwurzel</i>	480

MAX	539
Mehrwertigkeit	313
Menge	

<i>Differenzmenge</i>	229
<i>Schnittmenge</i>	229
<i>Vereinigungsmenge</i>	229

Mengenlehre	23
Mengenoperation	229, 236
<i>ORDER BY</i>	255
<i>WHERE</i>	253

Microsoft SQL Server Management

Studio	302
--------------	-----

Microsoft SQL Server → MS SQL Server

MIN	540
Modulo-Operator	451, 456
MS SQL Server	24

MySQL

<i>Dokumentation</i>	25
<i>FULL OUTER JOIN</i>	362

MySQL-Datenbank

<i>installieren</i>	28
<i>unterstützte Betriebssysteme</i>	25

N

Non-identifying	302
Normalform	309
<i>dritte</i>	316
<i>erste</i>	312
<i>zweite</i>	314
Normalisierung	309, 311
NOT	64
NOT BETWEEN	67
NOT EXISTS	573
NOT IN	77
NOT LIKE	76
NOT NULL	199, 200
NULL	79, 200
NULL-Wert	79, 82
<i>Gruppierung</i>	541
NUMERIC	171

O

ON	353
ON DELETE CASCADE	392
ON UPDATE CASCADE	337, 390
Operator	165
-	451
*	451
/	451

Operator (Forts.)	
%	451
+	451
Optimierung (Datenmodelle)	309
OR (Logik)	91, 94
OR NOT (Logik)	99
ORDER BY	113, 255, 490
View	581

P

Pattern	500
Performance	599
Pflichtattribut	286
Pflichtfeld	199
pgAdmin IV Client	302
Pipe-Operator	473
Plausibilitätsprüfung	207
POSITION	478
Potenz	479
POWER	479
Primärschlüssel	136, 192
<i>hochzählen</i>	194
<i>Kombination</i>	216
<i>Tabellenebene</i>	214
Projektion	47
Prozedur	
<i>anlegen</i>	632
<i>ausführen</i>	631

Q

Quadratwurzel	480
---------------	-----

R

Redundanz	309
Referenzielle Integrität	301, 334, 412
Relationale Datenbank	413
Relationales Datenbankmodell	24
Reservierte Schlüsselwörter	197
Restwert	456
Restwertoperator	451
REVOKE	272
RIGHT OUTER JOIN	351, 357
ROLLBACK	416
Rollen	270
<i>anlegen (CREATE ROLE)</i>	270
<i>Berechtigung</i>	271
<i>Berechtigung entziehen</i>	272
<i>entfernen</i>	272

<i>zuordnen</i>	271
root (Benutzer)	33, 36
Rückgabewert	618

S

Schema	140
Schlüsselattribut	281–283
Schlüsselbeziehung	
<i>aufheben</i>	399
<i>erstellen</i>	388
Schlüsselkandidat	283
Schlüsselvergleich	377
Schlüsselwert	152
Schnittmenge	229, 233, 245
SELECT	47, 48, 268, 488
<i>einschränken</i>	109
SELECT-Abfrage	22
Selektion	52
SELF JOIN	375
SET	146, 324
Sicht → View	
Skalarabfrage	562
Skalarfunktion	463
<i>CHAR_LENGTH</i>	477
<i>für Zahlenwerte</i>	476
<i>LOWER</i>	465
<i>POSITION</i>	477
<i>POWER</i>	477
<i>Spaltenwerte verketteten</i>	472
<i>SQRT</i>	477
<i>SUBSTRING</i>	469
<i>TRIM</i>	466
<i>UPPER</i>	465
Skalarfunktionen	
<i>entfernen</i>	627
<i>entwickeln</i>	617
<i>Parameter</i>	623
<i>Rückgabewert</i>	618
SOME	566, 567
Sonderzeichen	112
Sortieren	
<i>ASC</i>	114
<i>DESC</i>	114
<i>mehrere Spalten</i>	115
<i>nach Datumswerten</i>	117
<i>nach numerischen Spaltenwerten</i>	116
<i>nicht definierte Werte</i>	118
<i>Richtung</i>	114
Sortierreihenfolge	490

Spalte	47	AND NOT	98
alle Spalten abfragen	49	ANY	567
alle Werte ändern	150	AS	128
auf NULL prüfen	82	ASC	114
CHECK	207	BETWEEN	67
CONSTRAINTS	199	CASE	487–489
einem Alias zuordnen	128	CAST	179
entfernen	438	CREATE DATABASE	323, 324
fehlende Spaltenwerte	79	CREATE INDEX	606
Gleichheit	55	CREATE TABLE	165, 187
hinzufügen	438	CREATE VIEW	576
Intervall	66	CROSS JOIN	366
LIKE	71	DEFAULT CHARACTER	324
mehrere sortieren	115	DELETE	156
mehrere Spalten gruppieren	548	DESC	114
mehrere Werte einer Zeile ändern	147	DISTINCT	131, 132, 237
mehrere Zeilen gleichzeitig ändern	148	DROP INDEX	609
Mengenzugehörigkeit prüfen	76	DROP TABLE	191
mit Alias versehen	128	DROP VIEW	596
mit expliziter Angabe einfügen	136	ELSE	488
nach Datumswerten sortieren	117	END	489
nach numerischen Werten sortieren	116	EXCEPT	250
NOT NULL	199	EXISTS	571
NOT-Operator	64	FETCH	110
NULL-Zuweisung	151	FROM	48
ohne Angabe einfügen	140	FULL OUTER JOIN	360
Schlüsselwert ändern	152	GROUP BY	531
Spaltenauswahl abfragen	47	IN	76, 554, 561
Standardwert	205	INNER JOIN	352
umbenennen	425	INSERT	135
Ungleichheit	58	INTERSECT	233, 245
UNIQUE	203	INTO	136
Unterschiede	60, 62	IS NOT NULL	83
Werte ändern	146	IS NULL	83
Werte vergleichen	84	kommentieren	42
Spaltenwert		LEFT OUTER JOIN	354
addieren	452	LIKE	71
dividieren	455	LIMIT	109
ermitteln	23	NOT	64
gruppieren	529	NOT BETWEEN	67
multiplizieren	455	NOT EXISTS	573
Restwert berechnen	456	NOT IN	77
subtrahieren	454	NOT LIKE	76
SQL	21	ON DELETE CASCADE	392
Geschichte	23	ON UPDATE CASCADE	390
Grundlagen	21	OR	91
SQL_NO_CACHE	605	OR NOT	99
SQL-Anweisung		ORDER BY	113
ALL	232, 239, 567	RENAME TABLE	424
ALTER TABLE	423	RIGHT OUTER JOIN	357
AND	91	SELECT	47, 48

SQL-Anweisung (Forts.)		Elterntabellen	341
SELECT ... FROM	47	erstellen	187
SELF JOIN	375	filtern	51
SET	146, 324	Kindtabellen	341
SOME	567	löschen	191
THEN	488	m:n-Beziehung	330
TOP	110	Primärschlüssel erstellen	192
TRUNCATE	161	reservierte Schlüsselwörter	197
UNION	230	Selbstbezug	33
UNIQUE	203	Spalte	22
UPDATE	145	Spalten	47
VALUES	137	Spalten abfragen	48
WHEN	488	Struktur	22
WHERE	51, 52	Struktur verändern	423
SQL-Dialekt	26	umbenennen	423
SQL-Standard, ISO/IEC 9075-1	25	UNIQUE	218
SQL-Vergleichsoperator		virtuelle	575
BETWEEN	66	Zeile	22
Gleichheit	55	Zielstruktur	187
größer/gleich	62	Tabelle erstellen	187
kleiner/gleich	60	Autoinkrement-Schlüssel	194
LIKE	71	CHECK	207, 221
Mengenzugehörigkeit	76	DEFAULT	205
NOT	64	Einschränkung	199, 200, 203, 205, 207, 214, 218, 221
NULL-Value	79, 82	NOT NULL	199
Spaltenwerte vergleichen	84	NULL	200
Übersicht	52	PRIMARY KEY	189, 192
Ungleichheit	58	UNIQUE	203, 218
SQRT	480	Tabellenalias, JOIN	373
Standardwert	205	THEN	488, 492
String	173	TO_CHAR	505
Structured Query Language → SQL		TOP	109, 110
Subquery	553	Transaktion	411
korrelierende	553	ACID-Prinzip	413
korrelierende Abfragen	554	atomare	412
nicht korrelierende	560	Atomicity	413
SUBSTRING	469	BEGIN	416
SUM	535	COMMIT	416
Surrogatschlüssel → Schlüsselattribut		Consistency	413
		dauerhafte	413
		Durability	413
		Isolation	413
		isoliert ausführen	413, 419
		Konsistenz	412
		read committed	420
		read uncommitted	420
		repeatable read	420
		ROLLBACK	416
		serializable	420
		Snapshot	421

T

Tabelle

1:1-Beziehung	326
1:n-Beziehung	328
Beziehung	341
CHECK	221
Definition	21, 23
Eigenschaften	23
Einschränkungen	214
Eltern-Kind-Beziehung	341

Transaktionsprinzip	413
Transitive Abhängigkeit	317
Translation Mode	509
TRIM	466, 467
TRUNCATE	161
Typkonvertierung	180
<i>explizite</i>	180
<i>implizite</i>	181

U

Übungsdatenbank, Voraussetzungen	
für den Import	36
UML	302
UNICODE	173
Unified Modeling Language → UML	
UNION	229, 230, 236
<i>ALL</i>	232, 239
<i>DISTINCT</i>	237, 239
mit <i>WHERE</i> filtern	253
UNIQUE	203, 218
Unterabfrage	553
<i>korrelierende</i>	554
<i>nicht korrelierende</i>	560
<i>Vergleichsoperatoren</i>	566
UPDATE	145, 268
<i>View</i>	588
UPPER	465
UTC	495
UTF-8	173

V

VALUES	137, 191
VARCHAR	175
Vereinigungsmenge	229
<i>ORDER BY</i>	255
Vergleichsoperator	566
Verknüpfung, logische	91
View	140, 575
<i>allgemeine</i>	575
<i>anlegen</i>	576
<i>CREATE OR REPLACE</i>	596
<i>DELETE</i>	590
<i>Einschränkungen</i>	592
<i>entfernen</i>	596
<i>ersetzen</i>	596
<i>INSERT</i>	584
<i>ORDER BY</i>	581
<i>sortieren</i>	581

<i>UPDATE</i>	588
Virtuelle Tabelle	575

W

Wahrheitstabelle	91
Wertebereich	284
WHEN	488, 492
WHERE	51, 52
<i>Mengenoperationen</i>	253
Wildcard	71

Z

Zahlenwert	
<i>Funktionen</i>	476
Zeichenkette	56, 173
<i>Länge ermitteln</i>	477
<i>Position</i>	478
Zeichensatz	324
Zeile	
<i>löschen</i>	157
<i>mehrere Zeilen löschen</i>	158
<i>UPDATE</i>	145
<i>verbinden</i>	351, 375
Zeit	172
<i>einfügen</i>	497
<i>formatieren</i>	499
Zeit und Datum	495
<i>CURRENT_DATE</i>	496
<i>CURRENT_TIME</i>	496
<i>CURRENT_TIMESTAMP</i>	496
<i>DATE_ADD</i>	522
<i>DATE_FORMAT</i>	500
<i>DATE_SUB</i>	522
<i>DATEADD</i>	524
<i>DATEPART</i>	517
<i>EXTRACT</i>	515
<i>FORMAT</i>	510
<i>INTERVAL</i>	523
<i>MS SQL Server</i>	510, 517, 524
<i>MySQL</i>	500, 515, 522
<i>PostgreSQL</i>	505, 515, 523
<i>Skalarfunktionen</i>	496
<i>TO_CHAR</i>	505
Zeitangabe	
<i>berechnen</i>	520
Zeitstempel	
<i>einfügen</i>	497
Zielstruktur	187

Die Serviceseiten

Im Folgenden finden Sie Hinweise, wie Sie Kontakt zu uns aufnehmen können.

Lob und Tadel

Wir hoffen sehr, dass Ihnen dieses Buch gefallen hat. Wenn Sie zufrieden waren, empfehlen Sie das Buch bitte weiter. Wenn Sie meinen, es gebe doch etwas zu verbessern, schreiben Sie direkt an den Lektor dieses Buches: *christoph.meister@rheinwerk-verlag.de*. Wir freuen uns über jeden Verbesserungsvorschlag, aber über ein Lob freuen wir uns natürlich auch!

Auch über die Produktdetailseite zu diesem Buch haben Sie die Möglichkeit, Ihr Feedback an uns zu senden. Folgen Sie einfach diesem Link: *<https://www.rheinwerk-verlag.de/6020>*.

Zusatzmaterialien

Falls Zusatzmaterialien (Beispielcode, Übungsmaterial, Listen usw.) für dieses Buch verfügbar sind, finden Sie sie in Ihrer Online-Bibliothek sowie auf der Webkatalogseite zu diesem Buch: *<https://www.rheinwerk-verlag.de/6020>*. Wenn uns sinnentstellende Tippfehler oder inhaltliche Mängel bekannt werden, stellen wir Ihnen dort auch eine Liste mit Korrekturen zur Verfügung.

Technische Probleme

Im Falle von technischen Schwierigkeiten mit dem E-Book oder Ihrem E-Book-Konto beim Rheinwerk Verlag steht Ihnen gerne unser Leserservice zur Verfügung: *ebooks@rheinwerk-verlag.de*.

Über uns und unser Programm

Informationen zu unserem Verlag und weitere Kontaktmöglichkeiten bieten wir Ihnen auf unserer Verlagswebsite <https://www.rheinwerk-verlag.de>. Dort können Sie sich auch umfassend und aus erster Hand über unser aktuelles Verlagsprogramm informieren und alle unsere Bücher und E-Books schnell und komfortabel bestellen. Alle Buchbestellungen sind für Sie versandkostenfrei.

Rechtliche Hinweise

In diesem Abschnitt finden Sie die ausführlichen und rechtlich verbindlichen Nutzungsbedingungen für dieses E-Book.

Copyright-Vermerk

Das vorliegende Werk ist in all seinen Teilen urheberrechtlich geschützt. Alle Nutzungs- und Verwertungsrechte liegen bei den Autor*innen und beim Rheinwerk Verlag, insbesondere das Recht der Vervielfältigung und Verbreitung, sei es in gedruckter oder in elektronischer Form.

© **Rheinwerk Verlag GmbH, Bonn 2025**

Nutzungs- und Verwertungsrechte

Sie sind berechtigt, dieses E-Book ausschließlich für persönliche Zwecke zu nutzen. Insbesondere sind Sie berechtigt, das E-Book für Ihren eigenen Gebrauch auszudrucken oder eine Kopie herzustellen, sofern Sie diese Kopie auf einem von Ihnen alleine und persönlich genutzten Endgerät speichern. Zu anderen oder weitergehenden Nutzungen und Verwertungen sind Sie nicht berechtigt.

So ist es insbesondere unzulässig, eine elektronische oder gedruckte Kopie an Dritte weiterzugeben. Unzulässig und nicht erlaubt ist des Weiteren, das E-Book im Internet, in Intranets oder auf andere Weise zu verbreiten oder Dritten zur Verfügung zu stellen. Eine öffentliche Wiedergabe oder sonstige Weiterveröffentlichung und jegliche den persönlichen Gebrauch übersteigende Vervielfältigung des E-Books ist ausdrücklich untersagt. Das vorstehend Gesagte gilt nicht nur für das E-Book insgesamt, sondern auch für seine Teile (z. B. Grafiken, Fotos, Tabellen, Textabschnitte).

Urheberrechtsvermerke, Markenzeichen und andere Rechtsvorbehalte dürfen aus dem E-Book nicht entfernt werden, auch nicht das digitale Wasserzeichen.

Die automatisierte Analyse des Werkes, um daraus Informationen insbesondere über Muster, Trends und Korrelationen gemäß § 44b UrhG (»Text und Data Mining«) zu gewinnen, ist untersagt.

Digitales Wasserzeichen

Dieses E-Book-Exemplar ist mit einem **digitalen Wasserzeichen** versehen, einem Vermerk, der kenntlich macht, welche Person dieses Exemplar nutzen darf. Wenn Sie diese Person nicht sind, liegt ein Verstoß gegen das Urheberrecht vor, und wir bitten Sie freundlich, das E-Book nicht weiter zu nutzen und uns diesen Verstoß zu melden. Eine kurze E-Mail an service@rheinwerk-verlag.de reicht schon. Vielen Dank!

Markenschutz

Die in diesem Werk wiedergegebenen Gebrauchsnamen, Handelsnamen, Warenbezeichnungen usw. können auch ohne besondere Kennzeichnung Marken sein und als solche den gesetzlichen Bestimmungen unterliegen.

Haftungsausschluss

Ungeachtet der Sorgfalt, die auf die Erstellung von Text, Abbildungen und Programmen verwendet wurde, können weder Verlag noch Autor*innen, Herausgeber*innen oder Übersetzer*innen für mögliche Fehler und deren Folgen eine juristische Verantwortung oder irgendeine Haftung übernehmen.

Über den Autor



Michael Laube arbeitet schon seit vielen Jahren mit relationalen Datenbanken und SQL. Mit dieser Erfahrung aus der Praxis zeigt er Ihnen, was alles in der Abfragesprache steckt und wie Sie Probleme bei der Datenbankentwicklung elegant lösen.